

吴师兄学算法

从 / 初 / 学 / 者 / 的 / 角 / 度 / 学 / 算 / 法

大家好，我是吴师兄。

- 1、LeetCode 题目太多，**全部刷完肯定不是最好最有效的方式**，其中涉及到的基本知识点来来回回就那些，同时有非常多的类似题，所以刷经典题，掌握这些经典题是最有效的学习方式。
- 2、从算法训练营第一期开始到现在的第十二期，吴师兄整理和迭代了非常多的刷题顺序，经过这两年的同学们的反馈，终于整理出一份我认为是最适合新手小白转码同学的 LeetCode 刷题顺序。
- 3、本刷题顺序涵盖了 205 道算法题，如果你的自学能力强，每天能够投入 2 小时的刷题时间，两个月左右是可以做到完完整整的全部刷完。
- 4、所有题目均来源于算法训练营的内容，以动画和视频的形式讲解，弱化了文字讲解部分，**所以很多题目没有提供题目解析部分，而是以代码注释的形式进行讲解。**
- 5、**每道题目的文字解析部分会逐步补充好**，为后续实体书的出版提供物料。
- 6、更新好的内容会第一时间公布到个网站上：blog.algomooc.com
- 7、添加吴师兄的微信 **algomooc555**，后期将会在朋友圈赠送 205 道题目的 PDF 版题解。
- 8、结合专栏 [《玩转 Leetcode 算法高频 100》](#) 学习效果最佳。



基础内容

- 1、Python 语法知识
- 2、Java 语法知识

1、参考网站：<https://www.runoob.com/>

2、只需要掌握相关编程语言的基础语法、对象初始化、基本数据类型、变量类型、循环语句、条件语句这些知识即可开始刷题。

第二天

LeetCode 26、删除有序数组中的重复项

一、题目描述

给你一个 **升序排列** 的数组 `nums`，请你 **原地** 删除重复出现的元素，使每个元素 **只出现一次**，返回删除后数组的新长度。元素的 **相对顺序** 应该保持 **一致**。

由于在某些语言中不能改变数组的长度，所以必须将结果放在数组 `nums` 的第一部分。更规范地说，如果在删除重复项之后有 `k` 个元素，那么 `nums` 的前 `k` 个元素应该保存最终结果。

将最终结果插入 `nums` 的前 `k` 个位置后返回 `k`。

不要使用额外的空间，你必须在 **原地** 修改输入数组并在使用 $O(1)$ 额外空间的条件下完成。

判题标准:

系统会用下面的代码来测试你的题解:

```
int[] nums = [...]; // 输入数组
int[] expectedNums = [...]; // 长度正确的期望答案

int k = removeDuplicates(nums); // 调用

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

如果所有断言都通过，那么您的题解将被 **通过**。

示例 1:

输入: `nums = [1,1,2]` 输出: `2, nums = [1,2, ]` 解释: 函数应该返回新的长度 `2`，并且原数组 `nums` 的前两个元素被修改为 `1, 2`。不需要考虑数组中超出新长度后面的元素。

示例 2:

输入: nums = [0,0,1,1,1,2,2,3,3,4] 输出: 5, nums = [0,1,2,3,4] 解释: 函数应该返回新的长度 5 , 并且原数组 nums 的前五个元素被修改为 0, 1, 2, 3, 4 。不需要考虑数组中超出新长度后面的元素。

提示:

- `1 <= nums.length <= 3 * 104`
- `-104 <= nums[i] <= 104`
- `nums` 已按 升序 排列

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 删除有序数组中的重复项 (LeetCode 26) :https://leetcode.cn/problems/remove-duplicates-
// from-sorted-array/submissions/
class Solution {
    public int removeDuplicates(int[] nums) {

        // 指针 i 进行数组遍历
        int n = nums.length;

        // 指针 j 指向即将被赋值的位置
        int j = 0;

        // 开始对数组进行遍历
        for (int i = 0 ; i < n ; i++) {

            // 进行筛选
            if ( i == 0 ||  nums[i] != nums[i - 1]) {
                // 赋值
                nums[j] = nums[i];

                // j 移动
                j++;
            }
        }

        // 获取结果
        return j ;
    }
}
```

2、C++代码

```
class solution {
public:
    int removeDuplicates(vector<int>& nums) {

        // 指针 i 进行数组遍历
        int n = nums.size();

        // 指针 j 指向即将被赋值的位置
        int j = 0;

        // 开始对数组进行遍历
        for (int i = 0 ; i < n ; i++) {

            // 进行筛选
            if ( i == 0 ||  nums[i] != nums[i - 1]) {
                // 赋值
                nums[j] = nums[i];

                // j 移动
                j++;
            }
        }

        // 获取结果
        return j ;
    }
};
```

3、Python 代码

```
class solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        # 指针 i 进行数组遍历
        n = len(nums)

        # 指针 j 指向即将被赋值的位置
        j = 0

        # 开始对数组进行遍历
        for i in range(n):

            # 进行筛选
            if i == 0 or  nums[i] != nums[i - 1] :
                # 赋值
                nums[j] = nums[i]
```



```
# j 移动
j += 1

# 获取结果
return j
```

LeetCode 27、移除元素

一、题目描述

给你一个数组 `nums` 和一个值 `val`，你需要 [原地](#) 移除所有数值等于 `val` 的元素，并返回移除后数组的新长度。

不要使用额外的数组空间，你必须仅使用 `O(1)` 额外空间并 [原地](#) 修改输入数组。

元素的顺序可以改变。你不需要考虑数组中超出新长度后面的元素。

说明：

为什么返回数值是整数，但输出的答案是数组呢？

请注意，输入数组是以「引用」方式传递的，这意味着在函数里修改输入数组对于调用者是可见的。

你可以想象内部操作如下：

```
// nums 是以“引用”方式传递的。也就是说，不对实参作任何拷贝
int len = removeElement(nums, val);

// 在函数里修改输入数组对于调用者是可见的。
// 根据你的函数返回的长度，它会打印出数组中 该长度范围内 的所有元素。
for (int i = 0; i < len; i++) {
    print(nums[i]);
}
```

示例 1：

输入：nums = [3,2,2,3], val = 3 输出：2, nums = [2,2] 解释：函数应该返回新的长度 2，并且 nums 中的前两个元素均为 2。你不需要考虑数组中超出新长度后面的元素。例如，函数返回的新长度为 2，而 nums = [2,2,3,3] 或 nums = [2,2,0,0]，也会被视作正确答案。

示例 2：

```
输入：nums = [0,1,2,2,3,0,4,2], val = 2
输出：5, nums = [0,1,4,0,3]
解释：函数应该返回新的长度 5，并且 nums 中的前五个元素为 0，1，3，0，4。注意这五个元素可为任意顺序。
你不需要考虑数组中超出新长度后面的元素。
```

提示：

- `0 <= nums.length <= 100`
- `0 <= nums[i] <= 50`

- `0 <= val <= 100`

二、题目解析

三、参考代码

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:

        # 指针 i 进行数组遍历
        n = len(nums)

        # 指针 j 指向即将被赋值的位置
        j = 0

        # 开始对数组进行遍历
        for i in range(n):

            # 进行筛选
            if nums[i] != val:
                # 赋值
                nums[j] = nums[i]

                # j 移动
                j += 1

        # 获取结果
        return j
```

LeetCode 283、移动零

一、题目描述

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

```
输入: nums = [0,1,0,3,12]
输出: [1,3,12,0,0]
```

示例 2:

```
输入: nums = [0]
输出: [0]
```

提示:

- `1 <= nums.length <= 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

进阶：你能尽量减少完成的操作次数吗？

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 移动零 (LeetCode 283) : https://leetcode.cn/problems/move-zeroes/
class Solution {
    public void moveZeroes(int[] nums) {

        // 设置一个变量，用来指向经过一系列操作后数组中所有为 0 元素的第一个位置上
        // 一开始默认在索引为 0 的位置
        int slow = 0;

        // 从头到尾遍历数组
        // 遍历完毕之后，slow 指向了一个为 0 的元素，或者如果数组中不存在 0，就和 fast 一样，超过了数组的范围
        for (int fast = 0; fast < nums.length; fast++) {

            // 在遍历过程中，如果发现访问的元素是非 0 元素
            // 说明 slow 不在正确的位置上，需要向后移动，寻找合适的位置
            if (nums[fast] != 0) {

                // 这个时候，原先 slow 的值需要被 fast 的值覆盖
                nums[slow] = nums[fast];

                // slow 需要向后移动，寻找合适的位置
                slow++;

            }
        }

        // 接下来，只需要把 slow 及其后面所有的元素都设置为 0 就行
        for (int i = slow; i < nums.length; i++) {

            // 都设置为 0
            nums[i] = 0;

        }
    }
}
```

```
}
```

2、C++ 代码

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        // 设置一个变量，用来指向经过一系列操作后数组中所有为 0 元素的第一个位置上
        // 一开始默认在索引为 0 的位置
        int slow = 0;

        // 从头到尾遍历数组
        // 遍历完毕之后，slow 指向了一个为 0 的元素，或者如果数组中不存在 0，就和 fast 一样，超过了数组的范围
        for (int fast = 0; fast < nums.size(); fast++) {

            // 在遍历过程中，如果发现访问的元素是非 0 元素
            // 说明 slow 不在正确的位置上，需要向后移动，寻找合适的位置
            if (nums[fast] != 0) {

                // 这个时候，原先 slow 的值需要被 fast 的值覆盖
                nums[slow] = nums[fast];

                // slow 需要向后移动，寻找合适的位置
                slow++;
            }
        }

        // 接下来，只需要把 slow 及其后面所有的元素都设置为 0 就行
        for (int i = slow; i < nums.size(); i++) {

            // 都设置为 0
            nums[i] = 0;
        }
    }
};
```

3、Python 代码

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        # 设置一个变量，用来指向经过一系列操作后数组中所有为 0 元素的第一个位置上
        # 一开始默认在索引为 0 的位置
        slow = 0

        # 从头到尾遍历数组
```

```

        # 遍历完之后，slow 指向了一个为 0 的元素，或者如果数组中不存在 0，就和 fast 一样，超过了
        数组的范围
        for fast in range(len(nums)) :

            # 在遍历过程中，如果发现访问的元素是非 0 元素
            # 说明 slow 不在正确的位置上，需要向后移动，寻找合适的位置
            if nums[fast] != 0:

                # 这个时候，原先 slow 的值需要被 fast 的值覆盖
                nums[slow] = nums[fast]

                # slow 需要向后移动，寻找合适的位置
                slow += 1

            # 接下来，只需要把 slow 及其后面所有的元素都设置为 0 就行
            for i in range(slow, len(nums)) :

                # 都设置为 0
                nums[i] = 0

```

四、复杂度分析

- 时间复杂度：O(n)，其中 n 为序列长度。每个位置至多被遍历两次。
- 空间复杂度：O(1)。只需要常数的空间存放若干变量。

LeetCode 485、最大连续 1 的个数

一、题目描述

给定一个二进制数组 `nums`，计算其中最大连续 `1` 的个数。

示例 1:

输入：nums = [1,1,0,1,1,1] 输出：3 解释：开头的两位和最后的三位都是连续 1，所以最大连续 1 的个数是 3。

示例 2:

输入：nums = [1,0,1,1,0,1] 输出：2

提示:

- `1 <= nums.length <= 10^5`
- `nums[i]` 不是 0 就是 1。

二、题目解析

三、参考代码

```
class Solution(object):
    def findMaxConsecutiveOnes(self, nums):

        # 最后一个 0 所在的索引位置
        lastZero = -1

        # 结果
        ans = 0

        # 从左到右访问数组 nums
        for i, num in enumerate(nums):

            # 1、当前元素为 0 , 更新 lastZero
            if num == 0:
                lastZero = i

            # 2、否则说明当前元素为 1
            else:
                # 通过 lastZero 可以获取当前元素距离最前面的 1 的个数
                # 对比之前的 ans , 更新获取最大值
                ans = max(ans, i - lastZero)

        # 返回结果
        return ans
```

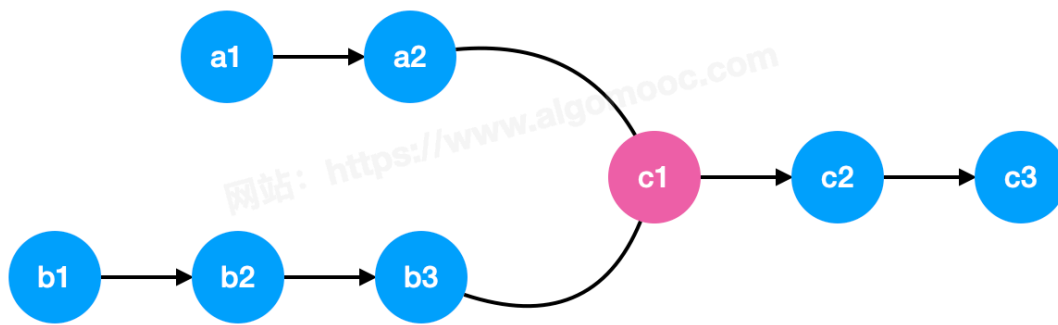
第三天

LeetCode 160、相交链表

一、题目描述

编写一个程序，找到两个单链表相交的起始节点。

如下面的两个链表：



在节点 c1 开始相交。

注意：

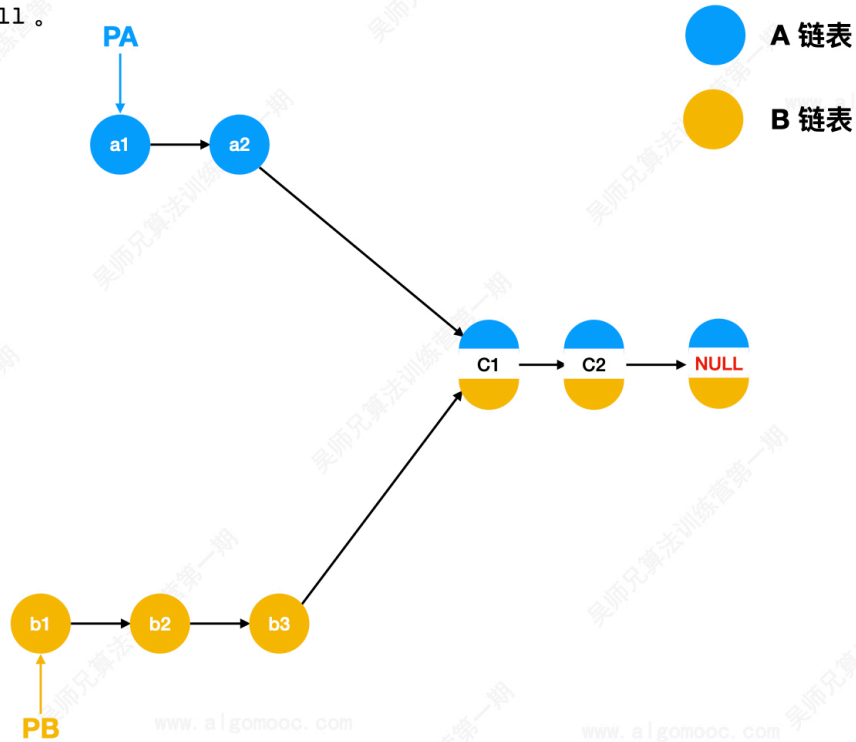
- 如果两个链表没有交点，返回 null。
- 在返回结果后，两个链表仍须保持原有的结构。
- 可假定整个链表结构中没有循环。
- 程序尽量满足 $O(n)$ 时间复杂度，且仅用 $O(1)$ 内存。

二、题目解析

1、模拟

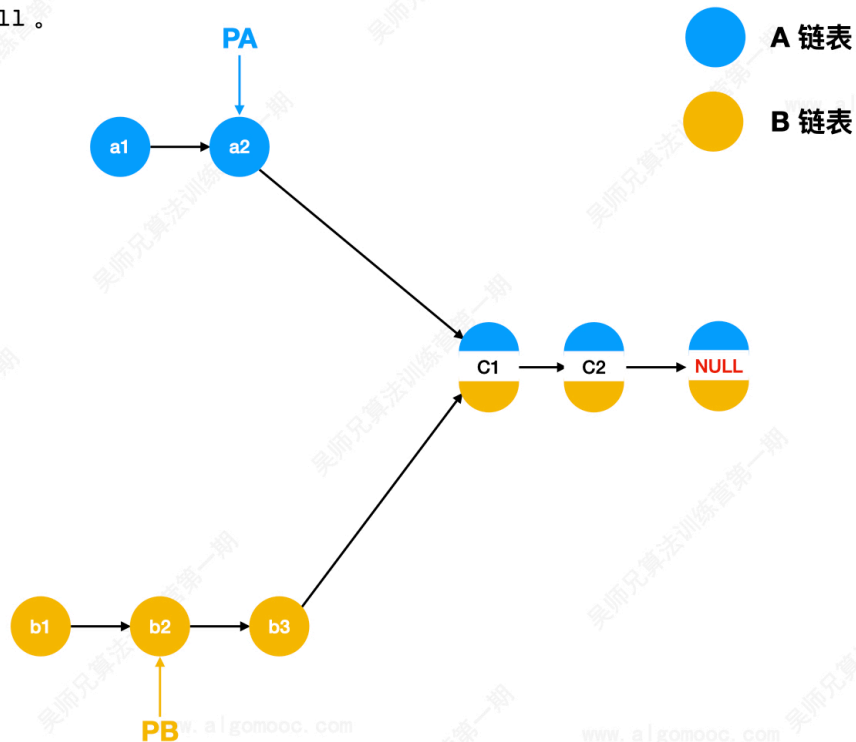
首先假设 A、B 两个链表是有相交节点的情况。

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



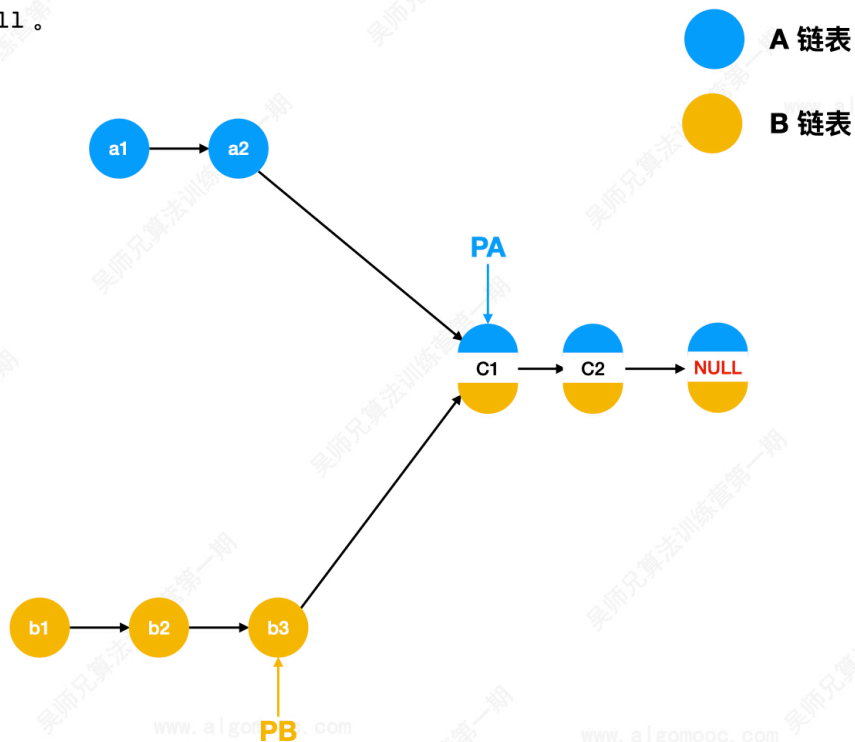
相交链表.002

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



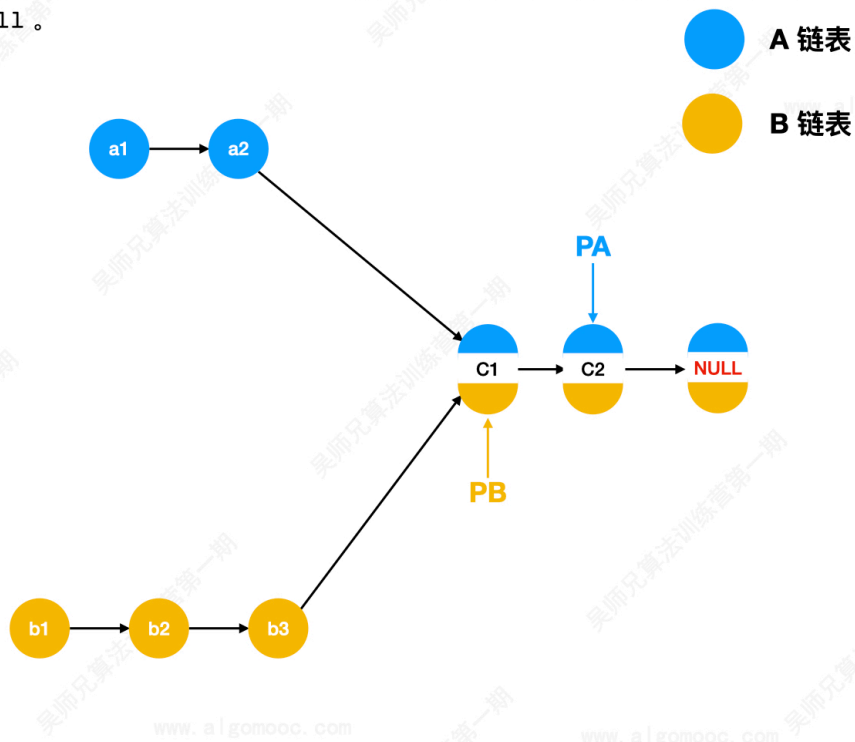
相交链表.003

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



相交链表.004

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



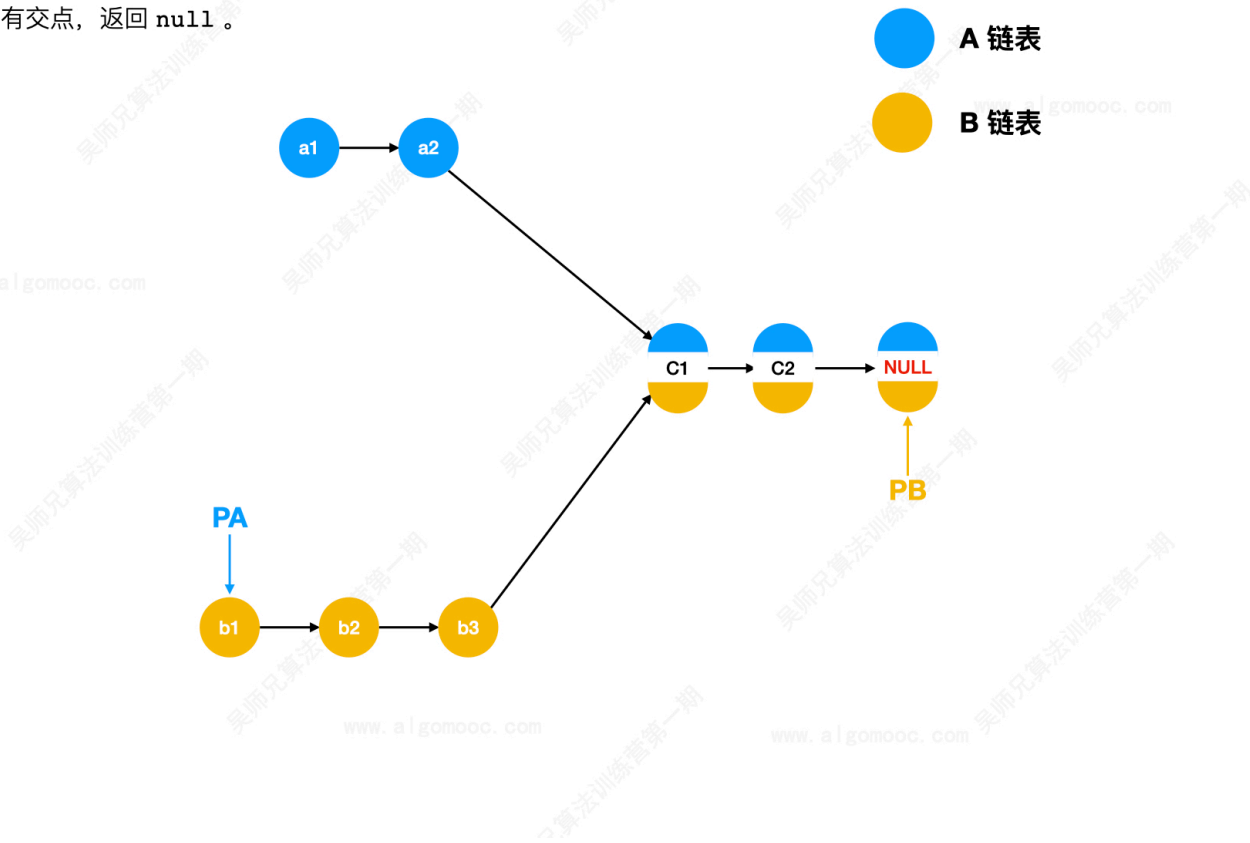
相交链表.005

相交链表.006

www. a | gomooc. com

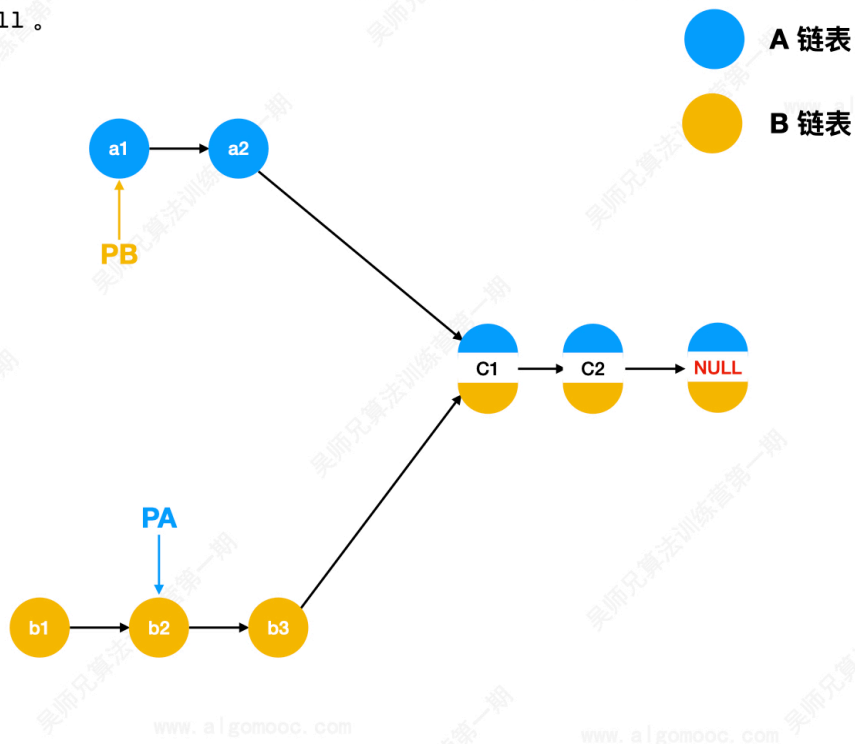


给你两个单链表的头节点 **headA** 和 **headB** ， 请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null** 。



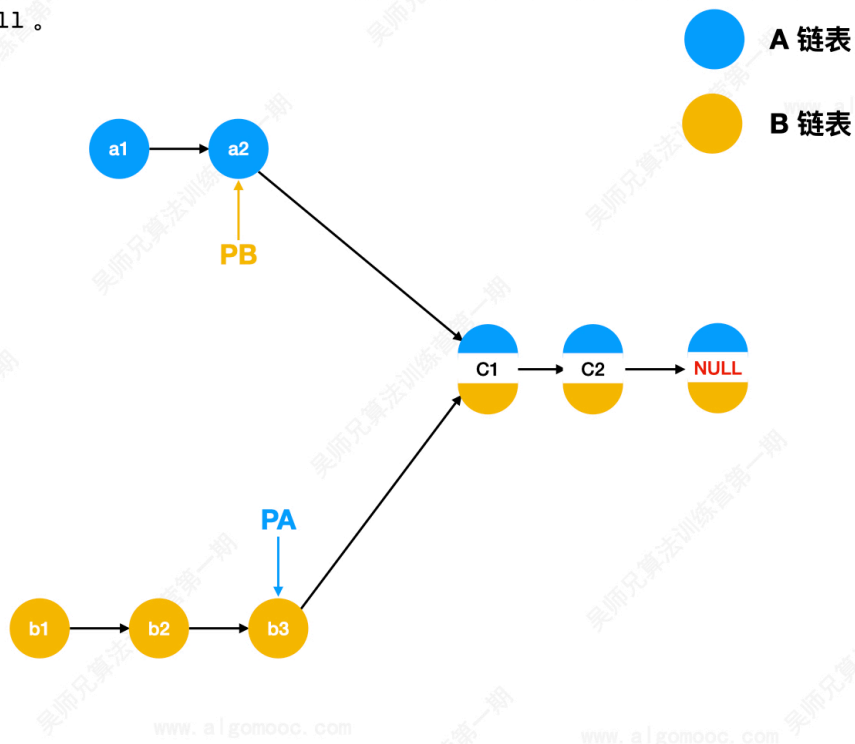
相交链表.007

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



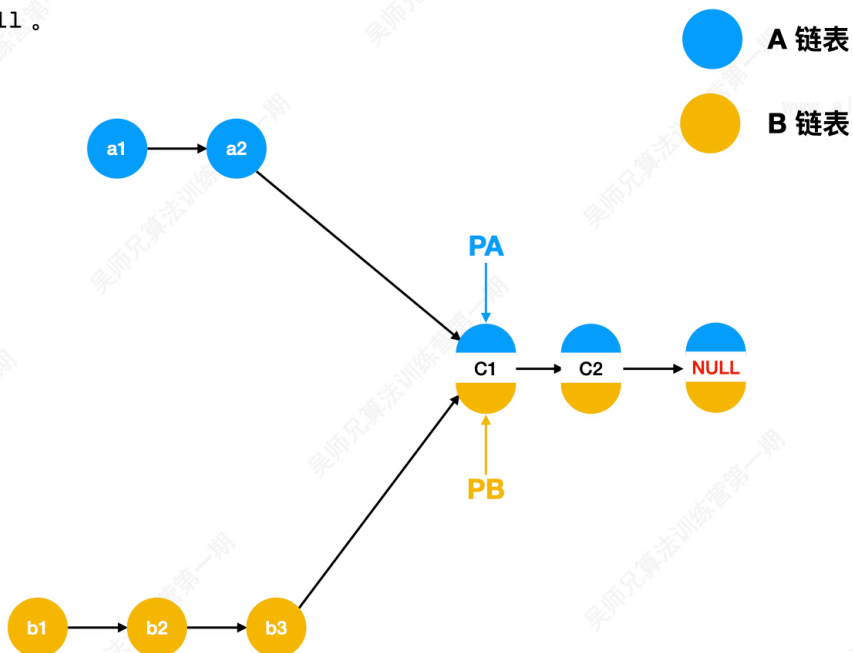
相交链表.008

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



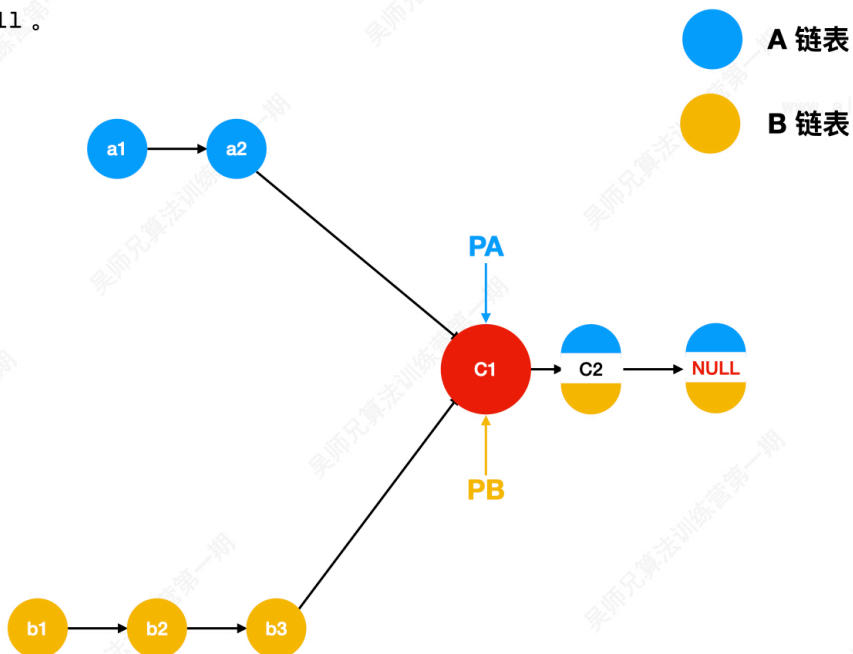
相交链表.009

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



相交链表.010

给你两个单链表的头节点 **headA** 和 **headB**，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 **null**。



相交链表.011

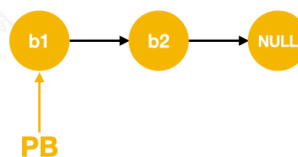
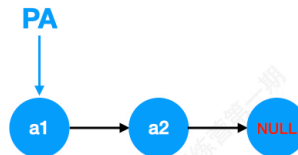
再假设 A、B 两个链表是没有相交节点的情况。

www.algomooc.com



给你两个单链表的头节点 `headA` 和 `headB`，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 `null`。

如果 A B 没有交点



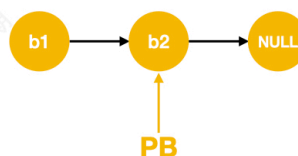
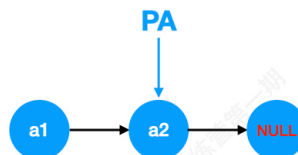
相交链表.012

www.algomooc.com



给你两个单链表的头节点 `headA` 和 `headB`，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 `null`。

如果 A B 没有交点



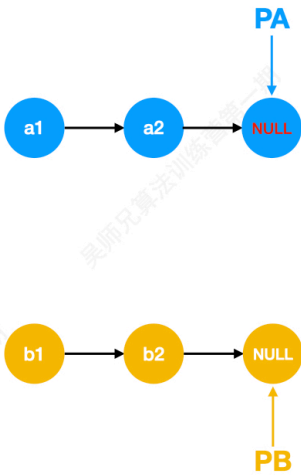
相交链表.013

www. a | gomooc. com



给你两个单链表的头节点 `headA` 和 `headB` ， 请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点， 返回 `null` 。

如果 A B 没有交点



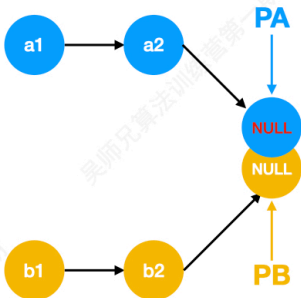
相交链表.014

www. a | gomooc. com



给你两个单链表的头节点 `headA` 和 `headB` ， 请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点， 返回 `null` 。

如果 A B 没有交点



同时指向 `NULL`， 代表不相交， 无公共尾部

2、规律

也就是说，无论 A、B 两个链表是否有相交点，最终都会指向一个相同的节点，要么是它们的公共尾部，要么是 NULL。

让指针 `pointA` 和 `pointB` 分别指向链表 A 和链表 B 的头结点，之后两个指针分别以步幅为 1 的速度向链表的尾部遍历。

- 当指针 `pointA` 遍历到链表 A 的尾节点时，此时指针 `pointA` 走了 a 个节点，将指针 `pointA` 指向链表 B 的头部，继续向后遍历，直至走到 `c1`，此时指针 `pointA` 总共走了 $a + (b - c)$ 步。
- 当指针 `pointB` 遍历到链表 B 的尾节点时，此时指针 `pointB` 走了 b 个节点，将指针 `pointB` 指向链表 A 的头部，继续向后遍历，直至走到 `c1`，此时指针 `pointB` 总共走了 $b + (a - c)$ 步。

根据数学知识， $a + (b - c) = b + (a - c)$ ，如果 $c > 0$ ，表明两链表有公共尾部，`c1` 存在，两两链表同时到达 `c1`；如果 $c = 0$ ，表明两链表没有公共尾部，指针 `pointA` 和 `pointB` 都指向 `NULL`。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        // 边界判断
        if (headA == null || headB == null) return null;

        // 设置一个指针 pointA，指向链表 A 的头节点
        ListNode pointA = headA;

        // 设置一个指针 pointB，指向链表 B 的头节点
        ListNode pointB = headB;

        // 指针 pointA 和 指针 pointB 不断向后遍历，直到找到相交点
        // 不用担心会跳出这个循环，实际上在链表 headA 长度和链表 headB 长度的之和减一
        // pointA 和 pointB 都会同时指向 null
        // 比如 headA 的长度是 7，headB 的长度是 11，这两个链表不相交
        // 那么 pointA 移动了 7 + 11 - 1 次之后，会指向 null
        // pointB 移动了 7 + 11 - 1 次之后，也指向 null
        // 这个时候就跳出了循环
        while (pointA != pointB) {
            // 指针 pointA 一开始在链表 A 上遍历，当走到链表 A 的尾部即 null 时，跳转到链表 B 上
            if (pointA == null){
                // 指针 pointA 跳转到链表 B 上
                pointA = headB;
            }else{
                // 指针 pointB 一开始在链表 B 上遍历，当走到链表 B 的尾部即 null 时，跳转到链表 A 上
                pointB = headA;
            }
        }
        return pointA;
    }
}
```

```

        // 否则的话 pointA 不断的向后移动
        pointA = pointA.next;
    }

    // 指针 pointB 一开始在链表 B 上遍历, 当走到链表 B 的尾部即 null 时, 跳转到链表 A 上
    if( pointB == null){
        // 指针 pointA 跳转到链表 B 上
        pointB = headA;
    }else{
        // 否则的话 pointB 不断的向后移动
        pointB = pointB.next;
    }
}

// 1、此时, pointA 和 pointB 指向那个相交的节点, 返回任意一个均可
// 2、此时, headA 和 headB 不相交, 那么 pointA 和 pointB 均为 null, 也返回任意一个均可
return pointA;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        // 边界判断
        if (headA == NULL || headB == NULL) return NULL;

        // 设置一个指针 pointA, 指向链表 A 的头节点
        ListNode *pointA = headA;

        // 设置一个指针 pointB, 指向链表 B 的头节点
        ListNode *pointB = headB;

        // 指针 pointA 和 指针 pointB 不断向后遍历, 直到找到相交点
        // 不用担心会跳不出这个循环, 实际上在链表 headA 长度和链表 headB 长度的之和减一
        // pointA 和 pointB 都会同时指向 null
        // 比如 headA 的长度是 7, headB 的长度是 11, 这两个链表不相交
        // 那么 pointA 移动了 7 + 11 - 1 次之后, 会指向 null
        // pointB 移动了 7 + 11 - 1 次之后, 也指向 null
        // 这个时候就跳出了循环
        while (pointA != pointB) {
            // 指针 pointA 一开始在链表 A 上遍历, 当走到链表 A 的尾部即 null 时, 跳转到链表 B 上
            if( pointA == NULL){
                // 指针 pointA 跳转到链表 B 上

```



```

        pointA = headB;
    }else{
        // 否则的话 pointA 不断的向后移动
        pointA = pointA->next;
    }

    // 指针 pointB 一开始在链表 B 上遍历, 当走到链表 B 的尾部即 null 时, 跳转到链表 A 上
    if( pointB == NULL){
        // 指针 pointA 跳转到链表 B 上
        pointB = headA;
    }else{
        // 否则的话 pointB 不断的向后移动
        pointB = pointB->next;
    }
}

// 1、此时, pointA 和 pointB 指向那个相交的节点, 返回任意一个均可
// 2、此时, headA 和 headB 不相交, 那么 pointA 和 pointB 均为 null, 也返回任意一个均可
return pointA;
}
};

```

3、Python 代码

```

class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> ListNode:

        # 边界判断
        if( headA == None or headB == None ):
            return None

        # 设置一个指针 pointA, 指向链表 A 的头节点
        pointA = headA

        # 设置一个指针 pointB, 指向链表 B 的头节点
        pointB = headB

        # 指针 pointA 和 指针 pointB 不断向后遍历, 直到找到相交点
        # 不用担心会跳不出这个循环, 实际上在链表 headA 长度和链表 headB 长度的之和减一
        # pointA 和 pointB 都会同时指向 null
        # 比如 headA 的长度是 7, headB 的长度是 11, 这两个链表不相交
        # 那么 pointA 移动了 7 + 11 - 1 次之后, 会指向 null
        # pointB 移动了 7 + 11 - 1 次之后, 也指向 null
        # 这个时候就跳出了循环
        while pointA != pointB:
            # 指针 pointA 一开始在链表 A 上遍历, 当走到链表 A 的尾部即 null 时, 跳转到链表 B 上
            if pointA == None:
                # 指针 pointA 跳转到链表 B 上
                pointA = headB

```

```

else:
    # 否则的话 pointA 不断的向后移动
    pointA = pointA.next
# 指针 pointB 一开始在链表 B 上遍历, 当走到链表 B 的尾部即 null 时, 跳转到链表 A 上
if pointB == None:
    # 指针 pointA 跳转到链表 B 上
    pointB = headA
else:
    # 否则的话 pointB 不断的向后移动
    pointB = pointB.next

# 1、此时, pointA 和 pointB 指向那个相交的节点, 返回任意一个均可
# 2、此时, headA 和 headB 不相交, 那么 pointA 和 pointB 均为 null, 也返回任意一个均可
return pointA

```

四、复杂度分析

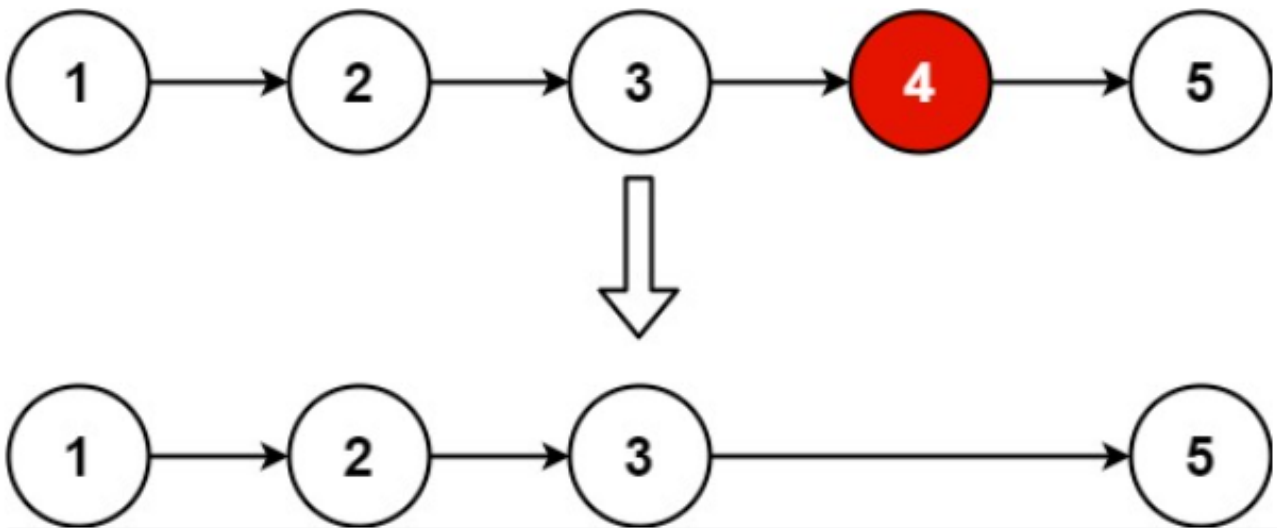
时间复杂度： $O(m+n)$ ，其中 m 和 n 是分别是链表 $headA$ 和 $headB$ 的长度。两个指针同时遍历两个链表，每个指针遍历两个链表各一次。

空间复杂度： $O(1)$ 。

LeetCode 19、删除链表的倒数第 N 个结点

一、题目描述

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。



示例 1:

输入: $head = [1,2,3,4,5]$, $n = 2$ 输出: $[1,2,3,5]$

示例 2:

输入: $head = [1]$, $n = 1$ 输出: $[]$

示例 3:

输入: head = [1,2], n = 1 输出: [1]

提示:

- 链表中结点的数目为 `sz`
- `1 <= sz <= 30`
- `0 <= Node.val <= 100`
- `1 <= n <= sz`

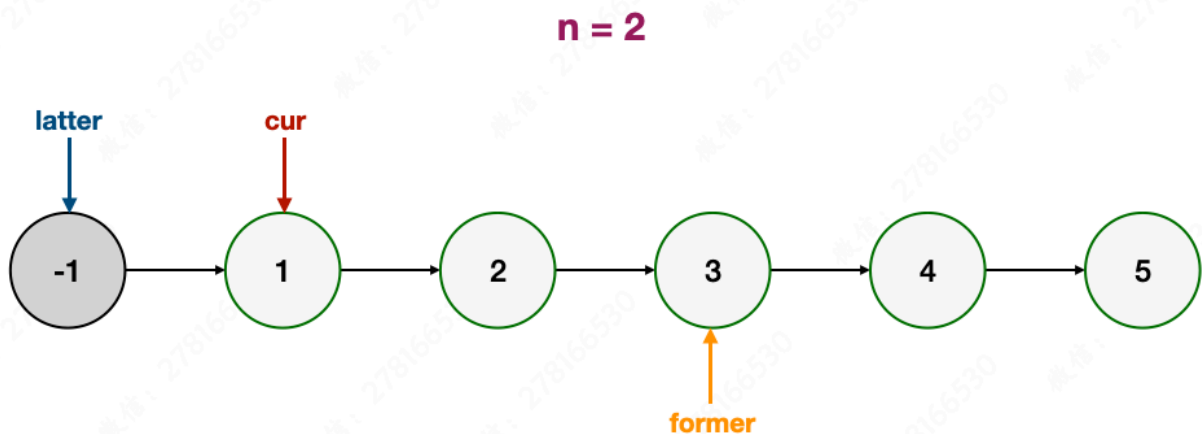
二、题目解析

一般来说, 链表相关的算法题考察的知识点有以下几个:

- 递归
- 反转
- 双指针
- 环

本题解题思路如下:

- 1、在原链表的前面添加一个**虚拟头结点**, 使得原链表的头结点和其余的结点地位一样, 进行删除操作时不需要进行区分处理。
- 2、在原链表的头部设置一个指针 `former`, 使用 `for` 循环让它向后移动 `n` 步。
- 3、在原链表的头部再设置一个指针 `cur`, 同时在虚拟头结点位置设置一个指针 `latter`。



- 4、接下来, 同时让这三个指针向后移动, 直到 `former` 指向了 `null`, 此时 `cur` 指向的恰好就是倒数第 `n` 个结点。
- 5、由于 `latter` 一直在 `cur` 的上一个结点位置, 这个时候只需要让 `latter` 指向 `cur` 的下一个结点, 那么也就完成了删除 `cur` 结点的操作。

三、参考代码

```
// 本题文字版详解请访问下方网站
// https://www.algomooc.com
// 作者: 程序员吴师兄
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
```

```

// 添加表头
ListNode dummy = new ListNode(-1);

dummy.next = head;

// 寻找需要删除的节点节点
ListNode cur = head;

// 指针 latter 指向虚拟头结点
ListNode latter = dummy;

ListNode former = head;

// 让 former 指针先向前走 n 步
for (int i = 0 ; i < n; i++) {
    // former 指针向后移动
    former = former.next;
}

// 接下来, 让这两个指针 former 和 latter 同时向前移动, 直到前指针 former 指向 NULL
while (former != null) {
    // former 指针向后移动
    former = former.next;

    // latter 来到 cur 的位置
    latter = cur;

    // cur 指针向后移动
    cur = cur.next;
}

// 删除 cur 这个位置的结点
latter.next = cur.next;

// 返回虚拟头结点的下一个结点
return dummy.next;
}
}

```

Python

```

#### 本题文字版详解请访问下方网站
#### https://www.algomooc.com
#### 作者: 程序员吴师兄
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # 添加表头
        dummy = ListNode(-1)
        dummy.next = head

```

```
# 寻找需要删除的节点节点
cur = head

# 指针 latter 指向虚拟头结点
latter = dummy

former = head

# 让 former 指针先向前走 n 步
for i in range(n):
    # former 指针向后移动
    former = former.next

# 接下来, 让这两个指针 former 和 latter 同时向前移动, 直到前指针 former 指向 NULL
while former is not None:
    # former 指针向后移动
    former = former.next

    # latter 来到 cur 的位置
    latter = cur

    # cur 指针向后移动
    cur = cur.next

# 删除 cur 这个位置的结点
latter.next = cur.next

# 返回虚拟头结点的下一个结点
return dummy.next
```

LeetCode 203、移除链表元素

一、题目描述

给你一个链表的头节点 head 和一个整数 val，请你删除链表中所有满足 Node.val == val 的节点，并返回 **新的头节点**。

二、题目解析

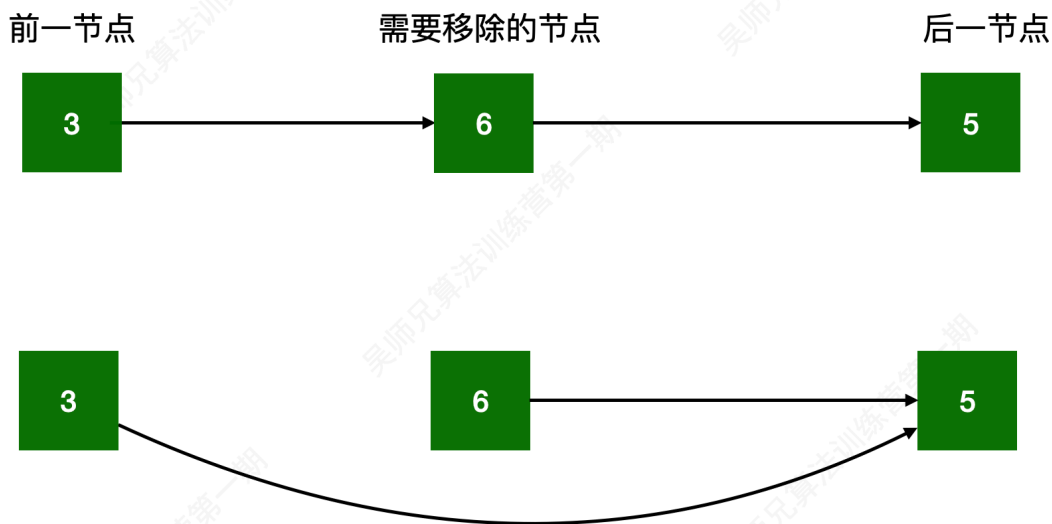
移除的节点有两种情况。

1、移除的是头节点

2、移除的是其它节点

对于**移除其它节点**这种情况，我们可以让它的前一个节点指向移除节点的下一节点来解决。

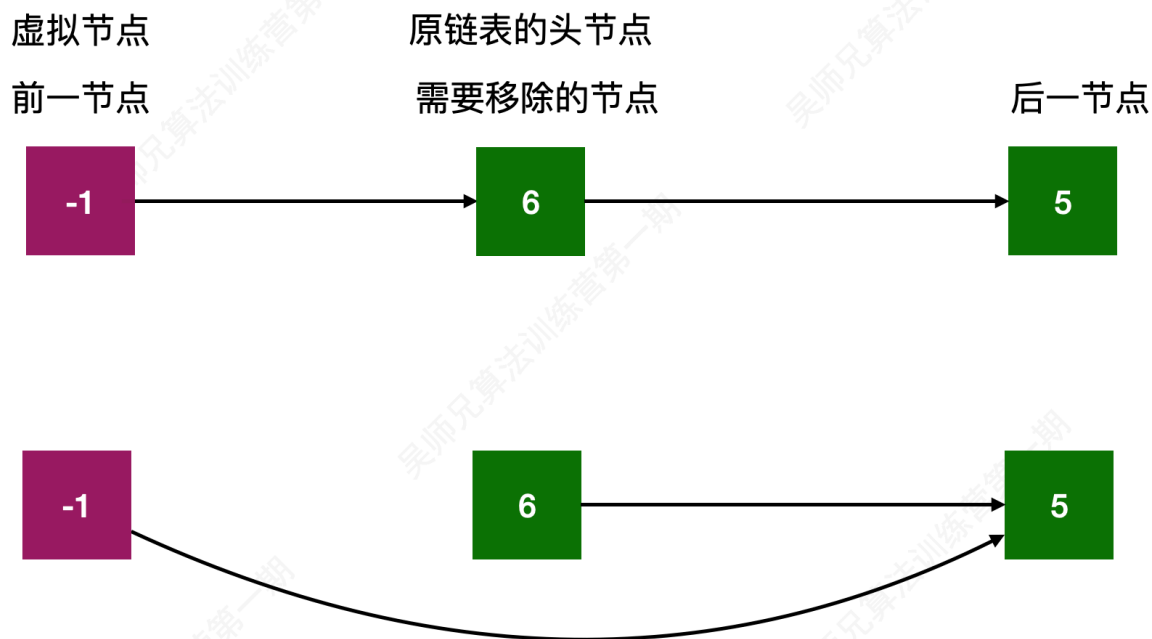
val = 6



对于**移除头节点**这种情况，为了让解决情况和上面的逻辑一样，我们借助一个**虚拟节点**来解决。

我们给链表添加一个虚拟头节点作为新的头节点。

val = 6



三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 移除链表元素 ( LeetCode 203 ) :https://leetcode-cn.com/problems/remove-linked-list-elements/
class Solution {
    public ListNode removeElements(ListNode head, int val) {

        // 边界情况，如果链表为空，那么没有元素可以删除，直接返回空
        if (head == null) return null;

        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        // 设置虚拟节点的目的是为了原链表中所有节点就都可以按照统一的方式进行移除
        // 因为如果不设置虚拟节点，如果删除的元素是原链表中的头节点，那么需要额外的做一些判断，比较繁琐

        ListNode dummy = new ListNode(-1);

        // 虚拟头节点的下一节点指向 head 节点
        // 如果原链表是 1 --> 2 --> 3
        // 那么加上虚拟头节点就是 -1 --> 1 --> 2 --> 3
        dummy.next = head;

        // 设置一个指针，指向此时的虚拟节点
        // pre: -1 --> 1 --> 2 --> 3
        ListNode pre = dummy;

        // 设置一个指针，指向原链表 head
        ListNode cur = head;

        // 让 cur 不断的向后移动，直到移动到链表的最尾部，指向 null 的那个位置
        // 此时 pre 还在指向 dummy
        // 也就是说一开始 pre 落后 cur 一个节点
        while (cur != null) {

            // 移动的过程中，如果发现当前的节点值和目标值一样
            // 我们就让指针 pre 所指向的节点的下一节点跳过这个值
            if (cur.val == val) {
                // 让指针 pre 所指向的节点的下一节点跳过这个值
                pre.next = cur.next;
            } else {
                // 否则的话，pre 跟上 cur 的位置
                pre = cur;
            }

            // 判断完当前的节点情况后，让 cur 向后移动
            cur = cur.next;
        }

        return dummy.next;
    }
}
```

```

        cur = cur.next;
    }

    // 最后返回 dummy 节点的下一节点
    // 因为这个时候 dummy 指向的是一个我们设置的节点，它的下一节点才是原链表中的节点
    return dummy.next;
}

}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 移除链表元素 ( LeetCode 203 ) :https://leetcode-cn.com/problems/remove-linked-list-
elements/
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        // 边界情况，如果链表为空，那么没有元素可以删除，直接返回空
        if(head == NULL) return NULL;

        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的
        // 值
        // 设置虚拟节点的目的是为了让原链表中所有节点就都可以按照统一的方式进行移除
        // 因为如果不设置虚拟节点，如果删除的元素是原链表中的头节点，那么需要额外的做一些判断，比较繁
        // 琐

        ListNode *dummy = new ListNode(-1);

        // 虚拟头节点的下一节点指向 head 节点
        // 如果原链表是 1 --> 2 --> 3
        // 那么加上虚拟头节点就是 -1 --> 1 --> 2 --> 3
        dummy->next = head;

        // 设置一个指针，指向此时的虚拟节点
        // pre: -1 --> 1 --> 2 --> 3
        ListNode *pre = dummy;

        // 设置一个指针，指向原链表 head
        ListNode *cur = head;

        // 让 cur 不断的向后移动，直到移动到链表的最尾部，指向 NULL 的那个位置
        // 此时 pre 还在指向 dummy
        // 也就是说一开始 pre 落后 cur 一个节点
        while ( cur != NULL ) {

            // 移动的过程中，如果发现当前的节点值和目标值一样

```



```

        // 我们就让指针 pre 所指向的节点的下一节点跳过这个值
        if (cur->val == val) {
            // 让指针 pre 所指向的节点的下一节点跳过这个值
            pre->next = cur->next;
        } else {
            // 否则的话, pre 跟上 cur 的位置
            pre = cur;
        }
        // 判断完当前的节点情况后, 让 cur 向后移动
        cur = cur->next;
    }

    // 最后返回 dummy 节点的下一节点
    // 因为这个时候 dummy 指向的是一个我们设置的节点, 它的下一节点才是原链表中的节点
    return dummy->next;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者: 程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 移除链表元素 ( LeetCode 203 ) : https://leetcode-cn.com/problems/remove-linked-list-elements/
class Solution:
    def removeElements(self, head: ListNode, val: int) -> ListNode:
        # 边界情况, 如果链表为空, 那么没有元素可以删除, 直接返回空
        if head == None :
            return None

        # 一开始设置一个虚拟节点, 它的值为 -1, 它的值可以设置为任何的数, 因为我们根本不需要使用它的值
        # 设置虚拟节点的目的是为了让原链表中所有节点就都可以按照统一的方式进行移除
        # 因为如果不设置虚拟节点, 如果删除的元素是原链表中的头节点, 那么需要额外的做一些判断, 比较繁琐
        dummy = ListNode(-1)

        # 虚拟头节点的下一节点指向 head 节点
        # 如果原链表是 1 --> 2 --> 3
        # 那么加上虚拟头节点就是 -1 --> 1 --> 2 --> 3
        dummy.next = head

        # 设置一个指针, 指向此时的虚拟节点
        # pre: -1 --> 1 --> 2 --> 3
        pre = dummy

        # 设置一个指针, 指向原链表 head
        cur = head

```

```

# 让 cur 不断的向后移动，直到移动到链表的最尾部，指向 None 的那个位置
# 此时 pre 还在指向 dummy
# 也就是说一开始 pre 落后 cur 一个节点
while cur != None :

    # 移动的过程中，如果发现当前的节点值和目标值一样
    # 我们就让指针 pre 所指向的节点的下一节点跳过这个值
    if cur.val == val :
        # 让指针 pre 所指向的节点的下一节点跳过这个值
        pre.next = cur.next
    else :
        # 否则的话，pre 跟上 cur 的位置
        pre = cur
        # 判断完当前的节点情况后，让 cur 向后移动
        cur = cur.next

# 最后返回 dummy 节点的下一节点
# 因为这个时候 dummy 指向的是一个我们设置的节点，它的下一节点才是原链表中的节点
return dummy.next

```

四、复杂度分析

时间复杂度：O(n)，其中 n 是链表的长度。需要遍历链表一次。

空间复杂度：O(1)。

LeetCode 24、两两交换链表中的节点

一、题目描述

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在**不修改节点内部的值**的情况下完成本题（即，只能进行节点交换）。

示例 1：

输入：head = [1,2,3,4]
输出：[2,1,4,3]

示例 2：

输入：head = []
输出：[]

示例 3：

输入：head = [1]
输出：[1]

提示：

- 链表中节点的数目在范围 [0, 100] 内
- $0 \leq \text{Node.val} \leq 100$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 两两交换链表中的节点 (LeetCode 24) : https://leetcode.cn/problems/swap-nodes-in-pairs/
class Solution {
    public ListNode swapPairs(ListNode head) {

        // 寻找递归终止条件
        // 1、head 指向的结点为 null
        // 2、head 指向的结点的下一个结点为 null
        // 在这两种情况下，一个节点或者空节点无论怎么交换操作，都是原来的 head
        if( head == null || head.next == null) {
            return head;
        }

        // 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点或者最后两个节点
        ListNode subHead = swapPairs(head.next.next);

        // 对于 head 这个节点来说，它是和它相邻的右边这个节点进行交换操作
        // 所以先要保存这个节点，并且经过上述递归终止条件的判断，head.next 是必然存在的
        ListNode headNext = head.next;

        // head 原来是指向 headNext
        // 交换之后
        // headNext 指向 head
        headNext.next = head;

        // headNext 原来是指向 subHead
        // 现在需要让 head 指向 subHead
        head.next = subHead;

        // 交换之后，原来的第二位的那个节点 headNext 变成了第一位
        // 把它返回就行
        return headNext;
    }
}
```

2、C++ 代码

```
class solution {
public:
    ListNode* swapPairs(ListNode* head) {

        // 寻找递归终止条件
        // 1、head 指向的结点为 null
        // 2、head 指向的结点的下一个结点为 null
        // 在这两种情况下，一个节点或者空节点无论怎么交换操作，都是原来的 head
        if( head == NULL || head->next == NULL) {
            return head;
        }

        // 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点或者最后两个节点
        ListNode *subHead = swapPairs(head->next->next);

        // 对于 head 这个节点来说，它是和它相邻的右边这个节点进行交换操作
        // 所以先要保存这个节点，并且经过上述递归终止条件的判断，head.next 是必然存在的
        ListNode *headNext = head->next;

        // head 原来是指向 headNext
        // 交换之后
        // headNext 指向 head
        headNext->next = head;

        // headNext 原来是指向 subHead
        // 现在需要让 head 指向 subHead
        head->next = subHead;

        // 交换之后，原来的第二位的那个节点 headNext 变成了第一位
        // 把它返回就行
        return headNext;
    }
};
```

3、Python 代码

```
class solution:
    def swapPairs(self, head: ListNode) -> ListNode:

        # 寻找递归终止条件
        # 1、head 指向的结点为 null
        # 2、head 指向的结点的下一个结点为 null
        # 在这两种情况下，一个节点或者空节点无论怎么交换操作，都是原来的 head
        if head == None or head.next == None :
            return head
```

```

# 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点或者最后两个节点
subHead = self.swapPairs(head.next.next)

# 对于 head 这个节点来说，它是和它相邻的右边这个节点进行交换操作
# 所以先要保存这个节点，并且经过上述递归终止条件的判断，head.next 是必然存在的
headNext = head.next

# head 原来是指向 headNext
# 交换之后
# headNext 指向 head
headNext.next = head

# headNext 原来是指向 subHead
# 现在需要让 head 指向 subHead
head.next = subHead

# 交换之后，原来的第二位的那个节点 headNext 变成了第一位
# 把它返回就行
return headNext

```

复杂度分析

时间复杂度：O(n)，其中 n 是链表的节点数量。需要对每个节点进行更新指针的操作。

空间复杂度：O(n)，其中 n 是链表的节点数量。空间复杂度主要取决于递归调用的栈空间。

第四天

LeetCode 21、合并两个有序链表

一、题目描述

将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

二、题目解析

无

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值

```

```

ListNode dummy = new ListNode(-1);

// 设置一个指针，指向虚拟节点
ListNode pre = dummy;

// 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
while (l1 != null && l2 != null) {
    // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
    if (l1.val <= l2.val) {
        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l1
        pre.next = l1;
        // 让 l1 向后移动
        l1 = l1.next;
    } else {
        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l2
        pre.next = l2;
        // 让 l2 向后移动
        l2 = l2.next;
    }
    // 让 pre 向后移动
    pre = pre.next;
}

// 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
// 直接把剩下的节点加入到 pre 的 next 指针位置

// 如果 l1 中还有节点
if (l1 != null) {
    // 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l1;
}

// 如果 l2 中还有节点
if (l2 != null) {
    // 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l2;
}

// 最后返回虚拟节点的 next 指针
return dummy.next;
}
}

```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class solution {
public:
    ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        ListNode *dummy = new ListNode(-1);

        // 设置一个指针，指向虚拟节点
        ListNode *pre = dummy;

        // 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
        while (l1 != NULL && l2 != NULL) {
            // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
            if (l1->val <= l2->val) {
                // 让 pre 指向节点的 next 指针指向这个更小值的节点
                // 即指向 l1
                pre->next = l1;
                // 让 l1 向后移动
                l1 = l1->next;
            } else {
                // 让 pre 指向节点的 next 指针指向这个更小值的节点
                // 即指向 l2
                pre->next = l2;
                // 让 l2 向后移动
                l2 = l2->next;
            }
            // 让 pre 向后移动
            pre = pre->next;
        }

        // 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
        // 直接把剩下的节点加入到 pre 的 next 指针位置

        // 如果 l1 中还有节点
        if (l1 != NULL) {
            // 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
            pre->next = l1;
        }

        // 如果 l2 中还有节点
        if (l2 != NULL) {
            // 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
            pre->next = l2;
        }
    }
};
```

```

        // 最后返回虚拟节点的 next 指针
        return dummy->next;
    }
};

```

3、Python 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution:
    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:

        # 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        dummy = ListNode(-1)

        # 设置一个指针，指向虚拟节点
        pre = dummy

        # 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
        while l1 and l2:
            # 如果 l1 当前节点的值小于等于了 l2 当前节点的值
            if l1.val <= l2.val :
                # 让 pre 指向节点的 next 指针指向这个更小值的节点
                # 即指向 l1
                pre.next = l1
                # 让 l1 向后移动
                l1 = l1.next
            else:
                # 让 pre 指向节点的 next 指针指向这个更小值的节点
                # 即指向 l2
                pre.next = l2
                # 让 l2 向后移动
                l2 = l2.next

            # 让 pre 向后移动
            pre = pre.next

        # 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
        # 直接把剩下的节点加入到 pre 的 next 指针位置

        # 如果 l1 中还有节点
        if l1 != None :
            # 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
            pre.next = l1

```



```
# 如果 l2 中还有节点
if l2 != None :
    # 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l2;

# 最后返回虚拟节点的 next 指针
return dummy.next
```

四、复杂度分析

时间复杂度： $O(n + m)$ ，其中 n 和 m 分别为两个链表的长度。因为每次循环迭代中， $l1$ 和 $l2$ 只有一个元素会被放进合并链表中，因此 `while` 循环的次数不会超过两个链表的长度之和。所有其他操作的时间复杂度都是常数级别的，因此总的时间复杂度为 $O(n+m)$ 。

空间复杂度： $O(1)$ 。只需要常数的空间存放若干变量。

LeetCode 237、删除链表的节点

一、题目描述

有一个单链表的 `head`，我们想删除它其中的一个节点 `node`。

给你一个需要删除的节点 `node`。你将 **无法访问** 第一个节点 `head`。

链表的所有值都是 **唯一的**，并且保证给定的节点 `node` 不是链表中的最后一个节点。

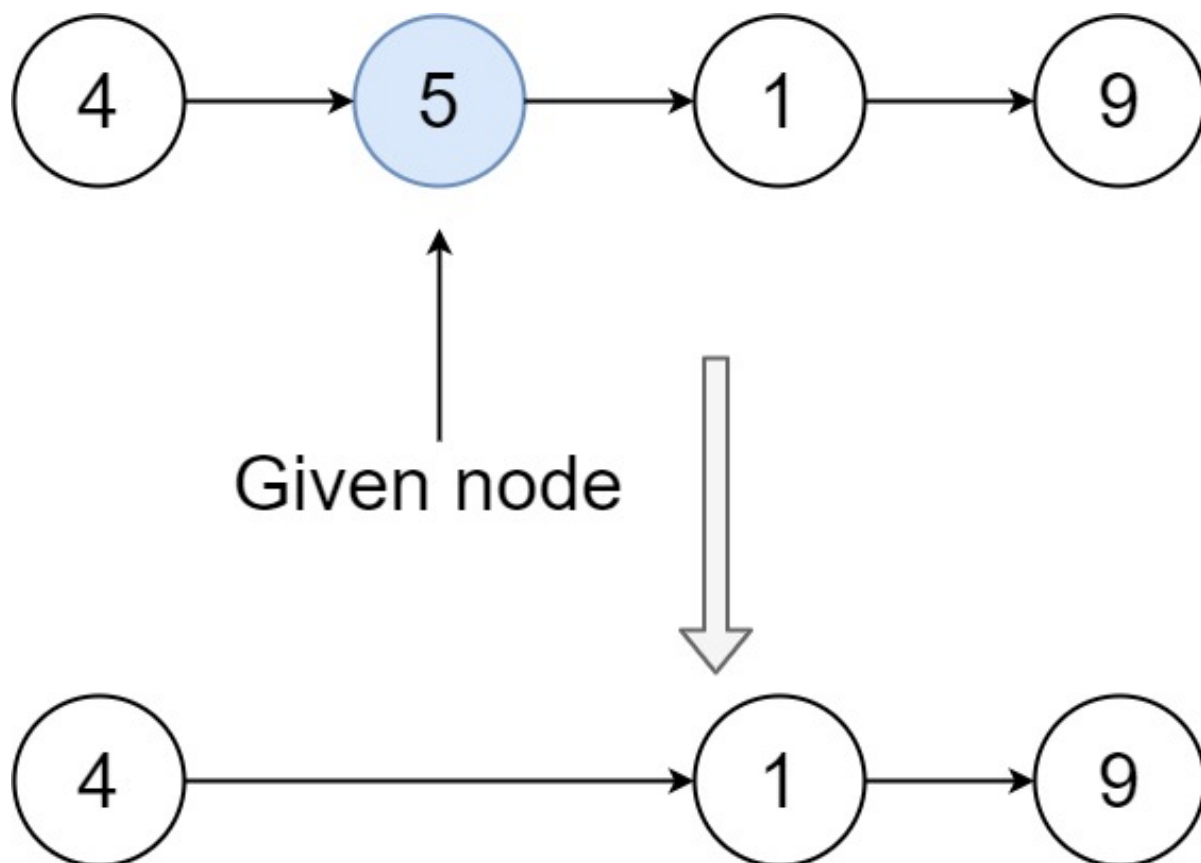
删除给定的节点。注意，删除节点并不是指从内存中删除它。这里的意思是：

- 给定节点的值不应该存在于链表中。
- 链表中的节点数应该减少 1。
- `node` 前面的所有值顺序相同。
- `node` 后面的所有值顺序相同。

自定义测试：

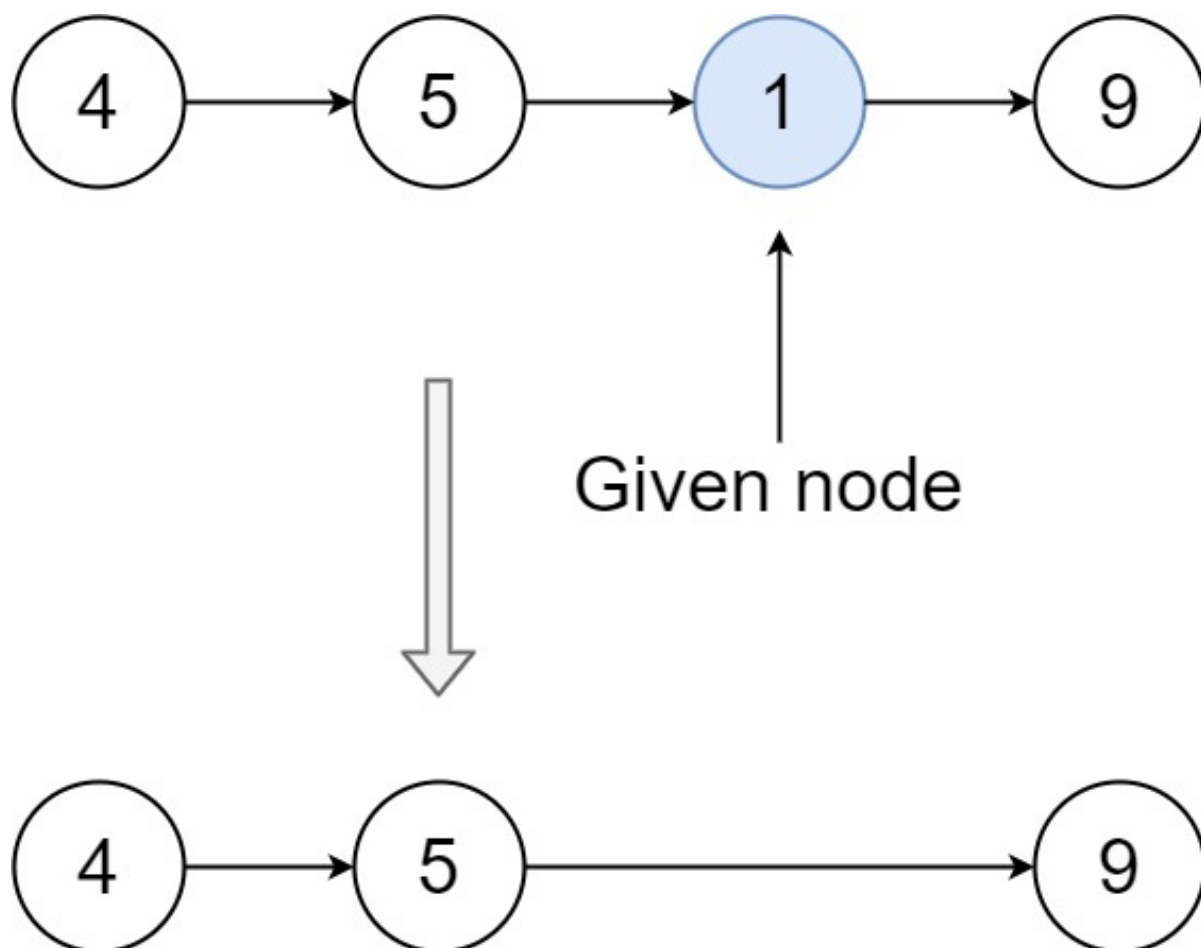
- 对于输入，你应该提供整个链表 `head` 和要给出的节点 `node`。`node` 不应该是链表的最后一个节点，而应该是链表中的一个实际节点。
- 我们将构建链表，并将节点传递给你的函数。
- 输出将是调用你函数后的整个链表。

示例 1：



输入: head = [4,5,1,9], node = 5 输出: [4,1,9] 解释: 指定链表中值为 5 的第二个节点, 那么在调用了你的函数之后, 该链表应变为 4 -> 1 -> 9

示例 2:



输入: head = [4,5,1,9], node = 1 输出: [4,5,9] 解释: 指定链表中值为 1 的第三个节点, 那么在调用了你的函数之后, 该链表应变为 4 -> 5 -> 9

提示:

- 链表中节点的数目范围是 [2, 1000]
- $-1000 \leq \text{Node.val} \leq 1000$
- 链表中每个节点的值都是 唯一 的
- 需要删除的节点 node 是 链表**中的节点** , 且 不是末尾节点

二、参考代码

```
class Solution:
    def deleteNode(self, node):
        """
        :type node: ListNode
        :rtype: void Do not return anything, modify node in-place instead.
        """
        node.val = node.next.val

        node.next = node.next.next
```

LeetCode 328、奇偶链表

一、题目描述

给定一个单链表, 把所有的奇数节点和偶数节点分别排在一起。请注意, 这里的奇数节点和偶数节点指的是节点编号的奇偶性, 而不是节点的值值的奇偶性。

请尝试使用原地算法完成。你的算法的空间复杂度应为 $O(1)$, 时间复杂度应为 $O(\text{nodes})$, nodes 为节点总数。

示例 1:

输入: 1->2->3->4->5->NULL
输出: 1->3->5->2->4->NULL

示例 2:

输入: 2->1->3->5->6->4->7->NULL
输出: 2->3->6->7->1->5->4->NULL

说明:

- 应当保持奇数节点和偶数节点的相对顺序。
- 链表的第一个节点视为奇数节点, 第二个节点视为偶数节点, 以此类推。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 奇偶链表 ( LeetCode 328 ) :https://leetcode-cn.com/problems/odd-even-linked-list/
class Solution {
    public ListNode oddEvenList(ListNode head) {

        // 边界情况处理，如果链表为空或者只有一个节点，返回 head 就行
        if(head == null || head.next == null ) return head;

        // 设置一个指针，指向链表的头节点，odd 代表奇数节点的头节点
        ListNode odd = head;

        // 设置一个指针，指向链表的头节点的下一个节点，even 代表偶数节点的头节点
        ListNode even = head.next;

        // 设置一个指针，指向偶数节点的头节点，最终让奇数节点的尾节点的 next 指针指向它
        ListNode evenHead = even;

        // 从偶数链表的头节点开始向后遍历
        // 如果当前节点为空，或者后一节点为空，那么说明整个链表已经查看完毕，不需要再遍历了
        while(even != null && even.next != null){
            // 原先奇数节点的下一个节点是偶数节点，即 even 这个节点
            // 根据数学知识，奇数后面一定是偶数，偶数后面一定是奇数
            // 那么 even.next 节点必然是奇数节点
            // 所以让 odd 这个奇数节点的 next 指针指向 even.next 这个奇数节点
            // 这样，odd 上面都是奇数
            odd.next = even.next;

            // 让 odd 移动到最新的由奇数节点组成的链表的尾部位置
            odd = odd.next;

            // 这个时候，odd.next 必然是偶数节点
            // 所以让 even 这个偶数节点的 next 指针指向 odd.next 这个偶数节点
            even.next = odd.next;

            // 让 even 移动到最新的由偶数节点组成的链表的尾部位置
            even = even.next;
        }

        // 此时，原链表所有的节点已经遍历完毕
        // odd 上都是奇数节点
        // even 都是偶数节点
        // 根据题目要求，奇数节点都在偶数节点之前
        // 所以让此时右奇数节点组成的链表的尾部的 next 指针指向由偶数节点组成的链表的头部
    }
}
```

```

        odd.next = evenHead;

        // 最后返回原链表的头部节点就可以了
        // 链表的头部节点没有发生过变化，因为它是奇数节点，并且是第一个奇数节点
        return head;
    }
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 奇偶链表 ( LeetCode 328 ) :https://leetcode-cn.com/problems/odd-even-linked-list/
class Solution {
public:
    ListNode* oddEvenList(ListNode* head) {
        // 边界情况处理，如果链表为空或者只有一个节点，返回 head 就行
        if(head == NULL || head->next == NULL ) return head;

        // 设置一个指针，指向链表的头节点，odd 代表奇数节点的头节点
        ListNode* odd = head;

        // 设置一个指针，指向链表的头节点的下一个节点，even 代表偶数节点的头节点
        ListNode* even = head->next;

        // 设置一个指针，指向偶数节点的头节点，最终让奇数节点的尾节点的 next 指针指向它
        ListNode* evenHead = even;

        // 从偶数链表的头节点开始向后遍历
        // 如果当前节点为空，或者后一节点为空，那么说明整个链表已经查看完毕，不需要再遍历了
        while(even != NULL && even->next != NULL){
            // 原先奇数节点的下一个节点是偶数节点，即 even 这个节点
            // 根据数学知识，奇数后面一定是偶数，偶数后面一定是奇数
            // 那么 even->next 节点必然是奇数节点
            // 所以让 odd 这个奇数节点的 next 指针指向 even->next 这个奇数节点
            // 这样，odd 上面都是奇数
            odd->next = even->next;
            // 让 odd 移动到最新的由奇数节点组成的链表的尾部位置
            odd = odd->next;

            // 这个时候，odd->next 必然是偶数节点
            // 所以让 even 这个偶数节点的 next 指针指向 odd->next 这个偶数节点
            even->next = odd->next;
            // 让 even 移动到最新的由偶数节点组成的链表的尾部位置
            even = even->next;
        }
    }
}

```

```

// 此时，原链表所有的节点已经遍历完毕
// odd 上都是奇数节点
// even 都是偶数节点
// 根据题目要求，奇数节点都在偶数节点之前
// 所以让此时右奇数节点组成的链表的尾部的 next 指针指向由偶数节点组成的链表的头部
odd->next = evenHead;

// 最后返回原链表的头部节点就可以了
// 链表的头部节点没有发生过变化，因为它是奇数节点，并且是第一个奇数节点
return head;
}
};

```

3、Python 代码

```

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        # 边界情况处理，如果链表为空或者只有一个节点，返回 head 就行
        if head == None or head.next == None :
            return head

        # 设置一个指针，指向链表的头节点，odd 代表奇数节点的头节点
        odd = head

        # 设置一个指针，指向链表的头节点的下一个节点，even 代表偶数节点的头节点
        even = head.next

        # 设置一个指针，指向偶数节点的头节点，最终让奇数节点的尾节点的 next 指针指向它
        evenHead = even

        # 从偶数链表的头节点开始向后遍历
        # 如果当前节点为空，或者后一节点为空，那么说明整个链表已经查看完毕，不需要再遍历了
        while even != None and even.next != None :
            # 原先奇数节点的下一个节点是偶数节点，即 even 这个节点
            # 根据数学知识，奇数后面一定是偶数，偶数后面一定是奇数
            # 那么 even.next 节点必然是奇数节点
            # 所以让 odd 这个奇数节点的 next 指针指向 even.next 这个奇数节点
            # 这样，odd 上面都是奇数
            odd.next = even.next
            # 让 odd 移动到最新的由奇数节点组成的链表的尾部位置
            odd = odd.next

            # 这个时候，odd.next 必然是偶数节点
            # 所以让 even 这个偶数节点的 next 指针指向 odd.next 这个偶数节点
            even.next = odd.next
            # 让 even 移动到最新的由偶数节点组成的链表的尾部位置
            even = even.next

```

```
# 此时，原链表所有的节点已经遍历完毕
# odd 上都是奇数节点
# even 都是偶数节点
# 根据题目要求，奇数节点都在偶数节点之前
# 所以让此时由奇数节点组成的链表的尾部的 next 指针指向由偶数节点组成的链表的头部
odd.next = evenHead

# 最后返回原链表的头部节点就可以了
# 链表的头部节点没有发生过变化，因为它是奇数节点，并且是第一个奇数节点
return head
```

四、复杂度分析

时间复杂度：O(n)，其中 n 是链表的节点数。需要遍历链表中的每个节点，并更新指针。

空间复杂度：O(1)。只需要维护有限的指针。

LeetCode 876、链表的中点

一、题目描述

给定一个头结点为 `head` 的非空单链表，返回链表的中点。

如果有两个中间结点，则返回第二个中间结点。

示例 1：

输入：[1,2,3,4,5]

输出：此列表中的结点 3（序列化形式：[3,4,5]）

返回的结点值为 3。（测评系统对该结点序列化表述是 [3,4,5]）。

注意，我们返回了一个 `ListNode` 类型的对象 `ans`，这样：

`ans.val = 3, ans.next.val = 4, ans.next.next.val = 5`，以及 `ans.next.next.next = NULL`。

示例 2：

输入：[1,2,3,4,5,6]

输出：此列表中的结点 4（序列化形式：[4,5,6]）

由于该列表有两个中间结点，值分别为 3 和 4，我们返回第二个结点。

提示：

- 给定链表的结点数介于 1 和 100 之间。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 链表的中间结点 (LeetCode 876) : https://leetcode.cn/problems/middle-of-the-linked-list/
class Solution {
    public ListNode middleNode(ListNode head) {

        // 设置两个指针，一开始都指向链表的头节点
        ListNode slow = head;

        ListNode fast = head;

        // 接下来，让这两个指针向前移动
        // 如果可以移动，那么就会让快指针每次移动两步，慢指针每次移动一步
        // 而快指针可以移动两步的前提就是当前节点不为空，同时下一节点也不为空
        // 这样才能保证 fast.next 有值、fast.next.next 有值
        while(fast != null && fast.next != null){

            // 慢指针每次移动一步
            slow = slow.next;

            // 快指针每次移动两步
            fast = fast.next.next;
        }

        // 最后，slow 指向的就是中间节点
        return slow;
    }
}
```

2、**C++** 代码

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {

        // 设置两个指针，一开始都指向链表的头节点
        ListNode* slow = head;

        ListNode* fast = head;

        // 接下来，让这两个指针向前移动
```



```

// 如果可以移动，那么就会让快指针每次移动两步，慢指针每次移动一步
// 而快指针可以移动两步的前提就是当前节点不为空，同时下一节点也不为空
// 这样才能保证 fast.next 有值、fast.next.next 有值
while(fast != NULL && fast->next != NULL){

    // 慢指针每次移动一步
    slow = slow->next;

    // 快指针每次移动两步
    fast = fast->next->next;
}

// 最后，slow 指向的就是中间节点
return slow;

}
};

```

3、Python 代码

```

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        # 设置两个指针，一开始都指向链表的头节点
        slow = head

        fast = head

        # 接下来，让这两个指针向前移动
        # 如果可以移动，那么就会让快指针每次移动两步，慢指针每次移动一步
        # 而快指针可以移动两步的前提就是当前节点不为空，同时下一节点也不为空
        # 这样才能保证 fast.next 有值、fast.next.next 有值
        while fast and fast.next :

            # 慢指针每次移动一步
            slow = slow.next

            # 快指针每次移动两步
            fast = fast.next.next

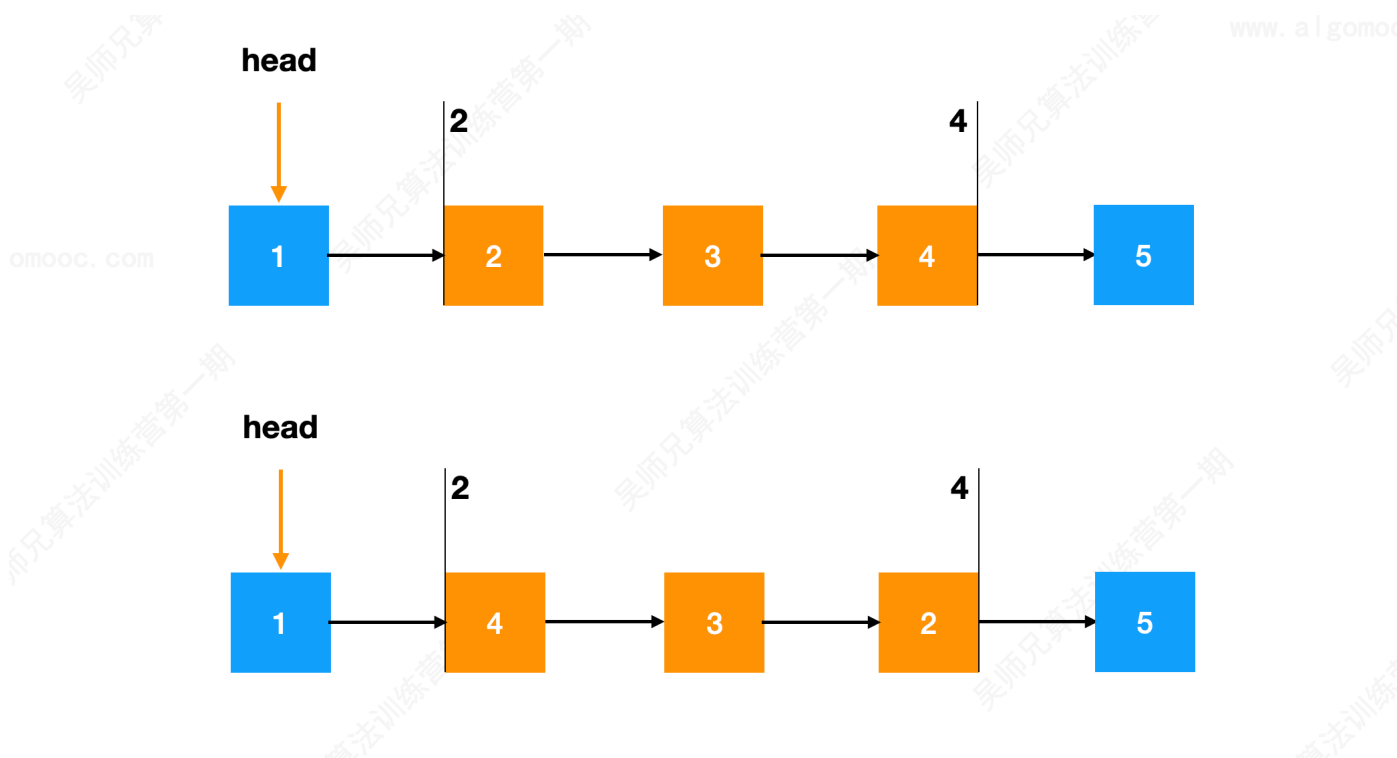
        # 最后，slow 指向的就是中间节点
        return slow

```

LeetCode 92 、反转链表 II

一、题目描述

给你单链表的头指针 head 和两个整数 left 和 right ，其中 $left \leq right$ 。请你反转从位置 left 到位置 right 的链表节点，返回 反转后的链表 。



二、题目解析

- 1、构建一个虚拟节点，让它指向原链表的头节点。
- 2、设置两个指针，pre 指针指向以虚拟头节点为链表的头部位置，cur 指针指向原链表的头部位置。
- 3、让着两个指针向前移动，直到 pre 指向了第一个要反转的节点的前面那个节点，而 cur 指向了第一个要反转的节点。
- 4、开始指向翻转操作
 - 1)、设置临时变量 temp，temp 是 cur 的 next 位置，保存当前需要翻转节点的后面的节点，我们需要交换 temp 和 cur
 - 2)、让 cur 的 next 位置变成 temp 的下一个节点
 - 3)、让 temp 的 next 位置变成 cur
 - 4)、让 pre 的 next 位置变成 temp

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 反转链表 II （ LeetCode 92 ）：https://leetcode-cn.com/problems/reverse-linked-list-ii
```

```

class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {

        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        // 设置虚拟节点的目的是为了让原链表中所有节点就都可以按照统一的方式进行翻转
        // 比如如果翻转的区间包含了原链表中的第一个位置，那么如果不设置 dummy
        // 在翻转的过程中需要设置其它的临时变量来保持第一位置节点的指针
        // 具体可以通过动画来理解
        ListNode dummy = new ListNode(-1);

        // 让虚拟节点指向原链表的头部
        dummy.next = head;

        // 设置一个指针，指向以虚拟头节点为链表的头部位置
        ListNode pre = dummy;

        // 设置一个指针，指向原链表的头部位置

        ListNode cur = head;

        // 从虚拟头节点出发，pre 走 left - 1 步找到需要翻转的左区间
        // for 循环结束后，pre 的右节点是需要翻转的节点
        // for 循环结束后，cur 指向的就是需要翻转的节点
        for (int i = 0; i < left - 1; i++) {

            // pre 不断的向右移动，直到走到翻转的左区间为止
            pre = pre.next;
            // cur 不断的向右移动，找到了需要翻转的第一个节点
            cur = cur.next;
        }

        // 开始翻转这些节点
        for (int i = 0; i < right - left; i++) {

            // 设置临时变量，保存当前需要翻转节点的后面的节点
            ListNode temp = cur.next;

            // 这个时候，让 temp 和 cur 两个节点翻转一下
            // 比如，一开始 i = 0 的时候 cur 为 2，temp 为 3
            // 执行完下面的代码，如果原链表是
            // 1 --> 2 --> 3 --> 4 --> 5
            // 变成了
            // 1 --> 3 --> 2 --> 4 --> 5

            // cur 的下一节点是等号右侧的值
            // i = 0 的时候，cur 为 2，cur.next.next 的值是 4
            // 所以，这行代码让 cur 的下一节点不是 3，而是 4

```

```

        // 2 --> 4
        // 等价于 cur.next = temp.next
        cur.next = cur.next.next;

        // temp 的下一节点是等号右侧的值
        // i = 0 的时候, temp 为 3, pre 为 1, pre 下一节点的值是 2
        // 所以, 这行代码让 temp 的下一节点不是 4, 而是 2
        // 3 --> 2
        temp.next = pre.next;

        // pre 的下一节点是等号右侧的值
        // i = 0 的时候, pre 为 1, temp 的值是 3
        // 所以, 这行代码让 pre 的下一节点为 3
        // 1 --> 3

        pre.next = temp;

        // i = 0 结束之后, 链表变成了
        // 1 --> 3 --> 2 --> 4 --> 5
    }

    // 最后返回虚拟头节点的下一个节点, 因为虚拟节点不在链表中
    return dummy.next;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 反转链表 II ( LeetCode 92 ) : https://leetcode-cn.com/problems/reverse-linked-list-ii
class Solution {
public:
    ListNode* reverseBetween(ListNode* head, int left, int right) {
        // 一开始设置一个虚拟节点, 它的值为 -1, 它的值可以设置为任何的数, 因为我们根本不需要使用它的值
        // 设置虚拟节点的目的是为了让原链表中所有节点就都可以按照统一的方式进行翻转
        // 比如如果翻转的区间包含了原链表中的第一个位置, 那么如果不设置 dummy
        // 在翻转的过程中需要设置其它的临时变量来保持第一位置节点的指针
        // 具体可以通过动画来理解
        ListNode *dummy = new ListNode(-1);

        // 让虚拟节点指向原链表的头部
        dummy->next = head;

        // 设置一个指针, 指向以虚拟头节点为链表的头部位置
        ListNode *pre = dummy;
    }
}

```

```

// 设置一个指针，指向原链表的头部位置
ListNode *cur = head;

// 从虚拟头节点出发，pre 走 left - 1 步找到需要翻转的左区间
// for 循环结束后，pre 的右节点是需要翻转的节点
// for 循环结束后，cur 指向的就是需要翻转的节点
for (int i = 0; i < left - 1; i++) {

    // pre 不断的向右移动，直到走到翻转的左区间为止
    pre = pre->next;
    // cur 不断的向右移动，找到了需要翻转的第一个节点
    cur = cur->next;
}

// 开始翻转这些节点
for (int i = 0; i < right - left; i++) {

    // 设置临时变量，保存当前需要翻转节点的后面的节点
    ListNode *temp = cur->next;

    // 这个时候，让 temp 和 cur 两个节点翻转一下
    // 比如，一开始 i = 0 的时候 cur 为 2， temp 为 3
    // 执行完下面的代码，如果原链表是
    // 1 --> 2 --> 3 --> 4 --> 5
    // 变成了
    // 1 --> 3 --> 2 --> 4 --> 5

    // cur 的下一节点是等号右侧的值
    // i = 0 的时候， cur 为 2， cur->next->next 的值是 4
    // 所以，这行代码让 cur 的下一节点不是 3，而是 4
    // 2 --> 4
    // 等价于 cur->next = temp->next
    cur->next = cur->next->next;

    // temp 的下一节点是等号右侧的值
    // i = 0 的时候， temp 为 3， pre 为 1， pre 下一节点的值是 2
    // 所以，这行代码让 temp 的下一节点不是 4，而是 2
    // 3 --> 2
    temp->next = pre->next;

    // pre 的下一节点是等号右侧的值
    // i = 0 的时候， pre 为 1， temp 的值是 3
    // 所以，这行代码让 pre 的下一节点为 3
    // 1 --> 3

    pre->next = temp;
}

```

```

        // i = 0 结束之后，链表变成了
        // 1 --> 3 --> 2 --> 4 --> 5

    }

    // 最后返回虚拟头节点的下一个节点，因为虚拟节点不在链表中
    return dummy->next;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 反转链表 II ( LeetCode 92 ) : https://leetcode-cn.com/problems/reverse-linked-list-ii
class Solution:
    def reverseBetween(self, head: ListNode, left: int, right: int) -> ListNode:
        # 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        # 设置虚拟节点的目的是为了原链表中所有节点就都可以按照统一的方式进行翻转
        # 比如如果翻转的区间包含了原链表中的第一个位置，那么如果不设置 dummy
        # 在翻转的过程中需要设置其它的临时变量来保持第一位置节点的指针
        # 具体可以通过动画来理解
        dummy = ListNode(-1)

        # 让虚拟节点指向原链表的头部
        dummy.next = head

        # 设置一个指针，指向以虚拟头节点为链表的头部位置
        pre = dummy

        # 设置一个指针，指向原链表的头部位置
        cur = head

        # 从虚拟头节点出发，pre 走 left - 1 步找到需要翻转的左区间
        # for 循环结束后，pre 的右节点是需要翻转的节点
        # for 循环结束后，cur 指向的就是需要翻转的节点
        for _ in range(left - 1):
            # pre 不断的向右移动，直到走到翻转的左区间为止
            pre = pre.next
            # cur 不断的向右移动，找到了需要翻转的第一个节点
            cur = cur.next

        # 开始翻转这些节点
        for _ in range(right - left):

```

```

# 设置临时变量，保存当前需要翻转节点的后面的节点
temp = cur.next

# 这个时候，让 temp 和 cur 两个节点翻转一下
# 比如，一开始 i = 0 的时候 cur 为 2， temp 为 3
# 执行完下面的代码，如果原链表是
# 1 --> 2 --> 3 --> 4 --> 5
# 变成了
# 1 --> 3 --> 2 --> 4 --> 5

# cur 的下一节点是等号右侧的值
# i = 0 的时候， cur 为 2， cur.next.next 的值是 4
# 所以，这行代码让 cur 的下一节点不是 3，而是 4
# 2 --> 4
# 等价于 cur.next = temp.next
cur.next = cur.next.next

# temp 的下一节点是等号右侧的值
# i = 0 的时候， temp 为 3， pre 为 1， pre 下一节点的值是 2
# 所以，这行代码让 temp 的下一节点不是 4，而是 2
# 3 --> 2
temp.next = pre.next

# pre 的下一节点是等号右侧的值
# i = 0 的时候， pre 为 1， temp 的值是 3
# 所以，这行代码让 pre 的下一节点为 3
# 1 --> 3

pre.next = temp

# i = 0 结束之后，链表变成了
# 1 --> 3 --> 2 --> 4 --> 5

# 最后返回虚拟头节点的下一个节点，因为虚拟节点不在链表中
return dummy.next

```

四、复杂度分析

时间复杂度：O(n)，其中 n 是链表总节点数，最多只遍历了链表一次，就完成了反转。

空间复杂度：O(1)，只使用到常数个变量。

第五天

LeetCode 150、逆波兰表达式求值

一、题目描述

根据 [逆波兰表示法](#)，求表达式的值。

有效的算符包括 `+`、`-`、`*`、`/`。每个运算对象可以是整数，也可以是另一个逆波兰表达式。

注意 两个整数之间的除法只保留整数部分。

可以保证给定的逆波兰表达式总是有效的。换句话说，表达式总会得出有效数值且不存在除数为 0 的情况。

示例 1:

输入: tokens = ["2","1","+","3","*"]
输出: 9
解释: 该算式转化为常见的中缀算术表达式为: ((2 + 1) * 3) = 9

示例 2:

输入: tokens = ["4","13","5","/","+"]
输出: 6
解释: 该算式转化为常见的中缀算术表达式为: (4 + (13 / 5)) = 6

示例 3:

输入: tokens = ["10","6","9","3","+","-11","*","/","*", "17","+","5","+"]
输出: 22
解释: 该算式转化为常见的中缀算术表达式为:
((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22

提示:

- `1 <= tokens.length <= 10^4`
- `tokens[i]` 是一个算符 (`+`、`-`、`*` 或 `/`)，或是在范围 `[-200, 200]` 内的一个整数

逆波兰表达式:

逆波兰表达式是一种后缀表达式，所谓后缀就是指算符写在后面。

- 平常使用的算式则是一种中缀表达式，如 `( 1 + 2 ) * ( 3 + 4 )`。
- 该算式的逆波兰表达式写法为 `(( 1 2 + ) ( 3 4 + ) * )`。

逆波兰表达式主要有以下两个优点:

- 去掉括号后表达式无歧义，上式即便写成 `1 2 + 3 4 + *` 也可以依据次序计算出正确结果。

- 适合用栈操作运算：遇到数字则入栈；遇到算符则取出栈顶两个数字进行计算，并将结果压入栈中

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 逆波兰表达式求值 (LeetCode 150) : https://leetcode.cn/problems/evaluate-reverse-polish-notation/
class Solution {
    public int evalRPN(String[] tokens) {

        // 使用一个栈存储操作数，从左到右遍历逆波兰表达式
        Stack<Integer> result = new Stack<>();

        // 对于一个表达式来说，由两个操作数和一个运算符构成
        // 比如加法：① + ②
        // 比如减法：① - ②
        // 比如乘法：① * ②
        // 比如除法：① / ②

        // 先出栈的是右操作数，后出栈的是左操作数

        // 先出栈的是右操作数
        int rightNum;

        // 后出栈的是左操作数
        int leftNum;

        // 一个表达式的计算结果
        int ans;

        // 遍历字符串数组
        for(String token : tokens){

            // 如果是 +
            if("+".equals(token)){

                // 先出栈的是右操作数
                rightNum = result.pop();

                // 后出栈的是左操作数
                leftNum = result.pop();

                // 计算结果
```

```
        ans = leftNum + rightNum;

// 如果是 -
    }else if("-".equals(token)){

        // 先出栈的是右操作数
        rightNum = result.pop();

        // 后出栈的是左操作数
        leftNum = result.pop();

        // 计算结果
        ans = leftNum - rightNum;

// 如果是 *
    }else if(".".equals(token)){

        // 先出栈的是右操作数
        rightNum = result.pop();

        // 后出栈的是左操作数
        leftNum = result.pop();

        // 计算结果
        ans = leftNum * rightNum;

// 如果是 /
    }else if("/".equals(token)){

        // 先出栈的是右操作数
        rightNum = result.pop();

        // 后出栈的是左操作数
        leftNum = result.pop();

        // 计算结果
        ans = leftNum / rightNum;

// 如果是非运算符
    }else{

        // 转换为数字
        ans = Integer.valueOf(token);

    }

// 存储结果
    result.push(ans);
```

```

    }

    // 返回栈顶元素
    return result.pop();
}
}

```

2、C++ 代码

```

class Solution {
public:
    int evalRPN(vector<string>& tokens) {

        // 使用一个栈存储操作数，从左到右遍历逆波兰表达式
        stack<int> result;

        // 对于一个表达式来说，由两个操作数和一个运算符构成
        // 比如加法：① + ②
        // 比如减法：① - ②
        // 比如乘法：① * ②
        // 比如除法：① / ②

        // 先出栈的是右操作数，后出栈的是左操作数

        // 先出栈的是右操作数
        long int rightNum;

        // 后出栈的是左操作数
        long int leftNum;

        // 一个表达式的计算结果
        long int ans;

        // 遍历字符串数组
        for (int i = 0; i < tokens.size(); i++) {

            // 如果是 +
            if(tokens[i] == "+"){

                // 先出栈的是右操作数
                rightNum = result.top();

                result.pop();

                // 后出栈的是左操作数
                leftNum = result.top();
            }
        }
    }
}

```

```
result.pop();

// 计算结果
ans = leftNum + rightNum;

// 如果是 -
}else if(tokens[i] == "-"){

    // 先出栈的是右操作数
    rightNum = result.top();

    result.pop();

    // 后出栈的是左操作数
    leftNum = result.top();

    result.pop();

    // 计算结果
    ans = leftNum - rightNum;

// 如果是 *
}else if(tokens[i] == "*"){

    // 先出栈的是右操作数
    rightNum = result.top();

    result.pop();

    // 后出栈的是左操作数
    leftNum = result.top();

    result.pop();

    // 计算结果
    ans = leftNum * rightNum;

// 如果是 /
}else if(tokens[i] == "/"){

    // 先出栈的是右操作数
    rightNum = result.top();

    result.pop();

    // 后出栈的是左操作数
    leftNum = result.top();
```

```

        result.pop();

        // 计算结果
        ans = leftNum / rightNum;

        // 如果是非运算符
    }else{

        // 转换为数字
        ans = stoi(tokens[i]);

    }

    // 存储结果
    result.push(ans);

}

// 返回栈顶元素
return result.top();

}

};

```

3、Python 代码

```

class Solution(object):
    def evalRPN(self, tokens):
        stack = []
        for token in tokens:
            try:
                stack.append(int(token))
            except:
                num2 = stack.pop()
                num1 = stack.pop()
                stack.append(self.evaluate(num1, num2, token))
        return stack[0]

    def evaluate(self, num1, num2, op):
        if op == "+":
            return num1 + num2
        elif op == "-":
            return num1 - num2
        elif op == "*":
            return num1 * num2
        elif op == "/":
            return int(num1 / float(num2))

```

四、动画理解（没有声音）

LeetCode 155、最小栈

一、题目描述

设计一个支持 push , pop , top 操作，并能在常数时间内检索到最小元素的栈。

- `push(x)` —— 将元素 x 推入栈中。
- `pop()` —— 删除栈顶的元素。
- `top()` —— 获取栈顶元素。
- `getMin()` —— 检索栈中的最小元素。

二、题目解析

由于需要在常数时间内找到最小的元素，那么说明肯定是不能使用遍历，因为遍历是 $O(n)$ 级别的时间，那么只能使用辅助空间进行存储，这是一种空间换时间的思想。

这里我们设置两个栈：**普通栈**和**辅助栈**。

1、**push** 操作

普通栈：直接添加 push 进来的值

辅助栈：每次 push 一个新元素的时候，将普通栈中最小的元素 push 进辅助栈中

2、pop 操作

普通栈：直接移除普通栈中的栈顶元素

辅助栈：判断普通栈中刚刚移除的栈顶元素值是否和此时辅助栈中的栈顶元素相同，如果是则将辅助栈中的栈顶元素移除，否则不执行操作，这样的目的是为了让辅助栈中的栈顶元素始终是普通栈中的最小值。

3、top 操作

普通栈：返回普通栈的栈顶元素

辅助栈：不执行操作

4、getMin 操作

普通栈：不执行操作

辅助栈：返回辅助栈的栈顶元素

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class MinStack {
    // 首先定义好两个栈

    // 一个栈叫做 stack，负责栈的正常操作
    Stack<Integer> stack;
    // 一个栈叫做 minStack，负责获取 stack 中的最小值，它等价于遍历 stack 中的所有元素，把升序的数字都删除掉，留下一个从栈底到栈顶降序的栈
    Stack<Integer> minStack;

    public MinStack() {
        // 在这个函数中初始化两个栈，传入的参数为空，返回也为空

        // 初始化 stack
        stack = new Stack<>();

        // 初始化 minStack
        minStack = new Stack<>();
    }

    public void push(int x) {
        // 新添加的元素添加到 stack 中
        stack.push(x);

        // 判断 minStack 是否为空，如果为空，直接同时把新添加的元素添加到 minStack 中

        // 如果 minStack 不为空
        if (!minStack.isEmpty()) {
            // 获取 minStack 的栈顶元素
            int top = minStack.peek();
            // 只有新添加的元素不大于 top 才允许添加到 minStack 中，目的是为了让 minStack 从栈底到栈顶是降序的
            if (x <= top) {
                // 此时，新添加的元素 x 小于 top，加入到 minStack 后依旧是从栈底到栈顶是降序的
                minStack.push(x);
            }
        } else {
            // 此时，minStack 中没有元素，所以直接把新添加的元素添加到 minStack 中
            minStack.push(x);
        }
    }

    public void pop() {
```

```

        // 让 stack 执行正常的 pop 操作就行
        int pop = stack.pop();

        // 由于 minStack 中的所有元素都是来自于 stack 中，所以 stack 删除元素后，minStack 也要考虑是否需要删除元素
        // 否则的话，minStack 有可能保存一个 stack 中不存在的元素

        // 首先，获取 minStack 的栈顶元素
        int top = minStack.peek();
        // 再判断 top 这个栈顶元素是否和 stack 移除的元素相等，如果相等，那么需要把 minStack 中的栈顶元素一并移除
        if (pop == top) {
            // 移除 minStack 的栈顶元素
            minStack.pop();
        }

    }

    public int top() {
        // 返回 stack 的栈顶元素
        return stack.peek();
    }

    public int getMin() {
        // 返回 minStack 的栈顶元素
        return minStack.peek();
    }
}

```

2、C++代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最小栈（ LeetCode 155 ）：https://leetcode-cn.com/problems/min-stack/
class MinStack {
public:
    // 首先定义好两个栈

    // 一个栈叫做 stk
    stack<int> stk;

    // 一个栈叫做 minstk，负责获取 stk 中的最小值，它等价于遍历 stk 中的所有元素，把升序的数字都删除掉，留下一个从栈底到栈顶降序的栈
    stack<int> minstk;

    MinStack() {

```



```

}

void push(int x) {
    // 新添加的元素添加到 stk 中
    stk.push(x);

    // 判断 minstk 是否为空，如果为空，直接同时把新添加的元素添加到 minstk 中

    // 如果 minstk 不为空
    if (!minstk.empty()) {
        // 获取 minstk 的栈顶元素
        int top = minstk.top();
        // 只有新添加的元素不大于 top 才允许添加到 minstk 中，目的是为了让 minstk 从栈底到栈
        顶是降序的
        if (x <= top) {
            // 此时，新添加的元素 x 小于 top，加入到 minstk 后依旧是从栈底到栈顶是降序的
            minstk.push(x);
        }
    } else {
        // 此时，minstk 中没有元素，所以直接把新添加的元素添加到 minstk 中
        minstk.push(x);
    }
}

void pop() {
    // 让 stk 执行正常的 pop 操作就行
    int pop = stk.top();

    stk.pop();

    // 由于 minstk 中的所有元素都是来自于 stk 中，所以 stk 删除元素后，minstk 也要考虑是否需要删除元素
    // 否则的话，minstk 有可能保存一个 stk 中不存在的元素

    // 首先，获取 minstk 的栈顶元素
    int top = minstk.top();
    // 再判断 top 这个栈顶元素是否和 stk 移除的元素相等，如果相等，那么需要把 minstk 中的栈顶元素一并移除
    if (pop == top) {
        // 移除 minstk 的栈顶元素
        minstk.pop();
    }

}

int top() {
    // 返回 stk 的栈顶元素

```

```

        return stk.top();
    }

    int getMin() {
        // 返回 minstk 的栈顶元素
        return minstk.top();
    }
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 最小栈 ( LeetCode 155 ) :https://leetcode-cn.com/problems/min-stack/
class MinStack:
    def __init__(self):
        # 首先定义好两个栈

        # 一个栈叫做 stack，负责栈的正常操作
        self.stack = []

        # 一个栈叫做 min_stack，负责获取 stack 中的最小值，它等价于遍历 stack 中的所有元素，把升序
        # 的数字都删除掉，留下一个从栈底到栈顶降序的栈
        self.min_stack = []

    def push(self, x: int) -> None:

        # 新添加的元素添加到 stack 中
        self.stack.append(x)

        # 判断 min_stack 是否为空，如果为空，直接同时把新添加的元素添加到 minStack 中
        # 如果 min_stack 不为空
        if self.min_stack:
            # 获取 min_stack 的栈顶元素
            top = self.min_stack[-1]

            # 只有新添加的元素不大于 top 才允许添加到 minStack 中，目的是为了让 minStack 从栈底到
            # 栈顶是降序的
            if x <= top :
                self.min_stack.append(x)

        # min_stack 中没有元素，所以直接把新添加的元素添加到 min_stack 中
        else:
            self.min_stack.append(x)

    def pop(self) -> None:

```

```

# 让 stack 执行正常的 pop 操作就行
pop = self.stack[-1]

self.stack.pop()

# 由于 minStack 中的所有元素都是来自于 stack 中，所以 stack 删除元素后，minStack 也要考虑是否需要删除元素
# 否则的话，minStack 有可能保存一个 stack 中不存在的元素

# 首先，获取 minStack 的栈顶元素
top = self.min_stack[-1]

# 再判断 top 这个栈顶元素是否和 stack 移除的元素相等，如果相等，那么需要把 minStack 中的栈顶元素一并移除
if pop == top:
    # 移除 min_stack 的栈顶元素
    self.min_stack.pop()

def top(self) -> int:
    # 返回 stack 的栈顶元素
    return self.stack[-1]
def getMin(self) -> int:
    # 返回 min_stack 的栈顶元素
    return self.min_stack[-1]

```

四、复杂度分析

时间复杂度：对于题目中的所有操作，时间复杂度均为 $O(1)$ 。因为栈的插入、删除与读取操作都是 $O(1)$ ，我们定义每个操作最多调用栈操作两次。

空间复杂度： $O(n)$ ，其中 n 为总操作数。最坏情况下，我们会连续插入 n 个元素，此时两个栈占用的空间为 $O(n)$ 。

LeetCode 1614、括号的嵌套深度

一、题目描述

如果字符串满足以下条件之一，则可以称之为 **有效括号字符串*** (valid parentheses string, 可以简称为 **VPS**)：

- 字符串是一个空字符串 `""`，或者是一个不为 `"(" 或 ")"` 的单字符。
- 字符串可以写为 `AB` (`A` 与 `B` 字符串连接)，其中 `A` 和 `B` 都是 **有效括号字符串**。
- 字符串可以写为 `(A)`，其中 `A` 是一个 **有效括号字符串**。

类似地，可以定义任何有效括号字符串 `S` 的 **嵌套深度** `depth(S)`：

- `depth("") = 0`
- `depth(C) = 0`，其中 `C` 是单个字符的字符串，且该字符不是 `"(" 或者 ")"`
- `depth(A + B) = max(depth(A), depth(B))`，其中 `A` 和 `B` 都是 **有效括号字符串**

- `depth("(" + A + ")") = 1 + depth(A)`，其中 `A` 是一个有效括号字符串

例如：`"`、`"()"`、`"()()()"` 都是有效括号字符串（嵌套深度分别为 0、1、2），而 `")("`、`"(("` 都不是有效括号字符串。

给你一个有效括号字符串 `s`，返回该字符串的 `s` 嵌套深度。

示例 1:

输入: `s = "(1+(2*3)+((8)/4))+1"` 输出: 3 解释: 数字 8 在嵌套的 3 层括号中。

示例 2:

输入: `s = "(1)+((2))+(((3)))"` 输出: 3

提示:

- `1 <= s.length <= 100`
- `s` 由数字 `0-9` 和字符 `'+'`、`'-'`、`'*'`、`'/'`、`'('`、`')'` 组成
- 题目数据保证括号表达式 `s` 是有效的括号表达式

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public int maxDepth(String s) {
        int ans = 0, size = 0;
        for (int i = 0; i < s.length(); ++i) {
            char ch = s.charAt(i);
            if (ch == '(') {
                ++size;
                ans = Math.max(ans, size);
            } else if (ch == ')') {
                --size;
            }
        }
        return ans;
    }
}
```

2、C++ 代码

```
class Solution {
public:
    int maxDepth(string s) {
        int ans = 0, size = 0;
        for (char ch : s) {
            if (ch == '(') {
                ++size;
            }
        }
    }
}
```

```

        ans = max(ans, size);
    } else if (ch == ')') {
        --size;
    }
}
return ans;
}
};

```

3、Python 代码

```

class solution:
    def maxDepth(self, s: str) -> int:

        # size 表示栈的大小
        # ans 表示 size 的最大值，也就是最终的结果值
        ans, size = 0, 0

        # 遍历字符串 s
        for ch in s:
            # 如果遇到了一个左括号，将其入栈
            if ch == '(':
                # 栈中元素的个数加 1
                size += 1
                # 更新深度的值
                ans = max(ans, size)
            # 如果遇到了一个右括号，弹出栈顶的左括号
            elif ch == ')':
                # 栈中元素的个数减 1
                size -= 1
        # 返回最大值
        return ans

```

LeetCode 20、有效的括号

一、题目描述

给定一个只包括 '(', ')', '{', '}', '[', ']' 的字符串 s，判断字符串是否有效。

有效字符串需满足：

- 1、左括号必须用相同类型的右括号闭合。
- 2、左括号必须以正确的顺序闭合。

二、题目解析

有效的括号满足以下几个条件：

- 1、字符串的长度一定是偶数。
- 2、括号的匹配遵循右括号和最近的一个左括号进行匹配，它们匹配成功才有可能有效的括号

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的括号 ( LeetCode 20 ) :https://leetcode-cn.com/problems/valid-parentheses
class Solution {

    public boolean isValid(String s) {

        // 当字符串长度为奇数的时候，属于无效情况，直接返回 false
        if (s.length() % 2 == 1) {
            // 无效情况，返回 false
            return false;
        }

        //构建一个栈，用来存储括号
        Stack<Character> stack = new Stack<Character>();

        // 把字符串转换为数组的形式，方便获取字符串中的每个字符
        char charArray[] = s.toCharArray();

        // 遍历字符串数组中的所有元素
        for( int i = 0 ; i < charArray.length ; i++){

            // 获取此时的字符
            char c = charArray[i];

            // 如果字符为左括号 ( ，那么就在栈中添加对左括号 (
            if(c == '('){

                // 添加对左括号 (
                stack.push('(');

                // 如果字符为左括号 [ ，那么就在栈中添加对左括号 [
            }else if(c == '['){

                // 添加对应的右括号 ]
                stack.push('[');
```

```

        // 如果字符为左括号 { , 那么就在栈中添加对左括号 {
    }else if( c == '{'){

        // 添加对应的右括号 }
        stack.push('}');

        // 否则的话, 说明此时 c 是 ) ] } 这三种符号中的一种
    }else {

        // 如果栈已经为空, 而现在遍历的字符 c 是 ) ] } 这三种符号中的一种
        // 找不到可以匹配的括号, 返回 false
        // 比如这种情况 }{, 直接从右括号开始, 此时栈为空
        if(stack.isEmpty()) return false;

        // 如果栈不为空, 获取栈顶元素
        char top = stack.peek();

        // 将栈顶元素和此时的元素 c 进行比较, 如果相同, 则将栈顶元素移除
        if( top == '(' && c == ')' || top == '[' && c == ']' || top == '{' && c
== '}' ){

            // 移除栈顶元素
            stack.pop();
        }else{
            // 如果不相同, 说明不匹配, 返回 false
            return false;
        }

    }

}

// 遍历完整整个字符数组, 判断栈是否为空
// 如果栈为空, 说明字符数组中的所有括号都是闭合的
// 如果栈不为空, 说明有未闭合的括号
return stack.isEmpty();
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的括号 ( LeetCode 20 ):https://leetcode-cn.com/problems/valid-parentheses
class Solution {
public:
    bool isValid(string s) {
        // 当字符串长度为奇数的时候, 属于无效情况, 直接返回 false
    }
}

```

```

if (s.size() % 2 == 1) {
    // 无效情况, 返回 false
    return false;
}

//构建一个栈, 用来存储括号
stack<char> stk ;

// 遍历字符串数组中的所有元素
for( int i = 0 ; i < s.size() ; i++){

    // 获取此时的字符
    char c = s[i];

    // 如果字符为左括号 ( , 那么就在栈中添加对左括号 (
    if(c == '('){

        // 添加对左括号 (
        stk.push('(');

        // 如果字符为左括号 [ , 那么就在栈中添加对左括号 [
    }else if(c == '['){

        // 添加对应的右括号 ]
        stk.push(']');

        // 如果字符为左括号 { , 那么就在栈中添加对左括号 {
    }else if( c == '{'){

        // 添加对应的右括号 }
        stk.push('}');

        // 否则的话, 说明此时 c 是 ) ] } 这三种符号中的一种
    }else {

        // 如果栈已经为空, 而现在遍历的字符 c 是 ) ] } 这三种符号中的一种
        // 找不到可以匹配的括号, 返回 false
        // 比如这种情况 }{, 直接从右括号开始, 此时栈为空
        if(stk.empty()) return false;

        // 如果栈不为空, 获取栈顶元素
        char top = stk.top();

        // 将栈顶元素和此时的元素 c 进行比较, 如果相同, 则将栈顶元素移除
        if( top == '(' && c == ')' || top == '[' && c == ']' || top == '{' && c
== '}' ){

            // 移除栈顶元素
            stk.pop();
        }else{

```



```

        // 如果不相同，说明不匹配，返回 false
        return false;
    }

}

}

// 遍历完整整个字符数组，判断栈是否为空
// 如果栈为空，说明字符数组中的所有括号都是闭合的
// 如果栈不为空，说明有未闭合的括号
return stk.empty();

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### // 有效的括号 ( LeetCode 20 ) :https://leetcode-cn.com/problems/valid-parentheses
class Solution:
    def isValid(self, s: str) -> bool:
        # 当字符串长度为奇数的时候，属于无效情况，直接返回 False
        if len(s) % 2 == 1:
            # 无效情况，返回 False
            return False

        # 构建一个栈，用来存储括号
        stack = list()

        # 遍历字符串数组中的所有元素
        for c in s :

            # 如果字符为左括号 ( ，那么就在栈中添加对左括号 (
            if c == '(' :

                # 添加对左括号 (
                stack.append('(')

            # 如果字符为左括号 [ ，那么就在栈中添加对左括号 [
            elif c == '[' :

                # 添加对应的右括号 ]
                stack.append('[')

```

```

        # 如果字符为左括号 { , 那么就在栈中添加对左括号 {
    elif c == '{' :

        # 添加对应的右括号 }
        stack.append('}')

        # 否则的话, 说明此时 c 是 ) ] } 这三种符号中的一种
    else :

        # 如果栈已经为空, 而现在遍历的字符 c 是 ) ] } 这三种符号中的一种
        # 找不到可以匹配的括号, 返回 False
        # 比如这种情况 }{, 直接从右括号开始, 此时栈为空
        if not stack :
            return False

        # 如果栈不为空, 获取栈顶元素
        top = stack[-1]

        # 将栈顶元素和此时的元素 c 进行比较, 如果相同, 则将栈顶元素移除
        if (top == '(' and c == ')') or (top == '[' and c == ']') or (top == '{' and c == '}') :
            # 移除栈顶元素
            stack.pop()
        else :
            # 如果不相同, 说明不匹配, 返回 False
            return False

    # 遍历完整个字符数组, 判断栈是否为空
    # 如果栈为空, 说明字符数组中的所有括号都是闭合的
    # 如果栈不为空, 说明有未闭合的括号
    return not stack

```

四、复杂度分析

时间复杂度： $O(n)$ ，其中 n 是字符串 s 的长度。

空间复杂度： $O(n+|\Sigma|)$ ，其中 Σ 表示字符集，本题中字符串只包含 6 种括号， $|\Sigma|=6$ 。栈中的字符数量为 $O(n)$ ，而哈希表使用的空间为 $O(|\Sigma|)$ ，相加即可得到总空间复杂度。

LeetCode 71、简化路径

一、题目描述

给你一个字符串 `path`，表示指向某一文件或目录的 Unix 风格 **绝对路径**（以 `'/'` 开头），请你将其转化为更加简洁的规范路径。

在 Unix 风格的文件系统中，一个点（`.`）表示当前目录本身；此外，两个点（`..`）表示将目录切换到上一级（指向父目录）；两者都可以是复杂相对路径的组成部分。任意多个连续的斜杠（即，`'//'`）都被视为单个斜杠 `'/'`。对于此问题，任何其他格式的点（例如，`'...'`）均被视为文件/目录名称。

请注意，返回的 **规范路径** 必须遵循下述格式：

- 始终以斜杠 `'/'` 开头。
- 两个目录名之间必须只有一个斜杠 `'/'`。
- 最后一个目录名（如果存在）**不能** 以 `'/'` 结尾。
- 此外，路径仅包含从根目录到目标文件或目录的路径上的目录（即，不含 `'.'` 或 `'..'`）。

返回简化后得到的 **规范路径**。

示例 1：

输入：`path = "/home/"` 输出：`"/home"` 解释：注意，最后一个目录名后面没有斜杠。

示例 2：

输入：`path = "/../"` 输出：`"/"` 解释：从根目录向上一级是不可行的，因为根目录是你到达的最高级。

示例 3：

输入：`path = "/home//foo/"` 输出：`"/home/foo"` 解释：在规范路径中，多个连续斜杠需要用一个斜杠替换。

示例 4：

输入：`path = "/a/./b/../../../../c/"` 输出：`"/c"`

提示：

- `1 <= path.length <= 3000`
- `path` 由英文字母，数字，`'.'`，`'/'` 或 `'_'` 组成。
- `path` 是一个有效的 Unix 风格绝对路径。

二、题目解析

我们首先将给定的字符串 `path` 根据 `/` 分割成一个由若干字符串组成的列表，记为 `names`。根据题目中规定的「规范路径的下述格式」，`names` 中包含的字符串只能为以下几种：

- 1、空字符串。例如当出现多个连续的 `/`，就会分割出空字符串；
- 2、一个点 `.`；
- 3、两个点 `..`；
- 4.只包含英文字母、数字或 `_` 的目录名。

对于「空字符串」以及「一个点」，我们实际上无需对它们进行处理，因为「空字符串」没有任何含义，而「一个点」表示当前目录本身，我们无需切换目录。

对于「两个点」或者「目录名」，我们则可以用一个栈来维护路径中的每一个目录名。当我们遇到「两个点」时，需要将目录切换到上一级，因此只要栈不为空，我们就弹出栈顶的目录。当我们遇到「目录名」时，就把它放入栈。

这样一来，我们只需要遍历names 中的每个字符串并进行上述操作即可。在所有的操作完成后，我们将从栈底到栈顶的字符串用 / 进行连接，再在最前面加上/ 表示根目录，就可以得到简化后的规范路径。

三、参考代码

```
class solution {
public:
    string simplifyPath(string path) {

        // 分割字符串为列表形式
        std::vector<std::string> names;
        std::stringstream ss(path);
        std::string name;

        while (getline(ss, name, '/')) {
            names.push_back(name);
        }

        // 利用栈来处理
        std::vector<std::string> stack;

        // 访问列表里面的元素
        for (const auto& name : names) {
            std::cout << name << std::endl;
            // 1、如果是 ..
            if (name == "..") {
                // 在栈不为空的情况下
                if (!stack.empty()) {
                    // 弹出栈顶元素
                    stack.pop_back();
                }
            }
            // 2、如果是有效值
            // 字母
            else if (!name.empty() && name != ".") {
                stack.push_back(name);
            }
        }

        std::stringstream result;
        result << "/";
        for (size_t i = 0; i < stack.size(); ++i) {
            result << stack[i];
            if (i != stack.size() - 1) {
                result << "/";
            }
        }

        return result.str();
    }
};
```

```
}  
};
```

```
class Solution:  
    def simplifyPath(self, path: str) -> str:  
  
        # 分割字符串为列表形式  
        names = path.split("/")  
  
        # 利用栈来处理  
        stack = list()  
  
        # 访问列表里面的元素  
        for name in names:  
            print(name)  
            # 1、如果是 ..  
            if name == "..":  
                # 在栈不为空的情况下  
                if stack:  
                    # 弹出栈顶元素  
                    stack.pop()  
            # 2、如果是有效值  
            # 字母  
            elif name and name != ".":  
                stack.append(name)  
        # stack = ['Python', 'world', 'Hello']  
        # new_string = '/ '.join(stack)  
        # print(new_string)  
        # 输出结果 Python/ world/ Hello  
        # 我们使用/ 作为分隔符，将栈stack中的元素连接成一个新的字符串。  
        # 每个元素之间使用/ 进行分隔，因此输出结果中每个元素都以/ 结尾（除了最后一个元素）。  
        return "/" + "/".join(stack)
```

第六天

LeetCode 224、基本计算器

一、题目描述

给你一个字符串表达式 `s`，请你实现一个基本计算器来计算并返回它的值。

示例 1:

输入: `s = "1 + 1"`

输出: 2

示例 2:

输入: `s = " 2-1 + 2 "`

输出: 3

示例 3:

输入: `s = "(1+(4+5+2)-3)+(6+8)"`

输出: 23

提示:

- `1 <= s.length <= 3 * 105`
- `s` 由数字、`'+'`、`'-'`、`'('`、`')'`、和 `' '` 组成
- `s` 表示一个有效的表达式

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 基本计算器 ( LeetCode 224 ) :https://leetcode-cn.com/problems/basic-calculator
class Solution {
    public int calculate(String s) {

        // 使用栈来储存字符串表达式中的数字
        Stack<Integer> stack = new Stack<Integer>();

        // 为了方便计算，所有的操作都视为加法操作
        // 那么原先的减法操作就相当于加一个负数
        // 默认都是正数
        int sign = 1;

        // 保存计算的结果
        int res = 0;

        // 获取字符串的长度，然后获取里面的每个字符
        int length = s.length();

        // 获取字符串里面的每个字符
```

```

for (int i = 0; i < length; i++) {

    // 获取此时的字符
    char ch = s.charAt(i);

    // 如果当前字符是数字的话
    if (Character.isDigit(ch)) {

        // 那么可以通过 - '0' 这个操作把字符转换为整数
        // 相当于转换成了 ascii 码进行的数字运算
        int value = ch - '0';

        // 去查看当前字符的后一位是否存在
        // 如果操作并且后一位依旧是数字，那么就需要把后面的数字累加起来

        while (i + 1 < length && Character.isDigit(s.charAt(i + 1))){

            // i 向后移动，直到遇到非数字为止
            i++;

            // i 每向后移动一位，当前的值就需要乘以 10
            // 比如 s 为 "123+456"
            // 此时 i = 0, 那么 value 为 1
            // i = 1, 那么 value 为 1 * 10 + 2 = 12
            // i = 2, 那么 value 为 12 * 10 + 3 = 123
            value = value * 10 + s.charAt(i) - '0';

        }

        // 那么把获取到的数累加到结果 res 上
        res = res + sign * value;

        // 如果是 '+'
    } else if (ch == '+') {

        // 那么说明直接加一个正数
        sign = 1;

        // 如果是 '-'
    } else if (ch == '-') {

        // 那么说明加一个负数
        sign = -1;

        // 如果是 '('
    } else if (ch == '(') {
        // 根据数学计算规则，需要先计算括号里面的数字
        // 而什么时候计算呢？
        // 遇到 ) 为止
    }
}

```

```

// 所以，在遇到 ) 之前需要把之前计算好的结果存储起来再计算
// ( ) 直接的计算规则和一开始是一样的

// 1、先把 ( 之前的结果存放到栈中
stack.push(res);

// 2、重新初始化 res 为 0
res = 0;
// 3、把 ( 左边的操作符号存放到栈中
// 比如如果是 5 - ( 2 + 3 ) ，那么就是把 -1 存放进去
// 比如如果是 5 + ( 2 + 3 ) ，那么就是把 1 存放进去
stack.push(sign);

// 4、为了方便计算，所有的操作都视为加法操作
// 那么原先的减法操作就相当于加一个负数
// 默认都是正数
sign = 1;

// 如果是 ')'
} else if (ch == ')') {

    // 那么就需要把栈中存放的元素取出来了
    // 在 ch == '(' 这个判断语句中，每次都会往栈中存放两个元素
    // 1、先存放左括号外面的结果
    // 2、再存放左括号外面的符号

    // 1、先获取栈顶元素，即左括号外面的符号，查看是 + 还是 -
    int formerSign = stack.peek();

    // 把栈顶元素弹出
    stack.pop();

    // 2、再获取栈顶元素，即左括号结果
    int formerRes = stack.peek();

    // 把栈顶元素弹出
    stack.pop();

    // 那结果就是括号外面的结果 + 括号里面的结果，至于是加正数还是负数，取决于左括号外面
    // 的符号
    res = formerRes + formerSign * res ;

}
}

// 返回计算好的结果
return res;
}
}

```


2、C++ 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 基本计算器 ( LeetCode 224 ) :https://leetcode-cn.com/problems/basic-calculator
class Solution {
public:
    int calculate(string s) {
        // 使用栈来储存字符串表达式中的数字
        stack<int> st;

        // 为了方便计算，所有的操作都视为加法操作
        // 那么原先的减法操作就相当于加一个负数
        // 默认都是正数
        int sign = 1;

        // 保存计算的结果
        int res = 0;

        // 获取字符串的长度，然后获取里面的每个字符
        int length = s.length();

        // 获取字符串里面的每个字符
        for(int i = 0; i < length; i++){

            // 获取此时的字符
            char ch = s[i];

            // 如果当前字符是数字的话
            if( ch >= '0' && ch <= '9' ) {
                // 那么可以通过 - '0' 这个操作把字符转换为整数
                // 相当于转换成了 ascii 码进行的数字运算
                long value = ch - '0';

                // 去查看当前字符的后一位是否存在
                // 如果操作并且后一位依旧是数字，那么就需要把后面的数字累加上来
                while (i + 1 < length && s[i+1] >= '0' && s[i+1] <= '9'){
                    // i 向后移动，直到遇到非数字为止
                    i++;

                    // i 每向后移动一位，当前的值就需要乘以 10
                    // 比如 s 为 "123+456"
                    // 此时 i = 0, 那么 value 为 1
                    // i = 1, 那么 value 为 1 * 10 + 2 = 12
                    // i = 2, 那么 value 为 12 * 10 + 3 = 123
                    value = value*10 + s[i] - '0';
                }
            }
        }
    }
};
```

```

    }
    // 那么把获取到的数累加到结果 res 上
    res = res + sign * value;

    // 如果是 '+'
}else if(ch == '+'){
    // 那么说明直接加一个正数
    sign = 1;
    // 如果是 '-'
}else if(ch == '-'){
    // 那么说明加一个负数
    sign = -1;

    // 如果是 '('
}else if(ch == '('){
    // 根据数学计算规则，需要先计算括号里面的数字
    // 而什么时候计算呢？
    // 遇到 ) 为止
    // 所以，在遇到 ) 之前需要把之前计算好的结果存储起来再计算
    // ( ) 直接的计算规则和一开始是一样的

    // 1、先把 ( 之前的结果存放到栈中
    st.push(res);

    // 2、重新初始化 res 为 0
    res = 0;

    // 3、把 ( 左边的操作符号存放到栈中
    // 比如如果是 5 - ( 2 + 3 ) ，那么就是把 -1 存放进去
    // 比如如果是 5 + ( 2 + 3 ) ，那么就是把 1 存放进去
    st.push(sign);

    // 4、为了方便计算，所有的操作都视为加法操作
    // 那么原先的减法操作就相当于加一个负数
    // 默认都是正数
    sign = 1;

    // 如果是 ')'
}else if(ch == ')'){
    // 那么就需要把栈中存放的元素取出来了
    // 在 ch == ' (' 这个判断语句中，每次都会往栈中存放两个元素
    // 1、先存放左括号外面的结果
    // 2、再存放左括号外面的符号

    // 1、先获取栈顶元素，即左括号外面的符号，查看是 + 还是 -
    int formerSign = st.top();

    // 把栈顶元素弹出
    st.pop();

```

```

        // 2、再获取栈顶元素，即左括号结果
        int formerRes = st.top();

        // 把栈顶元素弹出
        st.pop();

        // 那结果就是括号外面的结果 + 括号里面的结果，至于加正数还是负数，取决于左括号外面
        的符号
        res = formerRes + formerSign * res;
    }
}

// 返回计算好的结果
return res;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 基本计算器 ( LeetCode 224 ) :https://leetcode-cn.com/problems/basic-calculator
class Solution:
    def calculate(self, s: str) -> int:

        # 使用栈来储存字符串表达式中的数字
        stack = list()

        # 为了方便计算，所有的操作都视为加法操作
        # 那么原先的减法操作就相当于加一个负数
        # 默认都是正数
        sign = 1

        # 保存计算的结果
        res = 0

        # 获取字符串的长度，然后获取里面的每个字符
        length = len(s)

        # 从 0 开始访问字符串中的每个字符
        i = 0

        # 获取字符串里面的每个字符
        while i < length:
            # 获取此时的字符
            ch = s[i]

```

```

if ch == ' ':
    i += 1
# 如果当前字符是数字的话
elif ch.isdigit():
    # 那么把获取到的数累加到结果 res 上
    value = ord(s[i]) - ord('0')

    # 去查看当前字符的后一位是否存在
    # 如果操作并且后一位依旧是数字，那么就需要把后面的数字累加上来
    while i + 1 < length and s[i + 1].isdigit():
        i += 1
        value = value * 10 + ord(s[i]) - ord('0')

    # 那么把获取到的数累加到结果 res 上
    res += value * sign

    i += 1

# 如果是 '+'
elif ch == '+':
    # 那么说明直接加一个正数
    sign = 1

    i += 1

# 如果是 '-'
elif ch == '-':
    # 那么说明加一个负数
    sign = -1

    i += 1

# 如果是 '('
elif ch == '(':
    # 根据数学计算规则，需要先计算括号里面的数字
    # 而什么时候计算呢？
    # 遇到 ) 为止
    # 所以，在遇到 ) 之前需要把之前计算好的结果存储起来再计算
    # ( ) 直接的计算规则和一开始是一样的

    # 1、先把 ( 之前的结果存放到栈中
    stack.append(res)

    # 2、重新初始化 res 为 0
    res = 0

    # 3、把 ( 左边的操作符号存放到栈中
    # 比如如果是 5 - ( 2 + 3 ) ，那么就是把 -1 存放进去

```

```

# 比如如果是 5 + ( 2 + 3 ) ，那么就是把 1 存放进去
stack.append(sign)

# 4、为了方便计算，所有的操作都视为加法操作
# 那么原先的减法操作就相当于加一个负数
# 默认都是正数
sign = 1

i += 1

# 如果是 ')'
elif ch == ')' :

    # 那么就需要把栈中存放的元素取出来了
    # 在 ch == '(' 这个判断语句中，每次都会往栈中存放两个元素
    # 1、先存放左括号外面的结果
    # 2、再存放左括号外面的符号

    # 1、先获取栈顶元素，即左括号外面的符号，查看是 + 还是 -
    # 把栈顶元素弹出
    formerSign = stack.pop()

    # 2、再获取栈顶元素，即左括号结果
    # 把栈顶元素弹出
    formerRes = stack.pop()

    # 那结果就是括号外面的结果 + 括号里面的结果，至于加正数还是负数，取决于左括号外面
    的符号

    res = formerRes + formerSign * res

    i += 1

# 返回计算好的结果
return res

```

四、复杂度分析

时间复杂度： $O(n)$ ，其中 n 为字符串 s 的长度。需要遍历字符串 s 一次，计算表达式的值。

空间复杂度： $O(n)$ ，其中 n 为字符串 s 的长度。空间复杂度主要取决于栈的空间，栈中的元素数量不超过 n 。

五、直接调API解法

如果上述内容实在学得吃力，但机试中又不幸遇到。可以直接使用内置函数 `eval()` 解题。内置函数 `eval()` 能够直接计算一个字符串表达式的值，能够识别加减乘除以及括号。

```

s = input()
print(eval(s))

```

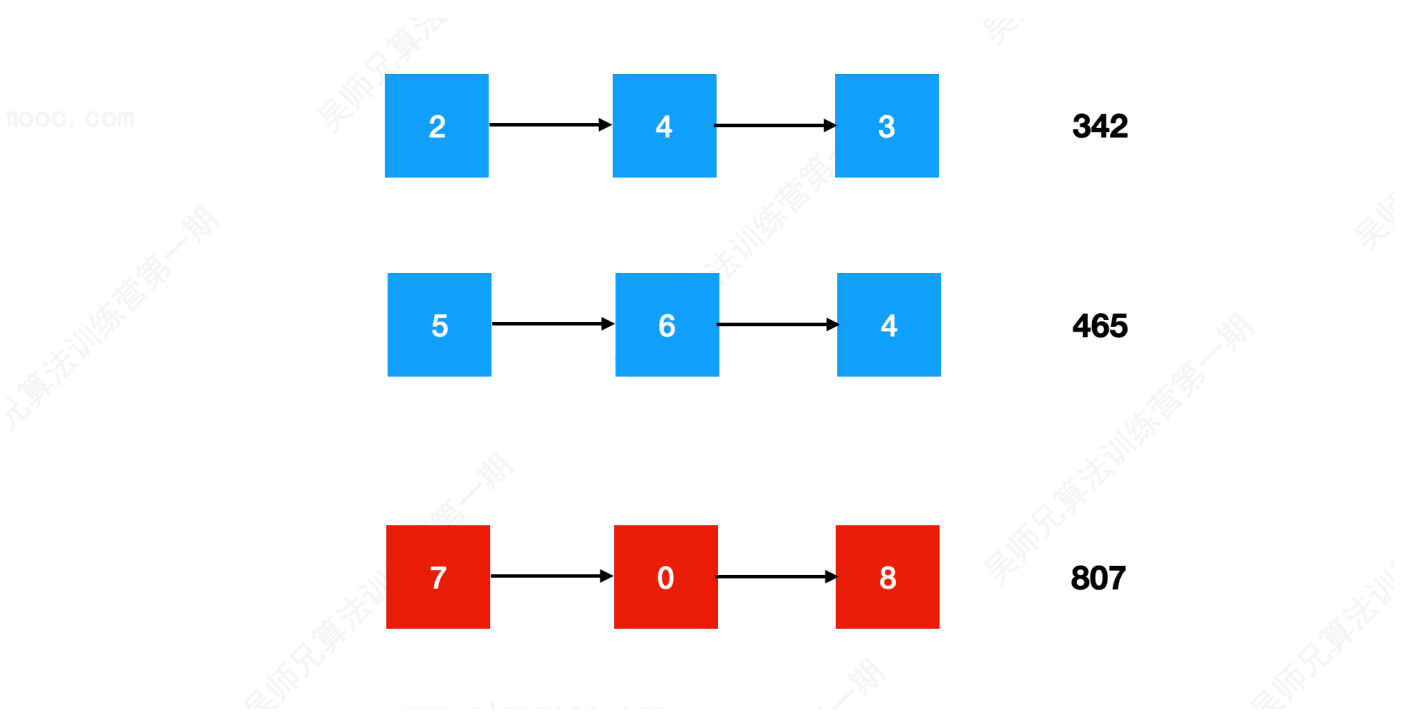
LeetCode 2、两数相加

一、题目描述

给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储一位 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。



二、题目解析

三、参考代码

Java

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

        // 构建一个链表用来存放 l1 和 l2 两个链表相加的结果
        // 其中 dummy 这个节点为虚拟头结点
        ListNode dummy = new ListNode(-1);

        // 设置一个进位，初始化为 0
        // 两个个位数相加，进位只能是 1 或者 0
        // 比如 7 + 8 = 15，进位是 1
        // 比如 2 + 3 = 6，没有进位，或者说进位是 0
        int carryBit = 0;

        // l1 和 l2 有可能为空，所以先默认结果链表从虚拟头结点位置开始
        ListNode cur = dummy;
```

```

// 同时遍历 l1 和 l2
// 只要此时 l1 和 l2 两个链表中的任意链表中节点有值，就一直加下去
// 直到两个链表中的所有节点都遍历完毕为止
while(l1 != null || l2 != null) {
    // 获取 l1 链表中节点的值
    int x;

    // 观察此时 l1 中的节点是否有值
    // 如果节点不存在，那么就用 0 来占位
    if( l1 == null){
        // 用 0 来占位
        x = 0;
    }else{
        // 否则，将 l1 的节点值赋值给 x
        x = l1.val;
    }

    // 获取 l2 链表中节点的值
    int y;

    // 观察此时 l2 中的节点是否有值
    // 如果节点不存在，那么就用 0 来占位
    if( l2 == null){
        // 用 0 来占位
        y = 0;
    }else{
        // 否则，将 l2 的节点值赋值给 y
        y = l2.val;
    }

    // 每一位计算的同时需要考虑上一位的进位情况
    int sum = x + y + carryBit;

    // 获取当前计算结果的十位数
    // 比如 7 + 8 = 15
    // 那么 sum / 10 = 1，进位为 1
    carryBit = sum / 10;

    // 获取当前计算结果的个位数
    // 比如 7 + 8 = 15
    // 那么 sum % 10 = 5
    int digit = sum % 10;

    // 构建一个节点用来存放这个个位数
    ListNode digitNode = new ListNode(digit);

    // 把这个节点加入到结果链表中
    cur.next = digitNode;
}

```

```

        // 移动 cur 的位置，观察后面应该存放什么节点
        cur = cur.next;

        // 11 链表中还有节点未遍历完毕就继续遍历下去
        if(l1 != null) l1 = l1.next;

        // 12 链表中还有节点未遍历完毕就继续遍历下去
        if(l2 != null) l2 = l2.next;
    }

    // 两个链表的尾节点相加之后，有可能产生进位的情况
    // 所以，需要构建一个新的节点用来存放这个进位的结果
    if(carryBit == 1) {
        // 构建一个节点用来存放这个进位
        ListNode carryBitNode = new ListNode(carryBit);

        // 把这个节点加入到结果链表中
        cur.next = carryBitNode;
    }

    // 最后返回结果链表的头节点就行，即虚拟头结点的下一个节点
    return dummy.next;
}
}

```

C++

```

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {

        // 构建一个链表用来存放 l1 和 l2 两个链表相加的结果
        // 其中 dummy 这个节点为虚拟头结点
        ListNode *dummy = new ListNode(-1);

        // 设置一个进位，初始化为 0
        // 两个个位数相加，进位只能是 1 或者 0
        // 比如 7 + 8 = 15，进位是 1
        // 比如 2 + 3 = 6，没有进位，或者说进位是 0
        int carryBit = 0;

        // l1 和 l2 有可能为空，所以先默认结果链表从虚拟头结点位置开始
        ListNode *cur = dummy;

        // 同时遍历 l1 和 l2
        // 只要此时 l1 和 l2 两个链表中的任意链表中节点有值，就一直加下去
        // 直到两个链表中的所有节点都遍历完毕为止
        while(l1 != NULL || l2 != NULL) {
            // 获取 l1 链表中的节点的值

```



```

int x;

// 观察此时 l1 中的节点是否有值
// 如果节点不存在，那么就用 0 来占位
if( l1 == NULL){
    // 用 0 来占位
    x = 0;
}else{
    // 否则，将 l1 的节点值赋值给 x
    x = l1->val;
}

// 获取 l2 链表中节点的值
int y;

// 观察此时 l2 中的节点是否有值
// 如果节点不存在，那么就用 0 来占位
if( l2 == NULL){
    // 用 0 来占位
    y = 0;
}else{
    // 否则，将 l2 的节点值赋值给 y
    y = l2->val;
}

// 每一位计算的同时需要考虑上一位的进位情况
int sum = x + y + carryBit;

// 获取当前计算结果的十位数
// 比如 7 + 8 = 15
// 那么 sum / 10 = 1, 进位为 1
carryBit = sum / 10;

// 获取当前计算结果的个位数
// 比如 7 + 8 = 15
// 那么 sum % 10 = 5
int digit = sum % 10;

// 构建一个节点用来存放这个个位数
ListNode *digitNode = new ListNode(digit);

// 把这个节点加入到结果链表中
cur->next = digitNode;

// 移动 cur 的位置，观察后面应该存放什么节点
cur = cur->next;

// l1 链表中还有节点未遍历完毕就继续遍历下去
if(l1 != NULL) l1 = l1->next;

```

```

        // 12 链表中还有节点未遍历完毕就继续遍历下去
        if(l2 != NULL) l2 = l2->next;
    }

    // 两个链表的尾节点相加之后，有可能产生进位的情况
    // 所以，需要构建一个新的节点用来存放这个进位的结果
    if(carryBit == 1) {
        // 构建一个节点用来存放这个进位
        ListNode *carryBitNode = new ListNode(carryBit);

        // 把这个节点加入到结果链表中
        cur->next = carryBitNode;
    }

    // 最后返回结果链表的头节点就行，即虚拟头结点的下一个节点
    return dummy->next;
}
};

```

Python

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        # 构建一个链表用来存放 l1 和 l2 两个链表相加的结果
        # 其中 dummy 这个节点为虚拟头结点
        dummy = ListNode()

        # 设置一个进位，初始化为 0
        # 两个个位数相加，进位只能是 1 或者 0
        # 比如 7 + 8 = 15，进位是 1
        # 比如 2 + 3 = 6，没有进位，或者说进位是 0
        carryBit = 0

        # l1 和 l2 有可能为空，所以先默认结果链表从虚拟头结点位置开始
        cur = dummy

        # 同时遍历 l1 和 l2
        # 只要此时 l1 和 l2 两个链表中的任意链表中节点有值，就一直加下去
        # 直到两个链表中的所有节点都遍历完毕为止
        while l1 or l2 :
            # 获取 l1 链表中节点的值
            # 观察此时 l1 中的节点是否有值
            # 如果节点不存在，那么就用 0 来占位
            # 否则，将 l1 的节点值赋值给 x
            x = l1.val if l1 else 0

            # 获取 l2 链表中节点的值

```

```
# 观察此时 l2 中的节点是否有值
# 如果节点不存在，那么就用 0 来占位
# 否则，将 l2 的节点值赋值给 y
y = l2.val if l2 else 0

# 每一位计算的同时需要考虑上一位的进位情况
sum = x + y + carryBit

# 获取当前计算结果的十位数
# 比如 7 + 8 = 15
# 那么 sum / 10 = 1, 进位为 1
carryBit = sum // 10

# 获取当前计算结果的个位数
# 比如 7 + 8 = 15
# 那么 sum % 10 = 5
digit = sum % 10

# 构建一个节点用来存放这个个位数
digitNode = ListNode(digit)

# 把这个节点加入到结果链表中
cur.next = digitNode

# 移动 cur 的位置，观察后面应该存放什么节点
cur = cur.next

# l1 链表中还有节点未遍历完毕就继续遍历下去
if l1:
    l1 = l1.next

# l2 链表中还有节点未遍历完毕就继续遍历下去
if l2:
    l2 = l2.next

# 两个链表的尾节点相加之后，有可能产生进位的情况
# 所以，需要构建一个新的节点用来存放这个进位的结果
if carryBit == 1:
    # 构建一个节点用来存放这个进位
    carryBitNode = ListNode(carryBit)

    # 把这个节点加入到结果链表中
    cur.next = carryBitNode

# 最后返回结果链表的头节点就行，即虚拟头结点的下一个节点
return dummy.next
```

四、复杂度分析

时间复杂度： $O(\max(m,n))$ ，其中 m 和 n 分别为两个链表的长度。我们要遍历两个链表的全部位置，而处理每个位置只需要 $O(1)$ 的时间。

空间复杂度： $O(1)$ 。注意返回值不计入空间复杂度。

LeetCode 32、最长有效括号

一、题目描述

给你一个只包含 '(' 和 ')' 的字符串，找出最长有效（格式正确且连续）括号子串的长度。

示例 1：

```
输入：s = "()"
输出：2
解释：最长有效括号子串是 "()"
```

示例 2：

```
输入：s = ")()())"
输出：4
解释：最长有效括号子串是 "()()"
```

示例 3：

```
输入：s = ""
输出：0
```

提示：

- $0 \leq s.length \leq 3 \times 10^4$
- $s[i]$ 为 '(' 或 ')'

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/longest-valid-parentheses/
class Solution {
    public int longestValidParentheses(String s) {
```

```

// 利用栈来保留访问路径
Stack<Integer> stack = new Stack<Integer>();

// maxLength 记录最长有效（格式正确且连续）括号子串的长度
int maxLength = 0;

// 设置变量 start 表示合法括号序列的起始索引位置，指向一个左括号，初始值为 0
int start = 0 ;

// 1、访问每个字符，假设当前索引为 i，结合访问的元素和栈是否为空，有以下几种情况：
for(int i = 0 ; i < s.length() ; i++){

    // 2、访问到左括号时，不管栈是否为空，都把它存储起来
    if( s.charAt(i) == '('){

        stack.add(i);

    // 3、访问到右括号时
    }else{

        // 4、如果当前栈为空，说明这个右括号在前面没有和它匹配的左括号，那么 start 需要发生
        改变，

        // 下一个访问的字符 i + 1 有可能是左括号，所以 start = i + 1
        if(stack.isEmpty()) {

            start = i + 1;

        // 5、如果当前栈不为空
        }else{

            // 6、弹出栈顶元素后
            stack.pop();

            // 7、如果栈为空，
            // 说明以当前右括号为右端点的合法括号序列的左端点为start，则更新长度 i -
            start + 1

            if(stack.isEmpty()){

                maxLength = Math.max(maxLength,i - start + 1);

            // 8、如果栈不为空，
            // 找到了一组合法的括号序列，左括号是刚刚弹出的元素，索引位置可以用栈顶元素索引
            加 1 获取到，

            // 那么这组合法括号序列的长度为 i - ( stack.peek() + 1 ) + 1
            }else{

                maxLength = Math.max(maxLength,i - (stack.peek() + 1) + 1);

            }

        }
    }
}

```

```

    }
    }
}
return maxLength;
}
}

```

**2、C++代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/longest-valid-parentheses/
class Solution {
public:
    int longestValidParentheses(string s) {
        // 利用栈来保留访问路径
        stack<int> stk;

        // maxLength 记录最长有效（格式正确且连续）括号子串的长度
        int maxLength = 0;

        // 设置变量 start 表示合法括号序列的起始索引位置，指向一个左括号，初始值为 0
        int start = 0 ;

        // 1、访问每个字符，假设当前索引为 i，结合访问的元素和栈是否为空，有以下几种情况：
        for(int i = 0 ; i < s.length() ; i++){

            // 2、访问到左括号时，不管栈是否为空，都把它存储起来
            if( s[i] == '('){

                stk.push(i);

            // 3、访问到右括号时
            }else{

                // 4、如果当前栈为空，说明这个右括号在前面没有和它匹配的左括号，那么 start 需要发生
                改变,

                // 下一个访问的字符 i + 1 有可能是左括号，所以 start = i + 1
                if(stk.empty()) {

                    start = i + 1;

                // 5、如果当前栈不为空
                }else{

                    // 6、弹出栈顶元素后
                    stk.pop();
                }
            }
        }

        return maxLength;
    }
};

```

```

        // 7、如果栈为空，
        // 说明以当前右括号为右端点的合法括号序列的左端点为start，则更新长度 i -
start + 1

        if(stk.empty()){

            maxLength = max(maxLength,i - start + 1);

        }

        // 8、如果栈不为空，
        // 找到了一组合法的括号序列，左括号是刚刚弹出的元素，索引位置可以用栈顶元素索引
加 1 获取到，

        // 那么这组合法括号序列的长度为 i - ( stk.peek() + 1 ) + 1
    }else{

        maxLength = max(maxLength,i - (stk.top() + 1) + 1);

    }

}

}

return maxLength;
}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### https://leetcode-cn.com/problems/longest-valid-parentheses/
class Solution:
    def longestValidParentheses(self, s: str) -> int:

        # 利用栈来保留访问路径
        stack = []

        # maxLength 记录最长有效（格式正确且连续）括号子串的长度
        maxLength = 0

        # 设置变量 start 表示合法括号序列的起始索引位置，指向一个左括号，初始值为 0
        start = 0

        # 1、访问每个字符，假设当前索引为 i，结合访问的元素和栈是否为空，有以下几种情况：
        for i in range(len(s)):

            # 2、访问到左括号时，不管栈是否为空，都把它存储起来
            if s[i] == '(' :

```

```

        stack.append(i)

# 3、访问到右括号时
else:

    # 4、如果当前栈为空，说明这个右括号在前面没有和它匹配的左括号，那么 start 需要发生
    改变，

    # 下一个访问的字符 i + 1 有可能是左括号，所以 start = i + 1
    if not stack :

        start = i + 1

    # 5、如果当前栈不为空
    else:

        # 6、弹出栈顶元素后
        stack.pop()

        # 7、如果栈为空，
        # 说明以当前右括号为右端点的合法括号序列的左端点为start，则更新长度 i - start
        + 1

        if not stack :

            maxLength = max(maxLength,i - start + 1)

        # 8、如果栈不为空，
        # 找到了一组合法的括号序列，左括号是刚刚弹出的元素，索引位置可以用栈顶元素索引加
        1 获取到，

        # 那么这组合法括号序列的长度为 i - ( stack[-1] + 1 ) + 1
        else:

            maxLength = max(maxLength,i - (stack[-1] + 1) + 1)

    return maxLength

```

四、复杂度分析

- 时间复杂度： $O(n)$ ， n 是给定字符串的长度。我们只需要遍历字符串一次即可。
- 空间复杂度： $O(n)$ 。栈的大小在最坏情况下会达到 n ，因此空间复杂度为 $O(n)$ 。

LeetCode 394、字符串解码

一、题目描述

给定一个经过编码的字符串，返回它解码后的字符串。

编码规则为: `k[encoded_string]`，表示其中方括号内部的 `encoded_string` 正好重复 `k` 次。注意 `k` 保证为正整数。

你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。

此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 `k`，例如不会出现像 `3a` 或 `2[4]` 的输入。

示例 1:

```
输入: s = "3[a]2[bc]"
输出: "aaabcbc"
```

示例 2:

```
输入: s = "3[a2[c]]"
输出: "accaccacc"
```

示例 3:

```
输入: s = "2[abc]3[cd]ef"
输出: "abccabccdcdef"
```

示例 4:

```
输入: s = "abc3[cd]xyz"
输出: "abccdcddxyz"
```

提示:

- `1 <= s.length <= 30`
- `s` 由小写英文字母、数字和方括号 `'[]'` 组成
- `s` 保证是一个 **有效** 的输入。
- `s` 中所有整数的取值范围为 `[1, 300]`

二、题目解析

注意示例 2，可以发现字符串中存在括号内有嵌套括号的情况，这个时候，只有先把内层括号解码成功，才能再去解码外层括号。

也就意味着后面访问的括号比前面访问的括号还更早的进行处理，与栈的先进后出特点对应。

所以，本题使用栈来处理。

具体操作如下：

1、构建两个栈，一个是数字栈 `numStack`，在遍历编码字符串过程中记录出现的数字；一个是字符串栈 `strStack`，在遍历编码字符串过程中记录出现的字符串。

2、初始化两个变量，一个是 `digit`，用来记录访问到字符串之前出现的数字；一个是 `res`，在访问编码字符串的过程中，把得到的结果存放到 `res` 中。

3、接下来，开始从头到尾访问编码字符串，在访问过程中，字符会出现 4 种情况。

- 4、如果是数字，需要把字符转成整型数字，然后更新到 `digit` 上，代表后续的字符串需要重复 `digit` 次。
- 5、如果是字符，说明它就出现一次，可以直接存放到 `res` 中。
- 6、如果是 "["，这个时候出现了嵌套的内层编码字符串，而外层的解码需要等待内层解码的结果，那么之前已经扫描的字符需要存放起来，等到内层解码之后再重新使用。
- 7、如果是 "]"，此时，内层解码已经有结果，需要把它和前面的字符串进行拼接。
- 8、拼接的方式就是先通过 `numsStack` 的栈顶元素获取重复的次数，再通过 `strStack` 的栈顶元素获取前面的字符串。
- 9、最后返回 `res` 就行。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 字符串解码(LeetCode 394):https://leetcode.cn/problems/decode-string/
class Solution {
    public String decodeString(String s) {

        // 创建数字栈，在遍历编码字符串过程中记录出现的数字
        Deque<Integer> numStack = new LinkedList<>();

        // 创建字符栈，在遍历编码字符串过程中记录出现的字符串
        Deque<StringBuilder> strStack = new LinkedList<>();

        // 在访问编码字符串的过程中，用来记录访问到字符串之前出现的数字，一开始为 0
        int digit = 0;

        // 在访问编码字符串的过程中，把得到的结果存放到 res 中
        StringBuilder res = new StringBuilder();

        // 从头到尾遍历编码字符串
        for (int i = 0; i < s.length(); i++) {

            // 在遍历过程中，字符会出现 4 种情况

            // 先获取此时访问的字符
            char ch = s.charAt(i);

            // 1、如果是数字，需要把字符转成整型数字
            // 注意数字不一定是 1 位，有可能是多位
            // 比如 123a，在字母 a 的前面出现了 123，表示 a 重复出现 123 次
            // 那么一开始 ch 为 1，当访问到 ch 为 2 的时候，1 和 2 要组成数字 12
```

```

// 再出现 3 , 12 和 3 组成数字 123
if (Character.isDigit(ch)) {

    // 先将字符转成整型数字 ch-'0'
    // 补充知识: 将字符'0'-'9'转换为数字, 只需将字符变量减去'0'就行
    // 因为字符和数字在内存里都是以 ASCII 码形式存储的
    // 减去 '0' , 其实就是减去字符 '0' 的 ASCII 码, 而字符 '0' 的 ASCII 码是 30
    // 所以减去'0'也就是减去30, 然后就可以得到字符对应的数字了。
    int num = ch - '0';

    // 再将这个数字和前面存储的数字进行组合
    // 1 和 2 组成数字 12, 也就是 1 * 10 + 2 = 12
    // 12 和 3 组成数字 123, 也就是 12 * 10 + 3 = 123
    digit = digit * 10 + num ;

// 2、如果是字符
}else if((ch >= 'a' && ch <= 'z') ){

    // 说明它就出现一次, 可以直接存放到 res 中
    res.append(ch);

// 3、如果是"["
// 出现了嵌套的内层编码字符串, 而外层的解码需要等待内层解码的结果
// 那么之前已经扫描的字符需要存放起来, 等到内层解码之后再重新使用
}else if (ch == '[') {

    // 把数字存放到数字栈
    numStack.push(digit);

    // 把外层的解码字符串存放到字符串栈
    strStack.push(res);

    // 开始新一轮解码
    // 于是, digit 归零
    digit = 0;

    // res 重新初始化
    res = new StringBuilder();

// 4、如果是"]"
}else if (ch == ']') {

    // 此时, 内层解码已经有结果, 需要把它和前面的字符串进行拼接

    // 第一步, 先去查看内层解码的字符串需要被重复输出几次
    // 比如 e3[abc], 比如内层解码结果 abc 需要输出 3 次
    // 通过数字栈提取出次数
    int count = numStack.poll();

```

```

        // 第二步，通过字符串栈提取出之前的解码字符串
        StringBuilder outString = strStack.poll();

        // 第三步，不断的把内层解码的字符串拼接起来
        for (int j = 0; j < count; j++) {

            // 拼接到 outString 后面，拼接 count 次
            outString.append(res.toString());
        }

        // 再把此时得到的结果赋值给 res
        res = outString;
    }
}

// 返回解码成功的字符串
return res.toString();
}
}

```

2、C++ 代码

```

class Solution {
public:
    string decodeString(string s) {
        // 创建数字栈，在遍历编码字符串过程中记录出现的数字
        stack<int> numStack;

        // 创建字符栈，在遍历编码字符串过程中记录出现的字符串
        stack<string> strStack;

        // 在访问编码字符串的过程中，用来记录访问到字符串之前出现的数字，一开始为 0
        int digit = 0;

        // 在访问编码字符串的过程中，把得到的结果存放到 res 中
        string res;

        // 从头到尾遍历编码字符串
        for (int i = 0; i < s.size(); i++) {

            // 在遍历过程中，字符会出现 4 种情况

            // 先获取此时访问的字符
            char ch = s[i];
            // 1、如果是数字，需要把字符转成整型数字
            // 注意数字不一定是 1 位，有可能是多位
            // 比如 123a，在字母 a 的前面出现了 123，表示 a 重复出现 123 次
            // 那么一开始 ch 为 1，当访问到 ch 为 2 的时候，1 和 2 要组成数字 12
            // 再出现 3，12 和 3 组成数字 123

```

```

if (ch>='0'&&ch<='9') {

    // 先将字符转成整型数字 ch-'0'
    // 补充知识：将字符'0'-'9'转换为数字，只需将字符变量减去'0'就行
    // 因为字符和数字在内存里都是以 ASCII 码形式存储的
    // 减去 '0'，其实就是减去字符 '0' 的 ASCII 码，而字符 '0' 的 ASCII 码是 30
    // 所以减去'0'也就是减去30，然后就可以得到字符对应的数字了。
    int num = ch - '0';

    // 再将这个数字和前面存储的数字进行组合
    // 1 和 2 组成数字 12，也就是 1 * 10 + 2 = 12
    // 12 和 3 组成数字 123，也就是 12 * 10 + 3 = 123
    digit = digit * 10 + num ;

// 2、如果是字符
}else if((ch >= 'a' && ch <= 'z') ){

    // 说明它就出现一次，可以直接存放到 res 中
    res += ch;

// 3、如果是 "["
// 出现了嵌套的内层编码字符串，而外层的解码需要等待内层解码的结果
// 那么之前已经扫描的字符需要存放起来，等到内层解码之后再重新使用
}else if (ch == '[') {

    // 把数字存放到数字栈
    numStack.push(digit);

    // 把外层的解码字符串存放到字符串栈
    strStack.push(res);

    // 开始新一轮解码
    // 于是, digit 归零
    digit = 0;

    // res 重新初始化
    res = "";

// 4、如果是 "]"
}else if (ch == ']') {

    // 此时，内层解码已经有结果，需要把它和前面的字符串进行拼接

    // 第一步，先去查看内层解码的字符串需要被重复输出几次
    // 比如 e3[abc]，比如内层解码结果 abc 需要输出 3 次
    // 通过数字栈提取出次数
    int count = numStack.top();

    numStack.pop();

```

```

        // 第二步，通过字符串栈提取出之前的解码字符串
        string outString = strStack.top();

        strStack.pop();

        // 第三步，不断的把内层解码的字符串拼接起来
        for (int j = 0; j < count; j++) {

            // 拼接到 outString 后面，拼接 count 次
            outString += res;
        }

        // 再把此时得到的结果赋值给 res
        res = outString;
    }
}

// 返回解码成功的字符串
return res;
}
};

```

3、Python 代码

```

class Solution:
    def decodeString(self, s: str) -> str:
        # 创建数字栈，在遍历编码字符串过程中记录出现的数字
        numStack = []

        # 创建字符栈，在遍历编码字符串过程中记录出现的字符串
        strStack = []

        # 在访问编码字符串的过程中，用来记录访问到字符串之前出现的数字，一开始为 0
        digit = 0

        # 在访问编码字符串的过程中，把得到的结果存放到 res 中
        res = ""

        # 从头到尾遍历编码字符串
        for ch in s:

            # 在遍历过程中，字符会出现 4 种情况
            # 先获取此时访问的字符
            # 1、如果是数字，需要把字符转成整型数字
            # 注意数字不一定是 1 位，有可能是多位
            # 比如 123a，在字母 a 的前面出现了 123，表示 a 重复出现 123 次
            # 那么一开始 ch 为 1，当访问到 ch 为 2 的时候，1 和 2 要组成数字 12
            # 再出现 3，12 和 3 组成数字 123

```

```

if '0' <= ch <= '9':

    # 先将字符转成整型数字 ch-'0'
    # 补充知识：将字符'0'-'9'转换为数字，只需将字符变量减去'0'就行
    # 因为字符和数字在内存里都是以 ASCII 码形式存储的
    # 减去 '0'，其实就是减去字符 '0' 的 ASCII 码，而字符 '0' 的 ASCII 码是 30
    # 所以减去'0'也就是减去30，然后就可以得到字符对应的数字了。
    num = int(ch)

    # 再将这个数字和前面存储的数字进行组合
    # 1 和 2 组成数字 12，也就是  $1 * 10 + 2 = 12$ 
    # 12 和 3 组成数字 123，也就是  $12 * 10 + 3 = 123$ 
    digit = digit * 10 + num

# 2、如果是字符
elif ch >= 'a' and ch <= 'z':

    # 说明它就出现一次，可以直接存放到 res 中
    res += ch

# 3、如果是"["
# 出现了嵌套的内层编码字符串，而外层的解码需要等待内层解码的结果
# 那么之前已经扫描的字符需要存放起来，等到内层解码之后再重新使用
elif ch == '[' :

    # 把数字存放到数字栈
    numStack.append(digit)

    # 把外层的解码字符串存放到字符串栈
    strStack.append(res)

    # 开始新一轮解码
    # 于是，digit 归零
    digit = 0

    # res 重新初始化
    res = ""

# 4、如果是"]"
elif ch == ']':

    # 此时，内层解码已经有结果，需要把它和前面的字符串进行拼接

    # 第一步，先去查看内层解码的字符串需要被重复输出几次
    # 比如 e3[abc]，比如内层解码结果 abc 需要输出 3 次
    # 通过数字栈提取出次数
    count = numStack.pop()

    # 第二步，通过字符串栈提取出之前的解码字符串

```

```

        outString = strStack.pop()

        # 第三步，不断的把内层解码的字符串拼接起来
        for j in range(0,count) :
            # 拼接到 outString 后面，拼接 count 次
            outString += res

        # 再把此时得到的结果赋值给 res
        res = outString

    # 返回解码成功的字符串
    return res

```

四、复杂度分析

- 时间复杂度 $O(N)$ ，一次遍历 `s`
- 空间复杂度 $O(N)$ ，辅助栈在极端情况下需要线性空间。

LeetCode 82、删除排序链表中的重复元素 II（迭代版）

一、题目描述

给定一个已排序的链表的头 `head`，删除原始链表中所有重复数字的节点，只留下不同的数字。返回已排序的链表。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list-ii/submissions/
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        ListNode dummy = new ListNode(-1);

        // 将虚拟头节点和原链表头节点连接起来
        // 添加虚拟头节点之后，原链表的每个节点的地位都是一样的
        dummy.next = head;

        // 从虚拟头节点位置开始访问
        ListNode cur = dummy;

```



```

// 只要当前访问节点的下一个节点与下下个节点都存在，就继续访问下去
while (cur.next != null && cur.next.next != null) {

    // 在访问过程中，会出现两种情况
    // 1、下一个节点与下下个节点相同，那么说明与这个节点值相同的所有节点都应该被删除掉
    if (cur.next.val == cur.next.next.val) {

        // 删除的方法是先记录这个值
        int value = cur.next.val;

        // 利用 while 循环，不断的查找出那些相同的节点值来
        while (cur.next != null && cur.next.val == value) {

            // 每次找到了一个相同的值，那么当前访问的节点 cur 就越过这个节点
            cur.next = cur.next.next;
        }

        // 2、下一个节点与下下个节点不相同，说明 cur 可以加入到最终的结果链表中
    } else {

        // 那么继续访问后面的节点
        cur = cur.next;
    }
}

// 最终返回虚拟头节点的下一个节点就行了
return dummy.next;
}
}

```

2、C++代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list-ii/submissions/
class Solution {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        ListNode *dummy = new ListNode(-1);

        // 将虚拟头节点和原链表头节点连接起来
        // 添加虚拟头节点之后，原链表的每个节点的地位都是一样的
        dummy->next = head;

        // 从虚拟头节点位置开始访问
        ListNode *cur = dummy;
    }
}

```

```

// 只要当前访问节点的下一个节点与下下个节点都存在，就继续访问下去
while (cur->next != NULL && cur->next->next != NULL) {

    // 在访问过程中，会出现两种情况
    // 1、下一个节点与下下个节点相同，那么说明与这个节点值相同的所有节点都应该被删除掉
    if (cur->next->val == cur->next->next->val) {

        // 删除的方法是先记录这个值
        int value = cur->next->val;

        // 利用 while 循环，不断的查找出那些相同的节点值来
        while (cur->next != NULL && cur->next->val == value) {

            // 每次找到了一个相同的值，那么当前访问的节点 cur 就越过这个节点
            cur->next = cur->next->next;
        }

        // 2、下一个节点与下下个节点不相同，说明 cur 可以加入到最终的结果链表中
    } else {

        // 那么继续访问后面的节点
        cur = cur->next;
    }
}

// 最终返回虚拟头节点的下一个节点就行了
return dummy->next;

}
};

```

3、Python 代码

```

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
        dummy = ListNode(-1)

        # 将虚拟头节点和原链表头节点连接起来
        # 添加虚拟头节点之后，原链表的每个节点的地位都是一样的
        dummy.next = head

        # 从虚拟头节点位置开始访问
        cur = dummy

        # 只要当前访问节点的下一个节点与下下个节点都存在，就继续访问下去
        while cur.next and cur.next.next :

```

```
# 在访问过程中，会出现两种情况
# 1、下一个节点与下下个节点相同，那么说明与这个节点值相同的所有节点都应该被删除掉
if cur.next.val == cur.next.next.val :

    # 删除的方法是先记录这个值
    value = cur.next.val

    # 利用 while 循环，不断的查找出那些相同的节点值来
    while cur.next and cur.next.val == value :

        # 每次找到了一个相同的值，那么当前访问的节点 cur 就越过这个节点
        cur.next = cur.next.next

# 2、下一个节点与下下个节点不相同，说明 cur 可以加入到最终的结果链表中
else:

    # 那么继续访问后面的节点
    cur = cur.next

# 最终返回虚拟头节点的下一个节点就行了
return dummy.next
```

四、复杂度分析

- 时间复杂度：O(n)，其中 n 是链表的长度。
- 空间复杂度：O(1)。

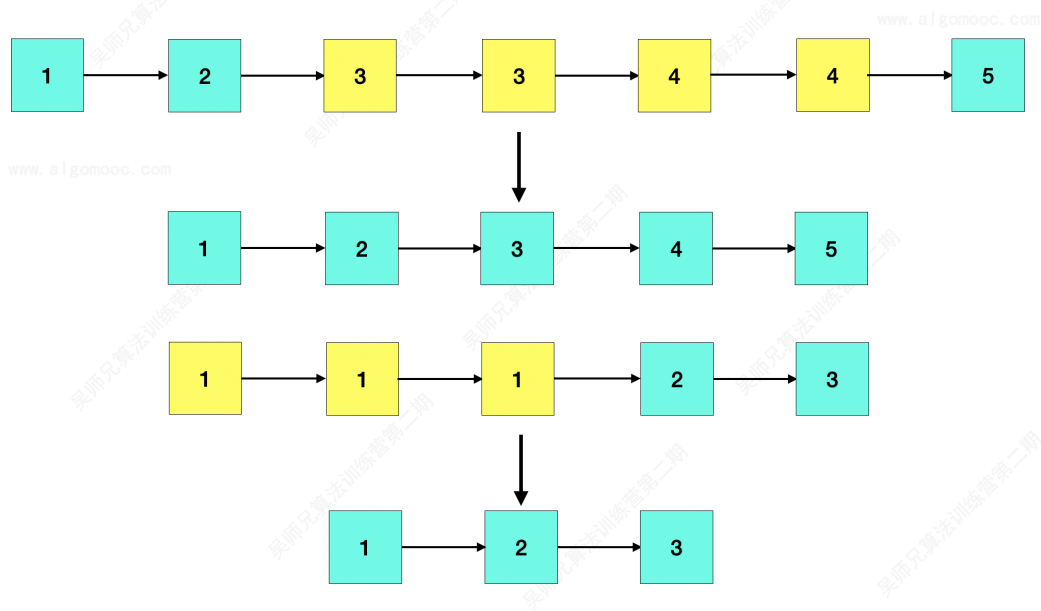
LeetCode 83、删除排序链表中的重复元素

一、题目描述

给定一个已排序的链表的头 `head`，删除所有重复的元素，使每个元素只出现一次。返回已排序的链表。

给定一个已排序的链表的头 `head`，删除所有重复的元素，使每个元素只出现一次。

返回 已排序的链表。



公众号：吴师兄算法

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list/
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        // 从链表的头节点开始访问每一个节点
        ListNode cur = head;

        // 在访问过程中，只要当前节点和当前节点的下一个节点有值，就不断访问下去
        while (cur != null && cur.next != null) {

            // 当前节点和当前节点的下一个节点有两种关系

            // 1、当前节点和当前节点的下一个节点相同，此时要删除重复元素
            // 由于链表已经是排序的，所以去重操作只需要跳过后面的这个重复的节点就行
            if (cur.val == cur.next.val) {

                // 执行这个操作之后，cur.next 被跳过去了
                cur.next = cur.next.next;

                // 2、当前节点和当前节点的下一个节点不相同，那么 cur 这个节点可以保留下来，继续访问后面的节点
            }
        }
    }
}
```

```

        } else {
            // 继续访问后面的节点
            cur = cur.next;
        }
    }

    // 返回链表的头节点就是结果
    return head;
}
}

```

2、C++代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list/
class Solution {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        // 从链表的头节点开始访问每一个节点
        ListNode *cur = head;

        // 在访问过程中，只要当前节点和当前节点的下一个节点有值，就不断访问下去
        while(cur != NULL && cur->next != NULL) {

            // 当前节点和当前节点的下一个节点有两种关系

            // 1、当前节点和当前节点的下一个节点相同，此时要删除重复元素
            // 由于链表已经是排序的，所以去重操作只需要跳过后面这个重复的节点就行
            if(cur->val == cur->next->val) {

                // 执行这个操作之后，cur->next 被跳过去了
                cur->next = cur->next->next;

                // 2、当前节点和当前节点的下一个节点不相同，那么 cur 这个节点可以保留下来，继续访问后面
                // 的节点
            } else {
                // 继续访问后面的节点
                cur = cur->next;
            }
        }

        // 返回链表的头节点就是结果
        return head;
    }
};

```

3、Python 代码

```
#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list/
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # 从链表的头节点开始访问每一个节点
        cur = head

        # 在访问过程中，只要当前节点和当前节点的下一个节点有值，就不断访问下去
        while cur and cur.next :

            # 当前节点和当前节点的下一个节点有两种关系

            # 1、当前节点和当前节点的下一个节点相同，此时要删除重复元素
            # 由于链表已经是排序的，所以去重操作只需要跳过后面这个重复的节点就行
            if cur.val == cur.next.val :

                # 执行这个操作之后，cur.next 被跳过去了
                cur.next = cur.next.next

            # 2、当前节点和当前节点的下一个节点不相同，那么 cur 这个节点可以保留下来，继续访问后面的
            # 节点
            else:
                # 继续访问后面的节点
                cur = cur.next

        # 返回链表的头节点就是结果
        return head
```

四、复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 是链表的长度。
- 空间复杂度： $O(1)$ 。

第七天

LeetCode 1475、商品折扣后的最终价格

一、题目描述

给你一个数组 `prices`，其中 `prices[i]` 是商店里第 `i` 件商品的价格。

商店里正在进行促销活动，如果你要买第 `i` 件商品，那么你可以得到与 `prices[j]` 相等的折扣，其中 `j` 是满足 `j > i` 且 `prices[j] <= prices[i]` 的最小下标，如果没有满足条件的 `j`，你将没有任何折扣。

请你返回一个数组，数组中第 `i` 个元素是折扣后你购买商品 `i` 最终需要支付的价格。

示例 1：

输入：`prices = [8,4,6,2,3]`

输出：[4,2,4,2,3]

解释：

商品 0 的价格为 `price[0]=8`，你将得到 `prices[1]=4` 的折扣，所以最终价格为 $8 - 4 = 4$ 。

商品 1 的价格为 `price[1]=4`，你将得到 `prices[3]=2` 的折扣，所以最终价格为 $4 - 2 = 2$ 。

商品 2 的价格为 `price[2]=6`，你将得到 `prices[3]=2` 的折扣，所以最终价格为 $6 - 2 = 4$ 。

商品 3 和 4 都没有折扣。

示例 2：

输入：`prices = [1,2,3,4,5]`

输出：[1,2,3,4,5]

解释：在这个例子中，所有商品都没有折扣。

示例 3：

输入：`prices = [10,1,1,6]`

输出：[9,0,1,6]

提示：

- `1 <= prices.length <= 500`
- `1 <= prices[i] <= 10^3`

二、题目解析

暂时无法在飞书文档外展示此内容

三、参考代码

代码一

单调栈解法

```
#### 题目：LC1475. 商品折扣后的最终价格
#### 难度：简单
#### 作者：吴师兄学算法
#### 算法：单调栈，时间复杂度O(N)
#### 代码看不懂的地方，请直接在群上提问
```

```

class Solution:
    def finalPrices(self, prices: List[int]) -> List[int]:

        # 原数组的长度
        n = len(prices)

        # 初始结果数组的内容，一开始都是 0
        ans = [0] * n

        # 设置一个栈
        stack = []

        # 从后向前去遍历原数组 prices
        # 从后向前去填充 ans
        for i in range(n - 1, -1, -1):
            # 获取当前的原始价格
            p = prices[i]

            # 不断执行如下的逻辑
            # 1、栈不为空
            # 2、栈顶元素大于当前的价格
            # 使得栈中的元素都小于 p
            while stack and stack[-1] > p:
                # 弹出栈顶元素
                stack.pop()

            # 此时，对于 i 这个位置的价格来说
            # 在它的右侧小于 prices[i] 并且最靠近它的价格是 stack 的栈顶元素
            # 计算差值，获取答案
            if stack:
                ans[i] = p - stack[-1]
            else:
                ans[i] = p

            # 再把当前的价格也压入栈
            # 栈是一个单调递减栈
            stack.append(p)

        # 返回答案
        return ans

```

代码二

暴力解法

题目：LC1475. 商品折扣后的最终价格
难度：简单
算法：直接暴力模拟，时间复杂度 $O(N^2)$
代码看不懂的地方，请直接在群上提问

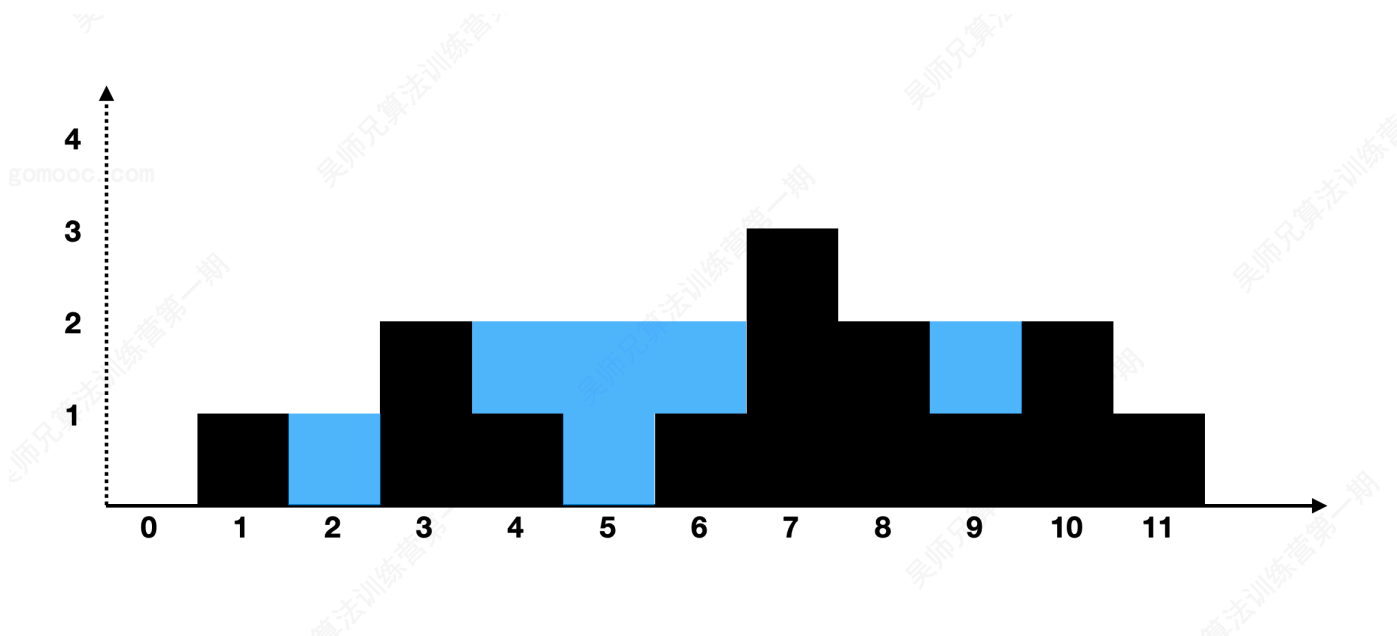
```
class Solution:
    def finalPrices(self, prices: List[int]) -> List[int]:
        for i in range(len(prices)):
            for j in range(i+1, len(prices)):
                if prices[j] <= prices[i]:
                    prices[i] -= prices[j]
                    break
        return prices
```

第一层循环，遍历整个prices数组
第二层循环，遍历索引为i+1以及之后元素
找到不大于prices[i]的第一个prices[j]
第i个元素的价格修改为折扣价
然后退出循环
返回修改后的prices数组

LeetCode 42、接雨水

一、题目描述

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。



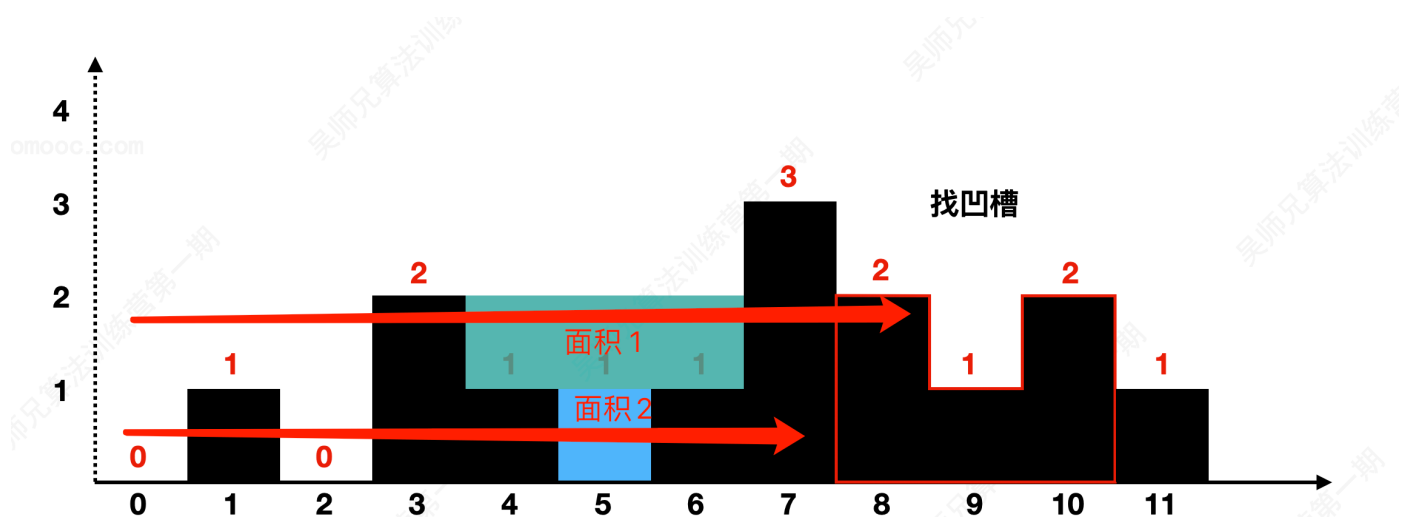
输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

二、题目解析

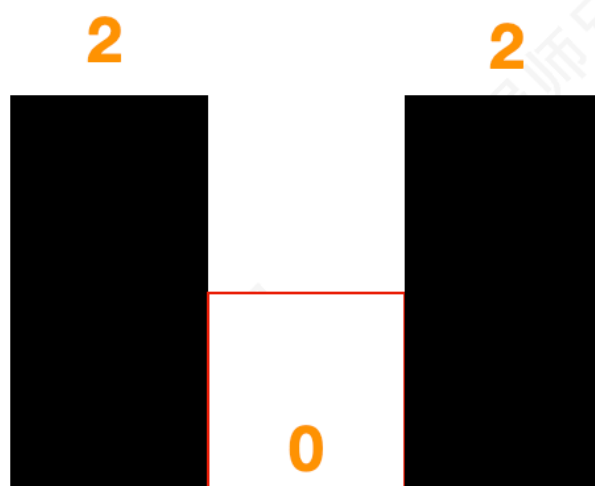
我们按照行方向去接雨水。



找出这样性质的数：它既小于左边，又小于右边

接下来我们去寻找凹槽。

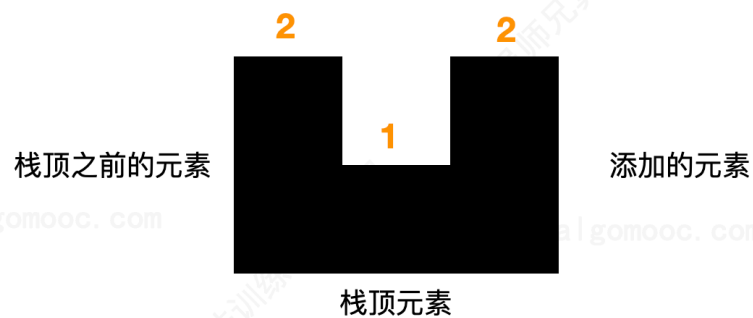
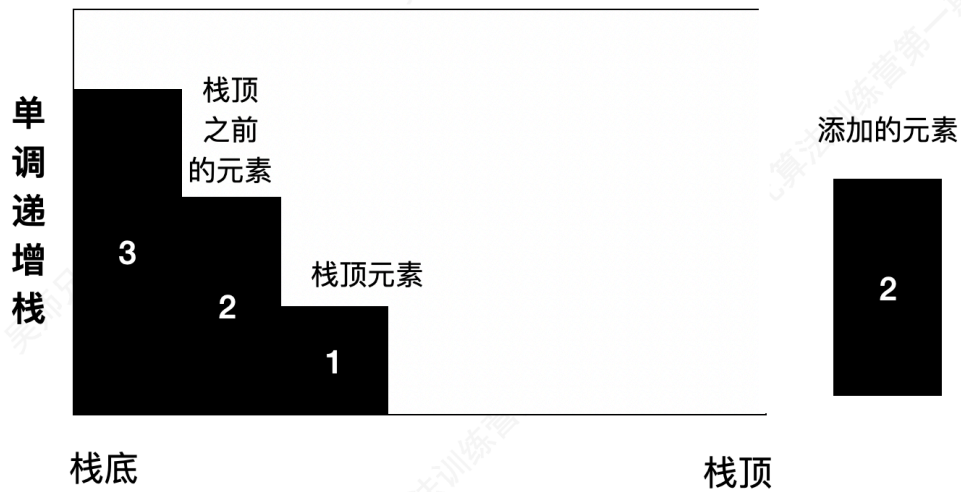
一个凹槽是由三个柱子围成的。（这里为了描述方便，我们把高度为 0 的柱子也当成存在的柱子）



对于这个凹槽来说，它的左侧和底部是由栈中挑选出来的，右侧是由新添加的柱子决定的。

什么情况会出现凹槽呢？

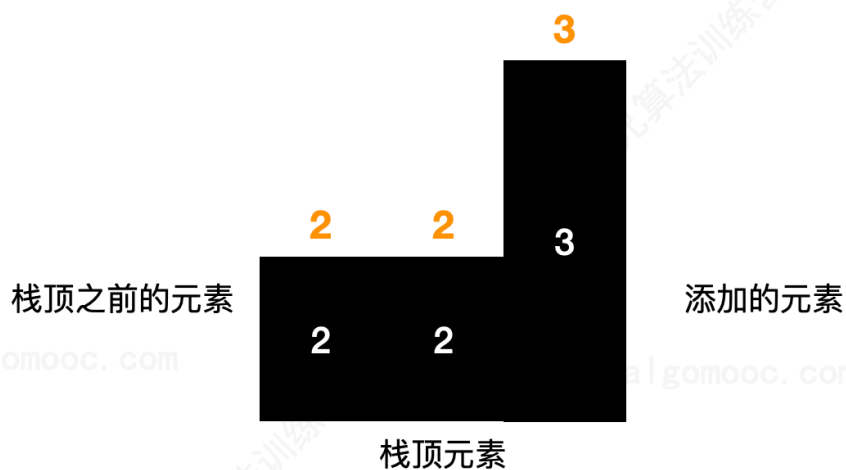
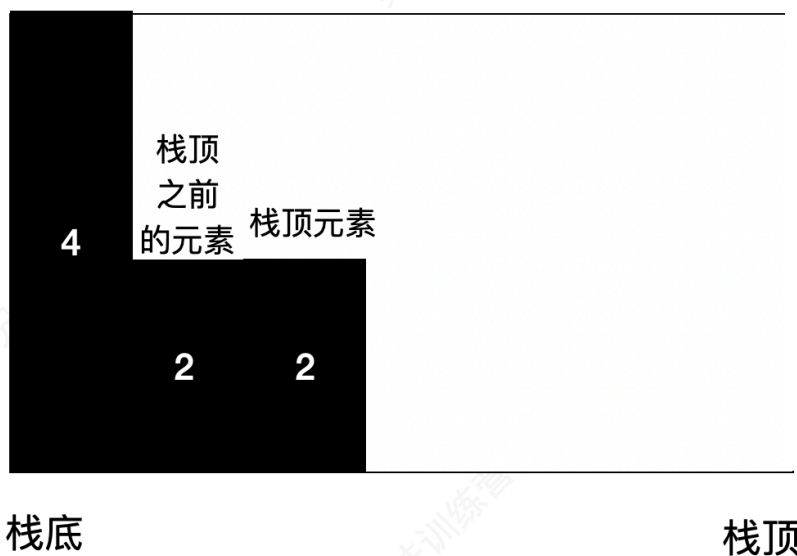
一旦新添加的柱子高度大于栈顶元素，就有可能形成凹槽！



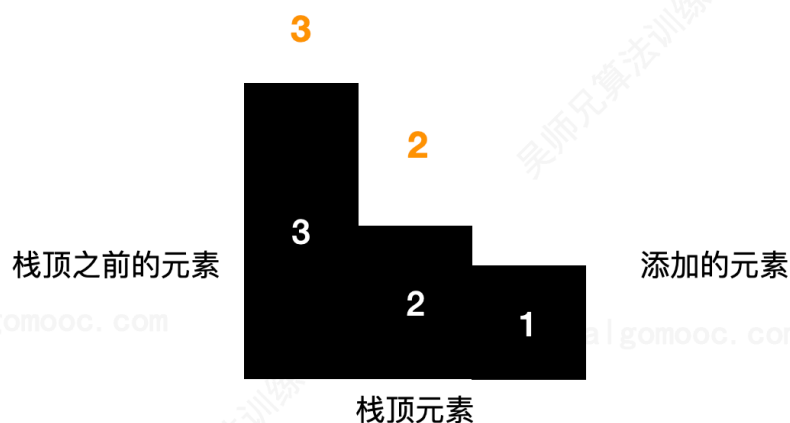
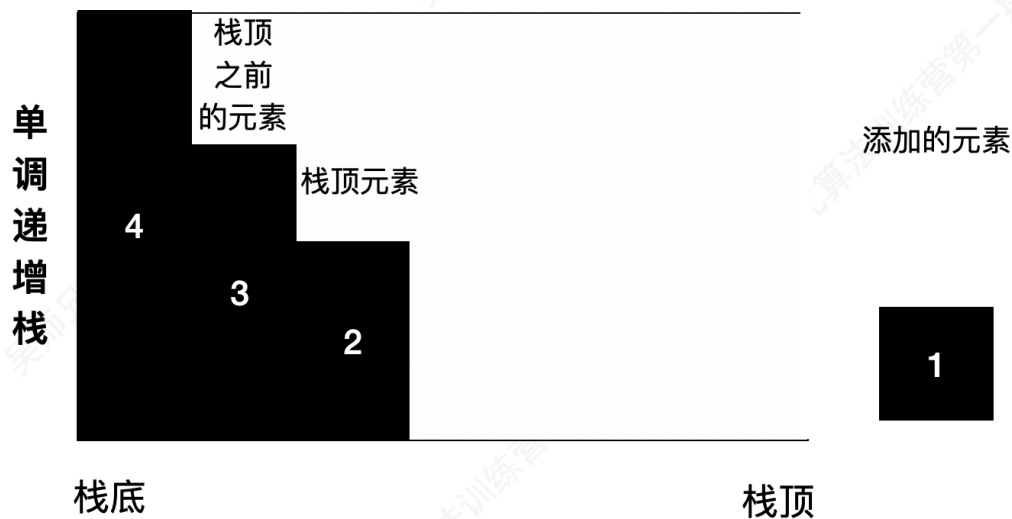
这个时候，栈顶元素是凹槽的底部，如果在栈中存在栈顶元素之前的元素，那么栈顶元素之前的元素就是凹槽的左侧，此时添加的元素是凹槽的右侧。

这里之所以是说有可能，是因为柱子里面可能是两根高度一样的柱子，即使新添加的柱子高度都大于它们，也是无法构成凹槽，或者说构成了一个面积为 0 的凹槽。

单调递增栈



如果新添加的柱子高度小于栈顶元素，那么必然无法和当前的栈中元素构成一个凹槽，因为这是一个单调栈，里面的元素越靠近栈顶越小，新添加的柱子高度都小于了栈顶元素，那必然小于里面其它的元素，这种情况下，无法围成一个凹槽，我们就把当前的柱子加入到我们的栈中，让它和里面的柱子一起等待接下来的柱子。

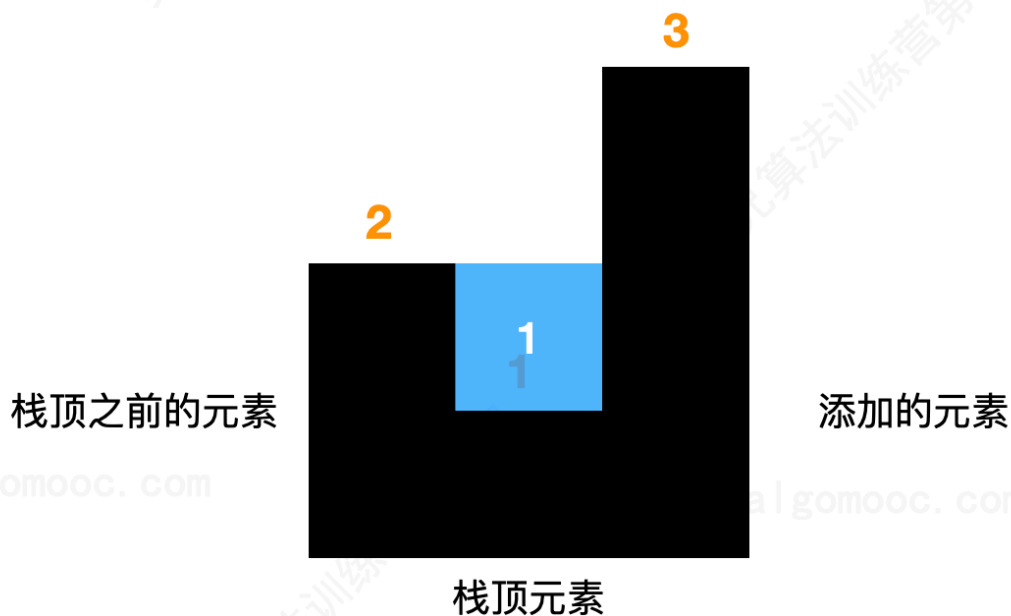


如果新添加的柱子高度等于栈顶元素，也是无法形成凹槽的，我们就把当前的柱子加入到我们的栈中，让它和里面的柱子一起等待接下来的柱子。

一旦形成了凹槽，我们去计算它的面积。

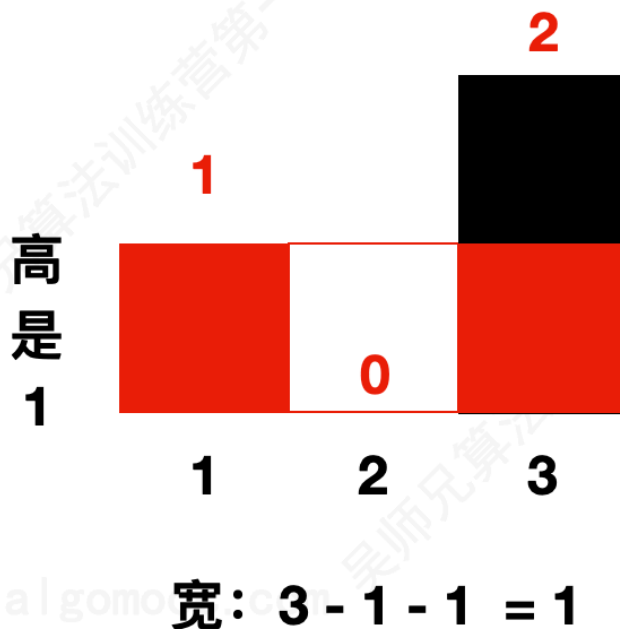
面积由高和宽决定。

凹槽的高度是由 $\min(\text{凹槽左边高度}, \text{凹槽右边高度}) - \text{凹槽底部高度}$ 来计算的。



凹槽的宽度是由凹槽右边的下标 - 凹槽左边的下标 - 1（因为只求中间宽度）来计算的。

栈顶的之前的元素、栈顶、要入栈的三个元素组成凹槽



计算完一个凹槽的面积之后，我们就把栈顶元素弹出，观察剩下的那些栈中的元素能否和新添加的元素再构成一个新的凹槽。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 接雨水 ( LeetCode 42 ) :https://leetcode-cn.com/problems/trapping-rain-water/
class Solution {
    public int trap(int[] height) {

        // 只有两根柱子，必然无法形成一个凹槽，那么水的面积就是 0
        if (height.length <= 2) return 0;

        // 构建一个栈，用来存储对应的柱子的下标
        // 注意：stack 存储的是下标而非高度
        Stack<Integer> stack = new Stack<>();

        // 一开始水的面积是 0
        int result = 0;

        // 从头开始遍历整个数组
        for (int i = 0; i < height.length; i++) {
            // 如果栈为空，那么直接把当前索引加入到栈中
            if(stack.isEmpty()){
                // 把当前索引加入到栈中
                stack.push(i);

                // 否则的话，栈里面是有值的，我们需要去判断此时的元素、栈顶元素、栈顶之前的元素能否形成
                // 一个凹槽

                // 情况一：此时的元素小于栈顶元素，凹槽的右侧不存在，无法形成凹槽
            }else if (height[i] < height[stack.peek()]) {
                // 把当前索引加入到栈中
                stack.push(i);

                // 情况二：此时的元素等于栈顶元素，也是无法形成凹槽
            } if (height[i] == height[stack.peek()]) {

                // 把当前索引加入到栈中
                stack.push(i);

                // 情况三：此时的元素大于栈顶元素，有可能形成凹槽
                // 注意是有可能形成，因为比如栈中的元素是 2 、2 ，此时的元素是 3，那么是无法形成凹槽
                // 的

                // 由于栈中有可能存在多个元素，移除栈顶元素之后，剩下的元素和此时的元素也有可能形成凹
                // 槽
            } else {
```

```

// 因此，我们需要不断的比较此时的元素和栈顶元素
// 此时的元素依旧大于栈顶元素时，我们去计算此时的凹槽面积
// 借助 while 循环来实现这个操作
while (!stack.empty() && height[i] > height[stack.peek()]) {

    // 1、获取栈顶的下标，bottom 为凹槽的底部位置
    int bottom = stack.peek();

    // 将栈顶元素推出，去判断栈顶之前的元素是否存在，即凹槽的左侧是否存在
    stack.pop();

    // 2、如果栈不为空，即栈中有元素，即凹槽的左侧存在
    if (!stack.empty()) {

        // 凹槽左侧的高度 height[stack.peek()] 和 凹槽右侧的高度 height[i]
        // 这两者的最小值减去凹槽底部的高度就是凹槽的高度
        int h = Math.min(height[stack.peek()], height[i]) -
height[bottom];

        // 凹槽的宽度等于凹槽右侧的下标值 i 减去凹槽左侧的下标值 stack.peek 再减
去 1

        int w = i - stack.peek() - 1;

        System.out.println("凹槽右侧-->" + i);
        System.out.println("凹槽左侧-->" + stack.peek());
        System.out.println("凹槽高度-->" + h);
        System.out.println("凹槽宽度-->" + w);
        System.out.println("凹槽面积-->" + h * w);
        System.out.println("-----");

        // 将计算的结果累加到最终的结果上去
        result += h * w;
    }
}

// 栈中和此时的元素可以形成栈的情况在上述 while 循环中都已经判断了
// 那么，此时栈中的元素必然不可能大于此时的元素，所以可以把此时的元素添加到栈中
stack.push(i);
}

// 最后返回结果即可
return result;
}
}

```


2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 接雨水 ( LeetCode 42 ) :https://leetcode-cn.com/problems/trapping-rain-water/
class Solution {
public:
    int trap(vector<int>& height) {
        // 只有两根柱子，必然无法形成一个凹槽，那么水的面积就是 0
        if (height.size() <= 2) return 0;

        // 构建一个栈，用来存储对应的柱子的下标
        // 注意：stk 存储的是下标而非高度
        stack<int> stk ;

        // 一开始水的面积是 0
        int result = 0;

        // 从头开始遍历整个数组
        for (int i = 0; i < height.size(); i++) {
            // 如果栈为空，那么直接把当前索引加入到栈中
            if(stk.empty()){
                // 把当前索引加入到栈中
                stk.push(i);

                // 否则的话，栈里面是有值的，我们需要去判断此时的元素、栈顶元素、栈顶之前的元素能否形成
                // 一个凹槽

                // 情况一：此时的元素小于栈顶元素，凹槽的右侧不存在，无法形成凹槽
            }else if (height[i] < height[stk.top()]) {
                // 把当前索引加入到栈中
                stk.push(i);

                // 情况二：此时的元素等于栈顶元素，也是无法形成凹槽
            } else if (height[i] == height[stk.top()]) {

                // 把当前索引加入到栈中
                stk.push(i);

                // 情况三：此时的元素大于栈顶元素，有可能形成凹槽
                // 注意是有可能形成，因为比如栈中的元素是 2 、2 ，此时的元素是 3，那么是无法形成凹槽
                // 的

                // 由于栈中有可能存在多个元素，移除栈顶元素之后，剩下的元素和此时的元素也有可能形成凹
                // 槽

                // 因此，我们需要不断的比较此时的元素和栈顶元素
                // 此时的元素依旧大于栈顶元素时，我们去计算此时的凹槽面积
            } else {
```

```

// 借助 while 循环来实现这个操作
while (!stk.empty() && height[i] > height[stk.top()]) {

    // 1、获取栈顶的下标，bottom 为凹槽的底部位置
    int bottom = stk.top();

    // 将栈顶元素推出，去判断栈顶之前的元素是否存在，即凹槽的左侧是否存在
    stk.pop();

    // 2、如果栈不为空，即栈中有元素，即凹槽的左侧存在
    if (!stk.empty()) {

        // 凹槽左侧的高度 height[stk.top()] 和 凹槽右侧的高度 height[i]
        // 这两者的最小值减去凹槽底部的高度就是凹槽的高度
        int h = min(height[stk.top()], height[i]) - height[bottom];

        // 凹槽的宽度等于凹槽右侧的下标值 i 减去凹槽左侧的下标值 stk.top 再减去
1
        int w = i - stk.top() - 1;

        // 将计算的结果累加到最终的结果上去
        result += h * w;
    }
}

// 栈中和此时的元素可以形成栈的情况在上述 while 循环中都已经判断了
// 那么，此时栈中的元素必然不可能大于此时的元素，所以可以把此时的元素添加到栈中
stk.push(i);
}
}

// 最后返回结果即可
return result;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 接雨水 ( LeetCode 42 ) :https://leetcode-cn.com/problems/trapping-rain-water/
class Solution:
    def trap(self, height: List[int]) -> int:
        # 构建一个栈，用来存储对应的柱子的下标
        # stack 存储的是下标而非高度
        stack = list()

```

```

# 一开始水的面积是 0
result = 0

# 获取数组的长度
n = len(height)

# 从头开始遍历整个数组
for i, h in enumerate(height):
    # 如果栈为空，那么直接把当前索引加入到栈中
    if not stack :
        # 把当前索引加入到栈中
        stack.append(i)

    # 否则的话，栈里面是有值的，我们需要去判断此时的元素、栈顶元素、栈顶之前的元素能否形成一
    # 个凹槽

    # 情况一：此时的元素小于栈顶元素，凹槽的右侧不存在，无法形成凹槽
    elif height[i] < height[stack[-1]]:
        # 把当前索引加入到栈中
        stack.append(i)

    # 情况二：此时的元素等于栈顶元素，也是无法形成凹槽
    elif height[i] == height[stack[-1]]:

        # 把当前索引加入到栈中
        stack.append(i)

    # 情况三：此时的元素大于栈顶元素，有可能形成凹槽
    # 注意是有可能形成，因为比如栈中的元素是 2、2，此时的元素是 3，那么是无法形成凹槽的
    else :

        # 由于栈中有可能存在多个元素，移除栈顶元素之后，剩下的元素和此时的元素也有可能形成凹
        # 槽

        # 因此，我们需要不断的比较此时的元素和栈顶元素
        # 此时的元素依旧大于栈顶元素时，我们去计算此时的凹槽面积
        # 借助 while 循环来实现这个操作
        while stack and height[i] > height[stack[-1]]:

            # 1、获取栈顶的下标，bottom 为凹槽的底部位置
            bottom = stack[-1]

            # 将栈顶元素推出，去判断栈顶之前的元素是否存在，即凹槽的左侧是否存在
            stack.pop()

            # 2、如果栈不为空，即栈中有元素，即凹槽的左侧存在
            if stack :

                # 凹槽左侧的高度 height[stack[-1]] 和 凹槽右侧的高度 height[i]
                # 这两者的最小值减去凹槽底部的高度就是凹槽的高度
                h = min(height[stack[-1]], height[i]) - height[bottom]

```

```

1          # 凹槽的宽度等于凹槽右侧的下标值 i 减去凹槽左侧的下标值 stack.top 再减去

          w = i - stack[-1] - 1

          # 将计算的结果累加到最终的结果上去
          result += h * w

          # 栈中和此时的元素可以形成栈的情况在上述 while 循环中都已经判断了
          # 那么，此时栈中的元素必然不可能大于此时的元素，所以可以把此时的元素添加到栈中
          stack.append(i)

          # 最后返回结果即可
          return result

```

四、复杂度分析

时间复杂度：O(n)，其中 n 是数组 height 的长度。从 0 到 n-1 的每个下标最多只会入栈和出栈各一次。

空间复杂度：O(n)，其中 nn 是数组 height 的长度。空间复杂度主要取决于栈空间，栈的大小不会超过 n。

LeetCode 739、每日温度

一、题目描述

根据每日 气温 列表，请重新生成一个列表，对应位置的输入是你需要再等待多久温度才会升高超过该日的天数。如果之后都不会升高，请在该位置用 0 来代替。

例如，给定一个列表 `temperatures = [73, 74, 75, 71, 69, 72, 76, 73]`，你的输出应该是 `[1, 1, 4, 2, 1, 1, 0, 0]`。

提示：气温 列表长度的范围是 `[1, 30000]`。每个气温的值的均为华氏度，都是在 `[30, 100]` 范围内的整数。

二、题目解析

这道题目最“难”的一个点是题目的理解。

给定列表 `temperatures = [73, 74, 75, 71, 69, 72, 76, 73]`，为啥输出就是 `[1, 1, 4, 2, 1, 1, 0, 0]`？

下面来一个个进行解释。

对于输入 73，它需要 经过一天 才能等到温度的升高，也就是在第二天的时候，温度升高到 74，所以对应的结果是 1。

对于输入 74，它需要 经过一天 才能等到温度的升高，也就是在第三天的时候，温度升高到 75，所以对应的结果是 1。

对于输入 75，它经过 1 天后发现温度是 71，没有超过它，继续等，一直 等了四天，在第七天才等到温度的升高，温度升高到 76，所以对应的结果是 4。

对于输入 71，它经过 1 天后发现温度是 69，没有超过它，继续等，一直等了两天，在第六天才等到温度的升高，温度升高到 72，所以对应的结果是 2。

对于输入 69，它经过一天后发现温度是 72，已经超过它，所以对应的结果是 1。

对于输入 72，它经过一天后发现温度是 76，已经超过它，所以对应的结果是 1。

对于输入 76，后续没有温度可以超过它，所以对应的结果是 0。

对于输入 73，后续没有温度可以超过它，所以对应的结果是 0。

也就是说，这道题目就是给你一个值，让你找到右边第一个比它大的数，它们两则的下标差就是输出结果。

好了，理解了题意我们来思考如何求解：借助单独递增栈来处理。

具体操作如下：

遍历整个数组，如果栈不空，且当前数字大于栈顶元素，那么如果直接入栈的话就不是递增栈，所以需要取出栈顶元素，由于当前数字大于栈顶元素的数字，而且一定是第一个大于栈顶元素的数，直接求出下标差就是二者的距离。

继续看新的栈顶元素，直到当前数字小于等于栈顶元素停止，再将数字入栈，这样就可以一直保持递增栈，且每个数字和第一个大于它的数的距离也可以算出来。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 每日温度 ( LeetCode 739 ) : https://leetcode-cn.com/problems/daily-temperatures
class Solution {
    /**
     * 维护一个单调递增栈，栈内元素从栈底到栈顶依次减小
     * 入栈的元素要和当前栈内栈首元素进行比较
     * 如果大于栈首元素，说明温度比之前更高，那么它们的下标差就是栈首元素等了多少天等到的更高温度的结果
     * 如果小于栈首元素，说明温度比之前更低，说明还没有等到更高的温度，直接放入到栈中
     */
    public static int[] dailyTemperatures(int[] temperatures) {

        // 构建一个栈,用来存放每日温度的下标
        Stack<Integer> stack = new Stack<>();

        // 构建一个数组，用来保存输出结果
        int[] res = new int[temperatures.length];

        // 从头开始遍历每天的温度
        for (int i = 0; i < temperatures.length; i++) {

            // 拿到当天的温度，需要和栈首元素进行比较
            // 如果此时栈不为空并且当天的温度大于栈首元素
```

```

        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {

            // 首先获取栈首元素的值，并将元素从栈中移除
            int preIndex = stack.pop();

            // 它们的下标差就是栈首元素等了多少天等到的更高温度的结果，保存到输出数组 res 中
            res[preIndex] = i - preIndex;
        }

        // 再把当天的温度的下标值存放到栈中
        stack.push(i);
    }
    // 最后输出 res 数组即可
    return res;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 每日温度 ( LeetCode 739 ) : https://leetcode-cn.com/problems/daily-temperatures
/**
 * 维护一个单调递增栈，栈内元素从栈底到栈顶依次减小
 * 入栈的元素要和当前栈内栈首元素进行比较
 * 如果大于栈首元素，说明温度比之前更高，那么它们的下标差就是栈首元素等了多少天等到的更高温度的结果
 * 如果小于栈首元素，说明温度比之前更低，说明还没有等到更高的温度，直接放入到栈中
 */
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& T) {
        // 构建一个栈,用来存放每日温度的下标
        stack<int> stk;

        // 构建一个数组，用来保存输出结果
        vector<int> res(T.size());

        // 从头开始遍历每天的温度
        for (int i = 0; i < T.size(); i++) {

            // 拿到当天的温度，需要和栈首元素进行比较
            // 如果此时栈不为空并且当天的温度大于栈首元素
            while (!stk.empty() && T[i] > T[stk.top()]) {

                // 首先获取栈首元素的值，并将元素从栈中移除
                int preIndex = stk.top();

```

```

        stk.pop();

        // 它们的下标差就是栈首元素等了多少天等到的更高温度的结果，保存到输出数组 res 中
        res[preIndex] = i - preIndex;
    }

    // 再把当天的温度的下标值存放到栈中
    stk.push(i);
}
// 最后输出 res 数组即可
return res;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 每日温度 ( LeetCode 739 ) : https://leetcode-cn.com/problems/daily-temperatures
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:

        # 构建一个栈,用来存放每日温度的下标
        stack = []

        # 获取数组长度
        length = len(temperatures)

        # 构建一个数组,用来保存输出结果
        res = [0] * length

        # 从头开始遍历每天的温度
        for i in range(length):
            # 拿到当天的温度,不断的和栈顶元素进行比较
            temperature = temperatures[i]

            # 如果栈顶元素存在并且当天的温度大于栈顶元素
            # 意味着栈顶元素等到了第一个温度比它更高的那一天
            # 它们的下标差就是栈顶元素等了多少天等到的更高温度的结果
            while stack and temperature > temperatures[stack[-1]]:

                # 首先获取栈顶元素的值,也就是每日温度的下标值
                preIndex = stack.pop()

                # 它们的下标差就是栈顶元素等了多少天等到的更高温度的结果,保存到输出数组 res 中
                res[preIndex] = i - preIndex

```

```
# 再把当天的温度的下标值存放到栈中
# 注意：是下标索引值
stack.append(i)

# 最后输出 res 数组即可
return res
```

四、复杂度分析

时间复杂度： $O(n)$ ，其中 n 是温度列表的长度。正向遍历温度列表一遍，对于温度列表中的每个下标，最多有一次进栈和出栈的操作。

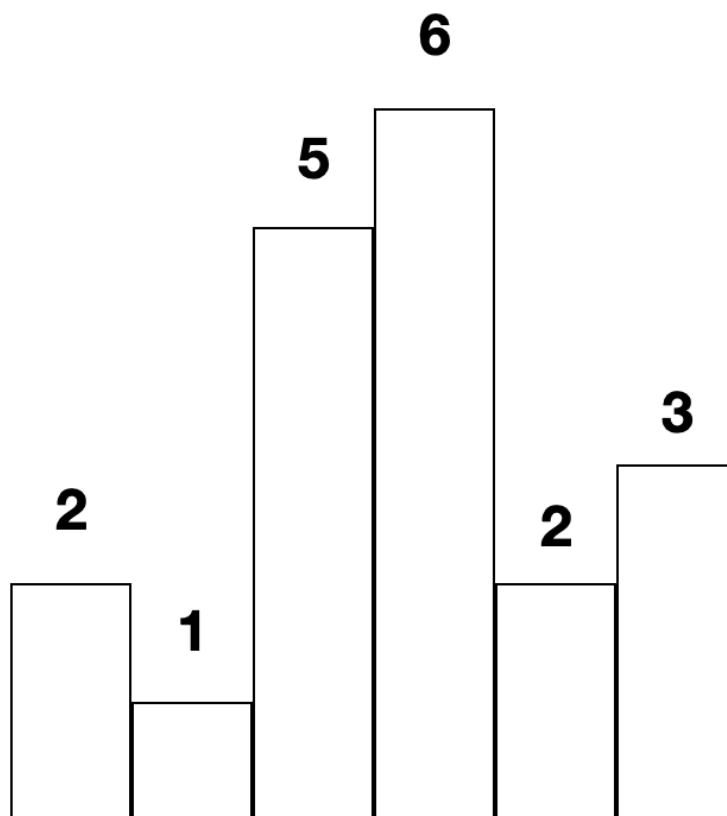
空间复杂度： $O(n)$ ，其中 n 是温度列表的长度。需要维护一个单调栈存储温度列表中的下标。

LeetCode 84、柱状图中的最大矩形

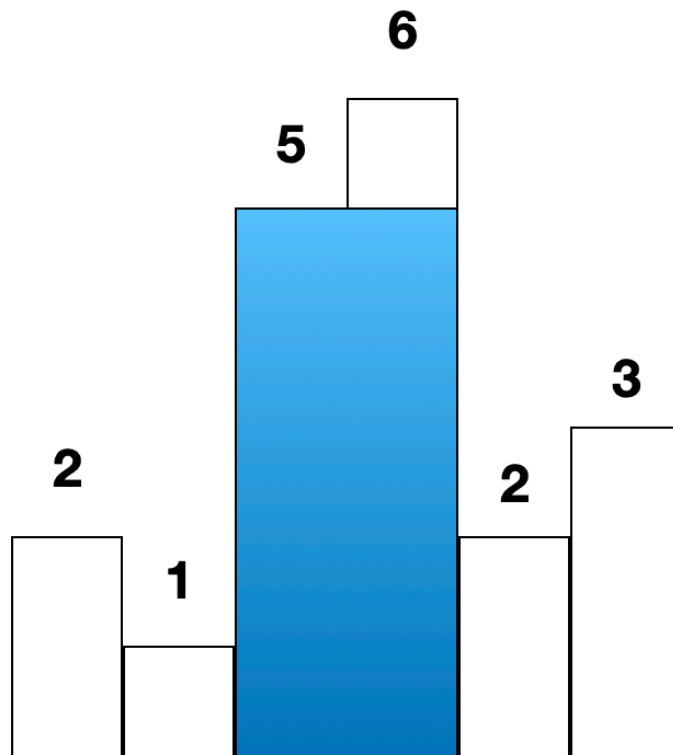
一、题目描述

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。

求在该柱状图中，能够勾勒出来的矩形的最大面积。



以上是柱状图的示例，其中每个柱子的宽度为 1，给定的高度为 `[2,1,5,6,2,3]`。



图中阴影部分为所能勾勒出的最大矩形面积，其面积为 10 个单位。

示例:

输入: [2,1,5,6,2,3]
输出: 10

二、题目解析

我们依旧用 **四步分析法** 来分析一下这道题目。

- 模拟：模拟题目的运行。
- 规律：尝试总结出题目的一般规律和特点。
- 匹配：找到符合这些特点的数据结构与算法。
- 边界：考虑特殊情况。

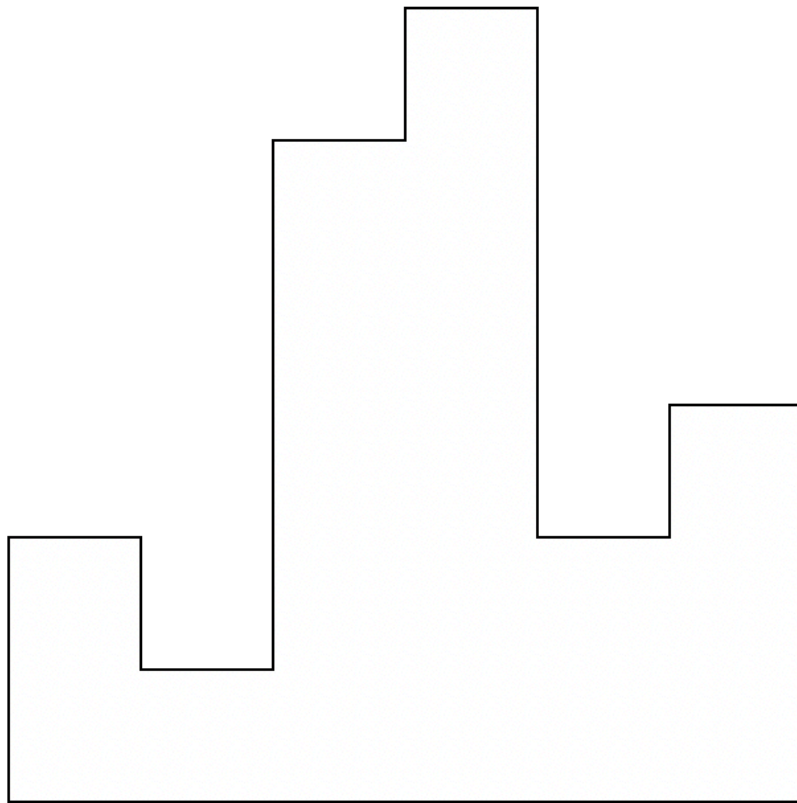
“

以下的整体设计思路参考于 liweiwei 的题解，liweiwei 是 LeetCode 大佬，欢迎前往关注：<https://leetcode-cn.com/u/liweiwei1419/>

”

模拟

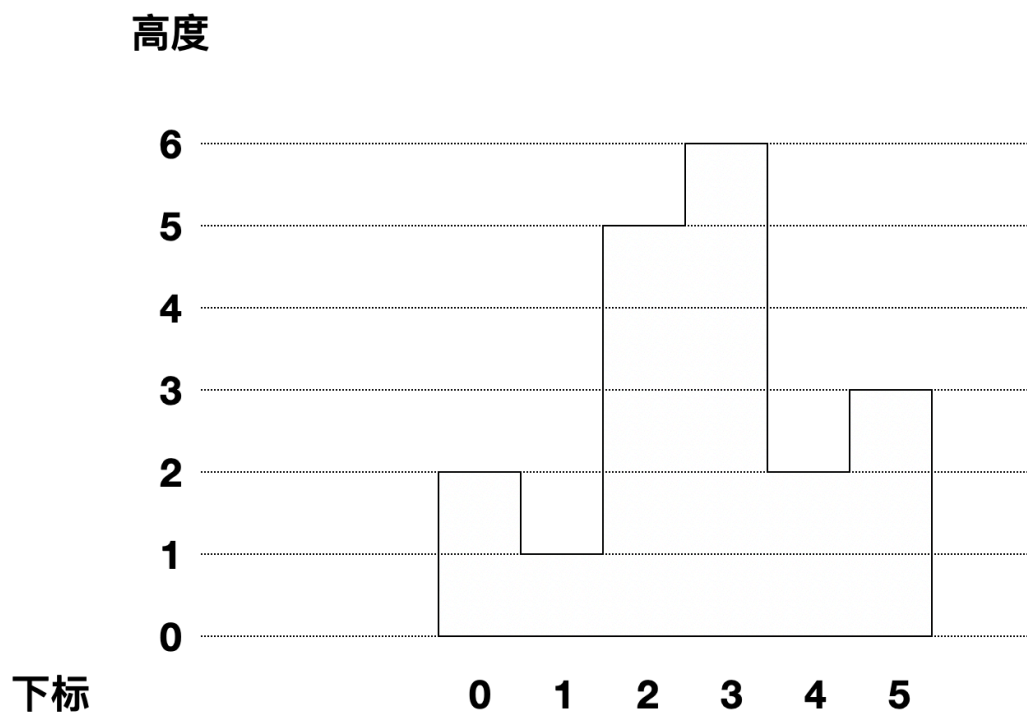
相当于给我们这样一块木板，怎么样去切才能得到一个面积最大的矩形。



矩形的面积由 **高** 和 **宽** 决定的，高确定的情况下，**宽** 的数值越大，矩形面积越大；同样的，宽确定的情况下，**高** 的数值越大，矩形面积越大。

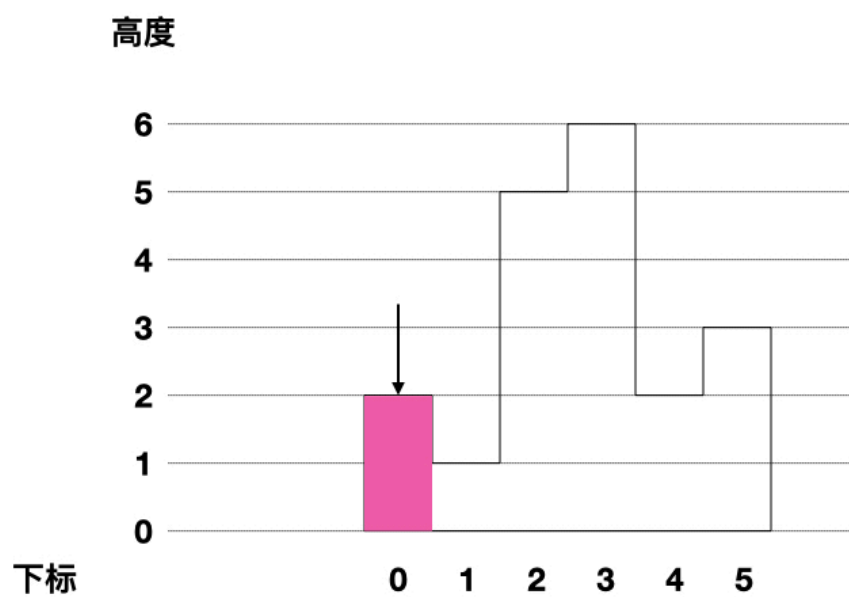
所以，一个可执行的思路就是**依次遍历柱形的高度**，求出每个高度下能切出的最大矩形时的宽度是多少。

以本题为例，有 6 种情况，高度是 2、1、5、6、2、3。



LeetCode 84. 柱状图中最大的矩形

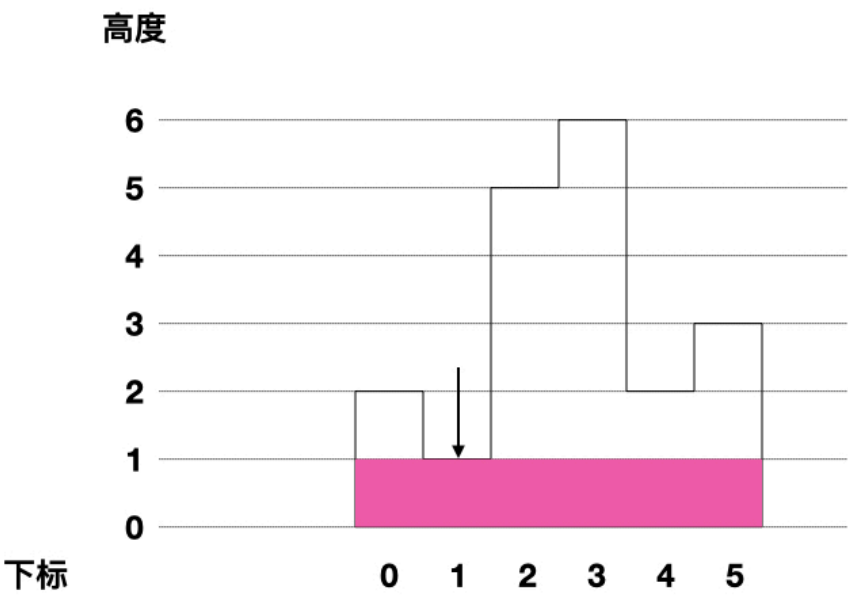
作者：程序员吴师兄



以 `heights[0]` 为高度的最大矩形

此时高度为 2，宽的选择只有 1 这种可能

LeetCode 84. 柱状图中最大的矩形
作者：程序员吴师兄

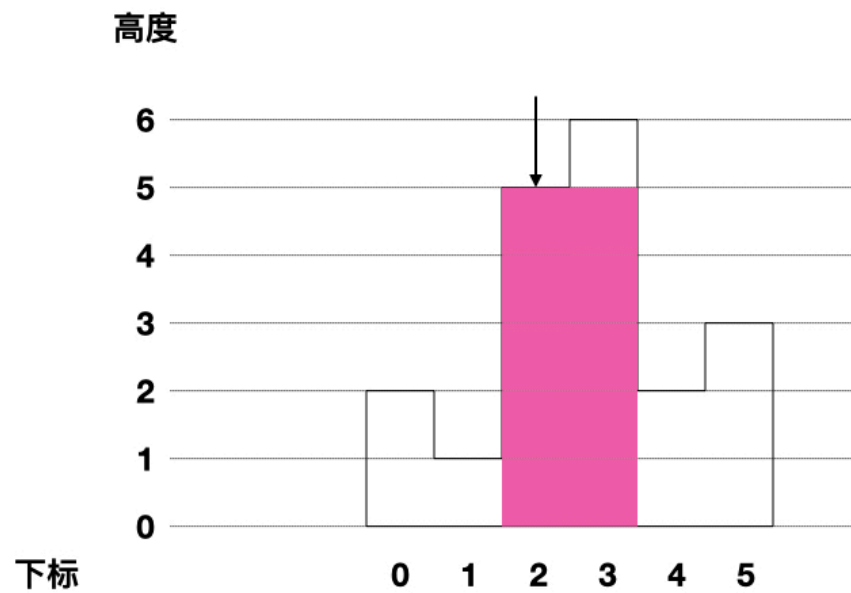


以 `heights[1]` 为高度的最大矩形

此时高度为 1，宽的选择有 1、2、3、4、5、6 这些可能

LeetCode 84. 柱状图中最大的矩形

作者：程序员吴师兄

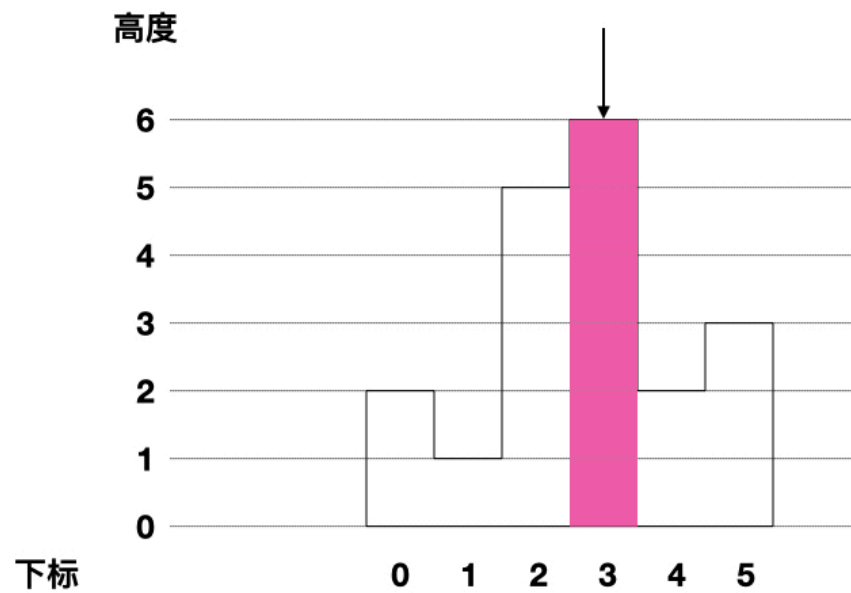


以 `heights[2]` 为高度的最大矩形

此时高度为 5，宽的选择有 1、2 这些可能

LeetCode 84. 柱状图中最大的矩形

作者：程序员吴师兄

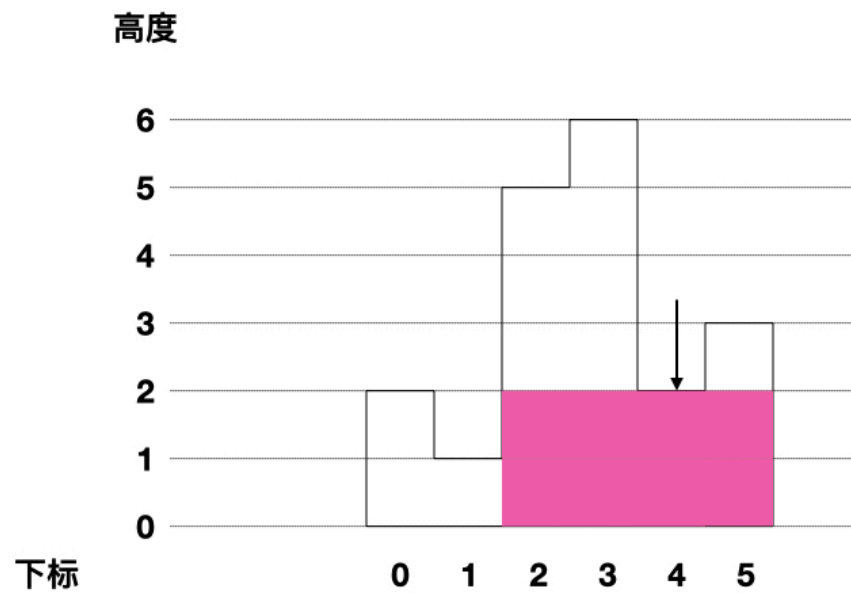


以 `heights[3]` 为高度的最大矩形

此时高度为 6，宽的选择只有 1 这个可能

LeetCode 84. 柱状图中最大的矩形

作者：程序员吴师兄

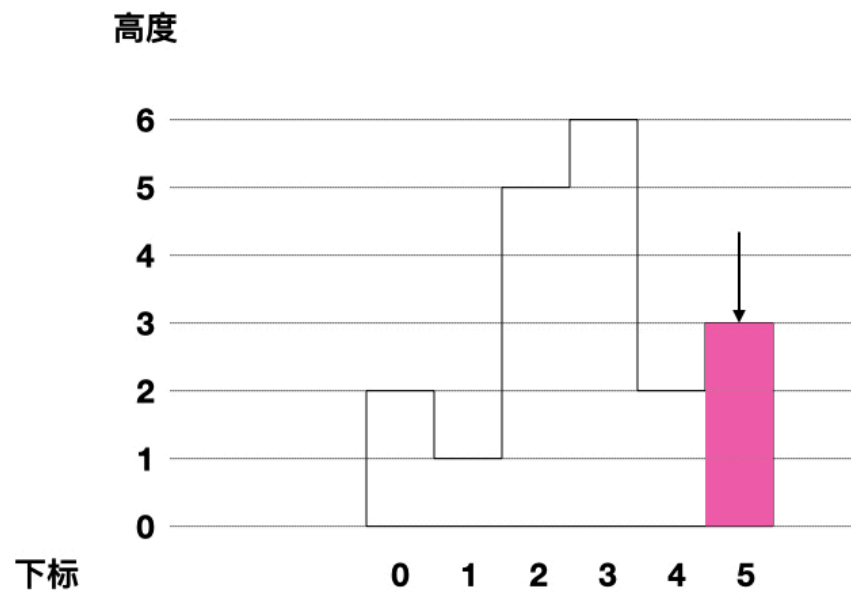


以 `heights[4]` 为高度的最大矩形

此时高度为 2，宽的选择有 1、2、3、4 这些可能

LeetCode 84. 柱状图中最大的矩形

作者：程序员吴师兄



以 `heights[5]` 为高度的最大矩形

此时高度为 3，宽的选择只有 1 这个可能

84图片版.011

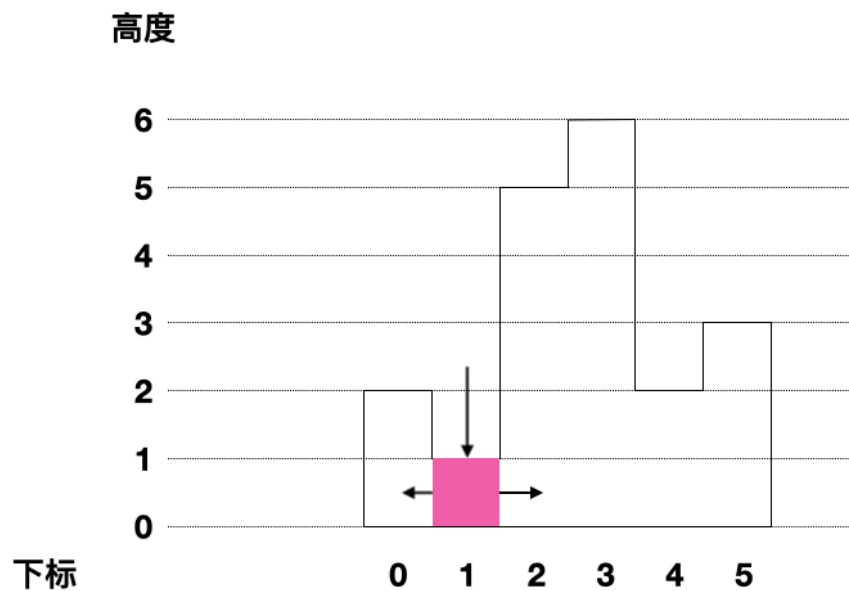
规律

也就是说，在高度确定的情况下，我们要尽可能的去扩充宽度，怎么扩充呢？

向左和向右延伸，直到碰壁。

LeetCode 84. 柱状图中最大的矩形

作者：程序员吴师兄



以 `heights[1]` 为高度的最大矩形

此时高度为 1，宽的选择有 1、2、3、4、5、6 这些可能

按照这样的思路，需要循环嵌套两次去遍历，再对比得出最大的矩形面积，这样的代码很好写，时间复杂度来到了 $O(N^2)$ 。

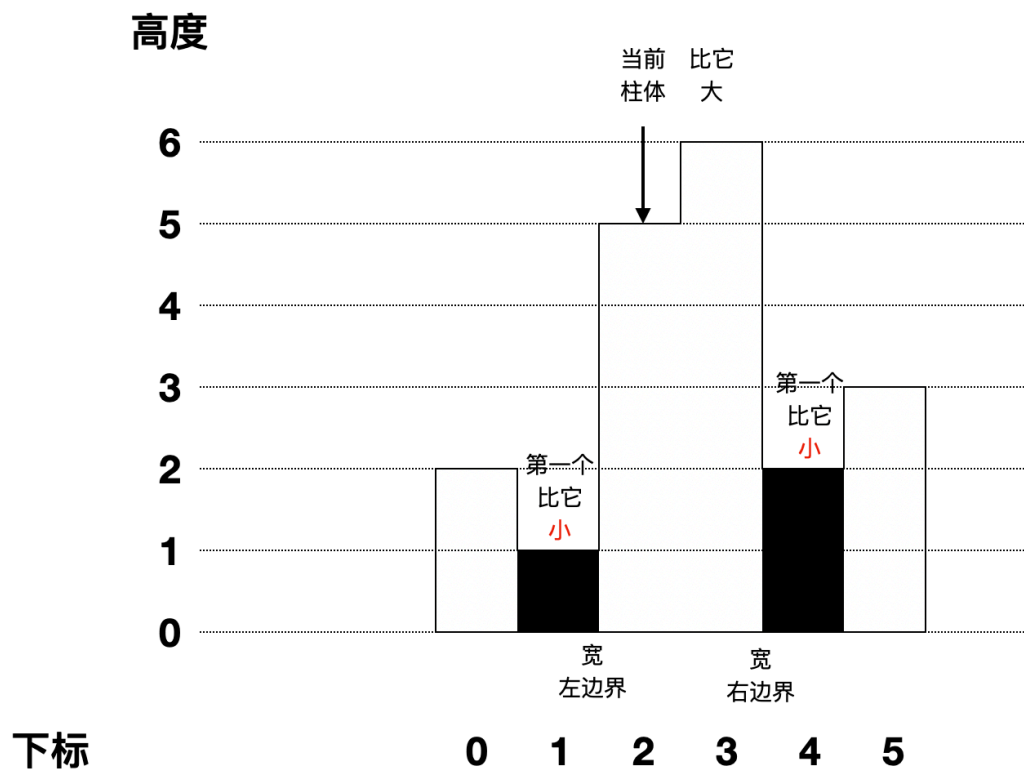
```
for (遍历高) {  
    while(遍历宽){  
    }  
}
```

需要优化。

再重复一次，矩形的面积是由 **高** 和 **宽** 共同决定的，遍历高这个操作无法省略，优化的方向在于能不能在 $O(1)$ 的时间里获取到它的最大宽度。

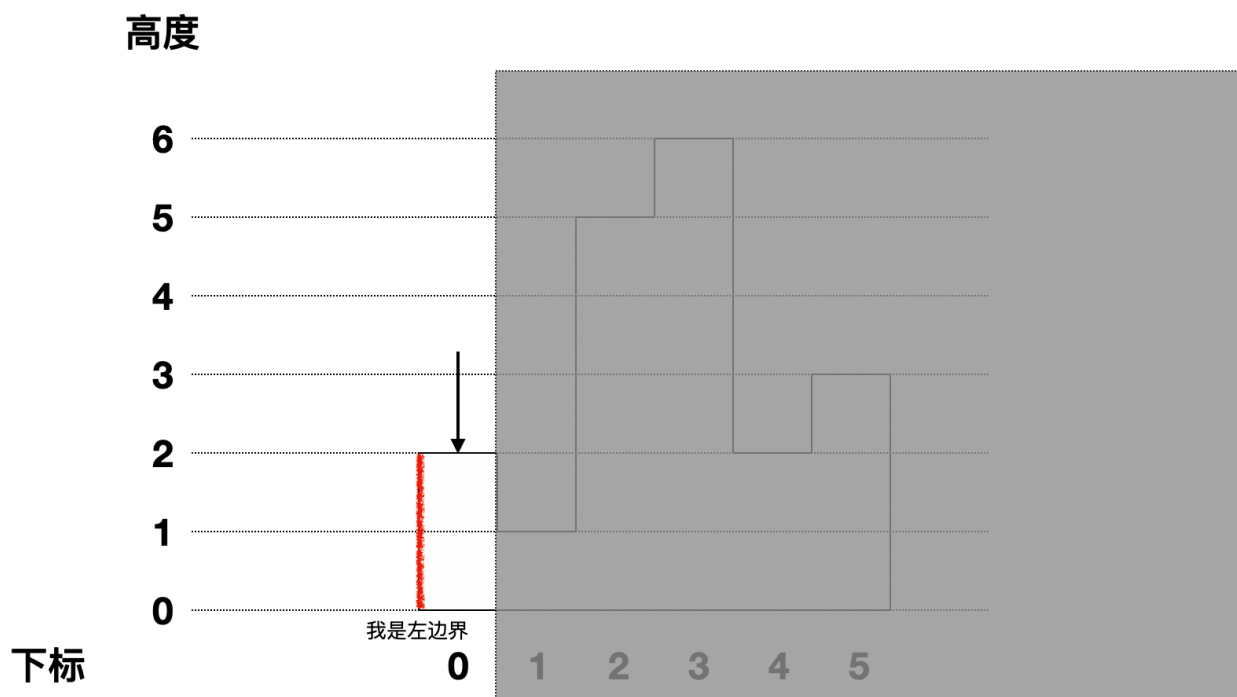
宽度是由什么决定的？

左边第一个比当前柱体小的柱体和右边第一个比当前柱体小的柱体。

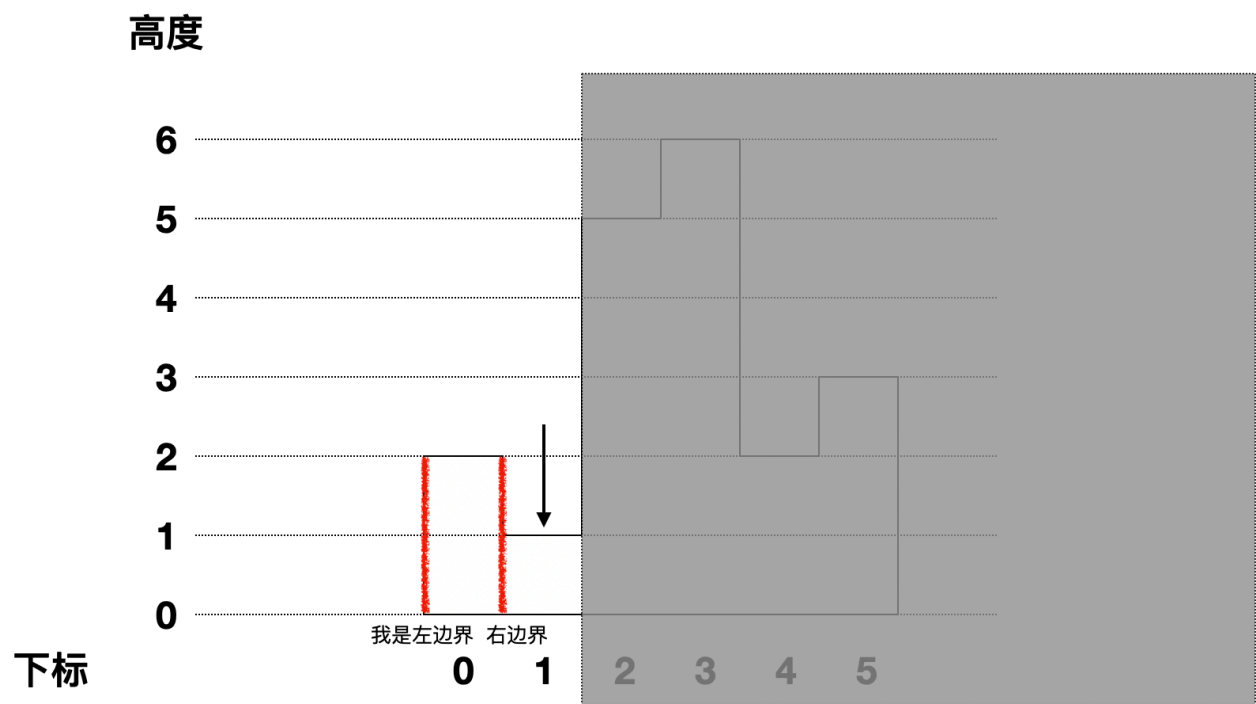


所以，我们重新来 模拟 一下整个过程。

模拟 2



请看上图，当遍历到高度为 2 时，左边界是确定了的，但右边是什么情况一无所知，被阴影笼罩，无法确定右边界，宽度也就无法确定，那先不管，继续前行，让阴影的面积变小。



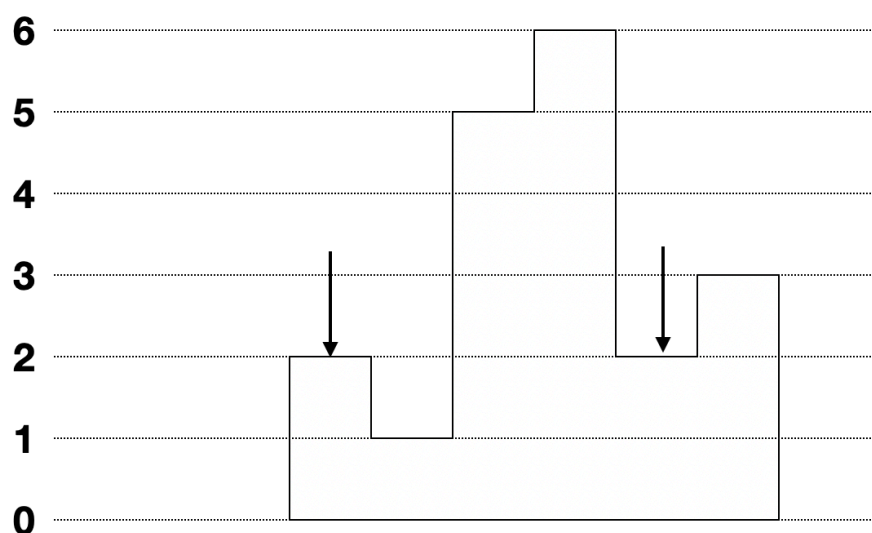
出现了第一个比高度为 2 时还小的那个柱体，那么高度为 2 的柱体的右边界确定了

也就是说，当出现了第一个比当前柱体高度还小的柱体时，可以确定当前柱体的右边界，以上图为例，左边界和右边界确定下来的宽度为 1，此时，高度为 2 的最大矩形面积是 $1 * 2 = 2$ 。

我们以上帝视角来看待这个不规则的矩形，高度为 2 的矩形有很多种。

高度

高度为 2 的矩形有多个



下标

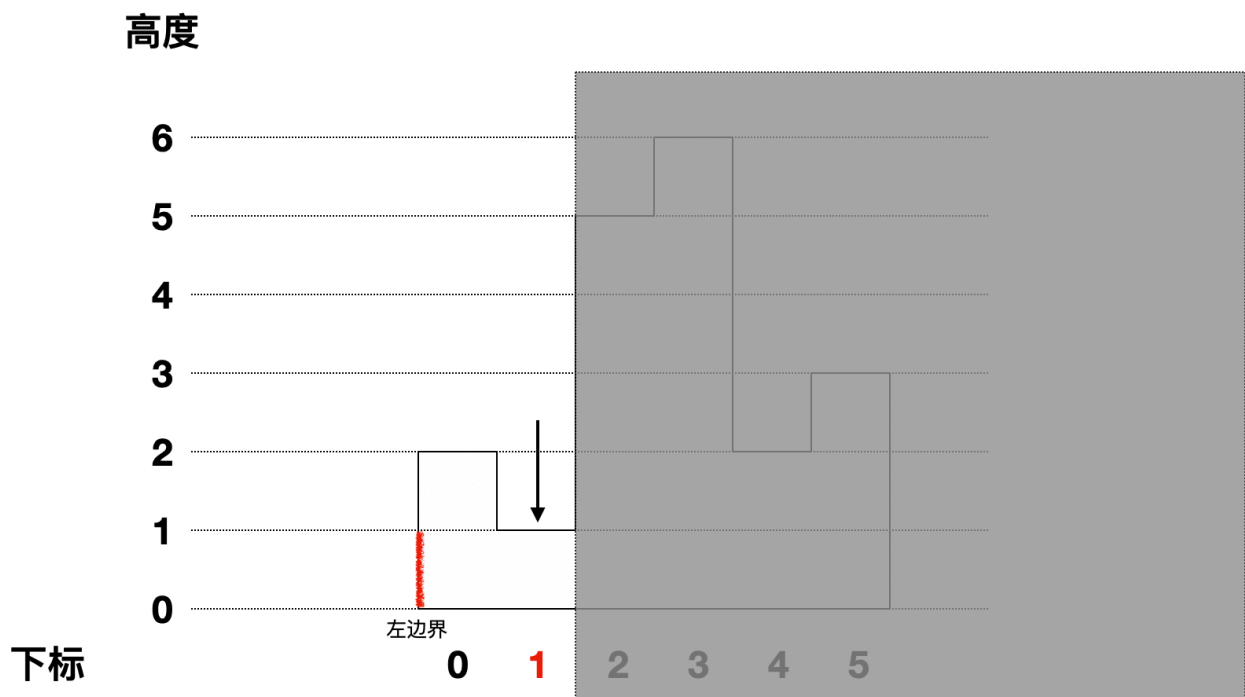
0 1 2 3 4 5

下标是唯一的

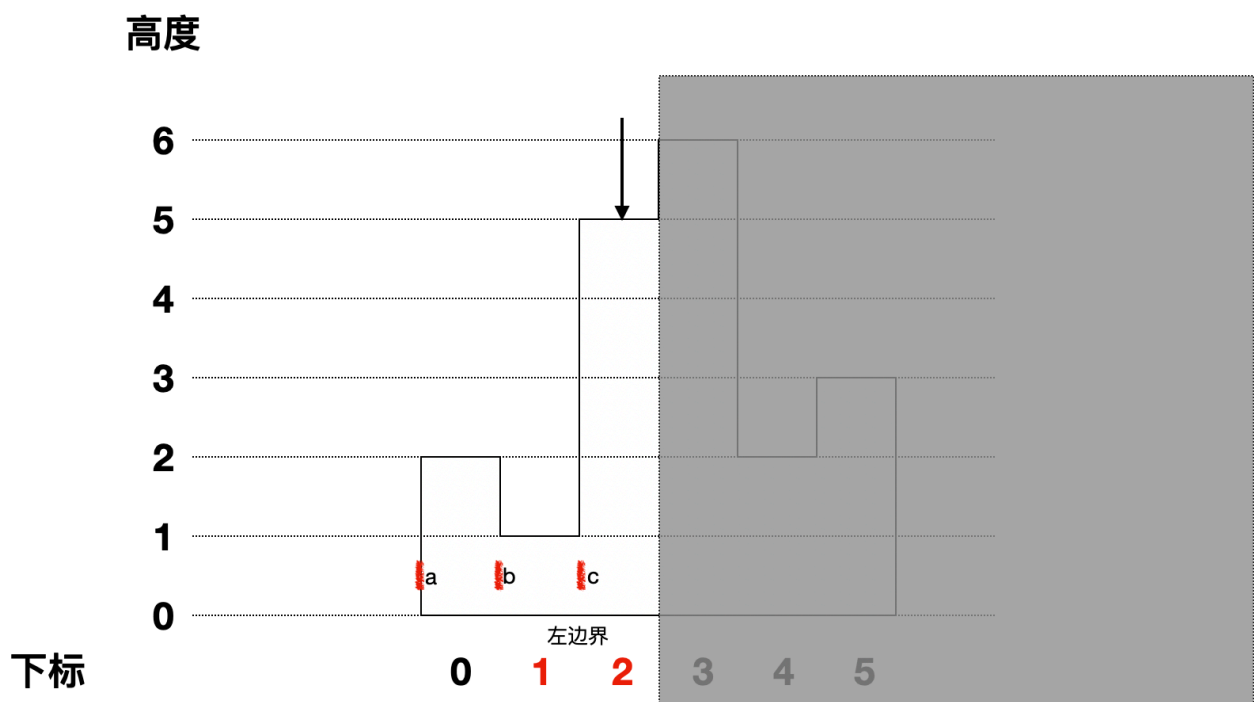
为了方便区分，我们用下标来表示，即下标为 0 的最大矩形面积为 $1 * 2 = 2$ 。

继续往下分析，下标为 1 的最大矩形面积是多少呢？

再次重复一下，矩形的面积是由 高 和 宽 来决定的，高确定了，需要去确定 宽，而宽是由左边界与右边界共同确定的。



此时，下标为 1 的矩形左边界是可以确定下来的，但右边是什么情况一无所知，被阴影笼罩，无法确定右边界，宽度也就无法确定，那先不管，继续前行，让阴影的面积变小。



当来到下标为 2 的位置时，我们先来确定下标 2 位置的左边界，有三个选择，分别是 a、b、c，而我们要选择的边界必须满足**左边第一个比当前柱体小的柱体**，即 c 的位置，否则如果你选 a、b 都会使得下标 2 位置的矩形高度变小。

image-20210321150021475

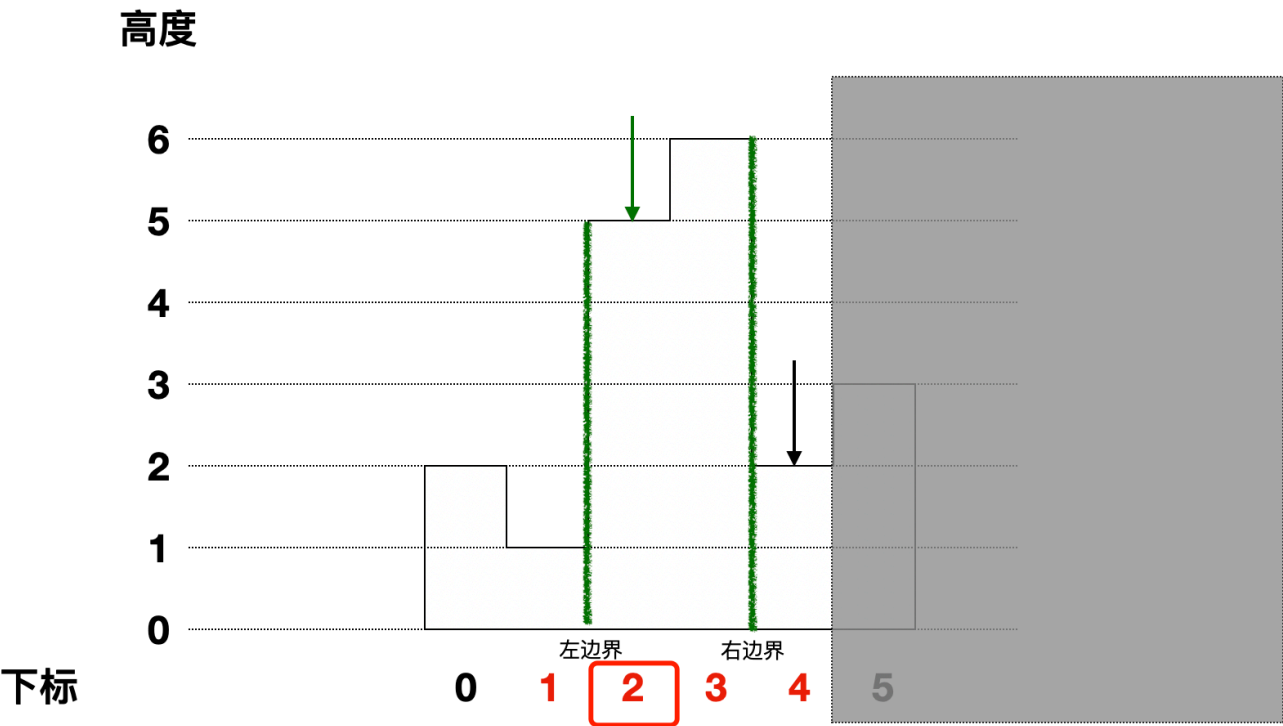
此时，下标为 2 的矩形左边界是可以确定下来的，但**右边是什么情况一无所知**，被阴影笼罩，无法确定右边界，宽度也就无法确定，那先不管，**继续前行**，让阴影的面积变小。

同样的，下标 3 位置的左边界确定了，右边界还无法确定，继续前行，让阴影的面积变小。

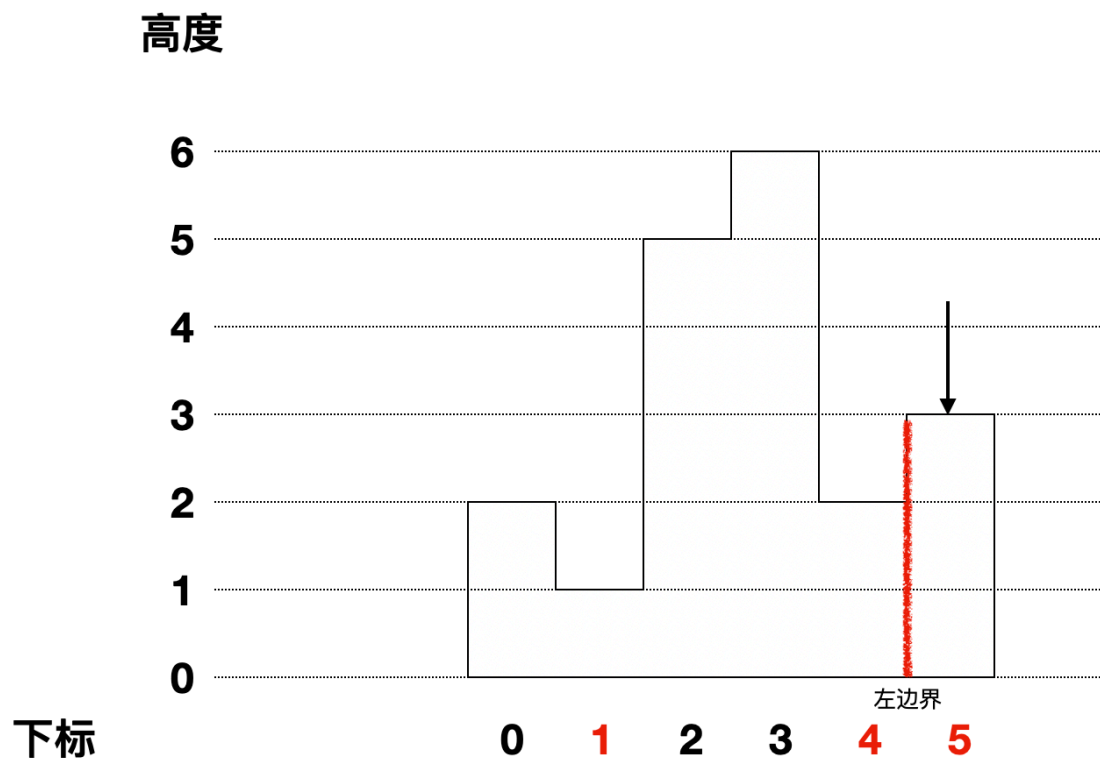
当来到下标 4 的位置时，它的左边界确定了，右边界还无法确定。

不过，下标 3 的右边界可以确定了，是右边下标为 4 位置的边界，此时，它的高度是 6，宽度是 1。

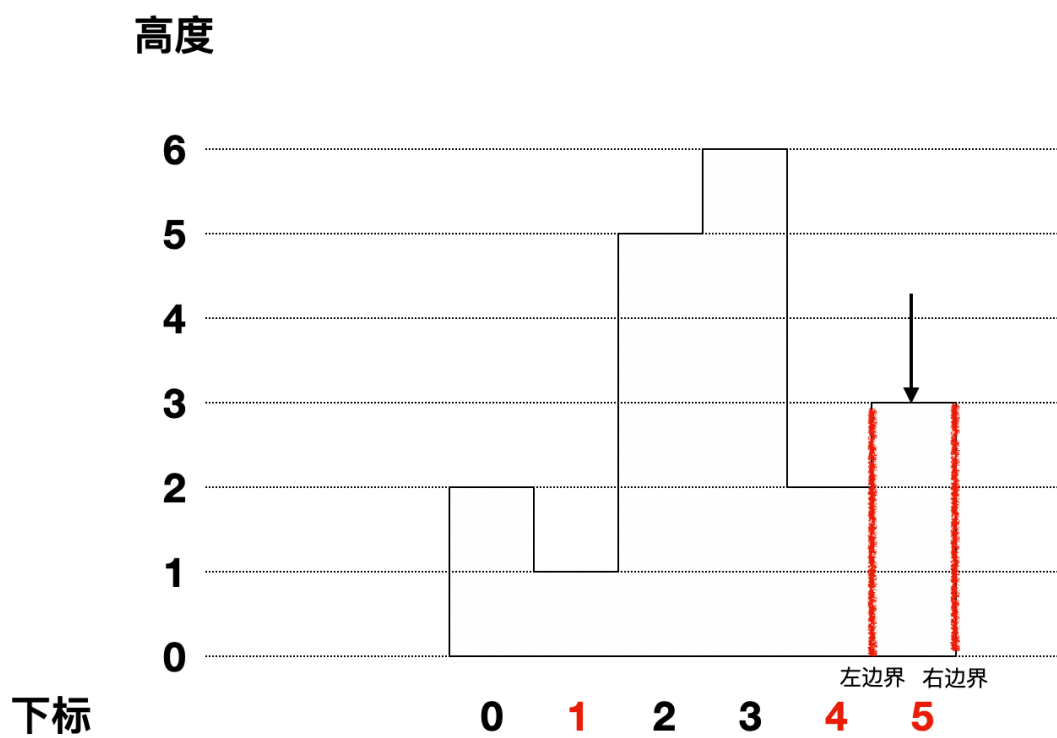
同样的，下标 2 的右边界可以确定了，是右边下标为 4 位置的边界，此时，它的高度是 5，宽度是 2。



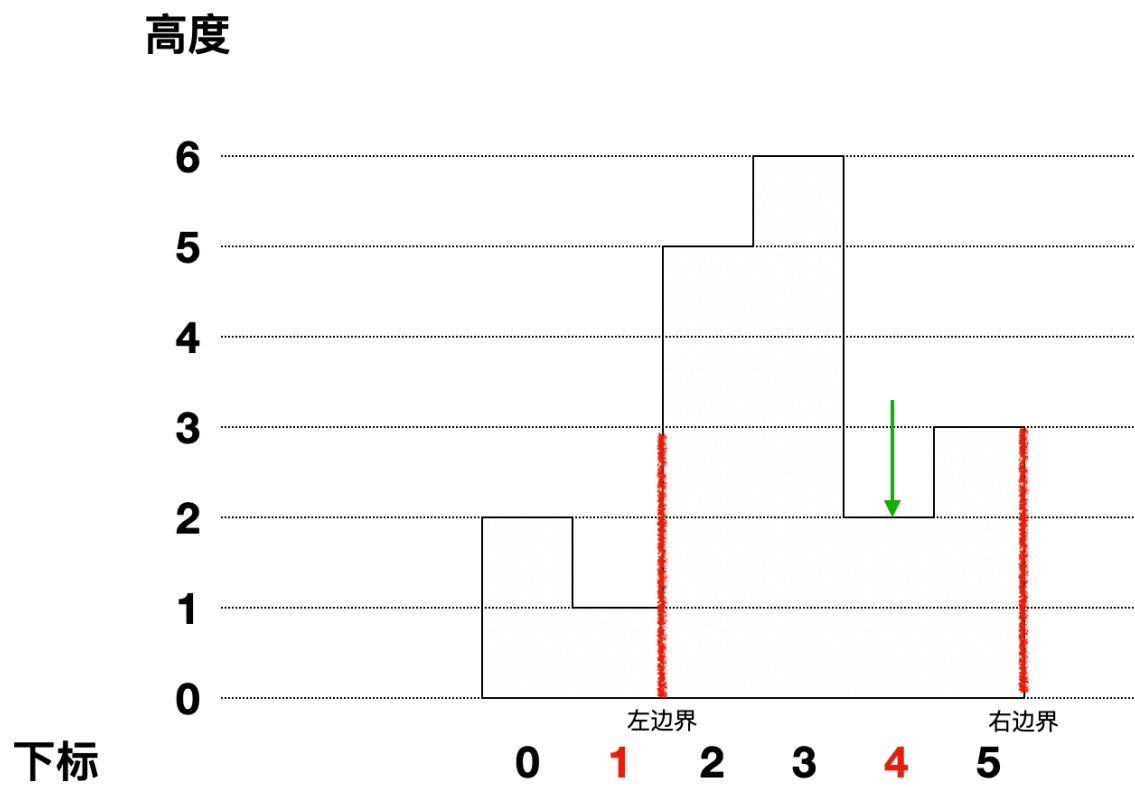
现在，剩下下标为 1、4、5 的边界还没有确定下来，继续前行。



当来到下标为 5 位置的时候，它的左边界和右边界可以确定下来了，此时，它的高度是 3，宽度是 1。

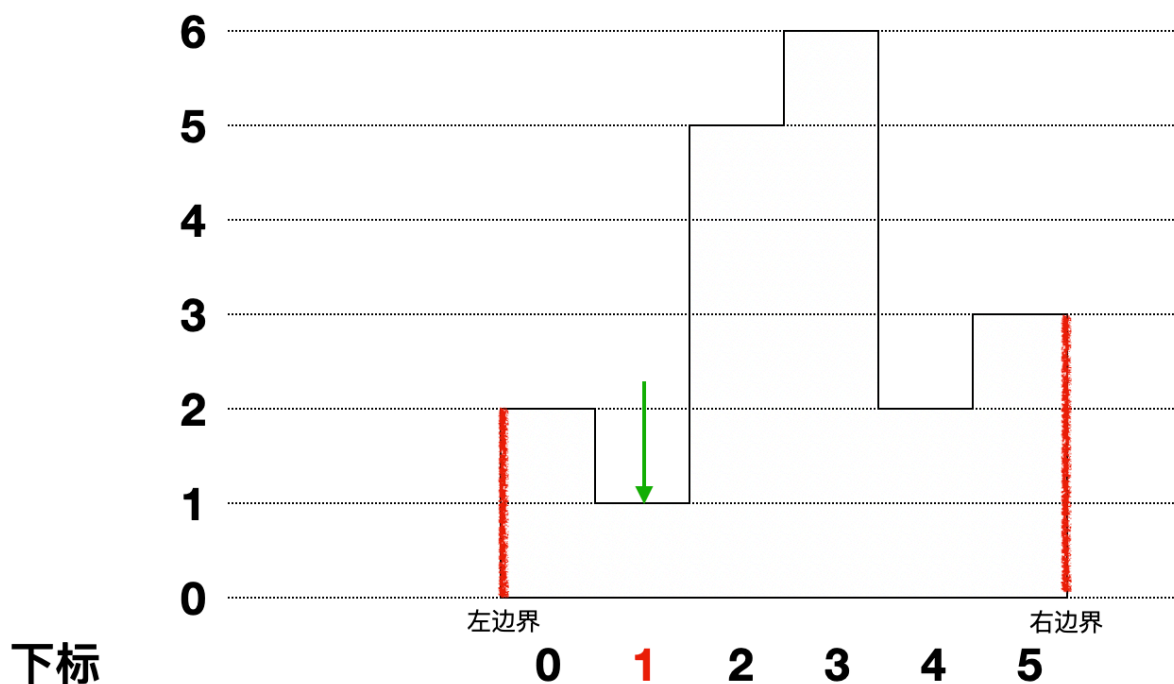


下标为 4 的右边界也可以确定下来，此时，它的高度是 2，宽度是 4。



最后，下标为 1 的右边界也确定下来，此时，它的高度是 1，宽度是 6。

高度



匹配

我们遍历的方式是**从左到右**进行遍历的，而对于右边界的确定分为两种情况，一种是**马上就确定了**，比如下标为 0 和下标为 5 的位置，另外一种是无法立马确定时先**存储**下来，遍历到后面才回过头来确定，比如下标 1、2、3、4 的位置，并且只要是遇到了当前柱形的高度比它上一个柱形的高度严格小的时候，一定可以确定它之前的某些柱形的最大宽度，并且确定的柱形宽度的顺序是**从右边向左边**，比如先确定好下标 3 的右边界才后，才再确定下标 2 的右边界。

很显然，**栈** 这种数据结构符合要求。

“

栈：先进后出，后进先出

”

边界

- 矩形不存在
- 矩形长度为 1

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 柱状图中最大的矩形 (LeetCode 84) : https://leetcode.cn/problems/largest-rectangle-in-histogram/
class Solution {
    public int largestRectangleArea(int[] heights) {

        // 初始化最终结果为0
        int res = 0;

        // 使用单调栈
        Stack<Integer> stack = new Stack<>();

        // 将给定的原数组左右各添加一个元素0，方便处理
        int[] newHeights = new int[heights.length + 2];

        // 左边界为 0
        newHeights[0] = 0;

        // 右边界边界为 0
        newHeights[newHeights.length-1] = 0;

        // 其余不变
        for (int i = 1; i < heights.length + 1; i++) {
            newHeights[i] = heights[i - 1];
        }

        // 整体思路：
        // 对于一个高度，如果能得到向左和向右的边界
        // 那么就能对每个高度求一次面积
        // 遍历所有高度，即可得出最大面积

        // 开始遍历
        for (int i = 0; i < newHeights.length; i++) {
            // 如果栈不为空且当前考察的元素值小于栈顶元素值，
            // 则表示以栈顶元素值为高的矩形面积可以确定
            while (!stack.isEmpty() && newHeights[i] < newHeights[stack.peek()]) {
                // 弹出栈顶元素
                int cur = stack.pop();
                // 获取栈顶元素对应的高
                int curHeight = newHeights[cur];

                // 栈顶元素弹出后，新的栈顶元素就是其左侧边界
                int leftIndex = stack.peek();
                // 右侧边界是当前考察的索引
                int rightIndex = i;
                // 计算矩形宽度
```

```

        int curwidth = rightIndex - leftIndex - 1;

        // 计算面积
        res = Math.max(res, curwidth * curHeight);
    }

    // 当前考察索引入栈
    stack.push(i);
}

// 返回结果
return res;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        // 初始化最终结果为0
        int res = 0;

        // 使用单调栈
        stack<int> stk;

        // 将给定的原数组左右各添加一个元素0，方便处理
        vector<int> newHeights(heights.size() + 2);
        // 左边界为 0
        newHeights[0] = 0 ;

        // 右边界边界为 0
        newHeights[heights.size() - 1] = 0;

        // 其余不变
        for ( int i = 1 ; i < heights.size() + 1 ; i++) {
            newHeights[i] = heights[i - 1];
        }

        // 整体思路：
        // 对于一个高度，如果能得到向左和向右的边界
        // 那么就能对每个高度求一次面积
        // 遍历所有高度，即可得出最大面积

        // 开始遍历
        for (int i = 0 ; i < newHeights.size() ; i++) {
            // 如果栈不为空且当前考察的元素值小于栈顶元素值，
            // 则表示以栈顶元素值为高的矩形面积可以确定

```

```

while (!stk.empty() && newHeights[i] < newHeights[stk.top()]) {
    // 弹出栈顶元素
    int cur = stk.top();

    stk.pop();

    // 获取栈顶元素对应的高
    int curHeight = newHeights[cur];

    // 栈顶元素弹出后，新的栈顶元素就是其左侧边界
    int leftIndex = stk.top();

    // 右侧边界是当前考察的索引

    int rightIndex = i;

    // 计算矩形宽度
    int curwidth = rightIndex - leftIndex - 1;

    // 计算面积
    res = max(res, curHeight * curwidth);

}

// 当前考察索引入栈
stk.push(i);
}

// 返回结果
return res;
}
};

```

3、Python 代码

```

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        # 初始化最终结果为0
        res = 0

        # 使用单调栈
        stack = list()

        # 将给定的原数组左右各添加一个元素0，方便处理
        newHeights = [0] * (len(heights) + 2)

        # 左边界为 0

```

```

newHeights[0] = 0

# 右边界边界为 0
newHeights[len(newHeights)-1] = 0

# 其余不变
for i in range( 1 ,len(heights) + 1 ) :
    newHeights[i] = heights[i - 1]

# 整体思路：
# 对于一个高度，如果能得到向左和向右的边界
# 那么就能对每个高度求一次面积
# 遍历所有高度，即可得出最大面积

# 开始遍历
for i in range(len(newHeights)) :
    # 如果栈不为空且当前考察的元素值小于栈顶元素值，
    # 则表示以栈顶元素值为高的矩形面积可以确定
    while stack and newHeights[i] < newHeights[stack[-1]] :
        # 弹出栈顶元素
        cur = stack[-1]

        stack.pop()

        # 获取栈顶元素对应的高
        curHeight = newHeights[cur]

        # 栈顶元素弹出后，新的栈顶元素就是其左侧边界
        leftIndex = stack[-1]

        # 右侧边界是当前考察的索引
        rightIndex = i
        # 计算矩形宽度
        curwidth = rightIndex - leftIndex - 1

        # 计算面积
        res = max(res, curwidth * curHeight)

    # 当前考察索引入栈
    stack.append(i)

# 返回结果
return res

```

第八天

LeetCode 232、用栈实现队列

一、题目描述

请你仅使用两个栈实现先入先出队列。

队列应当支持一般队列支持的所有操作（push、pop、peek、empty）：

实现 `MyQueue` 类：

- `void push(int x)` 将元素 `x` 推到队列的末尾
- `int pop()` 从队列的开头移除并返回元素
- `int peek()` 返回队列开头的元素
- `boolean empty()` 如果队列为空，返回 `True`；否则，返回 `False`

二、题目解析

入队操作

如果是栈的插入操作，那我们可以把元素都先插入到 `stackIn` 中，也就实现了队列的入队操作。

出队操作

- 当 `stackOut` 中不为空时，直接操作，此时在 `stackOut` 中的栈顶元素是最先进入队列的元素，返回该元素即可；
- 如果 `stackOut` 为空且 `stackIn` 不为空，首先需要把 `stackIn` 中的元素逐个弹出并压入到 `stackOut` 中，然后返回 `stackOut` 的栈顶元素即可。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 用栈实现队列 （ LeetCode 232 ） :https://leetcode-cn.com/problems/implement-queue-using-stacks/
class MyQueue {
    // 首先定义好两个栈

    // 一个栈叫做 stackIn，负责进栈操作，相当于队列 queue 中的入队操作
    Stack<Integer> stackIn;
    // 一个栈叫做 stackOut，负责出栈操作，相当于队列 queue 中的出队操作
    Stack<Integer> stackOut;

    public MyQueue() {
        // 在这个函数中初始化两个栈，传入的参数为空，返回也为空
    }
}
```

```

// 初始化 stackIn
stackIn = new Stack<>();
// 初始化 stackOut
stackOut = new Stack<>();
}

public void push(int x) {
    // 新添加的元素添加到 stackIn 中
    stackIn.push(x);
}

public int pop() {

    // 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
    // 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加
    // 进 stackOut 中
    if (stackOut.isEmpty()) {
        // 通过 while 循环将 stackIn 中的所有元素都取出
        while (!stackIn.isEmpty()) {
            // stackOut 不断的添加 stackIn 的栈顶元素
            stackOut.push(stackIn.pop());
        }
    }
    // 此时, stackIn 已经为空, 直接「移除」 stackOut 的栈顶元素
    return stackOut.pop();
}

public int peek() {

    // peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
    // 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
    // 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加
    // 进 stackOut 中
    if (stackOut.isEmpty()) {
        // 通过 while 循环将 stackIn 中的所有元素都取出
        while (!stackIn.isEmpty()) {
            // stackOut 不断的添加 stackIn 的栈顶元素
            stackOut.push(stackIn.pop());
        }
    }

    // peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
    // 此时, stackIn 已经为空, 直接「返回」 stackOut 的栈顶元素
    return stackOut.peek();
}

```

```

public boolean empty() {
    // 队列是否为空, 判断 stackIn 和 stackOut 是否同时不存在元素
    return stackIn.isEmpty() && stackOut.isEmpty();
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 用栈实现队列 ( LeetCode 232 ):https://leetcode-cn.com/problems/implement-queue-
using-stacks/
class MyQueue {
public:
    // 首先定义好两个栈

    // 一个栈叫做 stackIn, 负责进栈操作, 相当于队列 queue 中的入队操作
    stack<int> stackIn;
    // 一个栈叫做 stackOut, 负责出栈操作, 相当于队列 queue 中的出队操作
    stack<int> stackOut;

    MyQueue() {

    }

    void push(int x) {
        // 新添加的元素添加到 stackIn 中
        stackIn.push(x);
    }

    int pop() {

        // 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
        // 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加
        进 stackOut 中
        if (stackOut.empty()){
            // 通过 while 循环将 stackIn 中的所有元素都取出
            while (!stackIn.empty()) {
                // stackOut 不断的添加 stackIn 的栈顶元素
                int pop = stackIn.top();

                stackIn.pop();

                stackOut.push(pop);
            }
        }
    }
}

```



```

        // 此时, stackIn 已经为空, 直接「移除」 stackOut 的栈顶元素

        int pop = stackOut.top();

        stackOut.pop();

        return pop;
    }

    int peek() {

        // peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
        // 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
        // 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加
        // 进 stackOut 中
        if (stackOut.empty()) {
            // 通过 while 循环将 stackIn 中的所有元素都取出
            while (!stackIn.empty()) {
                // stackOut 不断的添加 stackIn 的栈顶元素
                int pop = stackIn.top();

                stackIn.pop();

                stackOut.push(pop);
            }
        }

        // peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
        // 此时, stackIn 已经为空, 直接「返回」 stackOut 的栈顶元素
        return stackOut.top();
    }

    bool empty() {
        // 队列是否为空, 判断 stackIn 和 stackOut 是否同时不存在元素
        return stackIn.empty() && stackOut.empty();
    }
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者: 程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 用栈实现队列 ( LeetCode 232 ): https://leetcode-cn.com/problems/implement-queue-using-stacks/
class MyQueue:

```

```
def __init__(self):
    # 一个栈叫做 stackIn, 负责进栈操作, 相当于队列 queue 中的入队操作
    self.stackIn = []
    # 一个栈叫做 stackOut, 负责出栈操作, 相当于队列 queue 中的出队操作
    self.stackOut = []

def push(self, x: int) -> None:
    # 新添加的元素添加到 stackIn 中
    self.stackIn.append(x)

def pop(self) -> int:
    # 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
    # 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加进
    # stackOut 中
    if not self.stackOut:
        # 通过 while 循环将 stackIn 中的所有元素都取出
        while self.stackIn:
            # stackOut 不断的添加 stackIn 的栈顶元素
            self.stackOut.append(self.stackIn.pop())
        # 此时, stackIn 已经为空, 直接「移除」 stackOut 的栈顶元素
        return self.stackOut.pop()

def peek(self) -> int:
    # peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
    # 如果 stackOut 为空, 首先需要将 stackIn 中的所有元素添加到 stackOut 中
    # 注意 stackIn 是栈, 栈的性质是先进后出, 后进先出, 所以是不断的将 stackIn 中的栈顶元素添加
    # 进 stackOut 中
    if not self.stackOut:
        # 通过 while 循环将 stackIn 中的所有元素都取出
        while self.stackIn:
            # stackOut 不断的添加 stackIn 的栈顶元素
            self.stackOut.append(self.stackIn.pop())

    # peek 和 pop 的区别在于是返回栈顶元素而非删除栈顶元素
    # 此时, stackIn 已经为空, 直接「返回」 stackOut 的栈顶元素
    return self.stackOut[-1]

def empty(self) -> bool:
    # 队列是否为空, 判断 stackIn 和 stackOut 是否同时不存在元素
    return not self.stackIn and not self.stackOut
```

LeetCode 264、丑数II

一、题目描述

给你一个整数 `n`，请你找出并返回第 `n` 个丑数。

丑数 就是只包含质因数 `2`、`3` 和/或 `5` 的正整数。

示例 1:

输入: `n = 10`

输出: `12`

解释: `[1, 2, 3, 4, 5, 6, 8, 9, 10, 12]` 是由前 10 个丑数组成的序列。

示例 2:

输入: `n = 1`

输出: `1`

解释: `1` 通常被视为丑数。

提示:

- `1 <= n <= 1690`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 丑数II (LeetCode 264) (优先队列解法)：https://leetcode.cn/problems/ugly-number-ii/
class Solution {
    public int nthUglyNumber(int n) {

        // 对任意一个丑数来说，去和 2、3、5 分别相乘，可以得到 3 个丑数
        // 那么对每一个丑数都去和 2 、 3 、 5 分别相乘，把那些结果进行排序即可
        // 因此，2、3、5 就是我们需要处理的质因数
        int[] factors = { 2 , 3 , 5 };

        // 由于一些丑数和 2 、 3 、 5 相乘会出现重复元素的情况
        // 比如丑数 2 和 2 、 3 、 5 相乘得到了 4、【6】、10
        // 而丑数 3 和 2 、 3 、 5 相乘得到了 【6】、9 、 15
        // 其中 【6】重复了，所以需要采取去重操作
        // 由于题目说明 n 最大为 1690，会导致丑数的范围超过了 int 范围，所以使用 long 来保存
        Set<Long> seen = new HashSet<Long>();
```

```

// 使用优先队列来获取每次集合中最小的数字
// 由于题目说明 n 最大为 1690, 会导致丑数的范围超过了 int 范围, 所以使用 long 来保存
PriorityQueue<Long> pq = new PriorityQueue<Long>();

// 集合中第一个数字是 1
// 常量后面跟这种一般是指类型
// 1L 表示 1 是长整型, 如果是 1f 表示是 float 型
seen.add(1L);

// 优先队列里面的元素来源于 seen
pq.offer(1L);

// 开始不断的迭代丑数的值, 直到迭代了 n 次为止
// 第一个丑数是 1
int ugly = 1;

// 开始迭代
for (int i = 0; i < n; i++) {

    // 下一个丑数来源于优先队列的队头元素
    long curr = pq.poll();

    // 本题需要返回 int 型, 所以强转一下
    ugly = (int) curr;

    // 再将获取到的丑数和 2 、 3 、 5 分别相乘
    for (int factor : factors) {

        // 获取到新的丑数
        long next = curr * factor;

        // 经过去重操作后
        if (seen.add(next)) {
            // 把这个丑数加入到优先队列里面
            // 优先队列里面会自动操作, 使得最小的元素位于队头
            pq.offer(next);
        }
    }
}

// 返回结果
return ugly;
}
}

```

2、**C++** 代码

```
class solution {
public:
    int nthUglyNumber(int n) {

        // 对任意一个丑数来说，去和 2、3、5 分别相乘，可以得到 3 个丑数
        // 那么对每一个丑数都去和 2 、 3 、 5 分别相乘，把那些结果进行排序即可
        // 因此，2、3、5 就是我们需要处理的质因数
        vector<int> factors = {2, 3, 5};

        // 由于一些丑数和 2 、 3 、 5 相乘会出现重复元素的情况
        // 比如丑数 2 和 2 、 3 、 5 相乘得到了 4、【6】、10
        // 而丑数 3 和 2 、 3 、 5 相乘得到了 【6】、9 、 15
        // 其中 【6】重复了，所以需要采取去重操作
        // 由于题目说明 n 最大为 1690，会导致丑数的范围超过了 int 范围，所以使用 long 来保存
        unordered_set<long> seen;

        // 使用优先队列来获取每次集合中最小的数字
        // 由于题目说明 n 最大为 1690，会导致丑数的范围超过了 int 范围，所以使用 long 来保存
        priority_queue<long, vector<long>, greater<long>> pq;

        // 集合中第一个数字是 1
        // 常量后面跟这种一般是指类型
        // 1L 表示 1 是长整型，如果是 1f 表示是 float 型
        seen.insert(1L);

        // 优先队列里面的元素来源于 seen
        pq.push(1L);

        // 开始不断的迭代丑数的值，直到迭代了 n 次为止
        // 第一个丑数是 1
        int ugly = 1;

        // 开始迭代
        for (int i = 0; i < n; i++) {

            // 下一个丑数来源于优先队列的队头元素
            long curr = pq.top();

            pq.pop();

            // 本题需要返回 int 型，所以强转一下
            ugly = (int) curr;

            // 再将获取到的丑数和 2 、 3 、 5 分别相乘
            for (int factor : factors) {

                // 获取到新的丑数
```

```

        long next = curr * factor;

        // 经过去重操作后
        if (!seen.count(next)) {
            // 把这个丑数加入到优先队列里面
            // 优先队列里面会自动操作，使得最小的元素位于队头
            seen.insert(next);
            pq.push(next);
        }
    }
}

// 返回结果
return ugly;
}
};

```

3、Python 代码

```

class Solution:
    def nthUglyNumber(self, n: int) -> int:
        # 对任意一个丑数来说，去和 2、3、5 分别相乘，可以得到 3 个丑数
        # 那么对每一个丑数都去和 2、3、5 分别相乘，把那些结果进行排序即可
        # 因此，2、3、5 就是我们需要处理的质因数
        factors = [2, 3, 5]

        # 由于一些丑数和 2、3、5 相乘会出现重复元素的情况
        # 比如丑数 2 和 2、3、5 相乘得到了 4、【6】、10
        # 而丑数 3 和 2、3、5 相乘得到了 【6】、9、15
        # 其中 【6】重复了，所以需要采取去重操作
        # 由于题目说明 n 最大为 1690，会导致丑数的范围超过了 int 范围，所以使用 long 来保存

        # 使用优先队列来获取每次集合中最小的数字
        # 由于题目说明 n 最大为 1690，会导致丑数的范围超过了 int 范围，所以使用 long 来保存

        # 集合中第一个数字是 1
        # 常量后面跟这种一般是指类型
        # 1L 表示 1 是长整型，如果是 1f 表示是 float 型
        seen = {1}

        # 优先队列里面的元素来源于 seen
        pq = [1]

        # 开始不断的迭代丑数的值，直到迭代了 n 次为止
        # 第一个丑数是 1

```

```

# 开始迭代
for i in range(n - 1):

    # 下一个丑数来源于优先队列的队头元素
    curr = heapq.heappop(pq)

    # 本题需要返回 int 型, 所以强转一下
    for factor in factors:
        if (nxt := curr * factor) not in seen:
            seen.add(nxt)
            heapq.heappush(pq, nxt)

# 返回结果
return heapq.heappop(pq)

```

LeetCode 373、查找和最小的 K 对数字

一、题目描述

给定两个以 **升序排列** 的整数数组 `nums1` 和 `nums2` , 以及一个整数 `k` 。

定义一对值 `(u,v)` , 其中第一个元素来自 `nums1` , 第二个元素来自 `nums2` 。

请找到和最小的 `k` 个数对 `(u1,v1)` , `(u2,v2)` ... `(uk,vk)` 。

示例 1:

输入: `nums1 = [1,7,11]` , `nums2 = [2,4,6]` , `k = 3`

输出: `[1,2]` , `[1,4]` , `[1,6]`

解释: 返回序列中的前 3 对数:

`[1,2]` , `[1,4]` , `[1,6]` , `[7,2]` , `[7,4]` , `[11,2]` , `[7,6]` , `[11,4]` , `[11,6]`

示例 2:

输入: `nums1 = [1,1,2]` , `nums2 = [1,2,3]` , `k = 2`

输出: `[1,1]` , `[1,1]`

解释: 返回序列中的前 2 对数:

`[1,1]` , `[1,1]` , `[1,2]` , `[2,1]` , `[1,2]` , `[2,2]` , `[1,3]` , `[1,3]` , `[2,3]`

示例 3:

输入: `nums1 = [1,2]` , `nums2 = [3]` , `k = 3`

输出: `[1,3]` , `[2,3]`

解释: 也可能序列中所有的数对都被返回: `[1,3]` , `[2,3]`

提示:

- `1 <= nums1.length, nums2.length <= 105`
- `-109 <= nums1[i], nums2[i] <= 109`
- `nums1` 和 `nums2` 均为升序排列
- `1 <= k <= 1000`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 查找和最小的 k 对数字 (LeetCode 373) : https://leetcode.cn/problems/find-k-pairs-with-smallest-sums/
class Solution {
    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        // 队列是遵循先进先出 (First-In-First-Out) 模式的，但有时需要在队列中基于优先级处理对象。
        // PriorityQueue 和队列 Queue 的区别在于，它的出队顺序与元素的优先级有关
        // 对 PriorityQueue 调用 remove() 或 poll() 方法，返回的总是优先级最高的元素
        // Java 中 PriorityQueue 通过二叉小顶堆实现
        // PriorityQueue 默认是一个【小顶堆】，可以通过传入自定义的 Comparator 函数来实现【大顶堆】

        // PriorityQueue 里面的每个元素是一个数组，这个数组包含了两个元素，表示 nums1 的索引和 nums2 的索引
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> nums1[a[0]] +
            nums2[a[1]] - (nums1[b[0]] + nums2[b[1]]));

        // 由于 nums1 和 nums2 都是升序序列，最小的组合就是 nums1[0] + nums2[0]
        // 接下来的组合需要经过比较
        // 和谁比较呢
        // 可以把 nums2 中的第一个元素和 nums1 中的每个元素都先组合一下，存放到优先队列当中
        for (int i = 0; i < nums1.length; i++) {
            // 优先队列存放的是索引
            pq.offer(new int[] {i, 0});
        }

        // 结果数组
        List<List<Integer>> ans = new ArrayList<>();

        // 访问优先队列，不断的弹出队头元素，最多弹 k 次就行
        while (k-- > 0 && !pq.isEmpty()) {
            // 取出队头元素
            // pos 是一个数组，包含两个元素，表示 nums1 的索引和 nums2 的索引
        }
    }
}
```



```

        int[] pos = pq.poll();

        int index1 = pos[0];

        int index2 = pos[1];

        // 利用这两个索引获取到对应的元素进行组合
        ans.add(Arrays.asList(nums1[index1], nums2[index2]));

        // 获取下一个索引
        index2++;

        // 判断 nums2 中是否还有元素
        if (index2 < nums2.length) {
            // 如果有，那么又是一个新的组合
            int[] newPos = { index1 ,index2 };

            // 加入到优先队列，在内部会自动进行排序操作
            pq.offer(newPos);
        }
    }

    // 返回结果
    return ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>& nums2, int k) {

        auto cmp = [&nums1, &nums2](const pair<int, int> & a, const pair<int, int> & b)
        {
            return nums1[a.first] + nums2[a.second] > nums1[b.first] + nums2[b.second];
        };

        int m = nums1.size();

        int n = nums2.size();
        // 结果数组
        vector<vector<int>> ans;

        // 队列是遵循先进先出 (First-In-First-Out) 模式的，但有时需要在队列中基于优先级处理对象。
        priority_queue<pair<int, int>, vector<pair<int, int>>, decltype(cmp)> pq(cmp);

        // 由于 nums1 和 nums2 都是升序序列，最小的组合就是 nums1[0] + nums2[0]
    }
}

```

```

// 接下来的组合需要经过比较
// 和谁比较呢
// 可以把 nums2 中的第一个元素和 nums1 中的每个元素都先组合一下，存放到优先队列当中
for (int i = 0; i < min(k, m); i++) {
    pq.emplace(i, 0);
}

// 访问优先队列，不断的弹出队头元素，最多弹 k 次就行
while (k-- > 0 && !pq.empty()) {
    // 取出队头元素
    auto [x, y] = pq.top();
    pq.pop();
    // 利用这两个索引获取到对应的元素进行组合
    ans.emplace_back(initializer_list<int>{nums1[x], nums2[y]});

    // 获取下一个索引
    if (y + 1 < n) {
        pq.emplace(x, y + 1);
    }
}
// 返回结果
return ans;
}
};

```

3、Python 代码

```

class Solution:
    def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
        # PriorityQueue 里面的每个元素是一个数组，这个数组包含了两个元素，表示 nums1 的索引和 nums2 的索引
        # 由于 nums1 和 nums2 都是升序序列，最小的组合就是 nums1[0] + nums2[0]
        # 接下来的组合需要经过比较
        # 和谁比较呢
        # 可以把 nums2 中的第一个元素和 nums1 中的每个元素都先组合一下，存放到优先队列当中

        m, n = len(nums1), len(nums2)
        pq = [(nums1[i] + nums2[0], i, 0) for i in range(min(k, m))]

        # 结果数组
        ans = []

        # 访问优先队列，不断的弹出队头元素，最多弹 k 次就行
        while pq and len(ans) < k:

            # 取出队头元素
            # pos 是一个数组，包含两个元素，表示 nums1 的索引和 nums2 的索引
            _, index1, index2 = heappop(pq)

```

```
# 利用这两个索引获取到对应的元素进行组合
ans.append([nums1[index1], nums2[index2]])

# 获取下一个索引
index2 += 1

# 判断 nums2 中是否还有元素
if index2 < n :
    # 如果有，那么又是一个新的组合

    # 加入到优先队列，在内部会自动进行排序操作
    heappush(pq, (nums1[index1] + nums2[index2], index1, index2))

# 返回结果
return ans
```

LeetCode 641、设计循环双端队列

一、题目描述

设计实现双端队列。你的实现需要支持以下操作：

- MyCircularDeque(k)：构造函数,双端队列的大小为k。
- insertFront()：将一个元素添加到双端队列头部。如果操作成功返回 true。
- insertLast()：将一个元素添加到双端队列尾部。如果操作成功返回 true。
- deleteFront()：从双端队列头部删除一个元素。如果操作成功返回 true。
- deleteLast()：从双端队列尾部删除一个元素。如果操作成功返回 true。
- getFront()：从双端队列头部获得一个元素。如果双端队列为空，返回 -1。
- getRear()：获得双端队列的最后一个元素。如果双端队列为空，返回 -1。
- isEmpty()：检查双端队列是否为空。
- isFull()：检查双端队列是否满了。

二、题目解析

- 1、初始化容量 $\text{capacity} = k + 1$ ，并建立大小为 capacity 的数组 arr；
- 2、设置两个指针 front 和 rear
- 3、front 指向当前队首元素的位置
- 4、rear 指向当前队尾元素的下一个位置，即下一个入队元素的位置
- 5、判空条件：front == rear
- 6、判满条件：(rear + 1) % capacity == front，也就是 rear 再后移一位就碰到了 front，因为是循环存储的，所以 rear + 1 可能会比 capacity 大，取模之后再和 front 比较

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class MyCircularDeque {

    // capacity 表示数组的容量，它的大小为 k + 1
    private int capacity ;

    // front 表示队头的位置
    private int front;

    // rear 表示队列尾部的下一个位置
    private int rear;

    // 这里使用数组来设计循环双端队列
    private int[] arr;

    public MyCircularDeque(int k) {

        // 需要空出一个额外的空间
        // 当队列中的元素有 k 个时，代表队列已满
        capacity = k + 1;

        // 这里使用数组来设计循环双端队列，数组的容量为 capacity
        // 即在数组中会空出一个额外的空间来不存储任何元素
        // 因为这样才能合理的区分出【队列为空】和【队列已满】这两种情况
        arr = new int[capacity];

        // 初始化队头指针
        front = 0;

        // 初始化队尾指针
        rear = 0;
    }

    // 在双端队列的队头添加元素 value
    public boolean insertFront(int value) {
        // 首先判断队列是否已满，没有其它的剩余空间
        // 如果队列已满，那么元素无法添加进去
        if(isFull()){
            // 返回 false
            return false;
        }
    }
```

```

        // front 向前移动
        // 由于 front 会移动到数组的最前面，如果再向前移动会出界
        // 那么这个时候就需要将 front 挪到数组结尾的位置
        // 可以通过取余运算来实现循环移动的这个操作
        front = ( front - 1 + capacity) % capacity;

        // 添加元素 value 到 front 这个位置
        arr[front] = value;

        // 添加元素成功，返回 true
        return true;
    }

    // 在双端队列的队尾添加元素 value
    public boolean insertLast(int value) {

        // 首先判断队列是否已满，没有其它的剩余空间
        // 如果队列已满，那么元素无法添加进去
        if(isFull()){
            // 返回 false
            return false;
        }

        // 添加元素 value 到 rear 这个位置
        arr[rear] = value;

        // rear 向后移动
        // 由于 rear 会移动到数组的最尾部，如果再向后移动会出界
        // 那么这个时候就需要将 rear 挪到数组开头的位置
        // 可以通过取余运算来实现循环移动的这个操作
        rear = ( rear + 1 ) % capacity;

        // 添加元素成功，返回 true
        return true;
    }

    // 删除双端队列的队头元素
    public boolean deleteFront() {
        // 首先判断队列是否为空
        if(isEmpty()){
            // 队列为空的话那就没有元素可删，返回 false
            return false;
        }

        // 队头指针向后移动
        front = ( front + 1 ) % capacity;
    }

```

```

        // 删除元素成功, 返回 true
        return true;
    }

    // 删除双端队列的队尾元素
    public boolean deleteLast() {
        // 首先判断队列是否为空
        if(isEmpty()){
            // 队列为空的话那就没有元素可删, 返回 false
            return false;
        }

        // 队尾指针向前移动
        rear = ( rear - 1 + capacity ) % capacity;

        // 删除元素成功, 返回 true
        return true;
    }

    public int getFront() {
        // 首先判断队列是否为空
        if(isEmpty()){
            // 队列为空的话那就没有元素可选, 根据题目要求返回 -1
            return -1;
        }

        // 否则返回 front 指向的元素就行
        return arr[front];
    }

    public int getRear() {
        // 首先判断队列是否为空
        if(isEmpty()){
            // 队列为空的话那就没有元素可选, 根据题目要求返回 -1
            return -1;
        }

        // 否则返回 rear 位置的前一个元素就行
        return arr[(rear - 1 + capacity) % capacity];
    }

    // 判断队列是否为空
    public boolean isEmpty() {
        // front 和 rear 指向同一个位置时, 队列为空
        return front == rear;
    }
}

```

```

public boolean isFull() {
    // 当队尾指针向后移动一个位置后，指向了队头时，那么说明队列已满
    return ( rear + 1 ) % capacity == front;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class MyCircularDeque {

public:
    // capacity 表示数组的容量，它的大小为 k + 1
    int capacity;

    // front 表示队头的位置
    int front;

    // rear 表示队列尾部的下一个位置
    int rear;

    // 这里使用数组来设计循环双端队列
    vector<int> arr;

    MyCircularDeque(int k) {

        // 需要空出一个额外的空间
        // 当队列中的元素有 k 个时，代表队列已满
        capacity = k + 1;

        // 这里使用数组来设计循环双端队列，数组的容量为 capacity
        // 即在数组中会空出一个额外的空间来不存储任何元素
        // 因为这样才能合理的区分出【队列为空】和【队列已满】这两种情况
        arr = vector<int>(capacity);

        // 初始化队头指针
        front = 0;

        // 初始化队尾指针
        rear = 0;
    }

    // 在双端队列的队头添加元素 value
    bool insertFront(int value) {
        // 首先判断队列是否已满，没有其它的剩余空间
        // 如果队列已满，那么元素无法添加进去
    }
}

```

```

    if(isFull()){
        // 返回 false
        return false;
    }

    // front 向前移动
    // 由于 front 会移动到数组的最前面，如果再向前移动会出界
    // 那么这个时候就需要将 front 挪到数组结尾的位置
    // 可以通过取余运算来实现循环移动的这个操作
    front = ( front - 1 + capacity ) % capacity;

    // 添加元素 value 到 front 这个位置
    arr[front] = value;

    // 添加元素成功，返回 true
    return true;
}

// 在双端队列的队尾添加元素 value
bool insertLast(int value) {

    // 首先判断队列是否已满，没有其它的剩余空间
    // 如果队列已满，那么元素无法添加进去
    if(isFull()){
        // 返回 false
        return false;
    }

    // 添加元素 value 到 rear 这个位置
    arr[rear] = value;

    // rear 向后移动
    // 由于 rear 会移动到数组的最尾部，如果再向后移动会出界
    // 那么这个时候就需要将 rear 挪到数组开头的位置
    // 可以通过取余运算来实现循环移动的这个操作
    rear = (rear + 1) % capacity;

    // 添加元素成功，返回 true
    return true;
}

// 删除双端队列的队头元素
bool deleteFront() {
    // 首先判断队列是否为空
    if(isEmpty()){
        // 队列为空的话那就没有元素可删，返回 false
        return false;
    }

```



```

    }

    // 队头指针向后移动
    front = (front + 1) % capacity;

    // 删除元素成功, 返回 true
    return true;
}

// 删除双端队列的队尾元素
bool deleteLast() {
    // 首先判断队列是否为空
    if(isEmpty()){
        // 队列为空的话那就没有元素可删, 返回 false
        return false;
    }

    // 队尾指针向前移动
    rear = (rear - 1 + capacity) % capacity;

    // 删除元素成功, 返回 true
    return true;
}

int getFront() {
    // 首先判断队列是否为空
    if(isEmpty()){
        // 队列为空的话那就没有元素可选, 根据题目要求返回 -1
        return - 1;
    }

    // 否则返回 front 指向的元素就行
    return arr[front];
}

int getRear() {
    // 首先判断队列是否为空
    if(isEmpty()){
        // 队列为空的话那就没有元素可选, 根据题目要求返回 -1
        return -1;
    }

    // 否则返回 rear 位置的前一个元素就行
    return arr[(rear - 1 + capacity) % capacity];
}

// 判断队列是否为空

```

```

bool isEmpty() {
    // front 和 rear 指向同一个位置时，队列为空
    return front == rear;
}

bool isFull() {
    // 当队尾指针向后移动一个位置后，指向了队头时，那么说明队列已满
    return (rear + 1) % capacity == front;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
class MyCircularDeque:

    # capacity 表示数组的容量，它的大小为 k + 1

    # front 表示队头的位置

    # rear 表示队列尾部的下一个位置

    # 这里使用数组来设计循环双端队列

    def __init__(self, k: int):

        # 需要空出一个额外的空间
        # 当队列中的元素有 k 个时，代表队列已满
        self.capacity = k + 1

        # 这里使用数组来设计循环双端队列，数组的容量为 capacity
        # 即在数组中会空出一个额外的空间来不存储任何元素
        # 因为这样才能合理的区分出【队列为空】和【队列已满】这两种情况
        self.arr = [0] * self.capacity

        # 初始化队头指针
        self.front = 0

        # 初始化队尾指针
        self.rear = 0

    # 在双端队列的队头添加元素 value
    def insertFront(self, value: int) -> bool:
        # 首先判断队列是否已满，没有其它的剩余空间
        # 如果队列已满，那么元素无法添加进去
        if self.isFull() :

```

```

        # 返回 false
        return False

    # front 向前移动
    # 由于 front 会移动到数组的最前面, 如果再向前移动会出界
    # 那么这个时候就需要将 front 挪到数组结尾的位置
    # 可以通过取余运算来实现循环移动的这个操作
    self.front = ( self.front - 1 + self.capacity ) % self.capacity

    # 添加元素 value 到 front 这个位置
    self.arr[self.front] = value

    # 添加元素成功, 返回 true
    return True

# 在双端队列的队尾添加元素 value
def insertLast(self, value: int) -> bool:

    # 首先判断队列是否已满, 没有其它的剩余空间
    # 如果队列已满, 那么元素无法添加进去
    if self.isFull() :
        # 返回 false
        return False

    # 添加元素 value 到 rear 这个位置
    self.arr[self.rear] = value

    # rear 向后移动
    # 由于 rear 会移动到数组的最尾部, 如果再向后移动会出界
    # 那么这个时候就需要将 rear 挪到数组开头的位置
    # 可以通过取余运算来实现循环移动的这个操作
    self.rear = (self.rear + 1) % self.capacity

    # 添加元素成功, 返回 true
    return True

# 删除双端队列的队头元素
def deleteFront(self) -> bool:
    # 首先判断队列是否为空
    if self.isEmpty():
        # 队列为空的话就没有元素可删, 返回 false
        return False

    # 队头指针向后移动
    self.front = (self.front + 1) % self.capacity

    # 删除元素成功, 返回 true
    return True

```

```

# 删除双端队列的队尾元素
def deleteLast(self) -> bool:
    # 首先判断队列是否为空
    if self.isEmpty():
        # 队列为空的话就没有元素可删，返回 false
        return False

    # 队尾指针向前移动
    self.rear = (self.rear - 1 + self.capacity) % self.capacity

    # 删除元素成功，返回 true
    return True

def getFront(self) -> int:
    if self.isEmpty():
        # 队列为空的话就没有元素可选，根据题目要求返回 -1
        return -1

    # 否则返回 front 指向的元素就行
    return self.arr[self.front]

def getRear(self) -> int:
    # 首先判断队列是否为空
    if self.isEmpty():
        # 队列为空的话就没有元素可选，根据题目要求返回 -1
        return -1

    # 否则返回 rear 位置的前一个元素就行
    return self.arr[(self.rear - 1 + self.capacity) % self.capacity]

# 判断队列是否为空
def isEmpty(self) -> bool:
    # front 和 rear 指向同一个位置时，队列为空
    return self.front == self.rear

def isFull(self) -> bool:
    # 当队尾指针向后移动一个位置后，指向了队头时，那么说明队列已满
    return (self.rear + 1) % self.capacity == self.front

```

四、复杂度分析

时间复杂度： $O(1)$

空间复杂度： $O(k)$

第九天

LeetCode 1、两数之和

一、题目描述

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值 `target`** 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素在答案里不能重复出现。

你可以按任意顺序返回答案。

例 1：

```
输入：nums = [2,7,11,15], target = 9
输出：[0,1]
解释：因为 nums[0] + nums[1] == 9 ，返回 [0, 1] 。
```

示例 2：

```
输入：nums = [3,2,4], target = 6
输出：[1,2]
```

示例 3：

```
输入：nums = [3,3], target = 6
输出：[0,1]
```

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 两数之和 (LeetCode 1) :https://leetcode-cn.com/problems/two-sum/
class Solution {
    public int[] twoSum(int[] nums, int target) {
```

```

// 首先构建一个哈希表，用来存放数组的元素值以及索引值
// 其中 key 是数组中的元素值
// value 为数组中元素值的索引
Map<Integer,Integer> map = new HashMap<>();

// 接下来，遍历整个数组
for(int i = 0; i < nums.length; i++) {

    // 目标值为 target，将 target 与 nums[i] 求差
    // 获取与 nums[i] 配对的那个数 anotherNum
    int anotherNum = target - nums[i];

    // 判断哈希表中是否存在那个与 nums[i] 配对的数 anotherNum
    if (map.containsKey(anotherNum)) {

        // 由于哈希表中所有 key 都是来自于数组中，
        // 所以，如果发现哈希表存在那个与 nums[i] 配对的数 anotherNum
        // 也就说明数组中存在一个数，可以和 nums[i] 相加为 target
        // 此时， anotherNum 这个 key 对应的 value 为这个数在数组中的索引
        // 所以，返回 map.get(anotherNum) 和 i 就是这两个值的下标
        return new int[]{map.get(anotherNum), i};
    }else{
        // 如果发现哈希表中目前不存在那个与 nums[i] 配对的数 anotherNum
        // 那么就把此时观察的数 nums[i] 和它的索引存放到哈希表中
        // 等待后面的数能和它配对为 target
        map.put(nums[i], i);
    }

}

// 如果遍历完整个数组都找不到和为 target 的两个数，返回 0
return new int[0];
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 两数之和 (LeetCode 1) :https://leetcode-cn.com/problems/two-sum/
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        // 首先构建一个哈希表，用来存放数组的元素值以及索引值
        // 其中 key 是数组中的元素值
        // value 为数组中元素值的索引
    }
};

```

```

unordered_map< int , int > map;

// 接下来，遍历整个数组
for(int i = 0; i < nums.size(); i++) {

    // 目标值为 target，将 target 与 nums[i] 求差
    // 获取与 nums[i] 配对的那个数 anotherNum
    int anotherNum = target - nums[i];

    // 判断哈希表中是否存在那个与 nums[i] 配对的数 anotherNum
    auto it = map.find(anotherNum);
    if (it != map.end()) {
        // 由于哈希表中所有 key 都是来自于数组中，
        // 所以，如果发现哈希表存在那个与 nums[i] 配对的数 anotherNum
        // 也就说明数组中存在一个数，可以和 nums[i] 相加为 target
        // 此时， anotherNum 这个 key 对应的 value 为这个数在数组中的索引
        // 所以，返回 map.get(anotherNum) 和 i 就是这两个值的下标
        return {map[anotherNum], i};
    }else{
        // 如果发现哈希表中目前不存在那个与 nums[i] 配对的数 anotherNum
        // 那么就把此时观察的数 nums[i] 和它的索引存放到哈希表中
        // 等待后面的数能和它配对为 target
        map[nums[i]] = i;
    }

}

// 如果遍历完整个数组都找不到和为 target 的两个数，返回 0
return {};
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 两数之和 (LeetCode 1) :https://leetcode-cn.com/problems/two-sum/
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        # 首先构建一个哈希表，用来存放数组的元素值以及索引值
        # 其中 key 是数组中的元素值
        # value 为数组中元素值的索引
        map = dict()

        # 接下来，遍历整个数组
        for i, num in enumerate(nums):
            # 目标值为 target，将 target 与 nums[i] 求差

```

```

# 获取与 nums[i] 配对的那个数 anotherNum
anotherNum = target - num

# 判断哈希表中是否存在那个与 nums[i] 配对的数 anotherNum
if anotherNum in map :

    # 由于哈希表中所有 key 都是来自于数组中，
    # 所以，如果发现哈希表存在那个与 nums[i] 配对的数 anotherNum
    # 也就说明数组中存在一个数，可以和 nums[i] 相加为 target
    # 此时， anotherNum 这个 key 对应的 value 为这个数在数组中的索引
    # 所以，返回 map.get(anotherNum) 和 i 就是这两个值的下标
    return [ map[ target - num ] , i ]
else:
    # 如果发现哈希表中目前不存在那个与 nums[i] 配对的数 anotherNum
    # 那么就把此时观察的数 nums[i] 和它的索引存放到哈希表中
    # 等待后面的数能和它配对为 target
    map[nums[i]] = i

# 如果遍历完整个数组都找不到和为 target 的两个数，返回 0
return []

```

四、复杂度分析

时间复杂度：O(N)，其中 N 是数组中的元素数量。对于每一个元素 x，我们可以 O(1) 地寻找 target - x。

空间复杂度：O(N)，其中 N 是数组中的元素数量。主要为哈希表的开销。

LeetCode 205、同构字符串

一、题目描述

给定两个字符串 `s` 和 `t`，判断它们是否是同构的。

如果 `s` 中的字符可以按某种映射关系替换得到 `t`，那么这两个字符串是同构的。

每个出现的字符都应当映射到另一个字符，同时不改变字符的顺序。不同字符不能映射到同一个字符上，相同字符只能映射到同一个字符上，字符可以映射到自己本身。

示例 1:

输入：s = "egg", t = "add" 输出：true

示例 2:

输入：s = "foo", t = "bar" 输出：false

示例 3:

输入：s = "paper", t = "title" 输出：true

提示：

- `1 <= s.length <= 5 * 104`

- `t.length == s.length`
- `s` 和 `t` 由任意有效的 ASCII 字符组成

二、题目解析

三、参考代码

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        // 设置一个哈希映射用来存储字符串s中的元素
        HashMap<Character, Character> map1 = new HashMap<>();

        // 设置一个哈希映射用来存储字符串t中的元素
        HashMap<Character, Character> map2 = new HashMap<>();

        // 由于t.length == s.length
        // 按顺序访问s和t中对应的元素
        for (int i = 0; i < s.length(); i++) {
            char charS = s.charAt(i);
            char charT = t.charAt(i);

            // 1. 如果元素charS已经存在于map1中
            // 并且对应的值与当前元素charT不相同，返回false
            if (map1.containsKey(charS) && map1.get(charS) != charT) {
                return false;
            }

            // 2. 如果元素charT已经存在于map2中
            // 并且对应的值与当前元素charS不相同，返回false
            if (map2.containsKey(charT) && map2.get(charT) != charS) {
                return false;
            }

            // 3. 如果元素charS不存在于map1中，将其放入哈希映射中
            if (!map1.containsKey(charS)) {
                map1.put(charS, charT);
            }

            // 4. 如果元素charT不存在于map2中，将其放入哈希映射中
            if (!map2.containsKey(charT)) {
                map2.put(charT, charS);
            }
        }

        // 返回true
        return true;
    }
}
```

```

class Solution {
public:
    bool isIsomorphic(string s, string t) {
        // 创建哈希映射用于存储字符串s中的元素
        unordered_map<char, char> map1;

        // 创建哈希映射用于存储字符串t中的元素
        unordered_map<char, char> map2;

        // 由于t.length == s.length
        // 按顺序访问s和t中对应的元素
        for (int i = 0; i < s.length(); i++) {
            char charS = s[i];
            char charT = t[i];

            // 1. 如果元素charS已经存在于map1中
            // 并且对应的值与当前元素charT不相同, 返回false
            if (map1.count(charS) && map1[charS] != charT) {
                return false;
            }

            // 2. 如果元素charT已经存在于map2中
            // 并且对应的值与当前元素charS不相同, 返回false
            if (map2.count(charT) && map2[charT] != charS) {
                return false;
            }

            // 3. 如果元素charS不存在于map1中, 将其放入哈希映射中
            if (!map1.count(charS)) {
                map1[charS] = charT;
            }

            // 4. 如果元素charT不存在于map2中, 将其放入哈希映射中
            if (!map2.count(charT)) {
                map2[charT] = charS;
            }
        }

        // 返回true
        return true;
    }
};

```

```

class Solution:
    def isIsomorphic(self, s: str, t: str) -> bool:

        # 设置一个哈希集合用来存储字符串 s 当中的元素
        dic1 = {}

```

```
# 设置一个哈希集合用来存储字符串 t 当中的元素
dic2 = {}

# 由于 t.length == s.length
# 按照顺序访问 s 和 t 中对应的元素
for i in range(len(s)):

    # 1、访问的元素 s[i] 存在于 dic1 中
    # 并且发现它对应的元素值和当前 t 中元素 t[i] 不相同
    # 返回 False
    if s[i] in dic1 and dic1[s[i]] != t[i]:
        return False

    # 2、访问的元素 t[i] 存在于 dic2 中
    # 并且发现它对应的元素值和当前 s 中元素 s[i] 不相同
    # 返回 False
    if t[i] in dic2 and dic2[t[i]] != s[i]:
        return False

    # 3、访问的元素 s[i] 不存在于 dic1 中
    # 存放到哈希集合中
    if s[i] not in dic1:
        # dic1[s[i]] = t[i]
        dic1[s[i]] = t[i]

    # 3、访问的元素 t[i] 不存在于 dic2 中
    # 存放到哈希集合中
    if t[i] not in dic2:
        # dic2[t[i]] = s[i]
        dic2[t[i]] = s[i]

# 返回 True
return True
```

LeetCode 217、存在重复元素

一、题目描述

给你一个整数数组 `nums` 。如果任一值在数组中出现 **至少两次** ，返回 `true` ；如果数组中每个元素互不相同，返回 `false` 。

示例 1：

```
输入：nums = [1,2,3,1]
输出：true
```

示例 2：

```
输入: nums = [1,2,3,4]
输出: false
```

示例 3:

```
输入: nums = [1,1,1,3,3,4,3,2,4,2]
输出: true
```

提示:

- `1 <= nums.length <= 105`
- `-109 <= nums[i] <= 109`

二、题目解析

三、参考代码

代码一

哈希集合调**API讨巧解法**

```
#### 题目: LC217. 存在重复元素
#### 难度: 简单
#### 作者: 闭着眼睛学数理化
#### 算法: 哈希集合调API讨巧解法
#### 代码看不懂的地方, 请直接在群上提问

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return (len(set(nums)) != len(nums))

#### 由于哈希集合具有不包含重复元素的性质,
#### 如果nums列表中存在重复元素, 将其转换成哈希集合后长度必然减少
#### 所以我们只需要判断set(nums)的长度和nums的长度是否一致即可得到答案
#### 如果长度一致, 则返回False, 说明nums不包含重复元素
#### 如果长度不一致, 则返回True, 说明nums包含重复元素
```

代码二

哈希集合遍历解法

1. Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 存在重复元素 (LeetCode 217) :https://leetcode.cn/problems/contains-duplicate/
```

```

class Solution {
    public boolean containsDuplicate(int[] nums) {

        // 使用数据结构 set 来存放 nums 里面的所有数字
        Set<Integer> set = new HashSet<>();

        // 遍历数组
        for (int num: nums) {

            // 如果数字已经存在于 set 中, 直接返回 true
            if (set.contains(num)) {
                return true;
            }

            // 把元素添加到 set 中
            set.add(num);
        }

        // 如果成功遍历完数组, 则表示没有重复元素, 返回 false
        return false;
    }
}

```

1. C++ 代码

```

class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {

        // 使用数据结构 unordered_set 来存放 nums 里面的所有数字
        unordered_set<int> s;

        // 遍历数组
        for (int x: nums) {

            // 如果数字已经存在于 set 中, 直接返回 true
            if (s.find(x) != s.end()) {
                return true;
            }

            // 把元素添加到 set 中
            s.insert(x);
        }

        // 如果成功遍历完数组, 则表示没有重复元素, 返回 false
        return false;
    }
};

```

1. Python 代码

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        # 使用数据结构 set 来存放 nums 里面的所有数字
        pre = set()
        # 遍历数组
        for num in nums:
            # 如果数字已经存在于 set 中, 直接返回 true
            if num in pre:
                return True
            # 把元素添加到 set 中
            pre.add(num)
        # 如果成功遍历完数组, 则表示没有重复元素, 返回 false
        return False
```

LeetCode 219、存在重复元素II

一、题目描述

给你一个整数数组 `nums` 和一个整数 `k`，判断数组中是否存在两个不同的索引 `i` 和 `j`，满足 `nums[i] == nums[j]` 且 `abs(i - j) <= k`。如果存在，返回 `true`；否则，返回 `false`。

示例 1:

输入: `nums = [1,2,3,1]`, `k = 3` 输出: `true`

示例 2:

输入: `nums = [1,0,1,1]`, `k = 1` 输出: `true`

示例 3:

输入: `nums = [1,2,3,1,2,3]`, `k = 2` 输出: `false`

提示:

- `1 <= nums.length <= 10^5`
- `-109 <= nums[i] <= 10^9`
- `0 <= k <= 10^5`

二、题目解析

三、参考代码

```
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:

        # 使用哈希集合
        pos = {}
```

```
# 如果想在 for 循环中同时获得列表的索引 i 和元素值 v
# 可以使用枚举内置函数 enumerate()
for i, v in enumerate(nums):
    # 1、如果发现当前这个元素 v 已经存在于哈希集合里面
    # 说明在此之前就已经访问到了一个元素，值为 v
    # 2、pos[v] 表示的是之前访问到的元素值所在的索引
    # 判断 i - pos[v] <= k
    if v in pos and i - pos[v] <= k:
        # 符合要求，就返回 True
        return True
    # 否则，把 v 和 i 存储到哈希集合里面
    pos[v] = i

# 最终没有找到，返回 False
return False
```

LeetCode 349、两个数组的交集

一、题目描述

给定两个数组 `nums1` 和 `nums2`，返回 它们的交集。输出结果中的每个元素一定是 唯一 的。我们可以 不考虑输出结果的顺序。

示例 1：

```
输入：nums1 = [1,2,2,1], nums2 = [2,2]
输出：[2]
```

示例 2：

```
输入：nums1 = [4,9,5], nums2 = [9,4,9,8,4]
输出：[9,4]
解释：[4,9] 也是可通过的
```

提示：

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 1000`

二、题目解析

三、参考代码

方法一

哈希集合调**API解法**

```
#### 题目：LC349. 两个数组的交集
#### 难度：简单
#### 作者：闭着眼睛学数理化
#### 算法：哈希集合调API
#### 代码看不懂的地方，请直接在群上提问

class Solution:
    def intersection(self, nums1: List[int], nums2: List[int]) -> List[int]:
        return list(set(nums1) & set(nums2))

#### 先将两个数组nums1和nums2用set(nums1)和set(nums2)转化为哈希集合
#### 再使用两个集合的取交集操作&，得到set(nums1)和set(nums2)的交集
#### 由于题目要求返回一个列表，我们还需要把交集再转化为list()，即可返回
```

哈希集合遍历解法

```
class Solution:
    def intersection(self, nums1: List[int], nums2: List[int]) -> List[int]:
        # 先获得nums1对应的哈希集合
        nums1_set = set(nums1)
        # 构建一个答案集合，初始化为空
        ans_set = set()

        # 遍历nums2中的元素num
        for num in nums2:
            # 如果num位于nums1对应的哈希集合nums1_set中
            if num in nums1_set:
                # 则说明num同时位于nums1和nums2中
                # 将其加入ans_set
                ans_set.add(num)

        # 最后将ans_set转化为列表后返回
        return (list(ans_set))
```

时空复杂度

时间复杂度： $O(m+n)$ 。nums1 和 nums2 各自需要遍历一次。

空间复杂度： $O(m+n)$ 。两个哈希集合所占用的空间。

方法二*

排序+双指针解法（暂不要求掌握）

1. Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/intersection-of-two-arrays/
class solution {
    public int[] intersection(int[] nums1, int[] nums2) {

        // 首先对两个数组进行排序
        Arrays.sort(nums1);

        Arrays.sort(nums2);

        // 计算出两个数组的长度
        int length1 = nums1.length ;

        int length2 = nums2.length;

        // 两个数组的交集结果数组长度必然是小于等于最短数组的长度
        int[] res = new int[length1 > length2 ? length2 : length1];

        // 设置三个索引指针

        // index 指向结果数组 res ，每当 index 指向的位置填充了元素就向后移动
        int index = 0;

        // index1 指向数组 nums1 中的元素，将该元素和 index2 指向数组 nums2 中的元素进行比较
        int index1 = 0;

        // index2 指向数组 nums2 中的元素，将该元素和 index1 指向数组 nums1 中的元素进行比较
        int index2 = 0;

        // 移动 index1 和 index2
        while (index1 < length1 && index2 < length2) {

            // 获取 index1 指向的元素值
            int num1 = nums1[index1];

            // 获取 index2 指向的元素值
            int num2 = nums2[index2];

            // num1 和 num2 的大小关系有三种
```

```

// 1、num1 == num2
if (num1 == num2) {

    // 由于输出结果中的每个元素一定是 【唯一】 的，所以要做一下判断
    // 如果 res 中的 index 在起始位置，说明还没有存放元素
    // 那么这个相等的元素可以存放 to res 中

    // 如果 res 中的 index 不在起始位置
    // 当它前面存放的元素并不是现在想要存放的元素
    // 那么这个相等的元素可以存放 to res 中
    if (index == 0 || num1 != res[index - 1]) {

        // 存放 to res 中
        res[index] = num1;
        // 移动 index
        index++;
    }

    // 移动 index1 ， 比较其它元素
    index1++;
    // 移动 index2 ， 比较其它元素
    index2++;

// 2、num1 < num2
} else if (num1 < num2) {

    // 由于两个数组已经排序了，说明此时 num1 肯定会小于 nums2 数组中 num2 后面所有的

    // 那 num1 肯定是在 nums2 中找到相等的元素
    // 移动 index1 ， 比较其它元素
    index1++;

// 3、num1 > num2
} else {

    // 由于两个数组已经排序了，说明此时 num2 肯定会小于 nums1 数组中 num1 后面所有的

    // 那 num2 肯定是在 nums1 中找到相等的元素
    // 移动 index2 ， 比较其它元素
    index2++;

}
}

// 最后返回结果数组中有值的那些元素就行
return Arrays.copyOfRange(res, 0, index);
}
}

```

数

数

1. C++ 代码

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {

        // 首先对两个数组进行排序
        sort(nums1.begin(), nums1.end());

        sort(nums2.begin(), nums2.end());

        // 计算出两个数组的长度
        int length1 = nums1.size();

        int length2 = nums2.size();

        // 两个数组的交集结果数组长度必然是小于等于最短数组的长度
        vector<int> res ;

        // 设置三个索引指针

        // index 指向结果数组 res , 每当 index 指向的位置填充了元素就向后移动
        // int index = 0;

        // index1 指向数组 nums1 中的元素, 将该元素和 index2 指向数组 nums2 中的元素进行比较
        int index1 = 0;

        // index2 指向数组 nums2 中的元素, 将该元素和 index1 指向数组 nums1 中的元素进行比较
        int index2 = 0;

        // 移动 index1 和 index2
        while (index1 < length1 && index2 < length2) {

            // 获取 index1 指向的元素值
            int num1 = nums1[index1];

            // 获取 index2 指向的元素值
            int num2 = nums2[index2];

            // num1 和 num2 的大小关系有三种

            // 1、num1 == num2
            if (num1 == num2) {

                // 由于输出结果中的每个元素一定是 【唯一】 的, 所以要做一下判断
                // 如果 res 中的 index 在起始位置, 说明还没有存放元素
```

数

数

};

```
// 那么这个相等的元素可以存放到 res 中

// 如果 res 中的 index 不在起始位置
// 当它前面存放的元素并不是现在想要存放的元素
// 那么这个相等的元素可以存放到 res 中
if ( !res.size() || num1 != res.back() ) {

    // 存放到 res 中
    res.push_back(num1);

}

// 移动 index1 , 比较其它元素
index1++;

// 移动 index2 , 比较其它元素
index2++;

// 2、num1 < num2
} else if (num1 < num2) {

    // 由于两个数组已经排序了, 说明此时 num1 肯定会小于 nums2 数组中 num2 后面所有的

    // 那 num1 肯定是无法在 nums2 中找到相等的元素
    // 移动 index1 , 比较其它元素
    index1++;

// 3、num1 > num2
} else {

    // 由于两个数组已经排序了, 说明此时 num2 肯定会小于 nums1 数组中 num1 后面所有的

    // 那 num2 肯定是无法在 nums1 中找到相等的元素
    // 移动 index2 , 比较其它元素
    index2++;

}

}

// 最后返回结果数组中有值的那些元素就行
return res;

}
```

1. Python 代码

```
class Solution:
    def intersection(self, nums1: List[int], nums2: List[int]) -> List[int]:
        # 首先对两个数组进行排序
        nums1.sort()

        nums2.sort()

        # 计算出两个数组的长度
        length1 = len(nums1)

        length2 = len(nums2)

        # 两个数组的交集结果数组长度必然是小于等于最短数组的长度
        res = list()

        # 设置三个索引指针

        # index 指向结果数组 res , 每当 index 指向的位置填充了元素就向后移动
        # index = 0

        # index1 指向数组 nums1 中的元素, 将该元素和 index2 指向数组 nums2 中的元素进行比较
        index1 = 0

        # index2 指向数组 nums2 中的元素, 将该元素和 index1 指向数组 nums1 中的元素进行比较
        index2 = 0

        # 移动 index1 和 index2
        while index1 < length1 and index2 < length2 :

            # 获取 index1 指向的元素值
            num1 = nums1[index1]

            # 获取 index2 指向的元素值
            num2 = nums2[index2]

            # num1 和 num2 的大小关系有三种

            # 1、num1 == num2
            if num1 == num2 :

                # 由于输出结果中的每个元素一定是 【唯一】 的, 所以要做一下判断
                # 如果 res 中的 index 在起始位置, 说明还没有存放元素
                # 那么这个相等的元素可以存放到 res 中

                # 如果 res 中的 index 不在起始位置
                # 当它前面存放的元素并不是现在想要存放的元素
```

```

        # 那么这个相等的元素可以存放到 res 中
        if not res or num1 != res[-1]:
            res.append(num1)

        # 移动 index1 , 比较其它元素
        index1 += 1
        # 移动 index2 , 比较其它元素
        index2 += 1

    # 2、num1 < num2
    elif num1 < num2 :

        # 由于两个数组已经排序了, 说明此时 num1 肯定会小于 nums2 数组中 num2 后面所有的数
        # 那 num1 肯定是在 nums2 中找不到相等的元素
        # 移动 index1 , 比较其它元素
        index1 += 1

    # 3、num1 > num2
    else:

        # 由于两个数组已经排序了, 说明此时 num2 肯定会小于 nums1 数组中 num1 后面所有的数
        # 那 num2 肯定是在 nums1 中找不到相等的元素
        # 移动 index2 , 比较其它元素
        index2 += 1

    # 最后返回结果数组中有值的那些元素就行
    return res

```

时空复杂度

时间复杂度： $O(m\log m + n\log n)$ 。两数组快排时间复杂度分别是 $O(m\log m)$ 、 $O(n\log n)$ ，双指针遍历需要 $O(m+n)$ ，复杂度取决于较大的 $O(m\log m + n\log n)$ 。

空间复杂度： $O(\log m + \log n)$ 。排序使用的额外空间。

第十天

LeetCode 242、有效的字母异位词

一、题目描述

给定两个字符串 `s` 和 `t`，编写一个函数来判断 `t` 是否是 `s` 的字母异位词。

注意：若 `s` 和 `t` 中每个字符出现的次数都相同，则称 `s` 和 `t` 互为字母异位词。

示例 1:

输入: `s = "anagram", t = "nagaram"`
输出: `true`

示例 2:

输入: `s = "rat", t = "car"`
输出: `false`

提示:

- `1 <= s.length, t.length <= 5 * 104`
- `s` 和 `t` 仅包含小写字母

进阶: 如果输入字符串包含 `unicode` 字符怎么办? 你能否调整你的解法来应对这种情况?

二、题目解析

题目讲的是让你判断两个字符串中的字母是否一致, 比如 **示例1** 中, `s` 包含字母 `a、n、g、r、m`, 而 `t` 中也包含 `a、n、g、r、m`, 都是只有这五个字母, 并且 **频次** 相同, 只是顺序不同。

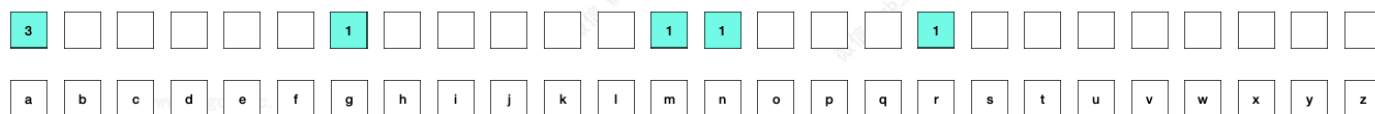
看到 **频次** 这个词, 你脑海中第一想法是什么?

没错, 就是 **哈希表**!

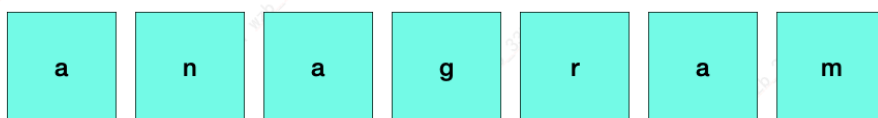
解法思路很简单。

1、首先先判断两个字符串长度是否相同, 不相同直接返回 `false`

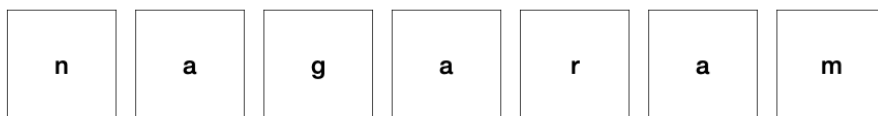
2、然后把 `s` 中所有的字符出现个数使用 **计数器** 统计起来, 存入一个大小为 26 的数组中 (注意题目的说明)



s

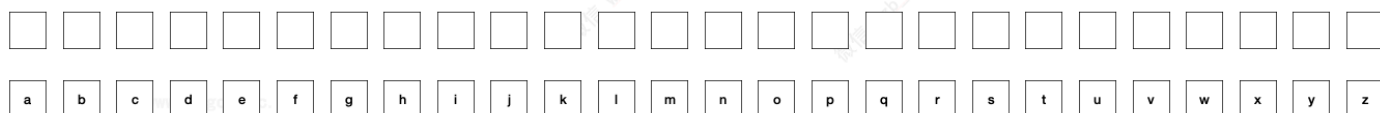


t



公众号: 吴师兄学算法

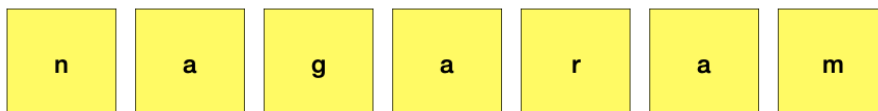
3、最后再来统计 `t` 字符串, 即遍历 `t` 时将对应的字母频次进行减少, 如果期间 **计数器** 出现小于零的情况, 则说明 `t` 中包含一个不存在于 `s` 中的字母, 直接返回 `false`。



s



t



公众号：吴师兄学算法

4、最后检查计数器是否归零。

三、参考代码

代码一

哈希集合调**API讨巧解法**

```
#### 题目：LC242. 有效的字母异位词
#### 难度：简单
#### 作者：闭着眼睛学数理化
#### 算法：哈希集合调API讨巧解法
#### 代码看不懂的地方，请直接在群上提问

#### 导入collections中的Counter计数器类
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)

#### 直接获得s和t的计数结果Counter(s)和Counter(t)
#### 若两者相等，说明s和t中的元素以及频率完全一致，互为字母异位词
#### 若两者不相等，说明s和t中的元素以及频率不完全一致，不是一组有效的字母异位词
```

代码二

哈希集合遍历解法

```
#### 题目：LC242. 有效的字母异位词
#### 难度：简单
#### 作者：闭着眼睛学数理化
#### 算法：哈希集合遍历解法
#### 代码看不懂的地方，请直接在群上提问
```



```

#### 导入collections中的Counter计数器类
from collections import Counter

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # 如果两个字符串长度不同，必然不可能构成一组有效的字母异位词，返回False
        if len(s) != len(t):
            return False
        # 使用Counter()计数器，得到两个字符串对应的哈希表
        cnt1, cnt2 = Counter(s), Counter(t)
        # 遍历cnt1中的键（即某个特定字符）
        for ch in cnt1:
            # 如果这个字符在cnt1中的频率和在cnt2中的频率不相等
            if cnt1[ch] != cnt2[ch]:
                # 返回False
                return False
        # 如果在上述循环中没有返回False，说明这两个字符串互为字母异位词
        return True

```

代码三

用列表表示哈希表进行统计

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的字母异位词（LeetCode 242）：https://leetcode.cn/problems/valid-anagram/
class Solution {
    public boolean isAnagram(String s, String t) {

        // 如果两个字符串的长度都不一致，那么肯定是无法成为字母异位词的
        if(s.length() != t.length()){

            // 直接返回 false
            return false;

        }

        // 让 a - z 这 26 个字母对应的下标变成 0 - 25 方便存到数组中
        // 比如 a 对应的索引就是 0
        // b 对应的索引就是 1
        int[] table = new int[26];

        // 从头到尾遍历字符串 s
        for( int i = 0 ; i < s.length() ; i++){

```

```

        // 把访问的字符转换为整数的形式
        // 比如访问字母 a, 那么 -'a' 之后就是 0, 就是 a 对应的索引为 0
        int index = s.charAt(i) - 'a';

        // 那么意味着这个字母出现的频次需要加 1
        table[index]++;

    }

    for( int i = 0 ; i < t.length() ; i++){

        // 把访问的字符转换为整数的形式
        // 比如访问字母 a, 那么 -'a' 之后就是 0, 就是 a 对应的索引为 0
        int index = t.charAt(i) - 'a';

        // 那么意味着这个字母出现的频次需要减 1
        table[index]--;

        // 如果说发现这个字母出现的频次小于了 0
        // 说明 t 中出现了 s 中不曾用的字母
        if(table[index] < 0){

            // 那就不是字母异位词
            return false;

        }

    }

    // 否则, 说明是字母异位词
    return true;

}
}

```

2、C++ 代码

```

class solution {
public:
    bool isAnagram(string s, string t) {
        // 如果两个字符串的长度都不一致, 那么肯定是无法成为字母异位词的
        if(s.length() != t.length()){

            // 直接返回 false
            return false;

        }
    }
}

```

```

// 让 a - z 这 26 个字母对应的下标变成 0 - 25 方便存到数组中
// 比如 a 对应的索引就是 0
// b 对应的索引就是 1
vector<int> table(26, 0);

// 从头到尾遍历字符串 s
for( int i = 0 ; i < s.length() ; i++){

    // 把访问的字符转换为整数的形式
    // 比如访问字母 a, 那么 -'a' 之后就是 0, 就是 a 对应的索引为 0
    int index = s[i] - 'a';

    // 那么意味着这个字母出现的频次需要加 1
    table[index]++;

}

for( int i = 0 ; i < t.length() ; i++){

    // 把访问的字符转换为整数的形式
    // 比如访问字母 a, 那么 -'a' 之后就是 0, 就是 a 对应的索引为 0
    int index = t[i] - 'a';

    // 那么意味着这个字母出现的频次需要减 1
    table[index]--;

    // 如果说发现这个字母出现的频次小于了 0
    // 说明 t 中出现了 s 中不曾用的字母
    if(table[index] < 0){

        // 那就不是字母异位词
        return false;

    }

}

// 否则, 说明是字母异位词
return true;

}

};

```

3、Python 代码

```

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:

        # 如果两个字符串的长度都不一致, 那么肯定无法成为字母异位词的
        if len(s) != len(t):

```

```

    # 直接返回 False
    return False

# 让 a - z 这 26 个字母对应的下标变成 0 - 25 方便存到数组中
# 比如 a 对应的索引就是 0
# b 对应的索引就是 1
table = [0] * 26

# 从头到尾遍历字符串 s
for i in s:

    # 把访问的字符转换为整数的形式
    # 比如访问字母 a, 那么 - 'a' 之后就是 0, 就是 a 对应的索引为 0
    index = ord(i) - ord('a')

    # 那么意味着这个字母出现的频次需要加 1
    table[index] += 1

for i in t:

    # 把访问的字符转换为整数的形式
    # 比如访问字母 a, 那么 - 'a' 之后就是 0, 就是 a 对应的索引为 0
    index = ord(i) - ord('a')

    # 那么意味着这个字母出现的频次需要减 1
    table[index] -= 1

    # 如果说发现这个字母出现的频次小于了 0
    # 说明 t 中出现了 s 中不曾用的字母
    if table[index] < 0 :

        # 那就不是字母异位词
        return False

# 否则, 说明是字母异位词
return True

```

复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 为 s 的长度。
- 空间复杂度： $O(S)$ ，其中 S 为字符集大小，此处 $S = 26$ 。

LeetCode 383、赎金信 (HJ81. 字符串字符匹配)

一、题目描述

给你两个字符串： `ransomNote` 和 `magazine`，判断 `ransomNote` 能不能由 `magazine` 里面的字符构成。

如果可以，返回 `true`；否则返回 `false`。

`magazine` 中的每个字符只能在 `ransomNote` 中使用一次。

示例 1:

```
输入: ransomNote = "a", magazine = "b"
输出: false
```

示例 2:

```
输入: ransomNote = "aa", magazine = "ab"
输出: false
```

示例 3:

```
输入: ransomNote = "aa", magazine = "aab"
输出: true
```

提示:

- `1 <= ransomNote.length, magazine.length <= 10^5`
- `ransomNote` 和 `magazine` 由小写英文字母组成

二、题目解析

三、参考代码

方法一

哈希集合遍历解法

```
#### 题目: LC383. 赎金信
#### 难度: 简单
#### 作者: 闭着眼睛学数理化
#### 算法: 哈希集合遍历解法
#### 代码看不懂的地方, 请直接在群上提问

#### 导入collections中的Counter计数器类, 使用dict()也可以但是代码就要多一些判断
from collections import Counter

class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        # 用两个哈希表保存R和M两个字符串中所有元素出现的频率
```

```

cnt_R, cnt_M = Counter(ransomNote), Counter(magazine)

# 由于要计算R是否被M完全包含，所以我们要遍历R中的字符
# 遍历cnt_R中的所有字符
for ch in cnt_R:
    # 如果发现ch在R中的数目大于在M中的数目，则返回False
    if cnt_R[ch] > cnt_M[ch]:
        return False

# 成功退出循环，返回True
return True

```

方法二

用列表表示哈希表进行统计

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 赎金信 (LeetCode 383) : https://leetcode.cn/problems/ransom-note/
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {

        // 手动设置哈希表
        // 这里的哈希表的作用是计数器
        // 由于小写字母的个数为 26 个，所以数组大小为 26
        int[] cnt = new int[26];

        // 记录 magazine 里字母出现的次数
        for (char c : magazine.toCharArray()) {
            cnt[c - 'a']++;
        }

        // 再用 ransomNote 去验证这个数组是否包含了 ransomNote 所需要的所有字母
        for (char c : ransomNote.toCharArray()) {
            // 在遍历过程中，每遍历一个字母，对应的频次减 1
            cnt[c - 'a']--;

            // 如果发现某个字母的频次小于了 0
            // 说明在 ransomNote 中出现了 magazine 未曾有的字母
            // 即 ransomNote 不能由 magazine 里面的字符构成
            if (cnt[c - 'a'] < 0) {

                // 返回 false
            }
        }
    }
}

```

```

        return false;
    }
}

// 说明可以, 返回 true
return true;
}
}

```

2、**C++** 代码

```

class solution {
public:
    bool canConstruct(string ransomNote, string magazine) {

        // 手动设置哈希表
        // 这里的哈希表的作用是计数器
        // 由于小写字母的个数为 26 个, 所以数组大小为 26
        vector<int> cnt(26);

        // 记录 magazine 里字母出现的次数
        for (auto & c : magazine) {
            cnt[c - 'a']++;
        }

        // 再用 ransomNote 去验证这个数组是否包含了 ransomNote 所需要的所有字母
        for (auto & c : ransomNote) {
            // 在遍历过程中, 每遍历一个字母, 对应的频次减 1
            cnt[c - 'a']--;

            // 如果发现某个字母的频次小于了 0
            // 说明在 ransomNote 中出现了 magazine 未曾有的字母
            // 即 ransomNote 不能由 magazine 里面的字符构成
            if(cnt[c - 'a'] < 0) {

                // 返回 false
                return false;
            }
        }

        // 说明可以, 返回 true
        return true;
    }
};

```

3、Python 代码

```
class solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        # 手动设置哈希表
        # 这里的哈希表的作用是计数器
        # 由于小写字母的个数为 26 个，所以数组大小为 26
        cnt = [0] * 26

        # 记录 magazine 里字母出现的次数
        for ch in magazine:
            # 对于数组类型，其下标为 int 类型
            # 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
            # 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
            idx = ord(ch) - ord('a')
            cnt[idx] += 1

        # 再用 ransomNote 去验证这个数组是否包含了 ransomNote 所需要的所有字母
        for ch in ransomNote:
            # 在遍历过程中，每遍历一个字母，对应的频次减 1
            cnt[ord(ch) - ord('a')] -= 1

            # 如果发现某个字母的频次小于了 0
            # 说明在 ransomNote 中出现了 magazine 未曾有的字母
            # 即 ransomNote 不能由 magazine 里面的字符构成
            if cnt[ord(ch) - ord('a')] < 0 :

                return False

        # 说明可以，返回 true
        return True
```

LeetCode 387、字符串中的第一个唯一字符

一、题目描述

给定一个字符串 `s`，找到 它的第一个不重复的字符，并返回它的索引。如果不存在，则返回 `-1`。

示例 1:

输入: `s = "leetcode"`

输出: 0

示例 2:

输入: `s = "loveleetcode"`

输出: 2

示例 3:

输入: s = "aabb"

输出: -1

提示:

- `1 <= s.length <= 10^5`
- `s` 只包含小写字母

二、题目解析

三、参考代码

```
class Solution {
    public int firstUniqChar(String s) {
        Map<Character, Integer> counter = new HashMap<>();

        // 统计字符频率
        for (char c : s.toCharArray()) {
            counter.put(c, counter.getDefault(c, 0) + 1);
        }

        // 找到第一个出现频率为 1 的字符
        for (int i = 0; i < s.length(); i++) {
            if (counter.get(s.charAt(i)) == 1) {
                return i;
            }
        }

        // 不存在唯一字符, 返回 -1
        return -1;
    }
}
```

```
class Solution {
public:
    int firstUniqChar(string s) {
        unordered_map<char, int> counter;

        // 统计字符频率
        for (char c : s) {
            counter[c]++;
        }

        // 找到第一个出现频率为 1 的字符
        for (int i = 0; i < s.size(); i++) {
```

```

        if (counter[s[i]] == 1) {
            return i;
        }
    }

    // 不存在唯一字符, 返回 -1
    return -1;
}
};

```

```

class Solution:
    def firstUniqChar(self, s: str) -> int:
        # Counter 可以直接统计字符串、列表等可迭代对象的元素频率
        # s = "leetcode"
        # cnt = Counter({'e': 3, 'l': 1, 't': 1, 'c': 1, 'o': 1, 'd': 1})
        cnt = Counter(s)
        print(cnt)

        # 如果想在for循环中同时获得列表的索引 i 和元素值 v
        # 可以使用枚举内置函数 enumerate()
        for i, v in enumerate(s):
            # 如果找到了某个字符出现的频率为 1
            if cnt[v] == 1:
                # 返回它的下标即可
                return i

        # 如果不存在, 则返回 -1
        return -1

```

LeetCode 409、最长回文串

一、题目描述

给定一个包含大写字母和小写字母的字符串 `s`，返回 *通过这些字母构造的 最长的回文串*。

在构造过程中，请注意 **区分大小写**。比如 `"Aa"` 不能当做一个回文字符串。

示例 1:

输入: `s = "abcccccdd"`

输出: 7

解释:

我们可以构造的最长的回文串是 `"dccaccd"`，它的长度是 7。

示例 2:

输入:s = "a"
输入:1

示例 3:

输入:s = "bb"
输入: 2

提示:

- `1 <= s.length <= 2000`
- `s` 只能由小写和/或大写英文字母组成

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最长回文串 (LeetCode 409) : https://leetcode.cn/problems/longest-palindrome/
class Solution {
    public int longestPalindrome(String s) {

        // 手动设置哈希表，哈希函数为 c - 'A'
        // 这里的哈希表的作用是计数器
        // 由于 大写字母 'A' 与小写字母 'a' 的 ASCII 相差 32
        // A : 65
        // a : 97
        // 所以设置哈希表的大小为 32 + 26 = 58
        int[] cnt = new int[58];

        // 统计每个字母出现的频次
        for (char c : s.toCharArray()) {

            cnt[c - 'A'] += 1;
        }

        // 初始化结果
        int ans = 0;

        for (int x: cnt) {

            // 根据奇偶性来判断当前字母的使用次数为多少
            // 按位与: &
        }
    }
}
```

```

        // 将参与运算的两操作数各对应的二进制位进行与操作
        // 只有对应的两个二进制位均为 1 时，结果的对应二进制位才为 1，否则为 0
        // 通过这个方法判断 x 的奇偶性，即统计这个字母出现的是偶数次还是奇数次
        // 1 的二进制为          0001
        // 比如 x 为 4，二进制为 0100，那么 x & 1 的结果是 0000，即为 0
        // 那么 x - (x & 1) 的结果是 4，那么这 4 个字母全部都可以出现在最终的回文串中
        // *****
        // 比如 x 为 5，二进制为 0101，那么 x & 1 的结果是 0001，即为 1
        // 那么 x - (x & 1) 的结果是 4，即出现了 5 次的那个字母只有其中 4 个可以被用上
        ans += x - (x & 1);

    }

    // 最后，会尝试一下，能不能再添加一个字符到回文串中
    // 如果最终的长度小于原字符串的长度，说明里面某个字符出现了奇数次，那么那个字符可以放在回文串的中间，所以额外再加一
    return ans < s.length() ? ans + 1 : ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int longestPalindrome(string s) {

        // 手动设置哈希表，哈希函数为 c - 'A'
        // 这里的哈希表的作用是计数器
        // 由于 大写字母 'A' 与小写字母 'a' 的 ASCII 相差 32
        // A : 65
        // a : 97
        // 所以设置哈希表的大小为 32 + 26 = 58
        int cnt[58] = {0};

        // 统计每个字母出现的频次
        for(string::size_type i = 0; i < s.size(); i++) {
            if('a' <= s[i] && s[i] <= 'z') {
                cnt[s[i] - 'a']++;
            } else {
                cnt[s[i] - 'A' + 26]++;
            }
        }

        // 初始化结果
        int ans = 0;

        for (int x: cnt) {

            // 根据奇偶性来判断当前字母的使用次数为多少

```

```

        // 按位与：&
        // 将参与运算的两操作数各对应的二进制位进行与操作
        // 只有对应的两个二进制位均为 1 时，结果的对应二进制位才为 1，否则为 0
        // 通过这个方法判断 x 的奇偶性，即统计这个字母出现的是偶数次还是奇数次
        // 1 的二进制为          0001
        // 比如 x 为 4，二进制为 0100，那么 x & 1 的结果是 0000，即为 0
        // 那么 x - (x & 1) 的结果是 4，那么这 4 个字母全部都可以出现在最终的回文串中
        // *****
        // 比如 x 为 5，二进制为 0101，那么 x & 1 的结果是 0001，即为 1
        // 那么 x - (x & 1) 的结果是 4，即出现了 5 次的那个字母只有其中 4 个可以被用上
        ans += x - (x & 1);
    }

    // 最后，会尝试一下，能不能再添加一个字符到回文串中
    // 如果最终的长度小于原字符串的长度，说明里面某个字符出现了奇数次，那么那个字符可以放在回文串的中间，所以额外再加一
    return ans < s.length() ? ans + 1 : ans;
}
};

```

3、Python 代码

```

class Solution:
    def longestPalindrome(self, s: str) -> int:

        # 手动设置哈希表，哈希函数为 c - 'A'
        # 这里的哈希表的作用是计数器
        # 由于 大写字母 'A' 与小写字母 'a' 的 ASCII 相差 32
        # A : 65
        # a : 97
        # 所以设置哈希表的大小为 32 + 26 = 58
        l_s = list(s)
        #字典来存储每个字母出现的个数
        cnt = {}
        for word in l_s:
            cnt[word] = 0

        for word in l_s:
            cnt[word] += 1

        ans = 0

        for key, value in cnt.items():
            #如果出现个数为偶数，直接加入
            if value % 2 == 0:
                ans += value
            else:

```

```
#如果为奇数，则加value - 1，并且，该字母的次数变为1
ans += value - 1
cnt[key] = 1

return ans + 1 if 1 in cnt.values() else ans
```

第十一天

LeetCode 1588、所有奇数长度子数组的和

一、题目描述

给你一个正整数数组 `arr`，请你计算所有可能的奇数长度子数组的和。

子数组 定义为原数组中的一个连续子序列。

请你返回 `arr` 中 所有奇数长度子数组的和 。

示例 1：

输入：arr = [1,4,2,5,3] 输出：58 解释：所有奇数长度子数组和它们的和为： [1] = 1 [4] = 4 [2] = 2 [5] = 5 [3] = 3 [1,4,2] = 7 [4,2,5] = 11 [2,5,3] = 10 [1,4,2,5,3] = 15 我们将所有值求和得到 1 + 4 + 2 + 5 + 3 + 7 + 11 + 10 + 15 = 58

示例 2：

输入：arr = [1,2] 输出：3 解释：总共只有 2 个长度为奇数的子数组，[1] 和 [2]。它们的和为 3。

示例 3：

输入：arr = [10,11,12] 输出：66

提示：

- `1 <= arr.length <= 100`
- `1 <= arr[i] <= 1000`

二、题目解析

三、参考代码

Java

```
class Solution {
    public int sumOddLengthSubarrays(int[] arr) {

        // 初始结果
        int ans = 0;

        // preSum[ i ] 表示数组 nums 从下标 0 到下标 i - 1 的所有元素之和
        int[] preSum = new int[arr.length + 1];
```

```

// 初始化 preSum
for (int i = 0; i < arr.length; i++) {
    preSum[i + 1] = preSum[i] + arr[i];
}

// 从索引为 0 的位置开始遍历出 1、3、5 等等奇数长度的子数组
for (int i = 0; i < arr.length; i++) {

    // 都是从索引为 i 的位置开始延伸出子数组
    // 一开始子数组长度为 1
    // 接下来子数组长度为 3
    for (int j = 1; j <= arr.length; j += 2) {

        // 边界判断
        if (i + j - 1 < arr.length) {

            // 累加当前子数组所有元素的和
            ans += preSum[i + j] - preSum[i];

        }
    }
}

// 返回结果
return ans;
}
}

```

C++

```

class Solution {
public:
    int sumOddLengthSubarrays(vector<int>& arr) {
        int ans = 0;
        vector<int> preSum(arr.size() + 1, 0);

        for (int i = 0; i < arr.size(); i++) {
            preSum[i + 1] = preSum[i] + arr[i];
        }

        for (int i = 0; i < arr.size(); i++) {
            for (int j = 1; j <= arr.size(); j += 2) {
                if (i + j - 1 < arr.size()) {
                    ans += preSum[i + j] - preSum[i];
                }
            }
        }
    }
}

```

```
        return ans;
    }
};
```

Python

```
class Solution:
    def sumOddLengthSubarrays(self, arr: List[int]) -> int:
        ans = 0
        preSum = [0] * (len(arr) + 1)

        for i in range(len(arr)):
            preSum[i + 1] = preSum[i] + arr[i]

        for i in range(len(arr)):
            for j in range(1, len(arr) + 1, 2):
                if i + j - 1 < len(arr):
                    ans += preSum[i + j] - preSum[i]

        return ans
```

LeetCode 303、区域和检索-数组不可变

一、题目描述

给定一个整数数组 `nums`，处理以下类型的多个查询：

1. 计算索引 `left` 和 `right`（包含 `left` 和 `right`）之间的 `nums` 元素的和，其中 `left <= right`

实现 `NumArray` 类：

- `NumArray(int[] nums)` 使用数组 `nums` 初始化对象
- `int sumRange(int i, int j)` 返回数组 `nums` 中索引 `left` 和 `right` 之间的元素的总和，包含 `left` 和 `right` 两点（也就是 `nums[left] + nums[left + 1] + ... + nums[right]`）

示例 1：

输入：

```
["NumArray", "sumRange", "sumRange", "sumRange"]
```

```
[[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
```

输出：

```
[null, 1, -1, -3]
```

解释：

```
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);
```

```
numArray.sumRange(0, 2); // return 1 ((-2) + 0 + 3)
```


numArray.sumRange(2, 5); // return -1 (3 + (-5) + 2 + (-1))

numArray.sumRange(0, 5); // return -3 ((-2) + 0 + 3 + (-5) + 2 + (-1))

提示:

- `1 <= nums.length <= 104`
- `-105 <= nums[i] <= 105`
- `0 <= i <= j < nums.length`
- 最多调用 `104` 次 `sumRange` 方法

二、题目解析

三、参考代码

Java

```
class NumArray {  
  
    // 前缀和数组  
    int[] sums;  
  
    public NumArray(int[] nums) {  
  
        int n = nums.length;  
  
        // sums[ i ] 表示数组 nums 从下标 0 到下标 i - 1 的所有元素之和  
        sums = new int[n + 1];  
  
        // 初始化 sums 数组  
        for (int i = 0; i < n; i++) {  
  
            sums[i + 1] = sums[i] + nums[i];  
        }  
    }  
  
    public int sumRange(int i, int j) {  
  
        // sums[ j + 1 ] 表示数组 nums 从下标 0 到下标 i 的所有元素之和  
        // sums[ i ] 表示数组 nums 从下标 0 到下标 i - 1 的所有元素之和  
        // 两者一相减得到结果  
        return sums[j + 1] - sums[i];  
    }  
}
```

C++

```
class NumArray {  
public:  
    vector<int> sums;
```

```

NumArray(vector<int>& nums) {
    int n = nums.size();
    sums.resize(n + 1);
    for (int i = 0; i < n; i++) {
        sums[i + 1] = sums[i] + nums[i];
    }
}

int sumRange(int i, int j) {
    return sums[j + 1] - sums[i];
}
};

```

Python

```

class NumArray:

    def __init__(self, nums: List[int]):
        self.sums = [0]
        _sums = self.sums

        for num in nums:
            _sums.append(_sums[-1] + num)

    def sumRange(self, i: int, j: int) -> int:
        _sums = self.sums
        return _sums[j + 1] - _sums[i]

```

LeetCode 724、寻找数组的中心下标

一、题目描述

给你一个整数数组 `nums`，请计算数组的 **中心下标**。

数组 **中心下标** 是数组的一个下标，其左侧所有元素相加的和等于右侧所有元素相加的和。

如果中心下标位于数组最左端，那么左侧数之和视为 `0`，因为在下标的左侧不存在元素。这一点对于中心下标位于数组最右端同样适用。

如果数组有多个中心下标，应该返回 **最靠近左边** 的那一个。如果数组不存在中心下标，返回 `-1`。

示例 1：

输入: `nums = [1, 7, 3, 6, 5, 6]`

输出: 3

解释:

中心下标是 3 。

左侧数之和 $\text{sum} = \text{nums}[0] + \text{nums}[1] + \text{nums}[2] = 1 + 7 + 3 = 11$,

右侧数之和 $\text{sum} = \text{nums}[4] + \text{nums}[5] = 5 + 6 = 11$, 二者相等。

示例 2:

输入: `nums = [1, 2, 3]`

输出: -1

解释:

数组中不存在满足此条件的中心下标。

示例 3:

输入: `nums = [2, 1, -1]`

输出: 0

解释:

中心下标是 0 。

左侧数之和 $\text{sum} = 0$, (下标 0 左侧不存在元素) ,

右侧数之和 $\text{sum} = \text{nums}[1] + \text{nums}[2] = 1 + -1 = 0$ 。

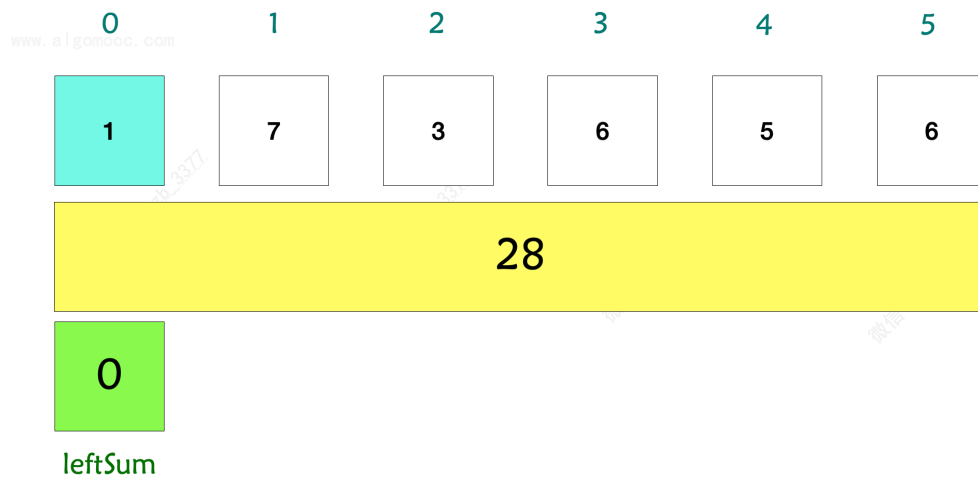
提示:

- `1 <= nums.length <= 10^4`
- `-1000 <= nums[i] <= 1000`

二、题目解析

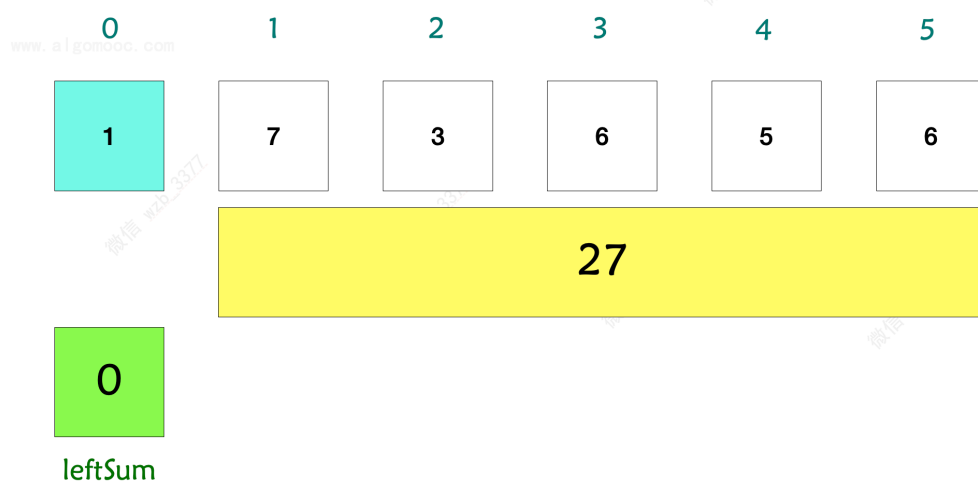
具体操作如下:

- 1、记数组的全部元素之和为 `sum`。
- 2、假设存在中心下标, 那么它的左侧所有元素相加的和是 `leftSum`, 一开始为 0。
- 3、从头到尾访问 `nums`, 查看 `i` 这个索引是否是中心下标。



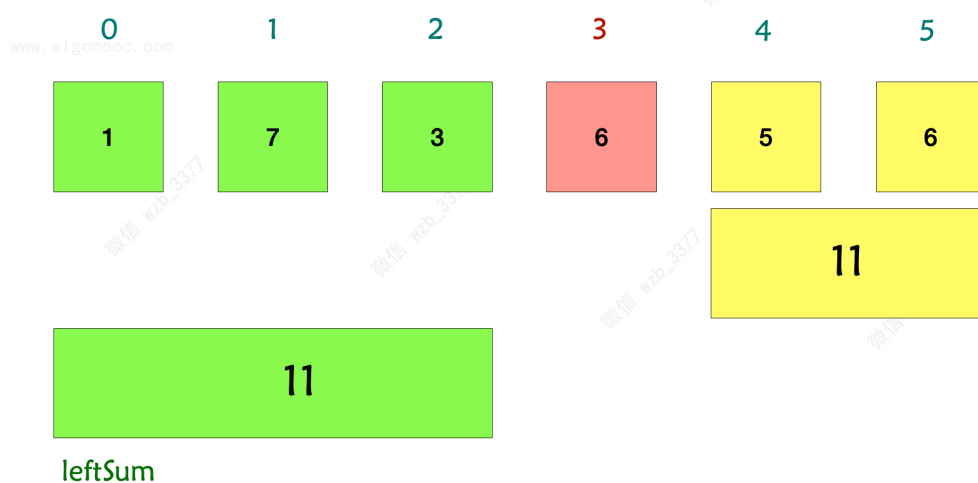
公众号：吴师兄学算法

4、在访问过程中，`i` 是否是中心下标取决于其左侧所有元素相加的和等于右侧所有元素相加的和，而右侧所有元素相加的和可以根据总和、`i` 左侧所有元素相加的和、当前元素的值获取到。



公众号：吴师兄学算法

5、如果发现其左侧所有元素相加的和等于右侧所有元素相加的和，找到中心下标了，就是 `i`。



公众号：吴师兄学算法

6、否则 `i` 不是中心下标，那么需要继续去访问，而开始访问下一个元素前，`nums[i]` 这个元素就要归于 `leftSum` 了。

7、遍历结束都没找到结果，说明不存在中心下标，返回 -1

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找数组的中心下标 (LeetCode 724) : https://leetcode.cn/problems/find-pivot-index/
class Solution {
    public int pivotIndex(int[] nums) {

        // 先去计算数组所有元素的和
        int sum = 0;

        // 先去计算数组所有元素的和
        for (int x : nums) {

            // 通过累加获取数组所有元素的和
            sum += x;
        }

        // 假设存在中心下标，那么它的左侧所有元素相加的和是 leftSum
        int leftSum = 0;

        // 从头到尾访问 nums，查看 i 这个索引是否是中心下标
        for (int i = 0; i < nums.length; i++) {

            // 那么 i 这个索引是否是中心下标呢？
            // 取决于其左侧所有元素相加的和等于右侧所有元素相加的和
            // 而右侧所有元素相加的和可以根据总和、i 左侧所有元素相加的和、当前元素的值获取到
            int rightSum = sum - nums[i] - leftSum;

            // 如果发现其左侧所有元素相加的和等于右侧所有元素相加的和
            if ( leftSum == rightSum ) {
                // 找到中心下标了，就是 i
                return i;
            }

            // 否则 i 不是中心下标
            // 那么需要继续去访问，而开始访问下一个元素前，nums[i] 这个元素就要归于 leftSum 了
            leftSum += nums[i];
        }

        // 遍历结束都没找到结果，说明不存在中心下标，返回 -1
        return -1;
    }
}
```

```
}
```

2、C++ 代码

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        // 先去计算数组所有元素的和
        int sum = 0;

        // 先去计算数组所有元素的和
        for (int x : nums) {

            // 通过累加获取数组所有元素的和
            sum += x;
        }

        // 假设存在中心下标，那么它的左侧所有元素相加的和是 leftSum
        int leftSum = 0;

        // 从头到尾访问 nums，查看 i 这个索引是否是中心下标
        for (int i = 0; i < nums.size(); i++) {

            // 那么 i 这个索引是否是中心下标呢？
            // 取决于其左侧所有元素相加的和等于右侧所有元素相加的和
            // 而右侧所有元素相加的和可以根据总和、i 左侧所有元素相加的和、当前元素的值获取到
            int rightSum = sum - nums[i] - leftSum;

            // 如果发现其左侧所有元素相加的和等于右侧所有元素相加的和
            if ( leftSum == rightSum ) {
                // 找到中心下标了，就是 i
                return i;
            }

            // 否则 i 不是中心下标
            // 那么需要继续去访问，而开始访问下一个元素前，nums[i] 这个元素就要归于 leftSum 了
            leftSum += nums[i];
        }

        // 遍历结束都没找到结果，说明不存在中心下标，返回 -1
        return -1;
    }
};
```

3、Python 代码

```
class solution:
    def pivotIndex(self, nums: List[int]) -> int:
        # 先去计算数组所有元素的和
        sum = 0

        # 先去计算数组所有元素的和
        for x in nums:
            # 通过累加获取数组所有元素的和
            sum += x

        # 假设存在中心下标，那么它的左侧所有元素相加的和是 leftSum
        leftSum = 0

        # 从头到尾访问 nums，查看 i 这个索引是否是中心下标
        for i in range(0, len(nums)):

            # 那么 i 这个索引是否是中心下标呢？
            # 取决于其左侧所有元素相加的和等于右侧所有元素相加的和
            # 而右侧所有元素相加的和可以根据总和、i 左侧所有元素相加的和、当前元素的值获取到
            rightSum = sum - nums[i] - leftSum

            # 如果发现其左侧所有元素相加的和等于右侧所有元素相加的和
            if leftSum == rightSum :
                # 找到中心下标了，就是 i
                return i

            # 否则 i 不是中心下标
            # 那么需要继续去访问，而开始访问下一个元素前，nums[i] 这个元素就要归于 leftSum 了
            leftSum += nums[i]

        # 遍历结束都没找到结果，说明不存在中心下标，返回 -1
        return -1
```

四、复杂度分析

- 时间复杂度：O(n)，其中 n 为数组的长度。
- 空间复杂度：O(1)。

第十二天

LeetCode 1248、统计「优美子数组」

一、题目描述

给你一个整数数组 `nums` 和一个整数 `k`。如果某个连续子数组中恰好有 `k` 个奇数数字，我们就认为这个子数组是「优美子数组」。

请返回这个数组中「优美子数组」的数目。

示例 1:

```
输入: nums = [1,1,2,1,1], k = 3
输出: 2
解释: 包含 3 个奇数的子数组是 [1,1,2,1] 和 [1,2,1,1]。
```

示例 2:

```
输入: nums = [2,4,6], k = 1
输出: 0
解释: 数列中不包含任何奇数，所以不存在优美子数组。
```

示例 3:

```
输入: nums = [2,2,2,1,2,2,1,2,2,2], k = 2
输出: 16
```

提示:

- `1 <= nums.length <= 50000`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= nums.length`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 统计「优美子数组」(LeetCode 1248)：https://leetcode.cn/problems/count-number-of-nice-subarrays/
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        // [1,1,0,1,1]
        // 统计和为 k 的子数组的数量
        int count = 0;
```



```

// 记录遍历到索引为 i 的这个元素时，前缀和的值是多少
int pre = 0;

// 利用哈希表，以前缀和为键，出现次数为对应的值，记录 pre[i] 出现的次数
HashMap <Integer,Integer> mp = new HashMap <>();

// 一开始，需要设置前缀和为 0 时，出现的次数为 1 次
// 这一行的作用就是为了应对 nums[0] +nums[1] + ... + nums[i] == k 这种情况
// 如数组 [1, 2, 3, 6]
// 这个数组的累加和数组为 [1, 3, 【6】 , 12]
// 如果 k = 6，假如 mp 中没有预先存储(0, 1)
// 那么来到累加和为 6 的位置时，这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
// 想去判断 mp.containsKey(pre - k)，这时 pre - k = 6 - 6 = 0
// 但 map 中没有 (0, 1)，
// 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
// 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
mp.put(0, 1);

// 开始从头到尾遍历 nums 数组，在遍历过程中，会执行两个操作
// 1、存储索引为 i 的这个元素时，前缀和的值是多少，并且把这个值出现的频次存储到 mp 中
// 2、判断之前有没有存储 pre - k 这种前缀和，如果有，说明 pre - k 到 pre 直接的那些元素值
之和就是 k
for (int i = 0; i < nums.length; i++) {

    // [1,3,2,3,5]
    // 奇数看成 1，偶数看成 0
    // 可以看成是下面这个数组
    // [1,1,0,1,1]
    // 存储索引为 i 的这个元素是 nums[i]
    // nums[i] 要么是 0，要么是 1
    // & 表示按位于操作
    // a & b 两个位都为1时，结果才为1
    // nums[i] = 1, nums[i] & 1 = 1，也就是原数组位置为奇数时，nums[i] & 1 的结果是
1
    // nums[i] = 0, nums[i] & 1 = 0，也就是原数组位置为偶数时，nums[i] & 1 的结果是
0

    // pre 是统计奇数的和
    // 存储索引为 i 的这个元素时，前缀和的值是多少
    pre += nums[i] & 1;

    // 判断之前有没有存储 pre - k 这种前缀和
    if (mp.containsKey(pre - k)) {

        // 如果有，说明 pre - k 到 pre 直接的那些元素值之和就是 k
        // 找到了一组，累加到 count 上
        count += mp.get(pre - k);

    }
}

```

```

        // 这个值出现的频次存储到 mp 中
        // getOrDefault: 当 Map 集合中有这个 key 时, 就使用这个 key 对应的 value 值
        // 如果没有就使用默认值 defaultValue
        mp.put(pre, mp.getOrDefault(pre, 0) + 1);
    }

    // 返回结果
    return count;
}

}

```

2、**C++** 代码

```

class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        // 统计和为 k 的子数组的数量
        int count = 0;

        // 记录遍历到索引为 i 的这个元素时, 前缀和的值是多少
        int pre = 0;

        // 利用哈希表, 以前缀和为键, 出现次数为对应的值, 记录 pre[i] 出现的次数
        unordered_map<int, int> mp;

        // 一开始, 需要设置前缀和为 0 时, 出现的次数为 1 次
        // 这一行的作用就是为了应对 nums[0] + nums[1] + ... + nums[i] == k 这种情况
        // 如数组 [1, 2, 3, 6]
        // 这个数组的累加和数组为 [1, 3, 【6】, 12]
        // 如果 k = 6, 假如 mp 中没有预先存储(0, 1)
        // 那么来到累加和为 6 的位置时, 这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
        // 想去判断 mp.containsKey(pre - k), 这时 pre - k = 6 - 6 = 0
        // 但 map 中没有 (0, 1),
        // 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
        // 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
        mp[0] = 1;

        // 开始从头到尾遍历 nums 数组, 在遍历过程中, 会执行两个操作
        // 1、存储索引为 i 的这个元素时, 前缀和的值是多少, 并且把这个值出现的频次存储到 mp 中
        // 2、判断之前有没有存储 pre - k 这种前缀和, 如果有, 说明 pre - k 到 pre 直接的那些元素值
        // 之和就是 k
        for (int i = 0; i < nums.size(); i++) {

            // 存储索引为 i 的这个元素时, 前缀和的值是多少
            pre += nums[i] & 1;

            // 判断之前有没有存储 pre - k 这种前缀和

```

```

        if (mp.find(pre - k) != mp.end()) {

            // 如果有, 说明 pre - k 到 pre 直接的那些元素值之和就是 k
            // 找到了一组, 累加到 count 上
            count += mp[pre - k];

        }

        // 这个值出现的频次存储到 mp 中
        mp[pre]++;
    }

    // 返回结果
    return count;
}

};

```

3、Python 代码

```

class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        # 统计和为 k 的子数组的数量
        count = 0

        # 记录遍历到索引为 i 的这个元素时, 前缀和的值是多少
        pre = 0

        # 利用哈希表, 以前缀和为键, 出现次数为对应的值, 记录 pre[i] 出现的次数
        mp = collections.defaultdict(int)

        # 一开始, 需要设置前缀和为 0 时, 出现的次数为 1 次
        # 这一行的作用就是为了应对 nums[0] + nums[1] + ... + nums[i] == k 这种情况
        # 如数组 [1, 2, 3, 6]
        # 这个数组的累加和数组为 [1, 3, 6, 12]
        # 如果 k = 6, 假如 mp 中没有预先存储(0, 1)
        # 那么来到累加和为 6 的位置时, 这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
        # 想去判断 mp.containsKey(pre - k), 这时 pre - k = 6 - 6 = 0
        # 但 map 中没有 (0, 1),
        # 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
        # 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
        mp[0] = 1

        # 开始从头到尾遍历 nums 数组, 在遍历过程中, 会执行两个操作
        # 1、存储索引为 i 的这个元素时, 前缀和的值是多少, 并且把这个值出现的频次存储到 mp 中
        # 2、判断之前有没有存储 pre - k 这种前缀和, 如果有, 说明 pre - k 到 pre 直接的那些元素值之和就是 k
        for i in range(len(nums)):

            # 存储索引为 i 的这个元素时, 前缀和的值是多少

```

```

pre += nums[i] & 1

# 判断之前有没有存储 pre - k 这种前缀和
# 如果有, 说明 pre - k 到 pre 直接的那些元素值之和就是 k
# 找到了一组, 累加到 count 上
# 利用 defaultdict 的特性, 当 presum - k 不存在时, 返回的是 0
count += mp[pre - k]

# 这个值出现的频次存储到 mp 中
# getOrDefault: 当 Map 集合中有这个 key 时, 就使用这个 key 对应的 value 值
# 如果没有就使用默认值 defaultValue
mp[pre] += 1

# 返回结果
return count

```

LeetCode 560、和为 K 的子数组

一、题目描述

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。

示例 1：

输入：nums = [1,1,1], k = 2
输出：2

示例 2：

输入：nums = [1,2,3], k = 3
输出：2

提示：

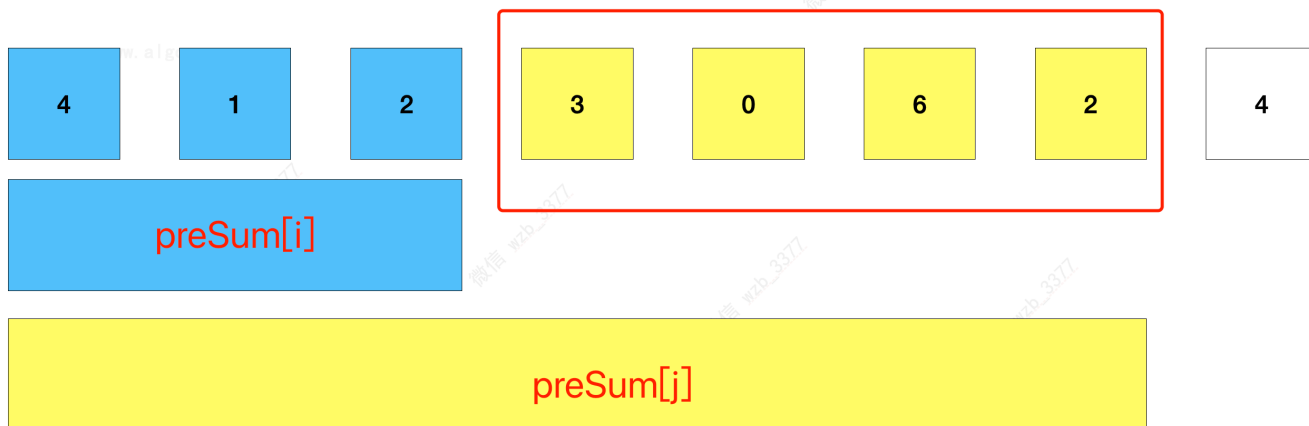
- `1 <= nums.length <= 2 * 104`
- `-1000 <= nums[i] <= 1000`
- `-107 <= k <= 107`

二、题目解析

补充知识点**前缀和**：前缀和指一个数组的某下标之前的所有数组元素的和（包含其自身）。

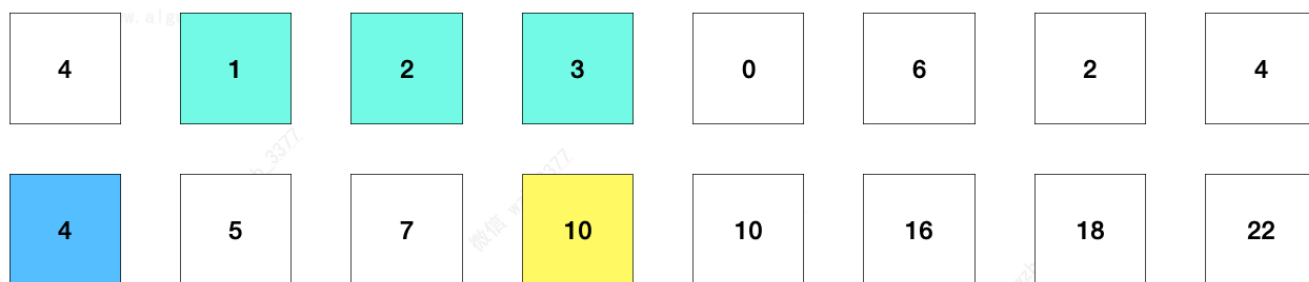
利用前缀和这种特点，可以快速计算某个区间内的和，比如前 `i` 个元素的前缀和为 `preSum[i] = num[0] + nums[1] + ... + nums[i]`，而前 `j` 个元素的前缀和为 `preSum[j] = num[0] + nums[1] + ... + nums[j]`。

那么区间 `[i, j]` 之间的子数组之和就是 `preSum[j] - preSum[i]`。



基于这种思路，可以先遍历一次数组，求出前缀和数组。

$$k = 6$$



题目这个时候就变成了需要寻找出多少个 `i` 和 `j` 的组合，使得 `[i, j]` 这个区间的和为 `k`。

```
class Solution {
    public int subarraySum(int[] nums, int k) {

        int len = nums.length;

        int[] preSum = new int[len + 1];

        preSum[0] = 0;

        for (int i = 0; i < len; i++) {
            preSum[i + 1] = preSum[i] + nums[i];
        }

        int count = 0;

        for (int i = 0; i < len; i++) {
            for (int j = i; j < len; j++) {
```

```

        if (preSum[j + 1] - preSum[i] == k) {
            count++;
        }
    }
}
return count;
}
}

```

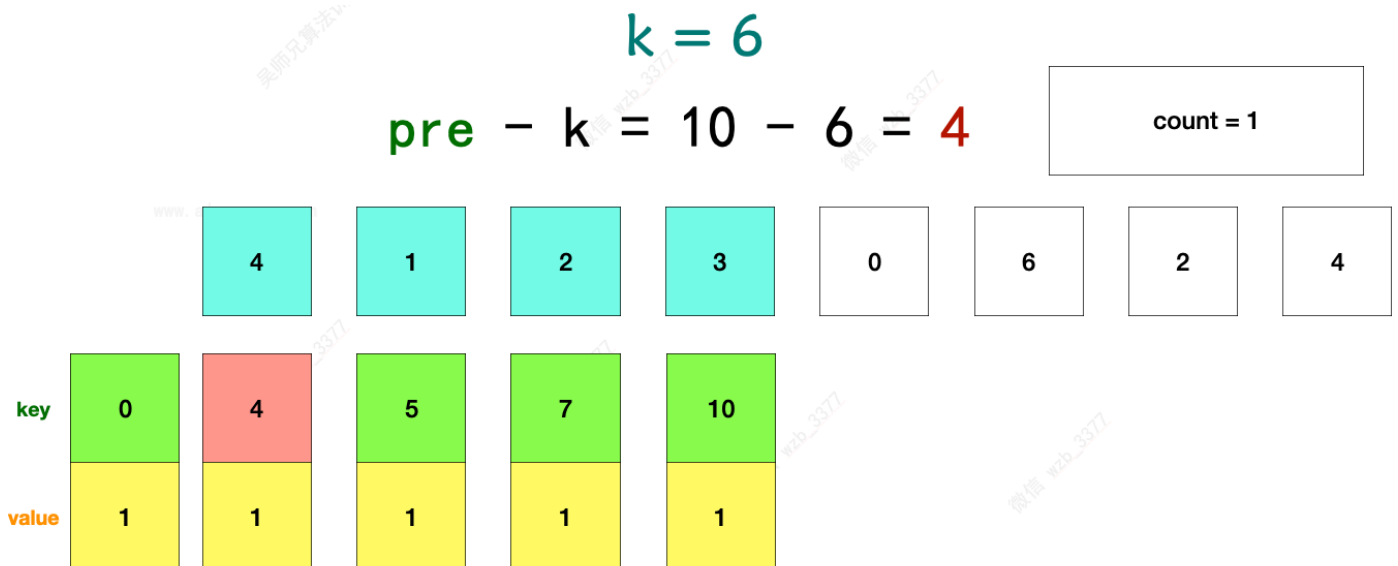
在计算过程中，有两个 for 循环发生了嵌套，时间复杂度来到了 $O(n^2)$ 级别。

需要优化。

事实上，我们不需要去计算出具体是哪两项的前缀和之差等于 k ，只需要知道等于 k 的前缀和之差出现的次数 **count**，所以我们可以遍历数组过程中，先去计算以 $nums[i]$ 结尾的前缀和 pre ，然后再去判断之前有没有存储 $pre - k$ 这种前缀和，如果有，那么 $pre - k$ 到 pre 这中间的元素和就是 k 了。

具体操作如下：

- 1、利用哈希表，以前缀和为键，出现次数为对应的值，记录 $pre[i]$ 出现的次数。
- 2、开始从头到尾遍历 $nums$ 数组，在遍历过程中，会执行两个操作。
- 3、存储索引为 i 的这个元素时，前缀和的值是多少，并且把这个值出现的频次存储到 mp 中。



这个哈希表以前缀和为键，出现次数为对应的值，记录 $pre[i]$ 出现的次数

公众号：吴师兄学算法

- 4、判断之前有没有存储 $pre - k$ 这种前缀和，如果有，说明 $pre - k$ 到 pre 直接的那些元素值之和就是 k 。
- 5、返回结果。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 和为 k 的子数组 (LeetCode 560) : https://leetcode.cn/problems/subarray-sum-equals-k/
class Solution {
    public int subarraySum(int[] nums, int k) {

        // 统计和为 k 的子数组的数量
        int count = 0;

        // 记录遍历到索引为 i 的这个元素时，前缀和的值是多少
        int pre = 0;

        // 利用哈希表，以前缀和为键，出现次数为对应的值，记录 pre[i] 出现的次数
        HashMap <Integer,Integer> mp = new HashMap <>();

        // 一开始，需要设置前缀和为 0 时，出现的次数为 1 次
        // 这一行的作用就是为了应对 nums[0] + nums[1] + ... + nums[i] == k 这种情况
        // 如数组 [1, 2, 3, 6]
        // 这个数组的累加和数组为 [1, 3, 【6】 , 12]
        // 如果 k = 6，假如 mp 中没有预先存储(0, 1)
        // 那么来到累加和为 6 的位置时，这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
        // 想去判断 mp.containsKey(pre - k)，这时 pre - k = 6 - 6 = 0
        // 但 map 中没有 (0, 1)，
        // 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
        // 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
        mp.put(0, 1);

        // 开始从头到尾遍历 nums 数组，在遍历过程中，会执行两个操作
        // 1、存储索引为 i 的这个元素时，前缀和的值是多少，并且把这个值出现的频次存储到 mp 中
        // 2、判断之前有没有存储 pre - k 这种前缀和，如果有，说明 pre - k 到 pre 直接的那些元素值之和就是 k
        for (int i = 0; i < nums.length; i++) {

            // 存储索引为 i 的这个元素时，前缀和的值是多少
            pre += nums[i];

            // 判断之前有没有存储 pre - k 这种前缀和
            if (mp.containsKey(pre - k)) {

                // 如果有，说明 pre - k 到 pre 直接的那些元素值之和就是 k
                // 找到了一组，累加到 count 上
                count += mp.get(pre - k);

            }

        }
    }
}
```

```

        // 这个值出现的频次存储到 mp 中
        // getOrDefault: 当 Map 集合中有这个 key 时, 就使用这个 key 对应的 value 值
        // 如果没有就使用默认值 defaultValue
        mp.put(pre, mp.getOrDefault(pre, 0) + 1);
    }

    // 返回结果
    return count;
}
}

```

2、C++ 代码

```

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {

        // 统计和为 k 的子数组的数量
        int count = 0;

        // 记录遍历到索引为 i 的这个元素时, 前缀和的值是多少
        int pre = 0;

        // 利用哈希表, 以前缀和为键, 出现次数为对应的值, 记录 pre[i] 出现的次数
        unordered_map<int, int> mp;

        // 一开始, 需要设置前缀和为 0 时, 出现的次数为 1 次
        // 这一行的作用就是为了应对 nums[0] + nums[1] + ... + nums[i] == k 这种情况
        // 如数组 [1, 2, 3, 6]
        // 这个数组的累加和数组为 [1, 3, 6, 12]
        // 如果 k = 6, 假如 mp 中没有预先存储(0, 1)
        // 那么来到累加和为 6 的位置时, 这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
        // 想去判断 mp.containsKey(pre - k), 这时 pre - k = 6 - 6 = 0
        // 但 map 中没有 (0, 1),
        // 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
        // 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
        mp[0] = 1;

        // 开始从头到尾遍历 nums 数组, 在遍历过程中, 会执行两个操作
        // 1、存储索引为 i 的这个元素时, 前缀和的值是多少, 并且把这个值出现的频次存储到 mp 中
        // 2、判断之前有没有存储 pre - k 这种前缀和, 如果有, 说明 pre - k 到 pre 直接的那些元素值之和就是 k
        for (int i = 0; i < nums.size(); i++) {

            // 存储索引为 i 的这个元素时, 前缀和的值是多少
            pre += nums[i];

            // 判断之前有没有存储 pre - k 这种前缀和
            if (mp.find(pre - k) != mp.end()) {

```



```

        // 如果有，说明 pre - k 到 pre 直接的那些元素值之和就是 k
        // 找到了一组，累加到 count 上
        count += mp[pre - k];

    }

    // 这个值出现的频次存储到 mp 中
    mp[pre]++;
}

// 返回结果
return count;

}

};

```

3、Python 代码

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        # 统计和为 k 的子数组的数量
        count = 0

        # 记录遍历到索引为 i 的这个元素时，前缀和的值是多少
        pre = 0

        # 利用哈希表，以前缀和为键，出现次数为对应的值，记录 pre[i] 出现的次数
        mp = collections.defaultdict(int)

        # 一开始，需要设置前缀和为 0 时，出现的次数为 1 次
        # 这一行的作用就是为了应对 nums[0] + nums[1] + ... + nums[i] == k 这种情况
        # 如数组 [1, 2, 3, 6]
        # 这个数组的累加和数组为 [1, 3, 6, 12]
        # 如果 k = 6，假如 mp 中没有预先存储(0, 1)
        # 那么来到累加和为 6 的位置时，这时 mp 中存储的就只有两个数据 (1, 1), (3, 1)
        # 想去判断 mp.containsKey(pre - k)，这时 pre - k = 6 - 6 = 0
        # 但 map 中没有 (0, 1)，
        # 因为这个时候忽略了从下标 0 累加到下标 i 等于 k 的情况
        # 仅仅是统计了从下标大于 0 到某个位置等于 k 的所有答案
        mp[0] = 1

        # 开始从头到尾遍历 nums 数组，在遍历过程中，会执行两个操作
        # 1、存储索引为 i 的这个元素时，前缀和的值是多少，并且把这个值出现的频次存储到 mp 中
        # 2、判断之前有没有存储 pre - k 这种前缀和，如果有，说明 pre - k 到 pre 直接的那些元素值之和就是 k
        for i in range(len(nums)):

            # 存储索引为 i 的这个元素时，前缀和的值是多少

```

```
pre += nums[i]

# 判断之前有没有存储 pre - k 这种前缀和
# 如果有, 说明 pre - k 到 pre 直接的那些元素值之和就是 k
# 找到了一组, 累加到 count 上
# 利用 defaultdict 的特性, 当 presum - k 不存在时, 返回的是 0
count += mp[pre - k]

# 这个值出现的频次存储到 mp 中
# getOrDefault: 当 Map 集合中有这个 key 时, 就使用这个 key 对应的 value 值
# 如果没有就使用默认值 defaultValue
mp[pre] += 1

# 返回结果
return count
```

四、复杂度分析

时间复杂度: $O(n)$, 其中 n 为数组的长度。我们遍历数组的时间复杂度为 $O(n)$, 中间利用哈希表查询删除的复杂度均为 $O(1)$, 因此总时间复杂度为 $O(n)$ 。

空间复杂度: $O(n)$, 其中 n 为数组的长度。哈希表在最坏情况下可能有 n 个不同的键值, 因此需要 $O(n)$ 的空间复杂度。

剑指 Offer 66. 构建乘积数组

一、题目描述

给定一个数组 $A[0,1,...,n-1]$, 请构建一个数组 $B[0,1,...,n-1]$, 其中 $B[i]$ 的值是数组 A 中除了下标 i 以外的元素的积, 即 $B[i]=A[0]\times A[1]\times...\times A[i-1]\times A[i+1]\times...\times A[n-1]$ 。

不能使用除法。

示例:

输入: $[1,2,3,4,5]$ 输出: $[120,60,40,30,24]$

提示:

- 所有元素乘积之和不会溢出 32 位整数
- `a.length <= 100000`

二、题目解析

按照正常的思路, 既然 $B[i]=A[0]\times A[1]\times...\times A[i-1]\times A[i+1]\times...\times A[n-1]$, 那么想要计算出 $B[i]$ 来那每次都去遍历数组 A , 把这些数字相乘就行了。

而数组 B 的长度为 n , 并且计算数组 B 中的每个元素时都需要完整的遍历一遍数组 A , 而数组 A 的长度为 n , 那么整体的时间复杂度就达到了 $O(N(2))$ 级别, 按照这个逻辑写出的代码提交会超时。

那么哪里可以优化呢?

上面的暴力解法中充斥着大量重复的计算!

比如数组 A 为 [1,2,3,4,5] 。

- 1、想要计算除了 2 以外的结果时，需要计算 $1 * 3 * 4 * 5$ 。
- 2、想要计算除了 3 以外的结果时，需要计算 $1 * 2 * 4 * 5$ 。

注意到，这两个计算过程都计算了 1 和 $4 * 5$ 。

所以，我们优化的方向就是去保存好计算的结果，避免重复计算。

1 出现在 2 和 3 的左侧， $4 * 5$ 出现在 2 和 3 的右侧，它们分别可以使用数组提前计算保存下来。

在公式 $B[i]=A[0] \times A[1] \times \dots \times A[i-1] \times A[i+1] \times \dots \times A[n-1]$ 中，实际上可以划分为两个部分，从 0 到 $i-1$ 和从 $i+1$ 到 $n-1$ ，因此，想要构建乘积数组后某下标对应元素的值，只要求出该下标对应原数组中其左边的元素的乘积和其右边的元素的乘积，最后将两个乘积再相乘即可。

具体操作如下：

数组 A 为 [1,2,3,4,5] 。

- 1、数组 left[i] 表示在数组 A 中下标为 i 的所有左边元素的乘积，如果左边没有元素，默认为 1。
- 2、数组 right[i] 表示在数组 A 中下标为 i 的所有右边元素的乘积，如果右边没有元素，默认为 1。
- 3、 $B[i] = left[i] * right[i]$ 。

	0	1	2	3	4
A	1	2	3	4	5
left	1	1	2	6	24
right	120	60	20	5	1
B					

公众号：吴师兄学算法

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
```

```
// 剑指 offer 66. 构建乘积数组:https://leetcode-cn.com/problems/gou-jian-cheng-ji-shu-zu-lcof/
class Solution {
    public int[] constructArr(int[] a) {

        // 边界判断
        if( a == null || a.length == 0 ) return a;

        // 获取数组 a 的长度
        int length = a.length;

        // 数组 leftA 表示数组 a 中每个元素左边所有元素的乘积
        // 比如 leftA[5] 表示数组 a 中下标为 5 的元素的左边所有元素的乘积
        // 即 leftA[5] = a[0] * a[1] * a[2] * a[3] * a[4]
        int[] leftA = new int[length];

        // 数组 rightA 表示数组 a 中每个元素右边所有元素的乘积
        // 比如 rightA[3] 表示数组 a 中下标为 3 的元素的右边所有元素的乘积
        // 即 rightA[3] = a[4] * a[5]
        int[] rightA = new int[length];

        // 由于任何数与 1 相乘都是它本身, 因此, 如果发现元素左边没有元素时, 默认值为 1
        // 这样就不会影响到后面的计算
        leftA[0] = 1;

        // 由于任何数与 1 相乘都是它本身, 因此, 如果发现元素右边没有元素时, 默认值为 1
        // 这样就不会影响到后面的计算
        rightA[length - 1] = 1;

        // 开始不断填充 leftA
        for( int i = 1 ; i < length ; i++ ){
            leftA[i] = leftA[ i - 1 ] * a[ i - 1 ];
        }

        // 开始不断填充 rightA
        for( int j = length - 2 ; j >= 0 ; j-- ){
            rightA[j] = rightA[ j + 1 ] * a[ j + 1 ];
        }

        // 数组 B 存放结果
        int[] B = new int[length];

        // 只求出该下标对应原数组中其左边的元素的乘积和其右边的元素的乘积
        // 最后将两个乘积再相乘即可
        for( int k = 0 ; k < length ; k++){
            B[k] = leftA[k] * rightA[k];
        }

        // 返回数组 B
    }
}
```

```
        return B;

    }

}
```

四、时间复杂度

- 1、时间复杂度 $O(N)$ ：其中 N 为数组长度，三轮遍历数组 a ，使用 $O(N)$ 时间。
- 2、空间复杂度 $O(N)$ ：额外创建了两个数组

第十三天

LeetCode 146、LRU缓存

一、题目描述

请你设计并实现一个满足 [LRU \(最近最少使用\) 缓存](#) 约束的数据结构。

实现 `LRUCache` 类：

- `LRUCache(int capacity)` 以 **正整数** 作为容量 `capacity` 初始化 LRU 缓存
- `int get(int key)` 如果关键字 `key` 存在于缓存中，则返回关键字的值，否则返回 `-1`。
- `void put(int key, int value)` 如果关键字 `key` 已经存在，则变更其数据值 `value`；如果不存在，则向缓存中插入该组 `key-value`。如果插入操作导致关键字数量超过 `capacity`，则应该 **逐出** 最久未使用的关键字。

函数 `get` 和 `put` 必须以 $O(1)$ 的平均时间复杂度运行。

示例：

输入 `["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]` `[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]` 输出 `[null, null, null, 1, null, -1, null, -1, 3, 4]` 解释 `LRUCache lRUCache = new LRUCache(2); lRUCache.put(1, 1); // 缓存是 {1=1} lRUCache.put(2, 2); // 缓存是 {1=1, 2=2} lRUCache.get(1); // 返回 1 lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3} lRUCache.get(2); // 返回 -1 (未找到) lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3} lRUCache.get(1); // 返回 -1 (未找到) lRUCache.get(3); // 返回 3 lRUCache.get(4); // 返回 4`

提示：

- `1 <= capacity <= 3000`
- `0 <= key <= 10000`
- `0 <= value <= 105`
- 最多调用 `2 * 105` 次 `get` 和 `put`

二、题目解析

三、参考代码

```
// 使用双向链表来实现
class ALNode{

    // 节点值
    public int val;
    // 当前节点的下一个节点
    public ALNode nextNode;
    // 当前节点的上一个节点
    public ALNode preNode;
    // 构造
    ALNode(int val){
        this.val = val;
    }
}

public class LRUCache {

    // 利用哈希表来存储元素
    public Map<Integer, Object[]> maps;
    // 为了方便使用，默认双向链表有两个节点
    // 这样，哪怕只有一个节点时，依旧有上节点、下节点
    // 头节点
    public ALNode head;
    // 尾节点
    public ALNode tail;
    // 实际容量，意味着最多存储这么多节点
    private int capacity;
    // 长度
    public int length;

    public LRUCache(int capacity) {
        // 初始化 head
        head = new ALNode(-1);
        // 初始化 tail
        tail = new ALNode(-1);
        // 初始化 maps
        maps = new HashMap<>();
        // 初始化 capacity
        this.capacity = capacity;
        // 连接 head 和 tail
        head.nextNode = tail;
        tail.preNode = head;
    }

    public int get(int key) {
```

```

// 判断哈希表中是否存储了 key
// 如果存在，不仅需要返回 key 的 value
// 同样，需要操作双向链表，使得
// 1、当前这个 key 对应的节点放到链表的最前面，即 head 的下一个节点
// 2、其余节点维持原来的顺序
if (maps.containsKey(key)) {

    // 获取节点值
    ALNode cur = (ALNode) maps.get(key)[0];

    // 获取当前节点的上一个节点
    ALNode preNode = cur.preNode;

    // 获取当前节点的下一个节点
    ALNode nextNode = cur.nextNode;

    // 让这两个上下节点连接起来，cur 也就消失了
    preNode.nextNode = nextNode;

    nextNode.preNode = preNode;

    // 把 cur 挪到 head 的 nextNode 位置
    // 1、先获取原先 head 的 nextNode 节点
    ALNode tmp = head.nextNode;

    // 2、修改 head 的 nextNode 节点为 cur
    head.nextNode = cur;

    // 3、cur 重新连接上 tmp
    cur.nextNode = tmp;

    // 4、tmp 也连接上 cur
    tmp.preNode = cur;

    // 5、cur 上一个节点指向 head
    cur.preNode = head;

    // 最后才返回 map 的值
    return (Integer) maps.get(key)[1];
}

// 否则返回 -1
return -1;
}

public void put(int key, int value) {

    // 判断哈希表中是否存储了 key

```

```

// 如果存在，不仅需要返回 key 的 value
// 同样，需要操作双向链表，使得
// 1、key 对应的节点值 value 需要修改，采取节点替换的操作
// 2、这个节点需要挪到最前面
if (maps.containsKey(key)) {

    // 获取节点值
    ALNode cur = (ALNode) maps.get(key)[0];

    // 获取当前节点的上一个节点
    ALNode preNode = cur.preNode;

    // 获取当前节点的下一个节点
    ALNode nextNode = cur.nextNode;

    // 让这两个上下节点连接起来，cur 也就消失了
    preNode.nextNode = nextNode;

    nextNode.preNode = preNode;

    // 把 cur 挪到 head 的 nextNode 位置
    // 1、先获取原先 head 的 nextNode 节点
    ALNode tmp = head.nextNode;

    // 2、修改 head 的 nextNode 节点为 cur
    head.nextNode = cur;

    // 3、cur 重新连接上 tmp
    cur.nextNode = tmp;

    // 4、tmp 也连接上 cur
    tmp.preNode = cur;

    // 5、cur 上一个节点指向 head
    cur.preNode = head;

    // 更新节点
    maps.put(key, new Object[]{cur, value});

    return;
}

// 如果哈希表中不包含 key 对应的节点，那么需要判断缓存是否满了
// 如果满了，需要把最后一个节点删除掉
if (length == capacity) {

    // 即将被删除的节点
    ALNode delNode = tail.preNode;

```



```

        // 即将被删除的节点的上一个节点
        ALNode delPreNode = tail.preNode.preNode;

        // delPreNode 跳过了 delNode
        delPreNode.nextNode = tail;

        tail.preNode = delPreNode;

        // 哈希表移除 delNode 对应的值
        maps.remove(delNode.val);

        // 链表的长度更新一下
        length--;
    }

    // 再把 key 节点添加到最前面去
    ALNode cur = new ALNode(key);

    // 把 cur 挪到 head 的 nextNode 位置
    // 1、先获取原先 head 的 nextNode 节点
    ALNode tmp = head.nextNode;

    // 2、修改 head 的 nextNode 节点为 cur
    head.nextNode = cur;

    // 3、cur 重新连接上 tmp
    cur.nextNode = tmp;

    // 4、tmp 也连接上 cur
    tmp.preNode = cur;

    // 5、cur 上一个节点指向 head
    cur.preNode = head;

    // 更新哈希表
    maps.put(key, new Object[]{cur, value});

    // 更新 length
    length++;

    return;
}
}

```

C++

```

class ALNode {
public:
    int val;

```

```

ALNode* nextNode;
ALNode* preNode;

ALNode(int val) {
    this->val = val;
    nextNode = nullptr;
    preNode = nullptr;
}

};

class LRUCache {
public:
    unordered_map<int, pair<ALNode*, int>> maps;
    ALNode* head;
    ALNode* tail;
    int capacity;
    int length;

    LRUCache(int capacity) {
        head = new ALNode(-1);
        tail = new ALNode(-1);
        this->capacity = capacity;
        length = 0;
        head->nextNode = tail;
        tail->preNode = head;
    }

    int get(int key) {
        if (maps.count(key)) {
            ALNode* cur = maps[key].first;
            ALNode* preNode = cur->preNode;
            ALNode* nextNode = cur->nextNode;
            preNode->nextNode = nextNode;
            nextNode->preNode = preNode;
            ALNode* tmp = head->nextNode;
            head->nextNode = cur;
            cur->nextNode = tmp;
            tmp->preNode = cur;
            cur->preNode = head;
            return maps[key].second;
        }
        return -1;
    }

    void put(int key, int value) {
        if (maps.count(key)) {
            ALNode* cur = maps[key].first;
            ALNode* preNode = cur->preNode;
            ALNode* nextNode = cur->nextNode;

```

```

        preNode->nextNode = nextNode;
        nextNode->preNode = preNode;
        ALNode* tmp = head->nextNode;
        head->nextNode = cur;
        cur->nextNode = tmp;
        tmp->preNode = cur;
        cur->preNode = head;
        maps[key].second = value;
        return;
    }
    if (length == capacity) {
        ALNode* delNode = tail->preNode;
        ALNode* delPreNode = delNode->preNode;
        delPreNode->nextNode = tail;
        tail->preNode = delPreNode;
        maps.erase(delNode->val);
        length--;
        delete delNode;
    }
    ALNode* cur = new ALNode(key);
    ALNode* tmp = head->nextNode;
    head->nextNode = cur;
    cur->nextNode = tmp;
    tmp->preNode = cur;
    cur->preNode = head;
    maps[key] = make_pair(cur, value);
    length++;
}
};

```

Python

```

class ALNode:
    def __init__(self, val):
        self.val = val
        self.nextNode = None
        self.preNode = None

class LRUCache:
    def __init__(self, capacity: int):
        self.maps = {}
        self.head = ALNode(-1)
        self.tail = ALNode(-1)
        self.capacity = capacity
        self.length = 0
        self.head.nextNode = self.tail
        self.tail.preNode = self.head

    def get(self, key: int) -> int:

```

```

    if key in self.maps:
        cur = self.maps[key][0]
        preNode = cur.preNode
        nextNode = cur.nextNode
        preNode.nextNode = nextNode
        nextNode.preNode = preNode
        tmp = self.head.nextNode
        self.head.nextNode = cur
        cur.nextNode = tmp
        tmp.preNode = cur
        cur.preNode = self.head
        return self.maps[key][1]
    return -1

def put(self, key: int, value: int) -> None:
    if key in self.maps:
        cur = self.maps[key][0]
        preNode = cur.preNode
        nextNode = cur.nextNode
        preNode.nextNode = nextNode
        nextNode.preNode = preNode
        tmp = self.head.nextNode
        self.head.nextNode = cur
        cur.nextNode = tmp
        tmp.preNode = cur
        cur.preNode = self.head
        self.maps[key] = [cur, value]
        return
    if self.length == self.capacity:
        delNode = self.tail.preNode
        delPreNode = self.tail.preNode.preNode
        delPreNode.nextNode = self.tail
        self.tail.preNode = delPreNode
        del self.maps[delNode.val]
        self.length -= 1
    cur = ALNode(key)
    tmp = self.head.nextNode
    self.head.nextNode = cur
    cur.nextNode = tmp
    tmp.preNode = cur
    cur.preNode = self.head
    self.maps[key] = [cur, value]
    self.length += 1

```

LeetCode 36、有效的数独

一、题目描述

请你判断一个 9 x 9 的数独是否有效。只需要 根据以下规则 ， 验证已经填入的数字是否有效即可。

- 1. 数字 1-9 在每一行只能出现一次。
- 2. 数字 1-9 在每一列只能出现一次。
- 3. 数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。（请参考示例图）

注意：

- 一个有效的数独（部分已被填充）不一定是可解的。
- 只需要根据以上规则，验证已经填入的数字是否有效即可。
- 空白格用 '.' 表示。

示例 1：

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

img

```
输入: board =
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[[".","9","8",".",".",".",".","6","."],
["8",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".","1"],
["7",".",".",".","2",".",".",".","6"],
[[".","6",".",".",".",".","2","8","."],
[[".","","","4","1","9",".","","5"],
[[".","","","","8",".","","7","9"]]
输出: true
```

示例 2：

```
输入: board =
[["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[["9","8",".",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8","3",".","1"],
["7",".","2",".","6"],
[["6",".","2","8","."],
[["4","1","9",".","5"],
[["8",".","7","9"]]
```

输出: false

解释: 除了第一行的第一个数字从 5 改为 8 以外, 空格内其他数字均与 示例1 相同。 但由于位于左上角的 3x3 宫内有 两个 8 存在, 因此这个数独是无效的。

提示:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` 是一位数字 (1-9) 或者 `'.'`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的数独 (LeetCode 36) : https://leetcode.cn/problems/valid-sudoku/
class Solution {
    public boolean isValidSudoku(char[][] board) {

        // 使用哈希表记录每一行每个数字出现的次数
        // 由于数字是从 1 - 9 , 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key , 数组的元素值作为哈希表的 value
        int[][] rows = new int[9][9];

        // 使用哈希表记录每一列每个数字出现的次数
        // 由于数字是从 1 - 9 , 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key , 数组的元素值作为哈希表的 value
        int[][] columns = new int[9][9];

        // 使用哈希表记录每一个【小九宫格】每个数字出现的次数
        // 由于数字是从 1 - 9 , 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key , 数组的元素值作为哈希表的 value
        int[][][] subboxes = new int[3][3][9];
```

```

// 一行行一列列去访问二维矩阵 board 的每个元素
for (int i = 0; i < 9; i++) {

    for (int j = 0; j < 9; j++) {

        // 获取当前小方格的字符
        char c = board[i][j];

        // 如果不是空白格而是数字
        if (c != '.') {

            // 由于数组的索引是从 0 开始的，而小方格的值是从 1 开始的
            // 因此，索引 0 代表 key 为 1
            // 索引 1 代表 key 为 2
            // 所以，c 为 9 时，存储在索引为 8 的位置
            // c 为 6 时，存储在索引为 5 的位置
            int index = c - '0' - 1;

            // 第 i 行数字 index 出现的次数增加 1 次
            rows[i][index]++;

            // 第 j 列数字 index 出现的次数增加 1 次
            columns[j][index]++;

            // 总共有 9 个 3 * 3 的小九宫格
            // 第一行有 3 个，包含了第 0 个、第 1 个、第 2 个
            // 第二行有 3 个，包含了第 3 个、第 4 个、第 5 个
            // 第三行有 3 个，包含了第 6 个、第 7 个、第 8 个
            // (i / 3, j / 3) 表示是第几个小九宫格
            // 当前这个小九宫格里面数字 index 出现的次数增加 1 次
            subboxes[i / 3][j / 3][index]++;

            // 如果发现如下任何一种情况，都说明是无效的
            // 1、这一行统计出 index 数字的次数大于 1 个
            // 2、这一列统计出 index 数字的次数大于 1 个
            // 3、这一小九宫格统计出 index 数字的次数大于 1 个
            if (rows[i][index] > 1 || columns[j][index] > 1 || subboxes[i / 3]
[j / 3][index] > 1) {

                // 返回 false
                return false;

            }

        }

    }

}

// 是有效的，返回 true
return true;
}

```

```
}
```

2、**C++** 代码

```
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {

        // 使用哈希表记录每一行每个数字出现的次数
        // 由于数字是从 1 - 9 ， 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key ， 数组的元素值作为哈希表的 value
        int rows[9][9];
        memset(rows,0,sizeof(rows));

        // 使用哈希表记录每一列每个数字出现的次数
        // 由于数字是从 1 - 9 ， 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key ， 数组的元素值作为哈希表的 value
        int columns[9][9];
        memset(columns,0,sizeof(columns));

        // 使用哈希表记录每一个【小九宫格】每个数字出现的次数
        // 由于数字是从 1 - 9 ， 因此可以采取数组的形式来表示哈希表
        // 即利用数组的索引作为哈希表的 key ， 数组的元素值作为哈希表的 value
        int subboxes[3][3][9];
        memset(subboxes,0,sizeof(subboxes));
        // 一行行一列列去访问二维矩阵 board 的每个元素
        for (int i = 0 ; i < 9 ; i++) {

            for (int j = 0 ; j < 9 ; j++) {

                // 获取当前小方格的字符
                char c = board[i][j];

                // 如果不是空白格而是数字
                if (c != '.') {

                    // 由于数组的索引是从 0 开始的，而小方格的值是从 1 开始的
                    // 因此，索引 0 代表 key 为 1
                    // 索引 1 代表 key 为 2
                    // 所以，c 为 9 时，存储在索引为 8 的位置
                    // c 为 6 时，存储在索引为 5 的位置
                    int index = c - '0' - 1;

                    // 第 i 行数字 index 出现的次数增加 1 次
                    rows[i][index]++;

                    // 第 j 列数字 index 出现的次数增加 1 次
                    columns[j][index]++;

                    // 总共有 9 个 3 * 3 的小九宫格
```


LeetCode 496、下一个更大元素 I

一、题目描述

`nums1` 中数字 `x` 的下一个更大元素 是指 `x` 在 `nums2` 中对应位置 右侧 的 第一个 比 `x` 大的元素。

给你两个 没有重复元素 的数组 `nums1` 和 `nums2`，下标从 0 开始计数，其中 `nums1` 是 `nums2` 的子集。

对于每个 `0 <= i < nums1.length`，找出满足 `nums1[i] == nums2[j]` 的下标 `j`，并且在 `nums2` 确定 `nums2[j]` 的下一个更大元素。如果不存在下一个更大元素，那么本次查询的答案是 `-1`。

返回一个长度为 `nums1.length` 的数组 `ans` 作为答案，满足 `ans[i]` 是如上所述的 下一个更大元素。

示例 1:

输入: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`. 输出: `[-1,3,-1]` 解释: `nums1` 中每个值的下一个更大元素如下所述: - 4，用加粗斜体标识，`nums2 = [1,3,4,2]`。不存在下一个更大元素，所以答案是 -1。 - 1，用加粗斜体标识，`nums2 = [1,3,4,2]`。下一个更大元素是 3。 - 2，用加粗斜体标识，`nums2 = [1,3,4,2]`。不存在下一个更大元素，所以答案是 -1。

示例 2:

输入: `nums1 = [2,4]`, `nums2 = [1,2,3,4]`. 输出: `[3,-1]` 解释: `nums1` 中每个值的下一个更大元素如下所述: - 2，用加粗斜体标识，`nums2 = [1,2,3,4]`。下一个更大元素是 3。 - 4，用加粗斜体标识，`nums2 = [1,2,3,4]`。不存在下一个更大元素，所以答案是 -1。

提示:

- `1 <= nums1.length <= nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 104`
- `nums1` 和 `nums2` 中所有整数 互不相同
- `nums1` 中的所有整数同样出现在 `nums2` 中

二、题目解析

三、参考代码

Java

```
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        // 使用 HashMap 存储 nums2 中每个元素右边的第一个更大的值
        // 键为 nums2 中的元素，值为这个元素右边的第一个更大的值
        HashMap<Integer, Integer> res = new HashMap<>();

        // 使用一个单调递增栈
        Stack<Integer> stack = new Stack<>();

        // 从后向前访问 nums2 中的元素
        for (int i = nums2.length - 1; i >= 0; i--) {
            int num = nums2[i];
```

```

        // 在访问过程中维护单调递增栈的性质
        // 不断比较栈顶元素和当前元素 num
        // 1、直到找到一个比 num 更大的元素为止
        // 2、或者栈为空
        while (!stack.isEmpty() && num >= stack.peek()) {
            // 出栈操作
            stack.pop();
        }

        // 1、如果栈不为空，栈顶元素就是比 num 更大的元素
        // 存放栈顶元素的值到哈希集合 res 中
        // 2、如果栈为空，说明在 num 的右侧没有任何一个元素比它更大
        // 存放 -1 到哈希集合 res 中
        res.put(num, stack.isEmpty() ? -1 : stack.peek());

        // 将当前元素入栈
        stack.push(num);
    }

    // 初始化结果数组
    int[] ans = new int[nums1.length];

    // 由于两个数组都是没有重复元素的数组
    // nums1 是 nums2 的子集
    for (int i = 0; i < nums1.length; i++) {
        // 从哈希集合 res 中找出对应元素的值
        ans[i] = res.get(nums1[i]);
    }

    // 返回结果
    return ans;
}
}

```

C++

```

class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        // 使用 unordered_map 存储 nums2 中每个元素右边的第一个更大的值
        // 键为 nums2 中的元素，值为这个元素右边的第一个更大的值
        unordered_map<int, int> res;

        // 使用一个单调递增栈
        stack<int> stack;

        // 从后向前访问 nums2 中的元素
        for (int i = nums2.size() - 1; i >= 0; i--) {
            int num = nums2[i];

```

```

        // 在访问过程中维护单调递增栈的性质
        // 不断比较栈顶元素和当前元素 num
        // 1、直到找到一个比 num 更大的元素为止
        // 2、或者栈为空
        while (!stack.empty() && num >= stack.top()) {
            // 出栈操作
            stack.pop();
        }

        // 1、如果栈不为空，栈顶元素就是比 num 更大的元素
        // 存放栈顶元素的值到哈希集合 res 中
        // 2、如果栈为空，说明在 num 的右侧没有任何一个元素比它更大
        // 存放 -1 到哈希集合 res 中
        res[num] = stack.empty() ? -1 : stack.top();

        // 将当前元素入栈
        stack.push(num);
    }

    // 初始化结果数组
    vector<int> ans(nums1.size());

    // 由于两个数组都是没有重复元素的数组
    // nums1 是 nums2 的子集
    for (int i = 0; i < nums1.size(); i++) {
        // 从哈希集合 res 中找出对应元素的值
        ans[i] = res[nums1[i]];
    }

    // 返回结果
    return ans;
}
};

```

Python

```

class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:

        # 先计算 nums2 中每个元素右边的第一个更大的值
        # 结果存放到一个哈希集合里面
        # key 是 nums2 中的元素
        # value 是这个元素右边的第一个更大的值
        res = {}

        # 设置一个栈
        # 这个栈是一个单调递增栈
        stack = []

```

```

# 从后向前访问 nums 中的元素
for num in reversed(nums2):

    # 在访问过程中
    # 维护单调递增栈的性质
    # 不断的去拿栈顶元素和当前元素 num 比较
    # 1、直到找到一个比 num 更大的元素为止
    # 2、或者栈为空位置
    while stack and num >= stack[-1]:
        # 出栈操作
        stack.pop()

    # 1、如果栈不为空，那么此时的栈顶元素就是一个比 num 更大的元素
    # 存放栈顶元素值到哈希集合 res 中
    # 2、如果栈为空，说明在 num 的右侧没有任何一个元素比它更大
    # 存放 -1 到哈希集合 res 中
    res[num] = stack[-1] if stack else -1

    # 把当前元素加入到栈中
    stack.append(num)

# 初始结果数组
ans = []

# 1、由于两个数组都是没有重复元素的数组
# 2、nums1 是 nums2 的子集
for num in nums1 :
    # 那么就去哈希集合 res 中找出对应元素的值来
    ans.append(res[num])

# 返回结果
return ans

```

LeetCode 49、字母异位词分组

一、题目描述

给你一个字符串数组，请你将 **字母异位词** 组合在一起。可以按任意顺序返回结果列表。

字母异位词 是由重新排列源单词的字母得到的一个新单词，所有源单词中的字母通常恰好只用一次。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"] 输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

示例 2:

输入: strs = [""] 输出: [[""]]

示例 3:

输入: strs = ["a"] 输出: [["a"]]

提示:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` 仅包含小写字母

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 字母异位词分组 (LeetCode 49) : https://leetcode.cn/problems/group-anagrams/
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {

        // 互为字母异位词的两个字符串包含的字母相同
        // 因此两个字符串中的相同字母出现的次数一定是相同的
        // 可以将每个字母出现的次数使用字符串表示, 作为哈希表的键
        Map<String, List<String>> map = new HashMap<String, List<String>>();

        for (String str : strs) {

            // 由于都是小写字母, 因此直接用 26 个长度的数组代替原来的 HashMap , 比直接使用
            // HashMap 更优秀
            // counts 代表每个小写字母出现的频次
            int[] counts = new int[26];

            // 利用 for 循环, 统计 str 当中每个字母出现的频次
            for (int i = 0; i < str.length(); i++) {

                // 对于数组类型, 其下标为 int 类型
                // 可以直接使用 char 类型变量, 默认强制转换, 存储的就是字母对应的 ASCII 码
                // 比如 str.charAt(i) 是 b 字符, 那么 b - a = 1, 即 counts[1] 表示记录 b 出
                // 现的频次
                counts[str.charAt(i) - 'a']++;

            }

            // String 类是不可变类, 即一旦一个 String 对象被创建以后, 包含在这个对象中的字符序列是
            // 不可改变的, 直至这个对象被销毁。
            // Java 提供了两个可变字符串类 StringBuffer 和 StringBuilder, 中文翻译为“字符串缓冲
            // 区”
```

```

// StringBuffer: 线程安全
// StringBuilder: 线程不安全
StringBuffer sb = new StringBuffer();

// 将每个出现次数大于 0 的字母和出现次数按顺序拼接成字符串, 作为哈希表的键
for (int i = 0; i < 26; i++) {

    if (counts[i] != 0) {

        // 先记录字母, 比如记录了字母 k
        sb.append((char) ('a' + i));
        // 再记录次数, 比如记录了次数 9
        sb.append(counts[i]);
    }
}

// 转换为字符串的形式, 比如为 a1k9x7
String key = sb.toString();

// 在哈希表 map 当中找出这个 key 对应的字符串 str 来
// 1、如果有这个 key, 那么这个 key 对应的 数组 会新增一个 str 进去
// 2、如果没有这个 key, 那么会初始化一个数组, 用来新增这个 str
List<String> list = map.getDefault(key, new ArrayList<String>());

// str 加入到 list 当中
list.add(str);

// map 进行更新
map.put(key, list);

}

// 返回结果
return new ArrayList<List<String>>(map.values());
}
}

```

2、**C++** 代码

```

class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        // 互为字母异位词的两个字符串包含的字母相同
        // 因此两个字符串中的相同字母出现的次数一定是相同的
        // 可以将每个字母出现的次数使用字符串表示, 作为哈希表的键
        unordered_map<string, vector<string>> mp;

        for (string& str : strs) {
            // counts 代表每个小写字母出现的频次

```

```

vector<int> counts(26);

// 利用 for 循环, 统计 str 当中每个字母出现的频次
for (char c : str) {
    counts[c - 'a']++;
}

// 将每个出现次数大于 0 的字母和出现次数按顺序拼接成字符串, 作为哈希表的键
string key;
for (int count : counts) {
    key += '#';
    key += to_string(count);
}

// 在哈希表 mp 当中找出这个 key 对应的字符串 str 来
// 1、如果有这个 key, 那么这个 key 对应的 数组 会新增一个 str 进去
// 2、如果没有这个 key, 那么会初始化一个数组, 用来新增这个 str
mp[key].emplace_back(str);
}

// 返回结果
vector<vector<string>> res;
for (auto& p : mp) {
    res.emplace_back(p.second);
}
return res;
}
};

```

3、Python 代码

```

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        # 互为字母异位词的两个字符串包含的字母相同
        # 因此两个字符串中的相同字母出现的次数一定是相同的
        # 可以将每个字母出现的次数使用字符串表示, 作为哈希表的键
        mp = collections.defaultdict(list)

        for s in strs:
            # counts 代表每个小写字母出现的频次
            counts = [0] * 26

            # 利用 for 循环, 统计 str 当中每个字母出现的频次
            for c in s:
                counts[ord(c) - ord('a')] += 1

            # 将每个出现次数大于 0 的字母和出现次数按顺序拼接成字符串, 作为哈希表的键
            key = ''.join(['#' + str(count) for count in counts])

```



```

# 在哈希表 mp 当中找出这个 key 对应的字符串 str 来
# 1、如果有这个 key，那么这个 key 对应的 数组 会新增一个 str 进去
# 2、如果没有这个 key，那么会初始化一个数组，用来新增这个 str
mp[key].append(s)

# 返回结果
return list(mp.values())

```

```

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        # 互为字母异位词的两个字符串包含的字母相同
        # 因此两个字符串中的相同字母出现的次数一定是相同的
        # 可以将每个字母出现的次数使用字符串表示，作为哈希表的键
        mp = collections.defaultdict(list)

        for s in strs:
            # counts 代表每个小写字母出现的频次
            counts = [0] * 26

            # 利用 for 循环，统计 str 当中每个字母出现的频次
            for c in s:
                counts[ord(c) - ord('a')] += 1

            # 将每个出现次数大于 0 的字母和出现次数按顺序拼接成字符串，作为哈希表的键
            key = ''.join(['#' + str(count) for count in counts])

            # 在哈希表 mp 当中找出这个 key 对应的字符串 str 来
            # 1、如果有这个 key，那么这个 key 对应的 数组 会新增一个 str 进去
            # 2、如果没有这个 key，那么会初始化一个数组，用来新增这个 str
            mp[key].append(s)

        # 返回结果
        return list(mp.values())

```

LeetCode 697、数组的度

一、题目描述

给定一个非空且只包含非负数的整数数组 `nums`，数组的 **度** 的定义是指数组里任一元素出现频数的最大值。

你的任务是在 `nums` 中找到与 `nums` 拥有相同大小的度的最短连续子数组，返回其长度。

示例 1：

输入: `nums = [1,2,2,3,1]`

输出: 2

解释:

输入数组的度是 2，因为元素 1 和 2 的出现频数最大，均为 2。

连续子数组里面拥有相同度的有如下所示：

`[1, 2, 2, 3, 1]`，`[1, 2, 2, 3]`，`[2, 2, 3, 1]`，`[1, 2, 2]`，`[2, 2, 3]`，`[2, 2]`

最短连续子数组 `[2, 2]` 的长度为 2，所以返回 2。

示例 2:

输入: `nums = [1,2,2,3,1,4,2]`

输出: 6

解释:

数组的度是 3，因为元素 2 重复出现 3 次。

所以 `[2,2,3,1,4,2]` 是最短子数组，因此返回 6。

提示:

- `nums.length` 在 1 到 50,000 范围内。
- `nums[i]` 是一个在 0 到 49,999 范围内的整数。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 数组的度 (LeetCode 697) : https://leetcode.cn/problems/degree-of-an-array/submissions/
class Solution {
    public int findShortestSubArray(int[] nums) {

        // 1、key 为数组中的值
        // 2、value 为一个数组
        // 2.1 数组的第一个元素为 key 出现的次数
        // 2.2 数组的第二个元素为 key 第一次出现的位置
        // 2.3 数组的第三个元素为 key 最后一次出现的位置
        Map<Integer,int[]> map = new HashMap<Integer,int[]>();

        // 遍历数组，统计每个元素出现的次数、第一次出现的位置、最后一次出现的位置
        // 把结果更新到 map 中
        for( int i = 0 ; i < nums.length ; i++ ){

            // 如果哈希表 map 中已经存在当前这个元素的 key
            // 说明在遇到 nums[i] 之前已经遇到了 nums[i] 这个值
```

```

        if(map.containsKey(nums[i])){

            // 更新这个元素出现的次数
            map.get(nums[i])[0]++;

            // 更新这个元素最后一次出现的位置
            map.get(nums[i])[2] = i;

            // 如果哈希表 map 中没有存在当前这个元素的 key
            // 说明现在是第一次遇到 nums[i] 这个值
        }else{

            // 更新这个元素出现的次数为 1
            // 更新这个元素第一次出现的位置为 i
            // 更新这个元素最后一次出现的位置为 i
            map.put(nums[i], new int[]{1,i,i});
        }
    }

    // 记录元素出现的最多次数
    int maxNum = 0;

    // 记录符合要求的元素并且前后位置差最小的数的长度
    int minLen = 0;

    // 遍历哈希表 map ， 找到元素出现次数最多， 且前后位置差最小的数
    for(Map.Entry<Integer, int[]> entry : map.entrySet()){

        // 获取当前元素里面的数组
        int[] temp = entry.getValue();

        // temp[0] 为 key 出现的次数
        // 如果这个值比 maxNum 更大， 那么数字需要更新
        if(temp[0] > maxNum){

            // 更新元素出现的最多次数
            maxNum = temp[0];

            // 记录符合要求的元素并且前后位置差最小的数的长度
            minLen = temp[2] - temp[1] + 1 ;

            // 如果这个值和 maxNum 相同， 那么 minLen 需要更新
        }else if(temp[0] == maxNum){
            // 对比 minLen 和当前这个元素的前后位置差
            if( minLen > temp[2]- temp[1] + 1 ){
                // 取较小值
                minLen = temp[2] - temp[1] + 1;
            }
        }
    }
}

```

```

    }
    // 返回结果
    return minLen;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int findShortestSubArray(vector<int>& nums) {
        unordered_map<int, vector<int>> mp;
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            if (mp.count(nums[i])) {
                mp[nums[i]][0]++;
                mp[nums[i]][2] = i;
            } else {
                mp[nums[i]] = {1, i, i};
            }
        }
        int maxNum = 0, minLen = 0;
        for (auto& [_, vec] : mp) {
            if (maxNum < vec[0]) {
                maxNum = vec[0];
                minLen = vec[2] - vec[1] + 1;
            } else if (maxNum == vec[0]) {
                if (minLen > vec[2] - vec[1] + 1) {
                    minLen = vec[2] - vec[1] + 1;
                }
            }
        }
        return minLen;
    }
};

```

3、Python 代码

```

class Solution:
    def findShortestSubArray(self, nums: List[int]) -> int:
        mp = dict()

        for i, num in enumerate(nums):
            if num in mp:
                mp[num][0] += 1
                mp[num][2] = i
            else:
                mp[num] = [1, i, i]

```

```
maxNum = minLen = 0
for count, left, right in mp.values():
    if maxNum < count:
        maxNum = count
        minLen = right - left + 1
    elif maxNum == count:
        if minLen > (span := right - left + 1):
            minLen = span

return minLen
```

第十四天

LeetCode 1124、表现良好的最长时间段

一、题目描述

给你一份工作时间表 `hours`，上面记录着某一位员工每天的工作小时数。

我们认为当员工一天中的工作小时数大于 `8` 小时的时候，那么这一天就是「劳累的一天」。

所谓「表现良好的时间段」，意味在这段时间内，「劳累的天数」是严格 大于「不劳累的天数」。

请你返回「表现良好时间段」的最大长度。

示例 1：

输入：hours = [9,9,6,0,6,6,9] 输出：3 解释：最长的表现良好时间段是 [9,9,6]。

示例 2：

输入：hours = [6,6,6] 输出：0

提示：

- `1 <= hours.length <= 10^4`
- `0 <= hours[i] <= 16`

二、题目解析

三、参考代码

```
// https://leetcode.cn/problems/longest-well-performing-interval/
class Solution {
    public int longestWPI(int[] hours) {
        // 记录当前子数组的长度
        // 表示 表现良好时间段 的最大长度
        int maxInterval = 0;
```

```

// 哈希表中记录每个 非零 前缀和 的 第一次 出现的 下标
Map<Integer, Integer> map = new HashMap<Integer, Integer>();

// preSum 表示前缀和
// 它是一个变量，一直迭代
int preSum = 0;

for (int i = 0; i < hours.length; i++) {

    // 如果 hours > 8
    // 则 preSum 加 1, 否则 preSum 减 1
    preSum += hours[i] > 8 ? 1 : -1;

    // 如果 preSum > 0
    // 表明从数组索引为 0 的位置出发到索引为 i 的位置
    // [ 0 , i ] 区间所有元素和大于 0
    if (preSum > 0) {

        // 从数组索引为 0 的位置出发到索引为 i 的位置组成的子数组就是 表现良好时间段
        maxInterval = i + 1;

    // 如果某一个位置的前缀和 preSum 不大于 0
    } else{

        // putIfAbsent() 方法会先判断指定的键 (key) 是否存在
        // 不存在则将键/值对插入到 HashMap 中
        // key 如果存在就不存储
        map.putIfAbsent(preSum, i);

        // 查找 [ 0 , i ] 这个区间当中，是否已经保存了前缀和为 preSum - 1 的情况
        // 如果有这种情况，表示某个区间 [ 0 , j ] 的元素和为 preSum - 1
        if (map.containsKey(preSum - 1)) {

            // 区间 [ 0 , j ] 的元素和为 preSum - 1
            int j = map.get(preSum - 1);

            // 此时，我们访问的是索引为 i 的元素，整个区间是这样的
            // [ 0 , ... , j , ... , i , .... ]
            // [ 0 , i ] 区间里面的所有元素和是 preSum
            // [ 0 , j ] 区间里面的所有元素和是 preSum - 1
            // ( j , i ] 区间里面的所有元素和是 1，是 表现良好的时间段
            int interval = i - j;

            // 更新最大值
            maxInterval = Math.max(maxInterval, interval);

        }

    }

}

```

```
        // 返回结果
        return maxInterval;
    }
}
```

LeetCode 238、除自身以外数组的乘积

一、题目描述

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积。

题目数据 保证 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。

请不要使用除法，且在 $O(n)$ 时间复杂度内完成此题。

示例 1:

输入: `nums = [1,2,3,4]` 输出: `[24,12,8,6]`

示例 2:

输入: `nums = [-1,1,0,-3,3]` 输出: `[0,0,9,0,0]`

提示:

- `2 <= nums.length <= 10^5`
- `-30 <= nums[i] <= 30`
- 保证 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内

进阶：你可以在 $O(1)$ 的额外空间复杂度内完成这个题目吗？（出于对空间复杂度分析的目的，输出数组不被视为额外空间。）

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode.cn/problems/product-of-array-except-self/
class Solution {
    public int[] productExceptSelf(int[] nums) {

        // 获取数组 nums 的长度
        int length = nums.length;

        // 出于对空间复杂度分析的目的，输出数组不被视为额外空间
        // answer 是输出数组
```

```

// 数组 answer 表示数组 nums 中每个元素左边所有元素的乘积
// 比如 answer[5] 表示数组 nums 中下标为 5 的元素的左边所有元素的乘积
// 即 answer[5] = nums[0] * nums[1] * nums[2] * nums[3] * nums[4]
int[] answer = new int[length];

// 由于任何数与 1 相乘都是它本身，因此，如果发现元素左边没有元素时，默认值为 1
// 这样就不会影响到后面的计算
answer[0] = 1;

// 开始不断填充 answer
for (int i = 1; i < length; i++) {
    answer[i] = nums[i - 1] * answer[i - 1];
    System.out.println(answer[i]);
}

// right 为右侧所有元素的乘积
// 由于任何数与 1 相乘都是它本身，因此，如果发现元素右边没有元素时，默认值为 1
int right = 1;

for (int i = length - 1; i >= 0; i--) {
    // 对于索引 i，左边的乘积为 answer[i]，右边的乘积为 right
    // 更新到 answer[i]
    answer[i] = answer[i] * right;

    // right 为右侧所有元素的乘积
    // 更新 right
    right *= nums[i];
}

// 输出结果数组
return answer;
}
}

```

LeetCode 23、合并K个升序链表（优先队列思路）

一、题目描述

给你一个链表数组，每个链表都已经按升序排列。

请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1：

输入: lists = [[1,4,5],[1,3,4],[2,6]]

输出: [1,1,2,3,4,4,5,6]

解释: 链表数组如下:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

将它们合并到一个有序链表中得到。

1->1->2->3->4->4->5->6

示例 2:

输入: lists = []

输出: []

示例 3:

输入: lists = [[]]

输出: []

提示:

- `k == lists.length`
- `0 <= k <= 10^4`
- `0 <= lists[i].length <= 500`
- `-10^4 <= lists[i][j] <= 10^4`
- `lists[i]` 按升序排列
- `lists[i].length` 的总和不超过 `10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: 278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 合并K个升序链表 (LeetCode 23) (优先队列做法):https://leetcode.cn/problems/merge-k-sorted-lists/
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {

        // 队列是遵循先进先出 (First-In-First-Out) 模式的, 但有时需要在队列中基于优先级处理对象。
        // PriorityQueue 和队列 Queue 的区别在于, 它的出队顺序与元素的优先级有关
```

堆】

```
// 对 PriorityQueue 调用 remove() 或 poll() 方法，返回的总是优先级最高的元素
// Java 中 PriorityQueue 通过二叉小顶堆实现
// PriorityQueue 默认是一个【小顶堆】，可以通过传入自定义的 Comparator 函数来实现【大顶堆】

Queue<ListNode> pq = new PriorityQueue<>((v1, v2) -> v1.val - v2.val);

// 遍历所有链表
for (ListNode node: lists) {
    // PriorityQueue 实现了 Queue 接口，不允许放入 null 元素
    if (node != null) {
        // 把所有链表都加入到优先队列当中
        // 优先队列会自己处理，把头节点最小的值放到前面去
        pq.offer(node);
    }

    // ListNode node = pq.peek();

    // System.out.println(node.val);
}

// 添加一个虚拟头节点（哨兵），帮助简化边界情况的判断
ListNode dummyHead = new ListNode(-1);

// 合并成功之后的尾节点位置
ListNode tail = dummyHead;

// 遍历优先队列，取出最小值出来
while (!pq.isEmpty()) {

    // 取出优先队列，即二叉堆的头节点，最小的节点
    ListNode minNode = pq.poll();

    // 把这个节点连接到合并链表的尾部
    tail.next = minNode;

    // tail 的位置也随之发生变化
    tail = minNode;

    // PriorityQueue 实现了 Queue 接口，不允许放入 null 元素
    if (minNode.next != null) {
        // 再把新的节点也加入到优先队列当中
        pq.offer(minNode.next);
    }
}

// 整个过程其实就是「多路归并」过程
// 返回结果
return dummyHead.next;
```

```
}  
}
```

2、C++ 代码

```
// 登录 AlgoMooc 官网获取更多算法图解  
// https://www.algomooc.com  
// 作者：程序员吴师兄  
// 微信：278166530  
// 代码有看不懂的地方一定要私聊咨询吴师兄呀  
// 合并K个升序链表（LeetCode 23）（优先队列做法）：https://leetcode.cn/problems/merge-k-sorted-lists/  
class Solution {  
public:  
    struct Status {  
        int val;  
        ListNode ptr;  
        bool operator < (const Status &rhs) const {  
            return val > rhs.val;  
        }  
    };  
  
    // 队列是遵循先进先出（First-In-First-Out）模式的，但有时需要在队列中基于优先级处理对象。  
    // PriorityQueue 和队列 Queue 的区别在于，它的出队顺序与元素的优先级有关  
    // 对 PriorityQueue 调用 remove() 或 poll() 方法，返回的总是优先级最高的元素  
    priority_queue<Status> pq;  
  
    ListNode mergeKLists(vector<ListNode*>& lists) {  
  
        // 遍历所有链表  
        for (auto node: lists) {  
            // 把所有链表都加入到优先队列当中  
            // 优先队列会自己处理，把头节点最小的值放到前面去  
            if (node) pq.push({node->val, node});  
        }  
  
        // 添加一个虚拟头节点（哨兵），帮助简化边界情况的判断  
  
        // 合并成功之后的尾节点位置  
        ListNode dummyHead, *tail = &dummyHead;  
  
        // 遍历优先队列，取出最小值出来  
        while (!pq.empty()) {  
  
            // 取出优先队列，即二叉堆的头节点，最小的节点  
            auto f = pq.top(); pq.pop();
```

```

        // 把这个节点连接到合并链表的尾部
        tail->next = f.ptr;

        // tail 的位置也随之发生变化
        tail = tail->next;

        // PriorityQueue 实现了 Queue 接口, 不允许放入 null 元素
        if (f.ptr->next) pq.push({f.ptr->next->val, f.ptr->next});
    }

    // 整个过程其实就是「多路归并」过程
    // 返回结果
    return dummyHead.next;
}
};

```

3、Python 代码

```

class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        if not lists or len(lists) == 0:
            return None
        import heapq
        heap = []
        # 和 Java有所不同, 是将所有元素都取出放入堆中
        for node in lists:
            while node:
                heapq.heappush(heap, node.val)
                node = node.next
        # 添加一个虚拟头节点(哨兵), 帮助简化边界情况的判断
        dummy = ListNode(None)
        # 合并成功之后的尾节点位置
        tail = dummy
        # 依次将堆中的元素取出(因为是小顶堆, 所以每次出来的都是目前堆中值最小的元素), 然后重新构建一个列表返回
        while heap:
            temp_node = ListNode(heapq.heappop(heap))
            tail.next = temp_node
            tail = temp_node
        return dummy.next

```

LeetCode 290、单词规律

一、题目描述

给定一种规律 `pattern` 和一个字符串 `s`，判断 `s` 是否遵循相同的规律。

这里的遵循指完全匹配，例如，`pattern` 里的每个字母和字符串 `s` 中的每个非空单词之间存在着双向连接的对应规律。

示例1:

输入: `pattern = "abba"`, `s = "dog cat cat dog"` 输出: `true`

示例 2:

输入: `pattern = "abba"`, `s = "dog cat cat fish"` 输出: `false`

示例 3:

输入: `pattern = "aaaa"`, `s = "dog cat cat dog"` 输出: `false`

提示:

- `1 <= pattern.length <= 300`
- `pattern` 只包含小写英文字母
- `1 <= s.length <= 3000`
- `s` 只包含小写英文字母和 `' '`
- `s` 不包含任何前导或尾随对空格
- `s` 中每个单词都被单个空格分隔

二、题目解析

三、参考代码

```
class Solution:
    def wordPattern(self, pattern: str, s: str) -> bool:

        s = s.split()

        if len(pattern) != len(s):
            return False

        t = pattern

        # 接下来的逻辑和 LC205. 同构字符串 一模一样

        # 设置一个哈希集合用来存储字符串 s 当中的元素
        dic1 = {}

        # 设置一个哈希集合用来存储字符串 t 当中的元素
        dic2 = {}
```

```

# 由于 t.length == s.length
# 按照顺序访问 s 和 t 中对应的元素
for i in range(len(pattern)):

    # 1、访问的元素 s[i] 存在于 dic1 中
    # 并且发现它对应的元素值和当前 t 中元素 t[i] 不相同
    # 返回 False
    if s[i] in dic1 and dic1[s[i]] != t[i]:
        return False

    # 2、访问的元素 t[i] 存在于 dic2 中
    # 并且发现它对应的元素值和当前 s 中元素 s[i] 不相同
    # 返回 False
    if t[i] in dic2 and dic2[t[i]] != s[i]:
        return False

    # 3、访问的元素 s[i] 不存在于 dic1 中
    # 存放哈希集合中
    if s[i] not in dic1:
        # dic1[s[i]] = t[i]
        dic1[s[i]] = t[i]

    # 3、访问的元素 t[i] 不存在于 dic2 中
    # 存放哈希集合中
    if t[i] not in dic2:
        # dic2[t[i]] = s[i]
        dic2[t[i]] = s[i]
# 返回 True
return True

```

LeetCode 304、二维区域和检索 - 矩阵不可变

一、题目描述

给定一个二维矩阵 `matrix`，以下类型的多个请求：

- 计算其子矩形范围内元素的总和，该子矩阵的 **左上角** 为 `(row1, col1)`，**右下角** 为 `(row2, col2)`。

实现 `NumMatrix` 类：

- `NumMatrix(int[][] matrix)` 给定整数矩阵 `matrix` 进行初始化
- `int sumRegion(int row1, int col1, int row2, int col2)` 返回 **左上角** `(row1, col1)`、**右下角** `(row2, col2)` 所描述的子矩阵的元素 **总和**。

示例 1：

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5

img

输入:

```
["NumMatrix","sumRegion","sumRegion","sumRegion"]
[[[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],[4,1,0,1,7],[1,0,3,0,5]], [2,1,4,3], [1,1,2,2], [1,2,2,4]]
```

输出:

```
[null, 8, 11, 12]
```

解释:

```
NumMatrix numMatrix = new NumMatrix([[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],[4,1,0,1,7],
[1,0,3,0,5]]);
numMatrix.sumRegion(2, 1, 4, 3); // return 8 (红色矩形框的元素总和)
numMatrix.sumRegion(1, 1, 2, 2); // return 11 (绿色矩形框的元素总和)
numMatrix.sumRegion(1, 2, 2, 4); // return 12 (蓝色矩形框的元素总和)
```

提示:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 200`
- `-10^5 <= matrix[i][j] <= 10^5`
- `0 <= row1 <= row2 < m`
- `0 <= col1 <= col2 < n`
- 最多调用 `10^4` 次 `sumRegion` 方法

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二维区域和检索 - 矩阵不可变(LeetCode 304): https://leetcode.cn/problems/range-sum-query-2d-immutable/submissions/
class NumMatrix {

    // 前缀和数组
    private int[][] preSum;

    public NumMatrix(int[][] matrix) {

        // 预处理二维数组 preSum
        // preSum[i][j] 表示 从 [ 0 , 0 ] 位置到 [ i , j ] 位置的子矩形所有元素之和
        // 默认都是 0
        preSum = new int[matrix.length + 1][matrix[0].length + 1];

        // 填充 preSum 数组
        for ( int i = 0 ; i < matrix.length ; i++ ) {

            for ( int j = 0 ; j < matrix[0].length ; j++ ) {
                // 由图可知, preSum[ i + 1 ][ j + 1 ] 的计算过程
                preSum[ i + 1 ][ j + 1 ] = preSum[ i ][ j + 1 ]
                    +
                    preSum[ i + 1 ][ j ]
                    -
                    preSum[ i ][ j ]
                    +
                    matrix[ i ][ j ];
            }
        }
    }

    public int sumRegion(int row1, int col1, int row2, int col2) {
        // 由图可知, 某个矩形的计算结果过程
        return preSum[row2 + 1][col2 + 1]
            -
            preSum[row2 + 1][col1]
            -
            preSum[row1][col2 + 1]
            +
            preSum[row1][col1];
    }
}
```



```
}  
}
```

2、C++代码

```
class NumMatrix {  
public:  
    // 前缀和数组  
    vector<vector<int>> sums;  
  
    NumMatrix(vector<vector<int>>& matrix) {  
        int m = matrix.size();  
        if (m > 0) {  
            int n = matrix[0].size();  
            sums.resize(m + 1, vector<int>(n + 1));  
            for (int i = 0; i < m; i++) {  
                for (int j = 0; j < n; j++) {  
                    sums[i + 1][j + 1] = sums[i][j + 1] + sums[i + 1][j] - sums[i][j] +  
matrix[i][j];  
                }  
            }  
        }  
    }  
  
    int sumRegion(int row1, int col1, int row2, int col2) {  
        return sums[row2 + 1][col2 + 1] - sums[row1][col2 + 1] - sums[row2 + 1][col1] +  
sums[row1][col1];  
    }  
};
```

3、Python 代码

```
class NumMatrix:  
  
    def __init__(self, matrix: List[List[int]]):  
        m, n = len(matrix), (len(matrix[0]) if matrix else 0)  
        self.sums = [[0] * (n + 1) for _ in range(m + 1)]  
        _sums = self.sums  
  
        for i in range(m):  
            for j in range(n):  
                _sums[i + 1][j + 1] = _sums[i][j + 1] + _sums[i + 1][j] - _sums[i][j] +  
matrix[i][j]  
  
    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:  
        _sums = self.sums
```

```
return _sums[row2 + 1][col2 + 1] - _sums[row1][col2 + 1] - _sums[row2 + 1][col1] + _sums[row1][col1]
```

第十五天

LeetCode 26、删除有序数组中的重复项

一、题目描述

给你一个 **升序排列** 的数组 `nums`，请你 **原地** 删除重复出现的元素，使每个元素 **只出现一次**，返回删除后数组的新长度。元素的 **相对顺序** 应该保持 **一致**。

由于在某些语言中不能改变数组的长度，所以必须将结果放在数组 `nums` 的第一部分。更规范地说，如果在删除重复项之后有 `k` 个元素，那么 `nums` 的前 `k` 个元素应该保存最终结果。

将最终结果插入 `nums` 的前 `k` 个位置后返回 `k`。

不要使用额外的空间，你必须在 **原地** ****修改输入数组**** 并在使用 $O(1)$ 额外空间的条件下完成。

判题标准:

系统会用下面的代码来测试你的题解:

```
int[] nums = [...]; // 输入数组
int[] expectedNums = [...]; // 长度正确的期望答案
int k = removeDuplicates(nums); // 调用
assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

如果所有断言都通过，那么您的题解将被 **通过**。

示例 1:

输入: `nums = [1,1,2]` 输出: `2, nums = [1,2,_]` 解释: 函数应该返回新的长度 `2`，并且原数组 `nums` 的前两个元素被修改为 `1, 2`。不需要考虑数组中超出新长度后面的元素。

示例 2:

输入: `nums = [0,0,1,1,1,2,2,3,3,4]` 输出: `5, nums = [0,1,2,3,4]` 解释: 函数应该返回新的长度 `5`，并且原数组 `nums` 的前五个元素被修改为 `0, 1, 2, 3, 4`。不需要考虑数组中超出新长度后面的元素。

提示:

- `1 <= nums.length <= 3 * 104`
- `-104 <= nums[i] <= 104`
- `nums` 已按 **升序** 排列

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 删除有序数组中的重复项 (LeetCode 26) :https://leetcode.cn/problems/remove-duplicates-
// from-sorted-array/submissions/
class Solution {
    public int removeDuplicates(int[] nums) {

        // 指针 i 进行数组遍历
        int n = nums.length;

        // 指针 j 指向即将被赋值的位置
        int j = 0;

        // 开始对数组进行遍历
        for (int i = 0 ; i < n ; i++) {

            // 进行筛选
            if ( i == 0 ||  nums[i] != nums[i - 1]) {
                // 赋值
                nums[j] = nums[i];

                // j 移动
                j++;
            }
        }

        // 获取结果
        return j ;
    }
}
```

2、C++代码

```
class Solution {
public:
    int removeDuplicates(vector<int>& nums) {

        // 指针 i 进行数组遍历
        int n = nums.size();

        // 指针 j 指向即将被赋值的位置
        int j = 0;
```

```

// 开始对数组进行遍历
for (int i = 0 ; i < n ; i++) {

    // 进行筛选
    if ( i == 0 ||  nums[i] != nums[i - 1]) {
        // 赋值
        nums[j] = nums[i];

        // j 移动
        j++;
    }
}

// 获取结果
return j ;

}
};

```

3、Python 代码

```

class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        # 指针 i 进行数组遍历
        n = len(nums)

        # 指针 j 指向即将被赋值的位置
        j = 0

        # 开始对数组进行遍历
        for i in range(n):

            # 进行筛选
            if i == 0 or  nums[i] != nums[i - 1] :
                # 赋值
                nums[j] = nums[i]

                # j 移动
                j += 1

        # 获取结果
        return j

```

LeetCode 319、灯泡开关

一、题目描述

初始时有 n 个灯泡处于关闭状态。第一轮，你将会打开所有灯泡。接下来的第二轮，你将会每两个灯泡关闭第二个。

第三轮，你每三个灯泡就切换第三个灯泡的开关（即，打开变关闭，关闭变打开）。第 i 轮，你每 i 个灯泡就切换第 i 个灯泡的开关。直到第 n 轮，你只需要切换最后一个灯泡的开关。

找出并返回 n 轮后有多少个亮着的灯泡。

示例 1：



输入： $n = 3$
输出： 1
解释：
初始时，灯泡状态 [关闭， 关闭， 关闭]。
第一轮后，灯泡状态 [开启， 开启， 开启]。
第二轮后，灯泡状态 [开启， 关闭， 开启]。
第三轮后，灯泡状态 [开启， 关闭， 关闭]。

你应该返回 1，因为只有一个灯泡还亮着。

示例 2：

输入： $n = 0$
输出： 0

示例 3：

输入: $n = 1$

输出: 1

提示:

- $0 \leq n \leq 10^9$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 灯泡开关 (LeetCode 319) :https://leetcode.cn/problems/bulb-switcher/
class Solution {
    public int bulbSwitch(int n) {
        // n = 14 , 3.xx
        // n = 15 , 3.xx
        // n = 16 , 4
        // 求小于等于 n 的完全平方数的个数
        return (int)Math.sqrt(n);
    }
}
```

2、**C++** 代码

```
class Solution {
public:
    int bulbSwitch(int n) {
        return sqrt(n + 0.5);
    }
};
```

3、Python 代码

```
class Solution:
    def bulbSwitch(self, n: int) -> int:
        return int(sqrt(n + 0.5))
```

LeetCode 80、删除有序数组中的重复项II

一、题目描述

给你一个有序数组 `nums`，请你`原地`删除重复出现的元素，使得出现次数超过两次的元素只出现两次，返回删除后数组的新长度。

不要使用额外的数组空间，你必须在 `原地` **修改输入数组** 并在使用 $O(1)$ 额外空间的条件下完成。

说明：

为什么返回数值是整数，但输出的答案是数组呢？

请注意，输入数组是以「引用」方式传递的，这意味着在函数里修改输入数组对于调用者是可见的。

你可以想象内部操作如下：

```
// nums 是以“引用”方式传递的。也就是说，不对实参做任何拷贝
int len = removeDuplicates(nums);

// 在函数里修改输入数组对于调用者是可见的。
// 根据你的函数返回的长度，它会打印出数组中 该长度范围内 的所有元素。
for (int i = 0; i < len; i++) {
    print(nums[i]);
}
```

示例 1：

输入：nums = [1,1,1,2,2,3] 输出：5, nums = [1,1,2,2,3] 解释：函数应返回新长度 length = 5, 并且原数组的前五个元素被修改为 1, 1, 2, 2, 3。不需要考虑数组中超出新长度后面的元素。

示例 2：

输入：nums = [0,0,1,1,1,2,3,3] 输出：7, nums = [0,0,1,1,2,3,3] 解释：函数应返回新长度 length = 7, 并且原数组的前五个元素被修改为 0, 0, 1, 1, 2, 3, 3。不需要考虑数组中超出新长度后面的元素。

提示：

- `1 <= nums.length <= 3 * 104`
- `-104 <= nums[i] <= 104`
- `nums` 已按升序排列

二、题目解析

三、参考代码

```
class Solution(object):
    def removeDuplicates(self, nums):

        # 指针 slow 指向即将被赋值的位置
        slow = 0

        # 开始对数组进行遍历
```

```

for fast in range(len(nums)):

    # 进行筛选
    if slow < 2 or nums[fast] != nums[slow - 2]:
        # 赋值
        nums[slow] = nums[fast]
        # slow 移动
        slow += 1

# 获取结果
return slow

```

LeetCode 88、合并两个有序数组

一、题目描述

给你两个有序整数数组 `nums1` 和 `nums2`，请你将 `nums2` 合并到 `nums1` 中，使 `nums1` 成为一个有序数组。

初始化 `nums1` 和 `nums2` 的元素数量分别为 `m` 和 `n`。

你可以假设 `nums1` 的空间大小等于 `m + n`，这样它就有足够的空间保存来自 `nums2` 的元素。

二、题目解析

设置两个索引 `i` 和 `j` 分别指向 `nums1` 和 `nums2` 的有效元素的尾部，从它们的尾部开始向前遍历，同时设置索引 `cur` 指向 `nums1` 的最末尾，在每次遍历过程中，比较 `i` 和 `j` 指向的元素值大小，把大的元素填充到 `cur` 的位置，填充完毕说明那个元素已经放置在它应该放置的位置，不需要再管它了，把 `cur` 向前移动，同时把 `i` 或者 `j` 向前移动，继续比较 `i` 和 `j` 指向的元素值大小，把大的元素填充到 `cur` 的位置。

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com/555.html
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 合并两个有序数组 ( LeetCode 88 ):https://leetcode-cn.com/problems/merge-sorted-array/
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        // 索引从有序数组 nums1 有效元素的末端开始
        // 数组的下标索引从零开始计数
        // 索引    0    1    2
        // 数组 [ 1 , 2 , 3 ]
        int i = m - 1;

        // 索引从有序数组 nums2 的末端开始
        int j = n - 1;

        // 从有序数组 nums1 最末端的位置开始保存元素

```



```

int cur = nums1.length - 1;

// 通过循环把 num2 的元素都移动到 num1 中
while( j >= 0 ){

    // 比较 num1 和 num2 中当前的元素大小

    // 如果 num1 中的索引位置为 i 的元素大于 num2 中索引位置为 j 的元素
    // 为了防止越界 i 必须是大于等于 0
    if( i >= 0 && nums1[i] > nums2[j] ){

        // 把 num1 中的索引位置为 i 的元素复制到索引为 cur 的位置
        // 此时 cur 的元素已经确定下来
        nums1[cur] = nums1[i];

        // 接下来去确定 cur 前面一个元素应该放什么数字
        cur--;
        // 此时，索引 i 需要向前移动
        i--;
        // 否则，如果 num1 中的索引位置为 i 的元素小于或者等于 num2 中索引位置为 j 的元素
    }else{

        // 把 num2 中的索引位置为 j 的元素复制到索引为 cur 的位置
        nums1[cur] = nums2[j];
        // 接下来去确定 cur 前面一个元素应该放什么数字
        cur--;
        // 此时，索引 j 需要向前移动
        j--;
    }
}
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com/555.html
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 合并两个有序数组( LeetCode 88 ): https://leetcode-cn.com/problems/merge-sorted-array/
class solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        // 索引从有序数组 nums1 有效元素的末端开始
        // 数组的下标索引从零开始计数
        // 索引    0    1    2
        // 数组 [ 1 , 2 , 3 ]
        int i = m - 1;
    }
}

```

```

// 索引从有序数组 nums2 的末端开始
int j = n - 1;

// 从有序数组 nums1 最末端的位置开始保存元素
int cur = nums1.size() - 1;

// 通过循环把 num2 的元素都移动到 num1 中
while( j >= 0 ){

    // 比较 num1 和 num2 中当前的元素大小

    // 如果 num1 中的索引位置为 i 的元素大于 num2 中索引位置为 j 的元素
    // 为了防止越界 i 必须是大于等于 0
    if( i >= 0 && nums1[i] > nums2[j] ){

        // 把 num1 中的索引位置为 i 的元素复制到索引为 cur 的位置
        // 此时 cur 的元素已经确定下来
        nums1[cur] = nums1[i];

        // 接下来去确定 cur 前面一个元素应该放什么数字
        cur--;
        // 此时，索引 i 需要向前移动
        i--;
        // 否则，如果 num1 中的索引位置为 i 的元素小于或者等于 num2 中索引位置为 j 的元素
    }else{

        // 把 num2 中的索引位置为 j 的元素复制到索引为 cur 的位置
        nums1[cur] = nums2[j];
        // 接下来去确定 cur 前面一个元素应该放什么数字
        cur--;
        // 此时，索引 j 需要向前移动
        j--;
    }
}
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com/555.html
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 合并两个有序数组( LeetCode 88 ):https://leetcode-cn.com/problems/merge-sorted-array/
class solution :
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        # 索引从有序数组 nums1 有效元素的末端开始
        # 数组的下标索引从零开始计数
        # 索引    0    1    2

```

```

# 数组 [ 1 , 2 , 3 ]
i = m - 1

# 索引从有序数组 nums2 的末端开始
j = n - 1

# 从有序数组 nums1 最末端的位置开始保存元素
cur = m + n - 1

# 通过循环把 num2 的元素都移动到 num1 中
while j >= 0 :

    # 比较 num1 和 num2 中当前的元素大小

    # 如果 num1 中的索引位置为 i 的元素大于 num2 中索引位置为 j 的元素
    # 为了防止越界 i 必须是大于等于 0
    if i >= 0 and nums1[i] > nums2[j]:

        # 把 num1 中的索引位置为 i 的元素复制到索引为 cur 的位置
        # 此时 cur 的元素已经确定下来
        nums1[cur] = nums1[i]

        # 接下来去确定 cur 前面一个元素应该放什么数字
        cur-=1
        # 此时，索引 i 需要向前移动
        i-=1
    # 否则，如果 num1 中的索引位置为 i 的元素小于或者等于 num2 中索引位置为 j 的元素
    else:

        # 把 num2 中的索引位置为 j 的元素复制到索引为 cur 的位置
        nums1[cur] = nums2[j]
        # 接下来去确定 cur 前面一个元素应该放什么数字
        cur-=1
        # 此时，索引 j 需要向前移动
        j-=1

```

四、复杂度分析

时间复杂度：O(m+n)。指针移动单调递增，最多移动 m+n 次，因此时间复杂度为 O(m+n)。

空间复杂度：O(m+n)。

第十六天

LeetCode 125、验证回文串

一、题目描述

给定一个字符串，验证它是否是回文串，只考虑字母和数字字符，可以忽略字母的大小写。

说明：本题中，我们将空字符串定义为有效的回文串。

示例 1:

```
输入: "A man, a plan, a canal: Panama"
输出: true
解释: "amanaplanacanalpanama" 是回文串
```

示例 2:

```
输入: "race a car"
输出: false
解释: "raceacar" 不是回文串
```

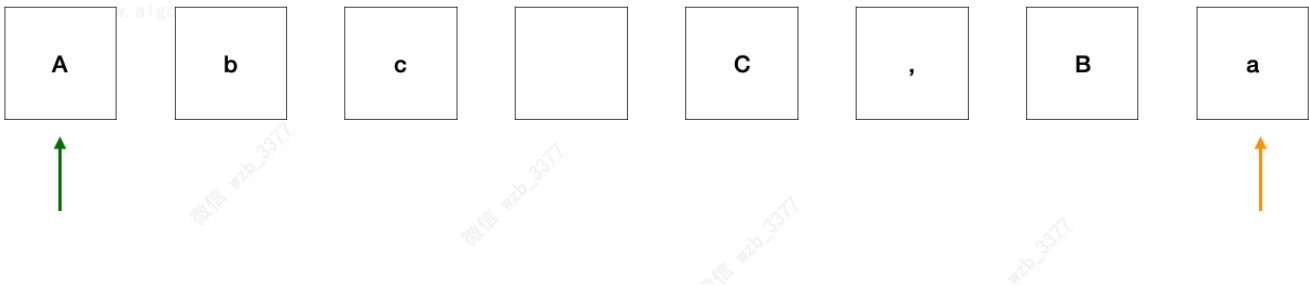
提示:

- `1 <= s.length <= 2 * 105`
- 字符串 `s` 由 ASCII 字符组成

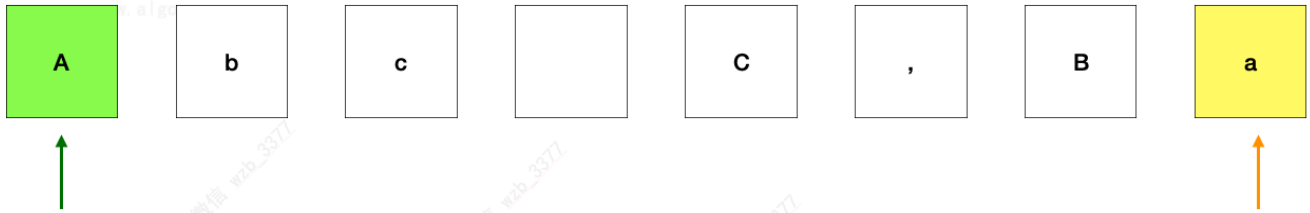
二、题目解析

具体操作如下：

1、设置左右两个指针，分别指向字符串的开头位置和结束位置。



- 2、移动和观察者两个指针所指向元素之间的关系。
- 3、如果 left 指向的元素不是字母、也不是数字，那么可以忽略掉这个元素，即让 left 向右移动。
- 4、如果 right 指向的元素不是字母、也不是数字，那么可以忽略掉这个元素，即让 right 向左移动。
- 5、进过 3、4 操作后，要么 left 和 right 相遇了，跳出循环；要么 left 和 right 还没有相遇，并且它们都是指向字母或者数字。
- 6、只需要判断一下 left 和 right 指向的元素值是否相同就行，如果不相同，直接返回 false。



7、否则，继续让两个指针向内移动，left 向右移动，right 向左移动。

8、最后，没有得到 false 的答案就说明是回文串，返回 true

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 验证回文串 (LeetCode 125) : https://leetcode.cn/problems/valid-palindrome/submissions/
class Solution {
    public boolean isPalindrome(String s) {

        // 设置左右两个指针
        int left = 0 ;

        int right = s.length() - 1 ;

        // 移动和观察者两个指针所指向元素之间的关系
        while (left < right) {

            // 这里的逻辑有点像快速排序的代码

            // 如果 left 指向的元素不是字母、也不是数字
            // 那么可以忽略掉这个元素，即让 left 向右移动
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
                // left 向右移动
                left++;
            }

            // 如果 right 指向的元素不是字母、也不是数字
            // 那么可以忽略掉这个元素，即让 right 向左移动
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
                // right 向左移动
            }
        }
    }
}
```

```

        right--;

    }

    // 来到这里时
    // 要么 left 和 right 相遇了，跳出循环
    // 要么 left 和 right 还没有相遇，并且它们都是指向字母或者数字
    if (left < right) {

        // 只需要判断一下 left 和 right 指向的元素值是否相同就行
        // 题目说明 可以忽略字母的大小写
        if (Character.toLowerCase(s.charAt(left)) !=
Character.toLowerCase(s.charAt(right))) {
            // 不相同就直接说明不是回文串
            return false;
        }

        // 否则，继续让两个指针向内移动

        // left 向右移动
        left++;

        // right 向左移动
        right--;
    }
}

// 最后，没有得到 false 的答案就说明是回文串，返回 true
return true;
}
}

```

2、C++ 代码

```

class Solution {
public:
    bool isPalindrome(string s) {
        // 设置左右两个指针
        int left = 0 ;

        int right = s.size() - 1 ;

        // 移动和观察者两个指针所指向元素之间的关系
        while (left < right) {

            // 这里的逻辑有点像快速排序的代码

            // 如果 left 指向的元素不是字母、也不是数字

```

```

// 那么可以忽略掉这个元素，即让 left 向右移动
while (left < right && !isalnum(s[left])) {
    // left 向右移动
    left++;
}

// 如果 right 指向的元素不是字母、也不是数字
// 那么可以忽略掉这个元素，即让 right 向左移动
while (left < right && !isalnum(s[right])) {
    // right 向左移动
    right--;
}

// 来到这里时
// 要么 left 和 right 相遇了，跳出循环
// 要么 left 和 right 还没有相遇，并且它们都是指向字母或者数字
if (left < right) {

    // 只需要判断一下 left 和 right 指向的元素值是否相同就行
    // 题目说明 可以忽略字母的大小写
    if (tolower(s[left]) != tolower(s[right])) {
        // 不相同就直接说明不是回文串
        return false;
    }

    // 否则，继续让两个指针向内移动

    // left 向右移动
    left++;

    // right 向左移动
    right--;
}
}

// 最后，没有得到 false 的答案就说明是回文串，返回 true
return true;
}
};

```

3、Python 代码

```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        # 设置左右两个指针
        left = 0

```

```

right = len(s) - 1

# 移动和观察者两个指针所指向元素之间的关系
while left < right :

    # 这里的逻辑有点像快速排序的代码

    # 如果 left 指向的元素不是字母、也不是数字
    # 那么可以忽略掉这个元素，即让 left 向右移动
    while left < right and not s[left].isalnum():
        # left 向右移动
        left += 1

    # 如果 right 指向的元素不是字母、也不是数字
    # 那么可以忽略掉这个元素，即让 right 向左移动
    while left < right and not s[right].isalnum():
        # right 向左移动
        right -= 1

    # 来到这里时
    # 要么 left 和 right 相遇了，跳出循环
    # 要么 left 和 right 还没有相遇，并且它们都是指向字母或者数字
    if left < right :

        # 只需要判断一下 left 和 right 指向的元素值是否相同就行
        # 题目说明 可以忽略字母的大小写
        if s[left].lower() != s[right].lower():
            # 不相同就直接说明不是回文串
            return False

        # 否则，继续让两个指针向内移动

        # left 向右移动
        left += 1

        # right 向左移动
        right -= 1

    # 最后，没有得到 false 的答案就说明是回文串，返回 true
    return True

class Solution:
    def isPalindrome(self, s: str) -> bool:

```



```

# isalnum() 方法检测字符串是否由字母和数字组成
# 转换为字符串数组的形式
xArray = ''.join(ch.lower() for ch in s if ch.isalnum())

# 左边索引的位置在 0
left = 0

# 右边索引的位置在 len(xArray) - 1
right = len(xArray) - 1

# 两个索引向内移动
# left 向右移动
# right 向左移动
while left <= right:
    # 判断这两个元素值是否相同
    if xArray[left] != xArray[right]:
        # 如果不同, 直接返回 False
        return False

    # 否则, left 向右移动
    left += 1

    # right 向左移动
    right -= 1

return True

```

四、复杂度分析

- 时间复杂度： $O(|s|)$ ，其中 $|s|$ 是字符串 s 的长度。
- 空间复杂度： $O(1)$ 。

LeetCode 143、重排链表

一、题目描述

给定一个单链表 $L: L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$,

将其重新排列后变为： $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

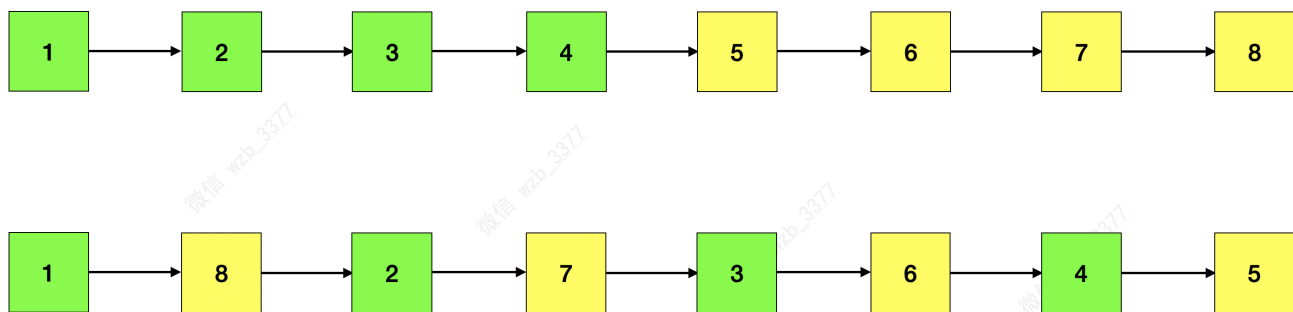
你不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

示例1:

给定链表 1->2->3->4->5, 重新排列为 1->5->2->4->3.

示例2:

给定链表 1->2->3->4, 重新排列为 1->4->2->3.



二、题目解析

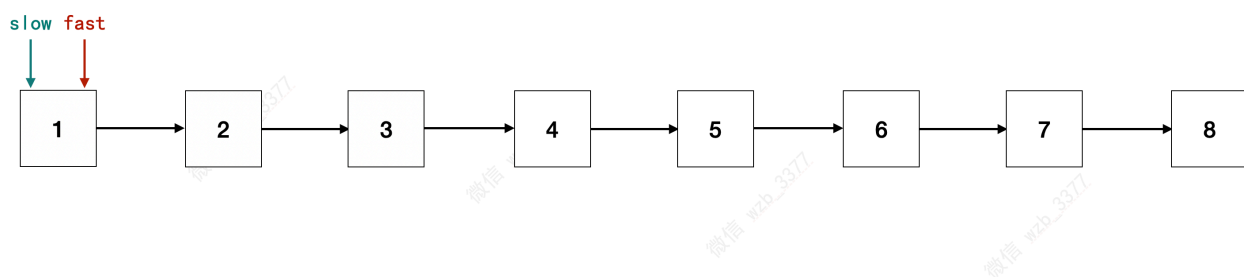
这道题目很考察基本功和观察能力，最终的结果就是将原**链表的前半部分和原链表的后半部分反转之后的链表进行合并得到的**。

所以，需要执行以下三个操作。

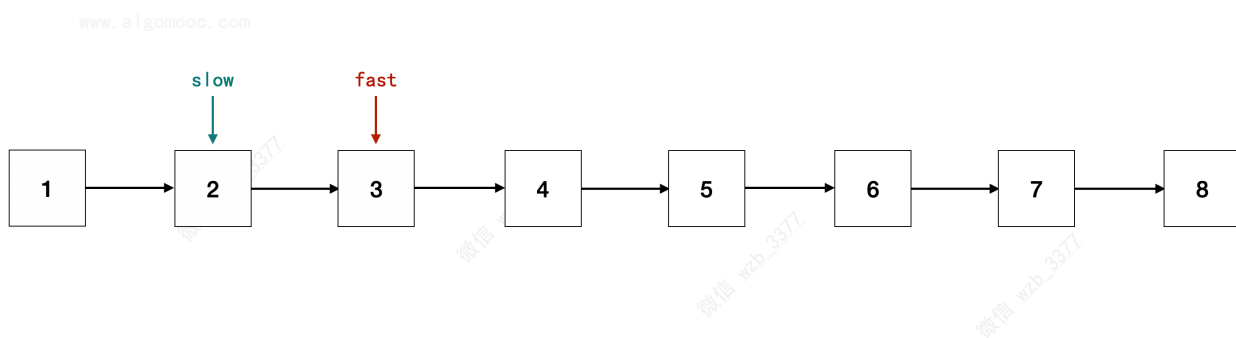
- 1、寻找出原链表的中点，把链表划分为两个区域
- 2、将右边的链表进行反转
- 3、把这两个区域进行交错合并

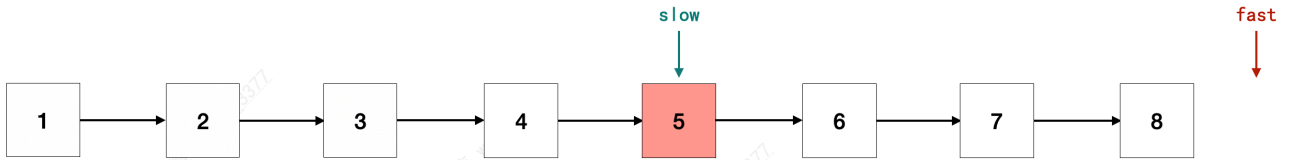
1、使用快慢指针寻找链表 midpoint

在链表的头节点设置两个指针 slow、fast，同时将它们向后移动。

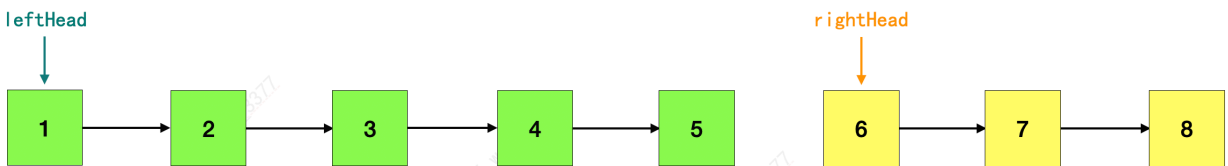


每一次，slow 向后移动一步，fast 向后移动两步。

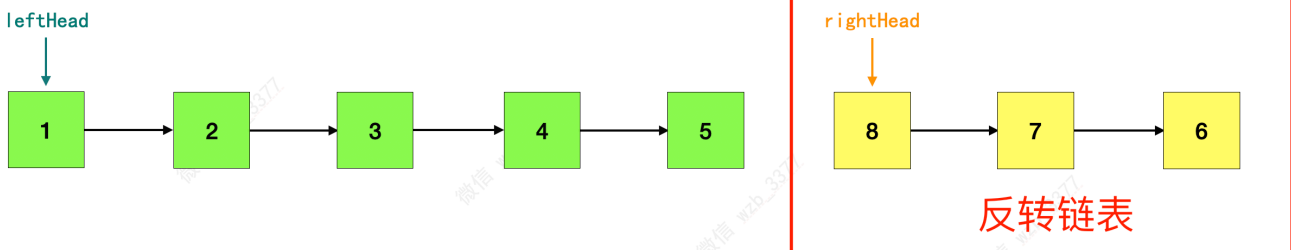




于是，找到了中间节点 5，把链表划分为两个区域。



2、将右边的链表进行反转



3、把这两个区域进行交错合并

属于归并排序的降维版本，这个操作不了解的话可以复习一下归并排序。

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 重排链表 (LeetCode 143) : https://leetcode.cn/problems/reorder-list/
class Solution {
    public void reorderList(ListNode head) {
        // a、寻找出原链表的中点，把链表划分为两个区域
  
```

```

// b、将右边的链表进行反转
// c、把这两个区域进行交错合并

// 1、使用快慢指针寻找出链表的中点来
// *****
// 对于 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
// 中间节点值为 5
// 所以左边区域为 1 -> 2 -> 3 -> 4 -> 5
// 右边区域为 6 -> 7 -> 8
// 但在视频讲解中，我把 5 归为了右边区域，这是一个错误
// 虽然这个错误并不影响结果，因为合并过程都是一样的逻辑
// *****
ListNode mid = middleNode(head);

// 2、基于 mid 这个中点，将链表划分为两个区域

// 左边的区域开头节点是 head
ListNode leftHead = head;

// 右边的区域开头节点是 mid.next
ListNode rightHead = mid.next;

// 将链表断开，就形成了两个链表了
mid.next = null;

// 3、将右边的链表进行反转
rightHead = reverseList(rightHead);

// 4、将这两个链表进行合并操作，即进行【交错拼接】
while( leftHead != null && rightHead != null){

    // 拼接过程如下
    // 5、先记录左区域、右区域【接下来将有访问的两个节点】
    ListNode leftHeadNext = leftHead.next;

    ListNode rightHeadNext = rightHead.next;

    // 6、左边连接右边的开头
    leftHead.next = rightHead;

    // 7、leftHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
    leftHead = leftHeadNext;

    // 8、右边连接左边的开头
    rightHead.next = leftHead;

    // 9、rightHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
    rightHead = rightHeadNext;
}

```

```

    }

}

// LeetCode 876 : 链表的中间节点
public ListNode middleNode(ListNode head) {

    ListNode fast = head;

    ListNode slow = head;

    while(fast != null && fast.next != null){

        fast = fast.next.next;

        slow = slow.next;
    }

    return slow;

}

// LeetCode 206 : 反转链表
public ListNode reverseList(ListNode head) {

    // 寻找递归终止条件
    // 1、head 指向的结点为 null
    // 2、head 指向的结点的下一个结点为 null
    // 在这两种情况下，反转之后的结果还是它自己本身
    if( head == null || head.next == null) return head;

    // 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点
    // 因为到最后一个节点的时候，由于当前节点 head 的 next 节点是空，所以会直接返回 head
    ListNode cur = reverseList(head.next);

    // 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
    // 第一次执行下面代码的时候，head 为 4，那么 head.next = 5
    // 那么 head.next.next 就是 5.next，意思就是去设置 5 的下一个节点
    // 等号右侧为 head，意思就是设置 5 的下一个节点是 4

    // 这里出现了两个 next
    // 第一个 next 是「获取」 head 的下一节点
    // 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
    head.next.next = head;

    // head 原来的下一节点指向自己，所以 head 自己本身就不能再指向原来的下一节点了
    // 否则会发生无限循环
    head.next = null;
}

```

```

    // 我们把每次反转后的结果传递给上一层
    return cur;
}
}

```

2、C++ 代码

```

class Solution {
public:
    void reorderList(ListNode* head) {
        // a、寻找出原链表的中点，把链表划分为两个区域
        // b、将右边的链表进行反转
        // c、把这两个区域进行交错合并

        // 1、使用快慢指针寻找出链表的中点来
        // *****
        // 对于 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
        // 中间节点值为 5
        // 所以左边区域为 1 -> 2 -> 3 -> 4 -> 5
        // 右边区域为 6 -> 7 -> 8
        // 但在视频讲解中，我把 5 归为了右边区域，这是一个错误
        // 虽然这个错误并不影响结果，因为合并过程都是一样的逻辑
        // *****
        ListNode* mid = middleNode(head);

        // 2、基于 mid 这个中点，将链表划分为两个区域

        // 左边的区域开头节点是 head
        ListNode* leftHead = head;

        // 右边的区域开头节点是 mid->next
        ListNode* rightHead = mid->next;

        // 将链表断开，就形成了两个链表了
        mid->next = nullptr;

        // 3、将右边的链表进行反转
        rightHead = reverseList(rightHead);

        // 4、将这两个链表进行合并操作，即进行【交错拼接】
        while( leftHead != nullptr && rightHead != nullptr){

            // 拼接过程如下
            // 5、先记录左区域、右区域【接下来将有访问的两个节点】
            ListNode* leftHeadNext = leftHead->next;

            ListNode* rightHeadNext = rightHead->next;

```

```

        // 6、左边连接右边的开头
        leftHead->next = rightHead;

        // 7、leftHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
        leftHead = leftHeadNext;

        // 8、右边连接左边的开头
        rightHead->next = leftHead;

        // 9、rightHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
        rightHead = rightHeadNext;

    }
}

```

```

ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

```

public:

```

ListNode* reverseList(ListNode* head) {

    // 寻找递归终止条件
    // 1、head 指向的结点为 null
    // 2、head 指向的结点的下一个结点为 null
    // 在这两种情况下，反转之后的结果还是它自己本身
    if( head == NULL || head->next == NULL) return head;

    // 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点
    // 因为到最后一个节点的时候，由于当前节点 head 的 next 节点是空，所以会直接返回 head
    ListNode *cur = reverseList(head->next);

    // 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
    // 第一次执行下面代码的时候，head 为 4，那么 head.next = 5
    // 那么 head.next.next 就是 5.next，意思就是去设置 5 的下一个节点
    // 等号右侧为 head，意思就是设置 5 的下一个节点是 4

    // 这里出现了两个 next
    // 第一个 next 是「获取」 head 的下一节点
    // 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
}

```

```

head->next->next = head;

// head 原来的下一节点指向自己，所以 head 自己本身就不能再指向原来的下一节点了
// 否则会发生无限循环
head->next = nullptr;

// 我们把每次反转后的结果传递给上一层
return cur;

}

};

```

3、Python 代码

```

class Solution:
    def reorderList(self, head: ListNode) -> None:
        # a、寻找出原链表的中点，把链表划分为两个区域
        # b、将右边的链表进行反转
        # c、把这两个区域进行交错合并

        # 1、使用快慢指针寻找出链表的中点来
        # *****
        # 对于 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
        # 中间节点值为 5
        # 所以左边区域为 1 -> 2 -> 3 -> 4 -> 5
        # 右边区域为 6 -> 7 -> 8
        # 但在视频讲解中，我把 5 归为了右边区域，这是一个错误
        # 虽然这个错误并不影响结果，因为合并过程都是一样的逻辑
        # *****
        mid = self.middleNode(head)

        # 2、基于 mid 这个中点，将链表划分为两个区域

        # 左边的区域开头节点是 head
        leftHead = head

        # 右边的区域开头节点是 mid.next
        rightHead = mid.next

        # 将链表断开，就形成了两个链表了
        mid.next = None

        # 3、将右边的链表进行反转
        rightHead = self.reverseList(rightHead)

        # 4、将这两个链表进行合并操作，即进行【交错拼接】
        while leftHead and rightHead :

```



```

# 拼接过程如下
# 5、先记录左区域、右区域【接下来将有访问的两个节点】
leftHeadNext = leftHead.next

rightHeadNext = rightHead.next

# 6、左边连接右边的开头
leftHead.next = rightHead

# 7、leftHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
leftHead = leftHeadNext

# 8、右边连接左边的开头
rightHead.next = leftHead

# 9、rightHead 已经处理好，移动到下一个节点，即刚刚记录好的节点
rightHead = rightHeadNext

```

```

def middleNode(self, head: ListNode) -> ListNode:
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

```

```

def reverseList(self, head):
    """
    :type head: ListNode
    :rtype: ListNode
    """
    # 寻找递归终止条件
    # 1、head 指向的结点为 null
    # 2、head 指向的结点的下一个结点为 null
    # 在这两种情况下，反转之后的结果还是它自己本身
    if(head == None or head.next == None):
        return head

    # 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点
    # 因为到最后一个节点的时候，由于当前节点 head 的 next 节点是空，所以会直接返回 head
    cur = self.reverseList(head.next)

    # 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
    # 第一次执行下面代码的时候，head 为 4，那么 head.next = 5
    # 那么 head.next.next 就是 5.next，意思就是去设置 5 的下一个节点
    # 等号右侧为 head，意思就是设置 5 的下一个节点是 4

```

```
# 这里出现了两个 next
# 第一个 next 是「获取」 head 的下一节点
# 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
head.next.next = head

# 原来的下一节点指向自己，所以 head 自己本身就不能再指向原来的下一节点了
# 否则会发生无限循环
head.next = None

# 我们把每次反转后的结果传递给上一层
return cur
```

四、复杂度分析

时间复杂度：O(N)，其中 N 是链表中的节点数。

空间复杂度：O(1)。

LeetCode 292、Nim 游戏

一、题目描述

你和你的朋友，两个人一起玩 [Nim 游戏](#)：

- 桌子上有一堆石头。
- 你们轮流进行自己的回合，你作为先手。
- 每一回合，轮到的人拿掉 1 - 3 块石头。
- 拿掉最后一块石头的人就是获胜者。

假设你们每一步都是最优解。请编写一个函数，来判断你是否可以在给定石头数量为 `n` 的情况下赢得游戏。如果可以赢，返回 `true`；否则，返回 `false`。

示例 1：

输入：n = 4

输出：false

解释：以下是可能的结果：

1. 移除1颗石头。你的朋友移走了3块石头，包括最后一块。你的朋友赢了。
2. 移除2个石子。你的朋友移走2块石头，包括最后一块。你的朋友赢了。
3. 你移走3颗石子。你的朋友移走了最后一块石头。你的朋友赢了。

在所有结果中，你的朋友是赢家。

示例 2：

输入：n = 1

输出：true

示例 3：

输入: $n = 2$

输出: true

提示:

- $1 \leq n \leq 2^{31} - 1$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// Nim游戏 (LeetCode 292) : https://leetcode.cn/problems/nim-game/
class Solution {
    public boolean canWinNim(int n) {

        return n % 4 != 0;

    }
}
```

2、**C++** 代码

```
class Solution {
public:
    bool canWinNim(int n) {
        return n % 4 != 0;
    }
};
```

3、Python 代码

```
class Solution:
    def canWinNim(self, n: int) -> bool:
        return n % 4 != 0
```

LeetCode 680、验证回文串II

一、题目描述

给你一个字符串 `s`，最多可以从中删除一个字符。

请你判断 `s` 是否能成为回文字符串：如果能，返回 `true`；否则，返回 `false`。

示例 1：

输入：`s = "aba"` 输出：`true`

示例 2：

输入：`s = "abca"` 输出：`true` 解释：你可以删除字符 'c'。

示例 3：

输入：`s = "abc"` 输出：`false`

提示：

- `1 <= s.length <= 105`
- `s` 由小写英文字母组成

二、题目解析

三、参考代码

```
class Solution:
    def validPalindrome(self, s: str) -> bool:

        # 判断 [ low , high ] 这个区间的字符串是否是回文串
        def isPalindrome(low, high):

            # 左边索引的位置在 low 开始
            left = low

            # 右边索引的位置在 high
            right = high

            # 两个索引向内移动
            # left 向右移动
            # right 向左移动
            while left < right:

                # 判断这两个元素值是否相同
                if s[left] != s[right]:
                    # 如果不同，直接返回 False
                    return False

                # 否则，left 向右移动
                left += 1
```

```

        # right 向左移动
        right -= 1

    # 返回结果
    return True

# 左边索引的位置在 0 开始
low = 0

# 右边索引的位置在 len(s) - 1
high = len(s) - 1

# 两个索引向内移动
# left 向右移动
# right 向左移动
while low < high:

    # 1、判断这两个元素值是否相同
    # 如果相同
    if s[low] == s[high]:

        # 两个索引向内移动
        low += 1

        high -= 1

    # 2、如果不相同
    else:
        # 3、删除 low 所在这个元素，判断 [ low + 1 , high ] 是否是回文串
        # 或者
        # 4、删除 high 所在这个元素，判断 [ low , high - 1 ] 是否是回文串
        return isPalindrome(low + 1, high) or isPalindrome(low, high - 1)

# 返回结果
return True

```

LeetCode 9、回文数

一、题目描述

给你一个整数 `x`，如果 `x` 是一个回文整数，返回 `true`；否则，返回 `false`。

回文数是指正序（从左向右）和倒序（从右向左）读都是一样的整数。

- 例如，`121` 是回文，而 `123` 不是。

示例 1：

输入：`x = 121` 输出：`true`

示例 2:

输入: $x = -121$ 输出: false 解释: 从左向右读, 为 -121 。从右向左读, 为 $121-$ 。因此它不是一个回文数。

示例 3:

输入: $x = 10$ 输出: false 解释: 从右向左读, 为 01 。因此它不是一个回文数。

二、题目解析

三、参考代码

```
class Solution:
    def isPalindrome(self, x: int) -> bool:

        # 转换为字符串数组的形式
        xArray = list(str(x))

        # 左边索引的位置在 0
        left = 0

        # 右边索引的位置在 len(xArray) - 1
        right = len(xArray) - 1

        # 两个索引向内移动
        # left 向右移动
        # right 向左移动
        while left <= right:
            # 判断这两个元素值是否相同
            if xArray[left] != xArray[right]:
                # 如果不同, 直接返回 False
                return False

            # 否则, left 向右移动
            left += 1

            # right 向左移动
            right -= 1

        return True
```

第十七天

LeetCode 122、买卖股票的最佳时机 II（贪心解法）

一、题目描述

给你一个整数数组 `prices`，其中 `prices[i]` 表示某支股票第 `i` 天的价格。

在每一天，你可以决定是否购买和/或出售股票。你在任何时候 **最多** 只能持有 **一股** 股票。你也可以先购买，然后在 **同一天** 出售。

返回 *你能获得的 最大 利润*。

示例 1：

输入：prices = [7,1,5,3,6,4] 输出：7 解释：在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5 - 1 = 4。随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出, 这笔交易所能获得利润 = 6 - 3 = 3。总利润为 4 + 3 = 7。

示例 2：

输入：prices = [1,2,3,4,5] 输出：4 解释：在第 1 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 5）的时候卖出, 这笔交易所能获得利润 = 5 - 1 = 4。总利润为 4。

示例 3：

输入：prices = [7,6,4,3,1] 输出：0 解释：在这种情况下, 交易无法获得正利润，所以不参与交易可以获得最大利润，最大利润为 0。

提示：

- `1 <= prices.length <= 3 * 104`
- `0 <= prices[i] <= 104`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机 II（LeetCode 122）（贪心解法）：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-ii/
class Solution {
    public int maxProfit(int[] prices) {
        // 最大利润初始为 0
        int profit = 0;

        // 从第 2 天开始（索引为 1）
        // 去查看当天是否需要采取【卖出】的操作
        for (int i = 1; i < prices.length; i++) {
```

```

        // 计算当天的股票价格与昨天的股票价格的差值
        int tmp = prices[i] - prices[i - 1];

        // 如果发现当天的股票价格大于了昨天的股票价格
        // 那么在当天采取【卖出】操作可以带来正向收益，即产生利润
        // 于是完全可以卖出
        // 而这个利润就可以进行累加起来
        if (tmp > 0) profit += tmp;

        // 如果发现当天的股票价格小于了昨天的股票价格
        // 那么不能采取【卖出】操作，因为这会带来负向收益，即产生亏损
        // 导致总的利润值变小
    }

    // 返回结果
    return profit;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {

        // 最大利润初始为 0
        int profit = 0;

        // 从第 2 天开始（索引为 1）
        // 去查看当天是否需要采取【卖出】的操作
        for (int i = 1; i < prices.size(); i++) {

            // 计算当天的股票价格与昨天的股票价格的差值
            int tmp = prices[i] - prices[i - 1];

            // 如果发现当天的股票价格大于了昨天的股票价格
            // 那么在当天采取【卖出】操作可以带来正向收益，即产生利润
            // 于是完全可以卖出
            // 而这个利润就可以进行累加起来
            if (tmp > 0) profit += tmp;

            // 如果发现当天的股票价格小于了昨天的股票价格
            // 那么不能采取【卖出】操作，因为这会带来负向收益，即产生亏损
            // 导致总的利润值变小
        }

        // 返回结果
        return profit;
    }
}

```



```
}  
};
```

3、Python 代码

```
#### 登录 AlgoMooc 官网获取更多算法图解  
#### https://www.algomooc.com  
#### 作者：程序员吴师兄  
#### 添加微信 278166530 获取最新课程  
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀  
class Solution:  
    def maxProfit(self, prices: List[int]) -> int:  
        # 最大利润初始为 0  
        profit = 0  
  
        # 从第 2 天开始（索引为 1）  
        # 去查看当天是否需要采取【卖出】的操作  
        for i in range(1, len(prices)):  
  
            # 计算当天的股票价格与昨天的股票价格的差值  
            tmp = prices[i] - prices[i - 1]  
  
            # 如果发现当天的股票价格大于了昨天的股票价格  
            # 那么在当天采取【卖出】操作可以带来正向收益，即产生利润  
            # 于是完全可以卖出  
            # 而这个利润就可以进行累加起来  
            if tmp > 0: profit += tmp  
  
            # 如果发现当天的股票价格小于了昨天的股票价格  
            # 那么不能采取【卖出】操作，因为这会带来负向收益，即产生亏损  
            # 导致总的利润值变小  
  
        # 返回结果  
        return profit
```

LeetCode 455、分发饼干

一、题目描述

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 i ，都有一个胃口值 $g[i]$ ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 $s[j]$ 。如果 $s[j] \geq g[i]$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。

你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 分发饼干 ( LeetCode 455 ) :https://leetcode-cn.com/problems/assign-cookies/
class Solution {
    public int findContentChildren(int[] g, int[] s) {

        // 1、将孩子们的胃口值按照从小到大的顺序排列
        // 优先满足胃口小的孩子
        Arrays.sort(g);

        // 2、将饼干按照从小到大的顺序排列
        Arrays.sort(s);

        // child 代表 g 的下标，即表示有多少孩子的胃口得到满足
        int child = 0;

        // child 代表 s 的下标，即表示目前有多少饼干被使用了
        int cookie = 0;

        // 遍历所有的饼干
        // 遍历过后，饼干只有两种状态
        // 1、要么找到了需要这个饼干的孩子
        // 2、要么剩下的孩子中，胃口值最低的孩子都大于这个饼干的值，那么这个饼干没人要
        while(cookie < s.length && child < g.length){

            // 孩子的胃口得到了满足
            if(s[cookie] >= g[child]){
                // 得到满足的孩子数量加 1
                child++;
            }

            // 查看下一个饼干能否找到需要的孩子
            cookie++;
        }

        // 最后返回孩子数量
        return child;
    }
}
```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 分发饼干 ( LeetCode 455 ) :https://leetcode-cn.com/problems/assign-cookies/
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        // 1、将孩子们的胃口值按照从小到大的顺序排列
        // 优先满足胃口小的孩子
        sort(g.begin(), g.end());

        // 2、将饼干按照从小到大的顺序排列
        sort(s.begin(), s.end());

        // child 代表 g 的下标，即表示有多少孩子的胃口得到满足
        int child = 0;

        // child 代表 s 的下标，即表示目前有多少饼干被使用了
        int cookie = 0;

        // 遍历所有的饼干
        // 遍历过后，饼干只有两种状态
        // 1、要么找到了需要这个饼干的孩子
        // 2、要么剩下的孩子中，胃口值最低的孩子都大于这个饼干的值，那么这个饼干没人要
        while(cookie < s.size() && child < g.size()){

            // 孩子的胃口得到了满足
            if(s[cookie] >= g[child]){
                // 得到满足的孩子数量加 1
                child++;
            }

            // 查看下一个饼干能否找到需要的孩子
            cookie++;
        }

        // 最后返回孩子数量
        return child;
    }
};
```

3、Python 代码

```
#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 分发饼干 ( LeetCode 455 ) :https://leetcode-cn.com/problems/assign-cookies/
class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        # 1、将孩子们的胃口值按照从小到大的顺序排列
        # 优先满足胃口小的孩子
        g.sort()

        # 2、将饼干按照从小到大的顺序排列
        s.sort()

        # child 代表 g 的下标，即表示有多少孩子的胃口得到满足
        child = 0

        # cookie 代表 s 的下标，即表示目前有多少饼干被使用了
        cookie = 0

        # 遍历所有的饼干
        # 遍历过后，饼干只有两种状态
        # 1、要么找到了需要这个饼干的孩子
        # 2、要么剩下的孩子中，胃口值最低的孩子都大于这个饼干的值，那么这个饼干没人要
        while cookie < len(s) and child < len(g) :
            # 孩子的胃口得到了满足
            if s[cookie] >= g[child] :
                # 得到满足的孩子数量加 1
                child += 1

            # 查看下一个饼干能否找到需要的孩子
            cookie += 1

        # 最后返回孩子数量
        return child
```

复杂度分析

时间复杂度： $O(m\log m + n\log n)$ ，其中 m 和 n 分别是数组 g 和 s 的长度。对两个数组排序的时间复杂度是 $O(m\log m + n\log n)$ ，遍历数组的时间复杂度是 $O(m+n)$ ，因此总时间复杂度是 $O(m\log m + n\log n)$ 。

空间复杂度： $O(\log m + \log n)$ ，其中 m 和 n 分别是数组 g 和 s 的长度。空间复杂度主要是排序的额外空间开销。

LeetCode 605、种花问题

一、题目描述

假设有一个很长的花坛，一部分地块种植了花，另一部分却没有。可是，花不能种植在相邻的地块上，它们会争夺水源，两者都会死去。

给你一个整数数组 `flowerbed` 表示花坛，由若干 `0` 和 `1` 组成，其中 `0` 表示没种植花，`1` 表示种植了花。另有一个数 `n`，能否在不打破种植规则的情况下种入 `n` 朵花？能则返回 `true`，不能则返回 `false`。

示例 1：

```
输入：flowerbed = [1,0,0,0,1], n = 1
输出：true
```

示例 2：

```
输入：flowerbed = [1,0,0,0,1], n = 2
输出：false
```

提示：

- `1 <= flowerbed.length <= 2 * 104`
- `flowerbed[i]` 为 `0` 或 `1`
- `flowerbed` 中不存在相邻的两朵花
- `0 <= n <= flowerbed.length`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/can-place-flowers/
class Solution {
    public boolean canPlaceFlowers(int[] flowerbed, int n) {
        // 遍历数组，在遍历过程中，采取贪心的思路，并不需要【每个位置】都去查看是否可以种花
        // 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去
        // 2、当前位置没有种花，需要考虑后面一个位置是否种花
        for( int i = 0 ; i < flowerbed.length && n > 0 ;){

            // 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去
            // 所以让 i 执行加 2 操作，跳过了加 1 后的那个位置
            if(flowerbed[i] == 1){

                // 让 i 执行加 2 操作
```

```

        i += 2;

        // 2、说明当前位置没有种花 flowerbed[i] == 0
        // 3、如果这个位置是数组的最后一个位置，说明后一个位置不存在，没有限制，说明
flowerbed[i] 可以种花
        // 4、如果这个位置【不是】数组的最后一个位置，那么只有后一个位置【没有种花】，才有资格在
flowerbed[i] 位置种花
        }else if(i == flowerbed.length - 1 || flowerbed[i + 1] == 0){

            // 以上两种条件都可以在 flowerbed[i] 位置种花
            // 成功之后，所需目标减 1
            n--;

            // 在 flowerbed[i] 位置种花之后，i + 1 位置不需要去考虑了，因为它明显不能种花，可
以跳过去

            // 让 i 执行加 2 操作
            i += 2;

            // 5、说明当前位置没有种花 flowerbed[i] == 0
            // 6、但是后一个位置已经种花了，那么当前位置无法采取种花操作
        }else{

            // i + 1 位置已经种花，不用再去访问一遍
            // i + 2 位置考虑到 i + 1 位置已经种花，所以也无法种花，不用再去访问
            // 让 i 执行加 3 操作
            i += 3;
        }
    }

    // 最后查看是否用完了 n
    return n <= 0;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    bool canPlaceFlowers(vector<int>& flowerbed, int n) {

        // 遍历数组，在遍历过程中，采取贪心的思路，并不需要【每个位置】都去查看是否可以种花
        // 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去
        // 2、当前位置没有种花，需要考虑后面一个位置是否种花
        for( int i = 0 ; i < flowerbed.size() && n > 0 ; ){

            // 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去

```

```

// 所以让 i 执行加 2 操作，跳过了加 1 后的那个位置
if(flowerbed[i] == 1){

    // 让 i 执行加 2 操作
    i += 2;

// 2、说明当前位置没有种花 flowerbed[i] == 0
// 3、如果这个位置是数组的最后一个位置，说明后一个位置不存在，没有限制，说明
flowerbed[i] 可以种花
// 4、如果这个位置【不是】数组的最后一个位置，那么只有后一个位置【没有种花】，才有资格在
flowerbed[i] 位置种花
}else if( i == flowerbed.size() - 1 || flowerbed[i + 1] == 0){

    // 以上两种条件都可以在 flowerbed[i] 位置种花
    // 成功之后，所需目标减 1
    n--;

    // 在 flowerbed[i] 位置种花之后，i + 1 位置不需要去考虑了，因为它明显不能种花，可
以跳过去

    // 让 i 执行加 2 操作
    i += 2;

// 5、说明当前位置没有种花 flowerbed[i] == 0
// 6、但是后一个位置已经种花了，那么当前位置无法采取种花操作
}else{

    // i + 1 位置已经种花，不用再去访问一遍
    // i + 2 位置考虑到 i + 1 位置已经种花，所以也无法种花，不用再去访问
    // 让 i 执行加 3 操作
    i += 3;
}

}

// 最后查看是否用完了 n
return n <= 0;

}

};

```

3、Python 代码

```

class Solution:
    def canPlaceFlowers(self, flowerbed: List[int], n: int) -> bool:
        # 遍历数组，在遍历过程中，采取贪心的思路，并不需要【每个位置】都去查看是否可以种花
        # 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去
        # 2、当前位置没有种花，需要考虑后面一个位置是否种花

```

```

i = 0

while i < len(flowerbed) and n > 0 :

    # 1、当前位置已经种花，那么后一个位置明显不能种花，可以跳过去
    # 所以让 i 执行加 2 操作，跳过了加 1 后的那个位置
    if flowerbed[i] == 1 :

        # 让 i 执行加 2 操作
        i += 2

    # 2、说明当前位置没有种花 flowerbed[i] == 0
    # 3、如果这个位置是数组的最后一个位置，说明后一个位置不存在，没有限制，说明 flowerbed[i]
可以种花
    # 4、如果这个位置【不是】数组的最后一个位置，那么只有后一个位置【没有种花】，才有资格在
flowerbed[i] 位置种花
    elif i == len(flowerbed) - 1 or flowerbed[i + 1] == 0 :

        # 以上两种条件都可以在 flowerbed[i] 位置种花
        # 成功之后，所需目标减 1
        n -= 1

        # 在 flowerbed[i] 位置种花之后，i + 1 位置不需要去考虑了，因为它明显不能种花，可以
跳过去

        # 让 i 执行加 2 操作
        i += 2

    # 5、说明当前位置没有种花 flowerbed[i] == 0
    # 6、但是后一个位置已经种花了，那么当前位置无法采取种花操作
    else:

        # i + 1 位置已经种花，不用再去访问一遍
        # i + 2 位置考虑到 i + 1 位置已经种花，所以也无法种花，不用再去访问
        # 让 i 执行加 3 操作
        i += 3

    # 最后查看是否用完了 n
return n <= 0

```

四、时空复杂度

- 时间复杂度： $O(n)$ 。
- 空间复杂度： $O(1)$ 。

LeetCode 860、柠檬水找零

一、题目描述

在柠檬水摊上，每一杯柠檬水的售价为 5 美元。顾客排队购买你的产品，（按账单 bills 支付的顺序）一次购买一杯。

每位顾客只买一杯柠檬水，然后向你付 5 美元、10 美元或 20 美元。

你必须给每个顾客正确找零，也就是说净交易是每位顾客向你支付 5 美元。

注意，一开始你手头没有任何零钱。

给你一个整数数组 bills，其中 bills[i] 是第 i 位顾客付的账。如果你能给每位顾客正确找零，返回 true，否则返回 false。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 柠檬水找零 ( LeetCode 860 ) :https://leetcode-cn.com/problems/lemonade-change/
class Solution {
    public boolean lemonadeChange(int[] bills) {

        // 用来记录 5 元钞票的数量
        int five = 0;

        // 用来记录 10 元钞票的数量
        int ten = 0;

        // 顾客开始按顺序购买，并找零
        for(int i = 0 ; i < bills.length ; i++){
            // 1、如果发现是 5 元面额
            if(bills[i] == 5){
                // 那么可以直接收钱，不需要找零
                // 并且 5 元钞票的数量加 1
                five++;

                // 2、如果发现是 10 元面额
            }else if(bills[i] == 10){

                // 如果手中有 5 元钞票，则找零 5 元
                if(five > 0){
                    // 5 元钞票的数量减 1
                    five--;
                    // 10 元钞票的数量加 1
```

```

        ten++;
    }else{
        // 如果手中没有 5 元钞票，说明找零失败
        return false;
    }

    // 3、如果发现是 20 元面额
}else{

    // 如果手中有 10 元 和 5 元钞票，则找零 1 张 10 元和 1 张 5 元的钞票
    if(ten > 0 && five > 0){
        // 5 元钞票的数量减 1
        five--;
        // 10 元钞票的数量减 1
        ten--;
    }else{
        // 如果手中只有 5 元的，并且数量超过或者等于 3 张
        // 那么找零 3 张 5 元的钞票
        if(five >= 3){
            // 5 元钞票的数量减 3
            five -= 3;
        }else{
            // 说明这个时候顾客付 20 元的时候
            // 1、手中没有 1 张 10 元的和 1 张 5 元的
            // 2、手中没有 3 张 5 元的
            // 说明找零失败
            return false;
        }
    }
}

// 所有顾客都找零了，成功
return true;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 柠檬水找零 ( LeetCode 860 ) :https://leetcode-cn.com/problems/lemonade-change/
class Solution {
public:
    bool lemonadeChange(vector<int>& bills) {
        // 用来记录 5 元钞票的数量
        int five = 0;
    }
}

```

```

// 用来记录 10 元钞票的数量
int ten = 0;

// 顾客开始按顺序购买，并找零
for(int i = 0 ; i < bills.size() ; i++){
    // 1、如果发现是 5 元面额
    if(bills[i] == 5){
        // 那么可以直接收钱，不需要找零
        // 并且 5 元钞票的数量加 1
        five++;

        // 2、如果发现是 10 元面额
    }else if(bills[i] == 10){

        // 如果手中有 5 元钞票，则找零 5 元
        if(five > 0){
            // 5 元钞票的数量减 1
            five--;
            // 10 元钞票的数量加 1
            ten++;
        }else{
            // 如果手中没有 5 元钞票，说明找零失败
            return false;
        }

        // 3、如果发现是 20 元面额
    }else{

        // 如果手中有 10 元 和 5 元钞票，则找零 1 张 10 元和 1 张 5 元的钞票
        if(ten > 0 && five > 0){
            // 5 元钞票的数量减 1
            five--;
            // 10 元钞票的数量减 1
            ten--;
        }else{
            // 如果手中只有 5 元的，并且数量超过或者等于 3 张
            // 那么找零 3 张 5 元的钞票
            if(five >= 3){
                // 5 元钞票的数量减 3
                five -= 3;
            }else{
                // 说明这个时候顾客付 20 元的时候
                // 1、手中没有 1 张 10 元的和 1 张 5 元的
                // 2、手中没有 3 张 5 元的
                // 说明找零失败
                return false;
            }
        }
    }
}

```

```

    }
    // 所有顾客都找零了，成功
    return true;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 柠檬水找零 ( LeetCode 860 ) :https://leetcode-cn.com/problems/lemonade-change/
class Solution:
    def lemonadeChange(self, bills: List[int]) -> bool:
        # 用来记录 5 元钞票的数量
        five = 0

        # 用来记录 10 元钞票的数量
        ten = 0

        # 顾客开始按顺序购买，并找零
        for i in range(len(bills)) :
            # 1、如果发现是 5 元面额
            if bills[i] == 5 :
                # 那么可以直接收钱，不需要找零
                # 并且 5 元钞票的数量加 1
                five += 1

            # 2、如果发现是 10 元面额
            elif bills[i] == 10 :

                # 如果手中有 5 元钞票，则找零 5 元
                if five > 0 :
                    # 5 元钞票的数量减 1
                    five -= 1
                    # 10 元钞票的数量加 1
                    ten += 1
                else :
                    # 如果手中没有 5 元钞票，说明找零失败
                    return False

            # 3、如果发现是 20 元面额
            else :

                # 如果手中有 10 元 和 5 元钞票，则找零 1 张 10 元和 1 张 5 元的钞票
                if ten > 0 and five > 0 :
                    # 5 元钞票的数量减 1
                    five -= 1

```

```

        # 10 元钞票的数量减 1
        ten -= 1
    else:
        # 如果手中只有 5 元的，并且数量超过或者等于 3 张
        # 那么找零 3 张 5 元的钞票
        if five >= 3 :
            # 5 元钞票的数量减 3
            five -= 3
        else :
            # 说明这个时候顾客付 20 元的时候
            # 1、手中没有 1 张 10 元的和 1 张 5 元的
            # 2、手中没有 3 张 5 元的
            # 说明找零失败
            return False

    # 所有顾客都找零了，成功
    return True

```

四、复杂度分析

时间复杂度：O(N)，其中 N 是 bills 的长度。

空间复杂度：O(1)。

第十八天

LeetCode 402 、移掉 K 位数字

一、题目描述

给你一个以字符串表示的非负整数 `num` 和一个整数 `k`，移除这个数中的 `k` 位数字，使得剩下的数字最小。请你以字符串形式返回这个最小的数字。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 移掉 k 位数字 ( LeetCode 402 ) :https://leetcode-cn.com/problems/remove-k-digits/submissions/
class Solution {
    public String removeKdigits(String num, int k) {

```

```

// 初始化栈，用来存储需要保留的数字
Stack<Character> stack = new Stack<Character>();

// 初始化字符串，用来保留最后的结果
StringBuilder result = new StringBuilder();

// 从左到右遍历字符串 num
for(int i = 0 ; i < num.length();i++){

    // 获取此时遍历的字符
    char digit = num.charAt(i);

    // 如果此时
    // 1、栈不为空
    // 2、栈顶元素大于此时遍历的字符
    // 3、还没有删除足够多的数字，即 k > 0
    // 那么这个时候需要把栈顶元素弹出
    while(!stack.isEmpty() && stack.peek() > digit && k > 0){
        // 把栈顶元素弹出
        stack.pop();
        // 记录删除的个数减 1
        k--;
    }

    // 如果发现此时遍历的字符为 0 并且栈为空
    // 那么就不要把 0 放入到栈中，否则最终的结果会以 0 开头
    if(digit == '0' && stack.isEmpty()){
        continue;
    }

    // 把符合要求的字符放入到栈中
    stack.push(digit);
}

// 遍历完所有的字符后，如果发现还没有删除足够多的元素，那么需要继续删除
// 什么数字会出现这种情况呢？
// 比如数字为 123456789 ，删除的数字 k 为 1
while(!stack.isEmpty() && k > 0){
    // 不断的弹出栈顶元素
    stack.pop();
    // 直到 k 为 0 位置
    k--;
}

// 操作完毕之后，如果发现栈为空，按上面逻辑我们会返回字符 "" ，
// 但根据题目的示例 3，num = "10"， k = 2 时，从原数字移除所有的数字，剩余为空就是 0
// 需要返回 "0"
if (stack.isEmpty()) return "0";

```

```

// 如果栈中还有元素
while(!stack.isEmpty()){
    // 那么从栈顶到栈底把字符添加到 result 上
    result.append(stack.peek());
    // 同时不断的弹出栈顶元素
    stack.pop();
}

// 由于 stack 中的栈底是数字的最高位，栈顶是最低位
// 所以此时 result 保存的顺序是最低位到最高位
// 需要执行一次翻转操作
return result.reverse().toString();

}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 移除 K 位数字 ( LeetCode 402 ) :https://leetcode-cn.com/problems/remove-k-digits/submissions/
class Solution {
public:
    string removeKdigits(string num, int k) {
        // 初始化栈，用来存储需要保留的数字
        stack<char> s;

        // 初始化字符串，用来保留最后的结果
        string result = "" ;

        // 从左到右遍历字符串 num
        for(int i = 0 ; i < num.length();i++){

            // 获取此时遍历的字符
            int digit = num[i];

            // 如果此时
            // 1、栈不为空
            // 2、栈顶元素大于此时遍历的字符
            // 3、还没有删除足够多的数字，即 k > 0
            // 那么这个时候需要把栈顶元素弹出
            while(!s.empty() && s.top() > digit && k > 0){
                // 把栈顶元素弹出
                s.pop();
                // 记录删除的个数减 1
                k--;
            }

```

```

    }

    // 如果发现此时遍历的字符为 0 并且栈为空
    // 那么就不要把 0 放入到栈中，否则最终的结果会以 0 开头
    if(digit == '0' && s.empty()){
        continue;
    }

    // 把符合要求的字符放入到栈中
    s.push(digit);
}

// 遍历完所有的字符后，如果发现还没有删除足够多的元素，那么需要继续删除
// 什么数字会出现这种情况呢？
// 比如数字为 123456789，删除的数字 k 为 1
while(!s.empty() && k > 0){
    // 不断的弹出栈顶元素
    s.pop();
    // 直到 k 为 0 位置
    k--;
}

// 操作完毕之后，如果发现栈为空，按上面逻辑我们会返回字符 ""，
// 但根据题目的示例 3，num = "10"，k = 2 时，从原数字移除所有的数字，剩余为空就是 0
// 需要返回 "0"
if (s.empty()) return "0";

// 如果栈中还有元素
while(!s.empty()){
    // 那么从栈顶到栈底把字符添加到 result 上
    result += s.top();
    // 同时不断的弹出栈顶元素
    s.pop();
}

// 由于 s 中的栈底是数字的最高位，栈顶是最低位
// 所以此时 result 保存的顺序是最低位到最高位
// 需要执行一次翻转操作
reverse(result.begin(), result.end());

return result;
}
};

```


3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

移除 K 位数字 (LeetCode 402) :<https://leetcode-cn.com/problems/remove-k-digits/submissions/>

class Solution:

def removeKdigits(self, num: str, k: int) -> str:

初始化栈，用来存储需要保留的数字

stack = []

从左到右遍历字符串 num

for digit in num:

如果此时

1、栈不为空

2、栈顶元素大于此时遍历的字符

3、还没有删除足够多的数字，即 $k > 0$

那么这个时候需要把栈顶元素弹出

while k and stack and stack[-1] > digit:

stack.pop()

k -= 1

把符合要求的字符放入到栈中

stack.append(digit)

return ''.join(stack[:len(stack) - k]).lstrip('0') or "0"

LeetCode 435、无重叠区间

一、题目描述

给定一个区间的集合 `intervals`，其中 `intervals[i] = [starti, endi]`。返回 需要移除区间的最小数量，使剩余区间互不重叠。

示例 1:

输入: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

输出: 1

解释: 移除 `[1,3]` 后，剩下的区间没有重叠。

示例 2:

输入: `intervals = [ [1,2], [1,2], [1,2] ]`
输出: 2
解释: 你需要移除两个 `[1,2]` 来使剩下的区间没有重叠。

示例 3:

输入: `intervals = [ [1,2], [2,3] ]`
输出: 0
解释: 你不需要移除任何区间, 因为它们已经是无重叠的了。

提示:

- `1 <= intervals.length <= 10^5`
- `intervals[i].length == 2`
- `-5 * 10^4 <= starti < endi <= 5 * 10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// LeetCode 435、无重叠区间: https://leetcode.cn/problems/non-overlapping-intervals/
class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {

        // 第一步, 先将原数组进行一个排序的操作
        // 左端排序和右端排序均可, 这里采取左端排序的方式
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        // 此时, 第一个元素的位置在第一个数组的末端, 即最右侧位置
        // 即末端位置
        int end = intervals[0][1];

        // 需要移除的数量
        int count = 0;

        // 开始遍历排序好后的那些区间元素
        // 这些元素按照开始端的大小进行了升序排序
        for (int i = 1; i < intervals.length; i++) {

            // 在遍历过程中, 只要发现当前元素的左端位置在之前某个元素的内部
            // 1、说明发生了重叠
            if (intervals[i][0] < end) {
```

```

        // 此时必须要移除一个
        // 移除尾部更大的那个元素
        // 因为它更长，会占据更大的区间
        // end 更新
        end = Math.min(end, intervals[i][1]);

        // 移除了一个
        count++;

        // 2、如果没有重叠，就不需要移除
    } else {

        // 只需要更新尾部的位置即可
        end = intervals[i][1];
    }
}

// 返回结果
return count;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        std::sort(intervals.begin(), intervals.end(), [](const std::vector<int>& a,
const std::vector<int>& b) {
            return a[0] < b[0];
        });

        int end = intervals[0][1];
        int count = 0;

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < end) {
                end = std::min(end, intervals[i][1]);
                count++;
            } else {
                end = intervals[i][1];
            }
        }

        return count;
    }
};

```

3、Python 代码

```
class solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[0]) # 左端排序

        end = intervals[0][1] # 末端位置
        count = 0 # 需要移除的数量

        for i in range(1, len(intervals)):
            if intervals[i][0] < end: # 发生重叠
                end = min(end, intervals[i][1]) # 更新末端位置
                count += 1 # 移除了一个
            else: # 没有重叠
                end = intervals[i][1] # 更新末端位置

        return count
```

LeetCode 452、用最少数量的箭引爆气球

一、题目描述

在二维空间中有许多球形的气球。对于每个气球，提供的输入是水平方向上，气球直径的开始和结束坐标。由于它是水平的，所以纵坐标并不重要，因此只要知道开始和结束的横坐标就足够了。开始坐标总是小于结束坐标。

一支弓箭可以沿着 x 轴从不同点完全垂直地射出。在坐标 x 处射出一支箭，若有一个气球的直径的开始和结束坐标为 $x(\text{start})$, $x(\text{end})$ ，且满足 $x(\text{start}) \leq x \leq x(\text{end})$ ，则该气球会被引爆。可以射出的弓箭的数量没有限制。弓箭一旦被射出之后，可以无限地前进。我们想找到使得所有气球全部被引爆，所需的弓箭的最小数量。

给你一个数组 `points`，其中 `points[i] = [xstart, xend]`，返回引爆所有气球所必须射出的最小弓箭数。

示例 1:

输入: `points = [[10,16],[2,8],[1,6],[7,12]]`

输出: 2

解释: 对于该样例， $x = 6$ 可以射爆 `[2,8]`, `[1,6]` 两个气球，以及 $x = 11$ 射爆另外两个气球

示例 2:

输入: `points = [[1,2],[3,4],[5,6],[7,8]]`

输出: 4

示例 3:

输入: `points = [[1,2],[2,3],[3,4],[4,5]]`

输出: 2

示例 4:

输入: points = [[1,2]]
输出: 1

示例 5:

输入: points = [[2,3],[2,3]]
输出: 1

提示:

- `1 <= points.length <= 104`
- `points[i].length == 2`
- `-231 <= xstart < xend <= 231 - 1`

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/minimum-number-of-arrows-to-burst-balloons/
class solution {
    public int findMinArrowShots(int[][] points) {

        // 第一步, 先将原数组进行一个排序的操作
        // 左端排序和右端排序均可, 但右端排序更方便一些
        // 注意: 是按照右端升序进行排序的, 原因在文章中进行说明
        // 本题包含了一个测试用例: [[-2147483646, -2147483645], [2147483646, 2147483647]]
        // 在 java 语言中, 如果使用 Arrays.sort(int[], Comparator) 这个 API, 由于重写
        // Comparator 中的 compare 方法需要返回一个 int 类型的值
        // 一般会直接使用两数相减来直接得出这两个数的相对顺序
        // 比如 return o1[0] - o2[0]
        // 那么这个测试用例得出 -2147483646 - 2147483646 就会溢出
        // 所以不使用这种返回方式, 而是使用三目运算符
        Arrays.sort(points, new Comparator<int[]>() {
            public int compare(int[] point1, int[] point2) {
                // points 是一个二维数组, 比如为 [[10,16],[2,8],[1,6],[7,12]]
                // point1 是其中的一个对象, 比如为 [10,16]
                // point2 是其中的一个对象, 比如为 [2,8]

                if (point1[1] > point2[1]) {
                    // 代表需要交换位置
                    return 1;
                } else if (point1[1] < point2[1]) {
                    // 代表不交换位置
                    return -1;
                }
            }
        });
    }
}
```

```

        } else {
            // 不交换位置，不排序
            return 0;
        }
    }
}

// 此时，第一只箭的位置在第一个数组的末端，即最右侧位置
int pos = points[0][1];

// res 代表箭的数量
int res = 1;

// 开始遍历排序好后的那些气球数组
// 这些气球数组按照末端的大小进行了升序排序
for (int[] balloon: points) {

    // 在遍历过程中，只要发现当前气球的左端位置超过了当前箭头的位置
    // 那么代表着此时这只 pos 箭无法引爆 balloon 这个气球
    // 那么需要增加其它的箭
    if (balloon[0] > pos) {

        // 新增的箭放在当前气球的最右端
        pos = balloon[1];

        // 同时，更新了箭的数量
        res++;
    }
}

return res;
}
}

```

补充：采取左端排序的版本

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/minimum-number-of-arrows-to-burst-balloons/
class Solution {
    public int findMinArrowShots(int[][] points) {

        /*
        左端排序版本
        */
        // 第一步，先将原数组进行一个排序的操作
        // 左端排序和右端排序均可，但右端排序更方便一些
        // 本题包含了一个测试用例：[[-2147483646, -2147483645], [2147483646, 2147483647]]

```

```

// 在 java 语言中, 如果使用 Arrays.sort(int[],Comparator) 这个 API , 由于重写
Comparator 中的 compare 方法需要返回一个 int 类型的值
// 一般会直接使用两数相减来直接得出这两个数的相对顺序
// 比如 return o1[0] - o2[0]
// 那么这个测试用例得出 -2147483646 - 2147483646 就会溢出
// 所以不使用这种返回方式, 而是使用三目运算符
Arrays.sort(points, new Comparator<int[]>() {
    public int compare(int[] point1, int[] point2) {
        // points 是一个二维数组, 比如为 [[10,16],[2,8],[1,6],[7,12]]
        // point1 是其中的一个对象, 比如为 [10,16]
        // point2 是其中的一个对象, 比如为 [2,8]
        // return point1[1] - point2[1];
        if (point1[0] > point2[0]) {
            // 代表需要交换位置
            return 1;
        } else if (point1[0] < point2[0]) {
            // 代表不交换位置
            return -1;
        } else {
            // 不交换位置, 不排序
            return 0;
        }
    }
});

// 此时, 第一只箭的位置在第一个数组的末端, 即最右侧位置
int pos = points[0][1];

// res 代表箭的数量
int res = 1;

// 开始遍历排序好后的那些气球数组
// 这些气球数组按照左端的大小进行了升序排序
for (int i = 0; i < points.length - 1; i++) {
    int[] balloon = points[ i + 1 ];
    // 在遍历过程中, 只要发现当前气球的左端位置超过了当前箭头的位置
    // 那么代表着此时这只 pos 箭无法引爆 balloon 这个气球
    // 那么需要增加其它的箭
    if (pos < balloon[0] ) {

        // 新增的箭放在当前气球的最右端
        pos = balloon[1];

        // 同时, 更新了箭的数量
        res++;
    } else if ( pos > balloon[1] ){

        pos = balloon[1];
    }
}

```

```

    }
}
return res;
}
}

```

C++

```

class Solution {
public:
    int findMinArrowShots(vector<vector<int>>& points) {

        // 第一步, 先将原数组进行一个排序的操作
        sort(points.begin(), points.end(), [](const vector<int>& u, const vector<int>&
v) {
            return u[1] < v[1];
        });

        // 此时, 第一只箭的位置在第一个数组的末端, 即最右侧位置
        int pos = points[0][1];

        // res 代表箭的数量
        int res = 1;

        // 开始遍历排序好后的那些气球数组
        // 这些气球数组按照末端的大小进行了升序排序
        for (vector<int> balloon: points) {

            // 在遍历过程中, 只要发现当前气球的左端位置超过了当前箭头的位置
            // 那么代表着此时这只 pos 箭无法引爆 balloon 这个气球
            // 那么需要增加其它的箭
            if (balloon[0] > pos) {

                // 新增的箭放在当前气球的最右端
                pos = balloon[1];

                // 同时, 更新了箭的数量
                res++;
            }
        }
        return res;
    }
};

```



```
class solution:
    def findMinArrowShots(self, points: List[List[int]]) -> int:

        # 第一步，先将原数组进行一个排序的操作
        points.sort(key=lambda balloon: balloon[1])

        # 此时，第一只箭的位置在第一个数组的末端，即最右侧位置
        pos = points[0][1]

        # res 代表箭的数量
        res = 1

        # 开始遍历排序好后的那些气球数组
        # 这些气球数组按照末端的大小进行了升序排序
        for balloon in points:

            # 在遍历过程中，只要发现当前气球的左端位置超过了当前箭头的位置
            # 那么代表着此时这只 pos 箭无法引爆 balloon 这个气球
            # 那么需要增加其它的箭
            if balloon[0] > pos :

                # 新增的箭放在当前气球的最右端
                pos = balloon[1]

            # 同时，更新了箭的数量
            res += 1

        return res
```

四、复杂度分析

- 时间复杂度： $O(n\log n)$ ，其中 n 是数组 `points` 的长度。排序的时间复杂度为 $O(n\log n)$ ，对所有气球进行遍历并计算答案的时间复杂度为 $O(n)$ ，其在渐进意义下小于前者，因此可以忽略。
- 空间复杂度： $O(\log n)$ ，即为排序需要使用的栈空间。

第十九天

LeetCode 1025、除数博弈

一、题目描述

爱丽丝和鲍勃一起玩游戏，他们轮流行动。爱丽丝先手开局。

最初，黑板上有一个数字 n 。在每个玩家的回合，玩家需要执行以下操作：

- 选出任一 x ，满足 $0 < x < n$ 且 $n \% x == 0$ 。
- 用 $n - x$ 替换黑板上的数字 n 。

如果玩家无法执行这些操作，就会输掉游戏。

只有在爱丽丝在游戏中取得胜利时才返回 `true`。假设两个玩家都以最佳状态参与游戏。

示例 1：

输入： $n = 2$

输出：`true`

解释：爱丽丝选择 1，鲍勃无法进行操作。

示例 2：

输入： $n = 3$

输出：`false`

解释：爱丽丝选择 1，鲍勃也选择 1，然后爱丽丝无法进行操作。

提示：

- $1 \leq n \leq 1000$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 除数博弈 (LeetCode 1025) : https://leetcode.cn/problems/divisor-game/
class Solution {
    public boolean divisorGame(int n) {

        return n % 2 == 0;

    }
}
```

2、**C++** 代码

```
class solution {
public:
    bool divisorGame(int n) {
        return n % 2 == 0;
    }
};
```

3、Python 代码

```
class Solution:
    def divisorGame(self, n: int) -> bool:
        return n % 2 == 0
```

LeetCode 15、三数之和

一、题目描述

给你一个包含 `n` 个整数的数组 `nums`，判断 `nums` 中是否存在三个元素 `a`，`b`，`c`，使得 $a + b + c = 0$ ？请你找出所有和为 `0` 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 三数之和(15): https://leetcode-cn.com/problems/3sum/
class Solution {
    public static List<List<Integer>> threeSum(int[] nums) {

        // 题目存在多组解，每一组解都是一个数组，所以使用二维数组存储所有的解
        List<List<Integer>> ans = new ArrayList();

        // 获取数组的长度
        int len = nums.length;

        // 边界情况判断
        if(nums == null || len < 3) return ans;

        // 先将数组进行排序操作，从小到大
        Arrays.sort(nums);
```

```

// 开始遍历整个数组
// 在遍历的过程中，观察设置的三个位置的元素之后的大小
// num[i] 为从左到右观察过去的元素
// left 为从 i 到 len - 1 的元素
// right 为从 len - 1 向左移动到 i 的元素
for (int i = 0; i < len ; i++) {

    // 如果发现 nums[i] > 0 ，由于 nums 是递增序列，left 在 nums[i] 的右侧，right 也在
nums[i] 的右侧
    // 那么 num[i]、nums[left]、nums[right] 都大于 0
    // 说明这三数之和一定大于 0 ， 结束循环
    if(nums[i] > 0) break;

    // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
    // 比如这个输入 [-4,-1,-1,0,1,2]
    // i 指向的为第一个 -1 时，left 指向的元素值为 0 ，right 指向的元素值为 1
    // i 指向的为第二个 -1 时，left 指向的元素值为 0 ，right 指向的元素值为 1
    // 这两组解都是 [ -1 ， 0 ， 1]，所以需要去重
    if(i > 0 && nums[i] == nums[ i - 1 ]) continue;

    // left 为从 i 到 len - 1 的元素，向右移动
    int left = i + 1;

    // right 为从 len - 1 向左移动到 i 的元素，向左移动
    int right = len - 1;

    // left 和 right 不断的向内移动
    while(left < right){

        // 计算三者之和
        int sum = nums[i] + nums[left] + nums[right];

        // 发现三者之和为 0
        if(sum == 0){

            // 把这个结果记录一下
            ans.add(Arrays.asList(nums[i],nums[left],nums[right]));

            // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
            // 比如这个输入 [-2,0,0,2,2]
            // i 指向的元素值为 -2 ，left 指向的元素值为第一个 0 ，right 指向的元素值为
倒数第一个 2 时
            // 它们的 sum 为 0 ，如果让 ，left 向右移动一下，，right 向左移动一下，它们
的 sum 也为 0
            // 但是这两组解都是 [ -2 ， 0 ， 2]，所以需要去重
            while( left < right && nums[left] == nums[ left + 1 ]) {
                // left 向右移动
                left++;
            }
        }
    }
}

```

```

    }

    while( left < right && nums[right] == nums[ right - 1]){
        // right 向左移动
        right--;
    }

    // left 向右移动
    left++;

    // right 向左移动
    right--;

    // 如果三者之和小于 0 ， 那么说明需要找更大的数
}else if (sum < 0){
    // left 向右移动
    left++;

    // 如果三者之和大于 0 ， 那么说明需要找更小的数
}else if (sum > 0) {
    // right 向左移动
    right--;
}
}
}
// 返回结果
return ans;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 三数之和(15)：https://leetcode-cn.com/problems/3sum/
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        // 题目存在多组解，每一组解都是一个数组，所以使用二维数组存储所有的解
        vector<vector<int>> ans;

        // 获取数组的长度
        int len = nums.size();

        // 边界情况判断
        if(len < 3) return ans;
    }
}

```

```

// 先将数组进行排序操作，从小到大
sort(nums.begin(), nums.begin() + len);

// 开始遍历整个数组
// 在遍历的过程中，观察设置的三个位置的元素之后的大小
// num[i] 为从左到右观察过去的元素
// left 为从 i 到 len - 1 的元素
// right 为从 len - 1 向左移动到 i 的元素
for (int i = 0; i < len ; i++) {

    // 如果发现 nums[i] > 0 ，由于 nums 是递增序列，left 在 nums[i] 的右侧，right 也在
    nums[i] 的右侧
    // 那么 num[i]、nums[left]、nums[right] 都大于 0
    // 说明这三数之和一定大于 0 ， 结束循环
    if(nums[i] > 0) break;

    // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
    // 比如这个输入 [-4,-1,-1,0,1,2]
    // i 指向的为第一个 -1 时，left 指向的元素值为 0 ， right 指向的元素值为 1
    // i 指向的为第二个 -1 时，left 指向的元素值为 0，right 指向的元素值为 1
    // 这两组解都是 [ -1 ， 0 ， 1]，所以需要去重
    if(i > 0 && nums[i] == nums[ i - 1 ]) continue;

    // left 为从 i 到 len - 1 的元素，向右移动
    int left = i + 1;

    // right 为从 len - 1 向左移动到 i 的元素，向左移动
    int right = len - 1;

    // left 和 right 不断的向内移动
    while(left < right){

        // 计算三者之和
        int sum = nums[i] + nums[left] + nums[right];

        // 发现三者之和为 0
        if(sum == 0){

            // 把这个结果记录一下
            vector<int>v = {nums[i], nums[left], nums[right]};
            ans.push_back(v);

            // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
            // 比如这个输入 [-2,0,0,2,2]
            // i 指向的元素值为 -2 ， left 指向的元素值为第一个 0 ， right 指向的元素值为
            倒数第一个 2 时
            // 它们的 sum 为 0 ， 如果让 ， left 向右移动一下， ， right 向左移动一下，它们
            的 sum 也为 0
            // 但是这两组解都是 [ -2 ， 0 ， 2]，所以需要去重

```

```

        while( left < right && nums[left] == nums[ left + 1 ]) {
            // left 向右移动
            left++;
        }

        while( left < right && nums[right] == nums[ right - 1]){
            // right 向左移动
            right--;
        }

        // left 向右移动
        left++;

        // right 向左移动
        right--;

        // 如果三者之和小于 0 ， 那么说明需要找更大的数
    }else if (sum < 0){
        // left 向右移动
        left++;

        // 如果三者之和大于 0 ， 那么说明需要找更小的数
    }else if (sum > 0) {
        // right 向左移动
        right--;
    }
    }
}
// 返回结果
return ans;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 三数之和(15): https://leetcode-cn.com/problems/3sum/
class solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        # 题目存在多组解，每一组解都是一个数组，所以使用二维数组存储所有的解
        ans = []

        # 边界情况判断
        if nums == None or len(nums) < 3 :
            return ans

```

```

# 先将数组进行排序操作，从小到大
nums.sort()

# 开始遍历整个数组
# 在遍历的过程中，观察设置的三个位置的元素之后的大小
# num[i] 为从左到右观察过去的元素
# left 为从 i 到 len - 1 的元素
# right 为从 len - 1 向左移动到 i 的元素
for i in range(len(nums)) :

    # 如果发现 nums[i] > 0 ，由于 nums 是递增序列，left 在 nums[i] 的右侧，right 也在
nums[i] 的右侧
    # 那么 num[i]、nums[left]、nums[right] 都大于 0
    # 说明这三数之和一定大于 0 ，结束循环
    if nums[i] > 0 :
        break

    # 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
    # 比如这个输入 [-4,-1,-1,0,1,2]
    # i 指向的为第一个 -1 时，left 指向的元素值为 0 ，right 指向的元素值为 1
    # i 指向的为第二个 -1 时，left 指向的元素值为 0 ，right 指向的元素值为 1
    # 这两组解都是 [-1 , 0 , 1]，所以需要去重
    if i > 0 and nums[i] == nums[ i - 1 ] :
        continue

    # left 为从 i 到 len - 1 的元素，向右移动
    left = i + 1

    # right 为从 len - 1 向左移动到 i 的元素，向左移动
    right = len(nums) - 1

    # left 和 right 不断的向内移动
    while left < right :

        # 计算这三者之和
        sum = nums[i] + nums[left] + nums[right]

        # 发现三者之和为 0
        if sum == 0 :

            # 把这个结果记录一下
            ans.append([nums[i],nums[left],nums[right]])

        # 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
        # 比如这个输入 [-2,0,0,2,2]
        # i 指向的元素值为 -2 ，left 指向的元素值为第一个 0 ，right 指向的元素值为倒
数第一个 2 时

```



```

sum 也为 0

# 它们的 sum 为 0 , 如果让 , left 向右移动一下, , right 向左移动一下, 它们的

# 但是这两组解都是 [ -2 , 0 , 2], 所以需要去重
while left < right and nums[left] == nums[ left + 1 ] :
    # left 向右移动
    left += 1

while left < right and nums[right] == nums[ right - 1 ] :
    # right 向左移动
    right -= 1

# left 向右移动
left += 1

# right 向左移动
right -= 1

# 如果三者之和小于 0 , 那么说明需要找更大的数
elif sum < 0 :
    # left 向右移动
    left += 1

# 如果三者之和大于 0 , 那么说明需要找更小的数
elif sum > 0 :
    # right 向左移动
    right -= 1

# 返回结果
return ans

```

四、复杂度分析

时间复杂度： $O(N^2)$ ，其中 N 是数组 `nums` 的长度。

空间复杂度： $O(\log N)$ 。

LeetCode 16、最接近三数之和

一、题目描述

给你一个长度为 `n` 的整数数组 `nums` 和一个目标值 `target`。请你从 `nums` 中选出三个整数，使它们的和与 `target` 最接近。

返回这三个数的和。

假定每组输入只存在恰好一个解。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最接近的三数之和(16): https://leetcode-cn.com/problems/3sum-closest/submissions/
class Solution {
    public int threeSumClosest(int[] nums, int target) {

        // 题目假定每组输入只存在恰好一个解，所以不需要处理边界情况
        int ans = nums[0] + nums[1] + nums[2];

        // 先将数组进行排序操作，从小到大
        Arrays.sort(nums);

        // 开始遍历整个数组
        // 在遍历的过程中，观察设置的三个位置的元素之后的大小
        // num[i] 为从左到右观察过去的元素
        // left 为从 i 到 len - 1 的元素
        // right 为从 len - 1 向左移动到 i 的元素
        for (int i = 0; i < nums.length ; i++) {

            // left 为从 i 到 len - 1 的元素，向右移动
            int left = i + 1;

            // right 为从 len - 1 向左移动到 i 的元素，向左移动
            int right = nums.length - 1;

            // left 和 right 不断的向内移动
            while(left < right){

                // 计算这三者之和
                int sum = nums[i] + nums[left] + nums[right];

                // 寻找出更接近于 target 的那个和
                if(Math.abs(target - sum) < Math.abs(target - ans)){
                    ans = sum;
                }

                // 如果三者之和小于 target ，那么说明需要找更大的数
                if (sum < target){
                    // left 向右移动
                    left++;
                }

                // 如果三者之和大于 target ，那么说明需要找更小的数
            }
        }
    }
}
```

```

        }else if (sum > target) {
            // right 向左移动
            right--;
        }else{
            return ans;
        }
    }
}
// 返回结果
return ans;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最接近的三数之和(16)：https://leetcode-cn.com/problems/3sum-closest/submissions/
class Solution {
public:
    int threeSumClosest(vector<int>& nums, int target) {
        // 题目假定每组输入只存在恰好一个解，所以不需要处理边界情况
        int ans = nums[0] + nums[1] + nums[2];

        // 先将数组进行排序操作，从小到大
        sort(nums.begin(), nums.end());

        // 开始遍历整个数组
        // 在遍历的过程中，观察设置的三个位置的元素之后的大小
        // num[i] 为从左到右观察过去的元素
        // left 为从 i 到 len - 1 的元素
        // right 为从 len - 1 向左移动到 i 的元素
        for (int i = 0; i < nums.size() ; i++) {

            // left 为从 i 到 len - 1 的元素，向右移动
            int left = i + 1;

            // right 为从 len - 1 向左移动到 i 的元素，向左移动
            int right = nums.size() - 1;

            // left 和 right 不断的向内移动
            while(left < right){

                // 计算三者之和
                int sum = nums[i] + nums[left] + nums[right];

                // 寻找出更接近于 target 的那个和
            }
        }
    }
}

```

```

        if(abs(target - sum) < abs(target - ans)){
            ans = sum;
        }

        // 如果三者之和小于 target , 那么说明需要找更大的数
        if (sum < target){
            // left 向右移动
            left++;

            // 如果三者之和大于 target , 那么说明需要找更小的数
        }else if (sum > target) {
            // right 向左移动
            right--;
        }else{
            return ans;
        }
    }
}
// 返回结果
return ans;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 最接近的三数之和(16): https://leetcode-cn.com/problems/3sum-closest/submissions/
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        # 题目假定每组输入只存在恰好一个解, 所以不需要处理边界情况
        ans = nums[0] + nums[1] + nums[2]

        # 先将数组进行排序操作, 从小到大
        nums.sort()

        # 开始遍历整个数组
        # 在遍历的过程中, 观察设置的三个位置的元素之后的大小
        # num[i] 为从左到右观察过去的元素
        # left 为从 i 到 len - 1 的元素
        # right 为从 len - 1 向左移动到 i 的元素
        for i in range(len(nums)) :

            # left 为从 i 到 len - 1 的元素, 向右移动
            left = i + 1

            # right 为从 len - 1 向左移动到 i 的元素, 向左移动

```

```

right = len(nums) - 1

# left 和 right 不断的向内移动
while left < right :

    # 计算这三者之和
    sum = nums[i] + nums[left] + nums[right]

    # 寻找出更接近于 target 的那个和
    if abs(target - sum) < abs(target - ans) :
        ans = sum

    # 如果三者之和小于 target ，那么说明需要找更大的数
    if sum < target :
        # left 向右移动
        left += 1

    # 如果三者之和大于 target ，那么说明需要找更小的数
    elif sum > target :
        # right 向左移动
        right -= 1
    else :
        return ans

# 返回结果
return ans

```

四、复杂度分析

时间复杂度： $O(N^2)$ ，其中 N 是数组 `nums` 的长度。

空间复杂度： $O(\log N)$ 。

LeetCode 18、四数之和

一、题目描述

给你一个由 n 个整数组成的数组 `nums`，和一个目标值 `target`。请你找出并返回满足下述全部条件且不重复的四元组 `[nums[a], nums[b], nums[c], nums[d]]`（若两个四元组元素一一对应，则认为两个四元组重复）：

- $0 \leq a, b, c, d < n$
- `a`、`b`、`c` 和 `d` 互不相同
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

你可以按任意顺序返回答案。

示例 1：

输入: nums = [1,0,-1,0,-2,2], target = 0
输出: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

示例 2:

输入: nums = [2,2,2,2,2], target = 8
输出: [[2,2,2,2]]

提示:

- $1 \leq \text{nums.length} \leq 200$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 四数之和 (LeetCode 18) : https://leetcode.cn/problems/4sum/submissions/
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {

        // 题目存在多组解, 每一组解都是一个数组, 所以使用二维数组存储所有的解
        List<List<Integer>> ans = new ArrayList();

        // 边界情况判断
        if (nums == null || nums.length < 4) {
            return ans;
        }

        // 先将数组进行排序操作, 从小到大
        Arrays.sort(nums);

        // 获取数组的长度
        int len = nums.length;

        // 开始遍历整个数组
        // 在遍历的过程中, 观察设置的四个位置的元素之后的大小
        // 1、第一层循环
        // nums[i] 作为四个元素当中最小的那个元素
        // 需要留下 nums[len - 3]、nums[len - 2]、nums[len - 1] 这三个元素
        // 所以 i 的范围是 [ 0 , len - 4 ]
```

```

for (int i = 0; i <= len - 4; i++) {

    // 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
    if (i > 0 && nums[i] == nums[i - 1]) {
        continue;
    }

    // 如果发现当前区间中，最小的四个元素之和都大于了 target
    // 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第一层循环
    if ((long) nums[i] + nums[i + 1] + nums[i + 2] + nums[i + 3] > target) {
        break;
    }

    // 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
    // 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
    // 因此第一层循环直接进入下一轮，即执行 i++
    if ((long) nums[i] + nums[len - 3] + nums[len - 2] + nums[len - 1] <
target) {
        continue;
    }

    // 来到这里时，通过第一层循环，已经确定 nums[i] 这个数
    // 在 [ i + 1 , len - 1 ] 这个区间中寻找剩下的两个数
    // 2、第二层循环
    // nums[j] 作为四个元素当中第二小的那个元素
    // 需要留下 nums[len - 2] 、nums[len - 1] 这三个元素
    // 所以 j 的范围是 [ i + 1 , len - 3 ]
    for (int j = i + 1; j <= len - 3; j++) {

        // 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
        if (j > i + 1 && nums[j] == nums[j - 1]) {
            continue;
        }

        // 如果发现当前区间中，最小的四个元素之和都大于了 target
        // 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第二层循环
        if ((long) nums[i] + nums[j] + nums[j + 1] + nums[j + 2] > target) {
            break;
        }

        // 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
        // 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
        // 因此第二层循环直接进入下一轮，即执行 j++
        if ((long) nums[i] + nums[j] + nums[len - 2] + nums[len - 1] < target)
{
            continue;
        }

        // 否则的话，开始去寻找剩下的两个数

```

```

// 在 [ i + 1 , len - 2 ] 这个区间
int left = j + 1 ;

int right = len - 1;

// left 和 right 不断的向内移动
// 逻辑和三数之和的逻辑一样
while (left < right) {

    // 计算这四个元素之和
    long sum = (long)nums[i] + (long)nums[j] + (long)nums[left] +
(long)nums[right];

    // 发现四者之和为 0
    if (sum == target) {
        // 把这个结果记录一下
        ans.add(Arrays.asList(nums[i], nums[j], nums[left],
nums[right]));

        // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
        // 比如这个输入 [-2,0,0,2,2]
        // i 指向的元素值为 -2 ， left 指向的元素值为第一个 0 ， right 指向的元素
值为倒数第一个 2 时
        // 它们的 sum 为 0 ， 如果让 ， left 向右移动一下， ， right 向左移动一下，
它们的 sum 也为 0

        // 但是这两组解都是 [ -2 ， 0 ， 2]，所以需要去重
        while (left < right && nums[left] == nums[left + 1]) {
            // left 向右移动
            left++;
        }

        while (left < right && nums[right] == nums[right - 1]) {
            // right 向左移动
            right--;
        }

        // left 向右移动
        left++;

        // right 向左移动
        right--;

        // 如果四者之和小于 0 ， 那么说明需要找更大的数
    } else if (sum < target) {
        // left 向右移动
        left++;

        // 如果四者之和大于 0 ， 那么说明需要找更小的数

```



```

        } else {
            // right 向左移动
            right--;
        }
    }
}

// 返回结果
return ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        // 题目存在多组解，每一组解都是一个数组，所以使用二维数组存储所有的解
        vector<vector<int>> ans;

        // 边界情况判断
        if ( nums.size() < 4) {
            return ans;
        }

        // 获取数组的长度
        int len = nums.size();

        // 先将数组进行排序操作，从小到大
        sort(nums.begin(), nums.begin() + len);

        // 开始遍历整个数组
        // 在遍历的过程中，观察设置的四个位置的元素之后的大小
        // 1、第一层循环
        // nums[i] 作为四个元素当中最小的那个元素
        // 需要留下 nums[len - 3] 、nums[len - 2] 、nums[len - 1] 这三个元素
        // 所以 i 的范围是 [ 0 , len - 4 ]
        for (int i = 0; i <= len - 4; i++) {

            // 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            // 如果发现当前区间中，最小的四个元素之和都大于了 target
            // 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第一层循环
            if ((long) nums[i] + nums[i + 1] + nums[i + 2] + nums[i + 3] > target) {

```

```

        break;
    }

    // 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
    // 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
    // 因此第一层循环直接进入下一轮，即执行 i++
    if ((long) nums[i] + nums[len - 3] + nums[len - 2] + nums[len - 1] <
target) {
        continue;
    }

    // 来到这里时，通过第一层循环，已经确定 nums[i] 这个数
    // 在 [ i + 1 , len - 1 ] 这个区间中寻找剩下的两个数
    // 2、第二层循环
    // nums[j] 作为四个元素当中第二小的那个元素
    // 需要留下 nums[len - 2] 、nums[len - 1] 这三个元素
    // 所以 j 的范围是 [ i + 1 , len - 3 ]
    for (int j = i + 1; j <= len - 3; j++) {

        // 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
        if (j > i + 1 && nums[j] == nums[j - 1]) {
            continue;
        }

        // 如果发现当前区间中，最小的四个元素之和都大于了 target
        // 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第二层循环
        if ((long) nums[i] + nums[j] + nums[j + 1] + nums[j + 2] > target) {
            break;
        }

        // 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
        // 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
        // 因此第二层循环直接进入下一轮，即执行 j++
        if ((long) nums[i] + nums[j] + nums[len - 2] + nums[len - 1] < target)
{
            continue;
        }

        // 否则的话，开始去寻找剩下的两个数
        // 在 [ i + 1 , len - 2 ] 这个区间
        int left = j + 1 ;

        int right = len - 1;

        // left 和 right 不断的向内移动
        // 逻辑和三数之和的逻辑一样
        while (left < right) {

            // 计算这四个元素之和

```

```

        long sum = (long)nums[i] + (long)nums[j] + (long)nums[left] +
(long)nums[right];

        // 发现四者之和为 0
        if (sum == target) {
            // 把这个结果记录一下
            vector<int>v = {nums[i], nums[j],nums[left], nums[right]};

            ans.push_back(v);

            // 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
            // 比如这个输入 [-2,0,0,2,2]
            // i 指向的元素值为 -2，left 指向的元素值为第一个 0，right 指向的元素
值为倒数第一个 2 时
            // 它们的 sum 为 0，如果让 left 向右移动一下，right 向左移动一下，
            它们的 sum 也为 0

            // 但是这两组解都是 [-2, 0, 2]，所以需要去重
            while (left < right && nums[left] == nums[left + 1]) {
                // left 向右移动
                left++;
            }

            while (left < right && nums[right] == nums[right - 1]) {
                // right 向左移动
                right--;
            }

            // left 向右移动
            left++;

            // right 向左移动
            right--;

            // 如果四者之和小于 0，那么说明需要找更大的数
        } else if (sum < target) {
            // left 向右移动
            left++;

            // 如果四者之和大于 0，那么说明需要找更小的数
        } else {
            // right 向左移动
            right--;
        }
    }
}

// 返回结果

```

```
        return ans;

    }

};
```

3、Python 代码

```
class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        # 题目存在多组解，每一组解都是一个数组，所以使用二维数组存储所有的解
        ans = []

        # 边界情况判断
        if nums == None or len(nums) < 4 :
            return ans

        # 先将数组进行排序操作，从小到大
        nums.sort()

        # 获取数组的长度
        length = len(nums)

        # 开始遍历整个数组
        # 在遍历的过程中，观察设置的四个位置的元素之后的大小
        # 1、第一层循环
        # nums[i] 作为四个元素当中最小的那个元素
        # 需要留下 nums[length - 3] 、nums[length - 2] 、nums[length - 1] 这三个元素
        # 所以 i 的范围是 [ 0 , length - 4 ]
        for i in range(length - 3) :

            # 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            # 如果发现当前区间中，最小的四个元素之和都大于了 target
            # 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第一层循环
            if nums[i] + nums[i + 1] + nums[i + 2] + nums[i + 3] > target:
                break

            # 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
            # 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
            # 因此第一层循环直接进入下一轮，即执行 i++
            if nums[i] + nums[length - 3] + nums[length - 2] + nums[length - 1] <
target :

                continue
```

```

# 来到这里时，通过第一层循环，已经确定 nums[i] 这个数
# 在 [ i + 1 , length - 1 ] 这个区间中寻找剩下的两个数
# 2、第二层循环
# nums[j] 作为四个元素当中第二小的那个元素
# 需要留下 nums[length - 2] 、nums[length - 1] 这三个元素
# 所以 j 的范围是 [ i + 1 , length - 3 ]
for j in range( i + 1, length - 2) :

    # 答案中不可以包含重复的四元组，所以需要执行一个去重的操作
    if j > i + 1 and nums[j] == nums[j - 1] :
        continue

    # 如果发现当前区间中，最小的四个元素之和都大于了 target
    # 此时，剩下的三个数无论取什么值，四数之和一定大于 target，可以直接退出第二层循环
    if nums[i] + nums[j] + nums[j + 1] + nums[j + 2] > target :
        break

    # 如果发现当前区间中，选择最大的三个数和 nums[i] 相加都小于了 target
    # 说明此时剩下的三个数无论取什么值，四数之和一定小于 target
    # 因此第二层循环直接进入下一轮，即执行 j++
    if nums[i] + nums[j] + nums[length - 2] + nums[length - 1] < target:
        continue

    # 否则的话，开始去寻找剩下的两个数
    # 在 [ i + 1 , length - 2 ] 这个区间
    left = j + 1

    right = length - 1

    # left 和 right 不断的向内移动
    # 逻辑和三数之和的逻辑一样
    while left < right :

        # 计算这四个元素之和
        sum = nums[i] + nums[j] + nums[left] + nums[right]

        # 发现四者之和为 0
        if sum == target :
            # 把这个结果记录一下
            ans.append([nums[i], nums[j], nums[left], nums[right]])

        # 答案中不可以包含重复的三元组，所以需要执行一个去重的操作
        # 比如这个输入 [-2,0,0,2,2]
        # i 指向的元素值为 -2，left 指向的元素值为第一个 0，right 指向的元素

```

值为倒数第一个 2 时

```

# 它们的 sum 为 0 , 如果让 , left 向右移动一下, , right 向左移动一下,
它们的 sum 也为 0

# 但是这两组解都是 [ -2 , 0 , 2], 所以需要去重
while left < right and nums[left] == nums[left + 1] :
    # left 向右移动
    left += 1

while left < right and nums[right] == nums[right - 1] :
    # right 向左移动
    right -= 1

# left 向右移动
left += 1

# right 向左移动
right -= 1

# 如果四者之和小于 0 , 那么说明需要找更大的数
elif sum < target :
    # left 向右移动
    left += 1

# 如果四者之和大于 0 , 那么说明需要找更小的数
else :
    # right 向左移动
    right -= 1

# 返回结果
return ans

```

四、复杂度分析

- 时间复杂度： $O(n^3)$ ，其中 n 是数组的长度。
- 空间复杂度： $O(\log n^*)$ ，其中 n 是数组的长度。

第二十天

LeetCode 11、盛水最多的容器

一、题目描述

给你 n 个非负整数 $a_1, a_2, \dots, a_n$ ，每个数代表坐标中的一个点 (i, a_i) 。在坐标内画 n 条垂直线，垂直线 i 的两个端点分别为 (i, a_i) 和 $(i, 0)$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

二、题目解析

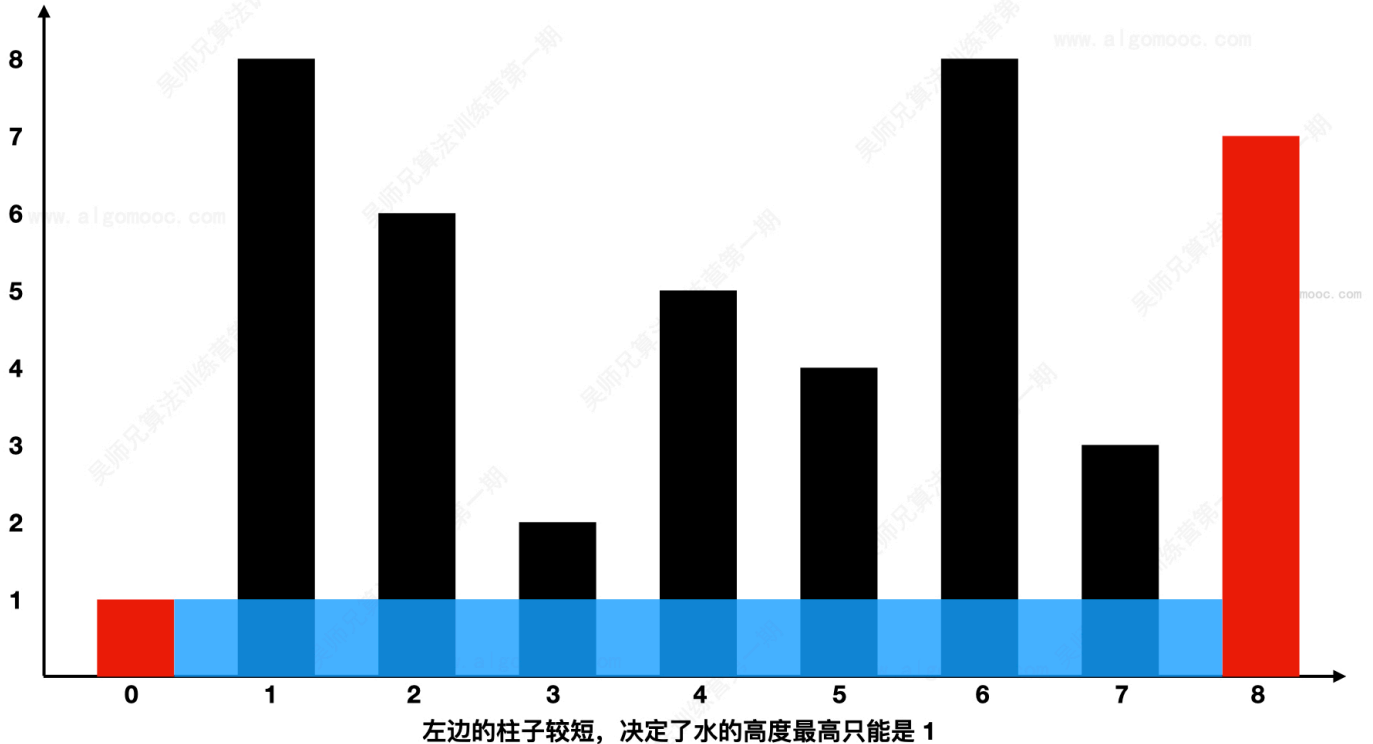
一开始，我们先去考虑相距最远的两个柱子所能容纳水的面积。

www.algomooc.com



给你 n 个非负整数 $a_1, a_2, \dots, a_n$ 每个数代表坐标中的一个点 (i, a_i) 。在坐标内画 n 条垂直线，垂直线 i 的两个端点分别为 (i, a_i) 和 $(i, 0)$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。



接下来去思考，我们去移动哪根柱子会更加合适？

这里我们需要注意一点：无论移动哪根柱子，柱子之间的宽度都是变小的。

移动右边那根更高的柱子？

由于水面高度是由最短的柱子决定的，所以移动右边那根更高的柱子的时候，水面高度一定是不会增加，甚至有可能遇到更短的柱子而变小，而宽度有一定再减少，所以水的面积也一定减少。

移动左边那根更短的柱子？

这时候，水的高度是不确定的，那么面积也是不确定的，有可能比之前更大，也有可能更小或者相等。

所以，我们可以得出一个结论：移动两根柱子之间更短的那根柱子，才有可能在宽度一定变小的情况下，找到一个更高的水面，从而使得面积有可能更大。

那接下来这道题目的解法也就有了：

1、设置两个索引，分别指向容器的两侧，即索引 `left` 指向最左边的柱子，索引 `right` 指向最右边的柱子。

2、记录下此时的水的面积，可以定义为 `res`

3、观察需要向内移动哪根柱子

- 1) 如果移动较高的柱子，由于水的宽度在变小，而水的高度一定不会增加，所以最终水的面积不会超过之前

记录的水的面积 res

- 2) 所以，只能移动较短的柱子，然后计算此时水的面积，再与之前记录的水的面积 res 进行比较，保存那个更大的值

4、再去判断应该向内移动哪根柱子

5、直到 left 和 right 相遇为止

三、参考代码

```
class solution {
    public int maxArea(int[] height) {

        // 设置两个索引，分别指向容器的两侧

        // 索引 left 指向最左边的柱子
        int left = 0;

        // 索引 right 指向最右边的柱子
        int right = height.length - 1;

        // 设置一个变量用来保存当下水的最大面积
        int res = 0;

        // 移动 left 和 right，直到 left 和 right 相遇为止
        while(left < right){

            // 水的宽度是 right - left
            int width = right - left;

            // 水的高度由两根柱子最短的那根决定
            int h = Math.min(height[left],height[right]);

            // 计算此时水的面积
            int area = width * h;

            // 如果此时水的面积大于了我们之前保存的那个值，我们需要更新一下
            if(area >= res){
                // 更新 res 的值为 area，确保 res 一直都是最大的值
                res = area;
            }

            // 接下来去观察需要移动哪根柱子：必定是最短的那根柱子

            // 如果左边的柱子更短，那么向内移动左边的柱子，因为只有这样，才有可能找到一个更高的水面
            // 在宽度一定变小的情况下，水的面积才有可能增大
            if(height[left] < height[right]){
                // 向内移动左边的柱子
                left++;
            }
        }
    }
}
```



```

        // 如果右边的柱子更短，那么向内移动右边的柱子，因为只有这样，才有可能找到一个更高的水面
        // 在宽度一定变小的情况下，水的面积才有可能增大
    }else{
        // 向内移动右边的柱子
        right--;
    }

}

// 最后返回最大的面积 res 即可
return res;
}

}

#### https://www.algomooc.com/587.html
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 盛最多水的容器 ( LeetCode 11) : https://leetcode-cn.com/problems/container-with-most-water/
class Solution:
    def maxArea(self, height: List[int]) -> int:
        # 设置两个索引，分别指向容器的两侧

        # 索引 left 指向最左边的柱子
        left = 0

        # 索引 right 指向最右边的柱子
        right = len(height) - 1

        # 设置一个变量用来保存当下水的最大面积
        res = 0

        # 移动 left 和 right，直到 left 和 right 相遇为止
        while left < right :

            # 水的宽度是 right - left
            width = right - left

            # 水的高度由两根柱子最短的那根决定
            h = min(height[left],height[right])

            # 计算此时水的面积
            area = width * h

            # 如果此时水的面积大于了我们之前保存的那个值，我们需要更新一下
            if area >= res :
                # 更新 res 的值为 area，确保 res 一直都是最大的值
                res = area

            # 接下来去观察需要移动哪根柱子：必定是最短的那根柱子

```

```

# 如果左边的柱子更短，那么向内移动左边的柱子，因为只有这样，才有可能找到一个更高的水面
# 在宽度一定变小的情况下，水的面积才有可能增大
if height[left] < height[right] :
    # 向内移动左边的柱子
    left += 1

# 如果右边的柱子更短，那么向内移动右边的柱子，因为只有这样，才有可能找到一个更高的水面
# 在宽度一定变小的情况下，水的面积才有可能增大
else:
    # 向内移动右边的柱子
    right -= 1

# 最后返回最大的面积 res 即可
return res

```

LeetCode 142、环形链表II

一、题目描述

给定一个链表，返回链表开始入环的第一个节点。

如果链表无环，则返回 null。为了表示给定链表中的环，我们使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。

如果 pos 是 -1，则在该链表中没有环。

注意，pos 仅仅是用于标识环的情况，并不会作为参数传递到函数中。

说明：不允许修改给定的链表。

二、题目解析

1、通过快慢指针的方式，在环中寻找它们的第一次相遇的节点位置

2、当快慢指针相遇的时候：

x 代表从头节点到环形入口节点的节点数（不包含头节点）

y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）

z 代表从第一次相遇节点到环形入口的节点数（不包含第一次相遇节点）

此时，快指针走了 $x + y + n(y + z)$ ，其中， $x + y$ 表示快指针第一次到达相遇节点， n 代表快指针在环里面绕了多少圈。

而慢指针走了 $x + y$ 步。

那么就出现了一个等式 $x + y = [x + y + n(y + z)] / 2$ ，即 $x = n(y + z) - y$ 。

$n(y + z) - y$ 代表的含义是一个指针从相遇节点开始出发，走了 n 圈之后回到原来的出发位置，往后退 y 步。

由于 x 代表从头节点到环形入口节点的节点数，并且 $x = n(y + z) - y$ ，所以 $n(y + z) - y$ 代表的含义就是一个指针从相遇节点开始出发，走了 n 圈之后回到原来的出发位置，往后退 y 步来到了环的入口位置。

那么，我们就可以设置两个指针，一个从链表的头节点开始出发，一个指针从相遇节点开始出发，当它们相遇的时候，代表着环的入口节点找到了。

补充：

- 1、当 $slow = 1$ ， $fast = 2$ ，为什么 $fast$ 和 $slow$ 一定会相遇？
- 2、当 $slow = 1$ ， $fast = 2$ ， $fast$ 和 $slow$ 相遇时， $slow$ 指针是否绕环超过一圈？
- 3、 $slow$ 和 $fast$ 的移动步数有什么规则？
- 4、能否设置为 $slow$ 每次移动 1 步， $fast$ 每次移动 3、4、5...步？
- 5、为什么设置 $slow = 1$ ， $fast = 2$ ？
- 6、为什么得出相反的结论？

为了方便大家理解与表达，所以回答尽可能的采取口语化的表达方式，面试的时候你直接拿去用就行。

1、当 $slow = 1$ ， $fast = 2$ ，为什么 $fast$ 和 $slow$ 一定会相遇？

首先，我们从现实角度去思考。

一个 400 米的圆形跑道上，有跑的慢的人、有跑的快的人，只要给予一定的时间，快的肯定会追上慢的，触发相遇。

环形链表在这个的基础上增加了一条限制：跑道并非连续，而是有一个个的节点组成的。

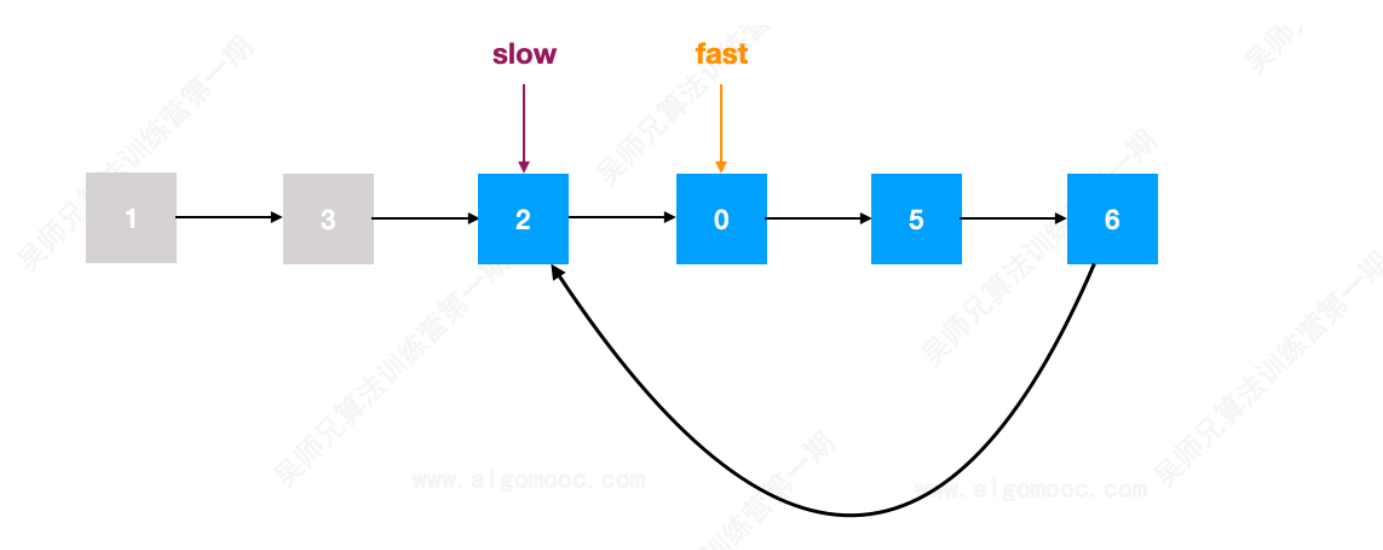
需要注意，如果两者没有在节点相遇，而是在节点与节点之间的位置相遇，那并非相遇只是相交。

- 1、假设， $fast$ 在 $slow$ 后方一个节点的位置，那么它们都跳一次之后， $fast$ 和 $slow$ 相遇了。
- 2、假设， $fast$ 在 $slow$ 后方两个节点的位置，那么它们都跳一次之后， $fast$ 和 $slow$ 的距离缩短为 1，变成了上述假设 1 的问题，可以相遇。
- 3、假设， $fast$ 在 $slow$ 后方 N 个节点的位置，那么它们都跳一次之后， $fast$ 和 $slow$ 的距离缩短为 $N - 1$ ，每条一次，都可以缩短一个单位，不断缩短，最终变成上述的假设 1。

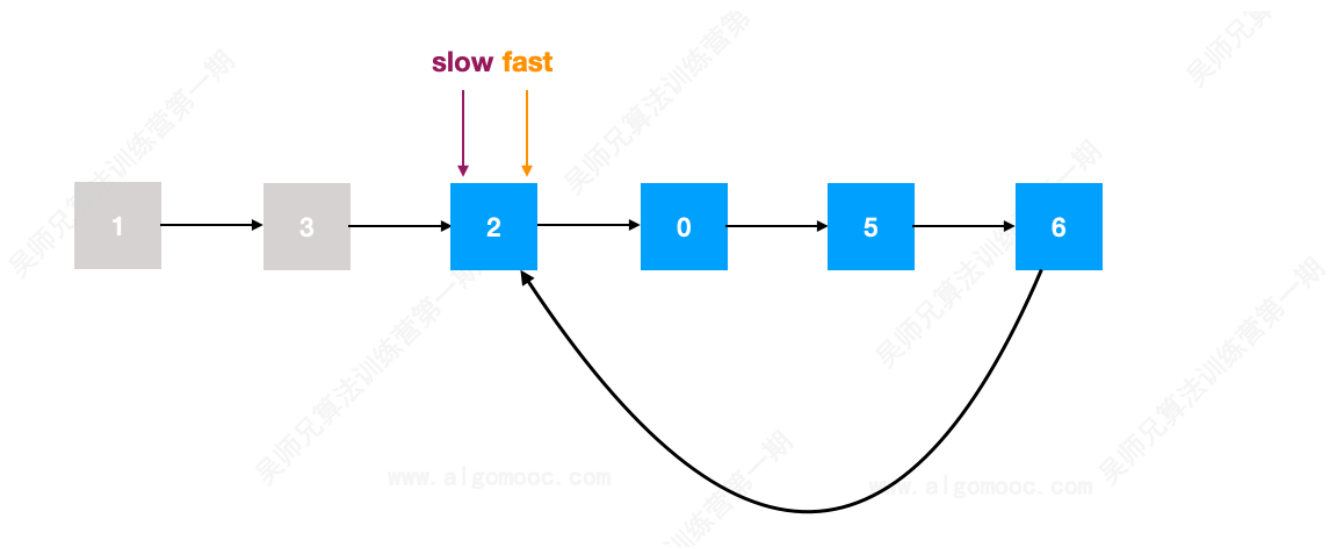
所以， $fast$ 和 $slow$ 一定会相遇。

2、当 $slow = 1$ ， $fast = 2$ ， $fast$ 和 $slow$ 相遇时， $slow$ 指针是否绕环超过一圈？

从 $slow$ 进入环之后开始统计， $fast$ 与 $slow$ 相距最远的距离是 $fast$ 在 $slow$ 前方一个节点的位置。



此时，假设一下，fast 与 slow 同时在节点 2 开始出发，相当于两者相距一个环的距离。



当 slow 跑完一圈来到起始位置节点 2 时，由于 fast 速度是 slow 的两倍，那么 fast 必然跑了两圈也来到起始位置节点 2，两者相遇。

而此时，fast 在 slow 的前方位置，意味着两者的距离是小于一个完整的环的节点数的，说明 fast 可以更快的追上 slow。

既然 fast 与 slow 相距一个环的距离相遇时，slow 才能跑一圈，那么相距更短的距离，slow 没有跑完一圈，两者相遇了。

3、slow 和 fast 的移动步数有什么规则？

只需要两者的步数不一致就行。

4、能否设置为 slow 每次移动 1 步，fast 每次移动 3、4、5...步？

先说答案：可以！

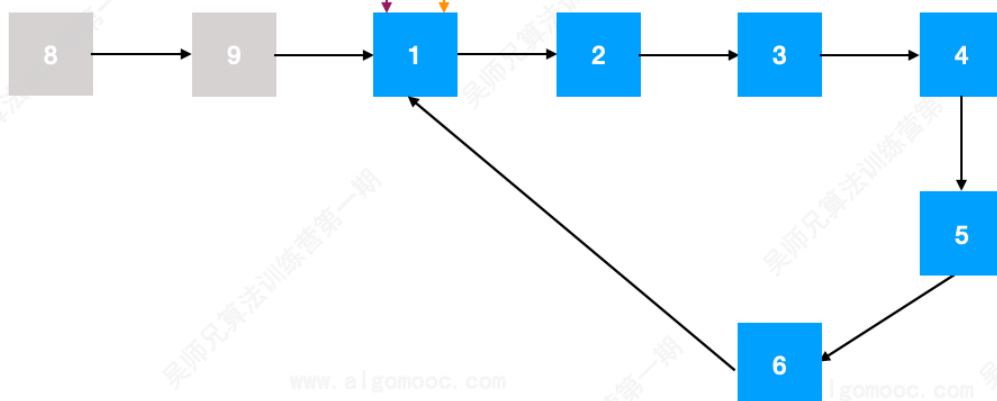
首先，只要快慢两个指针都进入环中，两者有速度差，快指针就一定能够追上慢指针。

追上可以分为两种情况：

1、恰好追上，即快慢指针在某个位置相遇了 2、快指针超过了慢指针，快指针跑到了慢指针的前面了

所以，我们需要分析的就是第二种情况：跑到前面之后能否多跑几圈之后刚好相遇。

假设环的周长为整数 L ，slow 每次走 a 步，fast 每次走 b 步， $a \neq b$ 。



公众号：吴师兄学算法

相遇时，两者都走了 t 次。

1、slow 走过的路程： $x + y + n(y+z)$, n 代表 slow 绕了 n 圈
 2、fast 走过的路程： $x + y + m(y+z)$, m 代表 fast 绕了 m 圈
 3、 $m > n$

对于 slow： $x + y + n(y+z) = a * t$

对于 fast： $x + y + m(y+z) = b * t$

两者一相减：

$$(m - n)(y + z) = (b - a) * t$$

$y + z$ 代表一圈的节点数， $y + z = L$

$$(m - n)L = (b - a) * t$$

$b - a$ 为一个正整数，假设为 c 。

$$t = (m - n)L / c。$$

其中 L 为常量， m 与 n 都可以随便设置，因此 $m - n$ 的值可以为 1、2、3、。。。，

无论 c 取任何的正整数值，始终可以找到 m 与 n ，计算得到一个正整数 t 。

也就是说 c 可以为 1、2、3、4、5。。。

说明 slow 与 fast 的速度差可以为 1、2、3、4、5。。。

由此也证明了 slow 每次走 1 步，fast 每次走 2 步、3 步、4 步。。。都是可以相遇的。

甚至，slow 每次走 2 步，fast 每次 3 步、4 步。。。都是可以相遇的。

只要 $a \neq b$ ，存在步数差。

5、为什么设置 $slow = 1$ ， $fast = 2$ ？

由于 $b - a = (**m** - n)L / t$ 。

说明速度差 $b - a$ 越小， $fast$ 和 $slow$ 跑的圈数就越小，经历的节点数也就越少，

由此 $b - a = 1$ 是一个最小的答案，即 $fast$ 和 $slow$ 相差一步最合理。

然后，再来看， $slow$ 进环的时候， $fast$ 已经在环里面了，当 $slow$ 来到环入口的时候，代表 $fast$ 已经快跑了一圈。

所以， $fast$ 和 $slow$ 相遇， m 至少为 1。

m 越小， $fast$ 总共走的路程就越小，时间复杂度就越低。

所以， $m = 1$ 时最低的时间复杂度。

所以， $fast$ 跑的越慢，越能在环中少跑几圈等到还在外面直线跑的 $slow$ 。

由于 $a \neq b$ ， $b - a = 1$ ， $a = 1$ ，所以 $b = 2$ 。

所以，设置 $slow$ 为 1， $fast$ 为 2 最合理！

6、为什么得出相反的结论？

关于 $fast$ 与 $slow$ 能否相遇的问题，我在网上看到这样的回答。

问题二：那么慢指针走一步，快指针走三步？走四步？或是走 n 步行不行？为什么？请证明。

不行，这样可能会追不上，证明如下：

假设慢指针进环的时候，快指针和慢指针之间的距离为 N
在他们追击的过程中，快指针往前走三步，慢指针走一步
每走一次，他们之间的距离缩小2
分两种情况：
1、若 N 为偶数，则他们之间的距离最终会减为0，即相遇。
2、若 N 为奇数，则他们之间的距离会减到1，然后减到-1，减到-1
也就意味着他们之间的距离又变为了 $C-1$ （ C 是环的周长），此时又分两种情况：
a、若 C 为奇数，则 $C-1$ 为偶数，因为他们之间的距离一次缩小2，所以他们会相遇。
b、若 C 为偶数，则 $C-1$ 还是为奇数，也就是说他们之间的距离还是为奇数，他们永远不会相遇。

总结一下：

当慢指针走一步，快指针走三步时。若慢指针进环时与快指针之间的距离为奇数，并且环的周长恰好为偶数，那么他们会一直在环里面打转转，永远不会相遇。

（当慢指针走一步，快指针走四步或是走 n 步时，证明过程类似）

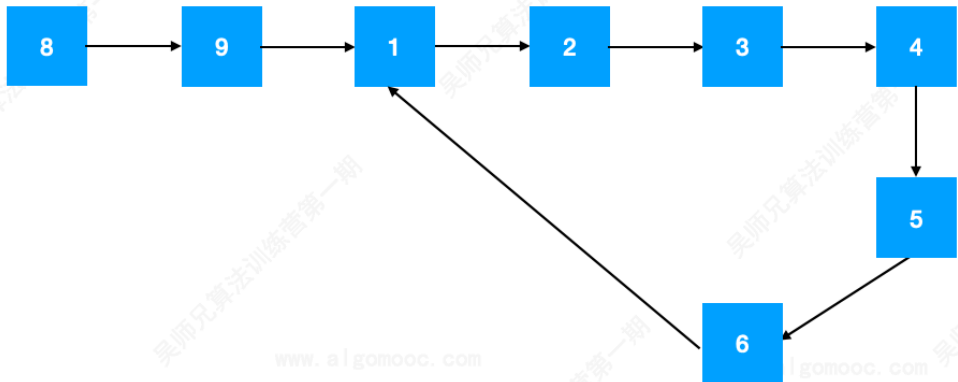


仔细看看，是不是很有道理，但得到的结论却和本文问题 4 得出的结论不符。

谁是对的呢？

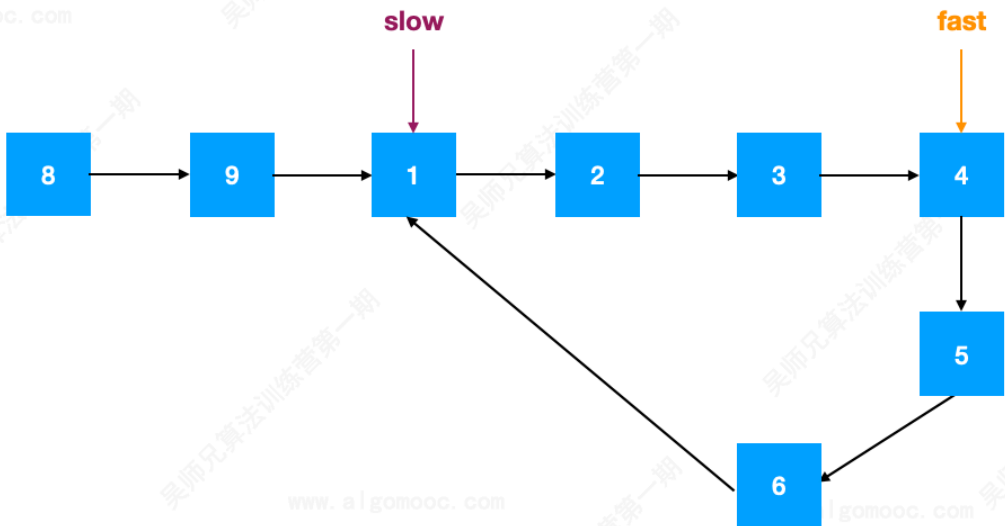
我们顺着它的假设来模拟一下。

假设环的周长为偶数，也就是六个节点。



公众号：吴师兄学算法

并且进入环后 $slow = 1$ ， $fast = 3$ ， $slow$ 与 $fast$ 的距离为奇数。



公众号：吴师兄学算法

模拟一下， $fast$ 始终会出现在节点 4 和节点 1 这两个位置，当 $slow$ 重新来到节点 1 的时候， $fast$ 又来到了节点 4，如此反复，两者无法相遇。

这个结论是对的？？！！

这种假设根本就不会成立！

即进入环后 $slow = 1$ ， $fast = 3$ ， $slow$ 与 $fast$ 的距离为奇数这种假设无法成立。

在进入环之前， $slow$ 和 $fast$ 有一段公共的直线路需要走。

假设这条直线的长度为 m 。

那么 $slow$ 需要跳 m 次才能来到环入口位置节点 1， $fast$ 需要跳 $m/3$ 次才能第一次来到环入口位置节点 1，环的节点数是 6 个， $fast$ 跑 2 次可以跑完一圈，假设跑了 n 圈，那么就跑了 $2n$ 次，那么再跑一次就来到了节点 4。

于是有了等式： $m = m / 3 + 2n + 1$ 。

计算一下得 $n = (2m / 3 - 1) / 2$ 。

由于 $2m$ 为偶数，根据数学知识一个能被 3 整除的偶数除以 3 仍为偶数，于是 $2m / 3$ 为偶数， $2m / 3 - 1$ 为奇数， $(2m / 3 - 1) / 2$ 无法是整数。

所以，找不到这样一个整数 m 和整数 n ，使得上面的假设成立。

假设不成立，结论也不成立！

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 环形链表 II ( LeetCode 142 ) : https://leetcode-cn.com/problems/linked-list-cycle-ii
public class Solution {
    public ListNode detectCycle(ListNode head) {

        // 1、通过快慢指针的方式，在环中寻找它们的第一次相遇的节点位置

        // 2、定义一个慢指针，每次只会向前移动 1 步
        ListNode slow = head;
        // 3、定义一个快指针，每次只会向前移动 2 步
        ListNode fast = head;

        // 4、如果链表有环，那么无论怎么移动，fast 指向的节点都是有值的
        while (fast != null && fast.next != null) {
            // 慢指针每次只会向前移动 1 步
            slow = slow.next;
            // 快指针每次只会向前移动 2 步
            fast = fast.next.next;

            // 快慢指针相遇，说明有环
            // x 代表从头节点到环形入口节点的节点数（不包含头节点）
            // y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
            // z 代表从第一次相遇节点到环形入口的节点数（不包含第一次相遇节点）
            // y + z 代表环的节点总数
            // 此时，快指针走了 x + y + n (y + z)
            // 其中，x + y 表示快指针第一次到达相遇节点，n 代表快指针在环里面绕了多少圈
            // 此时，慢指针走了 x + y 步

            // 由于快指针每次走 2 步，所以快慢指针第一次相遇的时候出现一个等式
            // x + y = [x + y + n (y + z)] / 2
            // 即 2 * (x + y) = x + y + n (y + z)
            // 即 x + y = n (y + z)
            // 即 x = n (y + z) - y
        }
    }
}
```



```

// 我们的目的就是去求 x

// 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a

// b 在环中绕圈圈，走了  $n(y + z)$  步会回到原处，即回到相遇节点处
// 由于 y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
// 所以  $n(y + z) - y$  时，b 到达了环形入口节点位置

// 由于 x 代表从头节点到环形入口节点的节点数（不包含头节点）
// 所以 a 走了 x 步时，a 到达了环形入口节点位置

// 当  $x = n(y + z) - y$  时，找到了环形入口节点位置

// 5、开始寻找环入口
if (slow == fast) {

    // 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a
    // 一个指向相遇节点，定义为 b
    ListNode b = fast;

    // 一个指向链表头节点，定义为 a
    ListNode a = head;

    // 让 a、b 两个指针向前移动，每次移动一步，直到相遇位置
    // 由于有环，必然相遇
    // 当 b 走了  $n(y + z) - y$  时，b 到达了环形入口节点位置
    // 当 a 走了 x 步时，a 到达了环形入口节点位置
    // a 与 b 相遇
    while (a != b) {
        // a 指针每次只会向前移动 1 步
        a = a.next;
        // b 指针每次只会向前移动 1 步
        b = b.next;
    }

    // 6、返回 a 和 b 相遇的节点位置就是环形入口节点位置
    return a;
}

// 没有环，返回 null
return null;
}
}

```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 环形链表 II ( LeetCode 142 ) : https://leetcode-cn.com/problems/linked-list-cycle-ii
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        // 1、通过快慢指针的方式，在环中寻找它们的第一次相遇的节点位置

        // 2、定义一个慢指针，每次只会向前移动 1 步
        ListNode *slow = head;
        // 3、定义一个快指针，每次只会向前移动 2 步
        ListNode *fast = head;

        // 4、如果链表有环，那么无论怎么移动，fast 指向的节点都是有值的
        while (fast != NULL && fast->next != NULL) {
            // 慢指针每次只会向前移动 1 步
            slow = slow->next;
            // 快指针每次只会向前移动 2 步
            fast = fast->next->next;

            // 快慢指针相遇，说明有环
            // x 代表从头节点到环形入口节点的节点数（不包含头节点）
            // y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
            // z 代表从第一次相遇节点到环形入口的节点数（不包含第一次相遇节点）
            // y + z 代表环的节点总数
            // 此时，快指针走了  $x + y + n(y + z)$ 
            // 其中， $x + y$  表示快指针第一次到达相遇节点， $n$  代表快指针在环里面绕了多少圈
            // 此时，慢指针走了  $x + y$  步

            // 由于快指针每次走 2 步，所以快慢指针第一次相遇的时候出现一个等式
            //  $x + y = [x + y + n(y + z)] / 2$ 
            // 即  $2 * (x + y) = x + y + n(y + z)$ 
            // 即  $x + y = n(y + z)$ 
            // 即  $x = n(y + z) - y$ 
            // 我们的目的就是去求  $x$ 

            // 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a

            // b 在环中绕圈圈，走了  $n(y + z)$  步会回到原处，即回到相遇节点处
            // 由于 y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
            // 所以  $n(y + z) - y$  时，b 到达了环形入口节点位置

            // 由于 x 代表从头节点到环形入口节点的节点数（不包含头节点）
            // 所以 a 走了 x 步时，a 到达了环形入口节点位置
        }
    }
};
```

```

// 当  $x = n(y + z) - y$  时，找到了环形入口节点位置

// 5、开始寻找环入口
if (slow == fast) {

    // 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a
    // 一个指向相遇节点，定义为 b
    ListNode *b = fast;

    // 一个指向链表头节点，定义为 a
    ListNode *a = head;

    // 让 a 、 b 两个指针向前移动，每次移动一步，直到相遇位置
    // 由于有环，必然相遇
    // 当 b 走了  $n(y + z) - y$  时，b 到达了环形入口节点位置
    // 当 a 走了 x 步时，a 到达了环形入口节点位置
    // a 与 b 相遇
    while (a != b) {
        // a 指针每次只会向前移动 1 步
        a = a->next;
        // b 指针每次只会向前移动 1 步
        b = b->next;
    }

    // 6、返回 a 和 b 相遇的节点位置就是环形入口节点位置
    return a;
}

// 没有环，返回 NULL
return NULL;
}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 环形链表 II ( LeetCode 142 ) : https://leetcode-cn.com/problems/linked-list-cycle-ii
class solution:
    def detectCycle(self, head: ListNode) -> ListNode:
        # 1、通过快慢指针的方式，在环中寻找它们的第一次相遇的节点位置

        # 2、定义一个慢指针，每次只会向前移动 1 步
        slow = head
        # 3、定义一个快指针，每次只会向前移动 2 步

```

```
fast = head
```

4、如果链表有环，那么无论怎么移动，fast 指向的节点都是有值的

```
while fast != None and fast.next != None :
```

```
    # 慢指针每次只会向前移动 1 步
```

```
    slow = slow.next
```

```
    # 快指针每次只会向前移动 2 步
```

```
    fast = fast.next.next
```

```
    # 快慢指针相遇，说明有环
```

```
    # x 代表从头节点到环形入口节点的节点数（不包含头节点）
```

```
    # y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
```

```
    # z 代表从第一次相遇节点到环形入口的节点数（不包含第一次相遇节点）
```

```
    # y + z 代表环的节点总数
```

```
    # 此时，快指针走了  $x + y + n(y + z)$ 
```

```
    # 其中， $x + y$  表示快指针第一次到达相遇节点，n 代表快指针在环里面绕了多少圈
```

```
    # 此时，慢指针走了  $x + y$  步
```

```
    # 由于快指针每次走 2 步，所以快慢指针第一次相遇的时候出现一个等式
```

```
    #  $x + y = [x + y + n(y + z)] / 2$ 
```

```
    # 即  $2 * (x + y) = x + y + n(y + z)$ 
```

```
    # 即  $x + y = n(y + z)$ 
```

```
    # 即  $x = n(y + z) - y$ 
```

```
    # 我们的目的就是去求 x
```

```
    # 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a
```

```
    # b 在环中绕圈圈，走了  $n(y + z)$  步会回到原处，即回到相遇节点处
```

```
    # 由于 y 代表从环形入口到第一次相遇节点的节点数（不包含环形入口节点）
```

```
    # 所以  $n(y + z) - y$  时，b 到达了环形入口节点位置
```

```
    # 由于 x 代表从头节点到环形入口节点的节点数（不包含头节点）
```

```
    # 所以 a 走了 x 步时，a 到达了环形入口节点位置
```

```
    # 当  $x = n(y + z) - y$  时，找到了环形入口节点位置
```

5、开始寻找环入口

```
if slow == fast :
```

```
    # 定义两个指针，一个指向相遇节点，定义为 b，一个指向链表头节点，定义为 a
```

```
    # 一个指向相遇节点，定义为 b
```

```
    b = fast
```

```
    # 一个指向链表头节点，定义为 a
```

```
    a = head
```

```
    # 让 a、b 两个指针向前移动，每次移动一步，直到相遇位置
```

```
    # 由于有环，必然相遇
```

```
    # 当 b 走了  $n(y + z) - y$  时，b 到达了环形入口节点位置
```

```

# 当 a 走了 x 步时, a 到达了环形入口节点位置
# a 与 b 相遇
while a != b :
    # a 指针每次只会向前移动 1 步
    a = a.next
    # b 指针每次只会向前移动 1 步
    b = b.next

# 6、返回 a 和 b 相遇的节点位置就是环形入口节点位置
return a

# 没有环, 返回 None
return None

```

四、复杂度分析

时间复杂度：O(N)，其中 N 为链表中节点的数目。在最初判断快慢指针是否相遇时，slow 指针走过的距离不会超过链表的总长度；随后寻找入环点时，走过的距离也不会超过链表的总长度。因此，总的执行时间为 O(N)+O(N)=O(N)。

空间复杂度：O(1)。

LeetCode 167、两数之和II- 输入有序数组

一、题目描述

给你一个下标从 1 开始的整数数组 `numbers`，该数组已按 **非递减顺序排列**，请你从数组中找出满足相加之和等于目标数 `target` 的两个数。如果设这两个数分别是 `numbers[index1]` 和 `numbers[index2]`，则 $1 \leq index1 < index2 \leq numbers.length$ 。

以长度为 2 的整数数组 `[index1, index2]` 的形式返回这两个整数的下标 `index1` 和 `index2`。

你可以假设每个输入 **只对应唯一的答案**，而且你 **不可以** 重复使用相同的元素。

你所设计的解决方案必须只使用常量级的额外空间。

示例 1：

输入：numbers = [2,7,11,15], target = 9

输出：[1,2]

解释：2 与 7 之和等于目标数 9。因此 index1 = 1, index2 = 2。返回 [1, 2]。

示例 2：

输入：numbers = [2,3,4], target = 6

输出：[1,3]

解释：2 与 4 之和等于目标数 6。因此 index1 = 1, index2 = 3。返回 [1, 3]。

示例 3：

输入: numbers = [-1,0], target = -1

输出: [1,2]

解释: -1 与 0 之和等于目标数 -1 。因此 index1 = 1, index2 = 2 。返回 [1, 2] 。

提示:

- `2 <= numbers.length <= 3 * 10^4`
- `-1000 <= numbers[i] <= 1000`
- numbers 按 非递减顺序 排列
- `-1000 <= target <= 1000`
- 仅存在一个有效答案

二、题目解析

三、参考代码

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:

        # 初始结果数组
        res = [0, 0]

        # 定义左侧指针left, 指向数组中第一个元素
        # 默认从数组的索引为 0 的位置开始
        left = 0

        # 定义右侧指针 right, 指向数组中最后一个元素
        right = len(numbers) - 1

        # 两个索引向内移动
        # left 向右移动
        # right 向左移动
        while left < right:
            # 1、如果左侧指针与右侧指针所指向的元素和等于目标值, 则返回结果
            if numbers[left] + numbers[right] == target:
                # 题目说明下标从 1 开始, 就操作一下
                res[0] = left + 1

                res[1] = right + 1

                # 返回结果
                return res

            # 2、如果左侧指针与右侧指针所指向的元素和小于目标值
            elif numbers[left] + numbers[right] < target:

                # 因为数组是升序排列的, 为了让两数之和变大一些
                # 因此应将左侧指针向右移动一位
                left += 1
```

```
# 3、如果左侧指针与右侧指针所指向的元素和大于目标值
elif numbers[left] + numbers[right] > target:

    # 因为数组是升序排列的，为了让两数之和变小一些
    # 因此应将右侧指针向左移动一位
    right -= 1

# 返回结果
return res
```

LeetCode 61、旋转链表

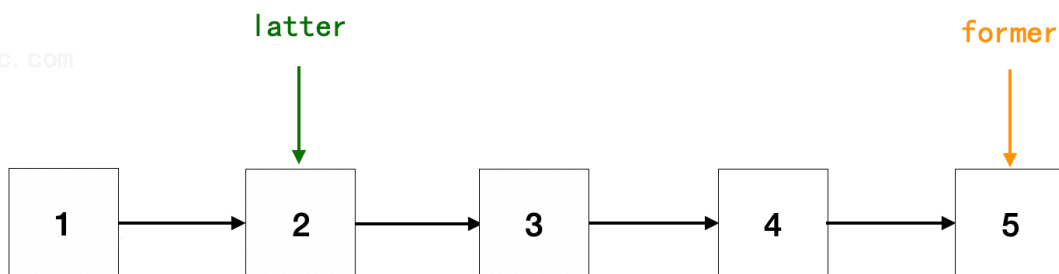
一、题目描述

给你一个链表的头节点 `head`，旋转链表，将链表每个节点向右移动 `k` 个位置。

二、题目解析

- 1、首先遍历整个链表，求出链表的长度 `len`
- 2、根据题目的提示，`k` 可能很大，远超链表的长度，这样就会导致一种情况，比如 $k = 1000$ ， $len = 999$ ，每个节点向右移动 1 个节点和向右移动 $k = 1000$ 个节点结果一样，所以进行一个取模的操作可以节省大量的移动操作。
- 3、接下来设置两个指针 `former`、`latter` 均指向链表的头节点，这两个指针的目的是去寻找出旋转之前的尾节点位置、旋转成功之后的尾节点位置。
- 4、先让 `former` 指针先向前移动 k 次，此时，`former` 就和 `latter` 相距 k 个节点了。
- 5、接下来，让 `former`、`latter` 同时向后移动，直到 `former` 来到了最后一个节点位置。

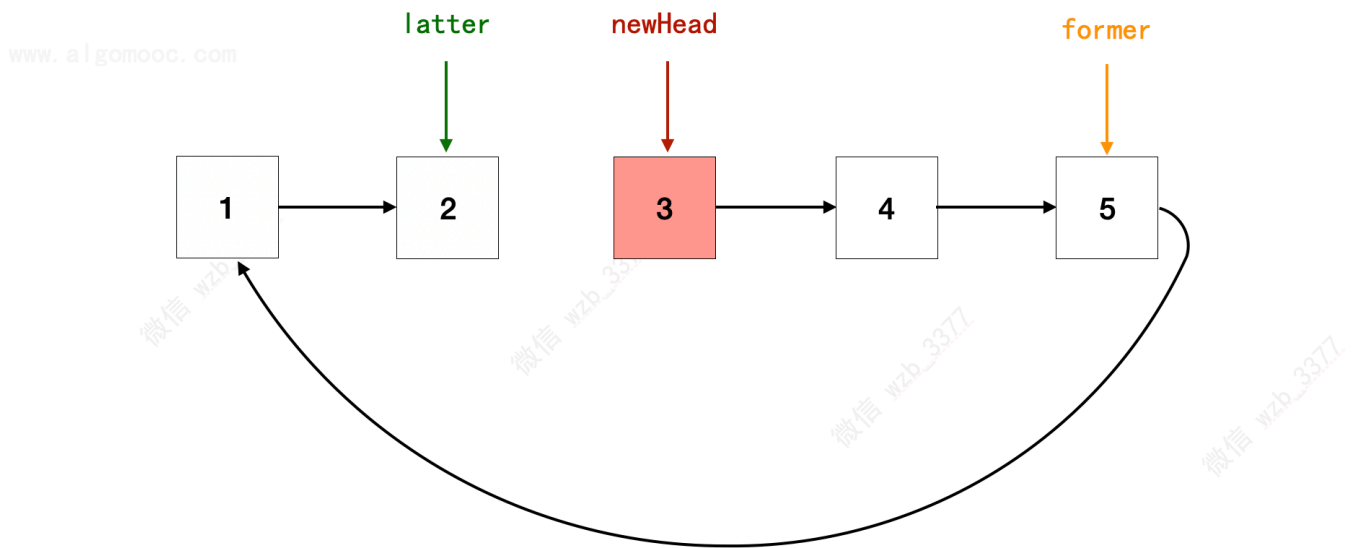
$$k = 3$$



- 6、这个时候，从 `head` 到 `latter` 有 $len - k$ 个节点，`latter + 1` 到 `former` 有 k 个节点。

7、也就意味着，latter + 1 这个节点是移动 k 个节点成功之后的头节点了。

$k = 3$



8、只需要返回 latter + 1 后面这个节点 newHead，并且断开连接就行了。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 旋转链表 (LeetCode 61) :https://leetcode.cn/problems/rotate-list/
class solution {
    public ListNode rotateRight(ListNode head, int k) {

        // 边界条件判断
        if( head == null) {
            return head;
        }

        // 1、第一步先要计算出链表的长度来
        int len = 0;

        // 2、这里我们设置一个指针指向链表的头节点的位置
        ListNode tempHead = head;

        // 3、通过不断的移动 tempHead，来获取链表的长度
        while (tempHead != null) {
```



```

        // tempHead 不断向后移动，直到为 null
        tempHead = tempHead.next;

        // 每次遍历一个新的节点，意味着链表长度新增 1
        len++;
    }

    // 由于题目中的 k 会超过链表的长度，因此进行一个取余的操作
    // 比如 k = 1000, len = 999
    // 实际上就是将链表每个节点向右移动  $1000 \% 999 = 1$  个位置就行了
    // 因为链表中的每个节点移动 len 次会回到原位
    k = k % len;

    // 4、接下来设置两个指针指向链表的头节点
    // 这两个指针的目的是去寻找出旋转之前的尾节点位置、旋转成功之后的尾节点位置

    // former 指针
    ListNode former = head;

    // latter 指针
    ListNode latter = head;

    // former 指针先向前移动 k 次
    for(int i = 0 ; i < k ; i++){

        // former 不断向后移动
        former = former.next;
    }

    // 这样 former 和 latter 就相差了 k 个节点
    // 5、接下来，两个指针同时向后移动，直到 former 来到了最后一个节点位置
    while(former.next != null){

        // former 不断向后移动
        former = former.next;

        // latter 不断向后移动
        latter = latter.next;
    }

    // 6、此时，former 指向了最后一个节点，需要将这个节点和原链表的头节点连接起来
    former.next = head;

    // 7、latter 指向的节点的【下一个节点】是倒数第 k 个节点，那就是旋转成功之后的头节点
    ListNode newHead = latter.next;

```

```

        // 8、断开链接，避免成环
        latter.next = null;

        // 9、返回 newHead
        return newHead;
    }
}

```

2、**C++** 代码

```

class solution {
public:
    ListNode* rotateRight(ListNode* head, int k) {

        // 边界条件判断
        if( head == NULL) {
            return head;
        }

        // 1、第一步先要计算出链表的长度来
        int len = 0;

        // 2、这里我们设置一个指针指向链表的头节点的位置
        ListNode *tempHead = head;

        // 3、通过不断的移动 tempHead ，来获取链表的长度
        while (tempHead != NULL) {

            // tempHead 不断向后移动，直到为 NULL
            tempHead = tempHead->next;

            // 每次遍历一个新的节点，意味着链表长度新增 1
            len++;
        }

        // 由于题目中的 k 会超过链表的长度，因此进行一个取余的操作
        // 比如 k = 1000, len = 999
        // 实际上就是将链表每个节点向右移动  $1000 \% 999 = 1$  个位置就行了
        // 因为链表中的每个节点移动 len 次会回到原位
        k = k % len;

        // 4、接下来设置两个指针指向链表的头节点
        // 这两个指针的目的是去寻找出旋转之前的尾节点位置、旋转成功之后的尾节点位置

        // former 指针
        ListNode *former = head;
    }
}

```

```

// latter 指针
ListNode *latter = head;

// former 指针先向前移动 k 次
for(int i = 0 ; i < k ; i++){

    // former 不断向后移动
    former = former->next;

}

// 这样 former 和 latter 就相差了 k 个节点
// 5、接下来，两个指针同时向后移动，直到 former 来到了最后一个节点位置
while(former->next != NULL){

    // former 不断向后移动
    former = former->next;

    // latter 不断向后移动
    latter = latter->next;

}

// 6、此时，former 指向了最后一个节点，需要将这个节点和原链表的头节点连接起来
former->next = head;

// 7、latter 指向的节点的【下一个节点】是倒数第 k 个节点，那就是旋转成功之后的头节点
ListNode *newHead = latter->next;

// 8、断开链接，避免成环
latter->next = NULL;

// 9、返回 newHead
return newHead;

}
};

```

3、Python 代码

```

class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        # 边界条件判断
        if not head or k == 0 :
            return head

        # 1、第一步先要计算出链表的长度来
        _len = 0

```

```
# 2、这里我们设置一个指针指向链表的头节点的位置
tempHead = head

# 3、通过不断的移动 tempHead，来获取链表的长度
while tempHead :

    # tempHead 不断向后移动，直到为 NULL
    tempHead = tempHead.next

    # 每次遍历一个新的节点，意味着链表长度新增 1
    _len += 1

# 由于题目中的 k 会超过链表的长度，因此进行一个取余的操作
# 比如 k = 1000, len = 999
# 实际上就是将链表每个节点向右移动  $1000 \% 999 = 1$  个位置就行了
# 因为链表中的每个节点移动 len 次会回到原位
k = k % _len

# 4、接下来设置两个指针指向链表的头节点
# 这两个指针的目的是去寻找出旋转之前的尾节点位置、旋转成功之后的尾节点位置

# former 指针
former = head

# latter 指针
latter = head

# former 指针先向前移动 k 次
for i in range(0, k) :

    # former 不断向后移动
    former = former.next

# 这样 former 和 latter 就相差了 k 个节点
# 5、接下来，两个指针同时向后移动，直到 former 来到了最后一个节点位置
while former.next :

    # former 不断向后移动
    former = former.next

    # latter 不断向后移动
    latter = latter.next

# 6、此时，former 指向了最后一个节点，需要将这个节点和原链表的头节点连接起来
```

```
former.next = head

# 7、latter 指向的节点的【下一个节点】是倒数第 k 个节点，那就是旋转成功之后的头节点
newHead = latter.next

# 8、断开链接，避免成环
latter.next = None

# 9、返回 newHead
return newHead
```

四、复杂度分析

时间复杂度：链表一共被遍历两次，因此总的时间复杂度为 $O(n)$ ， n 是链表的长度。

空间复杂度： $O(1)$ ，我们只需要常数的空间存储若干变量。

第二十一天

LeetCode 134、加油站

一、题目描述

在一条环路上有 N 个加油站，其中第 i 个加油站有汽油 `gas[i]` 升。

你有一辆油箱容量无限的汽车，从第 i 个加油站开往第 $i+1$ 个加油站需要消耗汽油 `cost[i]` 升。你从其中的一个加油站出发，开始时油箱为空。

如果你可以绕环路行驶一周，则返回出发时加油站的编号，否则返回 -1。

说明：

- 如果题目有解，该答案即为唯一答案。
- 输入数组均为非空数组，且长度相同。
- 输入数组中的元素均为非负数。

二、题目解析

证明命题：如果总的 `gas` 量大于 `cost`，则一定有解。

1、当 $n = 2$ 时，一定可以找到一个点 A ，它的 `gas` $\geq$ `cost`；假设另一个点为 B ，则 A 点可以到达 B 点，到达 B 点之后总 `gas` 量根据题设，大于 B 点的 `cost`，所以也可以回到 A 点；

2、当 $n = 3$ 时，一定可以找到两个点，它们的 `gas` 之和大于等于 `cost` 之和，把他们当作一个整体点 A ，剩余的点为 B 。那么根据 $n = 2$ 的情况， A 可以到达 B 再返回 A 。对于 A 内部来说，它们也符合 $n = 2$ 的情况，所以同样可以互通。

3、当 $n > 3$ 时，一定可以找到 $n - 1$ 个点，它们的 `gas` 之和大于等于 `cost` 之和，把他们当作一个整体点 A ，剩余的点为 B 。那么根据 $n - 1$ 的情况， A 可以到达 B 再返回 A 。对于 A 内部来说，它们也符合 $n - 1$ 的情况，所以同样可以互通。

所以，证明若 gas 总量大于 cost，则一定有解。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 加油站 (134) : https://leetcode-cn.com/problems/gas-station/
class solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {

        // 获取加油站的数量
        int n = gas.length;

        // 一开始，默认汽车储备的汽油数量为 0
        int remainGas = 0;

        // 需要统计一下，所有加油站的汽油总量能否支持汽车跑完所有的路程
        for( int i = 0 ; i < n ; i++ ){

            remainGas += gas[i] - cost[i];
        }

        // 如果发现加油站的汽油总量小于所有的路程需要消耗的汽油总量
        // 即汽车最终油箱汽油为负，那无论选择在哪出发，都不可能绕环路行驶一周
        if( remainGas < 0 ){

            return -1;
        }

        // 如果发现加油站的汽油总量大于或者等于所有的路程需要消耗的汽油总量
        // 那么可以绕环路行驶一周，接下来去寻找出发位置

        // 一开始，默认汽车此时储备的汽油数量为 0
        int currentGas = 0;

        // 一开始，默认汽车出发位置在索引为 0 的加油站
        int i = 0;

        // index 表示最终选择的出发点，默认为 0
        int index = 0;

        // 依次遍历每个加油站，在遍历的过程中，会执行一些【跳跃操作】
        while ( i < n ){

            // 汽车在 i 号加油站加了 gas[i]
            // 开到 i + 1 号加油站消耗了 cost[i] 的汽油
```

```

        currentGas += gas[i] - cost[i];

        // 如果发现油箱里面的汽油是非负数
        // 即汽车可以开到 j 号加油站, j >= i + 1, 那么就让汽车继续尝试往前开
        // 寻找出从 i 号加油站出发到最远的加油站的位置 j, 在这期间汽车是会从中间的加油站加油的
        if( currentGas >= 0){

            i++;

            // 如果发现油箱里面的汽油是负数
            // 意味着汽车无法从 i 号加油站开到 j 号加油站, 同时也意味着无法从 i + 1 号加油站开到 j
            // 那么就尝试让汽车从 j 号加油站开始重新出发
        }else{

            // 重新初始化汽车油箱油量
            currentGas = 0;

            // 最终选择的出发点为 j 号加油站
            index = i + 1;

            // 开始出发
            i++;
        }
    }

    // 最终找到了合适的出发点
    return index;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        // 获取加油站的数量
        int n = gas.size();

        // 一开始, 默认汽车储备的汽油数量为 0
        int remainGas = 0;

        // 需要统计一下, 所有加油站的汽油总量能否支持汽车跑完所有的路程
        for( int i = 0 ; i < n ; i++ ){

            remainGas += gas[i] - cost[i];
        }

        // 如果发现加油站的汽油总量小于所有的路程需要消耗的汽油总量
    }
}

```

```

// 即汽车最终油箱汽油为负，那无论选择在哪出发，都不可能绕环路行驶一周
if( remainGas < 0 ){

    return -1;
}

// 如果发现加油站的汽油总量大于或者等于所有的路程需要消耗的汽油总量
// 那么可以绕环路行驶一周，接下来去寻找出发位置

// 一开始，默认汽车此时储备的汽油数量为 0
int currentGas = 0;

// 一开始，默认汽车出发位置在索引为 0 的加油站
int i = 0;

// index 表示最终选择的出发点，默认为 0
int index = 0;

// 依次遍历每个加油站，在遍历的过程中，会执行一些【跳跃操作】
while ( i < n ){

    // 汽车在 i 号加油站加了 gas[i]
    // 开到 i + 1 号加油站消耗了 cost[i] 的汽油
    currentGas += gas[i] - cost[i];

    // 如果发现油箱里面的汽油是非负数
    // 即汽车可以开到 j 号加油站，j >= i + 1，那么就让汽车继续尝试往前开
    // 寻找出从 i 号加油站出发到最远的加油站的位置 j，在这期间汽车是会从中间的加油站加油的
    if( currentGas >= 0){

        i++;

        // 如果发现油箱里面的汽油是负数
        // 意味着汽车无法从 i 号加油站开到 j 号加油站，同时也意味着无法从 i + 1 号加油站开到 j
        // 那么就尝试让汽车从 j 号加油站开始重新出发
    }else{

        // 重新初始化汽车油箱油量
        currentGas = 0;

        // 最终选择的出发点为 j 号加油站
        index = i + 1;

        // 开始出发
        i++;
    }
}

```

号加油站


```

        // 最终找到了合适的出发点
        return index;
    }
};

```

3、Python 代码

```

class solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        # 获取加油站的数量
        n = len(gas)

        # 一开始，默认汽车储备的汽油数量为 0
        remainGas = 0

        # 需要统计一下，所有加油站的汽油总量能否支持汽车跑完所有的路程
        for i in range(n) :
            remainGas += gas[i] - cost[i]

        # 如果发现加油站的汽油总量小于所有的路程需要消耗的汽油总量
        # 即汽车最终油箱汽油为负，那无论选择在哪出发，都不可能绕环路行驶一周
        if remainGas < 0 :

            return -1

        # 如果发现加油站的汽油总量大于或者等于所有的路程需要消耗的汽油总量
        # 那么可以绕环路行驶一周，接下来去寻找出发位置

        # 一开始，默认汽车此时储备的汽油数量为 0
        currentGas = 0

        # 一开始，默认汽车出发位置在索引为 0 的加油站
        i = 0

        # index 表示最终选择的出发点，默认为 0
        index = 0

        # 依次遍历每个加油站，在遍历的过程中，会执行一些【跳跃操作】
        while i < n :

            # 汽车在 i 号加油站加了 gas[i]
            # 开到 i + 1 号加油站消耗了 cost[i] 的汽油
            currentGas += gas[i] - cost[i]

            # 如果发现油箱里面的汽油是非负数
            # 即汽车可以开到 j 号加油站，j >= i + 1，那么就让汽车继续尝试往前开
            # 寻找出从 i 号加油站出发到最远的加油站的位置 j，在这期间汽车是会从中间的加油站加油的
            if currentGas >= 0:

```

```

        i+=1

    # 如果发现油箱里面的汽油是负数
    # 意味着汽车无法从 i 号加油站开到 j 号加油站，同时也意味着无法从 i + 1 号加油站开到 j
号加油站

    # 那么就直接尝试让汽车从 j 号加油站开始重新出发
else:

    # 重新初始化汽车油箱油量
    currentGas = 0

    # 最终选择的出发点为 j 号加油站
    index = i + 1

    # 开始出发
    i+=1

# 最终找到了合适的出发点
return index

```

四、复杂度分析

时间复杂度：O(N)，其中 N 为数组的长度。

空间复杂度：O(1)。

LeetCode 135、分发糖果

一、题目描述

n 个孩子站成一排。给你一个整数数组 ratings 表示每个孩子的评分。

你需要按照以下要求，给这些孩子分发糖果：

- 每个孩子至少分配到 1 个糖果。
- 相邻两个孩子评分更高的孩子会获得更多的糖果。

请你给每个孩子分发糖果，计算并返回需要准备的 **最少糖果数目**。

示例 1：

输入：ratings = [1,0,2]

输出：5

解释：你可以分别给第一个、第二个、第三个孩子分发 2、1、2 颗糖果。

示例 2：

输入: ratings = [1,2,2]

输出: 4

解释: 你可以分别给第一个、第二个、第三个孩子分发 1、2、1 颗糖果。
第三个孩子只得到 1 颗糖果, 这满足题面中的两个条件。

提示:

- `n == ratings.length`
- `1 <= n <= 2 * 104`
- `0 <= ratings[i] <= 2 * 104`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 分发糖果 (LeetCode 135) : https://leetcode.cn/problems/candy/
class Solution {
    public int candy(int[] ratings) {

        // 存储每个孩子的糖果
        int[] candys = new int[ratings.length];

        // 先给所有孩子 1 颗糖
        Arrays.fill(candys, 1);

        // 从左至右遍历数组 ratings
        for(int i = 1; i < ratings.length; i++){

            // 如果发现当前孩子的评分比【左边】的更高
            if(ratings[i] > ratings[i - 1]){

                // 那么当前孩子的糖果数量需要比【左边】孩子的糖果数量多 1 个
                candys[i] = candys[i - 1] + 1;
            }
        }

        // 记录糖果数量, 初始为最右边的值
        int result = candys[ratings.length - 1];

        // 从右至左遍历数组 ratings
```

```

for(int i = ratings.length - 2; i >= 0; i--) {

    // 如果发现当前孩子的评分比【右边】的更高
    if(ratings[i] > ratings[i + 1]){

        // 那么当前孩子的糖果数量需要比【右边】孩子的糖果数量多 1 个
        // 当前孩子在第一轮循环中已经设置了值
        // 所以取这两个值的最大值
        // 这样同时满足左规则和右规则
        candys[i] = Math.max( candys[i + 1] + 1 , candys[i]) ;

    }

    // candys[i] 已经是符合左规则和右规则的值了
    // 累加到 result 上面
    result += candys[i];
}

// 返回结果
return result;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int candy(vector<int>& ratings) {

        // 先给所有孩子 1 颗糖
        vector<int> candys(ratings.size(),1);

        // 从左至右遍历数组 ratings
        for(int i = 1; i < ratings.size(); i++){

            // 如果发现当前孩子的评分比【左边】的更高
            if(ratings[i] > ratings[i - 1]){

                // 那么当前孩子的糖果数量需要比【左边】孩子的糖果数量多 1 个
                candys[i] = candys[i - 1] + 1;

            }

        }

        // 记录糖果数量，初始为最右边的值
        int result = candys[ratings.size() - 1];

        // 从右至左遍历数组 ratings
    }
}

```

```

for(int i = ratings.size() - 2; i >= 0; i--) {

    // 如果发现当前孩子的评分比【右边】的更高
    if(ratings[i] > ratings[i + 1]){

        // 那么当前孩子的糖果数量需要比【右边】孩子的糖果数量多 1 个
        // 当前孩子在第一轮循环中已经设置了值
        // 所以取这两个值的最大值
        // 这样同时满足左规则和右规则
        candys[i] = max( candys[i + 1] + 1 , candys[i]) ;

    }

    // candys[i] 已经是符合左规则和右规则的值了
    // 累加到 result 上面
    result += candys[i];
}

// 返回结果
return result;

}
};

```

3、Python 代码

```

class Solution:
    def candy(self, ratings: List[int]) -> int:
        # 存储每个孩子的糖果
        # 先给所有孩子 1 颗糖
        candys = [1 for _ in range(len(ratings))]

        # 从左至右遍历数组 ratings
        for i in range( 1 , len(ratings)) :

            # 如果发现当前孩子的评分比【左边】的更高
            if ratings[i] > ratings[i - 1] :

                # 那么当前孩子的糖果数量需要比【左边】孩子的糖果数量多 1 个
                candys[i] = candys[i - 1] + 1

        # 记录糖果数量，初始为最右边的值
        result = candys[-1]

        # 从右至左遍历数组 ratings
        for i in range( len(ratings) - 2 , -1 , - 1 ) :

            # 如果发现当前孩子的评分比【右边】的更高

```

```

        if ratings[i] > ratings[i + 1] :

            # 那么当前孩子的糖果数量需要比【右边】孩子的糖果数量多 1 个
            # 当前孩子在第一轮循环中已经设置了值
            # 所以取这两个值的最大值
            # 这样同时满足左规则和右规则
            candys[i] = max( candys[i + 1] + 1 , candys[i])

        # candys[i] 已经是符合左规则和右规则的值了
        # 累加到 result 上面
        result += candys[i]

    # 返回结果
    return result

```

LeetCode 55 、跳跃游戏

一、题目描述

给定一个非负整数数组 `nums`，你最初位于数组的 第一个下标。

数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 跳跃游戏（ LeetCode 55 ） :https://leetcode-cn.com/problems/jump-game/submissions/
class Solution {
    public boolean canJump(int[] nums) {

        // 设置数组，保存每个位置如果在当前位置出发，一次性可以到达的最远位置
        // 比如 nums 为 [ 2 , 4 , 5 , 3 , 1 , 0 , 6]
        // 那么对于 2 来说，可以跳到最远的位置是 5 那个位置，即索引为 2 的那个位置
        // 对于 4 来说，可以跳到最远的位置是 0 那个位置，即索引为 5 的那个位置
        // 对于 5 来说，可以跳到最远的位置超过了数组的范围
        int[] jump = new int[nums.length];

        // 初始化 jump

```

```

for(int i = 0 ; i < nums.length ; i++){
    // jump[i] 就是当前的索引值 i 加上该位置可以跳跃的最大长度 nums[i]
    jump[i] = i + nums[i];
}

// 从数组下标为 0 的位置开始起跳，索引为 0
int index = 0;

// 设置变量 maxJump，用来记录可以到达的最远位置
// 从数组下标为 0 的位置开始起跳，此时记录的最远距离为 jump[0]
int maxJump = jump[0];

// 开始起跳
// 直到 index 到达数组尾部，index >= nums.length
// 或者 index 超过 maxJump
while(index < nums.length && index <= maxJump){

    // 如果发现可以跳到的最远距离 maxJump 小于 jump[index]
    if(maxJump < jump[index]){
        // 那么需要更新 maxJump
        maxJump = jump[index];
    }

    // index 向前移动
    index++;
}

// 循环结束后，如果 index 已经访问了 nums 中所有的元素
if(index > nums.length - 1){
    // 说明可以到达最后一个下标
    return true;
}

// 否则说明无法到达最后一个下标
return false;

}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 跳跃游戏 ( LeetCode 55 ) :https://leetcode-cn.com/problems/jump-game/submissions/
class Solution {
public:
    bool canJump(vector<int>& nums) {

```

```

// 设置数组，保存每个位置如果在当前位置出发，一次性可以到达的最远位置
// 比如 nums 为 [ 2 , 4 , 5 , 3 , 1 , 0 , 6]
// 那么对于 2 来说，可以跳到最远的位置是 5 那个位置，即索引为 2 的那个位置
// 对于 4 来说，可以跳到最远的位置是 0 那个位置，即索引为 5 的那个位置
// 对于 5 来说，可以跳到最远的位置超过了数组的范围
vector<int> jump(nums.size());
// 初始化 jump
for(int i=0;i < nums.size(); i++){
    // jump[i] 就是当前的索引值 i 加上该位置可以跳跃的最大长度 nums[i]
    jump[i] = i + nums[i];
}
// 从数组下标为 0 的位置开始起跳，索引为 0
int index = 0;
// 设置变量 maxJump，用来记录可以到达的最远位置
// 从数组下标为 0 的位置开始起跳，此时记录的最远距离为 jump[0]
int maxJump = jump[0];
// 开始起跳
// 直到 index 到达数组尾部，index >= nums.length
// 或者 index 超过 maxJump
while(index<=maxJump && index < nums.size()){
    // 如果发现可以跳到的更远距离
    if(maxJump<jump[index]){
        maxJump = jump[index]; // 那么需要更新 maxJump
    }
    index++; // index 向前移动
}
// 循环结束后，如果发现 index 的位置在数组 nums 的最后一个位置
if(index > nums.size() - 1){
    return true;
}
return false;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 跳跃游戏（LeetCode 55）：https://leetcode-cn.com/problems/jump-game/submissions/
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        # 设置数组，保存每个位置如果在当前位置出发，一次性可以到达的最远位置
        # 比如 nums 为 [ 2 , 4 , 5 , 3 , 1 , 0 , 6]
        # 那么对于 2 来说，可以跳到最远的位置是 5 那个位置，即索引为 2 的那个位置
        # 对于 4 来说，可以跳到最远的位置是 0 那个位置，即索引为 5 的那个位置
        # 对于 5 来说，可以跳到最远的位置超过了数组的范围

```



```
jump = list(range(len(nums)))

# 初始化 jump
for i in range(len(nums)) :
    # jump[i] 就是当前的索引值 i 加上该位置可以跳跃的最大长度 nums[i]
    jump[i] = i + nums[i]

# 从数组下标为 0 的位置开始起跳, 索引为 0
index = 0

# 设置变量 maxJump, 用来记录可以到达的最远位置
# 从数组下标为 0 的位置开始起跳, 此时记录的最远距离为 jump[0]
maxJump = jump[0]

# 开始起跳
# 直到 index 到达数组尾部, index >= nums.length
# 或者 index 超过 maxJump
while index < len(nums) and index <= maxJump :

    # 如果发现可以跳到的最远距离 maxJump 小于 jump[index]
    if maxJump < jump[index] :
        # 那么需要更新 maxJump
        maxJump = jump[index]

    # index 向前移动
    index += 1

# 循环结束后, 如果 index 已经访问了 nums 中所有的元素
if index > len(nums) - 1 :
    # 说明可以到达最后一个下标
    return True

# 否则说明无法到达最后一个下标
return False
```

四、复杂度分析

时间复杂度： $O(n)$ ，其中 n 为数组的大小。只需要访问 `nums` 数组一遍，共 n 个位置。

空间复杂度： $O(1)$ ，不需要额外的空间开销。

LeetCode 881、救生艇

一、题目描述

给定数组 `people` 。`people[i]` 表示第 `i` 个人的体重，船的数量不限，每艘船可以承载的最大重量为 `limit`。

每艘船最多可同时载两人，但条件是这些人的重量之和最多为 `limit`。

返回 承载所有人所需的最小船数。

示例 1:

输入: `people = [1,2]`, `limit = 3` 输出: 1 解释: 1 艘船载 (1, 2)

示例 2:

输入: `people = [3,2,2,1]`, `limit = 3` 输出: 3 解释: 3 艘船分别载 (1, 2), (2) 和 (3)

示例 3:

输入: `people = [3,5,3,4]`, `limit = 5` 输出: 4 解释: 4 艘船分别载 (3), (3), (4), (5)

提示:

- `1 <= people.length <= 5 * 10^4`
- `1 <= people[i] <= limit <= 3 * 10^4`

二、题目解析

三、参考代码

```
class Solution:
    def numRescueBoats(self, people: List[int], limit: int) -> int:

        # 先排序
        people.sort()

        # 获取数组长度
        n = len(people)

        # 寻找两人组

        # 最左边的位置
        left = 0

        # 最右边的位置
        right = n - 1

        # 船的数量
        ans = 0

        while left <= right :

            # 1、如果 people[left] + people[right] <= limit
```

```
# 说明 people[left] 和 people[right] 可以同船

if people[left] + people[right] <= limit :

    # 此时船的数量加一
    ans += 1

    # 两个指针分别往中间靠拢

    left += 1

    right -= 1

# 如果 people[left] + people[right] > limit
# 说明 people[left] 和 people[right] 不可以同船
else:

    # 由于题目说明了人的重量不会超过 limit
    # people[right] 需要独自坐船，船的数量加一
    ans += 1

    # people[right] 坐船后
    # right 需要向内移动，寻找新的组合
    right -= 1

# 返回结果
return ans
```

第二十二天

LeetCode 136、只出现一次的数字

一、题目描述

给定一个非空整数数组，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。

说明：

你的算法应该具有线性时间复杂度。 你可以不使用额外空间来实现吗？

示例1：

输入: [2,2,1]
输出: 1

示例 2:

输入: [4,1,2,1,2]
输出: 4

二、题目解析

为了满足 算法应该具有线性时间复杂度与不使用额外空间来实现, 本题通过寻找位运算规律来解决。

假设 $a = 1, b = 0$

1、 $1 \oplus 0 = 1, 0 \oplus 0 = 0 \Rightarrow a \oplus 0 = 1, b \oplus 0 = 0 \Rightarrow$ 任何数与0做异或运算结果均为其本身

2、 $1 \oplus 1 = 0, 0 \oplus 0 = 0 \Rightarrow a \oplus a = 0, b \oplus b = 0 \Rightarrow$ 任何数与其本身做异或运算结果均为0

3、 $1 \oplus 0 \oplus 1 = 0 \Rightarrow 1 \oplus 1 = 0 \Rightarrow a \oplus b \oplus a = b$

$1 \oplus 1 \oplus 0 = 0 \Rightarrow 0 \oplus 0 = 0 \Rightarrow a \oplus a \oplus b = b$

$0 \oplus (1 \oplus 1) = 0 \Rightarrow 0 \oplus 0 = 0 \Rightarrow b \oplus (a \oplus a) = b$

$\Rightarrow$ 异或运算满足交换律和结合律

由于 非空整数数组, 除了某个元素只出现一次以外, 其余每个元素均出现两次, 即数组中元素数量恒为奇数。

假设数组中有 $2n+1$ 个元素, 有 n 个元素为出现两次的元素, 如果所有数进行连续异或操作可得:

```
A1  $\oplus$  A1  $\oplus$  A2  $\oplus$  A2 .....  $\oplus$  An  $\oplus$  An  $\oplus$  An+1  
= (A1  $\oplus$  A1)  $\oplus$  (A2  $\oplus$  A2) .....  $\oplus$  (An  $\oplus$  An)  $\oplus$  An+1 //通过规律3  
= (0)  $\oplus$  (0) .....  $\oplus$  (0)  $\oplus$  An+1 //通过规律2  
= An+1 //通过规律1
```

通过规律1可知, 我们在做连续异或操作的时候, 可以以0为起始数开始运算。

三、参考代码

```
class Solution {  
    public int singleNumber(int[] nums) {  
        int single = 0;  
        for (int num : nums) {  
            single ^= num;  
        }  
        return single;  
    }  
}
```

四、复杂度分析

时间复杂度：O(n)

空间复杂度：O(1)

LeetCode 137、只出现一次的数字 II

一、题目描述

给你一个整数数组 `nums`，除某个元素仅出现一次外，其余每个元素都恰出现三次。请你找出并返回那个只出现了一次的元素。

示例 1：

```
输入：nums = [2,2,3,2]
输出：3
```

示例 2：

```
输入：nums = [0,1,0,1,0,1,99]
输出：99
```

提示：

- `1 <= nums.length <= 3 * 104`
- `-231 <= nums[i] <= 231 - 1`
- `nums` 中，除某个元素仅出现一次外，其余每个元素都恰出现三次

进阶：你的算法应该具有线性时间复杂度。你可以不使用额外空间来实现吗？

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 只出现一次的数字 II ( LeetCode 137 ) :https://leetcode-cn.com/problems/single-number-ii
class Solution {
    public int singleNumber(int[] nums) {

        // 设置一个长度为 32 位的数组
        // 在这个数组中，每一位存储的就是 nums 数组中每一位二进制形式相加后 1 的个数
        int[] cnt = new int[32];
```

```
// 遍历 nums 数组
// 虽然这里有两个 for 循环，但是内层循环的 n 是常数，因此时间复杂度依旧为 o(n)
for (int i = 0 ; i < nums.length ; i++ ) {
```

```
    // 获取第 i 位的数字
    int num = nums[i];
```

```
    // 将 num 展开为二进制的形式，有 32 位
    for (int j = 0; j < 32; j++) {
```

被丢弃

```
        // >> 表示右移符号，右移运算是将一个二进制位的操作数按指定移动的位数向右移动，移出位
```

```
        // >> j 表示右移了 j 位
        // 比如 num 的二进制表示为
```

```
        // 0000 0000 0000 0000 0000 0000 0000 0010
```

```
        // >> 1 之后变成了
```

```
        // 000 0000 0000 0000 0000 0000 0000 0001
```

```
        // 在计算机中默认为 0 ，所以结果就是 0000 0000 0000 0000 0000 0000 0000
```

0001

```
        int temp = num >> j;
```

```
        // & 表示按位与运算符，只有对应的两个二进位都为 1 时，结果位才为 1
```

```
        // 每次将 num 向右移动之后，都将移动后的数和 1 进行按位与操作
```

```
        // 这样就得到了 num 中第 i 个二进制位
```

```
        // 比如 num 的二进制表示为
```

```
        // 0000 0000 0000 0000 0000 0000 0000 0010
```

```
        // 当 j = 0 时，temp 为
```

```
        // 0000 0000 0000 0000 0000 0000 0000 001 【0】 // temp 的二进制表示
```

```
        // &
```

```
        // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 1 的二进制表示
```

```
        // =
```

```
        // 0000 0000 0000 0000 0000 0000 0000 000 【0】 // 等于 0
```

```
        // 即 num 的第 0 个位置的二进制位 0
```

```
        // < ----->
```

```
        // 当 j = 1 时，temp 为
```

```
        // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // temp 的二进制表示
```

```
        // &
```

```
        // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 1 的二进制表示
```

```
        // =
```

```
        // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 等于 1
```

```
        // 即 num 的第 1 个位置的二进制位 1
```

```
        int x = temp & 1;
```

```
        // 如果第 j 位的二进制位是 1，把它累加到 cnt[j] 上
```

```
        // cnt[j] 表示 nums 数组中每一个数字二进制形式表示后，第 j 位有多少个 1
```

```
        if ( x == 1) {
```

```
            cnt[j]++;
```

```

    }
}

// 使用 ans 用来记录 nums 中那个只出现了一次的元素
// 一开始 ans 的二进制表示为
// 0000 0000 0000 0000 0000 0000 0000 0000
// 即由 32 个 0 组成
int ans = 0;

// 此时, cnt 已经存储的 nums 数组中每一位二进制形式相加后 1 的个数
// 遍历每个位置
for (int i = 0; i < 32; i++) {

    // 因为相同数字的二进制表示是一样的, 也就意味着如果它出现了三次, 那么这个数字相加三次之
    // 后,
    // 在某个位置上出现 1 的次数要么是 0 次, 要么是 3 次
    // 比如数字 2, 它的二进制是 0000 0000 0000 0000 0000 0000 0000 00【1】0
    // 出现三次时, 【1】也必然出现三次
    // 将某个位置相加的结果除以 3, 如果余数为 1, 表明只出现了一次
    int temp = cnt[i] % 3;

    // 把出现了一次的二进制位添加到 ans 上
    if (temp == 1) {

        // << 表示左移符号, 用来将一个数的各二进制位全部左移若干位
        // 移动的位数由右操作数指定, 右操作数必须是非负值, 其右边空出的位用 0 填补, 高位左移
        // 溢出则舍弃该高位
        // 这个操作就是不断的填充二进制
        int num = 1 << i;

        // 把二进制结果累加到 ans 上
        ans += num;
    }
}

// 返回 ans 就行
return ans;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 只出现一次的数字 II ( LeetCode 137 ) :https://leetcode-cn.com/problems/single-number-ii

```

```

class Solution {
public:
    int singleNumber(vector<int>& nums) {

        // 设置一个长度为 32 位的数组
        // 在这个数组中，每一位存储的就是 nums 数组中每一位二进制形式相加后 1 的个数
        vector<int> cnt(32, 0);

        // 遍历 nums 数组
        // 虽然这里有两个 for 循环，但是内层循环的 n 是常数，因此时间复杂度依旧为 O(n)
        for (int i = 0 ; i < nums.size() ; i++ ) {

            // 获取第 i 位的数字
            int num = nums[i];

            // 将 num 展开为二进制的形式，有 32 位
            for (int j = 0; j < 32; j++) {

                // >> 表示右移符号，右移运算是将一个二进制位的操作数按指定移动的位数向右移动，移出位
                // 被丢弃

                // >> j 表示右移了 j 位
                // 比如 num 的二进制表示为
                // 0000 0000 0000 0000 0000 0000 0000 0010
                // >> 1 之后变成了
                // 000 0000 0000 0000 0000 0000 0000 0001
                // 在计算机中默认为 0，所以结果就是 0000 0000 0000 0000 0000 0000 0000 0000
                // 0001

                int temp = num >> j;

                // & 表示按位与运算符，只有对应的两个二进制位都为 1 时，结果位才为 1
                // 每次将 num 向右移动之后，都将移动后的数和 1 进行按位与操作
                // 这样就得到了 num 中第 i 个二进制位
                // 比如 num 的二进制表示为
                // 0000 0000 0000 0000 0000 0000 0000 0010
                // 当 j = 0 时，temp 为
                // 0000 0000 0000 0000 0000 0000 0000 001 【0】 // temp 的二进制表示
                // &
                // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 1 的二进制表示
                // =
                // 0000 0000 0000 0000 0000 0000 0000 000 【0】 // 等于 0
                // 即 num 的第 0 个位置的二进制位 0
                // < ----->
                // 当 j = 1 时，temp 为
                // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // temp 的二进制表示
                // &
                // 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 1 的二进制表示
                // =
            }
        }
    }
};

```


后,

溢出则舍弃该高位

};

```
// 0000 0000 0000 0000 0000 0000 0000 000 【1】 // 等于 1
// 即 num 的第 1 个位置的二进制位 1
int x = temp & 1;

// 如果第 j 位的二进制位是 1, 把它累加到 cnt[j] 上
// cnt[j] 表示 nums 数组中每一个数字二进制形式表示后, 第 j 位有多少个 1
if ( x == 1) {
    cnt[j]++;
}

}

// 使用 ans 用来记录 nums 中那个只出现了一次的元素
// 一开始 ans 的二进制表示为
// 0000 0000 0000 0000 0000 0000 0000 0000
// 即由 32 个 0 组成
int ans = 0;

// 此时, cnt 已经存储的 nums 数组中每一位二进制形式相加后 1 的个数
// 遍历每个位置
for (int i = 0; i < 32; i++) {

    // 因为相同数字的二进制表示是一样的, 也就意味着如果它出现了三次, 那么这个数字相加三次之

    // 在某个位置上出现 1 的次数要么是 0 次, 要么是 3 次
    // 比如数字 2, 它的二进制是 0000 0000 0000 0000 0000 0000 0000 00 【1】 0
    // 出现三次时, 【1】 也必然出现三次
    // 将某个位置相加的结果除以 3, 如果余数为 1, 表明只出现了一次
    int temp = cnt[i] % 3;

    // 把出现了一次的二进制位添加到 ans 上
    if (temp == 1) {

        // << 表示左移符号, 用来将一个数的各二进制位全部左移若干位
        // 移动的位数由右操作数指定, 右操作数必须是非负值, 其右边空出的位用 0 填补, 高位左移
        // 溢出则舍弃该高位

        // 这个操作就是不断的填充二进制
        int num = 1 << i;

        // 把二进制结果累加到 ans 上
        ans += num;
    }
}

// 返回 ans 就行
return ans;
}
```

3、Python 代码

```
#登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 只出现一次的数字 II ( LeetCode 137 ) :https://leetcode-cn.com/problems/single-number-ii

class Solution:
    def singleNumber(self, nums: List[int]) -> int:

        # 设置一个长度为 32 位的数组
        # 在这个数组中，每一位存储的就是 nums 数组中每一位二进制形式相加后 1 的个数
        cnt = [0] * 32

        # 遍历 nums 数组
        # 虽然这里有两个 for 循环，但是内层循环的 n 是常数，因此时间复杂度依旧为 O(n)
        for i in range(len(nums)) :

            # 获取第 i 位的数字
            num = nums[i]

            # 将 num 展开为二进制的形式，有 32 位
            for j in range(32) :

                # >> 表示右移符号，右移运算是将一个二进制位的操作数按指定移动的位数向右移动，移出位
                # 被丢弃

                # >> j 表示右移了 j 位
                # 比如 num 的二进制表示为
                # 0000 0000 0000 0000 0000 0000 0000 0010
                # >> 1 之后变成了
                # 000 0000 0000 0000 0000 0000 0000 0001
                # 在计算机中默认为 0，所以结果就是 0000 0000 0000 0000 0000 0000 0000 0001
                temp = num >> j

                # & 表示按位与运算符，只有对应的两个二进位都为 1 时，结果位才为 1
                # 每次将 num 向右移动之后，都将移动后的数和 1 进行按位与操作
                # 这样就得到了 num 中第 i 个二进制位
                # 比如 num 的二进制表示为
                # 0000 0000 0000 0000 0000 0000 0000 0010
                # 当 j = 0 时，temp 为
                # 0000 0000 0000 0000 0000 0000 0000 001【0】 # temp 的二进制表示
                # &
                # 0000 0000 0000 0000 0000 0000 0000 000【1】 # 1 的二进制表示
                # =
                # 0000 0000 0000 0000 0000 0000 0000 000【0】 # 等于 0
                # 即 num 的第 0 个位置的二进制位 0
```

```

# < ----->
# 当 j = 1 时, temp 为
# 0000 0000 0000 0000 0000 0000 0000 000 【1】 # temp 的二进制表示
# &
# 0000 0000 0000 0000 0000 0000 0000 000 【1】 # 1 的二进制表示
# =
# 0000 0000 0000 0000 0000 0000 0000 000 【1】 # 等于 1
# 即 num 的第 1 个位置的二进制位 1
x = temp & 1

```

```

# 如果第 j 位的二进制位是 1, 把它累加到 cnt[j] 上
# cnt[j] 表示 nums 数组中每一个数字二进制形式表示后, 第 j 位有多少个 1
if x == 1 :
    cnt[j] += 1

```

```

# 使用 ans 用来记录 nums 中那个只出现了一次的元素
# 一开始 ans 的二进制表示为
# 0000 0000 0000 0000 0000 0000 0000 0000
# 即由 32 个 0 组成
ans = 0

```

```

# 此时, cnt 已经存储的 nums 数组中每一位二进制形式相加后 1 的个数
# 遍历每个位置
for i in range(32) :

```

```

    # 因为相同数字的二进制表示是一样的, 也就意味着如果它出现了三次, 那么这个数字相加三次之后,
    # 在某个位置上出现 1 的次数要么是 0 次, 要么是 3 次
    # 比如数字 2, 它的二进制是 0000 0000 0000 0000 0000 0000 0000 00 【1】 0
    # 出现三次时, 【1】 也必然出现三次
    # 将某个位置相加的结果除以 3, 如果余数为 1, 表明只出现了一次
    temp = cnt[i] % 3

```

```

    # 把出现了一次的二进制位添加到 ans 上
    if temp == 1 :

```

```

        # << 表示左移符号, 用来将一个数的各二进制位全部左移若干位
        # 移动的位数由右操作数指定, 右操作数必须是非负值, 其右边空出的位用 0 填补, 高位左移
        # 溢出则舍弃该高位
        # 这个操作就是不断的填充二进制
        num = 1 << i

        # 把二进制结果累加到 ans 上
        ans += num

```

```
# 返回 ans 就行
# 在 Python 中需要对最高位进行特殊判断。
return ans if cnt[31] % 3 == 0 else ~(ans ^ 0xffffffff)
```

LeetCode 169、多数元素

一、题目描述

数组中有一个数字出现的次数超过数组长度的一半，请找出这个数字。

你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例 1:

输入: [1, 2, 3, 2, 2, 2, 5, 4, 2]
输出: 2

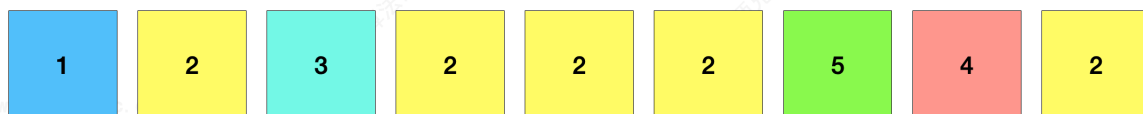
限制:

1 <= 数组长度 <= 50000

二、题目解析

题目明确说明在这个数组中肯定有一个数字出现的次数超过数组长度的一半，可以假设这个数字是 x ，并且假设每个数字的战斗力的都是 1，那么其它所有数字、即非 x 的数字加起来的战斗力肯定是不如所有 x 加起来的战斗力。

那么我们可以制定以下的规则：



擂台

公众号：吴师兄学算法

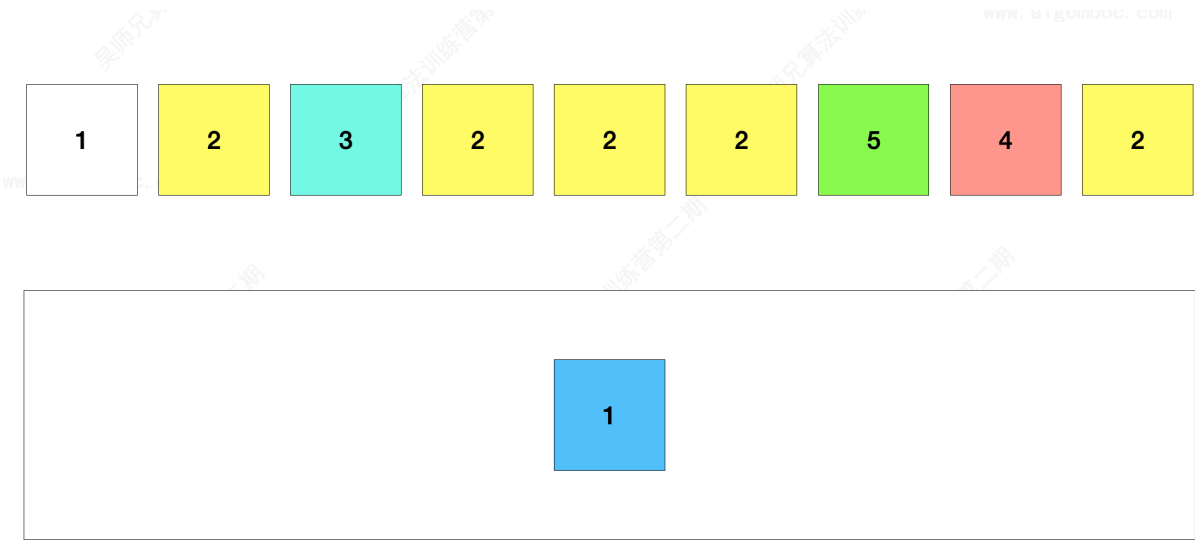
- 1、每个数字的战斗力都是 1，颜色相同的数字为同一组势力。
- 2、设置一个擂台，每个数字都需要轮番上擂台。

- 3、如果擂台上没有数字，那么该数字就是擂主。
- 4、如果擂台上已经有数字，但两方的势力不同，则同归于尽。
- 5、如果擂台上已经有数字，并且两方的势力一致，则擂主还是它，并且数量加 1。

具体操作如下：

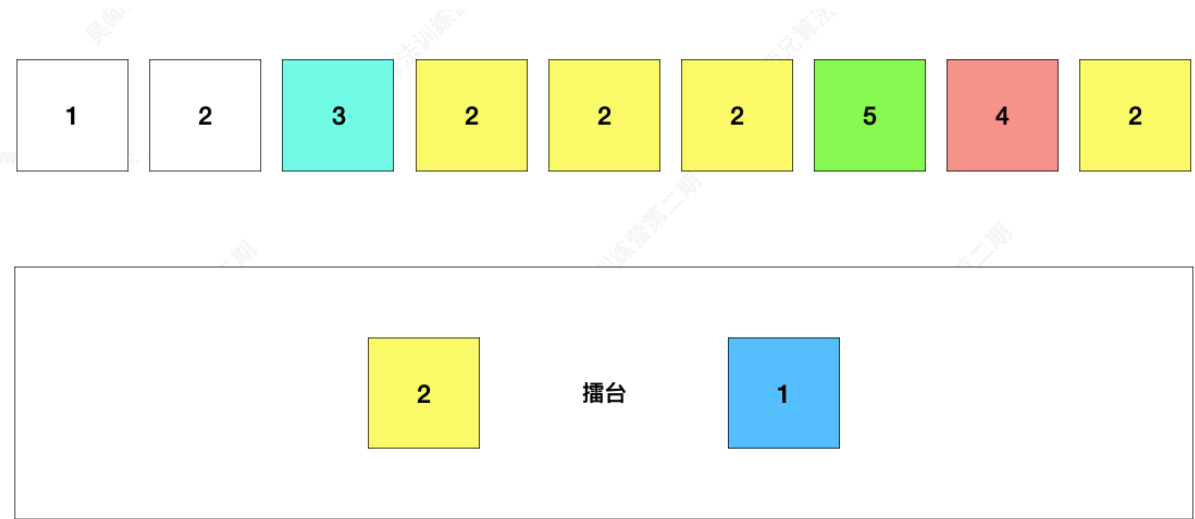
输入：[1, 2, 3, 2, 2, 2, 5, 4, 2]

一开始，蓝色势力先登场，由于擂台上之前没有数字，那么蓝色数字 1 就是擂主了。



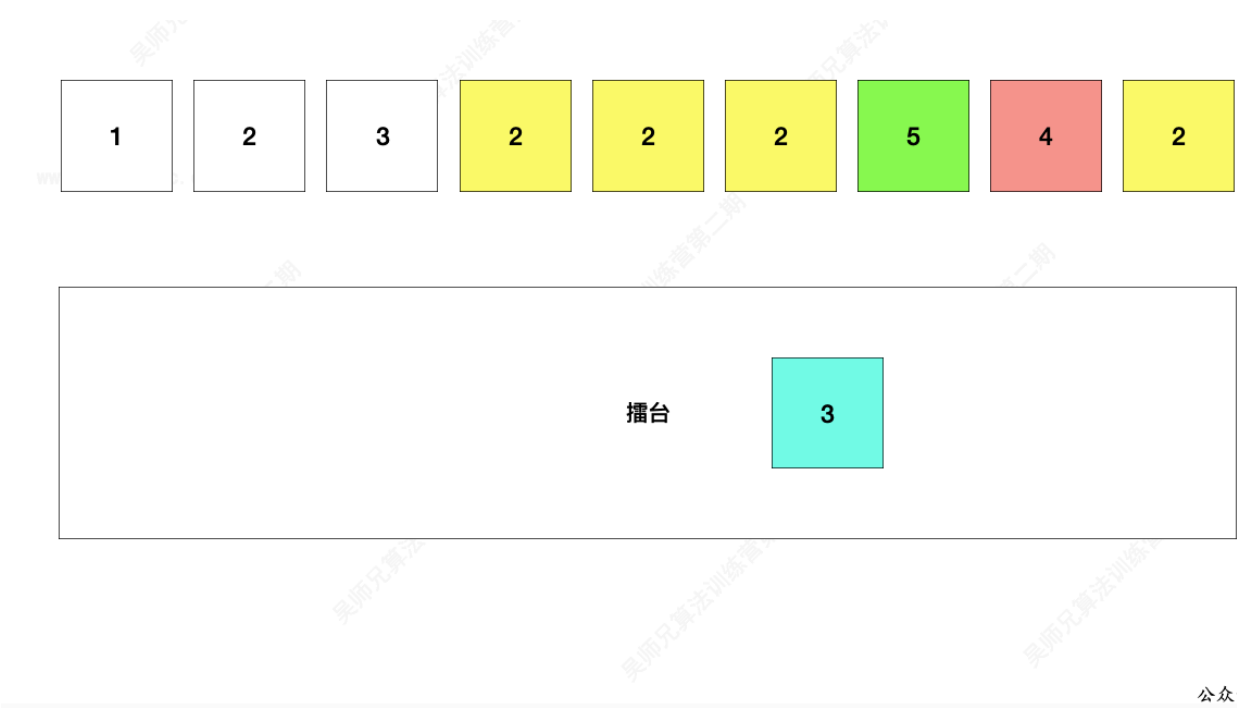
公众号：吴师兄学算法

接下来，黄色势力 2 开始挑战，由于蓝色和黄色属于不同的势力、战斗力一致、都只有一个，因此两者同归于尽，擂台被清空。



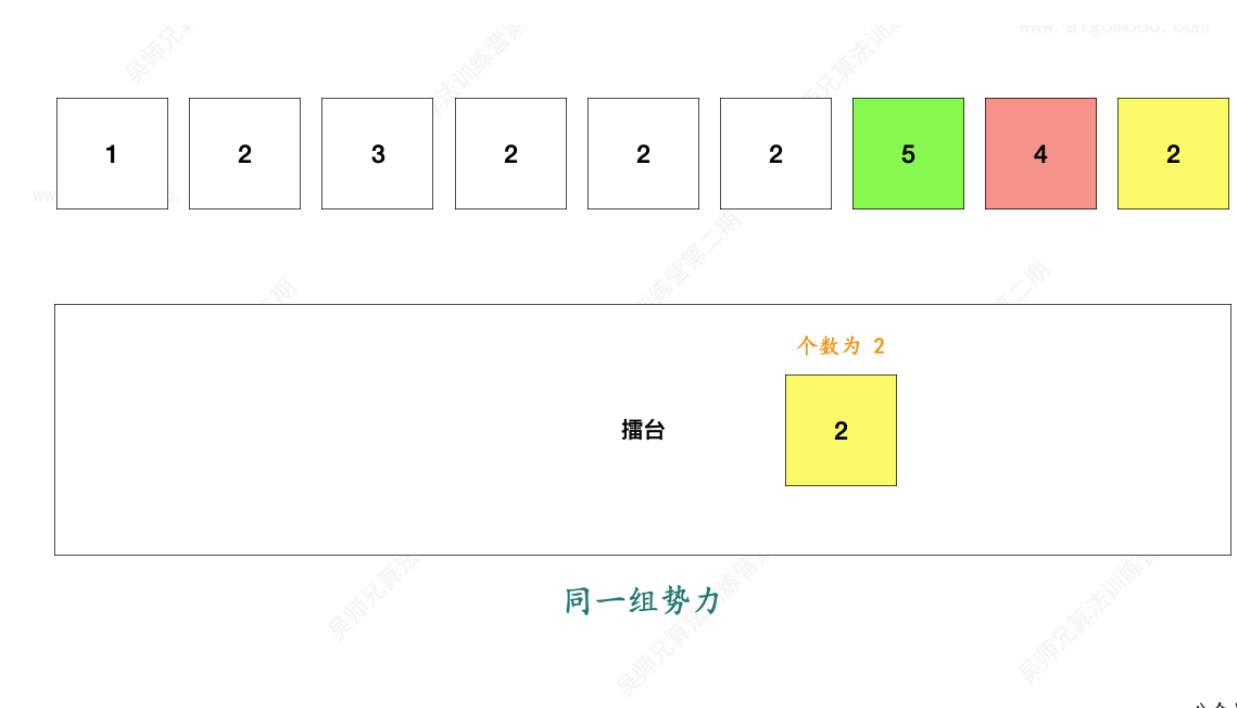
公众号：吴师兄学算法

擂台清空后，3 号登场成为了新的擂主。



公众号：吴师兄学算法

- 2 号登场，擂台被清空。
- 2 号登场，擂台擂主为 2 号。
- 2 号登场，擂台擂主依旧为 2 号，并且个数为 2 个。



公众号：吴师兄学算法

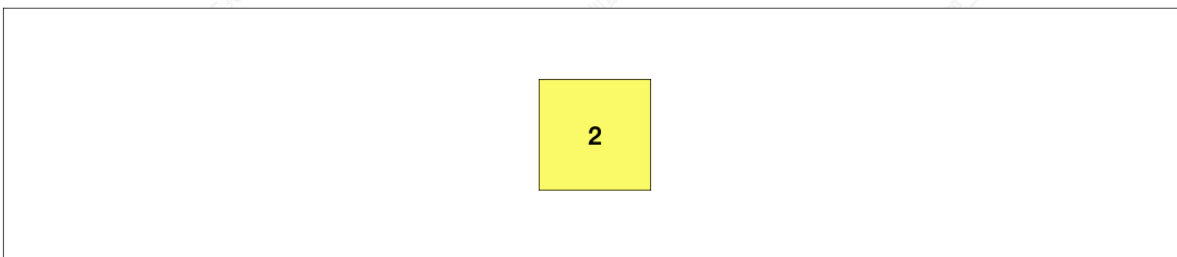
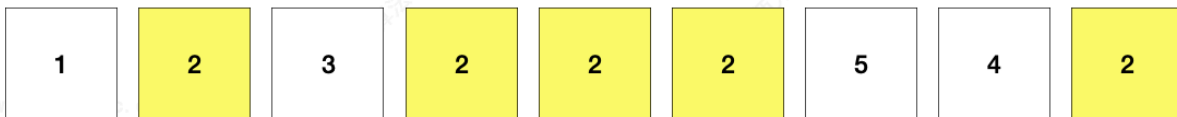
5 号登场，与 2 号不同势力，拉下一个擂主来，导致擂台上擂主依旧为 2，但个数下降为 1 个。



公众号：吴师兄学算法

4 号登场，与 2 号不同势力，拉下一个擂主来，擂台被清空。

2 号登场，成为新的擂主，全部数字挑战完毕，2 号留在场上，它就是所求的那个数字。



公众号：吴师兄学算法

为了帮助你更好的理解整个过程，我特意做了一组动画，[点开可以查看](#)：

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
```

// 剑指 offer 39. 数组中出现次数超过一半的数字 : <https://leetcode-cn.com/problems/shu-zu-zhong-chu-xian-ci-shu-chao-guo-yi-ban-de-shu-zi-lcof/>

```
class Solution {
    public int majorityElement(int[] nums) {

        // 统计擂台上擂主的个数
        int count = 0;

        // candidate 表示擂主的编号
        // 一开始，擂台上没有擂主
        int candidate = 0;

        // 数组中的所有数字开始轮番上擂台进行挑战，每个数字的战斗力的均为 1
        // 1、相同势力的数字可以都停留在擂台上
        // 2、不同势力的数字会同归于尽
        for (int num : nums) {

            // 擂台上没有擂主时
            if (count == 0) {
                // 此时登场的 num 就是擂主
                candidate = num;
            }

            // 擂台上有擂主
            // 并且此时登场的 num 和擂主属于相同的势力
            // 那么两者都停留在场上
            if (num == candidate){
                count += 1;

                // 否则说明此时登场的 num 和擂主属于不同的势力
                // num 会和一个擂主同归于尽
            }else{
                count -= 1;
            }

        }

        // 一轮遍历下来，停留在场上的擂主就是所求的数字
        return candidate;
    }
}
```

LeetCode 260、只出现一次的数字 III

一、题目描述

给定一个整数数组 `nums`，其中恰好有两个元素只出现一次，其余所有元素均出现两次。找出只出现一次的那两个元素。你可以按 **任意顺序** 返回答案。

进阶：你的算法应该具有线性时间复杂度。你能否仅使用常数空间复杂度来实现？

示例 1：

输入：nums = [1,2,1,3,2,5]

输出：[3,5]

解释：[5, 3] 也是有效的答案。

提示：

- $2 \leq \text{nums.length} \leq 3 \times 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- 除两个只出现一次的整数外，`nums` 中的其他数字都出现两次

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 只出现一次的数字 III ( LeetCode 260 ) : https://leetcode-cn.com/problems/single-number-iii/
class Solution {
    public int[] singleNumber(int[] nums) {

        // sum 表示数组 nums 所有元素进行异或的结果
        int sum = 0;

        // 遍历 nums 中的每个元素
        for (int num : nums){

            // 将每个元素都执行异或操作
            sum ^= num;
        }

        // 1、由于 nums 中其余元素均出现两次
        // 2、异或的特征：不同为 1，相同为 0
        // 因此，最终的 sum 就是那两个只出现一次的元素异或的结果
        // 假设这两个只出现一次的元素分别为 a 和 b
        // a = 011，b = 101，那么 sum = 110
        // 也就意味着 sum 上面的 1 必然是由 a 与 b 贡献的，因为两个相同元素的二进制必然相同，异或一下为 0
    }
}
```

```

// 假设 sum 上面的第 k 位是 1
// 那么，我们可以将 nums 划分为两部分数组
// 1、第 k 位是 1，包含了 b 与 nums 中第 k 位是 1 的元素，并且这些元素出现了两次
// 2、第 k 位是 0，包含了 a 与 nums 中第 k 位是 0 的元素，并且这些元素出现了两次
// 也就是说，a 与 b 划分到了不同的部分
// 而相同的元素进行异或为 0，那么每一部分进行异或最终就剩下了一个只出现一次的元素

// 先找出 sum 中任意一位为 1 的位置，我们这里去找最低位为 1 的位置
// sum & -sum 的结果是 nsum 的最低位的二进制
// 比如 sum = 1011，那么它的反码是 0100，补码就是 0101
// sum =          1011
// &
// -sum =         0101
// -----
// 结果就是    0001
// 即获取到了 sum 中最低位为 1 的位置
// 实际上也可以这样理解，-sum 是 sum 取反再加 1，那么 sum & -sum 取得的就是再加的那个

// 特殊情况，如果 sum = Integer.MIN_VALUE，即 sum = -2^31
// Integer.MIN_VALUE，即 -2147483648，二进制位如下：
// 1000 0000 0000 0000 0000 0000 0000 0000
// 取反后再加 1 会产生溢出的情况
int k = (sum == Integer.MIN_VALUE) ? sum : sum & -sum;

// 假设这两个只出现一次的元素分别为 a 和 b
int a = 0;

// 假设这两个只出现一次的元素分别为 a 和 b
int b = 0;

// 再次遍历 nums，将 num 划分为两部分
for(int num:nums){

    // 这部分表示第 k 位为 0 的元素，包含了 a 与 nums 中第 k 位是 0 的元素，并且这些元素
    // 出现了两次
    if( ( k & num ) == 0 ){

        // 这些元素进行异或，最终留下了那个只出现一次的元素 a
        a ^= num;

        // 这部分表示第 k 位为 1 的元素，包含了 b 与 nums 中第 k 位是 1 的元素，并且这些元素
        // 出现了两次
    }else{

        // 这些元素进行异或，最终留下了那个只出现一次的元素 b
        b ^= num;

    }

}
}

```

```
// 结合为数组的形式进行返回
int ans[] = { a , b } ;

// 返回结果
return ans;
}
}
```

第二十三天

LeetCode 43、字符串相乘

一、题目描述

给定两个以字符串形式表示的非负整数 `num1` 和 `num2`，返回 `num1` 和 `num2` 的乘积，它们的乘积也表示为字符串形式。

注意：不能使用任何内置的 `BigInteger` 库或直接将输入转换为整数。

示例 1:

```
输入: num1 = "2", num2 = "3"
输出: "6"
```

示例 2:

```
输入: num1 = "123", num2 = "456"
输出: "56088"
```

提示：

- `1 <= num1.length, num2.length <= 200`
- `num1` 和 `num2` 只能由数字组成。
- `num1` 和 `num2` 都不包含任何前导零，除了数字0本身。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
```

// 字符串相乘 (LeetCode 43) : <https://leetcode.cn/problems/multiply-strings/submissions/>

```
class Solution {
    public String multiply(String num1, String num2) {

        // 边界处理, 任何数和 0 相乘都为 0
        if (num1.equals("0") || num2.equals("0")) {
            // 直接返回 0
            return "0";
        }

        // 获取字符串 num1 的长度
        int m = num1.length();

        // 获取字符串 num2 的长度
        int n = num2.length();

        // 基于这两个长度可以构造一个长度为 m + n 的数组
        // 因为假设 m = 2 , 最大数是 99
        // n = 3 , 最大数是 999
        // 两者相乘的结果 98901 长度为 5
        int[] ansArr = new int[m + n];

        // 从后向前依次访问 num1 中的元素
        for (int i = m - 1; i >= 0; i--) {

            // '0' 的 ASCII 码是 48 , '1' 的是 49
            // 获取数字, 相当于字符串转整数操作
            int x = num1.charAt(i) - 48;

            // 从后向前依次访问 num2 中的元素
            for (int j = n - 1; j >= 0; j--) {

                // '0' 的 ASCII 码是 48 , '1' 的是 49
                // 获取数字, 相当于字符串转整数操作
                int y = num2.charAt(j) - 48;

                // 把 x 和 y 相乘的结果存放到 i + j + 1 位
                // 比如一开始 i = m - 1 、 j = n - 1
                // 那么就存放到 m - 1 + n - 1 + 1 = m + n - 1
                // 数组的索引是从 0 开始 , 即此时存放到了最后一位
                // 并且由于 x 和 y 必然都是个位数, 相乘的结果最多只能是两位数
                ansArr[ i + j + 1 ] += x * y;

            }
        }

        // 接下来, 需要把 ansArr 的每一位都拿出来查看一下
        // 使得每一个都只存放一位数字, 多余的传递给上一层
    }
}
```

```

for (int k = m + n - 1; k > 0; k--) {

    // 上一位的数字需要累加当前这一位的十位数
    // 比如 ansArr[k] = 24 , ansArr[k - 1] = 68
    // 那么 ansArr[k - 1] 需要加上 2 , 变成了 70
    ansArr[k - 1] += ansArr[k] / 10;

    // 然后 ansArr[k] 就更新为它的个位数
    ansArr[k] %= 10;
}

// 最后, 开始把数组转换为字符串的形式
StringBuffer ans = new StringBuffer();

// 从最高位开始接收
// 此时, 需要判断一下 ansArr[0] 是否为 0 , 如果为 0 , 那么就抛弃这一位, 从下一位在开始接收
int index = ansArr[0] == 0 ? 1 : 0;

// 不断的去填充字符串
while (index < m + n) {

    // 一位一位去接收
    ans.append(ansArr[index]);

    // 再去填充下一位
    index++;
}

// 最后返回结果
return ans.toString();
}
}

```

2、C++ 代码

```

class Solution {
public:
    string multiply(string num1, string num2) {
        // 边界处理, 任何数和 0 相乘都为 0
        if (num1 == "0" || num2 == "0") {
            // 直接返回 0
            return "0";
        }

        // 获取字符串 num1 的长度
        int m = num1.size() ;

        // 获取字符串 num2 的长度

```

```

int n = num2.size();

// 基于这两个长度可以构造一个长度为 m + n 的数组
// 因为假设 m = 2 , 最大数是 99
// n = 3 , 最大数是 999
// 两者相乘的结果 98901 长度为 5
auto ansArr = vector<int>(m + n);

// 从后向前依次访问 num1 中的元素
for (int i = m - 1; i >= 0; i--) {

    // '0' 的 ASCII 码是 48 , '1' 的是 49
    // 获取数字, 相当于字符串转整数操作
    int x = num1.at(i) - '0' ;

    // 从后向前依次访问 num2 中的元素
    for (int j = n - 1; j >= 0; j--) {

        // '0' 的 ASCII 码是 48 , '1' 的是 49
        // 获取数字, 相当于字符串转整数操作
        int y = num2.at(j) - '0';

        // 把 x 和 y 相乘的结果存放到 i + j + 1 位
        // 比如一开始 i = m - 1 、 j = n - 1
        // 那么就存放到 m - 1 + n - 1 + 1 = m + n - 1
        // 数组的索引是从 0 开始 , 即此时存放到了最后一位
        // 并且由于 x 和 y 必然都是个位数, 相乘的结果最多只能是两位数
        ansArr[ i + j + 1 ] += x * y;

    }
}

// 接下来, 需要把 ansArr 的每一位都拿出来查看一下
// 使得每一个都只存放一位数字, 多余的传递给上一层
for (int k = m + n - 1; k > 0; k--) {

    // 上一位的数字需要累加当前这一位的十位数
    // 比如 ansArr[k] = 24 , ansArr[k - 1] = 68
    // 那么 ansArr[k - 1] 需要加上 2 , 变成了 70
    ansArr[k - 1] += ansArr[k] / 10;

    // 然后 ansArr[k] 就更新为它的个位数
    ansArr[k] %= 10;
}

// 最后, 开始把数组转换为字符串的形式
string ans;

// 从最高位开始接收

```

```

// 此时，需要判断一下 ansArr[0] 是否为 0 ， 如果为 0 ， 那么就抛弃这一位，从下一位在开始接收
int index = ansArr[0] == 0 ? 1 : 0;

// 不断的去填充字符串
while (index < m + n) {

    // 一位一位去接收
    ans.push_back(ansArr[index]);

    // 再去填充下一位
    index++;
}

// 转换为数字的形式
for (auto &c: ans) {
    c += '0';
}

// 最后返回结果
return ans;
}
};

```

3、Python 代码

```

class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        # 边界处理，任何数和 0 相乘都为 0
        if num1 == "0" or num2 == "0":
            # 直接返回 0
            return "0"

        # 获取字符串 num1 的长度
        m = len(num1)

        # 获取字符串 num2 的长度
        n = len(num2)

        # 基于这两个长度可以构造一个长度为 m + n 的数组
        # 因为假设 m = 2 ， 最大数是 99
        # n = 3 ， 最大数是 999
        # 两者相乘的结果 98901 长度为 5
        ansArr = [0] * (m + n)

        # 从后向前依次访问 num1 中的元素
        for i in range(m - 1, -1, -1):

            # '0' 的 ASCII 码是 48 ， '1' 的是 49

```

```

# 获取数字，相当于字符串转整数操作
x = int(num1[i])

# 从后向前依次访问 num2 中的元素
for j in range(n - 1, -1, -1):

    # '0' 的 ASCII 码是 48，'1' 的是 49
    # 获取数字，相当于字符串转整数操作
    y = int(num2[j])

    # 把 x 和 y 相乘的结果存放到 i + j + 1 位
    # 比如一开始 i = m - 1、j = n - 1
    # 那么就存放到 m - 1 + n - 1 + 1 = m + n - 1
    # 数组的索引是从 0 开始，即此时存放到了最后一位
    # 并且由于 x 和 y 必然都是个位数，相乘的结果最多只能是两位数
    ansArr[ i + j + 1 ] += x * y

# 接下来，需要把 ansArr 的每一位都拿出来查看一下
# 使得每一个都只存放一位数字，多余的传递给上一层
for k in range(m + n - 1, 0, -1):

    # 上一位的数字需要累加当前这一位的十位数
    # 比如 ansArr[k] = 24，ansArr[k - 1] = 68
    # 那么 ansArr[k - 1] 需要加上 2，变成了 70
    ansArr[k - 1] += ansArr[k] // 10

    # 然后 ansArr[k] 就更新为它的个位数
    ansArr[k] %= 10

# 最后，开始把数组转换为字符串的形式

# 从最高位开始接收
# 此时，需要判断一下 ansArr[0] 是否为 0，如果为 0，那么就抛弃这一位，从下一位在开始接收
index = 1 if ansArr[0] == 0 else 0

# 不断的去填充字符串
ans = ''.join(str(x) for x in ansArr[index:])

# 最后返回结果
return ans

```

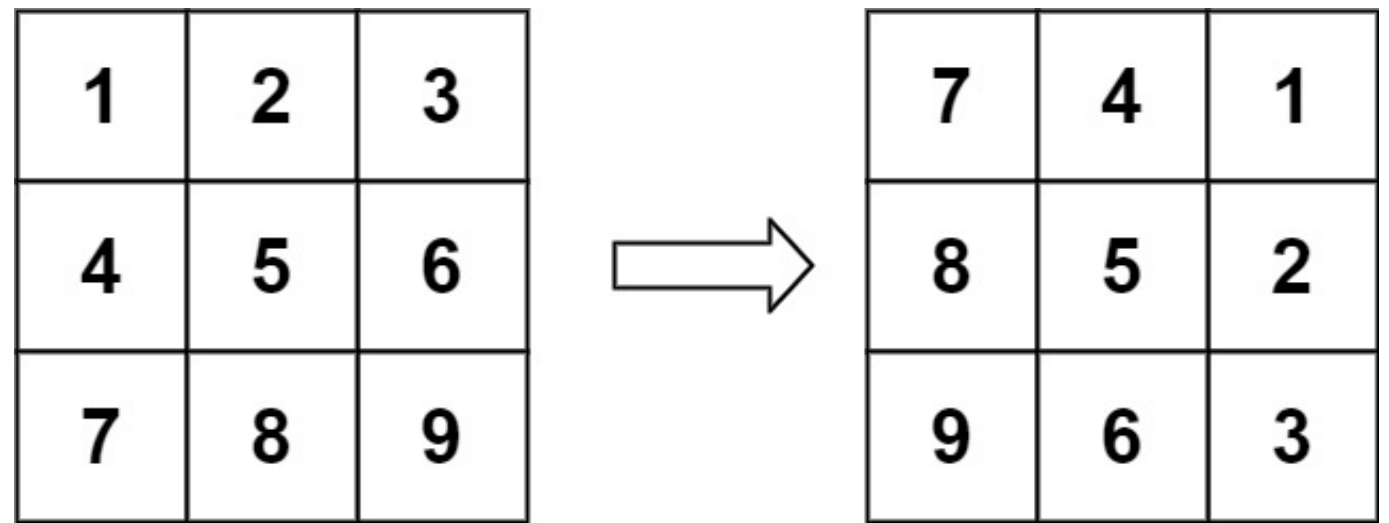

LeetCode 48、旋转图像

一、题目描述

给定一个 $n \times n$ 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。

你必须在 原地 旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要使用另一个矩阵来旋转图像。

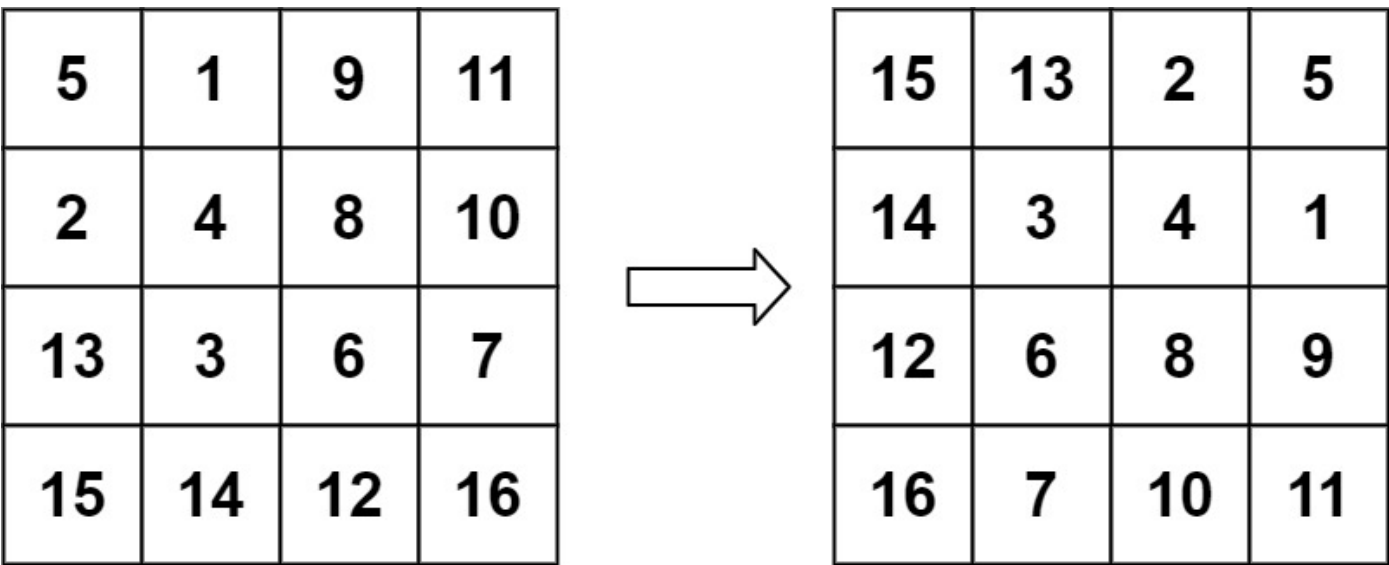
示例 1:



img

```
输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出: [[7,4,1],[8,5,2],[9,6,3]]
```

示例 2:



img

```
输入: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
输出: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```

提示:

- `n == matrix.length == matrix[i].length`
- `1 <= n <= 20`
- `-1000 <= matrix[i][j] <= 1000`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 旋转图像 (LeetCode 48) : https://leetcode.cn/problems/rotate-image/
class Solution {
    public void rotate(int[][] matrix) {

        // 这是一个 n 行 n 列的矩阵
        int n = matrix.length;

        // 只需要遍历 n / 2 行
        for (int i = 0; i < n / 2; i++) {

            // 每一列从 i 开始直到 n - i - 1
            for (int j = i; j < n - i - 1; j++) {

                // 每一轮变化的过程是四个元素旋转
                int temp = matrix[i][j];

                // 假设 matrix[i][j] 位于左上角的元素
                // 那么这个位置会被左下角元素替换，即 matrix[n - j - 1][i] 来到这个位置
                matrix[i][j] = matrix[n - j - 1][i];

                // 左下角元素被右下角的元素替换，即 matrix[n - i - 1][n - j - 1] 来到这个位置
                matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];

                // 右下角元素被右上角的元素替换，即 matrix[j][n - i - 1] 来到这个位置
                matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];

                // 右上角元素被左上角的元素替换，即 matrix[i][j] 来到这个位置
                matrix[j][n - i - 1] = temp;
            }
        }
    }
}
```

2、**C++** 代码

```
class solution {
public:
    void rotate(vector<vector<int>>& matrix) {

        // 这是一个 n 行 n 列的矩阵
        int n = matrix.size();

        // 只需要遍历 n / 2 行
        for (int i = 0 ; i < n / 2 ; i++) {

            // 每一列从 i 开始直到 n - i - 1
            for (int j = i ; j < n - i - 1 ; j++) {

                // 每一轮变化的过程是四个元素旋转

                int temp = matrix[i][j];

                // 假设 matrix[i][j] 位于左上角的元素
                // 那么这个位置会被左下角元素替换, 即 matrix[n - j - 1][i] 来到这个位置
                matrix[i][j] = matrix[ n - j - 1][i];

                // 左下角元素被右下角的元素替换, 即 matrix[n - i - 1][n - j - 1] 来到这个位置
                matrix[ n - j - 1][i] = matrix[n - i - 1][n - j - 1] ;

                // 右下角元素被右上角的元素替换, 即 matrix[j][n - i - 1] 来到这个位置
                matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1] ;

                // 右上角元素被左上角的元素替换, 即 matrix[i][j] 来到这个位置
                matrix[j][n - i - 1] = temp;

            }
        }

    }
};
```

3、Python 代码

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        # 这是一个 n 行 n 列的矩阵
        n = len(matrix)

        # 只需要遍历 n / 2 行
        for i in range( n // 2 ) :

            # 每一列从 i 开始直到 n - i - 1
```

```
for j in range(i, n - i - 1):

    # 每一轮变化的过程是四个元素旋转
    temp = matrix[i][j]

    # 假设 matrix[i][j] 位于左上角的元素
    # 那么这个位置会被左下角元素替换, 即 matrix[n - j - 1][i] 来到这个位置
    matrix[i][j] = matrix[n - j - 1][i]

    # 左下角元素被右下角的元素替换, 即 matrix[n - i - 1][n - j - 1] 来到这个位置
    matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1]

    # 右下角元素被右上角的元素替换, 即 matrix[j][n - i - 1] 来到这个位置
    matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1]

    # 右上角元素被左上角的元素替换, 即 matrix[i][j] 来到这个位置
    matrix[j][n - i - 1] = temp
```

LeetCode 50、Pow(x,n)

一、题目描述

实现 $\text{pow}(x, n)$, 即计算 x 的整数 n 次幂函数 (即, x^n)。

示例 1:

输入: $x = 2.00000$, $n = 10$
输出: 1024.00000

示例 2:

输入: $x = 2.10000$, $n = 3$
输出: 9.26100

示例 3:

输入: $x = 2.00000$, $n = -2$
输出: 0.25000
解释: $2^{-2} = 1/2^2 = 1/4 = 0.25$

提示:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31}-1$
- $-10^4 \leq x^n \leq 10^4$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// Pow(x,n) (LeetCode 50) : https://leetcode.cn/problems/powx-n/
class Solution {
    public double myPow(double x, int n) {

        // 递归终止条件
        if (n == 0) {

            // 任何数的 0 次方的结果都是 1，由于题目要求返回 double 类型，因此返回 1.0
            return 1.0;

        }

        // 判断 n 是奇数还是偶数
        // 1、如果是偶数
        if ((n & 1) == 0) {
            // 那么只需要先计算出 y = x * x 的结果
            // 然后 y 的次幂就是 n / 2
            return myPow(x * x, n / 2);

        }

        // 2、如果是奇数
        else {

            // 比如 n = 9，那么它就可以被划分为 4 + 4 + 1，即 x^4 * x^4 * x
            // 并且，这个时候还需要判断一下 n 是否为负数

            // 如果是正数
            if (n > 0){

                // 直接抽离出一个 x 来，然后和 myPow(x * x, n / 2) 进行相乘
                return x * myPow(x * x, n / 2);

            }

            // 如果是负数
            else{

                //测试*/
                // int a = -9;
                // int b = a / 2;
                // System.out.println(b);
                // 输出结果为 -4
                //结束测试*/
                // 比如 n = -9
            }
        }
    }
}
```

```

        // 无论正数还是负数，除 2 是向零取整。
        // 此时  $n / 2 = -4$ 
        //  $(x * x)^{-4} = x^{-8}$ 
        // 因此 myPow( $x * x$ ,  $n / 2$ ) 的结果还差一个  $x^{-1}$  才能还原原来的结果  $x^{-9}$ 
        // 由于题目要求返回 double 类型，因此这里使用 1.0
        return myPow(x * x, n / 2) * (1.0 / x);
    }
}
}
}

```

2、**C++** 代码

```

class Solution {
public:
    double myPow(double x, int n) {
        // 递归终止条件
        if (n == 0) {

            // 任何数的 0 次方的结果都是 1，由于题目要求返回 double 类型，因此返回 1.0
            return 1.0;

            // 判断 n 是奇数还是偶数
            // 1、如果是偶数
        } else if ((n & 1) == 0) {
            // 那么只需要先计算出  $y = x * x$  的结果
            // 然后 y 的次幂就是  $n / 2$ 
            return myPow(x * x, n / 2);

            // 2、如果是奇数
        } else {

            // 比如  $n = 9$ ，那么它就可以被划分为  $4 + 4 + 1$ ，即  $x^4 * x^4 * x$ 
            // 并且，这个时候还需要判断一下 n 是否为负数

            // 如果是正数
            if (n > 0) {

                // 直接抽离出一个 x 来，然后和 myPow( $x * x$ ,  $n / 2$ ) 进行相乘
                return x * myPow(x * x, n / 2);

                // 如果是负数
            } else {
                // 比如  $n = -9$ 
                // 无论正数还是负数，除 2 是向零取整。
                // 此时  $n / 2 = -4$ 
                //  $(x * x)^{-4} = x^{-8}$ 
                // 因此 myPow( $x * x$ ,  $n / 2$ ) 的结果还差一个  $x^{-1}$  才能还原原来的结果  $x^{-9}$ 
                // 由于题目要求返回 double 类型，因此这里使用 1.0
            }
        }
    }
}

```

```

        return myPow(x * x, n / 2) * (1.0 / x) ;
    }
}
};

```

3、Python 代码

```

class Solution:
    def myPow(self, x: float, n: int) -> float:

        # 递归终止条件
        if n == 0 :
            return 1.0
        elif n == -1 :
            # 任何数的 0 次方的结果都是 1 , 由于题目要求返回 double 类型, 因此返回 1.0
            return 1/x
        # 判断 n 是奇数还是偶数
        # 1、如果是偶数
        elif n & 1 == 0 :
            # 那么只需要先计算出 y = x * x 的结果
            # 然后 y 的次幂就是 n / 2
            return self.myPow(x * x, n // 2)

        # 2、如果是奇数
        else :

            # 比如 n = 9 , 那么它就可以被划分为 4 + 4 + 1, 即 x^4 * x^4 * x
            # 并且, 这个时候还需要判断一下 n 是否为负数

            # 如果是正数
            if n > 0 :

                # 直接抽离出一个 x 来, 然后和 myPow(x * x, n / 2) 进行相乘
                return x * self.myPow(x * x, n // 2)

            # 如果是负数
            else :
                # 比如 n = -9
                # Python 比较特殊, 正数负数除以 2 取整有区别
                # a = -9
                # b = a // 2
                # print(b) 输出为 -5
                # 此时 n / 2 = -5
                # (x * x)^-5 = x ^ -10
                # 因此 myPow(x * x, n / 2) 的结果还差一个 x ^ 1 才能还原原来的结果 x ^ -9
                # 由于题目要求返回 double 类型, 因此这里使用 1.0
                return self.myPow(x * x, n // 2) * x

```

LeetCode 67、二进制求和

一、题目描述

给你两个二进制字符串，返回它们的和（用二进制表示）。

输入为 非空 字符串且只包含数字 `1` 和 `0`。

示例 1:

```
输入: a = "11", b = "1"
输出: "100"
```

示例 2:

```
输入: a = "1010", b = "1011"
输出: "10101"
```

提示:

- 每个字符串仅由字符 `'0'` 或 `'1'` 组成。
- `1 <= a.length, b.length <= 10^4`
- 字符串如果不是 `"0"`，就都不含前导零。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二进制求和 (LeetCode 67) : https://leetcode.cn/problems/add-binary/
class Solution {
    public String addBinary(String a, String b) {

        // 使用字符串去拼接每一次计算的结果
        StringBuilder ans = new StringBuilder();

        // 二进制计算过程中产生的进位
        int carry = 0;

        // 开始计算
        for(int i = a.length() - 1, j = b.length() - 1; i >= 0 || j >= 0; i--, j--) {
```



```

        // 获取字符串 a 中对应的字符，如果有，那就是 a.charAt(i) - '0' 这个数字，否则设置为 0
        int numA = i >= 0 ? a.charAt(i) - '0' : 0;

        // 获取字符串 b 中对应的字符，如果有，那就是 b.charAt(j) - '0' 这个数字，否则设置为 0
        int numB = j >= 0 ? b.charAt(j) - '0' : 0;

        // 每次计算的过程都是 字符串 a 对应的字符 + 字符串 b 对应的字符 + 前面产生的进位
        // 这里采取的是【十进制】的计算规则，比如 1 + 1 = 2
        int sum = numA + numB + carry;

        // 需要把 sum 拆解为二进制的形式，比如十进制的结果 2 为二进制的 10
        // 通过 %2 的操作，可以把 0 存储到 ans 中
        ans.append(sum % 2);

        // 通过 /2 的操作，可以把 1 存储到 carry 中
        carry = sum / 2;

    }

    // 最后，需要观察最后的计算过程中是否还有进位没有使用到
    ans.append(carry == 1 ? carry : "");

    // 由于整个计算过程是从后向前计算，导致低位的计算结果存储在 ans 的前面
    // 所以需要翻转一下
    return ans.reverse().toString();
}
}

```

2、C++ 代码

```

class Solution {
public:
    string addBinary(string a, string b) {
        int carry = 0;
        string res = "";
        for (int i = a.size() - 1, j = b.size() - 1; i >= 0 || j >= 0; i--, j--)
        {
            int sum = carry;
            sum += i >= 0 ? a[i] - '0' : 0;
            sum += j >= 0 ? b[j] - '0' : 0;
            res += (sum % 2) + '0';
            carry = sum / 2;
        }
        if (carry == 1) res += '1';
        reverse(res.begin(), res.end());
        return res;
    }
};

```

3、Python 代码

```
class Solution(object):
    def addBinary(self, a, b):
        res = ""
        carry = 0
        i = len(a) - 1
        j = len(b) - 1
        while i >= 0 or j >= 0 or carry != 0:
            digitA = int(a[i]) if i >= 0 else 0
            digitB = int(b[j]) if j >= 0 else 0
            sum = digitA + digitB + carry
            carry = 1 if sum >= 2 else 0
            sum = sum - 2 if sum >= 2 else sum
            res += str(sum)
            i -= 1
            j -= 1
        return res[::-1]
```

LeetCode 7、整数反转

一、题目描述

给你一个 32 位的有符号整数 x ，返回将 x 中的数字部分反转后的结果。

如果反转后整数超过 32 位的有符号整数的范围 $[-2^{31}, 2^{31} - 1]$ ，就返回 0。

假设环境不允许存储 64 位整数（有符号或无符号）。

示例 1：

输入： $x = 123$

输出：321

示例 2：

输入： $x = -123$

输出：-321

示例 3：

输入： $x = 120$

输出：21

示例 4：

输入：x = 0

输出：0

提示：

- $-2^{31} \leq x \leq 2^{31} - 1$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 整数反转 (LeetCode 7) : https://leetcode.cn/problems/reverse-integer/
class Solution {
    public int reverse(int x) {

        // 记录反转成功之后的结果
        int res = 0;

        // 对于需要反转的数字 x
        // 每一次都按照【从后向前】的顺序去访问每个元素
        // 在访问过程中会执行一些逻辑判断
        // 判断过程中可能会由于越界直接返回 0
        // 否则会一直判断下去，直到 x 的每个元素均被访问，即 x = 0 时跳出循环
        while( x != 0 ) {

            // 每一次都先获取末尾数字
            int lastNum = x % 10;

            // 假设有 1147483649 这个数字，它是小于最大的 32 位整数 2147483647 的
            // 但是将这个数字反转过来后就变成了 9463847411，超过了最大的 32 位整数
            // 根据题目要求需要返回 0

            // 在获取到 lastNum 的过程中，需要判断一下当前的 res 是否合法
            // 如果不合法，就不用再添加 lastNum，结果直接返回 0 就行

            // 1、res 为正数并且大于了 214748364，即最小为 214748365，那么无论 lastNum 为多少，添加上去都会不合法
            // 比如 lastNum 最小为 0，形成了 2147483650，也是大于 2147483647
            // 2、res 等于了 214748364，那么在它的后面添加 8、9，就会形成 2147483648、2147483649 这两个数
            // 而这两个数都不合法，返回 0
            if (res > 214748364 || ( res == 214748364 && (lastNum == 8 || lastNum == 9
            ))) {
```

```

        // 返回 0
        return 0;

    }

    // 3、res 为负数并且小于了 -214748364 ，即最大为 214748365 ，那么无论 lastNum 为多少，添加上去都会不合法
    // 比如 lastNum 最小为 9 ，形成了 -2147483650 ，也是小于 -2147483648
    // 4、res 等于了 -214748364 ，那么在它的后面添加 9，就会形成 -2147483649 这个数
    // 而这个数不合法，返回 0
    if (res < -214748364 || (res == -214748364 && lastNum == -9 )) {
        // 返回 0
        return 0;

    }

    // 5、否则说明可以继续添加
    // res 全部元素向左移动，即 10
    res = res * 10 + lastNum;

    // 6、在去获取 x 的下一个数
    x /= 10;

}

// 返回结果
return res;
}
}

```

2、C++ 代码

```

class Solution {
public:
    int reverse(int x) {
        // 记录反转成功之后的结果
        int res = 0;

        // 对于需要反转的数字 x
        // 每一次都按照【从后向前】的顺序去访问每个元素
        // 在访问过程中会执行一些逻辑判断
        // 判断过程中可能会由于越界直接返回 0
        // 否则会一直判断下去，直到 x 的每个元素均被访问，即 x = 0 时跳出循环
        while( x != 0 ) {

            // 每一次都先获取末尾数字
            int lastNum = x % 10;

            // 假设有 1147483649 这个数字，它是小于最大的 32 位整数 2147483647 的
            // 但是将这个数字反转过来后就变成了 9463847411 ，超过了最大的 32 位整数

```

```

// 根据题目要求需要返回 0

// 在获取到 lastNum 的过程中，需要判断一下当前的 res 是否合法
// 如果不合法，就不用再添加 lastNum，结果直接返回 0 就行

// 1、res 为正数并且大于了 214748364，即最小为 214748365，那么无论 lastNum 为多少，添加上去都会不合法
// 比如 lastNum 最小为 0，形成了 2147483650，也是大于 2147483647
// 2、res 等于了 214748364，那么在它的后面添加 8、9，就会形成 2147483648、2147483649 这两个数
// 而这两个数都不合法，返回 0
if (res > 214748364 || (res == 214748364 && (lastNum == 8 || lastNum == 9))) {
    // 返回 0
    return 0;
}

// 3、res 为负数并且小于了 -214748364，即最大为 -214748365，那么无论 lastNum 为多少，添加上去都会不合法
// 比如 lastNum 最小为 9，形成了 -2147483650，也是小于 -2147483648
// 4、res 等于了 -214748364，那么在它的后面添加 9，就会形成 -2147483649 这个数
// 而这个数不合法，返回 0
if (res < -214748364 || (res == -214748364 && lastNum == -9)) {
    // 返回 0
    return 0;
}

// 5、否则说明可以继续添加
// res 全部元素向左移动，即 10
res = res * 10 + lastNum;

// 6、在去获取 x 的下一个数
x /= 10;
}

// 返回结果
return res;
}
};

```

3、Python 代码

```

class Solution:
    def reverse(self, x: int) -> int:
        # 记录反转成功之后的结果
        res = 0

```

```

# 对于需要反转的数字 x
# 每一次都按照【从后向前】的顺序去访问每个元素
# 在访问过程中会执行一些逻辑判断
# 判断过程中可能会由于越界直接返回 0
# 否则会一直判断下去，直到 x 的每个元素均被访问，即 x = 0 时跳出循环
while x != 0 :

    # 每一次都先获取末尾数字
    lastNum = x % 10

    # Python3 的取模运算在 x 为负数时也会返回 [0, 9) 以内的结果，因此这里需要进行特殊判断
    if x < 0 and lastNum > 0:
        lastNum -= 10

    # 假设有 1147483649 这个数字，它是小于最大的 32 位整数 2147483647 的
    # 但是将这个数反转过来后就变成了 9463847411，超过了最大的 32 位整数
    # 根据题目要求需要返回 0

    # 在获取到 lastNum 的过程中，需要判断一下当前的 res 是否合法
    # 如果不合法，就不用再添加 lastNum，结果直接返回 0 就行

    # 1、res 为正数并且大于了 214748364，即最小为 214748365，那么无论 lastNum 为多少，添加上去都会不合法
    # 比如 lastNum 最小为 0，形成了 2147483650，也是大于 2147483647
    # 2、res 等于了 214748364，那么在它的后面添加 8、9，就会形成 2147483648、2147483649 这两个数
    # 而这两个数都不合法，返回 0
    if res > 214748364 or ( res == 214748364 and (lastNum == 8 or lastNum == 9
)) :

        # 返回 0
        return 0

    # 3、res 为负数并且小于了 -214748364，即最大为 -214748365，那么无论 lastNum 为多少，添加上去都会不合法
    # 比如 lastNum 最小为 9，形成了 -2147483650，也是小于 -2147483648
    # 4、res 等于了 -214748364，那么在它的后面添加 9，就会形成 -2147483649 这个数
    # 而这个数不合法，返回 0
    if res < -214748364 or (res == -214748364 and lastNum == -9 ) :

        # 返回 0
        return 0

    # 5、否则说明可以继续添加
    # res 全部元素向左移动，即 10
    res = res * 10 + lastNum

```

```
# 6、在去获取 x 的下一个数
# Python3 的整数除法在 x 为负数时会向下（更小的负数）取整，因此不能写成 x // = 10
x = (x - lastNum) // 10

# 返回结果
return res
```

LeetCode 8、字符串转换整数 (atoi)

一、题目描述

请你来实现一个 `myAtoi(string s)` 函数，使其能将字符串转换成一个 32 位有符号整数（类似 C/C++ 中的 `atoi` 函数）。

函数 `myAtoi(string s)` 的算法如下：

1. 读入字符串并丢弃无用的前导空格
2. 检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。确定最终结果是负数还是正数。如果两者都不存在，则假定结果为正。
3. 读入下一个字符，直到到达下一个非数字字符或到达输入的结尾。字符串的其余部分将被忽略。
4. 将前面步骤读入的这些数字转换为整数（即，"123" -> 123，"0032" -> 32）。如果没有读入数字，则整数为 0。必要时更改符号（从步骤 2 开始）。
5. 如果整数数超过 32 位有符号整数范围 $[-2^{31}, 2^{31} - 1]$ ，需要截断这个整数，使其保持在这个范围内。具体来说，小于 -2^{31} 的整数应该被固定为 -2^{31} ，大于 $2^{31} - 1$ 的整数应该被固定为 $2^{31} - 1$ 。
6. 返回整数作为最终结果。

注意：

- 本题中的空白字符只包括空格字符 ' '。
- 除前导空格或数字后的其余字符串外，请勿忽略任何其他字符。

示例 1：

输入：s = "42"

输出：42

解释：加粗的字符串为已经读入的字符，插入符号是当前读取的字符。

第 1 步："42"（当前没有读入字符，因为没有前导空格）

^

第 2 步："42"（当前没有读入字符，因为这里不存在 '-' 或者 '+'）

^

第 3 步："42"（读入 "42"）

^

解析得到整数 42。

由于 "42" 在范围 $[-2^{31}, 2^{31} - 1]$ 内，最终结果为 42。

示例 2：

输入: `s = " -42"`

输出: `-42`

解释:

第 1 步: `" -42"` (读入前导空格, 但忽视掉)

^

第 2 步: `" -42"` (读入 `'-'` 字符, 所以结果应该是负数)

^

第 3 步: `" -42"` (读入 `"42"`)

^

解析得到整数 `-42` 。

由于 `"-42"` 在范围 `[-231, 231 - 1]` 内, 最终结果为 `-42` 。

示例 3:

输入: `s = "4193 with words"`

输出: `4193`

解释:

第 1 步: `"4193 with words"` (当前没有读入字符, 因为没有前导空格)

^

第 2 步: `"4193 with words"` (当前没有读入字符, 因为这里不存在 `'-'` 或者 `'+'`)

^

第 3 步: `"4193 with words"` (读入 `"4193"`; 由于下一个字符不是一个数字, 所以读入停止)

^

解析得到整数 `4193` 。

由于 `"4193"` 在范围 `[-231, 231 - 1]` 内, 最终结果为 `4193` 。

提示:

- `0 <= s.length <= 200`
- `s` 由英文字母 (大写和小写)、数字 (`0-9`)、`' '`、`'+'`、`'-'` 和 `'.'` 组成

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
//
class Solution {
    public int myAtoi(String str) {

        // 字符串的长度
        int n = str.length();
```



```

        // 设置有效数字的索引位置，初始化在第 0 个
int index = 0;

        // 接收最终的结果
int res = 0;

        // 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。 确定最终结果是负数还是正数。
        // 所以在扫描的过程中，需要不断的判断最终的结果是正数还是负数
        // 根据字符的正和负来判断，最后一个字符决定
        // 默认为正数，即不是负数
boolean negative = false;

        // 根据第一点要求：读入字符串并丢弃无用的前导空格
while( index < n && str.charAt(index) == ' ' ) {

        // 有效数字的索引位置不断移动
index++;

}

// 根据第四点要求：如果字符串全是空格直接返回 0
if( index == n ) {

        // 直接返回 0
return 0;

}

        // 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。 确定最终结果是负数还是正数。
        // 如果发现是负数，修改判断
if( str.charAt(index) == '-' ) {

        // 认为是负数
negative = true;

}

// 跳过正负号符号位
if( str.charAt(index) == '-' || str.charAt(index) == '+' ) {

        // 有效数字的索引位置不断移动
index++;

}

```

```

// 开始获取数字，一个数字一个数字去获取
// 由于在字符串中可能存在数字之间夹杂的其它字符，所以出现这种情况需要截断
while( index < n && str.charAt(index) >= '0' && str.charAt(index) <= '9' ) {

    // '0'的 ASCII 码是48, '1' 的是 49
    // 获取数字，相当于字符串转整数操作
    int lastNum = str.charAt(index) - 48 ;

    // 根据第五点要求：需要判断数字是否合理
    // 先判断正数情况，大于 2147483647 的整数应该被固定为 2147483647
    // 即如何发现之前积累的数字都大于了 214748364，比如最小为 214748365
    // 那么无论 lastNum 为多少，添加到尾部都会大于 214748364
    if( !negative && ( res > 214748364 || ( res == 214748364 && (lastNum == 8 || lastNum
== 9 ) ) ) ) {

        // 应该被固定为 2147483647
        return 2147483647;

    }

    // 根据第五点要求：需要判断数字是否合理
    // 再判断负数情况，小于 -2147483648 的整数应该被固定为 -2147483648
    if( negative && ( -res < -214748364 || ( -res == -214748364 && (lastNum == 8 ||
lastNum == 9 ) ) ) ) {

        // 应该被固定为 -2147483648
        return -2147483648;

    }

    // 否则说明可以添加上去
    res = res * 10 + lastNum;

    // 有效数字的索引位置不断移动
    index++;

}

// 返回整数作为最终结果。
return negative ? - res : res;
}
}

```

2、C++ 代码

```

class Solution {
public:
    int myAtoi(string s) {
        // 字符串的长度
        int n = s.length();
    }
}

```

```
// 设置有效数字的索引位置，初始化在第 0 个
```

```
int index = 0;
```

```
// 接收最终的结果
```

```
int res = 0;
```

```
// 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。确定最终结果是负数还是正数。
```

```
// 所以在扫描的过程中，需要不断的判断最终的结果是正数还是负数
```

```
// 根据字符的正和负来判断，最后一个字符决定
```

```
// 默认为正数，即不是负数
```

```
bool negative = false;
```

```
// 根据第一点要求：读入字符串并丢弃无用的前导空格
```

```
while( index < n && s[index] == ' ') {
```

```
    // 有效数字的索引位置不断移动
```

```
    index++;
```

```
}
```

```
// 根据第四点要求：如果字符串全是空格直接返回 0
```

```
if( index == n) {
```

```
    // 直接返回 0
```

```
    return 0;
```

```
}
```

```
// 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。确定最终结果是负数还是正数。
```

```
// 如果发现是负数，修改判断
```

```
if( s[index] == '-' ) {
```

```
    // 认为是负数
```

```
    negative = true;
```

```
}
```

```
// 跳过正负号符号位
```

```
if( s[index] == '-' || s[index] == '+' ) {
```

```
    // 有效数字的索引位置不断移动
```

```
    index++;
```

```
}
```

```
// 开始获取数字，一个数字一个数字去获取
```

```

// 由于在字符串中可能存在数字之间夹杂的其它字符，所以出现这种情况需要截断
while( index < n && s[index] >= '0' && s[index] <= '9' ) {

    // '0'的 ASCII 码是48, '1' 的是 49
    // 获取数字，相当于字符串转整数操作
    int lastNum = s[index] - 48 ;

    // 根据第五点要求：需要判断数字是否合理
    // 先判断正数情况，大于 2147483647 的整数应该被固定为 2147483647
    // 即如何发现之前积累的数字都大于了 214748364，比如最小为 214748365
    // 那么无论 lastNum 为多少，添加到尾部都会大于 214748364
    if( !negative && ( res > 214748364 || ( res == 214748364 && (lastNum == 8
|| lastNum == 9 ) ) ) ) {

        // 应该被固定为 2147483647
        return 2147483647;

    }

    // 根据第五点要求：需要判断数字是否合理
    // 再判断负数情况，小于 -2147483648 的整数应该被固定为 -2147483648
    if( negative && ( -res < -214748364 || ( -res == -214748364 && (lastNum ==
8 || lastNum == 9 ) ) ) ) {

        // 应该被固定为 -2147483648
        return -2147483648;

    }

    // 否则说明可以添加上去
    res = res * 10 + lastNum;

    // 有效数字的索引位置不断移动
    index++;

}

// 返回整数作为最终结果。
return negative ? - res : res;

}

};

```

3、Python 代码

```

class Solution:
    def myAtoi(self, s: str) -> int:
        # 字符串的长度
        n = len(s)

        # 设置有效数字的索引位置，初始化在第 0 个

```

```
index = 0
```

```
# 接收最终的结果
```

```
res = 0
```

```
# 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。确定最终结果是负数还是正数。
```

```
# 所以在扫描的过程中，需要不断的判断最终的结果是正数还是负数
```

```
# 根据字符的正和负来判断，最后一个字符决定
```

```
# 默认为正数，即不是负数
```

```
negative = False
```

```
# 根据第一点要求：读入字符串并丢弃无用的前导空格
```

```
while index < n and s[index] == ' ' :
```

```
    # 有效数字的索引位置不断移动
```

```
    index += 1
```

```
# 根据第四点要求：如果字符串全是空格直接返回 0
```

```
if index == n :
```

```
    # 直接返回 0
```

```
    return 0
```

```
# 根据第二点要求：检查下一个字符（假设还未到字符末尾）为正还是负号，读取该字符（如果有）。确定最终结果是负数还是正数。
```

```
# 如果发现是负数，修改判断
```

```
if s[index] == '-' :
```

```
    # 认为是负数
```

```
    negative = True
```

```
# 跳过正负号符号位
```

```
if s[index] == '-' or s[index] == '+' :
```

```
    # 有效数字的索引位置不断移动
```

```
    index += 1
```

```
# 开始获取数字，一个数字一个数字去获取
```

```
# 由于在字符串中可能存在数字之间夹杂的其它字符，所以出现这种情况需要截断
```

```
while index < n and s[index] >= '0' and s[index] <= '9' :
```

```
    # '0' 的 ASCII 码是48, '1' 的是 49
```

```

# 获取数字，相当于字符串转整数操作
lastNum = ord(s[index]) - 48

# 根据第五点要求：需要判断数字是否合理
# 先判断正数情况，大于 2147483647 的整数应该被固定为 2147483647
# 即如何发现之前积累的数字都大于了 214748364，比如最小为 214748365
# 那么无论 lastNum 为多少，添加到尾部都会大于 214748364
if not negative and ( res > 214748364 or ( res == 214748364 and (lastNum ==
8 or lastNum == 9 )) ) :

    # 应该被固定为 2147483647
    return 2147483647

# 根据第五点要求：需要判断数字是否合理
# 再判断负数情况，小于 -2147483648 的整数应该被固定为 -2147483648
if negative and ( -res < -214748364 or ( -res == -214748364 and (lastNum ==
8 or lastNum == 9 ) ) ) :
    # 应该被固定为 -2147483648
    return -2147483648

# 否则说明可以添加上去
res = res * 10 + lastNum

# 有效数字的索引位置不断移动
index += 1

# 返回整数作为最终结果。
return -res if negative else res

```

第二十四天

LeetCode 153、寻找旋转排序数组中的最小值

一、题目描述

已知一个长度为 n 的数组，预先按照升序排列，经由 1 到 n 次 **旋转** 后，得到输入数组。例如，原数组 `nums = [0,1,2,4,5,6,7]` 在变化后可能得到：

- 若旋转 4 次，则可以得到 `[4,5,6,7,0,1,2]`
- 若旋转 7 次，则可以得到 `[0,1,2,4,5,6,7]`

注意，数组 `[a[0], a[1], a[2], ..., a[n-1]]` 旋转一次的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`。

给你一个元素值 **互不相同** 的数组 `nums`，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的 **最小元素**。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1:

输入: `nums = [3,4,5,1,2]` 输出: 1 解释: 原数组为 `[1,2,3,4,5]`，旋转 3 次得到输入数组。

示例 2:

输入: `nums = [4,5,6,7,0,1,2]` 输出: 0 解释: 原数组为 `[0,1,2,4,5,6,7]`，旋转 4 次得到输入数组。

示例 3:

输入: `nums = [11,13,15,17]` 输出: 11 解释: 原数组为 `[11,13,15,17]`，旋转 4 次得到输入数组。

提示:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- `nums` 中的所有整数 **互不相同**
- `nums` 原来是一个升序排序的数组，并进行了 `1` 至 `n` 次旋转

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找旋转排序数组中的最小值（LeetCode 153）：https://leetcode.cn/problems/find-minimum-in-rotated-sorted-array/
class Solution {
    public int findMin(int[] nums) {

        // 搜索区间的左边界
        int left = 0;

        // 搜索区间的右边界
        int right = nums.length - 1;

        // 在搜索区间 [ left , right ] 中去搜索
        // 在 while 里面就可以获取到结果
        while ( left < right ) {
```

```

// 先去获取搜索区间的中间位置元素
int mid = left + (right - left) / 2;

// 如果 nums[mid] < nums[right]
// 最小值一定在 mid 左侧区间或者就是 nums[mid]
if (nums[mid] < nums[right]) {

    // right 移到 mid 的位置。
    // 疑问: 为什么 right = mid ; 而不是 right = mid - 1;
    // 解答: {4,5,1,2,3}, 如果 right = mid - 1 , 则丢失了最小值 1
    right = mid;

    // 由于 nums 里面的元素都互不相同
    // 所以, 如果 nums[mid] > nums[right]
    // 最小值一定在 mid 右侧侧区间
} else {

    // left 移到 mid + 1 的位置。
    // 疑问: 为什么 left = mid + 1 ; 而不是 left = mid;
    // 解答: {4,5,6,1,2,3}, nums[mid] = 6, left = mid + 1 , 刚好 nums[left] = 1
    left = mid + 1;
}
}

// 返回结果
return nums[right];
}
}

```

LeetCode 154、寻找旋转排序数组中的最小值 II

一、题目描述

已知一个长度为 `n` 的数组，预先按照升序排列，经由 `1` 到 `n` 次 **旋转** 后，得到输入数组。例如，原数组 `nums = [0,1,4,4,5,6,7]` 在变化后可能得到：

- 若旋转 `4` 次，则可以得到 `[4,5,6,7,0,1,4]`
- 若旋转 `7` 次，则可以得到 `[0,1,4,4,5,6,7]`

注意，数组 `[a[0], a[1], a[2], ..., a[n-1]]` **旋转一次** 的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`。

给你一个可能存在 **重复** 元素值的数组 `nums`，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的 **最小元素**。

你必须尽可能减少整个过程的操作步骤。

示例 1：

输入: nums = [1,3,5] 输出: 1

示例 2:

输入: nums = [2,2,2,0,1] 输出: 0

提示:

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- `nums` 原来是一个升序排序的数组, 并进行了 `1` 至 `n` 次旋转

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找旋转排序数组中的最小值 II (LeetCode 154) : https://leetcode.cn/problems/find-minimum-in-rotated-sorted-array-ii/
class Solution {
    public int findMin(int[] nums) {
        // 搜索区间的左边界
        int left = 0;

        // 搜索区间的右边界
        int right = nums.length - 1;

        // 在搜索区间 [ left , right ] 中去搜索
        // 在 while 里面就可以获取到结果
        while (left < right) {

            // 先去获取搜索区间的中间位置元素
            int mid = left + (right - left) / 2;

            // 如果 nums[mid] < nums[right]
            // 最小值一定在 mid 左侧区间或者就是 nums[mid]
            if (nums[mid] < nums[right]) {

                // right 移到 mid 的位置。
                right = mid ;

            }

            // 由于 nums 里面的元素存在相同的情况
            // 所以, 如果 nums[mid] > nums[right]
```

```

        // 最小值一定在 mid 右侧侧区间
    } else if(nums[mid] > nums[right]) {

        // left 移到 mid + 1 的位置。
        left = mid + 1;

        // 如果 nums[mid] = nums[right]
    }else{

        // right 按顺序遍历一样，一个个向左移动
        // 为什么不能让 right = mid 呢?
        // 给一个测试用例 [3,3,1,3]
        // nums[left] = 3 、 nums[right] = 3 、 nums[mid] = 3
        // 如果让 right = mid, 跳过了 1
        right--;

    }

}

// 返回结果
return nums[left];
}
}

```

LeetCode 189、轮转数组

一、题目描述

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

输入: `nums = [1,2,3,4,5,6,7]`, `k = 3` 输出: `[5,6,7,1,2,3,4]` 解释: 向右轮转 1 步: `[7,1,2,3,4,5,6]` 向右轮转 2 步: `[6,7,1,2,3,4,5]` 向右轮转 3 步: `[5,6,7,1,2,3,4]`

示例 2:

输入: `nums = [-1,-100,3,99]`, `k = 2` 输出: `[3,99,-1,-100]` 解释: 向右轮转 1 步: `[99,-1,-100,3]` 向右轮转 2 步: `[3,99,-1,-100]`

提示:

- `1 <= nums.length <= 10^5`
- `-2^31 <= nums[i] <= 2^31 - 1`
- `0 <= k <= 10^5`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// LeetCode 189、轮转数组：https://leetcode.cn/problems/rotate-array/
class Solution {
    public void rotate(int[] nums, int k) {

        // 1、对 k 进行取余操作
        k %= nums.length;

        // 第一步，翻转整个数组
        reverse(nums, 0, nums.length - 1);
        // 第二步，翻转前 k 个元素
        reverse(nums, 0, k - 1);
        // 第三步，翻转后面的所有元素
        reverse(nums, k, nums.length - 1);
    }

    // 翻转区间 [ start , end ] 里面的元素
    public void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start += 1;
            end -= 1;
        }
    }
}
```

LeetCode 33、搜索旋转排序数组

一、题目描述

整数数组 `nums` 按升序排列，数组中的值 **互不相同**。

在传递给函数之前，`nums` 在预先未知的某个下标 `k` ($0 \leq k < \text{nums.length}$) 上进行了 **旋转**，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`（下标从 0 开始计数）。例如，`[0,1,2,4,5,6,7]` 在下标 3 处经旋转后可能变为 `[4,5,6,7,0,1,2]`。

给你 **旋转后** 的数组 `nums` 和一个整数 `target`，如果 `nums` 中存在这个目标值 `target`，则返回它的下标，否则返回 `-1`。

示例 1:

输入: `nums = [4,5,6,7,0,1,2]`, `target = 0`
输出: `4`

示例 2:

输入: `nums = [4,5,6,7,0,1,2]`, `target = 3`
输出: `-1`

示例 3:

输入: `nums = [1]`, `target = 0`
输出: `-1`

提示:

- `1 <= nums.length <= 5000``
- `-10^4 <= nums[i] <= 10^4`
- `nums` 中的每个值都 **独一无二**
- 题目数据保证 `nums` 在预先未知的某个下标上进行了旋转
- `-10^4 <= target <= 10^4`

进阶：你可以设计一个时间复杂度为 `O(log n)` 的解决方案吗？

二、题目解析

三、参考代码

```
class solution {
    public int search(int[] nums, int target) {

        // left 指向当前区间的最左边位置，所以初始化为 0
        int left = 0;

        // right 指向当前区间的最右边位置，所以初始化为 nums.length - 1
        int right = nums.length - 1;

        // 循环进行二分查找，直到左端点位置超过了右端点
        // 或者在循环过程中找到了 target
        while( left <= right){

            // 计算当前区间的中间位置，向下取整
            int mid = (left + right) / 2;
```

```

// 如果中间位置数字 nums[mid] 等于目标值 target, 那么说明找到了
// 返回当前的下标 mid
if(nums[mid] == target) return mid;

// 否则的话需要先确定 mid 的左边还是右边为有序区间

// 如果当前区间最左端的值 nums[left] 小于等于 nums[mid]
// 说明从 left 到 mid 这段区间是递增的, 为有序区间
// 即 mid 的左侧为有序区间, 右侧为无序区间
if(nums[left] <= nums[mid]){

    // 先去判断 target 是否在左侧有序区间内
    // 如果目标值 target 大于这段有序区间的最小值 nums[left] 同时小于这段有序区间的最大
    大值 nums[mid]
    // 那么说明需要在这段有序区间去寻找 target
    if(target >= nums[left] && target <= nums[mid]){

        // 所以缩小范围为 left 到 mid - 1
        // 当前区间的左侧为 left, 右侧 right = mid - 1
        right = mid - 1;

        // 否则说明需要在 mid 的右侧无序区间搜索
    }else{

        // 所以缩小范围为 mid + 1 到 right
        // 当前区间的左侧为 left = mid + 1, 右侧 right
        left = mid + 1;
    }

    // 否则说明当前区间最左端的值 nums[left] 大于 nums[mid]
    // 说明从 left 到 mid 这段区间是无序区间
    // 即 mid 的左侧为无序区间, 右侧为有序区间
}else{

    // 先去判断 target 是否在右侧有序区间内
    // 如果目标值 target 大于这段有序区间的最小值 nums[mid] 同时小于这段有序区间的最大
    大值 nums[right]
    // 那么说明需要在这段有序区间去寻找 target
    if(target >= nums[mid] && target <= nums[right]){

        // 所以缩小范围为 mid + 1 到 right
        // 当前区间的左侧为 left = mid + 1, 右侧 right
        left = mid + 1;

        // 否则说明需要在 mid 的左侧无序区间搜索
    }else{

        // 所以缩小范围为 left 到 mid - 1
        // 当前区间的左侧为 left, 右侧 right = mid - 1
    }
}

```

```
        right = mid - 1;

    }

}

// 目标值不存在, 返回 -1
return - 1;

}

}
```

LeetCode 34、在排序数组中查找元素的第一个和最后一个位置

一、题目描述

给定一个按照升序排列的整数数组 `nums`，和一个目标值 `target`。找出给定目标值在数组中的开始位置和结束位置。

如果数组中不存在目标值 `target`，返回 `[-1, -1]`。

进阶：

- 你可以设计并实现时间复杂度为 $O(\log n)$ 的算法解决此问题吗？

示例 1：

输入：nums = [5,7,7,8,8,10], target = 8
输出：[3,4]

示例 2：

输入：nums = [5,7,7,8,8,10], target = 6
输出：[-1,-1]

示例 3：

输入：nums = [], target = 0
输出：[-1,-1]

提示：

- $0 \leq \text{nums.length} \leq 105$
- $-10(9) \leq \text{nums}[i] \leq 10(9)$
- nums 是一个非递减数组
- $-10(9) \leq \text{target} \leq 10(9)$

二、题目解析

三、参考代码

```
class Solution {
    public int[] searchRange(int[] nums, int target) {

        // 寻找目标值在数组中的开始位置
        int firstIdx = findBeginPostion(nums,target);

        // 寻找目标值在数组中的结束位置
        int lastIdx = findEndPostion(nums,target);

        // 返回寻找的结果
        return new int[]{firstIdx,lastIdx};

    }

    // 寻找目标值在数组中的开始位置
    private int findBeginPostion(int[] nums , int target){

        // left 指向当前区间的最左边位置，所以初始化为 0
        int left = 0;

        // right 指向当前区间的最右边位置，所以初始化为 nums.length - 1
        int right = nums.length - 1;

        // 循环进行二分查找，直到左端点位置超过了右端点
        // 或者在循环过程中找到了起始位置
        while(left <= right){

            // 计算当前区间的中间位置，向下取整
            int mid = (left + right) / 2;

            // 如果中间位置的元素值等于目标值 target
            if(nums[mid] == target){

                // 并且中间位置 mid 的左边没有元素，即中间位置 mid 为当前区间的起始位置
                // 或者中间位置 mid 的前一个元素小于目标值 target
                // 说明 mid 指向了 target 的起始位置
                if(mid == 0 || nums[mid - 1] < target){
                    // mid 指向了 target 的起始位置，返回这个结果
                    return mid;
                }

                // 否则，说明 mid 的左边依然有元素值等于 target
                // 那么 mid 就不是 target 的起始位置，需要在 mid 的左边进行查找
                // 所以缩小范围为 left 到 mid - 1
                // 当前区间的左侧为 left，右侧 right = mid - 1
            }
        }
    }
}
```

```

        right = mid - 1;

        // 如果中间位置的元素值大于目标值 target
        // 说明需要在 mid 的左边进行查找
    }else if( nums[mid] > target){

        // 所以缩小范围为 left 到 mid - 1
        // 当前区间的左侧为 left, 右侧 right = mid - 1
        right = mid - 1;

        // 如果中间位置的元素值小于目标值 target
        // 说明需要在 mid 的右边进行查找
    }else{

        // 所以缩小范围为 mid + 1 到 right
        // 当前区间的左侧为 left = mid + 1, 右侧 right
        left = mid + 1;

    }
}

// 如果循环结束后还没有返回, 说明找不到目标值 target, 返回 -1
return - 1;
}

// 寻找目标值在数组中的结束位置
private int findEndPostion(int[] nums , int target){

    // left 指向当前区间的最左边位置, 所以初始化为 0
    int left = 0;

    // right 指向当前区间的最右边位置, 所以初始化为 nums.length - 1
    int right = nums.length - 1;

    // 循环进行二分查找, 直到左端点位置超过了右端点
    // 或者在循环过程中找到了结束位置
    while(left <= right){

        // 计算当前区间的中间位置, 向下取整
        int mid = (left + right) / 2;

        // 如果中间位置的元素值等于目标值 target
        if(nums[mid] == target){
            // 并且中间位置 mid 的右边没有元素, 即中间位置 mid 为当前区间的结束位置
            // 或者中间位置 mid 的后一个元素大于目标值 target
            // 说明 mid 指向了 target 的结束位置
            if(mid == nums.length - 1 || nums[mid + 1] > target){
                // mid 指向了 target 的结束位置, 返回这个结果
                return mid;
            }
        }
    }
}

```



```

    }

    // 否则, 说明 mid 的右边依然有元素值等于 target
    // 那么 mid 就不是 target 的结束位置, 需要在 mid 的右边进行查找
    // 所以缩小范围为 mid + 1 到 right
    // 当前区间的左侧为 left = mid + 1, 右侧为 right
    left = mid + 1;

    // 如果中间位置的元素值大于目标值 target
    // 说明需要在 mid 的左边进行查找
} else if( nums[mid] > target){

    // 所以缩小范围为 left 到 mid - 1
    // 当前区间的左侧为 left, 右侧 right = mid - 1
    right = mid - 1;

    // 如果中间位置的元素值小于目标值 target
    // 说明需要在 mid 的右边进行查找
} else{

    // 所以缩小范围为 mid + 1 到 right
    // 当前区间的左侧为 left = mid + 1, 右侧 right
    left = mid + 1;

}

}

// 如果循环结束后还没有返回, 说明找不到目标值 target, 返回 -1
return - 1;
}

}

```

LeetCode 35、搜索插入位置

一、题目描述

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。

请必须使用时间复杂度为 $O(\log n)$ 的算法。

示例 1:

输入: nums = [1,3,5,6], target = 5
输出: 2

示例 2:

输入: nums = [1,3,5,6], target = 2
输出: 1

示例 3:

输入: nums = [1,3,5,6], target = 7
输出: 4

示例 4:

输入: nums = [1,3,5,6], target = 0
输出: 0

示例 5:

输入: nums = [1], target = 0
输出: 0

提示:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` 为无重复元素的升序排列数组
- $-10^4 \leq \text{target} \leq 10^4$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 搜索插入位置(35):https://leetcode-cn.com/problems/search-insert-position/
class Solution {
    public int searchInsert(int[] nums, int target) {
        // left 为当前区间最左侧的元素, 可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素, 可以获取到
        int right = nums.length - 1;

        // 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
        // 在区间 nums[left..right] 里查找第 1 个大于等于 target 的元素的下标
        while(left <= right) {
```

```

// left + (right - left) / 2 和 (left + right) / 2 的结果相同
// 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
的结果溢出的问题
// 比如数据 int 的最大值为 2,147,483,647
// 此时，left 为 2,147,483,640
// 此时，right 为 2,147,483,646
// 两者直接相加超过了 2,147,483,647，产生了溢出
int mid = left + (right - left) / 2;

// 中间的元素和目标值 target 相等，直接返回下标即可，不需要插入
if(nums[mid] == target) {
    // 直接返回下标即可
    return mid;

// 中间的元素大于了目标值 target
// 1、数组为有序数组，比如为递增数组
// 2、数组中不存在重复元素
// 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素都大于目标值
target

// 因此，缩小查找区间为 [ left , mid - 1]
} else if(nums[mid] > target) {
    // 也就是说缩小之后的区间最右侧位置为 mid - 1
    right = mid - 1;

// 中间的元素小于了目标值 target
// 1、数组为有序数组，比如为递增数组
// 2、数组中不存在重复元素
// 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素都小于目标值
target

// 因此，缩小查找区间为 [ mid + 1 , right]
} else if(nums[mid] < target){
    // 也就是说缩小之后的区间最左侧位置为 mid + 1
    left = mid + 1;
}
}

// 执行到这里，说明退出了 while 循环，在 nums 里面没有找到和目标值 target 相同的元素，需要
将 target 插入到 nums 中
// 在上面 while 循环每次的操作中，都会使得 [ left , right ] 的这个区间中的元素减少，直到
left = right + 1 出现区间不存在元素位置
// 在出现这种情况之前，也就是 while 循环操作的最后一次时，left = right，那么此时计算的 mid
= left = right
// 此时 nums[mid] 左边的数全部小于目标值 target ，nums[mid] 右边的数全部大于目标值
target

// 1、如果 nums[mid] 大于 target，那么接下来 right 会向左移动，即 right = left - 1，此
时搜索区间不存在
// 那么说明 target 应该插入到 nums[mid] 的前一个位置，也就是顶替了 nums[mid] 的位置，
nums[mid] 后面所有元素都向后移动
// 而此时 left 指向的就是 nums[mid] 这个元素，所以 left 就是插入的位置

```

```

        // 2、如果 nums[mid] 小于 target，那么接下来 left 会向右移动，即 left = right + 1，此时搜索区间不存在
        // 那么说明 target 应该插入到 nums[mid] 的后一个位置，而 left 向后移动了一次，这个位置就是插入位置
        return left;
    }
}

```

2、C++ 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 搜索插入位置(35):https://leetcode-cn.com/problems/search-insert-position/
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        // left 为当前区间最左侧的元素，可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素，可以获取到
        int right = nums.size() - 1;

        // 在 while 循环里面，left 不断的 ++，而 right 不断的 --
        // 在区间 nums[left..right] 里查找第 1 个大于等于 target 的元素的下标
        while(left <= right) {

            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left、right 同时太大，导致相加的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时，left 为 2,147,483,640
            // 此时，right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647，产生了溢出
            int mid = left + (right - left) / 2;

            // 中间的元素和目标值 target 相等，直接返回下标即可，不需要插入
            if(nums[mid] == target) {
                // 直接返回下标即可
                return mid;
            }

            // 中间的元素大于了目标值 target
            // 1、数组为有序数组，比如为递增数组
            // 2、数组中不存在重复元素
            // 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素都大于目标值 target

            // 因此，缩小查找区间为 [ left , mid - 1]

```

```

    } else if(nums[mid] > target) {
        // 也就是说缩小之后的区间最右侧位置为 mid - 1
        right = mid - 1;

        // 中间的元素小于了目标值 target
        // 1、数组为有序数组，比如为递增数组
        // 2、数组中不存在重复元素
        // 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素都小于目标值
target
        // 因此，缩小查找区间为 [ mid + 1 , right]
    } else if(nums[mid] < target){
        // 也就是说缩小之后的区间最左侧位置为 mid + 1
        left = mid + 1;
    }
}

// 执行到这里，说明退出了 while 循环，在 nums 里面没有找到和目标值 target 相同的元素，需要
将 target 插入到 nums 中
// 在上面 while 循环每次的操作中，都会使得 [ left , right ] 的这个区间中的元素减少，直到
left = right + 1 出现区间不存在元素位置
// 在出现这种情况之前，也就是 while 循环操作的最后一次时，left = right，那么此时计算的 mid
= left = right
// 此时 nums[mid] 左边的数全部小于目标值 target ， nums[mid] 右边的数全部大于目标值
target
// 1、如果 nums[mid] 大于 target，那么接下来 right 会向左移动，即 right = left - 1，此
时搜索区间不存在
// 那么说明 target 应该插入到 nums[mid] 的前一个位置，也就是顶替了 nums[mid] 的位置，
nums[mid] 后面所有元素都向后移动
// 而此时 left 指向的就是 nums[mid] 这个元素，所以 left 就是插入的位置
// 2、如果 nums[mid] 小于 target，那么接下来 left 会向右移动，即 left = right + 1，此
时搜索区间不存在
// 那么说明 target 应该插入到 nums[mid] 的后一个位置，而 left 向后移动了一次，这个位置就
是插入位置
return left;

}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二分查找(704):https://leetcode-cn.com/problems/binary-search/
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:

        # left 为当前区间最左侧的元素，可以获取到

```

```

left = 0

# right 为当前区间最右侧的元素，可以获取到
right = len(nums) - 1

# 在 while 循环里面，left 不断的 ++，而 right 不断的 --
# 当 [ left , right ] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left >
right 时跳出循环
# 即当 left = right + 1 时，搜索区间没有元素了
# 由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [ left , right] 区间，这个
区间就一个元素
# 所以此时 while 循环的判断可以使用等号
while left <= right :

    # left + (right - left) / 2 和 (left + right) / 2 的结果相同
    # 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
    的结果溢出的问题
    # 比如数据 int 的最大值为 2,147,483,647
    # 此时，left 为 2,147,483,640
    # 此时，right 为 2,147,483,646
    # 两者直接相加超过了 2,147,483,647，产生了溢出
    mid = left + (right - left) // 2

    # 中间的元素和目标值 target 相等，直接返回下标即可
    if nums[mid] == target :
        # 直接返回下标即可
        return mid

    # 中间的元素大于了目标值 target
    # 1、数组为有序数组，比如为递增数组
    # 2、数组中不存在重复元素
    # 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素都大于目标值
    target

    # 因此，缩小查找区间为 [ left , mid - 1]
    elif nums[mid] > target :
        # 也就是说缩小之后的区间最右侧位置为 mid - 1
        right = mid - 1

    # 中间的元素小于了目标值 target
    # 1、数组为有序数组，比如为递增数组
    # 2、数组中不存在重复元素
    # 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素都小于目标值
    target

    # 因此，缩小查找区间为 [ mid + 1 , right]
    elif nums[mid] < target :
        # 也就是说缩小之后的区间最左侧位置为 mid + 1
        left = mid + 1

```

```
        # 执行到这里，说明退出了 while 循环，在 nums 里面没有找到和目标值 target 相同的元素，需要
        将 target 插入到 nums 中
        # 在上面 while 循环每次的操作中，都会使得 [ left , right ] 的这个区间中的元素减少，直到
        left = right + 1 出现区间不存在元素位置
        # 在出现这种情况之前，也就是 while 循环操作的最后一次时，left = right，那么此时计算的 mid
        = left = right
        # 此时 nums[mid] 左边的数全部小于目标值 target ， nums[mid] 右边的数全部大于目标值
        target
        # 1、如果 nums[mid] 大于 target，那么接下来 right 会向左移动，即 right = left - 1，此
        时搜索区间不存在
        # 那么说明 target 应该插入到 nums[mid] 的前一个位置，也就是顶替了 nums[mid] 的位置，
        nums[mid] 后面所有元素都向后移动
        # 而此时 left 指向的就是 nums[mid] 这个元素，所以 left 就是插入的位置
        # 2、如果 nums[mid] 小于 target，那么接下来 left 会向右移动，即 left = right + 1，此时
        搜索区间不存在
        # 那么说明 target 应该插入到 nums[mid] 的后一个位置，而 left 向后移动了一次，这个位置就是
        插入位置
        return left
```

四、复杂度分析

时间复杂度： $O(\log n)$ ，其中 n 是给定版本的数量。

空间复杂度： $O(1)$ 。我们只需要常数的空间保存若干变量。

LeetCode 410、分割数组的最大值

一、题目描述

给定一个非负整数数组 `nums` 和一个整数 `m`，你需要将这个数组分成 `m` 个非空的连续子数组。

设计一个算法使得这 `m` 个子数组各自和的最大值最小。

示例 1：

输入：`nums = [7,2,5,10,8]`，`m = 2` 输出：18 解释：一共有四种方法将 `nums` 分割为 2 个子数组。其中最好的方式是将其为 `[7,2,5]` 和 `[10,8]`。因为此时这两个子数组各自的和的最大值为18，在所有情况中最小。

示例 2：

输入：`nums = [1,2,3,4,5]`，`m = 2` 输出：9

示例 3：

输入：`nums = [1,4,4]`，`m = 3` 输出：4

提示：

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 106`
- `1 <= m <= min(50, nums.length)`

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 分割数组的最大值 (LeetCode 410) : https://leetcode.cn/problems/split-array-largest-sum/
class Solution {
    public int splitArray(int[] nums, int m) {
        // m 表示将数组 nums 划分为 m 个子数组
        // 对于数组 nums 来说
        // 即可以划分为 1 个完整的子数组，子数组包含 nums 里面所有的元素
        // 也可以划分为 n 个子数组，每个子数组只包含 1 个元素
        // 也就是说，m 的取值范围为  $1 \leq m \leq n$ 
        // 假设 nums = [ 7 , 2 , 5 , 10 , 8 ]
        // 1、如果 m = 1，那么整个数组作为一部分，只有一种划法，最小的最大值为 32
        // 2、如果 m = n，那么每个元素作为一个子数组，只有一种划法，从所有元素选取最大值，最小的最大值为 10
        // 因此，最大值的最小值的范围为 [10, 32]
        // 这里说明一下最大值和最小值的概念
        // 最大值：在一个确定的划分之后，有很多个子数组，从所有子数组和里面挑选出一个最大值来
        // 最小值：在 m 等于某个数字时，有很多种划分的方法，每一种划法都能有最大值，比较这些最大值，选出最小的来

        // left 记录子数组和最大值下界
        int left = 0;

        // right记录子数组和最大值上界
        int right = 0;

        for(int num : nums){

            // 子数组和最大值下界为数组中元素的最大值，这个时候每个元素表示 1 个子数组
            left = Math.max(left, num);

            // 子数组和最大值上界为数组中所有元素和，这个时候只有 1 个子数组
            right += num;
        }

        // 在区间 [ left , right ] 中寻找出一个合理的值
        // 这个值是使得这 m 个子数组各自和的最大值最小
        while(left <= right){

            // 猜测中间值为答案
            int mid = left + (right - left) / 2;
```



```

// 按照这种方案把数组进行划分
// 得到的子区间数 cnt
int cnt = subSplit( nums , mid );

// 开始分析这种划分是否合理
// 1、划分的子数组个数多了
if(cnt > m){

    // 为什么会划分的子数组个数多了
    // 因为限定的子数组的元素和值小，导致只能不断地去划分子数组
    // 为了可以少划分一些，需要提高子数组的元素和值
    left = mid + 1;

// 2、划分的子数组个数少了
}else if(cnt < m){

    // 为什么会划分的子数组个数少了
    // 因为限定的子数组的元素和值大，导致每个子数组可以装很多元素
    // 为了可以多划分一些，需要减小子数组的元素和值
    right = mid - 1;

// 3、划分的子数组个数刚刚好
}else if(cnt == m){
    // 此时，找到了满足分割数 m 的子数组最大值
    // 在此基础上继续在左区间寻找
    // 这样，才能使得满足分割数且最大值最小
    right = mid - 1;
}
}

// while 终止条件是 left > right
// 最后一次跳出 while 之前，left 会等于 right
// 1、如果此时，在这种情况下进行切分的结果刚好等于分割数 m
// 那么为了获取更小的值，right 会向左移动，从而导致 left > right
// 于是，left 就是那个满足分割数为 m 情况下最大值最小的数
// 2、如果此时，在这种情况下进行切分的结果不等于分割数 m
// left 会向右移动，从而导致 left > right
// 最终 left 也恰好等于上一次分割数为 m 的 mid 值
// 结合视频里面的动画进行理解
return left;
}

// 根据子数组和 maxSum 划分数组，返回子数组个数
private int subSplit(int[] nums, int maxSum){

    // 默认为划分为一个子数组，即就是 nums 数组

```

```

    int cnt = 1;

    // 当前子数组的元素和
    int tmpArrSum = 0;

    // 对每个元素进行考虑它是否有资格划分进行
    for(int num : nums){

        // 如果发现把当前元素和归纳到目前这个数组后
        // 导致子数组元素和大于了 maxSum
        if(tmpArrSum + num > maxSum){

            // 说明，这个元素需要开始另外分配一个子数组了
            // 以该元素为界限开始进行划分
            cnt++;

            // 当前子数组的元素和归零
            tmpArrSum = 0;
        }

        // 当前子数组的元素和进行累加
        tmpArrSum += num;
    }

    // 返回子数组的个数
    // 这些子数组每个子数组元素和的最大值都不会超过 maxSum
    return cnt;
}
}

```

Python

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 微信：wzb_3377
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 分割数组的最大值（LeetCode 410）：https://leetcode.cn/problems/split-array-largest-sum/
class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:

        # m 表示将数组 nums 划分为 m 个子数组
        # 对于数组 nums 来说
        # 即可以划分为 1 个完整的子数组，子数组包含 nums 里面所有的元素
        # 也可以划分为 n 个子数组，每个子数组只包含 1 个元素
        # 也就是说，m 的取值范围为 1 <= m <= n
        # 假设 nums = [ 7 , 2 , 5 , 10 , 8 ]
        # 1、如果 m = 1 ，那么整个数组作为一部分，只有一种划法，最小的最大值为 32

```

为 10

2、如果 $m = n$ ，那么每个元素作为一个子数组，只有一种划法，从所有元素选取最大值，最小的最大值为 10

因此，最大值的最小值的范围为 [10, 32]

这里说明一下最大值和最小值的概念

最大值：在一个确定的划分之后，有很多个子数组，从所有子数组和里面挑选出一个最大值来

最小值：在 m 等于某个数字时，有很多种划分的方法，每一种划法都能有最大值，比较这些最大值，选出最小的来

```
# left 记录子数组和最大值下界
left = 0

# right记录子数组和最大值上界
right = 0

for num in nums:

    # 子数组和最大值下界为数组中元素的最大值，这个时候每个元素表示 1 个子数组
    left = max(left, num)

    # 子数组和最大值上界为数组中所有元素和，这个时候只有 1 个子数组
    right += num

# 在区间 [ left , right ] 中寻找出一个合理的值
# 这个值是使得这 m 个子数组各自和的最大值最小
while left <= right :

    # 猜测中间值为答案
    mid = left + (right - left) // 2

    # 按照这种方案把数组进行划分
    # 得到的子区间数 cnt
    cnt = self.subSplit( nums , mid )

    # 开始分析这种划分是否合理
    # 1、划分的子数组个数多了
    if cnt > k :

        # 为什么会划分的子数组个数多了
        # 因为限定的子数组的元素和值小，导致只能不断地去划分子数组
        # 为了可以少划分一些，需要提高子数组的元素和值
        left = mid + 1

    # 2、划分的子数组个数少了
    elif cnt < k :

        # 为什么会划分的子数组个数少了
        # 因为限定的子数组的元素和值大，导致每个子数组可以装很多元素
        # 为了可以多划分一些，需要减小子数组的元素和值
        right = mid - 1
```

```

# 3、划分的子数组个数刚刚好
elif cnt == k :
    # 此时，找到了满足分割数 m 的子数组最大值
    # 在此基础上继续在左区间寻找
    # 这样，才能使得满足分割数且最大值最小
    right = mid - 1

# while 终止条件是 left > right
# 最后一次跳出 while 之前，left 会等于 right
# 1、如果此时，在这种情况下进行切分的结果刚好等于分割数 m
# 那么为了获取更小的值，right 会向左移动，从而导致 left > right
# 于是，left 就是那个满足分割数为 m 情况下最大值最小的数
# 2、如果此时，在这种情况下进行切分的结果不等于分割数 m
# left 会向右移动，从而导致 left > right
# 最终 left 也恰好等于上一次分割数为 m 的 mid 值
# 结合视频里面的动画进行理解
return left

# 根据子数组和 maxSum 划分数组，返回子数组个数
def subSplit(self, nums: List[int], maxSum: int) -> int:

    # 默认为划分为一个子数组，即就是 nums 数组
    cnt = 1

    # 当前子数组的元素和
    tmpArrSum = 0

    # 对每个元素进行考虑它是否有资格划分进行
    for num in nums :

        # 如果发现把当前元素和归纳到目前这个数组后
        # 导致子数组元素和大于了 maxSum
        if tmpArrSum + num > maxSum :

            # 说明，这个元素需要开始另外分配一个子数组了
            # 以该元素为界限开始进行划分
            cnt += 1

            # 当前子数组的元素和归零
            tmpArrSum = 0

        # 当前子数组的元素和进行累加
        tmpArrSum += num

    # 返回子数组的个数
    # 这些子数组每个子数组元素和的最大值都不会超过 maxSum

```

```
return cnt
```

LeetCode 704、二分查找

一、题目描述

给定一个 n 个元素有序的（升序）整型数组 `nums` 和一个目标值 `target`，写一个函数搜索 `nums` 中的 `target`，如果目标值存在返回下标，否则返回 `-1`。

示例 1:

输入: `nums = [-1,0,3,5,9,12]`, `target = 9` 输出: `4` 解释: `9` 出现在 `nums` 中并且下标为 `4`

示例 2:

输入: `nums = [-1,0,3,5,9,12]`, `target = 2` 输出: `-1` 解释: `2` 不存在 `nums` 中因此返回 `-1`

提示:

- 你可以假设 `nums` 中的所有元素是不重复的。
- n 将在 $[1, 10000]$ 之间。
- `nums` 的每个元素都将在 $[-9999, 9999]$ 之间。

二、题目解析

二分查找法有以下几个前提:

- 1、数组为有序数组
- 2、数组中不存在重复元素

所以，如果题目中包含以上两个条件，那么就可以思考能否使用二分查找法；相反，如果想使用二分查找法，就得去构造以上这两个条件。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二分查找(704):https://leetcode-cn.com/problems/binary-search/
class Solution {
    public int search(int[] nums, int target) {

        // 二分查找法的一种写法，在左闭右闭的区间里去查找目标值

        // left 为当前区间最左侧的元素，可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素，可以获取到
        int right = nums.length - 1;
```

```

// 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
// 当 [ left , right ] 这个区间不存在元素的时候, 才跳出 while 循环, 也就是当 left >
right 时跳出循环
// 即当 left = right + 1 时, 搜索区间没有元素了
// 由于 left 和 right 相遇的时候指向同一个元素, 这个元素是存在于 [ left , right] 区间,
这个区间就一个元素
// 所以此时 while 循环的判断可以使用等号
while(left <= right) {

    // left + (right - left) / 2 和 (left + right) / 2 的结果相同
    // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大, 导致相加
的结果溢出的问题
    // 比如数据 int 的最大值为 2,147,483,647
    // 此时, left 为 2,147,483,640
    // 此时, right 为 2,147,483,646
    // 两者直接相加超过了 2,147,483,647, 产生了溢出
    int mid = left + (right - left) / 2;

    // 中间的元素和目标值 target 相等, 直接返回下标即可
    if(nums[mid] == target) {
        // 直接返回下标即可
        return mid;

        // 中间的元素大于了目标值 target
        // 1、数组为有序数组, 比如为递增数组
        // 2、数组中不存在重复元素
        // 由于该数组存在以上两个特点, 所以 [ mid , right ] 这个区间中的所有元素都大于目标值
target

        // 因此, 缩小查找区间为 [ left , mid - 1]
    } else if(nums[mid] > target) {
        // 也就是说缩小之后的区间最右侧位置为 mid - 1
        right = mid - 1;

        // 中间的元素小于了目标值 target
        // 1、数组为有序数组, 比如为递增数组
        // 2、数组中不存在重复元素
        // 由于该数组存在以上两个特点, 所以 [ left , mid ] 这个区间中的所有元素都小于目标值
target

        // 因此, 缩小查找区间为 [ mid + 1 , right]
    } else if(nums[mid] < target){
        // 也就是说缩小之后的区间最左侧位置为 mid + 1
        left = mid + 1;
    }
}

// 查找完区间中的所有元素都没有找到, 返回 -1
return -1;
}

```

```
}
```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二分查找(704):https://leetcode-cn.com/problems/binary-search/
class solution {
public:
    int search(vector<int>& nums, int target) {

        // 二分查找法的一种写法，在左闭右闭的区间里去查找目标值

        // left 为当前区间最左侧的元素，可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素，可以获取到
        int right = nums.size() - 1;

        // 在 while 循环里面，left 不断的 ++，而 right 不断的 --
        // 当 [ left , right ] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left >
        right 时跳出循环
        // 即当 left = right + 1 时，搜索区间没有元素了
        // 由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [ left , right] 区间，
        这个区间就一个元素
        // 所以此时 while 循环的判断可以使用等号
        while(left <= right) {

            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
            的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时，left 为 2,147,483,640
            // 此时，right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647，产生了溢出
            int mid = left + (right - left) / 2;

            // 中间的元素和目标值 target 相等，直接返回下标即可
            if(nums[mid] == target) {
                // 直接返回下标即可
                return mid;

                // 中间的元素大于了目标值 target
                // 1、数组为有序数组，比如为递增数组
                // 2、数组中不存在重复元素
            }
        }
    }
};
```

```

        // 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素都大于目标值
target
        // 因此，缩小查找区间为 [ left , mid - 1]
        } else if(nums[mid] > target) {
            // 也就是说缩小之后的区间最右侧位置为 mid - 1
            right = mid - 1;

        // 中间的元素小于了目标值 target
        // 1、数组为有序数组，比如为递增数组
        // 2、数组中不存在重复元素
        // 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素都小于目标值
target
        // 因此，缩小查找区间为 [ mid + 1 , right]
        } else if(nums[mid] < target){
            // 也就是说缩小之后的区间最左侧位置为 mid + 1
            left = mid + 1;
        }
    }

    // 查找完区间中的所有元素都没有找到，返回 -1
    return -1;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二分查找(704):https://leetcode-cn.com/problems/binary-search/
class Solution:
    def search(self, nums: List[int], target: int) -> int:

        # 二分查找法的一种写法，在左闭右闭的区间里去查找目标值

        # left 为当前区间最左侧的元素，可以获取到
        left = 0

        # right 为当前区间最右侧的元素，可以获取到
        right = len(nums) - 1

        # 在 while 循环里面，left 不断的 ++，而 right 不断的 --
        # 当 [ left , right ] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left >
right 时跳出循环
        # 即当 left = right + 1 时，搜索区间没有元素了
        # 由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [ left , right] 区间，这个
区间就一个元素

```



```

# 所以此时 while 循环的判断可以使用等号
while left <= right :

    # left + (right - left) / 2 和 (left + right) / 2 的结果相同
    # 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
    # 的结果溢出的问题
    # 比如数据 int 的最大值为 2,147,483,647
    # 此时，left 为 2,147,483,640
    # 此时，right 为 2,147,483,646
    # 两者直接相加超过了 2,147,483,647，产生了溢出
    mid = left + (right - left) // 2

    # 中间的元素和目标值 target 相等，直接返回下标即可
    if nums[mid] == target :
        # 直接返回下标即可
        return mid

    # 中间的元素大于了目标值 target
    # 1、数组为有序数组，比如为递增数组
    # 2、数组中不存在重复元素
    # 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素都大于目标值
target
    # 因此，缩小查找区间为 [ left , mid - 1]
    elif nums[mid] > target :
        # 也就是说缩小之后的区间最右侧位置为 mid - 1
        right = mid - 1

    # 中间的元素小于了目标值 target
    # 1、数组为有序数组，比如为递增数组
    # 2、数组中不存在重复元素
    # 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素都小于目标值
target
    # 因此，缩小查找区间为 [ mid + 1 , right]
    elif nums[mid] < target :
        # 也就是说缩小之后的区间最左侧位置为 mid + 1
        left = mid + 1

# 查找完区间中的所有元素都没有找到，返回 -1
return -1

```

LeetCode 81、搜索旋转排序数组II

一、题目描述

已知存在一个按非降序排列的整数数组 `nums`，数组中的值不必互不相同。

在传递给函数之前，`nums` 在预先未知的某个下标 `k` ($0 \leq k < \text{nums.length}$) 上进行了 **旋转**，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`（下标从 **0** 开始计数）。例如，`[0,1,2,4,4,4,5,6,6,7]` 在下标 **5** 处经旋转后可能变为 `[4,5,6,6,7,0,1,2,4,4]`。

给你 **旋转后** 的数组 `nums` 和一个整数 `target`，请你编写一个函数来判断给定的目标值是否存在于数组中。如果 `nums` 中存在这个目标值 `target`，则返回 `true`，否则返回 `false`。

你必须尽可能减少整个操作步骤。

示例 1：

输入：`nums = [2,5,6,0,0,1,2]`, `target = 0` 输出：`true`

示例 2：

输入：`nums = [2,5,6,0,0,1,2]`, `target = 3` 输出：`false`

提示：

- `1 \leq \text{nums.length} \leq 5000`
- `-10^4 \leq \text{nums}[i] \leq 10^4`
- 题目数据保证 `nums` 在预先未知的某个下标上进行了旋转
- `-10^4 \leq \text{target} \leq 10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 搜索旋转排序数组 II (LeetCode 81) : https://leetcode.cn/problems/search-in-rotated-sorted-array-ii/
class Solution {
    public boolean search(int[] nums, int target) {
        // left 指向当前区间的最左边位置，所以初始化为 0
        int left = 0;

        // right 指向当前区间的最右边位置，所以初始化为 nums.length - 1
        int right = nums.length - 1;

        // 循环进行二分查找，直到左端点位置超过了右端点
        // 或者在循环过程中找到了 target
        while( left <= right){
```

```

// 计算当前区间的中间位置，向下取整
int mid = (left + right) / 2;

// 如果中间位置数字 nums[mid] 等于目标值 target，那么说明找到了
// 返回当前的下标 mid
if(nums[mid] == target) return true;

// 按顺序遍历一样，一个个向右移动
if(nums[mid] == nums[left]){

    left++;

    continue;
}

// 否则的话需要先确定 mid 的左边还是右边为有序区间

// 如果当前区间最左端的值 nums[left] 小于 nums[mid]
// 说明从 left 到 mid 这段区间是递增的，为有序区间
// 即 mid 的左侧为有序区间，右侧为无序区间
if(nums[left] < nums[mid]){

    // 先去判断 target 是否在左侧有序区间内
    // 如果目标值 target 大于这段有序区间的最小值 nums[left] 同时小于这段有序区间的最大
    大值 nums[mid]
    // 那么说明需要在这段有序区间去寻找 target
    if(target >= nums[left] && target <= nums[mid]){

        // 所以缩小范围为 left 到 mid - 1
        // 当前区间的左侧为 left，右侧 right = mid - 1
        right = mid - 1;

        // 否则说明需要在 mid 的右侧无序区间搜索
    }else{

        // 所以缩小范围为 mid + 1 到 right
        // 当前区间的左侧为 left = mid + 1，右侧 right
        left = mid + 1;
    }

    // 否则说明当前区间最左端的值 nums[left] 大于 nums[mid]
    // 说明从 left 到 mid 这段区间是无序区间
    // 即 mid 的左侧为无序区间，右侧为有序区间
}else{

    // 先去判断 target 是否在右侧有序区间内
    // 如果目标值 target 大于这段有序区间的最小值 nums[mid] 同时小于这段有序区间的最大
    大值 nums[right]
    // 那么说明需要在这段有序区间去寻找 target

```

```

        if(target >= nums[mid] && target <= nums[right]){

            // 所以缩小范围为 mid + 1 到 right
            // 当前区间的左侧为 left = mid + 1, 右侧 right
            left = mid + 1;

            // 否则说明需要在 mid 的左侧无序区间搜索
        }else{

            // 所以缩小范围为 left 到 mid - 1
            // 当前区间的左侧为 left, 右侧 right = mid - 1
            right = mid - 1;

        }
    }

    // 目标值不存在, 返回 -1
    return false;

}
}

```

面试题 10.03. 搜索旋转数组

一、题目描述

搜索旋转数组。给定一个排序后的数组，包含n个整数，但这个数组已被旋转过很多次了，次数不详。请编写代码找出数组中的某个元素，假设数组元素原先是按升序排列的。若有多个相同元素，返回索引值最小的一个。

示例1:

输入: arr = [15, 16, 19, 20, 25, 1, 3, 4, 5, 7, 10, 14], target = 5 输出: 8 (元素5在该数组中的索引)

示例2:

输入: arr = [15, 16, 19, 20, 25, 1, 3, 4, 5, 7, 10, 14], target = 11 输出: -1 (没有找到)

提示:

1. arr 长度范围在[1, 1000000]之间

二、题目解析

三、参考代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

// 面试题 10.03. 搜索旋转数组: <https://leetcode.cn/problems/search-rotate-array-lcci/>

```
class Solution {
```

```
    public int search(int[] arr, int target) {
```

```
        /**
```

```
        本题和 LeetCode 81、搜索旋转排序数组II 非常类似，只需要修改部分代码即可
```

```
        修改位置标注了 星号
```

```
        LeetCode 81、搜索旋转排序数组II 思路：
```

```
https://r07na4yqwor.feishu.cn/docx/Pe13drtqMoBrHJxDT32ceMnCnMH
```

```
        */
```

```
        // 重点：特殊情况处理
```

```
        // 否则 [5,5,5,1,2,3,4,5]
```

```
        // 5
```

```
        // 这个测试用例会出错，预期结果是 0 ，下方代码输出的是 7
```

```
        if(arr[0] == target){
```

```
            return 0;
```

```
        }
```

```
        // left 指向当前区间的最左边位置，所以初始化为 0
```

```
        int left = 0;
```

```
        // right 指向当前区间的最右边位置，所以初始化为 arr.length - 1
```

```
        int right = arr.length - 1;
```

```
        // 循环进行二分查找，直到左端点位置超过了右端点
```

```
        // 或者在循环过程中找到了 target
```

```
        while( left <= right){
```

```
            // 计算当前区间的中间位置，向下取整
```

```
            int mid = (left + right) / 2;
```

```
            // 如果中间位置数字 arr[mid] 等于目标值 target，那么说明找到了
```

```
            // 返回当前的下标 mid
```

```
            /*****下方代码修改*****/
```

```
            if(arr[mid] == target){
```

```
                while (mid > 1 && arr[mid - 1] == arr[mid]) {
```

```
                    mid--;
```

```
                }
```

```
                return mid;
```

```
            }
```

```
            /*****上方代码修改*****/
```

```
            // 否则的话需要先确定 mid 的左边还是右边为有序区间
```

```
            if(arr[left] == arr[mid]){
```

```

        left++;

        continue;

    }

    // 如果当前区间最左端的值 arr[left] 小于等于 arr[mid]
    // 说明从 left 到 mid 这段区间是递增的，为有序区间
    // 即 mid 的左侧为有序区间，右侧为无序区间
    if(arr[left] < arr[mid]){

        // 先去判断 target 是否在左侧有序区间内
        // 如果目标值 target 大于这段有序区间的最小值 arr[left] 同时小于这段有序区间的最大
        值 arr[mid]

        // 那么说明需要在这段有序区间去寻找 target
        if(target >= arr[left] && target <= arr[mid]){

            // 所以缩小范围为 left 到 mid - 1
            // 当前区间的左侧为 left，右侧 right = mid - 1
            right = mid - 1;

            // 否则说明需要在 mid 的右侧无序区间搜索
        }else{

            // 所以缩小范围为 mid + 1 到 right
            // 当前区间的左侧为 left = mid + 1，右侧 right
            left = mid + 1;

        }

        // 否则说明当前区间最左端的值 arr[left] 大于 arr[mid]
        // 说明从 left 到 mid 这段区间是无序区间
        // 即 mid 的左侧为无序区间，右侧为有序区间
    }else{

        // 先去判断 target 是否在右侧有序区间内
        // 如果目标值 target 大于这段有序区间的最小值 arr[mid] 同时小于这段有序区间的最大
        值 arr[right]

        // 那么说明需要在这段有序区间去寻找 target
        if(target >= arr[mid] && target <= arr[right]){

            // 所以缩小范围为 mid + 1 到 right
            // 当前区间的左侧为 left = mid + 1，右侧 right
            left = mid + 1;

            // 否则说明需要在 mid 的左侧无序区间搜索
        }else{

            // 所以缩小范围为 left 到 mid - 1
            // 当前区间的左侧为 left，右侧 right = mid - 1

```

```
        right = mid - 1;

    }

}

/*****下方代码修改*****/
return -1;
/*****上方代码修改*****/

}

}
```

第二十五天

LeetCode 278、第一个错误版本

一、题目描述

你是产品经理，目前正在带领一个团队开发新的产品。不幸的是，你的产品的最新版本没有通过质量检测。由于每个版本都是基于之前的版本开发的，所以错误的版本之后的所有版本都是错的。

假设你有 n 个版本 $[1, 2, \dots, n]$ ，你想找出导致之后所有版本出错的第一个错误的版本。

你可以通过调用 `bool isBadVersion(version)` 接口来判断版本号 `version` 是否在单元测试中出错。实现一个函数来查找第一个错误的版本。你应该尽量减少对调用 API 的次数。

示例 1:

```
输入: n = 5, bad = 4
输出: 4
解释: 调用 isBadVersion(3) -> false
调用 isBadVersion(5) -> true
调用 isBadVersion(4) -> true
所以, 4 是第一个错误的版本。
```

提示:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 第一个错误的版本 ( LeetCode 278 ) :https://leetcode-cn.com/problems/first-bad-version/
/* The isBadVersion API is defined in the parent class VersionControl.
    boolean isBadVersion(int version); */
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {

        // 注意到题目的第一个版本下标为 1，所以边界情况为 [ 1, n ]
        int left = 1;
        int right = n;

        // 利用二分查找的方法，去定位出【第一个错误的版本】
        // 在 while 循环里面，left 不断的 ++，而 right 不断的 --
        // 当 left 和 right 相等，[ left, right ] 这个区间存在一个版本的时候
        // 这个版本就是【第一个错误的版本】
        // 所以，当 left == right 时，跳出循环
        // 此时，while 循环的判断不可以使用等号
        while (left < right) {
            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left、right 同时太大，导致相加
            // 的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时，left 为 2,147,483,640
            // 此时，right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647，产生了溢出
            int mid = left + (right - left) / 2;

            // 调用系统函数，查看当前的版本是否是错误的版本
            // 1、如果当前版本为错误的版本，那么此时 mid 指向的版本有可能是【第一个错误的版本】
            // 因此，区间缩小为 [ left, mid ]
            if (isBadVersion(mid)) {

                // 再次注意，mid 指向的版本有可能是【第一个错误的版本】，因此，right 移动到 mid 的
                // 位置
                right = mid;

                // 2、如果当前版本为正确的版本，那么此时 mid 指向的版本绝对不可能是【第一个错误的版本】
                // 因此，区间缩小为 [ mid + 1, right ]
            } else {

                // 再次注意，mid 指向的版本已经是正确的版本了
                // 意味着 [ left, mid ] 的所有版本都是正确的，错误的版本发生在 [ mid + 1,
                // right ] 这个区间里面
                left = mid + 1;
            }
        }
    }
}
```



```

    }
}

// 当 left 和 right 相等, [ left , right ] 这个区间存在一个版本的时候
// 这个版本就是【第一个错误的版本】
return left;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int firstBadVersion(int n) {
        // 注意到题目的第一个版本下标为 1 , 所以边界情况为 [ 1 , n ]
        int left = 1 ;
        int right = n ;

        // 利用二分查找的方法, 去定位出【第一个错误的版本】
        // 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
        // 当 left 和 right 相等, [ left , right ] 这个区间存在一个版本的时候
        // 这个版本就是【第一个错误的版本】
        // 所以, 当 left == right 时, 跳出循环
        // 此时, while 循环的判断不可以使用等号
        while (left < right) {
            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大, 导致相加
            // 的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时, left 为 2,147,483,640
            // 此时, right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647, 产生了溢出
            int mid = left + (right - left) / 2;

            // 调用系统函数, 查看当前的版本是否是错误的版本
            // 1、如果当前版本为错误的版本, 那么此时 mid 指向的版本有可能是【第一个错误的版本】
            // 因此, 区间缩小为 [ left , mid ]
            if (isBadVersion(mid)) {

                // 再次注意, mid 指向的版本有可能是【第一个错误的版本】, 因此, right 移动到 mid 的
                // 位置
                right = mid;

            }

            // 2、如果当前版本为正确的版本, 那么此时 mid 指向的版本绝对不可能是【第一个错误的版本】
            // 因此, 区间缩小为 [ mid + 1 , right ]
        } else {

            // 再次注意, mid 指向的版本已经是正确的版本了

```

```
        // 意味着 [ left , mid ] 的所有版本都是正确的，错误的版本发生在 [ mid + 1 ,
right ] 这个区间里面
        left = mid + 1;

    }
}

// 当 left 和 right 相等， [ left , right ] 这个区间存在一个版本的时候
// 这个版本就是【第一个错误的版本】
return left;
}
};
```

3、Python 代码

四、复杂度分析

时间复杂度： $O(\log n)$ ，其中 n 是给定版本的数量。

空间复杂度： $O(1)$ 。我们只需要常数的空间保存若干变量。

LeetCode 367、有效的完全平方数

一、题目描述

给定一个 正整数 `num`，编写一个函数，如果 `num` 是一个完全平方数，则返回 `true`，否则返回 `false`。

进阶：不要使用任何内置的库函数，如 `sqrt`。

示例 1：

```
输入：num = 16
输出：true
```

示例 2：

```
输入：num = 14
输出：false
```

提示：

- $1 \leq \text{num} \leq 2^{31} - 1$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的完全平方数 ( LeetCode 367 ) :https://leetcode-cn.com/problems/valid-perfect-square
class Solution {
    public boolean isPerfectSquare(int num) {

        // left 为当前区间最左侧的元素，可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素，可以获取到
        int right = num;

        // 在区间 [left , right] 这个左闭右闭的区间里面去查找
        // 查找什么呢？
        // 查找是否存在一个数 a，使得  $a * a = num$ 

        // 在 while 循环里面，left 不断的向右移动，而 right 不断的向左移动
        // 当 [ left , right ] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left >
right 时跳出循环
        // 即当 left = right + 1 时，搜索区间没有元素了
        // 由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [ left , right] 区间，
        这个区间就一个元素
        // 所以此时 while 循环的判断可以使用等号
        while (left <= right) {

            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
            的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时，left 为 2,147,483,640
            // 此时，right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647，产生了溢出
            int mid = left + (right - left) / 2;

            // 判断中间元素的平方与 num 的大小
            // 如果 mid * mid < num

            // mid * mid 的数值有可能超过 int 类型的最大数
            // 使用 long
            long square = (long) mid * mid;

            // 中间元素的平方小于了目标值 num
```

```

        // 数组为 [ 0 , n ]
        // 1、数组为有序数组，比如为递增数组
        // 2、数组中不存在重复元素
        // 由于该数组存在以上两个特点，所以 [ left , mid ] 这个区间中的所有元素的平方都小于目
标值 num
        // 因此，缩小查找区间为 [ mid + 1 , right]
        if (square < num){
            // 也就是说缩小之后的区间最左侧位置为 mid + 1
            left = mid + 1;

            // 中间元素的平方大于了目标值 num
            // 数组为 [ 0 , n ]
            // 1、数组为有序数组，比如为递增数组
            // 2、数组中不存在重复元素
            // 由于该数组存在以上两个特点，所以 [ mid , right ] 这个区间中的所有元素的平方都大于目
标值 num
            // 因此，缩小查找区间为 [ left , mid - 1]
        }else if( square > num){
            // 也就是说缩小之后的区间最右侧位置为 mid - 1
            right = mid - 1 ;

            // 中间元素的平方等于了目标值 num
        }else if( square == num){
            // 直接返回结果即可
            return true;
        }
    }

    // 查找完区间中的所有元素都没有找到，返回 false
    return false;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的完全平方数 ( LeetCode 367 ) :https://leetcode-cn.com/problems/valid-perfect-square
class Solution {
public:
    bool isPerfectSquare(int num) {
        // left 为当前区间最左侧的元素，可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素，可以获取到
        int right = num;
    }
}

```

```

// 在区间 [left , right] 这个左闭右闭的区间里面去查找
// 查找什么呢?
// 查找是否存在一个数 a, 使得 a * a = num

// 在 while 循环里面, left 不断的向右移动, 而 right 不断的向左移动
// 当 [ left , right ] 这个区间不存在元素的时候, 才跳出 while 循环, 也就是当 left >
right 时跳出循环
// 即当 left = right + 1 时, 搜索区间没有元素了
// 由于 left 和 right 相遇的时候指向同一个元素, 这个元素是存在于 [ left , right] 区间,
这个区间就一个元素
// 所以此时 while 循环的判断可以使用等号
while (left <= right) {

    // left + (right - left) / 2 和 (left + right) / 2 的结果相同
    // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大, 导致相加
    的结果溢出的问题
    // 比如数据 int 的最大值为 2,147,483,647
    // 此时, left 为 2,147,483,640
    // 此时, right 为 2,147,483,646
    // 两者直接相加超过了 2,147,483,647, 产生了溢出
    int mid = left + (right - left) / 2;

    // 判断中间元素的平方与 num 的大小
    // 如果 mid * mid < num

    // mid * mid 的数值有可能超过 int 类型的最大数
    // 使用 long
    long square = (long) mid * mid;

    // 中间元素的平方小于了目标值 num
    // 数组为 [ 0 , n ]
    // 1、数组为有序数组, 比如为递增数组
    // 2、数组中不存在重复元素
    // 由于该数组存在以上两个特点, 所以 [ left , mid ] 这个区间中的所有元素的平方都小于目
    标值 num

    // 因此, 缩小查找区间为 [ mid + 1 , right]
    if (square < num){
        // 也就是说缩小之后的区间最左侧位置为 mid + 1
        left = mid + 1;

    // 中间元素的平方大于了目标值 num
    // 数组为 [ 0 , n ]
    // 1、数组为有序数组, 比如为递增数组
    // 2、数组中不存在重复元素
    // 由于该数组存在以上两个特点, 所以 [ mid , right ] 这个区间中的所有元素的平方都大于目
    标值 num

    // 因此, 缩小查找区间为 [ left , mid - 1]
    }else if( square > num){

```

```

        // 也就是说缩小之后的区间最右侧位置为 mid - 1
        right = mid - 1 ;

        // 中间元素的平方等于了目标值 num
    }else if( square == num){
        // 直接返回结果即可
        return true;
    }
}

// 查找完区间中的所有元素都没有找到，返回 false
return false;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

有效的完全平方数 (LeetCode 367) :<https://leetcode-cn.com/problems/valid-perfect-square>

class Solution:

def isPerfectSquare(self, num: int) -> bool:

left 为当前区间最左侧的元素，可以获取到

left = 0

right 为当前区间最右侧的元素，可以获取到

right = num

在区间 [left , right] 这个左闭右闭的区间里面去查找

查找什么呢?

查找是否存在一个数 a，使得 $a * a = num$

在 while 循环里面，left 不断的向右移动，而 right 不断的向左移动

当 [left , right] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left > right 时跳出循环

即当 left = right + 1 时，搜索区间没有元素了

由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [left , right] 区间，这个区间就一个元素

所以此时 while 循环的判断可以使用等号

while left <= right :

left + (right - left) / 2 和 (left + right) / 2 的结果相同

但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加的结果溢出的问题

比如数据 的最大值为 2,147,483,647

```

# 此时, left 为 2,147,483,640
# 此时, right 为 2,147,483,646
# 两者直接相加超过了 2,147,483,647, 产生了溢出
mid = left + (right - left) // 2

# 判断中间元素的平方与 num 的大小
# 如果 mid * mid < num

# mid * mid 的数值有可能超过 类型的最大数
# 使用 long
square = mid * mid

# 中间元素的平方小于了目标值 num
# 数组为 [ 0 , n ]
# 1、数组为有序数组, 比如为递增数组
# 2、数组中不存在重复元素
# 由于该数组存在以上两个特点, 所以 [ left , mid ] 这个区间中的所有元素的平方都小于目标
值 num

# 因此, 缩小查找区间为 [ mid + 1 , right]
if square < num:
    # 也就是说缩小之后的区间最左侧位置为 mid + 1
    left = mid + 1

# 中间元素的平方大于了目标值 num
# 数组为 [ 0 , n ]
# 1、数组为有序数组, 比如为递增数组
# 2、数组中不存在重复元素
# 由于该数组存在以上两个特点, 所以 [ mid , right ] 这个区间中的所有元素的平方都大于目
标值 num

# 因此, 缩小查找区间为 [ left , mid - 1]
elif square > num :
    # 也就是说缩小之后的区间最右侧位置为 mid - 1
    right = mid - 1

# 中间元素的平方等于了目标值 num
elif square == num :
    # 直接返回结果即可
    return True

# 查找完区间中的所有元素都没有找到, 返回 False
return False

```

LeetCode 69、x 的平方根

一、题目描述

给你一个非负整数 x ，计算并返回 x 的算术平方根。

由于返回类型是整数，结果只保留整数部分，小数部分将被舍去。

注意：不允许使用任何内置指数函数和算符，例如 `pow(x, 0.5)` 或者 `x ** 0.5`。

示例 1：

输入：x = 4
输出：2

示例 2：

输入：x = 8
输出：2
解释：8 的算术平方根是 2.82842...，由于返回类型是整数，小数部分将被舍去。

提示：

- $0 \leq x \leq 2^{31} - 1$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// x 的平方根（LeetCode 69）：https://leetcode.cn/problems/sqrtx/
class Solution {
    public int mySqrt(int x) {

        // 需要寻找出一个数 ans，使得 ans * ans <= x，并且 ans 总是尽可能更大
        // 于是，开始在 [0, x] 这个区间中去查找
        // 在查找过程中，不断的去缩小区间

        // 左区间开始位置为 0
        int left = 0;

        // 右区间开始位置为 x
        int right = x;

        // 算法平方根的结果，一开始为一个不可能的数 -1
```



```

int ans = -1;

// 开始在区间中查找
while (left <= right) {

    // 先定位中间元素
    int mid = left + (right - left) / 2;

    // 由于 x 的范围能到达 int 的最大值
    // 所以 mid 也有可能很大, 导致 mid * mid 超过 int 的范围
    // 因此使用 long
    // 判断 mid 是否符合要求
    // 1、如果发现不够
    if ((long) mid * mid <= x) {

        // 保留结果
        ans = mid;

        // 同时, 可以舍去 left 左边的所有元素
        left = mid + 1;

        // 2、如果发现超过
    } else {

        // 同时, 可以舍去 right 右边的所有元素
        right = mid - 1;
    }
}

// 返回结果
return ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int mySqrt(int x) {
        // 需要寻找出一个数 ans , 使得 ans * ans <= x , 并且 ans 总是尽可能更大
        // 于是, 开始在 [ 0 , x ] 这个区间中去查找
        // 在查找过程中, 不断的去缩小区间

        // 左区间开始位置为 0
        int left = 0 ;

        // 右区间开始位置为 x
        int right = x ;
    }
}

```

```

// 算法平方根的结果，一开始为一个不可能的数 -1
int ans = -1;

// 开始在区间中查找
while (left <= right) {

    // 先定位中间元素
    int mid = left + (right - left) / 2;

    // 由于 x 的范围能到达 int 的最大值
    // 所以 mid 也有可能很大，导致 mid * mid 超过 int 的范围
    // 因此使用 long
    // 判断 mid 是否符合要求
    // 1、如果发现不够
    if ((long) mid * mid <= x) {

        // 保留结果
        ans = mid;

        // 同时，可以舍去 left 左边的所有元素
        left = mid + 1;

        // 2、如果发现超过
    } else {

        // 同时，可以舍去 right 右边的所有元素
        right = mid - 1;
    }
}

// 返回结果
return ans;
}
};

```

3、Python 代码

```

class solution:
    def mySqrt(self, x: int) -> int:
        # 需要寻找出一个数 ans，使得 ans * ans <= x，并且 ans 总是尽可能更大
        # 于是，开始在 [ 0，x ] 这个区间中去查找
        # 在查找过程中，不断的去缩小区间

        # 左区间开始位置为 0
        left = 0

        # 右区间开始位置为 x
        right = x

```

```

# 算法平方根的结果，一开始为一个不可能的数 -1
ans = -1

# 开始在区间中查找
while left <= right :

    # 先定位中间元素
    mid = left + (right - left) // 2

    # 由于 x 的范围能到达 的最大值
    # 所以 mid 也有可能很大，导致 mid * mid 超过 的范围
    # 因此使用 long
    # 判断 mid 是否符合要求
    # 1、如果发现不够
    if mid * mid <= x :

        # 保留结果
        ans = mid

        # 同时，可以舍去 left 左边的所有元素
        left = mid + 1

    # 2、如果发现超过
    else :
        # 同时，可以舍去 right 右边的所有元素
        right = mid - 1

# 返回结果
return ans

```

四、复杂度分析

- 时间复杂度： $O(\log x)$ ，即为二分查找需要的次数。
- 空间复杂度： $O(1)$ 。

第二十六天

LeetCode 162、寻找峰值

一、题目描述

峰值元素是指其值严格大于左右相邻值的元素。

给你一个整数数组 `nums`，找到峰值元素并返回其索引。数组可能包含多个峰值，在这种情况下，返回 **任何一个峰值** 所在位置即可。

你可以假设 `nums[-1] = nums[n] = -∞`。

你必须实现时间复杂度为 $O(\log n)$ 的算法来解决此问题。

示例 1:

输入: `nums = [1,2,3,1]`

输出: 2

解释: 3 是峰值元素, 你的函数应该返回其索引 2。

示例 2:

输入: `nums = [1,2,1,3,5,6,4]`

输出: 1 或 5

解释: 你的函数可以返回索引 1, 其峰值元素为 2;
或者返回索引 5, 其峰值元素为 6。

提示:

- $1 \leq \text{nums.length} \leq 1000$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- 对于所有有效的 i 都有 $\text{nums}[i] \neq \text{nums}[i + 1]$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找峰值(162):https://leetcode-cn.com/problems/find-peak-element/
class Solution {
    public int findPeakElement(int[] nums) {

        // left 为当前区间最左侧的元素, 可以获取到
        int left = 0;

        // right 为当前区间最右侧的元素, 可以获取到
        int right = nums.length - 1;

        // 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
        // 当 left 和 right 相等, [ left , right ] 这个区间存在一个元素的时候
        // 这也意味着这个区间里面的元素找不到其它元素和它进行比较, 也就无法得知这个元素是否严格大于它
        // 左右相邻值的元素了
        // 所以, 当 left == right 时, 跳出循环
        // 此时, while 循环的判断不可以使用等号
        while(left < right) {
```

```

        // left + (right - left) / 2 和 (left + right) / 2 的结果相同
        // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
        的结果溢出的问题
        // 比如数据 int 的最大值为 2,147,483,647
        // 此时，left 为 2,147,483,640
        // 此时，right 为 2,147,483,646
        // 两者直接相加超过了 2,147,483,647，产生了溢出
        int mid = left + (right - left) / 2;

        // 题目告诉我们 nums[-1] = nums[n] = -∞
        // 这暗示我们只要数组中存在一个元素比它相邻的元素大，那么沿着它一定可以找到一个峰值
        // 就像爬山一样，较小的山高度是 100，前面的山高度是 200，在前面是一个深渊，那么高度为
        200 的那座山就是山峰
        // 所以比较 nums[mid] 与 nums[mid + 1] 的值

        // 如果 nums[mid] > nums[mid + 1]
        // 所以，如果存在山峰，那么这一段是右侧下降的那一段，因此需要在左侧去寻找上升的那段
        if (nums[mid] > nums[mid + 1]) {

            // 缩小区间，从 [ left , mid ] 里面去找
            right = mid;

        // 如果 nums[mid] < nums[mid + 1]
        // 所以，如果存在山峰，那么这一段是左侧上升的那一段，因此需要在右侧去寻找下降的那段

        } else {

            // 缩小区间，从 [ mid + 1 , right ] 里面去找
            left = mid + 1;

        }
    }

    // 跳出循环，此时 left == right，返回这个下标即可
    return left;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找峰值(162):https://leetcode-cn.com/problems/find-peak-element/
class Solution {
public:
    int findPeakElement(vector<int>& nums) {

```

```

// left 为当前区间最左侧的元素，可以获取到
int left = 0;

// right 为当前区间最右侧的元素，可以获取到
int right = nums.size() - 1;

// 在 while 循环里面，left 不断的 ++，而 right 不断的 --
// 当 left 和 right 相等，[ left , right ] 这个区间存在一个元素的时候
// 这也意味着这个区间里面的元素找不到其它元素和它进行比较，也就无法得知这个元素是否严格大于它
左右相邻值的元素了
// 所以，当 left == right 时，跳出循环
// 此时，while 循环的判断不可以使用等号
while(left < right) {

    // left + (right - left) / 2 和 (left + right) / 2 的结果相同
    // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
    的结果溢出的问题
    // 比如数据 int 的最大值为 2,147,483,647
    // 此时，left 为 2,147,483,640
    // 此时，right 为 2,147,483,646
    // 两者直接相加超过了 2,147,483,647，产生了溢出
    int mid = left + (right - left) / 2;

    // 题目告诉我们 nums[-1] = nums[n] = -∞
    // 这暗示我们只要数组中存在一个元素比它相邻的元素大，那么沿着它一定可以找到一个峰值
    // 就像爬山一样，较小的山高度是 100，前面的山高度是 200，在前面是一个深渊，那么高度为
    200 的那座山就是山峰
    // 所以比较 nums[mid] 与 nums[mid + 1] 的值

    // 如果 nums[mid] > nums[mid + 1]
    // 所以，如果存在山峰，那么这一段是右侧下降的那一段，因此需要在左侧去寻找上升的那段
    if (nums[mid] > nums[mid + 1]) {

        // 缩小区间，从 [ left , mid ] 里面去找
        right = mid;

    // 如果 nums[mid] < nums[mid + 1]
    // 所以，如果存在山峰，那么这一段是左侧上升的那一段，因此需要在右侧去寻找下降的那段

    } else {

        // 缩小区间，从 [ mid + 1 , right ] 里面去找
        left = mid + 1;

    }

}
}

```

```

        // 跳出循环, 此时 left == right, 返回这个下标即可
        return left;
    }
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者: 程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

寻找峰值(162):<https://leetcode-cn.com/problems/find-peak-element/>

class Solution:

def findPeakElement(self, nums: List[int]) -> int:

left 为当前区间最左侧的元素, 可以获取到

left = 0

right 为当前区间最右侧的元素, 可以获取到

right = len(nums) - 1

在 while 循环里面, left 不断的 ++, 而 right 不断的 --

当 left 和 right 相等, [left , right] 这个区间存在一个元素的时候

这也意味着这个区间里面的元素找不到其它元素和它进行比较, 也就无法得知这个元素是否严格大于它左右相邻值的元素了

所以, 当 left == right 时, 跳出循环

此时, while 循环的判断不可以使用等号

while left < right :

left + (right - left) / 2 和 (left + right) / 2 的结果相同

但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大, 导致相加的结果溢出的问题

比如数据 的最大值为 2,147,483,647

此时, left 为 2,147,483,640

此时, right 为 2,147,483,646

两者直接相加超过了 2,147,483,647, 产生了溢出

mid = left + (right - left) // 2

题目告诉我们 nums[-1] = nums[n] = -∞

这暗示我们只要数组中存在一个元素比它相邻的元素大, 那么沿着它一定可以找到一个峰值

就像爬山一样, 较小的山高度是 100, 前面的山高度是 200, 在前面是一个深渊, 那么高度为 200 的那座山就是山峰

所以比较 nums[mid] 与 nums[mid + 1] 的值

如果 nums[mid] > nums[mid + 1]

所以, 如果存在山峰, 那么这一段是右侧下降的那一段, 因此需要在左侧去寻找上升的那段

if nums[mid] > nums[mid + 1] :

```
# 缩小区间, 从 [ left , mid ] 里面去找
right = mid

# 如果 nums[mid] < nums[mid + 1]
# 所以, 如果存在山峰, 那么这一段是左侧上升的那一段, 因此需要在右侧去寻找下降的那段

else :

    # 缩小区间, 从 [ mid + 1 , right ] 里面去找
    left = mid + 1

# 跳出循环, 此时 left == right, 返回这个下标即可
return left
```

LeetCode 240、搜索二维矩阵 II

一、题目描述

在一个 $n * m$ 的二维数组中, 每一行都按照从左到右递增的顺序排序, 每一列都按照从上到下递增的顺序排序。请完成一个函数, 输入这样的一个二维数组和一个整数, 判断数组中是否含有该整数。

示例:

现有矩阵 matrix 如下:

[[1, 4, 7, 11, 15], [2, 5, 8, 12, 19], [3, 6, 9, 16, 22], [10, 13, 14, 17, 24], [18, 21, 23, 26, 30]]

给定 target = 5, 返回 true。

给定 target = 20, 返回 false。

限制:

- $0 \leq n \leq 1000$
- $0 \leq m \leq 1000$

二、题目解析

仔细观察矩阵, 可以发现: **左下角元素** 为所在**列最大元素**, 所在**行最小元素**

如果 **左下角元素** 大于了目标值, 则目标值一定在该行的上方, **左下角元素** 所在行可以消去。

如果 **左下角元素** 小于了目标值, 则目标值一定在该列的右方, **左下角元素** 所在列可以消去。

具体操作为从矩阵左下角元素开始遍历, 并与目标值对比:

- 当 `matrix[i][j] > target` 时: 行索引向上移动一格 (即 `i--`), 即消去矩阵第 `i` 行元素;
- 当 `matrix[i][j] < target` 时: 列索引向右移动一格 (即 `j++`), 即消去矩阵第 `j` 列元素;
- 当 `matrix[i][j] == target` 时: 返回 true。
- 如果越界, 则返回 false。

三、动画描述

四、图片描述

剑指offer 面试题精讲图解

作者：程序员吴师兄

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
	10	13	15	18	23
i	18	19	23	25	28
	j				

从矩阵**左下角**元素开始遍历，将其与目标值进行对比

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
	10	13	15	18	23
i	18	19	23	25	28
j					

从矩阵**左下角**元素开始遍历，将其与目标值进行对比

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
	10	13	15	18	23
i	18	19	23	25	28
j					

元素 18 大于目标值 7，行索引向上移动一格

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
i	10	13	15	18	23
	18	19	23	25	28
	j				

元素 18 大于目标值 7，行索引向上移动一格

此时原来的那一行都不符合要求

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
i	10	13	15	18	23
	18	19	23	25	28
	j				

元素 18 大于目标值 7，行索引向上移动一格

此时原来的那一行都不符合要求

面试题04. 二维数组中的查找
作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
	3	8	9	16	22
i	10	13	15	18	23
	18	19	23	25	28
	j				

元素 10 大于目标值 7，行索引向上移动一格

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
i	3	8	9	16	22
	10	13	15	18	23
	18	19	23	25	28
j					

元素 10 大于目标值 7，行索引向上移动一格
此时原来的那一行都不符合要求

面试题04. 二维数组中的查找

作者：程序员吴师兄

target = 7

	1	4	6	9	15
	2	5	7	12	19
i	3	8	9	16	22
	10	13	15	18	23
	18	19	23	25	28
j					

元素 10 大于目标值 7，行索引向上移动一格
此时原来的那一行都不符合要求

面试题04. 二维数组中的查找
作者：程序员吴师兄

target = 7

	1	4	6	9	15
i	2	5	7	12	19
	3	8	9	16	22
	10	13	15	18	23
	18	19	23	25	28
		j			

元素 8 大于目标值 7，行索引向上移动一格
此时原来的那一行都不符合要求

五、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// LeetCode 240、搜索二维矩阵 II: https://leetcode.cn/problems/search-a-2d-matrix-ii/submissions/
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {

        // 从数组的最左下角位置开始去搜索整个二维数组
        // 1、当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
        // 那么就不用去搜索这一行了
        // 2、当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
        // 那么就不用去搜索这一列了

        // 初始化 i 和 j 为数组左下角元素
        // 最后一行
        int i = matrix.length - 1;

        // 第 0 列
```

```

int j = 0;

// 从数组的左下角开始出发，只要 i 和 j 没有越界继续判断
while( i >= 0 && j <= matrix[0].length - 1 ){

    // 当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
    if(matrix[i][j] > target){

        // 行索引向上移动一格（即 i-- ），即消去矩阵第 i 行元素
        i--;

        // 当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
    }else if(matrix[i][j] < target){

        // 列索引向右移动一格（即 j++ ），即消去矩阵第 j 列元素
        j++;

        // 否则，说明找到目标值
    }else{

        // 直接返回 true
        return true;
    }
}

// 遍历了整个二维数组，没有找到目标值，返回 false
return false;
}
}

```

六、复杂度分析

时间复杂度

时间复杂度为 $O(M+N)$ ，其中， N 和 M 分别为矩阵行数和列数，此算法最多循环 $M + N$ 次。

空间复杂度

空间复杂度为 $O(1)$ 。

七、相关标签

- 数组
 - 双指针
 - 二分法
-

LeetCode 74、搜索二维矩阵

一、题目描述

编写一个高效的算法来判断 $m \times n$ 矩阵中，是否存在一个目标值。该矩阵具有如下特性：

- 每行中的整数从左到右按升序排列。
- 每行的第一个整数大于前一行的最后一个整数。

示例：

现有矩阵 matrix 如下：

```
[
  [1,   4,   7,  11, 15],
  [2,   5,   8,  12, 19],
  [3,   6,   9,  16, 22],
  [10, 13, 14, 17, 24],
  [18, 21, 23, 26, 30]
]
```

给定 target = 5，返回 true。

给定 target = 20，返回 false。

限制：

- $0 \leq n \leq 1000$
- $0 \leq m \leq 1000$

二、保姆级参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 搜索二维矩阵 ( LeetCode 74 ) : https://leetcode-cn.com/problems/search-a-2d-matrix/
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        // 从数组的最左下角位置开始去搜索整个二维数组
        // 1、当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
        // 那么就不用去搜索这一行了
        // 2、当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
        // 那么就不用去搜索这一列了

        // 初始化 i 和 j 为数组左下角元素
        // 最后一行
        int i = matrix.length - 1;

        // 第 0 列
```



```

int j = 0;

// 从数组的左下角开始出发，只要 i 和 j 没有越界继续判断
while(i >= 0 && j < matrix[0].length){

    // 当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
    if(matrix[i][j] > target){

        // 行索引向上移动一格（即 i-- ），即消去矩阵第 i 行元素
        i--;

        // 当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
    }else if(matrix[i][j] < target){

        // 列索引向右移动一格（即 j++ ），即消去矩阵第 j 列元素
        j++;

        // 否则，说明找到目标值
    }else{

        // 直接返回 true
        return true;
    }
}

// 遍历了整个二维数组，没有找到目标值，返回 false
return false;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 搜索二维矩阵（ LeetCode 74 ）：https://leetcode-cn.com/problems/search-a-2d-matrix/
class solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        // 从数组的最左下角位置开始去搜索整个二维数组
        // 1、当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
        // 那么就不用去搜索这一行了
        // 2、当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
        // 那么就不用去搜索这一行了

        // 初始化 i 和 j 为数组左下角元素
        // 最后一行
    }
}

```

```

int i = matrix.size() - 1;

// 第 0 列
int j = 0;

// 从数组的左下角开始出发，只要 i 和 j 没有越界继续判断
while(i >= 0 && j < matrix[0].size()){

    // 当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
    if(matrix[i][j] > target){

        // 行索引向上移动一格（即 i-- ），即消去矩阵第 i 行元素
        i--;

        // 当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
    }else if(matrix[i][j] < target){

        // 列索引向右移动一格（即 j++ ），即消去矩阵第 j 列元素
        j++;

        // 否则，说明找到目标值
    }else{

        // 直接返回 true
        return true;
    }
}

// 遍历了整个二维数组，没有找到目标值，返回 false
return false;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 搜索二维矩阵（LeetCode 74）：https://leetcode-cn.com/problems/search-a-2d-matrix/
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        # 从数组的最左下角位置开始去搜索整个二维数组
        # 1、当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
        # 那么就不用去搜索这一行了
        # 2、当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
        # 那么就不用去搜索这一行了

```

```

# 初始化 i 和 j 为数组左下角元素
# 最后一行
i = len(matrix) - 1

# 第 0 列
j = 0

# 从数组的左下角开始出发，只要 i 和 j 没有越界继续判断
while i >= 0 and j < len(matrix[0]) :

    # 当发现当前遍历的元素大于 target 时，意味着这个元素后面的所有元素也都大于 target
    if matrix[i][j] > target :

        # 行索引向上移动一格（即 i-- ），即消去矩阵第 i 行元素
        i -= 1

    # 当发现当前遍历的元素小于 target 时，意味着这个元素上面的所有元素也都小于 target
    elif matrix[i][j] < target :

        # 列索引向右移动一格（即 j++ ），即消去矩阵第 j 列元素
        j += 1

    # 否则，说明找到目标值
    else :

        # 直接返回 ture
        return True

# 遍历了整个二维数组，没有找到目标值，返回 False
return False

```

LeetCode 852、山脉数组的顶峰索引

一、题目描述

符合下列属性的数组 `arr` 称为 **山脉数组**：

- `arr.length >= 3`
- 存在 `i` ($0 < i < arr.length - 1$) 使得：
 - `arr[0] < arr[1] < ... arr[i-1] < arr[i]`
 - `arr[i] > arr[i+1] > ... > arr[arr.length - 1]`

给你由整数组成的山脉数组 `arr`，返回任何满足 `arr[0] < arr[1] < ... arr[i - 1] < arr[i] > arr[i + 1] > ... > arr[arr.length - 1]` 的下标 `i`。

示例 1：

输入：arr = [0,1,0] 输出：1

示例 2:

输入: arr = [0,2,1,0] 输出: 1

示例 3:

输入: arr = [0,10,5,2] 输出: 1

示例 4:

输入: arr = [3,4,5,1] 输出: 2

提示

- `3 <= arr.length <= 104`
- `0 <= arr[i] <= 106`
- 题目数据保证 arr 是一个山脉数组

进阶: 很容易想到时间复杂度 $O(n)$ 的解决方案, 你可以设计一个 $O(\log(n))$ 的解决方案吗?

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 山脉数组的峰顶索引 ( LeetCode 852 ):https://leetcode-cn.com/problems/peak-index-in-a-mountain-array/
class Solution {
    public int peakIndexInMountainArray(int[] arr) {

        // 题目告诉我们 3 <= arr.length <= 10^4
        // 也就是说 arr 的最左侧和最右侧肯定不是山峰

        // 所以如果存在山脉, 数组中最左侧的元素肯定不是山脉的山峰位置, 因为它的左侧没有元素了
        // 因此, 搜索区间需要从索引为 1 的位置开始
        int left = 1;

        // 同样的, 如果存在山脉, 数组中最右侧的元素肯定不是山脉的山峰位置, 因为它的右侧没有元素了
        // 因此, 搜索区间需要从索引为 nums.length - 2 的位置结束
        int right = arr.length - 2;

        // 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
        // 当 left 和 right 相等, [ left , right ] 这个区间存在一个元素的时候
        // 这也意味着这个区间里面的元素找不到其它元素和它进行比较, 它已经傲视群峰, 就是山脉, 得到结果
        // 所以, 当 left == right 时, 跳出循环
        // 此时, while 循环的判断不可以使用等号
        while(left < right) {
```

```

// left + (right - left) / 2 和 (left + right) / 2 的结果相同
// 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
的结果溢出的问题
// 比如数据 int 的最大值为 2,147,483,647
// 此时，left 为 2,147,483,640
// 此时，right 为 2,147,483,646
// 两者直接相加超过了 2,147,483,647，产生了溢出
int mid = left + (right - left) / 2;

// 由于题目中提示【题目数据保证 arr 是一个山脉数组】
// 所以对于 arr 中任意位置的元素，比如 mid 所在的位置，只有以下三种情况

// 1、mid 处于山峰位置，也就是它的值大于左侧，也大于右侧
// arr[mid] > arr[mid + 1] && arr[mid] > arr[mid - 1]
// 此时，找到了结果
if( arr[mid] > arr[mid - 1] && arr[mid] > arr[mid + 1] ){

    // 返回结果
    return mid;

}

// 2、mid 处于上升状态，也就是它的值大于左侧，同时小于右侧
// arr[mid] > arr[mid - 1] && arr[mid] < arr[mid + 1]
// 那么，我们需要在右侧区间去寻找出下降的那段
if( arr[mid] > arr[mid - 1] && arr[mid] < arr[mid + 1]){

    // 缩小区间，从 [ mid + 1 , right ] 里面去找
    left = mid + 1;

}

// 3、mid 处于下降状态，也就是它的值小于左侧，同时小于右侧
// arr[mid] < arr[mid - 1] && arr[mid] > arr[mid + 1]
// 那么，我们需要在左侧区间去寻找出上升的那段
if( arr[mid] < arr[mid - 1] && arr[mid] > arr[mid + 1] ){

    // 缩小区间，从 [ left , mid - 1 ] 里面去找
    right = mid - 1;

}

}

// 跳出循环，此时 left == right，区间为 [ left, right ]，只有一个元素
// 这也意味着这个区间里面的元素找不到其它元素和它进行比较，它已经傲视群峰，就是山脉，得到结果
// 返回这个下标即可
return left;
}

```

```
}
```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 山脉数组的峰顶索引 ( LeetCode 852 ) :https://leetcode-cn.com/problems/peak-index-in-a-mountain-array/
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {

        // 题目告诉我们  $3 \leq arr.length \leq 10^4$ 
        // 也就是说 arr 的最左侧和最右侧肯定不是山峰

        // 所以如果存在山脉，数组中最左侧的元素肯定不是山脉的山峰位置，因为它的左侧没有元素了
        // 因此，搜索区间需要从索引为 1 的位置开始
        int left = 1;

        // 同样的，如果存在山脉，数组中最右侧的元素肯定不是山脉的山峰位置，因为它的右侧没有元素了
        // 因此，搜索区间需要从索引为 nums.length - 2 的位置结束
        int right = arr.size() - 2;

        // 在 while 循环里面，left 不断的 ++，而 right 不断的 --
        // 当 left 和 right 相等，[ left , right ] 这个区间存在一个元素的时候
        // 这也意味着这个区间里面的元素找不到其它元素和它进行比较，它已经傲视群峰，就是山脉，得到结果
        // 所以，当 left == right 时，跳出循环
        // 此时，while 循环的判断不可以可以使用等号
        while(left < right) {

            // left + (right - left) / 2 和 (left + right) / 2 的结果相同
            // 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加的结果溢出的问题
            // 比如数据 int 的最大值为 2,147,483,647
            // 此时，left 为 2,147,483,640
            // 此时，right 为 2,147,483,646
            // 两者直接相加超过了 2,147,483,647，产生了溢出
            int mid = left + (right - left) / 2;

            // 由于题目中提示【题目数据保证 arr 是一个山脉数组】
            // 所以对于 arr 中任意位置的元素，比如 mid 所在的位置，只有以下三种情况

            // 1、mid 处于山峰位置，也就是它的值大于左侧，也大于右侧
            // arr[mid] > arr[mid + 1] && arr[mid] > arr[mid - 1]
            // 此时，找到了结果
            if( arr[mid] > arr[mid - 1] && arr[mid] > arr[mid + 1] ){
```

```

        // 返回结果
        return mid;

    }

    // 2、mid 处于上升状态，也就是它的值大于左侧，同时小于右侧
    // arr[mid] > arr[mid - 1] && arr[mid] < arr[mid + 1]
    // 那么，我们需要在右侧区间去寻找出下降的那段
    if( arr[mid] > arr[mid - 1] && arr[mid] < arr[mid + 1]){

        // 缩小区间，从 [ mid + 1 , right ] 里面去找
        left = mid + 1;

    }

    // 3、mid 处于下降状态，也就是它的值小于左侧，同时小于右侧
    // arr[mid] < arr[mid - 1] && arr[mid] > arr[mid + 1]
    // 那么，我们需要在左侧区间去寻找出上升的那段
    if( arr[mid] < arr[mid - 1] && arr[mid] > arr[mid + 1] ){

        // 缩小区间，从 [ left , mid - 1 ] 里面去找
        right = mid - 1;

    }

}

// 跳出循环，此时 left == right，区间为 [ left, right ]，只有一个元素
// 这也意味着这个区间里面的元素找不到其它元素和它进行比较，它已经傲视群峰，就是山脉，得到结果
// 返回这个下标即可
return left;
}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 山脉数组的峰顶索引 ( LeetCode 852 ) :https://leetcode-cn.com/problems/peak-index-in-a-mountain-array/
class solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:

        # 题目告诉我们 3 <= arr.length <= 10^4
        # 也就是说 arr 的最左侧和最右侧肯定不是山峰

        # 所以如果存在山脉，数组中最左侧的元素肯定不是山脉的山峰位置，因为它的左侧没有元素了

```

```

# 因此，搜索区间需要从索引为 1 的位置开始
left = 1

# 同样的，如果存在山脉，数组中最右侧的元素肯定不是山脉的山峰位置，因为它的右侧没有元素了
# 因此，搜索区间需要从索引为 nums.length - 2 的位置结束
right = len(arr) - 2

# 在 while 循环里面，left 不断的 ++，而 right 不断的 --
# 当 left 和 right 相等，[ left , right ] 这个区间存在一个元素的时候
# 这也意味着这个区间里面的元素找不到其它元素和它进行比较，它已经傲视群峰，就是山脉，得到结果
# 所以，当 left == right 时，跳出循环
# 此时，while 循环的判断不可以使用等号
while left < right :

    # left + (right - left) / 2 和 (left + right) / 2 的结果相同
    # 但是使用 left + (right - left) / 2 可以防止由于 left 、right 同时太大，导致相加
    # 的结果溢出的问题
    # 比如数据 int 的最大值为 2,147,483,647
    # 此时，left 为 2,147,483,640
    # 此时，right 为 2,147,483,646
    # 两者直接相加超过了 2,147,483,647，产生了溢出
    mid = left + (right - left) // 2

    # 由于题目中提示【题目数据保证 arr 是一个山脉数组】
    # 所以对于 arr 中任意位置的元素，比如 mid 所在的位置，只有以下三种情况

    # 1、mid 处于山峰位置，也就是它的值大于左侧，也大于右侧
    # arr[mid] > arr[mid + 1] and arr[mid] > arr[mid - 1]
    # 此时，找到了结果
    if arr[mid] > arr[mid - 1] and arr[mid] > arr[mid + 1] :

        # 返回结果
        return mid

    # 2、mid 处于上升状态，也就是它的值大于左侧，同时小于右侧
    # arr[mid] > arr[mid - 1] and arr[mid] < arr[mid + 1]
    # 那么，我们需要在右侧区间去寻找出下降的那段
    if arr[mid] > arr[mid - 1] and arr[mid] < arr[mid + 1] :

        # 缩小区间，从 [ mid + 1 , right ] 里面去找
        left = mid + 1

    # 3、mid 处于下降状态，也就是它的值小于左侧，同时小于右侧
    # arr[mid] < arr[mid - 1] and arr[mid] > arr[mid + 1]
    # 那么，我们需要在左侧区间去寻找出上升的那段
    if arr[mid] < arr[mid - 1] and arr[mid] > arr[mid + 1] :

```



```
# 缩小区间, 从 [ left , mid - 1 ] 里面去找
right = mid - 1

# 跳出循环, 此时 left == right, 区间为 [ left, right ], 只有一个元素
# 这也意味着这个区间里面的元素找不到其它元素和它进行比较, 它已经傲视群峰, 就是山脉, 得到结果
# 返回这个下标即可
return left
```

第二十七天

LeetCode 1695、删除子数组的最大得分

```
class solution:
    def maximumUniqueSubarray(self, nums: List[int]) -> int:
        # 滑动窗口模板化解题, 五步走策略

        # 【1、定义需要维护的变量】

        # 对于此题来说, 需要维护当前滑动窗口的元素和、滑动过程中得出的最大得分
        # 一开始, 滑动窗口没有元素, 元素和为 0
        sums = 0

        # 由于数组都是正数, 所以可以初始化为 0
        largest = 0

        # 同时又涉及去重, 因此需要一个哈希表
        hash = set()

        # 【2、定义窗口的首尾端 (start, end), 然后滑动窗口】

        # 窗口的左端位置从 0 开始
        start = 0

        # 窗口的右端位置从 0 开始, 可以一直移动到尾部
        for end in range(len(nums)) :

            # 【3、更新需要维护的变量, 有的变量需要一个 if 语句来维护 (比如最大最小长度)】

            # 【4、如果题目的窗口长度可变: 这个时候一般涉及到窗口是否合法的问题】
            # 如果当前窗口不合法时, 用一个 while 去不断移动窗口左指针, 从而剔除非法元素直到窗口再次合法

            # 如果哈希表中存储了即将加入滑动窗口的元素
            while nums[end] in hash :
```

```

# 那么需要不断的把窗口左边的元素移除窗口

# 元素和需要减去移除的值
sums -= nums[start]

# 把 nums[start] 移除记录
hash.remove(nums[start])

# 窗口左端向右移动
start += 1

# 此时，滑动窗口可以接纳 nums[end]
hash.add(nums[end])

# 维护变量 sum
sums += nums[end]

# 维护变量 largest
largest = max(largest, sums)

# 【5、返回所需要的答案】
return largest

```

四、复杂度分析

时间复杂度：O(N)

LeetCode 209、长度最小的子数组

一、题目描述

给定一个含有 n 个正整数的数组和一个正整数 `target` 。

找出该数组中满足其和 $\geq target$ 的长度最小的连续子数组 $[nums_l, nums_l+1, \dots, nums_r-1, nums_r]$ ，并返回其长度。如果不存在符合条件的子数组，返回 0。

示例 1：

输入：target = 7, nums = [2,3,1,2,4,3]
 输出：2
 解释：子数组 [4,3] 是该条件下的长度最小的子数组。

示例 2：

输入：target = 4, nums = [1,4,4]
 输出：1

示例 3：

```
输入: target = 11, nums = [1,1,1,1,1,1,1,1]
输出: 0
```

提示:

- `1 <= target <= 109`
- `1 <= nums.length <= 105`
- `1 <= nums[i] <= 105`

进阶:

- 如果你已经实现 `O(n)` 时间复杂度的解法, 请尝试设计一个 `O(n log(n))` 时间复杂度的解法。

二、题目解析

很经典的滑动窗口的题目。

具体操作如下:

- 1、初始化一个滑动窗口, 左边界位置为 0, 接下来开始设置右边界的位置, 通过数组不断的去调整右边界的位置, 所以一开始右边界的位置也是 0, 如果数组为空, 那么这个窗口就不存在; 如果数组不为空, 那么窗口就是第一个元素。
- 2、计算此时滑动窗口里面所有元素的值。
- 3、如果滑动窗口的元素和小于了 target, 那么说明还没有找到了一个满足条件的连续子数组, 需要继续扩充滑动窗口, 扩充的方式就是让右边界向右移动。
- 4、如果滑动窗口的元素和大于等于了 target, 那么说明找到了一个满足条件的连续子数组, 计算出这个连续子数组的长度来, 把这个结果 result 和之前的保存的结果进行对比取最小值。
- 5、基于 4 的情况, 需要开始尝试缩小滑动窗口, 去寻找更短的滑动窗口, 缩小的方法就是不断的移动左边界, 直到滑动窗口里面的元素和小于了 target 为止, 在移动过程中, 如果发现滑动窗口的元素和依旧大于等于了 target, 继续缩小同时更新 result。
- 6、最后需要注意一点, 一开始, result 可以初始化为一个超过数组长度的值, 这样的目的是为了最后返回结果的时候判断 result 有没有被更新, 如果没有被更新, 并且滑动窗口的长度不可能为 result, 因为超过了数组的长度, 那就代表不存在符合条件的子数组, 需要返回 0, 比如 `target = 11`, `nums = [1,1,1,1,1,1,1,1]`, 先设置 `result = 9`, 执行完后续代码, result 依旧为 9, 代表 nums 里面找不到一个子数组和大于等于 11, 需要返回 0。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 长度最小的子数组 (LeetCode 209) : https://leetcode.cn/problems/minimum-size-subarray-sum/
```

```

class Solution {
    public int minSubArrayLen(int target, int[] nums) {

        // 滑动窗口的左端
        int left = 0;

        // 滑动窗口中所有元素的和
        int sum = 0;

        // 记录滑动窗口的长度，并且不断更新获取最小的那个
        // 一开始，result 可以初始化为一个超过数组长度的值
        // 这样的目的是为了最后返回结果的时候判断 result 有没有被更新
        // 如果没有被更新，并且滑动窗口的长度不可能为 result，因为超过了数组的长度
        // 那就代表不存在符合条件的子数组，需要返回 0
        // 比如 target = 11, nums = [1,1,1,1,1,1,1,1]
        // 先设置 result = 9, 执行完后续代码，result 依旧为 9
        // 代表 nums 里面找不到一个子数组和大于等于 11，需要返回 0
        int result = nums.length + 1;

        // 滑动窗口的右端从 0 开始，这样，当 nums 为空时，可以直接跳出 for 循环
        for (int right = 0; right < nums.length; right++) {

            // 滑动窗口中加入了 nums[right] 这个元素
            // 滑动窗口元素和需要发生变化
            sum += nums[right];

            // 变化之后需要判断一下，如果滑动窗口的元素和大于等于了 target
            // 那么这个时候就需要不断的向右移动 left，缩小滑动窗口的长度
            while (sum >= target) {

                // 在获取到一个满足要求的子数组时，更新 result 的值
                result = Math.min(result, right - left + 1);

                // 把 nums[left] 移除滑动窗口
                sum -= nums[left];

                // 即 left 向右移动
                left++;
            }
        }

        // 返回结果
        return result == nums.length + 1 ? 0 : result;
    }
}

```

2、C++ 代码

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        // 滑动窗口的左端
        int left = 0;

        // 滑动窗口中所有元素的和
        int sum = 0;

        // 记录滑动窗口的长度，并且不断更新获取最小的那个
        // 一开始，result 可以初始化为一个超过数组长度的值
        // 这样的目的是为了最后返回结果的时候判断 result 有没有被更新
        // 如果没有被更新，并且滑动窗口的长度不可能为 result，因为超过了数组的长度
        // 那就代表不存在符合条件的子数组，需要返回 0
        // 比如 target = 11, nums = [1,1,1,1,1,1,1,1]
        // 先设置 result = 9, 执行完后续代码，result 依旧为 9
        // 代表 nums 里面找不到一个子数组和大于等于 11，需要返回 0
        int result = nums.size() + 1;

        // 滑动窗口的右端从 0 开始，这样，当 nums 为空时，可以直接跳出 for 循环
        for (int right = 0; right < nums.size(); right++) {

            // 滑动窗口中加入了 nums[right] 这个元素
            // 滑动窗口元素和需要发生变化
            sum += nums[right];

            // 变化之后需要判断一下，如果滑动窗口的元素和大于等于了 target
            // 那么这个时候就需要不断的向右移动 left，缩小滑动窗口的长度
            while (sum >= target) {

                // 在获取到一个满足要求的子数组时，更新 result 的值
                result = min(result, right - left + 1);

                // 把 nums[left] 移除滑动窗口
                sum -= nums[left];

                // 即 left 向右移动
                left++;
            }
        }

        // 返回结果
        return result == nums.size() + 1 ? 0 : result;
    }
};
```

3、Python 代码

```
class solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        # 滑动窗口的左端
        left = 0

        # 滑动窗口中所有元素的和
        sum = 0

        # 记录滑动窗口的长度，并且不断更新获取最小的那个
        # 一开始，result 可以初始化为一个超过数组长度的值
        # 这样的目的是为了最后返回结果的时候判断 result 有没有被更新
        # 如果没有被更新，并且滑动窗口的长度不可能为 result，因为超过了数组的长度
        # 那就代表不存在符合条件的子数组，需要返回 0
        # 比如 target = 11, nums = [1,1,1,1,1,1,1,1]
        # 先设置 result = 9, 执行完后续代码，result 依旧为 9
        # 代表 nums 里面找不到一个子数组和大于等于 11，需要返回 0
        result = len(nums) + 1

        # 滑动窗口的右端从 0 开始，这样，当 nums 为空时，可以直接跳出 for 循环
        for right in range(len(nums)) :

            # 滑动窗口中加入了 nums[right] 这个元素
            # 滑动窗口元素和需要发生变化
            sum += nums[right]

            # 变化之后需要判断一下，如果滑动窗口的元素和大于等于了 target
            # 那么这个时候就需要不断的向右移动 left，缩小滑动窗口的长度
            while sum >= target :

                # 在获取到一个满足要求的子数组时，更新 result 的值
                result = min(result, right - left + 1)

                # 把 nums[left] 移除滑动窗口
                sum -= nums[left]

                # 即 left 向右移动
                left += 1

        # 返回结果
        return 0 if result == len(nums) + 1 else result
```

四、复杂度分析

时间复杂度： $O(n)$ ，其中 n 是数组的长度。

空间复杂度： $O(1)$ 。

LeetCode 3、无重复字符的最长子串

一、题目描述

给定一个字符串 `s`，请你找出其中不含有重复字符的 **最长子串** 的长度。

示例 1:

输入：s = "abcabcbb"

输出：3

解释：因为无重复字符的最长子串是 "abc"，所以其长度为 3。

示例 2:

输入：s = "bbbbbb"

输出：1

解释：因为无重复字符的最长子串是 "b"，所以其长度为 1。

示例 3:

输入：s = "pwwkew"

输出：3

解释：因为无重复字符的最长子串是 "wke"，所以其长度为 3。

请注意，你的答案必须是 **子串** 的长度，"pwke" 是一个子序列，不是子串。

提示：

- `0 <= s.length <= 5 * 104`
- `s` 由英文字母、数字、符号和空格组成

二、题目解析

很经典的**滑动窗口**的题目。

具体操作如下：

- 1、定义需要维护的变量，对于此题来说，要求是最大长度，同时又涉及去重，因此需要一个哈希表。
- 2、定义窗口的首尾端 (start, end)，然后滑动窗口。
- 3、窗口的右端位置从 0 开始，可以一直移动到尾部。
- 4、如果哈希表中存储了即将加入滑动窗口的元素，那么需要不断的把窗口左边的元素移除窗口。
- 5、此时，滑动窗口可以接纳新增元素。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 无重复字符的最长子串 (LeetCode 3) : https://leetcode.cn/problems/longest-substring-
without-repeating-characters/
class Solution {
    public int lengthOfLongestSubstring(String s) {

        // 滑动窗口模板化解题，五步走策略

        // 【1、定义需要维护的变量】

        // 对于此题来说，要求是最大长度
        int maxLen = 0;

        // 同时又涉及去重，因此需要一个哈希表
        HashSet<Character> hash = new HashSet<Character>();

        // 【2、定义窗口的首尾端 (start, end)， 然后滑动窗口】

        // 窗口的左端位置从 0 开始
        int start = 0;

        // 窗口的右端位置从 0 开始，可以一直移动到尾部
        for( int end = 0 ; end < s.length() ; end++){

            // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

            // 【4、如果题目的窗口长度可变：这个时候一般涉及到窗口是否合法的问题】
            // 如果当前窗口不合法时，用一个 while 去不断移动窗口左指针，从而剔除非法元素直到窗口再
            次合法

            // 如果哈希表中存储了即将加入滑动窗口的元素
            while(hash.contains(s.charAt(end))){

                // 那么需要不断的把窗口左边的元素移除窗口

                // 把 s.charAt(start) 移除记录
                hash.remove(s.charAt(start));

                // 窗口左端向右移动
                start++;
            }
        }
    }
}
```



```

        // 此时，滑动窗口可以接纳 s.charAt(end)
        hash.add(s.charAt(end));

        // 维护变量 maxLen
        maxLen = Math.max(maxLen, end - start + 1);

    }

    // 【5、返回所需要的答案】
    return maxLen;

}

}

```

2、C++ 代码

```

class Solution {
public:
    int lengthOfLongestSubstring(string s) {

        // 滑动窗口模板化解题，五步走策略

        // 【1、定义需要维护的变量】

        // 对于此题来说，要求是最大长度
        int maxLen = 0;

        // 同时又涉及去重，因此需要一个哈希表
        unordered_set<char> hash;

        // 【2、定义窗口的首尾端 (start, end)， 然后滑动窗口】

        // 窗口的左端位置从 0 开始
        int start = 0;

        // 窗口的右端位置从 0 开始，可以一直移动到尾部
        for( int end = 0 ; end < s.length() ; end++){

            // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

            // 【4、如果题目的窗口长度可变：这个时候一般涉及到窗口是否合法的问题】
            // 如果当前窗口不合法时，用一个 while 去不断移动窗口左指针，从而剔除非法元素直到窗口再
            次合法

            // 如果哈希表中存储了即将加入滑动窗口的元素
            while(hash.count(s[end])){

                // 那么需要不断的把窗口左边的元素移除窗口
            }
        }
    }
};

```

```

        // 把 s.charAt(start) 移除记录
        hash.erase(s[start]);

        // 窗口左端向右移动
        start++;
    }

    // 此时，滑动窗口可以接纳 s.charAt(end)
    hash.insert(s[end]);

    // 维护变量 maxLen
    maxLen = max(maxLen, end - start + 1);

}

// 【5、返回所需要的答案】
return maxLen;

}

};

```

3、Python 代码

```

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # 滑动窗口模板化解题，五步走策略

        # 【1、定义需要维护的变量】

        # 对于此题来说，要求是最大长度
        maxLen = 0

        # 同时又涉及去重，因此需要一个哈希表
        hash = set()

        # 【2、定义窗口的首尾端 (start, end)， 然后滑动窗口】

        # 窗口的左端位置从 0 开始
        start = 0

        # 窗口的右端位置从 0 开始，可以一直移动到尾部
        for end in range(len(s)) :

            # 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

            # 【4、如果题目的窗口长度可变：这个时候一般涉及到窗口是否合法的问题】
            # 如果当前窗口不合法时，用一个 while 去不断移动窗口左指针，从而剔除非法元素直到窗口再
            次合法

```

```

# 如果哈希表中存储了即将加入滑动窗口的元素
while s[end] in hash :

    # 那么需要不断的把窗口左边的元素移除窗口

    # 把 s.charAt(start) 移除记录
    hash.remove(s[start])

    # 窗口左端向右移动
    start += 1

# 此时，滑动窗口可以接纳 s.charAt(end)
hash.add(s[end])

# 维护变量 maxLen
maxLen = max(maxLen, end - start + 1)

# 【5、返回所需要的答案】
return maxLen

```

四、复杂度分析

- 时间复杂度：O(N)，其中 N 是字符串的长度。左指针和右指针分别会遍历整个字符串一次。
- 空间复杂度：O(|Σ|)，其中 Σ 表示字符集（即字符串中可以出现的字符），|Σ| 表示字符集的大小。

LeetCode 76、最小覆盖子串

一、题目描述

给你一个字符串 `s`、一个字符串 `t`。返回 `s` 中涵盖 `t` 所有字符的最小子串。如果 `s` 中不存在涵盖 `t` 所有字符的子串，则返回空字符串 `""`。

注意：

- 对于 `t` 中重复字符，我们寻找的子字符串中该字符数量必须不少于 `t` 中该字符数量。
- 如果 `s` 中存在这样的子串，我们保证它是唯一的答案。

示例 1：

```

输入：s = "ADOBECODEBANC", t = "ABC"
输出："BANC"

```

示例 2：

```

输入：s = "a", t = "a"
输出："a"

```

示例 3：

输入: `s = "a", t = "aa"`

输出: `""`

解释: `t` 中两个字符 `'a'` 均应包含在 `s` 的子串中, 因此没有符合条件的子字符串, 返回空字符串。

提示:

- `1 <= s.length, t.length <= 105`
- `s` 和 `t` 由英文字母组成

进阶: 你能设计一个在 `O(n)` 时间内解决此问题的算法吗?

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最小覆盖子串 (LeetCode 76) : https://leetcode.cn/problems/minimum-window-substring/
class Solution {
    public static String minWindow(String s, String t) {

        // 使用整型数组来表示每个字符在 t 中的数量, 初始化都为 0, 利用 ASCII 码的方式把字符存储到整型数组中
        // 数组的长度设置为 128
        // 其中 65 ~ 90 号为 26 个大写英文字母, 97 ~ 122 号为 26 个小写英文字母, 其余为一些标点符号、运算符等
        // 由于 t 是由英文字母组成, 所以数组中有些位置的值始终不会被操作, 造成了空间的浪费, 比如 map[20]map[30]
        // 但这样做方便理解, 比如看到 map[98] = 5, 能知道 b 出现的频次是 5 次
        int[] map = new int[128];

        // 开始统计 t 中每个字符出现的频次
        for (int i = 0; i < t.length(); i++) {
            // 对于数组类型, 其下标为 int 类型
            // 可以直接使用 char 类型变量, 默认强制转换, 存储的就是字母对应的 ASCII 码
            // 比如 t.charAt(i) = 'a', 存储的就是 map[97]
            map[t.charAt(i)]++;
        }

        // 记录滑动窗口的长度, 并且不断更新获取最小的那个
        int windowLength = s.length() + 1;

        // 滑动窗口的左端
        int left = 0;
```

```

// 滑动窗口的右端
int right = 0;

// t 中字符的总个数
int count = t.length();

// 滑动窗口左端新的位置
int start = 0;

// 滑动窗口的右端开始移动
while (right < s.length()) {

    // 获取此时将要加入到滑动窗口的元素
    char c = s.charAt(right);

    // 如果说 map 数组中 c 出现的频次大于了 0，说明此时字符 c 加入到滑动窗口距离找到这样一个子串更近了一步
    // 那么滑动窗口需要搜罗的特定元素个数变少了
    if (map[c] > 0) {
        // 需要搜罗的和 t 中字符一样的元素个数变少了
        count--;
    }

    // 既然滑动窗口中新增了一个字符 c，那么 map 数组中对应的频次就需要减 1
    map[c]--;

    System.out.println( c + " : " + map[c]);

    // 如果此时 count == 0，表明滑动窗口中包含了 t 中全部的字符
    // 此时，找到了一个符合条件的子串
    // 但想尝试一下，能否满足条件的情况下子串更短一些
    // 于是，去尝试把滑动窗口的左端向右移动一下
    // 可以移动的前提是，滑动窗口的左端元素抛弃后剩下的元素依旧满足条件
    // 意思就是实际上左端元素是多余的
    // 而如果这个元素对应的值在 map[] 数组中小于 0，说明它是一个多余元素
    // 反复的删除这些多余的元素
    while(count == 0){

        // 如果当前的这个窗口值比之前维护的窗口值更小，需要进行更新
        if (right - left + 1 < windowLength) {

            // 更新滑动窗口的长度
            windowLength = right - left + 1;

            // 更新滑动窗口起始位置，来到了 left 这个位置
            start = left;
        }
    }
}

```

```

// 接下来左端位置开始向右移动，也就是一个删除操作
// 删除操作需要执行以下三个步骤
// 如果这个元素不是多余的元素，比如滑动窗口为 ADBC，t 为 ABC
// 移除了 A，那么滑动窗口又需要去新增其它的元素了
// 所以通过 map[s.charAt(left)] == 0 来判断它是否是多余的元素
if( map[s.charAt(left)] == 0){
    // 需要搜罗的和 t 中字符一样的元素个数要增加一个了，因为删除了关键元素
    count++;
}

// 2、这个元素离开了滑动窗口，那么在 map 中这个的值就需要加 1
// 对应着上面新增一个元素到滑动窗口，map[c]--
map[s.charAt(left)]++;

// 3、left 向右移动，那么这个元素就离开了
left++;

}

// 可以开始查看新的元素了
right++;

}

// 根据 s 中是否涵盖了 t 所有字符的子串来获取结果
return windowLength == s.length() + 1 ? "" : s.substring(start, start +
windowLength);
}
}

```

2、C++ 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最小覆盖子串 (LeetCode 76) : https://leetcode.cn/problems/minimum-window-substring/
class Solution {
public:
    string minWindow(string s, string t) {
        // 使用整型数组来表示每个字符在 t 中的数量，初始化都为 0，利用 ASCII 码的方式把字符存储到整型数组中
        // 数组的长度设置为 128
        // 其中 65 ~ 90 号为 26 个大写英文字母，97 ~ 122 号为 26 个小写英文字母，其余为一些标点符号、运算符等
        // 由于 t 是由英文字母组成，所以数组中有些位置的值始终不会被操作，造成了空间的浪费，比如 map[20]map[30]
        // 但这样做方便理解，比如看到 map[98] = 5，能知道 b 出现的频次是 5 次
    }
};

```

```

vector<int> map(128, 0);

// 开始统计 t 中每个字符出现的频次
for (auto& c : t){
    // 对于数组类型，其下标为 int 类型
    // 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
    // 比如 t.[i] = 'a', 存储的就是 map[97]
    map[c]++;
}

// 记录滑动窗口的长度，并且不断更新获取最小的那个
int windowLength = s.length() + 1;

// 滑动窗口的左端
int left = 0;

// 滑动窗口的右端
int right = 0;

// t 中字符的总个数
int count = t.length();

// 滑动窗口左端新的位置
int start = 0;

// 滑动窗口的右端开始移动
while (right < s.length()) {

    // 获取此时将要加入到滑动窗口的元素
    auto c = s[right];

    // 如果说 map 数组中 c 出现的频次大于了 0，说明此时字符 c 加入到滑动窗口距离找到这样一个子串更近了一步
    // 那么滑动窗口需要搜罗的特定元素个数变少了
    if (map[c] > 0) {
        // 需要搜罗的和 t 中字符一样的元素个数变少了
        count--;
    }

    // 既然滑动窗口中新增了一个字符 c，那么 map 数组中对应的频次就需要减 1
    map[c]--;

    // System.out.println( c + " : " + map[c]);

    // 如果此时 count == 0，表明滑动窗口中包含了 t 中全部的字符
    // 此时，找到了一个符合条件的子串
    // 但想尝试一下，能否满足条件的情况下子串更短一些
    // 于是，去尝试把滑动窗口的左端向右移动一下

```

```

// 可以移动的前提是，滑动窗口的左端元素抛弃后剩下的元素依旧满足条件
// 意思就是实际上左端元素是多余的
// 而如果这个元素对应的值在 map[] 数组中小于 0，说明它是一个多余元素
// 反复的删除这些多余的元素
while(count == 0){

    // 如果当前的这个窗口值比之前维护的窗口值更小，需要进行更新
    if (right - left + 1 < windowLength) {

        // 更新滑动窗口的长度
        windowLength = right - left + 1;

        // 更新滑动窗口起始位置，来到了 left 这个位置
        start = left;
    }

    // 接下来左端位置开始向右移动，也就是一个删除操作
    // 删除操作需要执行以下三个步骤
    // 如果这个元素不是多余的元素，比如滑动窗口为 ADBC，t 为 ABC
    // 移除了 A，那么滑动窗口又需要去新增其它的元素了
    // 所以通过 map[s[left]] == 0 来判断它是否是多余的元素
    if( map[s[left]] == 0){
        // 需要搜罗的和 t 中字符一样的元素个数要增加一个了，因为删除了关键元素
        count++;
    }

    // 2、这个元素离开了滑动窗口，那么在 map 中这个的值就需要加 1
    // 对应着上面新增一个元素到滑动窗口，map[c]--
    map[s[left]]++;

    // 3、left 向右移动，那么这个元素就离开了
    left++;

}

// 可以开始查看新的元素了
right++;

}

// 根据 s 中是否涵盖了 t 所有字符的子串来获取结果
return windowLength == s.length() + 1 ? "" : s.substr(start, windowLength);

}
};

```


3、Python 代码

```
class solution:
    def minwindow(self, s: str, t: str) -> str:
        # 使用整型数组来表示每个字符在 t 中的数量，初始化都为 0，利用 ASCII 码的方式把字符存储到整型
        # 数组中
        # 数组的长度设置为 128
        # 其中 65 ~ 90 号为 26 个大写英文字母，97 ~ 122 号为 26 个小写英文字母，其余为一些标点符
        # 号、运算符等
        # 由于 t 是由英文字母组成，所以数组中有些位置的值始终不会被操作，造成了空间的浪费，比如
        map[20]map[30]
        # 但这样做方便理解，比如看到 map[98] = 5，能知道 b 出现的频次是 5 次
        map = collections.defaultdict(int)

        # 开始统计 t 中每个字符出现的频次
        for ch in t:    #初始化需要的必要字符数量
            map[ch] += 1

        # 记录滑动窗口的长度，并且不断更新获取最小的那个
        windowLength = len(s) + 1

        # 滑动窗口的左端
        left = 0

        # 滑动窗口的右端
        right = 0

        # t 中字符的总个数
        count = len(t)

        # 滑动窗口左端新的位置
        start = 0

        # 滑动窗口的右端开始移动
        while right < len(s) :

            # 获取此时将要加入到滑动窗口的元素
            c = s[right]

            # 如果说 map 数组中 c 出现的频次大于了 0，说明此时字符 c 加入到滑动窗口距离找到这样一个
            # 子串更近了一步
            # 那么滑动窗口需要搜罗的特定元素个数变少了
            if map[c] > 0 :
                # 需要搜罗的和 t 中字符一样的元素个数变少了
                count -= 1

            # 既然滑动窗口中新增了一个字符 c，那么 map 数组中对应的频次就需要减 1
            map[c] -= 1
```

```

# 如果此时 count == 0 , 表明滑动窗口中包含了 t 中全部的字符
# 此时, 找到了一个符合条件的子串
# 但想尝试一下, 能否满足条件的情况下子串更短一些
# 于是, 去尝试把滑动窗口的左端向右移动一下
# 可以移动的前提是, 滑动窗口的左端元素抛弃后剩下的元素依旧满足条件
# 意思就是实际上左端元素是多余的
# 而如果这个元素对应的值在 map[] 数组中小于 0, 说明它是一个多余元素
# 反复的删除这些多余的元素
while count == 0 :

    # 如果当前的这个窗口值比之前维护的窗口值更小, 需要进行更新
    if right - left + 1 < windowLength :

        # 更新滑动窗口的长度
        windowLength = right - left + 1

        # 更新滑动窗口起始位置, 来到了 left 这个位置
        start = left

    # 接下来左端位置开始向右移动, 也就是一个删除操作
    # 删除操作需要执行以下三个步骤
    # 如果这个元素不是多余的元素, 比如滑动窗口为 ADBC, t 为 ABC
    # 移除了 A, 那么滑动窗口又需要去新增其它的元素了
    # 所以通过 map[s.charAt(left)] == 0 来判断它是否是多余的元素
    if map[s[left]] == 0 :
        # 需要搜罗的和 t 中字符一样的元素个数要增加一个了, 因为删除了关键元素
        count += 1

    # 2、这个元素离开了滑动窗口, 那么在 map 中这个的值就需要加 1
    # 对应着上面新增一个元素到滑动窗口, map[c]--
    map[s[left]] += 1

    # 3、left 向右移动, 那么这个元素就离开了
    left += 1

# 可以开始查看新的元素了
right += 1

# 根据 s 中是否涵盖了 t 所有字符的子串来获取结果

```

```
return '' if windowLength == (len(s) + 1) else s[start:start + windowLength]
```

四、复杂度分析

时间复杂度：两个指针都严格递增，最多移动 n 次，所以总时间复杂度是 $O(n)$ 。

空间复杂度： $O(1)$

第二十八天

LeetCode 239、滑动窗口最大值

一、题目描述

给定一个数组 `nums` 和滑动窗口的大小 `k`，请找出所有滑动窗口里的最大值。

示例：

```
输入：nums = [1,3,-1,-3,5,3,6,7]，和 k = 3
输出：[3,3,5,5,6,7]
```

解释：

提示：

- 你可以假设 `k` 总是有效的，在输入数组不为空的情况下， $1 \leq k \leq$ 输入数组的大小。

二、题目解析

滑动窗口这个词包含两个概念，一个是滑动，一个是窗口。

首先是窗口需要生成，一开始里面是没有任何元素的。

然后再滑动，随着窗口不断的滑动，窗口里面的元素个数从 0 到 1，再到 2，再到 3，此时，从窗口里面宣传最大值。

想要找出当前窗口里面的最大值，自然而然的想法就是遍历窗口中的所有元素，从中选出最大值，这样的复杂度是 $O(k*n)$ 级别，复杂度有点高。

首先滑动窗口滑过所有的元素必然要经历 $O(n)$ 的时间，这没法调整，所以可以优化的方向在于获取当前窗口的最大值，即想办法从 $O(k)$ 优化到 $O(\log k)$ 或者直接优化到 $O(1)$ 。

使用双端队列！

让窗口移动的过程，维护好队列里面的元素，做到每次窗口移动后都能马上知道当前窗口的最大值，由于想要做到 $O(1)$ 的级别拿到最大值，那么必须是它的队首始终是最大值，也就是说我们需要维护一个递减队列用来保存队列中所有递减的元素。

一开始，窗口中只有 1，队列中放入 1。

窗口滑动，包含了 1 和 3，3 大于 1，如果直接放入队列，队列为 1 3，不是递减队列，所以需要先移除 1 再放入 3。

继续滑动，窗口元素为 1 3 -1。

滑动窗口已经有三个元素，是合格的窗口，获取它的最大值只需要获取队列的队首元素就行。

同样的，窗口不断的向右移动，每次窗口都会增加新的元素，为了让队列中的队首元素始终是当前窗口的最大值，需要把队列中所有小于新元素值的那些元素移除。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 滑动窗口最大值（LeetCode 239）：https://leetcode-cn.com/problems/sliding-window-maximum/
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        // 边界情况
        if(nums.length == 0 || k == 0){
            return new int[0];
        }
        // 构建双端队列
        Deque<Integer> deque = new LinkedList<>();
        // 构建存储最大值的数组
        int[] res = new int[nums.length - k + 1];
        // 一开始滑动窗口不包含 k 个元素，不是合格的滑动窗口
        for(int i = 0; i < k; i++) {
            // 在滑动过程中，维护好 deque，确保它是单调递减队列

            // 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。
            // 直到考察元素可以放入到队列中
            while(!deque.isEmpty() && deque.peekLast() < nums[i]){
                deque.removeLast();
            }

            // 考察元素可以放入到队列中
            deque.addLast(nums[i]);
        }
        // 这个时候，滑动窗口刚刚好有 k 个元素，是合格的滑动窗口，那么最大值就是队列中的队首元素
```

```

res[0] = deque.peekFirst();

// 现在让滑动窗口滑动
for(int i = k; i < nums.length; i++) {
    // 滑动窗口已经装满了元素，向右移动会把窗口最左边的元素抛弃
    // i - k 为滑动窗口的最左边
    // 如果队列的队首元素和窗口最左边的元素相等，需要将队首元素抛出
    // 如果不写这个判断，会导致队列中包含非当前窗口的元素
    // 比如窗口大小为 1，队列为 1 -1，此时窗口为 【 1 】，队列为 1，输出最大值 1，下一个窗口为 【 -1 】，准备移动的时候队列 1 和数组最左端元素一样，必须移除，否则队列中是 【 1, -1 】，输出的结果是 1，而 1 不在窗口 【 -1 】中
    if(deque.peekFirst() == nums[i - k]){
        deque.removeFirst();
    }

    // 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。
    // 直到考察元素可以放入到队列中
    while(!deque.isEmpty() && deque.peekLast() < nums[i]){
        deque.removeLast();
    }

    // 考察元素可以放入到队列中
    deque.addLast(nums[i]);
    // 此时，结果数组的值就是队列的队首元素
    res[i - k + 1] = deque.peekFirst();
}

// 最后返回 res
return res;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 滑动窗口最大值（LeetCode 239）：https://leetcode-cn.com/problems/sliding-window-maximum/
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        // 构建存储最大值的数组
        vector<int> res;

        if(nums.size() == 0 || k == 0){

```

```

        return res;
    }

    // 构建双端队列
    deque<int> deque;

    // 一开始滑动窗口不包含 k 个元素，不是合格的滑动窗口
    for(int i = 0; i < k; i++) {
        // 在滑动过程中，维护好 deque，确保它是单调递减队列

        // 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。
        // 直到考察元素可以放入到队列中
        while(!deque.empty() && deque.back() < nums[i]){
            deque.pop_back();
        }
        // 考察元素可以放入到队列中
        deque.push_back(nums[i]);
    }
    // 这个时候，滑动窗口刚刚好有 k 个元素，是合格的滑动窗口，那么最大值就是队列中的队首元素
    res.push_back(deque.front());
    // 现在让滑动窗口滑动
    for(int i = k; i < nums.size(); i++) {
        // 滑动窗口已经装满了元素，向右移动会把窗口最左边的元素抛弃
        // i - k 为滑动窗口的最左边
        // 如果队列的队首元素和窗口最左边的元素相等，需要将队首元素抛出
        // 如果不写这个判断，会导致队列中会包含非当前窗口的元素
        // 比如窗口大小为 1，队列为 1 -1，此时窗口为 【 1 】，队列为 1，输出最大值 1，下一个窗口为 【 -1 】，准备移动的时候队列 1 和数组最左端元素一样，必须移除，否则队列中是 【 1, -1 】，输出的结果是 1，而 1 不在窗口 【 -1 】中
        if(deque.front() == nums[i - k]){
            deque.pop_front();
        }
        // 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。
        // 直到考察元素可以放入到队列中
        while(!deque.empty() && deque.back() < nums[i]){
            deque.pop_back();
        }

        // 考察元素可以放入到队列中
        deque.push_back(nums[i]);
        // 此时，结果数组的值就是队列的队首元素
        res.push_back(deque.front());
    }
    // 最后返回 res
    return res;
}
};

```

3、Python 代码 1

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

滑动窗口最大值 (LeetCode 239) : <https://leetcode-cn.com/problems/sliding-window-maximum/>

class Solution:

def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:

获取数组长度

n = len(nums)

构建双端队列

q = collections.deque()

建存储最大值的数组

res = []

边界情况

if k < 1:

return res

一开始滑动窗口不包含 k 个元素，不是合格的滑动窗口

for i in range(k):

在滑动过程中，维护好 deque，确保它是单调递减队列

反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。

直到考察元素可以放入到队列中

while q and q[-1] < nums[i]:

q.pop()

考察元素可以放入到队列中

q.append(nums[i])

这个时候，滑动窗口刚好有 k 个元素，是合格的滑动窗口，那么最大值就是队列中的队首元素

res.append(q[0])

现在让滑动窗口滑动

for i in range(k, n):

滑动窗口已经装满了元素，向右移动会把窗口最左边的元素抛弃

i - k 为滑动窗口的最左边

如果队列的队首元素和窗口最左边的元素相等，需要将队首元素抛出

如果不写这个判断，会导致队列中会包含非当前窗口的元素

比如窗口大小为 1，队列为 1 -1，此时窗口为 【 1 】，队列为 1，输出最大值 1，下一个窗口为 【 -1 】，准备移动的时候队列 1 和数组最左端元素一样，必须移除，否则队列中是 【 1, -1 】，输出的结果是 1，而 1 不在窗口 【 -1 】中

while q and nums[q[-1]] <= nums[i]:

q.pop()

if q[0] == nums[i-k]:

q.popleft()

反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。

```

        # 直到考察元素可以放入到队列中
        while q and q[-1] < nums[i]:
            q.pop()

        q.append(nums[i])

        # 考察元素可以放入到队列中
        res.append(q[0])
    # 最后返回 res
    return res

```

3、Python 代码 2

添加索引的方式

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

滑动窗口最大值 (LeetCode 239) : <https://leetcode-cn.com/problems/sliding-window-maximum/>

class Solution:

```

    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:

```

```

        # 获取数组长度

```

```

        n = len(nums)

```

```

        # 构建双端队列，存储的是 nums 数组中元素的下标

```

```

        q = collections.deque()

```

```

        # 建存储最大值的数组

```

```

        res = []

```

```

        # 边界情况

```

```

        if k < 1:

```

```

            return res

```

```

        # 一开始滑动窗口不包含 k 个元素，不是合格的滑动窗口

```

```

        for i in range(k):

```

```

            # 在滑动过程中，维护好 deque，确保它是单调递减队列

```

```

            # 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。

```

```

            # 直到考察元素可以放入到队列中

```

```

            # q[-1] 表示队尾元素下标，nums[q[-1]] 则是获取这个下标的元素

```

```

            while q and nums[i] >= nums[q[-1]] :

```

```

                q.pop()

```

```

            # 考察元素的下标可以放入到队列中

```

```

            q.append(i)

```

```

        # 这个时候，滑动窗口刚好有 k 个元素，是合格的滑动窗口，那么最大值就是队列中的队首元素

```

```

        # q[-1] 表示队首元素下标，nums[q[0]] 则是获取这个下标的元素

```

```

        res.append(nums[q[0]])

```



```

# 现在让滑动窗口滑动
for i in range(k, n):
    # 滑动窗口已经装满了元素，向右移动会把窗口最左边的元素抛弃
    # i - k 为滑动窗口的最左边
    # 如果队列的队首元素和窗口最左边的元素相等，需要将队首元素抛出
    # 如果不写这个判断，会导致队列中会包含非当前窗口的元素
    # 比如窗口大小为 1，队列为 1 -1，此时窗口为 【 1 】，队列为 1，输出最大值 1，下一个窗口
    # 为 【 -1 】，准备移动的时候队列 1 和数组最左端元素一样，必须移除，否则队列中是 【 1, -1 】，输出的结果
    # 是 1，而 1 不在窗口 【 -1 】中
    # q 存储的是下标
    while q and q[0] <= i-k:
        q.popleft()

    # 反复判断，如果队列不为空且当前考察元素大于等于队尾元素，则将队尾元素移除。
    # 直到考察元素可以放入到队列中
    while q and nums[q[-1]] <= nums[i]:
        q.pop()

    q.append(i)

    # 考察元素可以放入到队列中
    res.append(nums[q[0]])
# 最后返回 res
return res

```

LeetCode 438、找到字符串中所有字母异位词

一、题目描述

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

异位词 指由相同字母重排列形成的字符串（包括相同的字符串）。

示例 1:

输入: `s = "cbaebabacd"`, `p = "abc"`

输出: `[0,6]`

解释:

起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。

示例 2:

输入: `s = "abab"`, `p = "ab"`

输出: `[0,1,2]`

解释:

起始索引等于 0 的子串是 "ab", 它是 "ab" 的异位词。

起始索引等于 1 的子串是 "ba", 它是 "ab" 的异位词。

起始索引等于 2 的子串是 "ab", 它是 "ab" 的异位词。

提示:

- `1 <= s.length, p.length <= 3 * 10^4`
- `s` 和 `p` 仅包含小写字母

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 找到字符串中所有字母异位词 (LeetCode 438) :https://leetcode.cn/problems/find-all-anagrams-in-a-string/
class Solution {
    public List<Integer> findAnagrams(String s, String p) {

        // 滑动窗口模板化解题, 五步走策略

        // 【1、定义需要维护的变量】

        // 题目要求返回这些子串的起始索引, 因此使用数组来保存结果, 存储的是整型
        List<Integer> res = new ArrayList<>();

        // 由于都是小写字母, 因此直接用 26 个长度的数组代替原来的 HashMap, 比直接使用 HashMap 更
        // 优秀
        // needs 代表 p 中每个字符出现的频次
        int[] needs = new int[26];

        for(char ch : p.toCharArray()){
            // 对于数组类型, 其下标为 int 类型
            // 可以直接使用 char 类型变量, 默认强制转换, 存储的就是字母对应的 ASCII 码
            // 比如 ch 是 b 字符, 那么 b - a = 1, 即 needs[1] 表示记录 b 出现的频次
            needs[ch - 'a']++;
        }

        // window 代表在滑动窗口中每个字符出现的频次
```

```

int[] window = new int[26];

// 【2、定义窗口的首尾端 (start, end), 然后滑动窗口】

// 窗口的左端位置从 0 开始
int start = 0;

// 窗口的右端位置从 0 开始, 可以一直移动到尾部
for(int end = 0; end < s.length(); end++){

    // 【3、更新需要维护的变量, 有的变量需要一个 if 语句来维护 (比如最大最小长度)】

    // 获取此时将要加入到滑动窗口的元素
    char cur = s.charAt(end);

    // 这个字符的频次需要发生变化
    window[cur - 'a'] ++;

    // 加入之后, 去对比一下 window 中 cur 这个字符是否满足要求
    if(isSame(window, needs)){

        // 相同情况下, 找到一个符合条件的索引
        res.add(start);
    }

    // 【4、如果题目的窗口长度固定: 用一个 if 语句判断一下当前窗口长度是否达到了限定长度 】

    // 4.1、如果达到了, 窗口左指针前移一个单位, 从而保证下一次右指针右移时, 窗口长度保持不变,
    if( end >= p.length() - 1 ){

        // 4.2、更新 (部分或所有) 维护变量
        char chr = s.charAt(start);

        // 最左端那个元素的频次发生变化
        window[chr - 'a']--;

        // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
        start++;

    }

}

// 【5、返回所需要的答案】
return res;
}

boolean isSame (int[] a, int [] b){

```

```

        for(int i = 0; i < a.length; i++){

            if(a[i] != b[i])return false;

        }
        return true;
    }
}

```

2、C++ 代码

```

class Solution {
public:
    vector<int> findAnagrams(string s, string p) {

        // 滑动窗口模板化解题，五步走策略

        // 【1、定义需要维护的变量】

        // 题目要求返回这些子串的起始索引，因此使用数组来保存结果，存储的是整型
        vector<int> res;

        // 由于都是小写字母，因此直接用 26 个长度的数组代替原来的 HashMap，比直接使用 HashMap 更
        // needs 代表 p 中每个字符出现的频次
        vector<int> needs(26, 0);

        for(auto& ch : p){
            // 对于数组类型，其下标为 int 类型
            // 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
            // 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
            needs[ch - 'a']++;
        }

        // window 代表在滑动窗口中每个字符出现的频次
        vector<int> window(26, 0);

        // 【2、定义窗口的首尾端 (start, end)，然后滑动窗口】

        // 窗口的左端位置从 0 开始
        int start = 0;

        // 窗口的右端位置从 0 开始，可以一直移动到尾部
        for(int end = 0; end < s.size(); end++){

            // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护 (比如最大最小长度)】

```

```

// 获取此时将要加入到滑动窗口的元素
auto cur = s[end];

// 这个字符的频次需要发生变化
window[cur - 'a']++;

// 加入之后，去对比一下 window 中 cur 这个字符是否满足要求
if(window == needs){

    // 相同情况下，找到一个符合条件的索引
    res.push_back(start);
}

// 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度】

// 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
if( end >= p.size() - 1 ){

    // 4.2、更新（部分或所有）维护变量
    auto chr = s[start];

    // 最左端那个元素的频次发生变化
    window[chr - 'a']--;

    // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
    start++;

}

}

// 【5、返回所需要的答案】
return res;
}
};

```

3、Python 代码

```

class solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        # 滑动窗口模板化解题，五步走策略

        # 【1、定义需要维护的变量】

        # 题目要求返回这些子串的起始索引，因此使用数组来保存结果，存储的是整型
        res = []

```

秀

```
# 由于都是小写字母，因此直接用 26 个长度的数组代替原来的 HashMap，比直接使用 HashMap 更优

# needs 代表 p 中每个字符出现的频次
needs = [0] * 26

# 开始统计 t 中每个字符出现的频次
for ch in p:
    # 对于数组类型，其下标为 int 类型
    # 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
    # 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
    idx = ord(ch) - ord('a')
    needs[idx] += 1

# window 代表在滑动窗口中每个字符出现的频次
window = [0] * 26

# 【2、定义窗口的首尾端 (start, end)，然后滑动窗口】

# 窗口的左端位置从 0 开始
start = 0

# 窗口的右端位置从 0 开始，可以一直移动到尾部
for end in range(len(s)):

    # 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

    # 获取此时将要加入到滑动窗口的元素
    cur = s[end]

    # 这个字符的频次需要发生变化
    window[ord(cur) - ord('a')] += 1

    # 加入之后，去对比一下 window 中 cur 这个字符是否满足要求
    if window == needs:

        # 相同情况下，找到一个符合条件的索引
        res.append(start)

# 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度】

# 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
if end >= len(p) - 1:

    # 4.2、更新（部分或所有）维护变量
    chr = s[start]

    # 最左端那个元素的频次发生变化
```

```
        window[ord(chr) - ord('a')] -= 1

        # 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
        start += 1

    # 【5、返回所需要的答案】
    return res
```

LeetCode 567、字符串的排列

一、题目描述

给你两个字符串 `s1` 和 `s2`，写一个函数来判断 `s2` 是否包含 `s1` 的排列。如果是，返回 `true`；否则，返回 `false`。

换句话说，`s1` 的排列之一是 `s2` 的子串。

示例 1:

```
输入: s1 = "ab" s2 = "eidbaooo"
输出: true
解释: s2 包含 s1 的排列之一 ("ba").
```

示例 2:

```
输入: s1= "ab" s2 = "eidboaoo"
输出: false
```

提示:

- `1 <= s1.length, s2.length <= 104`
- `s1` 和 `s2` 仅包含小写字母

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
//
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        // 滑动窗口模板化解题，五步走策略
```

优秀

```
// 【1、定义需要维护的变量】

// 题目要求返回这些子串的起始索引，因此使用数组来保存结果，存储的是整型
List<Integer> res = new ArrayList<>();

// 由于都是小写字母，因此直接用 26 个长度的数组代替原来的 HashMap，比直接使用 HashMap 更

// needs 代表 p 中每个字符出现的频次
int[] needs = new int[26];

for(char ch : s1.toCharArray()){
    // 对于数组类型，其下标为 int 类型
    // 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
    // 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
    needs[ch - 'a']++;
}

// window 代表在滑动窗口中每个字符出现的频次
int[] window = new int[26];

// 【2、定义窗口的首尾端 (start, end)，然后滑动窗口】

// 窗口的左端位置从 0 开始
int start = 0;

// 窗口的右端位置从 0 开始，可以一直移动到尾部
for(int end = 0; end < s2.length(); end++){

    // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

    // 获取此时将要加入到滑动窗口的元素
    char cur = s2.charAt(end);

    // 这个字符的频次需要发生变化
    window[cur - 'a'] ++;

    // 加入之后，去对比一下 window 中 cur 这个字符是否满足要求
    if(isSame(window,needs)){

        // 相同情况下，找到了满足条件的一个解，返回 true
        return true;
    }

    // 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度】

    // 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
```



```

        if( end >= s1.length() - 1 ){

            // 4.2、更新（部分或所有）维护变量
            char chr = s2.charAt(start);

            // 最左端那个元素的频次发生变化
            window[chr - 'a']--;

            // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
            start++;

        }

    }

    // 【5、返回所需要的答案】
    return false;
}

boolean isSame (int[] a, int [] b){

    for(int i = 0; i < a.length; i++){

        if(a[i] != b[i])return false;

    }
    return true;
}

}

```

2、C++ 代码

```

class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        // 滑动窗口模板化解题，五步走策略

        // 【1、定义需要维护的变量】

        // 题目要求返回这些子串的起始索引，因此使用数组来保存结果，存储的是整型
        vector<int> res;

        // 由于都是小写字母，因此直接用 26 个长度的数组代替原来的 HashMap，比直接使用 HashMap 更
        // needs 代表 p 中每个字符出现的频次
        vector<int> needs(26, 0);
    }
}

```

优秀

```

for(auto& ch : s1){
    // 对于数组类型，其下标为 int 类型
    // 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
    // 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
    needs[ch - 'a']++;
}

// window 代表在滑动窗口中每个字符出现的频次
vector<int> window(26, 0);

// 【2、定义窗口的首尾端 (start, end)，然后滑动窗口】

// 窗口的左端位置从 0 开始
int start = 0;

// 窗口的右端位置从 0 开始，可以一直移动到尾部
for(int end = 0; end < s2.size(); end++){

    // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

    // 获取此时将要加入到滑动窗口的元素
    auto cur = s2[end];

    // 这个字符的频次需要发生变化
    window[cur - 'a']++;

    // 加入之后，去对比一下 window 中 cur 这个字符是否满足要求
    if(window == needs){

        // 相同情况下，找到了满足条件的一个解，返回 true
        return true;
    }

    // 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度】

    // 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
    if( end >= s1.size() - 1 ){

        // 4.2、更新（部分或所有）维护变量
        auto chr = s2[start];

        // 最左端那个元素的频次发生变化
        window[chr - 'a']--;

        // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
        start++;
    }
}

```

```

    }

}

// 【5、返回所需要的答案】
return false;

}

};

```

3、Python 代码

```

class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        # 滑动窗口模板化解题，五步走策略

        # 【1、定义需要维护的变量】

        # 题目要求返回这些子串的起始索引，因此使用数组来保存结果，存储的是整型
        res = []

        # 由于都是小写字母，因此直接用 26 个长度的数组代替原来的 HashMap，比直接使用 HashMap 更优
        # needs 代表 p 中每个字符出现的频次
        needs = [0] * 26

        # 开始统计 t 中每个字符出现的频次
        for ch in s1:
            # 对于数组类型，其下标为 int 类型
            # 可以直接使用 char 类型变量，默认强制转换，存储的就是字母对应的 ASCII 码
            # 比如 ch 是 b 字符，那么 b - a = 1，即 needs[1] 表示记录 b 出现的频次
            idx = ord(ch) - ord('a')
            needs[idx] += 1

        # window 代表在滑动窗口中每个字符出现的频次
        window = [0] * 26

        # 【2、定义窗口的首尾端（start，end），然后滑动窗口】

        # 窗口的左端位置从 0 开始
        start = 0

        # 窗口的右端位置从 0 开始，可以一直移动到尾部
        for end in range(len(s2)):

            # 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

            # 获取此时将要加入到滑动窗口的元素
            cur = s2[end]

```

```

# 这个字符的频次需要发生变化
window[ord(cur) - ord('a')] += 1

# 加入之后，去对比一下 window 中 cur 这个字符是否满足要求
if window == needs:

    # 相同情况下，找到一个符合条件的索引
    return True

# 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度】

# 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
if end >= len(s1) - 1 :

    # 4.2、更新（部分或所有）维护变量
    chr = s2[start]

    # 最左端那个元素的频次发生变化
    window[ord(chr) - ord('a')] -= 1

# 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
start += 1

# 【5、返回所需要的答案】
return False

```

LeetCode 643、子数组最大平均数 I

一、题目描述

给你一个由 n 个元素组成的整数数组 `nums` 和一个整数 `k`。

请你找出平均数最大且 长度为 `k` 的连续子数组，并输出该最大平均数。

任何误差小于 10^{-5} 的答案都将被视为正确答案。

示例 1：

输入：nums = [1,12,-5,-6,50,3]，k = 4

输出：12.75

解释：最大平均数 $(12-5-6+50)/4 = 51/4 = 12.75$

示例 2：

输入: nums = [5], k = 1
输出: 5.00000

提示:

- `n == nums.length`
- `1 <= k <= n <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

二、题目解析

很经典的滑动窗口的题目。

具体操作如下:

- 1、定义需要维护的变量, 即元素和、最大平均数。
- 2、定义窗口的首尾端 (start, end), 然后滑动窗口。
- 3、窗口的右端位置从 0 开始, 可以一直移动到尾部。
- 4、更新需要维护的变量, 维护元素和的值, 窗口右端新增一个元素进来, 元素和发生变化。
- 5、如果窗口的长度为 k, 需要计算平均数。
- 6、如果题目的窗口长度固定: 用一个 if 语句判断一下当前窗口长度是否达到了限定长度, 如果达到了, 窗口左指针前移一个单位, 从而保证下一次右指针右移时, 窗口长度保持不变。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 子数组最大平均数 I (LeetCode 643) : https://leetcode.cn/problems/maximum-average-subarray-i/
class Solution {
    public double findMaxAverage(int[] nums, int k) {

        // 滑动窗口模板化解题, 五步走策略

        // 【1、定义需要维护的变量】

        // 对于此题来说, 需要维护当前滑动窗口的元素和、滑动过程中得出的最大平均数
        // 一开始, 滑动窗口没有元素, 元素和为 0
        double sums = 0;

        // 由于数组存在负数, 所以 largest 设置为一个最小值
        double largest = Integer.MIN_VALUE;
```

```

// 【2、定义窗口的首尾端 (start, end), 然后滑动窗口】

// 窗口的左端位置从 0 开始
int start = 0;

// 窗口的右端位置从 0 开始, 可以一直移动到尾部
for( int end = 0 ; end < nums.length ; end++ ){

    // 【3、更新需要维护的变量, 有的变量需要一个 if 语句来维护 (比如最大最小长度)】

    // 维护元素和的值, 窗口右端新增一个元素进来, 元素和发生变化
    sums += nums[end];

    // if 语句
    // 如果窗口的长度为 k, 需要计算平均数
    if( end - start + 1 == k ){

        // 维护 largest 变量
        largest = Math.max( largest, sums / k );

    }

    // 【4、如果题目的窗口长度固定: 用一个 if 语句判断一下当前窗口长度是否达到了限定长度 】

    // 4.1、如果达到了, 窗口左指针前移一个单位, 从而保证下一次右指针右移时, 窗口长度保持不变,
    if( end >= k - 1 ){

        // 4.2、更新 (部分或所有) 维护变量
        sums -= nums[start];

        // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
        start++;

    }

}

// 【5、返回所需要的答案】
return largest;
}
}

```

2、C++ 代码

```

class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {

        // 滑动窗口模板化解题, 五步走策略
    }
}

```

```

// 【1、定义需要维护的变量】

// 对于此题来说，需要维护当前滑动窗口的元素和、滑动过程中得出的最大平均数
// 一开始，滑动窗口没有元素，元素和为 0
double sums = 0;

// 由于数组存在负数，所以 largest 设置为一个最小值
double largest = INT_MIN;

// 【2、定义窗口的首尾端 (start, end)， 然后滑动窗口】

// 窗口的左端位置从 0 开始
int start = 0;

// 窗口的右端位置从 0 开始，可以一直移动到尾部
for( int end = 0 ; end < nums.size() ; end++){

    // 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

    // 维护元素和的值，窗口右端新增一个元素进来，元素和发生变化
    sums += nums[end];

    // if 语句
    // 如果窗口的长度为 k，需要计算平均数
    if( end - start + 1 == k ){

        // 维护 largest 变量
        largest = max( largest, sums / k );

    }

    // 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度 】

    // 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
    if( end >= k - 1 ){

        // 4.2、更新（部分或所有）维护变量
        sums -= nums[start];

        // 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
        start++;

    }

}

// 【5、返回所需要的答案】
return largest;

}

};

```

3、Python 代码

```
class solution:
    def findMaxAverage(self, nums: List[int], k: int) -> float:
        # 滑动窗口模板化解题，五步走策略

        # 【1、定义需要维护的变量】

        # 对于此题来说，需要维护当前滑动窗口的元素和、滑动过程中得出的最大平均数
        # 一开始，滑动窗口没有元素，元素和为 0
        sums = 0

        # 由于数组存在负数，所以 largest 设置为一个最小值
        largest = float('-inf')

        # 【2、定义窗口的首尾端 (start, end)， 然后滑动窗口】

        # 窗口的左端位置从 0 开始
        start = 0

        # 窗口的右端位置从 0 开始，可以一直移动到尾部
        for end in range(len(nums)) :

            # 【3、更新需要维护的变量，有的变量需要一个 if 语句来维护（比如最大最小长度）】

            # 维护元素和的值，窗口右端新增一个元素进来，元素和发生变化
            sums += nums[end]

            # if 语句
            # 如果窗口的长度为 k，需要计算平均数
            if end - start + 1 == k :

                # 维护 largest 变量
                largest = max( largest, sums / k )

            # 【4、如果题目的窗口长度固定：用一个 if 语句判断一下当前窗口长度是否达到了限定长度 】

            # 4.1、如果达到了，窗口左指针前移一个单位，从而保证下一次右指针右移时，窗口长度保持不变，
            if end >= k - 1 :

                # 4.2、更新（部分或所有）维护变量
                sums -= nums[start]

                # 4.3、窗口左指针前移一个单位保证下一次右指针右移时窗口长度保持不变
                start += 1

            # 【5、返回所需要的答案】
            return largest
```


四、复杂度分析

- 时间复杂度： $O(n)$ ，其中 n 是数组 `nums` 的长度。遍历数组一次。
- 空间复杂度： $O(1)$ 。

第二十九天

LeetCode 206、反转链表

一、题目描述

给你单链表的头节点 `head`，请你反转链表，并返回反转后的链表。

二、题目解析

这道题目有很多种解法，递归解法最难以理解整个过程，今天我就带大家来探究一次！

先来看分析过程，这里看不懂没关系，一定要看最后的动画过程。

- 1、通过递归函数，一直递归到链表的最后一个结点为止，此时，该结点就是反转成功后的头结点，是最终的返回结果。
- 2、在递归函数中，让当前节点的下一个节点的 `next` 指针指向当前节点。
- 3、在递归函数中，让当前节点的 `next` 指针指向 `null`
- 4、通过二三步的操作，已经让递归函数中的链表实现了局部反转，将结果返回给上一层递归函数
- 5、所有递归结束后，链表反转成功。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public ListNode reverseList(ListNode head) {

        // 寻找递归终止条件
        // 1、head 指向的结点为 null
        // 2、head 指向的结点的下一个结点为 null
        // 在这两种情况下，反转之后的结果还是它自己本身
        if( head == null || head.next == null) return head;

        // 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点
        // 因为到最后一个节点的时候，由于当前节点 head 的 next 节点是空，所以会直接返回 head
```

```

ListNode cur = reverseList(head.next);

// 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
// 第一次执行下面代码的时候, head 为 4, 那么 head.next = 5
// 那么 head.next.next 就是 5.next, 意思就是去设置 5 的下一个节点
// 等号右侧为 head, 意思就是设置 5 的下一个节点是 4

// 这里出现了两个 next
// 第一个 next 是「获取」 head 的下一节点
// 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
head.next.next = head;

// head 原来的下一节点指向自己, 所以 head 自己本身就不能再指向原来的下一节点了
// 否则会发生无限循环
head.next = null;

// 我们把每次反转后的结果传递给上一层
return cur;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
public:
    ListNode* reverseList(ListNode* head) {

// 寻找递归终止条件
// 1、head 指向的结点为 null
// 2、head 指向的结点的下一个结点为 null
// 在这两种情况下, 反转之后的结果还是它自己本身
if( head == NULL || head->next == NULL) return head;

// 不断的通过递归调用, 直到无法递归下去, 递归的最小粒度是在最后一个节点
// 因为到最后一个节点的时候, 由于当前节点 head 的 next 节点是空, 所以会直接返回 head
ListNode *cur = reverseList(head->next);

// 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
// 第一次执行下面代码的时候, head 为 4, 那么 head.next = 5
// 那么 head.next.next 就是 5.next, 意思就是去设置 5 的下一个节点
// 等号右侧为 head, 意思就是设置 5 的下一个节点是 4

// 这里出现了两个 next
// 第一个 next 是「获取」 head 的下一节点

```

```

        // 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
head->next->next = head;

// head 原来的下一节点指向自己，所以 head 自己本身就不能再指向原来的下一节点了
// 否则会发生无限循环
head->next = nullptr;

// 我们把每次反转后的结果传递给上一层
return cur;

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution(object):
    def reverseList(self, head):
        """
        :type head: ListNode
        :rtype: ListNode
        """
        # 寻找递归终止条件
        # 1、head 指向的结点为 null
        # 2、head 指向的结点的下一个结点为 null
        # 在这两种情况下，反转之后的结果还是它自己本身
        if(head == None or head.next == None):
            return head

        # 不断的通过递归调用，直到无法递归下去，递归的最小粒度是在最后一个节点
        # 因为到最后一个节点的时候，由于当前节点 head 的 next 节点是空，所以会直接返回 head
        cur = self.reverseList(head.next)

        # 比如原链表为 1 --> 2 --> 3 --> 4 --> 5
        # 第一次执行下面代码的时候，head 为 4，那么 head.next = 5
        # 那么 head.next.next 就是 5.next，意思就是去设置 5 的下一个节点
        # 等号右侧为 head，意思就是设置 5 的下一个节点是 4

        # 这里出现了两个 next
        # 第一个 next 是「获取」 head 的下一节点
        # 第二个 next 是「设置」 当前节点的下一节点为等号右侧的值
        head.next.next = head

        # 原来的下一节点指向自己，所以 head 自己本身就不能再指向原来的下一节点了
        # 否则会发生无限循环

```

```
head.next = None
```

```
# 我们把每次反转后的结果传递给上一层  
return cur
```

四、复杂度分析

时间复杂度：O(n)，其中 n 是链表的长度。需要对链表的每个节点进行反转操作。

空间复杂度：O(n)，其中 n 是链表的长度。空间复杂度主要取决于递归调用的栈空间，最多为 n 层。

LeetCode 82、删除排序链表中的重复元素 II（递归版）

一、题目描述

给定一个已排序的链表的头 `head`，删除原始链表中所有重复数字的节点，只留下不同的数字。返回已排序的链表。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解  
// https://www.algomooc.com  
// 作者：程序员吴师兄  
// 代码有看不懂的地方一定要私聊咨询吴师兄呀  
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list-ii/submissions/  
class Solution {  
    public ListNode deleteDuplicates(ListNode head) {  
  
        // 递归终止条件  
        if(head == null || head.next == null){  
            return head;  
        }  
  
        // 在访问过程中，会出现两种情况  
        // 1、如果访问的节点值和下一个节点值不相同  
        if(head.val != head.next.val){  
  
            // 那么说明当前访问的节点 head 需要保留下来，它和后面经过删除操作的链表连接起来  
  
            // a、利用递归，获取后续删除了链表重复元素的链表  
            ListNode resNode = deleteDuplicates(head.next);  
  
            // 将 head 和后续已经删除了重复元素的链表连接起来  
            head.next = resNode;  
  
            // 返回这个结果就行
```

```

        return head;

// 2、如果访问的节点值和下一个节点值相同
    }else{

        // 那说明 head 这个节点就不应该留下来

        // 现在的目的就是找出和 head 这个节点值不一样的节点来
        ListNode newNode = head.next;

        // 利用 while 循环，找出和 head 这个节点值不一样的节点来
        while(newNode != null && head.val == newNode.val){
            newNode = newNode.next;
        }

        // 此时，经过 while 循环后，newNode 指向了一个不等于 head 的节点
        // 递归调用 deleteDuplicates 函数，处理后面的所有节点
        // 处理好后
        // 1、要么是直接是最终的结果
        // 2、要么是这个结果链表会连接在某个节点后面，比如上述的 a 操作这行代码
        return deleteDuplicates(newNode);
    }

}

}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list-ii/submissions/
class Solution {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        // 递归终止条件
        if(head == NULL || head->next == NULL){
            return head;
        }

        // 在访问过程中，会出现两种情况
        // 1、如果访问的节点值和下一个节点值不相同
        if(head->val != head->next->val){

            // 那么说明当前访问的节点 head 需要保留下来，它和后面经过删除操作的链表连接起来

            // a、利用递归，获取后续删除了链表重复元素的链表

```

```

        ListNode *resNode = deleteDuplicates(head->next);

        // 将 head 和后续已经删除了重复元素的链表连接起来
        head->next = resNode;

        // 返回这个结果就行
        return head;

// 2、如果访问的节点值和下一个节点值相同
}else{

    // 那说明 head 这个节点就不应该留下来

    // 现在的目的就是去找出和 head 这个节点值不一样的节点来
    ListNode *newNode = head->next;

    // 利用 while 循环，找出和 head 这个节点值不一样的节点来
    while(newNode != NULL && head->val == newNode->val){
        newNode = newNode->next;
    }

    // 此时，经过 while 循环后，newNode 指向了一个不等于 head 的节点
    // 递归调用 deleteDuplicates 函数，处理后面的所有节点
    // 处理好后
    // 1、要么是直接是最终的结果
    // 2、要么是这个结果链表会连接在某个节点后面，比如上述的 a 操作这行代码
    return deleteDuplicates(newNode);

}

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### https://leetcode-cn.com/problems/remove-duplicates-from-sorted-list-ii/submissions/
class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        # 递归终止条件
        if head == None or head.next == None:
            return head

        # 在访问过程中，会出现两种情况

```

```

# 1、如果访问的节点值和下一个节点值不相同
if head.val != head.next.val:

    # 那么说明当前访问的节点 head 需要保留下来，它和后面经过删除操作的链表连接起来

    # a、利用递归，获取后续删除了链表重复元素的链表
    resNode = self.deleteDuplicates(head.next)

    # 将 head 和后续已经删除了重复元素的链表连接起来
    head.next = resNode

    # 返回这个结果就行
    return head

# 2、如果访问的节点值和下一个节点值相同
else:

    # 那说明 head 这个节点就不应该留下来

    # 现在的目的就是找出和 head 这个节点值不一样的节点来
    newNode = head.next

    # 利用 while 循环，找出和 head 这个节点值不一样的节点来
    while newNode != None and head.val == newNode.val:
        newNode = newNode.next

    # 此时，经过 while 循环后，newNode 指向了一个不等于 head 的节点
    # 递归调用 deleteDuplicates 函数，处理后面的所有节点
    # 处理好后
    # 1、要么是直接是最终的结果
    # 2、要么是这个结果链表会连接在某个节点后面，比如上述的 a 操作这行代码
    return self.deleteDuplicates(newNode)

```

四、复杂度分析

- 时间复杂度：O(n)，其中 n 是链表的长度。
- 空间复杂度：O(n)。

第三十天

LeetCode 148、排序链表

一、题目描述

给你链表的头结点 `head`，请将其按 **升序** 排列并返回 **排序后的链表**。示例 1：

img

```
输入：head = [4,2,1,3]
输出：[1,2,3,4]
```

示例 2：

img

```
输入：head = [-1,5,3,4,0]
输出：[-1,0,3,4,5]
```

示例 3：

```
输入：head = []
输出：[]
```

提示：

- 链表中节点的数目在范围 `[0, 5 * 104]` 内
- `-105 <= Node.val <= 105`

进阶：你可以在 `O(n log n)` 时间复杂度和常数级空间复杂度下，对链表进行排序吗？

二、题目解析

题目要求时间空间复杂度分别为 $O(n\log n)$ 和 $O(1)$ ，根据时间复杂度和题目中的**排序**两个字，可以联想到**归并排序**和**快速排序**。

所以，本题有两种解法：**归并排序**和**快速排序**。

这里我们先用**归并排序**的思路进行处理，其中**合并**的基本操作如下：

- 1、长度为 1 的链表和长度为 1 的链表合并后，形成一个长度为 2 的链表
- 2、长度为 2 的链表和长度为 2 的链表合并后，形成一个长度为 4 的链表
- ...

而由于合并过程中操作的是链表，所以需要有**断链**和**重新连接**的过程。

具体操作步骤如下：

- 1、先获取链表长度，基于这个长度才能知道后续合并到什么时候截止
- 2、设置三个指针 `prev`、`curr`、`next`。

其中，`prev` 表示已经排序好的链表的【尾节点】。

`curr` 一开始设置为准备排序的那些节点的【首节点】，然后向后移动，获取相应的节点，到达所有正在准备排序的那些节点的【尾节点】位置。

`next` 表示接下来需要排序的那些节点的【首节点】。

3、断开 `prev` 与 `curr` 的连接，再断开 `curr` 与 `next` 的连接。

4、把 `curr` 访问的这些节点划分为两个区域，区域的长度取决于此时进行到了长度为多少的链表进行合并操作，一个是左链表，一个是右链表，把这两个链表进行合并操作。

5、合并成功之后，`prev` 移动到尾部，`curr` 来到 `next` 的位置，继续后面的归并操作。

6、这样一轮下来，已经把长度为 2 的链表和长度为 2 的链表合并，形成了一个长度为 4 的链表。

7、接下来，只需要执行上述同样的操作，唯一的修改点在于合并的子链表长度变成了 4。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 排序链表 (LeetCode 148) : https://leetcode.cn/problems/sort-list/
class Solution {
    public ListNode sortList(ListNode head) {

        // 获链表的总长度
        int length = 0;

        // 从链表的头节点开始访问
        ListNode node = head;

        // 利用 while 循环，可以统计出链表的节点个数，即长度
        while (node != null) {

            length++;

            node = node.next;

        }

        // 在原链表的头部设置一个虚拟头节点
        // 因为可能会操作到原链表的头节点
        // 设置了虚拟头节点后，原链表的头节点和原链表的其它节点地位一样
        ListNode dummyHead = new ListNode(0, head);
```

```

// 利用 for 循环，执行合并的操作
// 长度为 1 的链表和长度为 1 的链表合并后，形成一个长度为 2 的链表
// 长度为 2 的链表和长度为 2 的链表合并后，形成一个长度为 4 的链表
// 长度为 4 的链表和长度为 4 的链表合并后，形成一个长度为 8 的链表
// 长度为 8 的链表和长度为 8 的链表合并后，形成一个长度为 16 的链表
// 也有可能是，长度为 8 的链表和长度为 5 的链表合并后，形成一个长度为 13 的链表
// 但是，每次合并过程中，子链表都会想要扩充为原来的两倍
// 直到子链表想要扩充的长度超过了 length
for (int subLength = 1; subLength < length; subLength *= 2) {

    // 整个归并过程分为三个步骤
    // 1、不停的划分，直到无法划分为止
    // 2、开始两两合并
    // 3、每次合并之后的结果都需要连接起来

    // 每次都把结果连接到 dummyHead，因此先记录一下
    // prev 表示已经排序好的链表的【尾节点】
    ListNode prev = dummyHead;

    // dummyHead 的后面节点才是原链表的节点，需要把它们进行划分
    // curr 表示所有正在准备排序的那些节点的【尾节点】
    ListNode curr = dummyHead.next;

    // 利用 while 循环，寻找出每次划分后子链表的头节点
    while (curr != null) {

        // 每次都是两个子链表开始合并

        // 1、先寻找出【左子链表】，长度为 subLength
        ListNode head1 = curr;

        // 通过 for 循环，找出 subLength 个节点来
        // curr 的索引为 0，需要再找 subLength - 1 个节点来
        for (int i = 1; i < subLength && curr.next != null; i++) {

            curr = curr.next;

        }

        // 2、再寻找出【右子链表】，长度最多为 subLength，甚至有可能长度为 0
        ListNode head2 = curr.next;

        // 此时，需要将【左子链表】与【右子链表】的连接断开
        curr.next = null;

        // curr 来到【右子链表】的头部
        curr = head2;
    }
}

```

```

// 通过 for 循环，找出【右子链表】的那些节点来
// 【右子链表】的节点个数可能达不到 subLength，甚至只有 1 个或者 0 个节点
for (int i = 1; i < subLength && curr != null && curr.next != null;
i++) {

    curr = curr.next;

}

// 获取到【右子链表】之后，需要把它和后续链表断开
// next 表示接下来需要排序的那些节点的【首节点】
ListNode next = null;

// 如果 curr != null，那么说明【右子链表】的节点个数达到了 subLength 个，并且后
续还有节点
if (curr != null) {

    // 记录一下后面节点
    next = curr.next;

    // 再将【右子链表】和后续链表断开
    curr.next = null;

}

// 将【左子链表】与【右子链表】合并
ListNode merged = mergeTwoLists(head1, head2);

// 合并之后的结果需要连接到前一个链表
prev.next = merged;

// prev 来到链表的尾部，是下一个即将合成链表之后的前一个链表的尾节点
while (prev.next != null) {

    prev = prev.next;

}

// curr 来到 next，处理后面的节点
curr = next;

}

}

return dummyHead.next;
}

// 合并两个有序链表的代码
private ListNode mergeTwoLists(ListNode l1, ListNode l2) {

```

```

// 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
ListNode dummy = new ListNode(-1);

// 设置一个指针，指向虚拟节点
ListNode pre = dummy;

// 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
while (l1 != null && l2 != null) {
    // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
    if (l1.val <= l2.val) {
        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l1
        pre.next = l1;
        // 让 l1 向后移动
        l1 = l1.next;
    } else {
        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l2
        pre.next = l2;
        // 让 l2 向后移动
        l2 = l2.next;
    }
    // 让 pre 向后移动
    pre = pre.next;
}

// 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
// 直接把剩下的节点加入到 pre 的 next 指针位置

// 如果 l1 中还有节点
if (l1 != null) {
    // 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l1;
}

// 如果 l2 中还有节点
if (l2 != null) {
    // 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l2;
}

// 最后返回虚拟节点的 next 指针
return dummy.next;
}
}

```

2、C++ 代码

```
class solution {
public:
    ListNode* sortList(ListNode* head) {
        // 获链表的总长度
        int length = 0;

        // 从链表的头节点开始访问
        ListNode* node = head;

        // 利用 while 循环，可以统计出链表的节点个数，即长度
        while (node != NULL) {

            length++;

            node = node->next;

        }

        // 在原链表的头部设置一个虚拟头节点
        // 因为可能会操作到原链表的头节点
        // 设置了虚拟头节点后，原链表的头节点和原链表的其它节点地位一样
        ListNode* dummyHead = new ListNode(0, head);

        // 利用 for 循环，执行合并的操作
        // 长度为 1 的链表和长度为 1 的链表合并后，形成一个长度为 2 的链表
        // 长度为 2 的链表和长度为 2 的链表合并后，形成一个长度为 4 的链表
        // 长度为 4 的链表和长度为 4 的链表合并后，形成一个长度为 8 的链表
        // 长度为 8 的链表和长度为 8 的链表合并后，形成一个长度为 16 的链表
        // 也有可能是，长度为 8 的链表和长度为 5 的链表合并后，形成一个长度为 13 的链表
        // 但是，每次合并过程中，子链表都会想要扩充为原来的两倍
        // 直到子链表想要扩充的长度超过了 length
        for (int subLength = 1; subLength < length; subLength = 2) {

            // 整个归并过程分为三个步骤
            // 1、不停的划分，直到无法划分为止
            // 2、开始两两合并
            // 3、每次合并之后的结果都需要连接起来

            // 每次都把结果连接到 dummyHead，因此先记录一下
            // prev 表示已经排序好的链表的【尾节点】
            ListNode prev = dummyHead;

            // dummyHead 的后面节点才是原链表的节点，需要把它们进行划分
            // curr 表示所有正在准备排序的那些节点的【尾节点】
            ListNode* curr = dummyHead->next;

            // 利用 while 循环，寻找出每次划分后子链表的头节点
```

```

while (curr != NULL) {

    // 每次都是两个子链表开始合并

    // 1、先寻找出【左子链表】，长度为 subLength
    ListNode* head1 = curr;

    // 通过 for 循环，找出 subLength 个节点来
    // curr 的索引为 0，需要再找 subLength - 1 个节点来
    for (int i = 1; i < subLength && curr->next != NULL; i++) {

        curr = curr->next;

    }

    // 2、再寻找出【右子链表】，长度最多为 subLength，甚至有可能长度为 0
    ListNode* head2 = curr->next;

    // 此时，需要将【左子链表】与【右子链表】的连接断开
    curr->next = NULL;

    // curr 来到【右子链表】的头部
    curr = head2;

    // 通过 for 循环，找出【右子链表】的那些节点来
    // 【右子链表】的节点个数可能达不到 subLength，甚至只有 1 个或者 0 个节点
    for (int i = 1; i < subLength && curr != NULL && curr->next != NULL;
i++) {

        curr = curr->next;

    }

    // 获取到【右子链表】之后，需要把它和后续链表断开
    //->next 表示接下来需要排序的那些节点的【首节点】
    ListNode* next = NULL;

    // 如果 curr != NULL，那么说明【右子链表】的节点个数达到了 subLength 个，并且后
续还有节点

    if (curr != NULL) {

        // 记录一下后面节点
        next = curr->next;

        // 再将【右子链表】和后续链表断开
        curr->next = NULL;

    }
}

```

```

        // 将【左子链表】与【右子链表】合并
        ListNode* merged = mergeTwoLists(head1, head2);

        // 合并之后的结果需要连接到前一个链表
        prev->next = merged;

        // prev 来到链表的尾部，是下一个即将合成链表之后的前一个链表的尾节点
        while (prev->next != NULL) {

            prev = prev->next;

        }

        // curr 来到 next，处理后面的节点
        curr = next;
    }

}

return dummyHead->next;

}

```

public:

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    // 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
    ListNode *dummy = new ListNode(-1);

    // 设置一个指针，指向虚拟节点
    ListNode *pre = dummy;

    // 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
    while (l1 != NULL && l2 != NULL) {
        // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
        if (l1->val <= l2->val) {
            // 让 pre 指向节点的 next 指针指向这个更小值的节点
            // 即指向 l1
            pre->next = l1;
            // 让 l1 向后移动
            l1 = l1->next;
        } else {
            // 让 pre 指向节点的 next 指针指向这个更小值的节点
            // 即指向 l2
            pre->next = l2;
            // 让 l2 向后移动
            l2 = l2->next;
        }
        // 让 pre 向后移动
    }
}

```

```

        pre = pre->next;
    }

    // 跳出循环后, l1 或者 l2 中可能有剩余的节点没有被观察过
    // 直接把剩下的节点加入到 pre 的 next 指针位置

    // 如果 l1 中还有节点
    if (l1 != NULL) {
        // 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
        pre->next = l1;
    }

    // 如果 l2 中还有节点
    if (l2 != NULL) {
        // 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
        pre->next = l2;
    }

    // 最后返回虚拟节点的 next 指针
    return dummy->next;
}

};

```

3、Python 代码

```

class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # 获链表的总长度
        length = 0

        # 从链表的头节点开始访问
        node = head

        # 利用 while 循环, 可以统计出链表的节点个数, 即长度
        while node :

            length += 1

            node = node.next

        # 在原链表的头部设置一个虚拟头节点
        # 因为可能会操作到原链表的头节点
        # 设置了虚拟头节点后, 原链表的头节点和原链表的其它节点地位一样
        dummyHead = ListNode(0)

        dummyHead.next = head

```



```

# 利用 for 循环，执行合并的操作
# 长度为 1 的链表和长度为 1 的链表合并后，形成一个长度为 2 的链表
# 长度为 2 的链表和长度为 2 的链表合并后，形成一个长度为 4 的链表
# 长度为 4 的链表和长度为 4 的链表合并后，形成一个长度为 8 的链表
# 长度为 8 的链表和长度为 8 的链表合并后，形成一个长度为 16 的链表
# 也有可能是，长度为 8 的链表和长度为 5 的链表合并后，形成一个长度为 13 的链表
# 但是，每次合并过程中，子链表都会想要扩充为原来的两倍
# 直到子链表想要扩充的长度超过了 length
subLength = 1
while subLength < length :

    # 整个归并过程分为三个步骤
    # 1、不停的划分，直到无法划分为止
    # 2、开始两两合并
    # 3、每次合并之后的结果都需要连接起来

    # 每次都把结果连接到 dummyHead，因此先记录一下
    # prev 表示已经排序好的链表的【尾节点】
    prev = dummyHead

    # dummyHead 的后面节点才是原链表的节点，需要把它们进行划分
    # curr 表示所有正在准备排序的那些节点的【尾节点】
    curr = dummyHead.next

    # 利用 while 循环，寻找出每次划分后子链表的头节点
    while curr :

        # 每次都是两个子链表开始合并

        # 1、先寻找出【左子链表】，长度为 subLength
        head1 = curr

        # 通过 for 循环，找出 subLength 个节点来
        # curr 的索引为 0，需要再找 subLength - 1 个节点来
        for i in range( 1, subLength ) :
            if curr.next:
                curr = curr.next

        # 2、再寻找出【右子链表】，长度最多为 subLength，甚至有可能长度为 0
        head2 = curr.next

        # 此时，需要将【左子链表】与【右子链表】的连接断开
        curr.next = None

        # curr 来到【右子链表】的头部
        curr = head2

```

还有节点

```
# 通过 for 循环，找出【右子链表】的那些节点来
# 【右子链表】的节点个数可能达不到 subLength，甚至只有 1 个或者 0 个节点
for i in range( 1 , subLength ) :
    if curr and curr.next:
        curr = curr.next

# 获取到【右子链表】之后，需要把它和后续链表断开
# next 表示接下来需要排序的那些节点的【首节点】
nextNode = None

# 如果 curr != None，那么说明【右子链表】的节点个数达到了 subLength 个，并且后续

if curr :

    # 记录一下后面节点
    nextNode = curr.next

    # 再将【右子链表】和后续链表断开
    curr.next = None

# 将【左子链表】与【右子链表】合并
merged = self.mergeTwoLists(head1, head2)

# 合并之后的结果需要连接到前一个链表
prev.next = merged

# prev 来到链表的尾部，是下一个即将合成链表之后的前一个链表的尾节点
while prev.next :

    prev = prev.next

# curr 来到 next，处理后面的节点
curr = nextNode

subLength = subLength * 2

return dummyHead.next

def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:

    # 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
    dummy = ListNode(-1)

    # 设置一个指针，指向虚拟节点
```

```

pre = dummy

# 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
while l1 and l2:
    # 如果 l1 当前节点的值小于等于了 l2 当前节点的值
    if l1.val <= l2.val :
        # 让 pre 指向节点的 next 指针指向这个更小值的节点
        # 即指向 l1
        pre.next = l1
        # 让 l1 向后移动
        l1 = l1.next
    else:
        # 让 pre 指向节点的 next 指针指向这个更小值的节点
        # 即指向 l2
        pre.next = l2
        # 让 l2 向后移动
        l2 = l2.next

    # 让 pre 向后移动
    pre = pre.next

# 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
# 直接把剩下的节点加入到 pre 的 next 指针位置

# 如果 l1 中还有节点
if l1 != None :
    # 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l1

# 如果 l2 中还有节点
if l2 != None :
    # 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l2;

# 最后返回虚拟节点的 next 指针
return dummy.next

```

四、复杂度分析

时间复杂度：O(nlogn)

空间复杂度：O(1)

LeetCode 23、合并K个升序链表（归并思路）

一、题目描述

给你一个链表数组，每个链表都已经按升序排列。

请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1:

输入: lists = [[1,4,5],[1,3,4],[2,6]]

输出: [1,1,2,3,4,4,5,6]

解释: 链表数组如下:

```
[
  1->4->5,
  1->3->4,
  2->6
]
```

将它们合并到一个有序链表中得到。

1->1->2->3->4->4->5->6

提示:

- `k == lists.length`
- `0 <= k <= 10^4`
- `0 <= lists[i].length <= 500`
- `-10^4 <= lists[i][j] <= 10^4`
- `lists[i]` 按升序排列
- `lists[i].length` 的总和不超过 `10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 合并K个升序链表: https://leetcode-cn.com/problems/merge-k-sorted-lists/
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {

        // 如果划分的 lists 总的链表个数为 0
        if(lists.length == 0){
            // 返回空
            return null;
        }
    }
}
```

```

// 如果划分的 lists 总的链表个数为 1
if(lists.length == 1){
    // 不需要再划分, 返回这个链表
    return lists[0];
}

// 如果划分的 lists 总的链表个数为 2
if(lists.length == 2){
    // 将这两个链表合并
    return mergeTwoLists(lists[0],lists[1]);
}

// 计算中点
int mid = lists.length / 2;

// 设置两个存放链表的数组
// sub1 用来存放前部分的那些链表
ListNode[] sub1 = new ListNode[mid];
for(int i = 0 ; i < mid ; i++){
    sub1[i] = lists[i];
}

// sub2 用来存放后部分的那些链表
ListNode[] sub2 = new ListNode[lists.length - mid ];
for(int i = mid ; i < lists.length ; i++){
    sub2[i - mid] = lists[i];
}

// 获取 sub1 合并之后的结果
ListNode l1 = mergeKLists(sub1);

// 获取 sub2 合并之后的结果
ListNode l2 = mergeKLists(sub2);

// 将这两个链表合并返回
return mergeTwoLists(l1,l2);
}

public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    // 一开始设置一个虚拟节点, 它的值为 -1, 它的值可以设置为任何的数, 因为我们根本不需要使用它的值
    ListNode dummy = new ListNode(-1);

    // 设置一个指针, 指向虚拟节点
    ListNode pre = dummy;

    // 通过一个循环, 不断的比较 l1 和 l2 中当前节点值的大小, 直到 l1 或者 l2 遍历完毕为止
    while (l1 != null && l2 != null) {
        // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
        if (l1.val <= l2.val) {

```

```

        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l1
        pre.next = l1;
        // 让 l1 向后移动
        l1 = l1.next;
    }else {
        // 让 pre 指向节点的 next 指针指向这个更小值的节点
        // 即指向 l2
        pre.next = l2;
        // 让 l2 向后移动
        l2 = l2.next;
    }
    // 让 pre 向后移动
    pre = pre.next;
}

// 跳出循环后, l1 或者 l2 中可能有剩余的节点没有被观察过
// 直接把剩下的节点加入到 pre 的 next 指针位置

// 如果 l1 中还有节点
if (l1 != null) {
    // 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l1;
}

// 如果 l2 中还有节点
if (l2 != null) {
    // 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
    pre.next = l2;
}

// 最后返回虚拟节点的 next 指针
return dummy.next;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 合并K个升序链表: https://leetcode-cn.com/problems/merge-k-sorted-lists/
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*> lists) {
        // 如果划分的 lists 总的链表个数为 0
        if(lists.size() == 0){
            // 返回空

```

```

        return NULL;
    }

    // 如果划分的 lists 总的链表个数为 1
    if(lists.size() == 1){
        // 不需要再划分, 返回这个链表
        return lists[0];
    }

    // 如果划分的 lists 总的链表个数为 2
    if(lists.size() == 2){
        // 将这两个链表合并
        return mergeTwoLists(lists[0],lists[1]);
    }

    return merge(lists , 0 , lists.size() - 1);
}

private:
ListNode* merge(vector<ListNode*>& lists,int left,int right){
    if( left > right ){
        return NULL;
    }
    if( left == right ){
        return lists[left];
    }
    int mid = ( left + right )/2;
    return mergeTwoLists(merge(lists,left,mid),merge(lists,mid+1,right));
}

private:
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    // 一开始设置一个虚拟节点, 它的值为 -1, 它的值可以设置为任何的数, 因为我们根本不需要使用它的值
    ListNode *dummy = new ListNode(-1);

    // 设置一个指针, 指向虚拟节点
    ListNode *pre = dummy;

    // 通过一个循环, 不断的比较 l1 和 l2 中当前节点值的大小, 直到 l1 或者 l2 遍历完毕为止
    while (l1 != NULL && l2 != NULL) {
        // 如果 l1 当前节点的值小于等于了 l2 当前节点的值
        if (l1->val <= l2->val) {
            // 让 pre 指向节点的 next 指针指向这个更小值的节点
            // 即指向 l1
            pre->next = l1;
            // 让 l1 向后移动
            l1 = l1->next;
        }else {
            // 让 pre 指向节点的 next 指针指向这个更小值的节点
            // 即指向 l2
            pre->next =l2;

```

```

        // 让 12 向后移动
        12 = 12->next;
    }
    // 让 pre 向后移动
    pre = pre->next;
}

// 跳出循环后, 11 或者 12 中可能有剩余的节点没有被观察过
// 直接把剩下的节点加入到 pre 的 next 指针位置

// 如果 11 中还有节点
if (l1 != NULL) {
    // 把 11 中剩下的节点全部加入到 pre 的 next 指针位置
    pre->next = l1;
}

// 如果 12 中还有节点
if (l2 != NULL) {
    // 把 12 中剩下的节点全部加入到 pre 的 next 指针位置
    pre->next = l2;
}

// 最后返回虚拟节点的 next 指针
return dummy->next;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 合并K个升序链表: https://leetcode-cn.com/problems/merge-k-sorted-lists/
class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        if not lists: return
        return self.merge(lists, 0, len(lists) - 1)

    def merge(self, lists, left, right):

        if left == right:
            return lists[left]

        # 计算中点
        mid = left + (right - left) // 2

        # 获取 l1 合并之后的结果
        l1 = self.merge(lists, left, mid)

```



```

# 获取 l2 合并之后的结果
l2 = self.merge(lists, mid+1, right)

# 将这两个链表合并返回
return self.mergeTwoLists(l1,l2)

def mergeTwoLists(self, l1, l2):

    # 一开始设置一个虚拟节点，它的值为 -1，它的值可以设置为任何的数，因为我们根本不需要使用它的值
    dummy = ListNode(-1)

    # 设置一个指针，指向虚拟节点
    pre = dummy

    # 通过一个循环，不断的比较 l1 和 l2 中当前节点值的大小，直到 l1 或者 l2 遍历完毕为止
    while l1 and l2:
        # 如果 l1 当前节点的值小于等于了 l2 当前节点的值
        if l1.val <= l2.val :
            # 让 pre 指向节点的 next 指针指向这个更小值的节点
            # 即指向 l1
            pre.next = l1
            # 让 l1 向后移动
            l1 = l1.next
        else:
            # 让 pre 指向节点的 next 指针指向这个更小值的节点
            # 即指向 l2
            pre.next = l2
            # 让 l2 向后移动
            l2 = l2.next

        # 让 pre 向后移动
        pre = pre.next

    # 跳出循环后，l1 或者 l2 中可能有剩余的节点没有被观察过
    # 直接把剩下的节点加入到 pre 的 next 指针位置

    # 如果 l1 中还有节点
    if l1 != None :
        # 把 l1 中剩下的节点全部加入到 pre 的 next 指针位置
        pre.next = l1

    # 如果 l2 中还有节点
    if l2 != None :
        # 把 l2 中剩下的节点全部加入到 pre 的 next 指针位置
        pre.next = l2;

```

```
# 最后返回虚拟节点的 next 指针
return dummy.next
```

四、复杂度分析

时间复杂度： $O(kn \times \log k)$ 。

空间复杂度：递归会使用到 $O(\log k)$ 空间代价的栈空间。

LeetCode 4、寻找两个正序数组的中位数

一、题目描述

给定两个大小分别为 m 和 n 的正序（从小到大）数组 `nums1` 和 `nums2`。请你找出并返回这两个正序数组的 **中位数**。

算法的时间复杂度应该为 $O(\log(m+n))$ 。

示例 1：

```
输入：nums1 = [1,3]，nums2 = [2]
输出：2.00000
解释：合并数组 = [1,2,3]，中位数 2
```

示例 2：

```
输入：nums1 = [1,2]，nums2 = [3,4]
输出：2.50000
解释：合并数组 = [1,2,3,4]，中位数  $(2 + 3) / 2 = 2.5$ 
```

示例 3：

```
输入：nums1 = [0,0]，nums2 = [0,0]
输出：0.00000
```

示例 4：

```
输入：nums1 = []，nums2 = [1]
输出：1.00000
```

示例 5：

```
输入：nums1 = [2]，nums2 = []
输出：2.00000
```

提示：

- `nums1.length == m`
- `nums2.length == n`
- `0 <= m <= 1000`
- `0 <= n <= 1000`
- `1 <= m + n <= 2000`
- `-10^6 <= nums1[i], nums2[i] <= 10^6`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 寻找两个正序数组的中位数(4):https://leetcode-cn.com/problems/median-of-two-sorted-arrays/submissions/
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {

        // 中位数的概念：将一个集合划分为两个长度相等的子集，其中一个子集中的元素总是大于另一个子集中的元素

        // 由于数组的长度有两种情况：奇数、偶数
        // 奇数组：[2 3 5] 对应的中位数为 3
        // 偶数组：[1 4 7 9] 对应的中位数为 ( 4 + 7 ) / 2 = 5.5

        // 获取 nums1 的长度
        int n = nums1.length;

        // 获取 nums2 的长度
        int m = nums2.length;

        if( n > m){
            // 通过这个方法，可以将 nums1 和 nums2 进行一个交换操作，确保 nums1 的长度一定小于
            nums2

            return findMedianSortedArrays(nums2,nums1);
        }

        // nums1、nums2 均为有序数组
        // 如果将有序数组切割分成两部分，那么切的那个位置的左侧为左边最大值，切的那个位置的右侧为右边
        最小值

        //      |
        // [1 4 | 7 9]
        //      |
        // 比如 4 就是左部分的最大值，7 就是右部分的最小值
```

```

// 设置两个变量 Cut1、Cut2 方便切割 nums1、nums2
// 不断的挪动 Cut1 和 Cut2 切的位置，如果说 Cut1、Cut2 找到了合适的位置进行切割
// 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的右部分
) 个数】
// 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2 的右部分
) 最小数
// 那么也就找到了中位数

// 一开始 Cut1 为 0
int Cut1 = 0;

// 一开始 Cut2 为 0
int Cut2 = 0;

// nums1、nums2 均为有序数组
// 显然, LMax1 <= RMin1, LMax2 <= RMin2

// LMax1 表示 nums1 被 Cut1 切割后, 左部分的最大值
int LMax1 = 0;

// LMax2 表示 nums2 被 Cut2 切割后, 左部分的最大值
int LMax2 = 0;

// RMin1 表示 nums1 被 Cut1 切割后, 右部分的最小值
int RMin1 = 0;

// RMin2 表示 nums2 被 Cut2 切割后, 右部分的最小值
int RMin2 = 0;

// 一开始 Cut1 = 0, Cut2 = 0 , 也就是说 nums1 的左部分 0 个元素,  nums2 的左部分 0 个元
素
// 因此需要挪动 Cut1、Cut2 的位置, 使得 ( nums1 的左部分 + nums2 的左部分 ) 个数 = (
nums1 的右部分 + nums2 的右部分 ) 个数

// 所以, 一开始直接把 Cut1 挪到 nums1 的中间位置进行切割, 把 Cut2 挪到 nums2 的中间位置进
行切割
// 如果 LMax1 <= RMin2, LMax2 <= RMin1
// 又由于 LMax1 <= RMin1, LMax2 <= RMin2
// 意味着
// 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的右部分 )
个数
// 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2 的右部分
) 最小数

// 当发现 LMax1 > RMin2 时, 说明 Cut1 切的位置不对, 左边的元素太多了, 需要把一些元素挪到右
边来, 这样才能减小 LMax1 的值
// 所以 Cut1 的值减小了
// 而因为 ( nums1 的左部分 + nums2 的左部分 ) 个数 = Cut1 + Cut2
// Cut2 变大了, 也就是说可以通过 Cut1 的位置找到 Cut2 的位置

```

```

// 由于 m + n 可能为奇数，也可能为偶数，为了方便统一处理，这里加入一个技巧
// 在数组的 开头、结尾、数字直接加入一个 “#”
// 这样 nums2 的长度 m 变成了 2m + 1
// 这样 nums1 的长度 n 变成了 2n + 1
// 两数之和变成了 2m + 2n + 2，恒为偶数
// 比如 [ 1 、 3 、 5 、 7 ] 变成了 [ #、 1 、# 、 3 、 # 、 5 、 # 、 7 、 # ]
// 比如 [ 2 、 4 、 6 ] 变成了 [ #、 2 、# 、 4 、 # 、 6 、 # ]
// 因此，每个位置的下标位置发生了改变，但可以通过 /2 得到原来元素的位置：
// 比如 1，原来在 0 位，现在是 1 位，1 / 2 = 0
// 比如 3，原来在 1 位，现在是 3 位，3 / 2 = 1
// 比如 6，原来在 2 位，现在是 5 位，5 / 2 = 2

// 此时，LMax1 = ( Cut1 - 1 ) / 2 位置上的元素
// RMin1 = Cut1 / 2 位置上的元素

// 接下来，开始找 Cut1 和 Cut2 的位置了

// left 为 nums1 最左侧的元素，可以获取到
int left = 0;

// 添加了 “#” 后，nums1 的长度 n 变成了 2n + 1
// right 为 nums1 最右侧的元素，可以获取到
int right = 2 * n ;

// 在 while 循环里面，left 不断的 ++，而 right 不断的 --
// 当 [ left , right ] 这个区间不存在元素的时候，才跳出 while 循环，也就是当 left >
right 时跳出循环
// 即当 left = right + 1 时，搜索区间没有元素了
// 由于 left 和 right 相遇的时候指向同一个元素，这个元素是存在于 [ left , right] 区间，
这个区间就一个元素
// 所以此时 while 循环的判断可以使用等号
while( left <= right){

    // Cut1 切在 nums1 的中间位置
    Cut1 = left + ( right - left ) / 2;

    // ( nums1 的左部分 + nums2 的左部分 ) 个数 = m + n
    // 所以 Cut2 = m + n - Cut1
    Cut2 = m + n - Cut1;

    // 注意几种边界情况

    // 1、nums1 左部分没有元素，全部都在右部分，并且元素值都比中值大
    // 所以，中值在 nums2 中，这里就可以假定 LMax1 = INT_MIN
    if( Cut1 == 0 ){
        LMax1 = Integer.MIN_VALUE;
    }else{
        // 否则 LMax1 切割位置的左边元素

```

```

        LMax1 = nums1[ (Cut1 - 1) / 2 ];
    }

    // 2、nums1 右部分没有元素，全部都在左部分，并且元素值都比中值小
    // 所以，中值在 nums2 中，这里就可以假定 RMin1 = MAX_VALUE
    if( Cut1 == 2 * n){
        RMin1 = Integer.MAX_VALUE;
    }else{
        // 否则 RMin1 切割位置的右边元素
        RMin1 = nums1[ Cut1 / 2 ];
    }

    // 3、nums2 左部分没有元素，全部都在右部分，并且元素值都比中值大
    // 所以，中值在 nums1 中，这里就可以假定 LMax2 = INT_MIN
    if( Cut2 == 0){
        LMax2 = Integer.MIN_VALUE;
    }else{
        // 否则 LMax2 切割位置的左边元素
        LMax2 = nums2[ (Cut2 - 1) / 2 ];
    }

    // 2、nums2 右部分没有元素，全部都在左部分，并且元素值都比中值小
    // 所以，中值在 nums1 中，这里就可以假定 RMin2 = MAX_VALUE
    if( Cut2 == 2 * m){
        RMin2 = Integer.MAX_VALUE;
    }else{
        // 否则 RMin2 切割位置的右边元素
        RMin2 = nums2[ Cut2 / 2 ];
    }

    // LMax1 > RMin2
    // 说明 Cut1 切的位置不对，左边的元素太多了，需要把一些元素挪到右边来，这样才能减小
LMax1

    // Cut1 切的位置向左边挪一些
    if( LMax1 > RMin2 ){
        // 也就是说缩小之后的区间最右侧位置为 Cut1 - 1
        right = Cut1 - 1;

    // LMax2 > RMin1
    // 说明 Cut1 切的位置不对，左边的元素太少了，需要把一些元素挪到左边来，这样才能增大
RMin1

    // Cut1 切的位置向右边挪一些
    }else if( LMax2 > RMin1 ){
        // 也就是说缩小之后的区间最左侧位置为 Cut1 + 1
        left = Cut1 + 1;
    }else{
        // 否则就是说明切的位置合适，不用再找其它位置了
        // 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的
右部分 ) 个数

```

```

        // 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2
的右部分 ) 最小数
        break;
    }

}

// 那么再获取 LMax1 和 LMax2 较大值 + RMin1 和 RMin2 的较小值
// 两者相加除以 2 就是结果
return (Math.max(LMax1,LMax2) + Math.min(RMin1,RMin2)) / 2.0;

}
}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

寻找两个正序数组的中位数(4):<https://leetcode-cn.com/problems/median-of-two-sorted-arrays/submissions/>

class Solution:

def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:

中位数的概念：将一个集合划分为两个长度相等的子集，其中一个子集中的元素总是大于另一个子集中的元素

由于数组的长度有两种情况：奇数、偶数

奇数组：[2 3 5] 对应的中位数为 3

偶数组：[1 4 7 9] 对应的中位数为 (4 + 7) / 2 = 5.5

获取 nums1 的长度

n = len(nums1)

获取 nums2 的长度

m = len(nums2)

if n > m :

通过这个方法，可以将 nums1 和 nums2 进行一个交换操作，确保 nums1 的长度一定小于 nums2

return self.findMedianSortedArrays(nums2,nums1)

nums1、nums2 均为有序数组

如果将有序数组切割分成两部分，那么切的那个位置的左侧为左边最大值，切的那个位置的右侧为右边最小值

```

#      |
# [1 4 | 7 9]
#      |
# 比如 4 就是左部分的最大值, 7 就是右部分的最小值

# 设置两个变量 Cut1、Cut2 方便切割 nums1、nums2
# 不断的挪动 Cut1 和 Cut2 切的位置, 如果说 Cut1、Cut2 找到了合适的位置进行切割
# 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的右部分 )
个数】
# 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2 的右部分
) 最小数
# 那么也就找到了中位数

# 一开始 Cut1 为 0
Cut1 = 0

# 一开始 Cut2 为 0
Cut2 = 0

# nums1、nums2 均为有序数组
# 显然, LMax1 <= RMin1, LMax2 <= RMin2

# LMax1 表示 nums1 被 Cut1 切割后, 左部分的最大值
LMax1 = 0

# LMax2 表示 nums2 被 Cut2 切割后, 左部分的最大值
LMax2 = 0

# RMin1 表示 nums1 被 Cut1 切割后, 右部分的最小值
RMin1 = 0

# RMin2 表示 nums2 被 Cut2 切割后, 右部分的最小值
RMin2 = 0

# 一开始 Cut1 = 0, Cut2 = 0 , 也就是说 nums1 的左部分 0 个元素,  nums2 的左部分 0 个元
素
# 因此需要挪动 Cut1、Cut2 的位置, 使得 ( nums1 的左部分 + nums2 的左部分 ) 个数 = (
nums1 的右部分 + nums2 的右部分 ) 个数

# 所以, 一开始直接把 Cut1 挪到 nums1 的中间位置进行切割, 把 Cut2 挪到 nums2 的中间位置进
行切割
# 如果 LMax1 <= RMin2, LMax2 <= RMin1
# 又由于 LMax1 <= RMin1, LMax2 <= RMin2
# 意味着
# 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的右部分 ) 个
数
# 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2 的右部分
) 最小数

```



```

# 当发现 LMax1 > RMin2 时, 说明 Cut1 切的位置不对, 左边的元素太多了, 需要把一些元素挪到右边来, 这样才能减小 LMax1 的值
# 所以 Cut1 的值减小了
# 而因为 ( nums1 的左部分 + nums2 的左部分 ) 个数 = Cut1 + Cut2
# Cut2 变大了, 也就是说可以通过 Cut1 的位置找到 Cut2 的位置

# 由于 m + n 可能为奇数, 也可能为偶数, 为了方便统一处理, 这里加入一个技巧
# 在数组的 开头、结尾、数字直接加入一个 “#”
# 这样 nums2 的长度 m 变成了 2m + 1
# 这样 nums1 的长度 n 变成了 2n + 1
# 两数之和变成了 2m + 2n + 2, 恒为偶数
# 比如 [ 1 、 3 、 5 、 7 ] 变成了 [ # 、 1 、 # 、 3 、 # 、 5 、 # 、 7 、 # ]
# 比如 [ 2 、 4 、 6 ] 变成了 [ # 、 2 、 # 、 4 、 # 、 6 、 # ]
# 因此, 每个位置的下标位置发生了改变, 但可以通过 /2 得到原来元素的位置:
# 比如 1, 原来在 0 位, 现在是 1 位, 1 / 2 = 0
# 比如 3, 原来在 1 位, 现在是 3 位, 3 / 2 = 1
# 比如 6, 原来在 2 位, 现在是 5 位, 5 / 2 = 2

# 此时, LMax1 = ( Cut1 - 1 ) / 2 位置上的元素
# RMin1 = Cut1 / 2 位置上的元素

# 接下来, 开始找 Cut1 和 Cut2 的位置了

# left 为 nums1 最左侧的元素, 可以获取到
left = 0

# 添加了 “#” 后, nums1 的长度 n 变成了 2n + 1
# right 为 nums1 最右侧的元素, 可以获取到
right = 2 * n

# 在 while 循环里面, left 不断的 ++, 而 right 不断的 --
# 当 [ left , right ] 这个区间不存在元素的时候, 才跳出 while 循环, 也就是当 left >
right 时跳出循环
# 即当 left = right + 1 时, 搜索区间没有元素了
# 由于 left 和 right 相遇的时候指向同一个元素, 这个元素是存在于 [ left , right] 区间, 这个区间就一个元素
# 所以此时 while 循环的判断可以使用等号
while left <= right :

    # Cut1 切在 nums1 的中间位置
    Cut1 = left + ( right - left ) // 2

    # ( nums1 的左部分 + nums2 的左部分 ) 个数 = m + n
    # 所以 Cut2 = m + n - Cut1
    Cut2 = m + n - Cut1

    # 注意几种边界情况

    # 1、nums1 左部分没有元素, 全部都在右部分, 并且元素值都比中值大

```

```

# 所以，中值在 nums2 中，这里就可以假定 LMax1 = INT_MIN
if Cut1 == 0 :
    LMax1 = -1000001
else :
    # 否则 LMax1 切割位置的左边元素
    LMax1 = nums1[ (Cut1 - 1) // 2 ]

# 2、nums1 右部分没有元素，全部都在左部分，并且元素值都比中值小
# 所以，中值在 nums2 中，这里就可以假定 RMin1 = MAX_VALUE
if Cut1 == 2 * n :
    RMin1 = 1000001
else:
    # 否则 RMin1 切割位置的右边元素
    RMin1 = nums1[ Cut1 // 2 ]

```

```

# 3、nums2 左部分没有元素，全部都在右部分，并且元素值都比中值大
# 所以，中值在 nums1 中，这里就可以假定 LMax2 = INT_MIN
if Cut2 == 0 :
    LMax2 = -1000001
else:
    # 否则 LMax2 切割位置的左边元素
    LMax2 = nums2[ (Cut2 - 1) // 2 ]

```

```

# 2、nums2 右部分没有元素，全部都在左部分，并且元素值都比中值小
# 所以，中值在 nums1 中，这里就可以假定 RMin2 = MAX_VALUE
if Cut2 == 2 * m :
    RMin2 = 1000001
else:
    # 否则 RMin2 切割位置的右边元素
    RMin2 = nums2[ Cut2 // 2 ]

```

```

# LMax1 > RMin2
# 说明 Cut1 切的位置不对，左边的元素太多了，需要把一些元素挪到右边来，这样才能减小

```

LMax1

```

# Cut1 切的位置向左边挪一些
if LMax1 > RMin2 :
    # 也就是说缩小之后的区间最右侧位置为 Cut1 - 1
    right = Cut1 - 1

```

```

# LMax2 > RMin1
# 说明 Cut1 切的位置不对，左边的元素太少了，需要把一些元素挪到左边来，这样才能增大

```

RMin1

```

# Cut1 切的位置向右边挪一些
elif LMax2 > RMin1 :
    # 也就是说缩小之后的区间最左侧位置为 Cut1 + 1

```

```

        left = cut1 + 1
    else:
        # 否则就是说明切的位置合适，不用再找其它位置了
        # 1、( nums1 的左部分 + nums2 的左部分 ) 个数 = ( nums1 的右部分 + nums2 的
右部分 ) 个数
        # 2、( nums1 的左部分 + nums2 的左部分 ) 最大数 <= ( nums1 的右部分 + nums2
的右部分 ) 最小数
        break

    # 那么再获取 LMax1 和 LMax2 较大值 + RMin1 和 RMin2 的较小值
    # 两者相加除以 2 就是结果
    return (max(LMax1,LMax2) + min(RMin1,RMin2)) / 2.0

```

LeetCode 56、合并区间

一、题目描述

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例 1:

输入: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
 输出: `[[1,6],[8,10],[15,18]]`
 解释: 区间 `[1,3]` 和 `[2,6]` 重叠，将它们合并为 `[1,6]`。

示例 2:

输入: `intervals = [[1,4],[4,5]]`
 输出: `[[1,5]]`
 解释: 区间 `[1,4]` 和 `[4,5]` 可被视为重叠区间。

提示:

- `1 <= intervals.length <= 10^4`
- `intervals[i].length == 2`
- `0 <= starti <= endi <= 10^4`

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/merge-intervals/

```

```

class Solution {
    public int[][] merge(int[][] intervals) {

        //通过 sort 函数对二维数组每一行按第一列元素进行排序
        //重写比较器方法,o1[] - o2[] 表示当 o1 大于 o2 时,将 o1 放在 o2 后面,即基本的升序排序
        //而 o1[0] - o2[0] 表示按二维数组的每一行第一列元素排序,类似的 o[1] - o2[1]代表按第二列进
        行排序
        Arrays.sort(intervals, new Comparator<int[]>() {
            @Override
            public int compare(int[] o1, int[] o2) {
                return o1[0] - o2[0];
            }
        });

        // 最终的合并区间
        List<int[]> result = new ArrayList<int[]>();

        for (int i = 0; i < intervals.length; i++) {

            // 比如 intervals = [[1,3],[2,6],[8,10],[15,18]]
            // i = 1
            // intervals[i] = [2,6]

            // left 表示第 i 个数组的最左边那个元素
            // left = 2
            int left = intervals[i][0];

            // left 表示第 i 个数组的最右边那个元素
            // right = 6
            int right = intervals[i][1];

            // 一开始, result 为空, 那么遇到第一个区间, 可以直接把它加进去
            if(result.size() == 0 ){

                // 直接把当前的区间加入到 result
                result.add(intervals[i]);

                // 开始执行下一次循环
                continue;
            }

            // 如果 result 不为空, 里面存在区间
            // 那么需要判断一下, result 里面的最后一个区间的最右端是否可以和 left 重合
            int[] last = result.get(result.size() - 1);

            // 如果发现最后一个区间的最右端在 left 的左侧, 说明此时无法合并
            if( last[1] < left ){

```

```

        // 直接把当前的区间加入到 result
        result.add(intervals[i]);

        // 否则, 说明需要合并
        // 将 result 的最后一项的 right 进行更新
    }else{

        // 此时, last[1] >= left 了, 需要判断一下 last[1] 和 right 的大小
        int newRight = Math.max(last[1],right);

        // 把较大值作为这个区间的最右端
        result.get(result.size() - 1)[1] = newRight;

    }

}

// 最后, 将 list 转二维数组
return result.toArray(new int[result.size()][]);
}
}

```

2、**C++** 代码

```

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        //通过 sort 函数对二维数组每一行按第一列元素进行排序
        //重写比较器方法,o1[] - o2[] 表示当 o1 大于 o2 时,将 o1 放在 o2 后面,即基本的升序排序
        //而 o1[0] - o2[0] 表示按二维数组的每一行第一列元素排序,类似的 o[1] - o2[1]代表按第二列进
        行排序
        sort(intervals.begin(), intervals.end());

        // 最终的合并区间
        vector<vector<int>> result;

        for (int i = 0; i < intervals.size(); i++) {

            // 比如 intervals = [[1,3],[2,6],[8,10],[15,18]]
            // i = 1
            // intervals[i] = [2,6]

            // left 表示第 i 个数组的最左边那个元素
            // left = 2
            int left = intervals[i][0];

            // left 表示第 i 个数组的最右边那个元素

```

```

// right = 6
int right = intervals[i][1];

// 一开始, result 为空, 那么遇到第一个区间, 可以直接把它加进去
if(result.size() == 0 ){

    // 直接把当前的区间加入到 result
    result.push_back(intervals[i]);

    // 开始执行下一次循环
    continue;
}

// 如果 result 不为空, 里面存在区间
// 那么需要判断一下, result 里面的最后一个区间的最右端是否可以和 left 重合
// int[] last = result.back();

// 如果发现最后一个区间的最右端在 left 的左侧, 说明此时无法合并
if( result.back()[1] < left ){

    // 直接把当前的区间加入到 result
    result.push_back(intervals[i]);

// 否则, 说明需要合并
// 将 result 的最后一项的 right 进行更新
}else{

    // 此时, last[1] >= left 了, 需要判断一下 last[1] 和 right 的大小
    int newRight = max(result.back()[1],right);

    // 把较大值作为这个区间的最右端
    result.back()[1] = newRight;

}

}

// 最后, 将 list 转二维数组
return result;

}

};

```

3、Python 代码

```
class solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:

        intervals.sort(key=lambda x: x[0])

        # 最终的合并区间
        result = []

        for i in range(len(intervals)) :

            # 比如 intervals = [[1,3],[2,6],[8,10],[15,18]]
            # i = 1
            # intervals[i] = [2,6]

            # left 表示第 i 个数组的最左边那个元素
            # left = 2
            left = intervals[i][0]

            # left 表示第 i 个数组的最右边那个元素
            # right = 6
            right = intervals[i][1]

            # 一开始, result 为空, 那么遇到第一个区间, 可以直接把它加进去
            if not result :

                # 直接把当前的区间加入到 result
                result.append(intervals[i])

                # 开始执行下一次循环
                continue

            # 如果 result 不为空, 里面存在区间
            # 那么需要判断一下, result 里面的最后一个区间的最右端是否可以和 left 重合
            last = result[-1]

            # 如果发现最后一个区间的最右端在 left 的左侧, 说明此时无法合并
            if last[1] < left :

                # 直接把当前的区间加入到 result
                result.append(intervals[i])

            # 否则, 说明需要合并
            # 将 result 的最后一项的 right 进行更新
            else:
```

```
# 此时, last[1] >= left 了, 需要判断一下 last[1] 和 right 的大小
newRight = max(last[1],right)

# 把较大值作为这个区间的最右端
result[-1][1] = newRight

# 最后, 将 list 转二维数组
return result
```

四、复杂度分析

时间复杂度: $O(n \log n)$

空间复杂度: $O(\log n)$

第三十一天

LeetCode 179、最大数

一、题目描述

给定一组非负整数 `nums`，重新排列每个数的顺序（每个数不可拆分）使之组成一个最大的整数。

注意：输出结果可能非常大，所以你需要返回一个字符串而不是整数。

示例 1：

```
输入: nums = [10,2]
输出: "210"
```

示例 2：

```
输入: nums = [3,30,34,5,9]
输出: "9534330"
```

提示：

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/largest-number/submissions/
class Solution {
    public String largestNumber(int[] nums) {

        // 先将 nums 转换为字符串数组的形式
        String[] strs = new String[nums.length];

        for(int i = 0 ; i < nums.length; i++){
            strs[i] = String.valueOf(nums[i]);
        }
        // 通过快速排序的方式，将字符串数组的每个字符按照约定的顺序进行排序
        quickSort(strs,0,strs.length - 1);

        // 如果 nums 中最大的数是 0 ，那么只需要显示 1 个 0 就行
        if (strs[strs.length - 1].equals("0")) {
            return "0";
        }

        // 再把字符串数组转字符串的形式
        StringBuilder ans = new StringBuilder();

        // 逆序输出结果就是最大的数
        for(int i = strs.length - 1 ; i >= 0 ; i--){
            String s = strs[i];
            ans.append(s);
        }

        return ans.toString();
    }

    // 函数传入待排序数组 nums
    // 排序区间的左端点 left
    // 排序区间的右端点 right
    private void quickSort(String[] strs,int left, int right){

        // 如果 left 大于等于 right，说明该区间只有 1 个或者没有元素
        if( left >= right ){
            // 无需再递归划分后再排序，直接返回
            return;
        }
    }
}
```

```

// 调用函数 partition, 将 left 和 right 之间的元素划分为左右两部分
int mid = partition(strs, left, right);

// 划分之后, 再对 mid 左侧的元素进行快速排序
quickSort(strs, left, mid-1);

// 划分之后, 再对 mid 右侧的元素进行快速排序
quickSort(strs, mid+1, right);
}

// 直接套之前的快速排序的代码进行修改
// 原先的小于的含义指的是数值上的小于, 比如 1 < 10
// 但现在的小于含义为: a + b 拼凑的字符串小于 b + a 拼凑的字符串
// 比如 a = 1, b = 10
// 那么 a + b = "110", b + a = "101"
// 显然, b + a < a + b
// 也就是说 a 应该放到 b 的后面来拼凑字符串
private int partition(String[] strs, int left, int right){

    // 经典快速排序的写法
    // 设置当前区间的第一个元素为基准元素
    String pivot = strs[left];

    // left 向右移动, right 向左移动, 直到 left 和 right 指向同一元素为止
    while( left < right ){

        // 当 pivot + strs[right] 的字符串小于 strs[right] + pivot 的字符串时
        // 说明 strs[right] 在正确的位置上, right 向左移动
        // 1 3
        while( left < right && (pivot + strs[right]).compareTo(strs[right] + pivot)
<= 0 ){

            // right 不断的向左移动
            right--;

        }

        // 此时, 跳出了上面这个 while 循环, 说明 pivot + strs[right] 的字符串大于
strs[right] + pivot 的字符串了
        // 说明 strs[right] 不在正确的位置上
        // 将此时的 strs[left] 赋值为 strs[right]
        // 执行完这个操作, 比 pivot 小的这个元素被移动到了左侧
        strs[left] = strs[right];

        // 当 strs[left] + pivot 的字符串小于 pivot + strs[left] 的字符串时
        // 说明 strs[left] 在正确的位置上, left 向右移动
        while( left < right && (strs[left] + pivot ).compareTo(pivot+ strs[left])
<= 0 ){

            // left 不断的向右移动
            left++;

        }
    }
}

```

```

        // 此时, 跳出了上面这个 while 循环, 说明 strs[left] + pivot 的字符串大于 pivot +
        strs[left] 的字符串了
        // 说明 strs[left] 不在正确的位置上
        // 将此时的 strs[right] 赋值为 strs[left]
        // 执行完这个操作, 比 pivot 大的这个元素被移动到了右侧
        strs[right] = strs[left];

    }

    // 此时, left 和 right 相遇, 那么需要将此时的元素设置为 pivot
    // 这个时候, pivot 的左侧元素都小于它, 右侧元素都大于它
    strs[left] = pivot;

    // 返回 left
    return left;
}
}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者: 程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

<https://leetcode-cn.com/problems/largest-number/>

class Solution:

def largestNumber(self, nums: List[int]) -> str:

先将 nums 转换为字符串数组的形式

strs = [str(num) for num in nums]

通过快速排序的方式, 将字符串数组的每个字符按照约定的顺序进行排序

self.quickSort(strs, 0, len(strs) - 1)

strs.reverse()

再把字符串数组转字符串的形式

return ''.join(strs) if strs[0] != '0' else '0'

函数传入待排序数组 nums

排序区间的左端点 left

排序区间的右端点 right

```

def quickSort(self, strs: List[str] , left : int ,right : int) :

    # 如果 left 大于等于 right, 说明该区间只有 1 个或者没有元素
    if left >= right :
        # 无需再递归划分后再排序, 直接返回
        return

    # 调用函数 partition, 将 left 和 right 之间的元素划分为左右两部分
    mid = self.partition(strs, left, right)

    # 划分之后, 再对 mid 左侧的元素进行快速排序
    self.quickSort(strs, left, mid-1)

    # 划分之后, 再对 mid 右侧的元素进行快速排序
    self.quickSort(strs, mid+1, right)

# 直接套之前的快速排序的代码进行修改
# 原先的小于的含义指的是数值上的小于, 比如 1 < 10
# 但现在的小于含义为: a + b 拼凑的字符串小于 b + a 拼凑的字符串
# 比如 a = 1 , b = 10
# 那么 a + b = "110", b + a = "101"
# 显然, b + a < a + b
# 也就是说 a 应该放到 b 的后面来拼凑字符串
def partition( self, strs: List[str] , left : int ,right) -> int :

    # 经典快速排序的写法
    # 设置当前区间的第一个元素为基准元素
    pivot = strs[left]

    # left 向右移动, right 向左移动, 直到 left 和 right 指向同一元素为止
    while left < right :

        # 当 pivot + strs[right] 的字符串小于 strs[right] + pivot 的字符串时
        # 说明 strs[right] 在正确的位置上, right 向左移动
        # 1 3
        while left < right and pivot + strs[right] <= strs[right] + pivot :
            # right 不断的向左移动
            right -= 1

        # 此时, 跳出了上面这个 while 循环, 说明 pivot + strs[right] 的字符串大于
        # strs[right] + pivot 的字符串了
        # 说明 strs[right] 不在正确的位置上
        # 将此时的 strs[left] 赋值为 strs[right]
        # 执行完这个操作, 比 pivot 小的这个元素被移动到了左侧
        strs[left] = strs[right]

        # 当 strs[left] + pivot 的字符串小于 pivot + strs[left] 的字符串时

```

```
# 说明 strs[left] 在正确的位置上, left 向右移动
while left < right and strs[left] + pivot <= pivot + strs[left] :
    # left 不断的向右移动
    left+=1

# 此时, 跳出了上面这个 while 循环, 说明 strs[left] + pivot 的字符串大于 pivot +
strs[left] 的字符串了
# 说明 strs[left] 不在正确的位置上
# 将此时的 strs[right] 赋值为 strs[left]
# 执行完这个操作, 比 pivot 大的这个元素被移动到了右侧
strs[right] = strs[left]

# 此时, left 和 right 相遇, 那么需要将此时的元素设置为 pivot
# 这个时候, pivot 的左侧元素都小于它, 右侧元素都大于它
strs[left] = pivot

# 返回 left
return left
```

LeetCode 215、数组中的第 K 个最大元素

一、题目描述

给定整数数组 `nums` 和整数 `k`, 请返回数组中第 `k` 个最大的元素。

请注意, 你需要找的是数组排序后的第 `k` 个最大的元素, 而不是第 `k` 个不同的元素。

示例 1:

输入: `[3,2,1,5,6,4]` 和 `k = 2`
输出: `5`

示例 2:

输入: `[3,2,3,1,2,4,5,5,6]` 和 `k = 4`
输出: `4`

提示:

- `1 <= k <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/kth-largest-element-in-an-array/
class Solution {
    public int findKthLargest(int[] nums, int k) {
        // 执行快速排序操作，定位找到下标为 k - 1 的那个元素
        int[] res = quickSort(nums, 0, nums.length - 1, k - 1);

        return res[k-1];
    }

    // 函数传入待排序数组 nums
    // 排序区间的左端点 left
    // 排序区间的右端点 right
    private int[] quickSort(int[] nums, int left, int right, int index){

        // 调用函数 partition，将 left 和 right 之间的元素划分为左右两部分
        int mid = partition(nums, left, right);

        // 如果 mid 下标恰巧为 index，那么找到了最小的 k 个数
        if (mid == index) {
            // 直接返回
            return Arrays.copyOf(nums, mid + 1);
        }

        // 如果 mid 下标大于 index，那么说明需要在左侧元素中去切分
        }else if( mid > index ){

            // 对 mid 左侧的元素进行快速排序
            return quickSort(nums, left, mid - 1, index );
        }else{

            // 对 mid 右侧的元素进行快速排序
            return quickSort(nums, mid + 1, right, index );
        }
    }

    private int partition(int[] nums, int left, int right){

        // 经典快速排序的写法
        // 设置当前区间的第一个元素为基准元素
```

```

int pivot = nums[left];

// left 向右移动, right 向左移动, 直到 left 和 right 指向同一元素为止
while( left < right ){

    // 只有当遇到小于 pivot 的元素时, right 才停止移动
    // 此时, right 指向了一个小于 pivot 的元素, 这个元素不在它该在的位置上
    while( left < right && nums[right] <= pivot ){
        // 如果 right 指向的元素是大于 pivot 的, 那么
        // right 不断的向左移动
        right--;
    }

    // 将此时的 nums[left] 赋值为 nums[right]
    // 执行完这个操作, 比 pivot 小的这个元素被移动到了左侧
    nums[left] = nums[right];

    // 只有当遇到大于 pivot left 才停止移动
    // 此时, left 指向了一个大于 pivot 的元素, 这个元素不在它该在的位置上
    while( left < right && nums[left] >= pivot){
        // 如果 left 指向的元素是小于 pivot 的, 那么
        // left 不断的向右移动
        left++;
    }

    // 将此时的 nums[right] 赋值为 nums[left]
    // 执行完这个操作, 比 pivot 大的这个元素被移动到了右侧
    nums[right] = nums[left];

}

// 此时, left 和 right 相遇, 那么需要将此时的元素设置为 pivot
// 这个时候, pivot 的左侧元素都小于它, 右侧元素都大于它
nums[left] = pivot;

// 返回 left
return left;

}
}

```

C++

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode-cn.com/problems/kth-largest-element-in-an-array/

```

```

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        // 执行快速排序操作，定位找到下标为 k - 1 的那个元素
        vector<int> res = quickSort(nums,0,nums.size() - 1,k - 1);

        return res[k-1];
    }

    // 函数传入待排序数组 nums
    // 排序区间的左端点 left
    // 排序区间的右端点 right
    vector<int> quickSort(vector<int>& nums,int left, int right , int index){

        // 调用函数 partition, 将 left 和 right 之间的元素划分为左右两部分
        int mid = partition(nums,left,right);

        // 如果 mid 下标恰巧为 index, 那么找到了最小的 k 个数
        if (mid == index) {
            // 直接返回
            return nums;

        // 如果 mid 下标大于 index, 那么说明需要在左侧元素中去切分
        }else if( mid > index ){

            // 对 mid 左侧的元素进行快速排序
            return quickSort(nums,left,mid - 1, index );
        }else{

            // 对 mid 右侧的元素进行快速排序
            return quickSort(nums,mid + 1,right, index );
        }
    }

    int partition(vector<int>& nums, int left ,int right){

        // 经典快速排序的写法
        // 设置当前区间的第一个元素为基准元素
        int pivot = nums[left];

        // left 向右移动, right 向左移动, 直到 left 和 right 指向同一元素为止
        while( left < right ){

            // 只有当遇到小于 pivot 的元素时, right 才停止移动
            // 此时, right 指向了一个小于 pivot 的元素, 这个元素不在它该在的位置上
            while( left < right && nums[right] <= pivot ){

```



```

        // 如果 right 指向的元素是大于 pivot 的, 那么
        // right 不断的向左移动
        right--;
    }

    // 将此时的 nums[left] 赋值为 nums[right]
    // 执行完这个操作, 比 pivot 小的这个元素被移动到了左侧
    nums[left] = nums[right];

    // 只有当遇到大于 pivot left 才停止移动
    // 此时, left 指向了一个大于 pivot 的元素, 这个元素不在它该在的位置上
    while( left < right && nums[left] >= pivot){
        // 如果 left 指向的元素是小于 pivot 的, 那么
        // left 不断的向右移动
        left++;
    }

    // 将此时的 nums[right] 赋值为 nums[left]
    // 执行完这个操作, 比 pivot 大的这个元素被移动到了右侧
    nums[right] = nums[left];

}

// 此时, left 和 right 相遇, 那么需要将此时的元素设置为 pivot
// 这个时候, pivot 的左侧元素都小于它, 右侧元素都大于它
nums[left] = pivot;

// 返回 left
return left;
}
};

```

四、复杂度分析

- 时间复杂度：O(N)，遍历数据 O(N)，堆内元素调整 O(log K)
- 空间复杂度：O(logn)

剑指 Offer 40. 最小的 K 个数

一、题目描述

输入整数数组 `arr`，找出其中最小的 `k` 个数。例如，输入 4、5、1、6、2、7、3、8 这 8 个数字，则最小的 4 个数字是 1、2、3、4。

示例 1：

输入: `arr = [3,2,1]`, `k = 2`
输出: `[1,2]` 或者 `[2,1]`

示例 2:

输入: `arr = [0,1,2,1]`, `k = 1`
输出: `[0]`

限制:

- `0 <= k <= arr.length <= 10000`
- `0 <= arr[i] <= 10000`

二、题目解析

这道题目最简单粗暴的方法当然是将数组 `arr` 按照从小到大的顺序整体排序之后, 获取数组的前 `k` 个数就行。

而整体排序的算法有很多种选择, 比如冒泡、选择、快速、堆排序等等。

这种暴力解法肯定不是面试官想要的回答, 因为我们没有利用好题目的全部条件。

再读一下这句话: **找出其中最小的 `k` 个数**。

这句话隐藏着以下几个意思:

- 1、找出的这 `k` 个数并不需要按照顺序排列。
- 2、如果一开始就知道某个数不在这 `k` 个数中, 完全可以将它丢到一旁。

也就意味着, 在排序过程中, 我们可以去不断的缩小排序的区间, 这里我们借助快速排序的代码, 稍微的改动几行就完成了这道题目。

具体操作如下:

- 1、以当前区间的第一个元素为基准元素 `pivot`, 根据快速排序的操作, 将当前区间划分为了三个区间。
 - 1、左侧区间均是小于等于基准元素 `pivot` 的元素
 - 2、中间区间均是等于基准元素 `pivot` 的元素
 - 3、右侧区间均是大于等于基准元素 `pivot` 的元素
- 2、对比基准元素 `pivot` 所在的下标 `index` 与 `k` 的关系
 - 1、**`index` 小于 `k`**, 说明从 0 到 `index` 这个左侧区间中的元素不足 `k` 个, 那么最小的 `k` 个数肯定部分是在这个区间, 还需要继续在右侧区间中寻找出一部分元素来填充, 因此对右侧区间进行快速排序即可
 - 2、**`index` 等于 `k`**, 说明从 0 到 `index` 这个区间中的所有元素就是那些最小的 `k` 个数, 将其返回。
 - 3、**`index` 大于 `k`**, 说明从 0 到 `index` 这个左侧区间中的元素超过了 `k` 个, 那么最小的 `k` 个数肯定都是在这个区间, 而中间、右侧区间均可以不去处理, 只需要继续对左侧区间进行快速排序即可, 找到那 `k` 个数。

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 剑指 offer 40. 最小的k个数: https://leetcode-cn.com/problems/zui-xiao-de-kge-shu-lcof/
class Solution {
    public int[] getLeastNumbers(int[] arr, int k) {

        if (k == 0 || arr.length == 0) {
            return new int[0];
        }

        // 执行快速排序操作，定位找到下标为 k - 1 的那个元素
        return quickSort(arr, 0, arr.length - 1, k - 1);
    }

    // 函数传入待排序数组 nums
    // 排序区间的左端点 left
    // 排序区间的右端点 right
    private int[] quickSort(int[] nums, int left, int right, int index) {

        // 调用函数 partition，将 left 和 right 之间的元素划分为左右两部分
        int mid = partition(nums, left, right);

        // 如果 mid 下标恰巧为 index，那么找到了最小的 k 个数
        if (mid == index) {
            // 直接返回
            return Arrays.copyOf(nums, mid + 1);
        }

        // 如果 mid 下标大于 index，那么说明需要在左侧元素中去切分
        }else if( mid > index ){

            // 对 mid 左侧的元素进行快速排序
            return quickSort(nums, left, mid - 1, index );
        }else{

            // 对 mid 右侧的元素进行快速排序
            return quickSort(nums, mid + 1, right, index );
        }
    }

    private int partition(int[] nums, int left, int right){

        // 经典快速排序的写法
        // 设置当前区间的第一个元素为基准元素
```

```

int pivot = nums[left];

// left 向右移动, right 向左移动, 直到 left 和 right 指向同一元素为止
while( left < right ){

    // 只有当遇到小于 pivot 的元素时, right 才停止移动
    // 此时, right 指向了一个小于 pivot 的元素, 这个元素不在它该在的位置上
    while( left < right && nums[right] >= pivot ){
        // 如果 right 指向的元素是大于 pivot 的, 那么
        // right 不断的向左移动
        right--;
    }

    // 将此时的 nums[left] 赋值为 nums[right]
    // 执行完这个操作, 比 pivot 小的这个元素被移动到了左侧
    nums[left] = nums[right];

    // 只有当遇到大于 pivot left 才停止移动
    // 此时, left 指向了一个大于 pivot 的元素, 这个元素不在它该在的位置上
    while( left < right && nums[left] <= pivot){
        // 如果 left 指向的元素是小于 pivot 的, 那么
        // left 不断的向右移动
        left++;
    }

    // 将此时的 nums[right] 赋值为 nums[left]
    // 执行完这个操作, 比 pivot 大的这个元素被移动到了右侧
    nums[right] = nums[left];

}

// 此时, left 和 right 相遇, 那么需要将此时的元素设置为 pivot
// 这个时候, pivot 的左侧元素都小于它, 右侧元素都大于它
nums[left] = pivot;

// 返回 left
return left;

}
}

```

算法的复杂度分析

空间复杂度 $O(1)$, 不需要额外空间。

时间复杂度的分析方法和快速排序类似。由于快速选择只需要递归一边的数组, 时间复杂度小于快速排序, 期望时间复杂度为 $O(n)$, 最坏情况下的时间复杂度为 $O(n^2)$ 。

第三十二天

LeetCode 46、全排列

一、题目描述

给定一个不含重复数字的数组 `nums`，返回其 *所有可能的全排列*。你可以 **按任意顺序** 返回答案。

示例 1:

输入: `nums = [1,2,3]`

输出: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

示例 2:

输入: `nums = [0,1]`

输出: `[[0,1],[1,0]]`

示例 3:

输入: `nums = [1]`

输出: `[[1]]`

提示:

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有整数 **互不相同**

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public List<List<Integer>> permute(int[] nums) {

        // 结果集合
        List<List<Integer>> sets = new ArrayList<List<Integer>>();

        // 每次的子集
        List<Integer> path = new ArrayList<Integer>();
```

```

        Boolean[] used = new Boolean[nums.length];

        // 每个元素默认一开始都没有被选择
        Arrays.fill(used, false);

        // 执行回溯算法
        backtrack( nums , path,used , sets);

        // 返回结果
        return sets;
    }

    // used 表示递归时正在访问的数组元素是否之前已经被访问过
    // nums 表示当前集合中的元素
    // path 表示每次递归后生成的子集
    // sets 表示最终生成的所有子集合
    private void backtrack( int[] nums , List<Integer> path , Boolean[] used ,
List<List<Integer>> sets ){

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        if( path.size() == nums.length){
            // 每次确定好一个子集，都把它加入到结果集合中
            sets.add(new ArrayList<Integer>(path));
            return;
        }

        for( int i = 0 ; i < nums.length ; i++ ){

            if(!used[i]){
                // 把本次递归访问的元素加入到 subset 数组中
                path.add(nums[i]);

                used[i] = true;

                // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
                // 本题不需要剪枝
                // 5、做出选择，递归调用该函数，进入下一层继续搜索
                // 递归
                backtrack( nums , path ,used, sets);

                // 6、撤销选择，回到上一层的状态
                // 取消对 nums[i] 的选择
                used[i] = false;
                path.remove(path.size() - 1);
            }
        }
    }
}

```

```

    }

}

}

```

Python

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        """
        :param nums: List[int]
        :return: List[List[int]]
        """
        # 结果集合
        sets = []

        # 每次的子集
        path = []

        used = [False] * len(nums)

        # 执行回溯算法
        self.backtrack(nums, path, used, sets)

        # 返回结果
        return sets

    def backtrack(self, nums, path, used, sets):
        """
        :param nums: List[int]
        :param path: List[int]
        :param used: List[bool]
        :param sets: List[List[int]]
        """
        # 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        if len(path) == len(nums):
            # 每次确定好一个子集，都把它加入到结果集合中
            sets.append(path[:])
            return

        for i in range(len(nums)):
            if not used[i]:
                # 把本次递归访问的元素加入到 subset 数组中
                path.append(nums[i])

                used[i] = True

                # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过

```

```
# 本题不需要剪枝
# 5、做出选择，递归调用该函数，进入下一层继续搜索
# 递归
self.backtrack(nums, path, used, sets)

# 6、撤销选择，回到上一层的状态
# 取消对 nums[i] 的选择
used[i] = False
path.pop()
```

LeetCode 47、全排列II

一、题目描述

给定一个可包含重复数字的序列 `nums`，*按任意顺序* 返回所有不重复的全排列。

示例 1：

```
输入：nums = [1,1,2]
输出：
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

示例 2：

```
输入：nums = [1,2,3]
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

提示：

- `1 <= nums.length <= 8`
- `-10 <= nums[i] <= 10`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {

        // 排序（升序或者降序都可以），排序是剪枝的前提
        Arrays.sort(nums);
```



```

// 结果集合
List<List<Integer>> sets = new ArrayList<List<Integer>>();

// 每次的子集
List<Integer> path = new ArrayList<Integer>();

Boolean[] used = new Boolean[nums.length];

// 每个元素默认一开始都没有被选择
Arrays.fill(used, false);

// 执行回溯算法
backtrack( nums , path,used , sets);

// 返回结果
return sets;
}

// used 表示递归时正在访问的数组元素是否之前已经被访问过
// nums 表示当前集合中的元素
// path 表示每次递归后生成的子集
// sets 表示最终生成的所有子集合
private void backtrack( int[] nums , List<Integer> path , Boolean[] used ,
List<List<Integer>> sets ){

// 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
if( path.size() == nums.length){
    // 每次确定好一个子集，都把它加入到结果集合中
    sets.add(new ArrayList<Integer>(path));
    return;
}

for( int i = 0 ; i < nums.length ; i++ ){

    if(!used[i]){

        if( i > 0 && nums[i] == nums[i - 1] && used[i-1]){
            continue;
        }
        // 把本次递归访问的元素加入到 subset 数组中
        path.add(nums[i]);

        used[i] = true;

        // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        // 本题不需要剪枝
    }
}
}

```

```

        // 5、做出选择，递归调用该函数，进入下一层继续搜索
        // 递归
        backtrack( nums , path ,used, sets);

        // 6、撤销选择，回到上一层的状态
        // 取消对 nums[i] 的选择
        used[i] = false;
        path.remove(path.size() - 1);

    }

}

}
}

```

Python

```

class Solution:
    def permuteUnique(self, nums: List[int]) -> List[List[int]]:
        """
        :param nums: List[int]
        :return: List[List[int]]
        """
        # 排序（升序或者降序都可以），排序是剪枝的前提
        nums.sort()

        # 结果集合
        sets = []

        # 每次的子集
        path = []

        used = [False] * len(nums)

        # 执行回溯算法
        self.backtrack(nums, path, used, sets)

        # 返回结果
        return sets

    def backtrack(self, nums, path, used, sets):
        """
        :param nums: List[int]
        :param path: List[int]
        :param used: List[bool]
        :param sets: List[List[int]]
        """
        # 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件

```

```

if len(path) == len(nums):
    # 每次确定好一个子集，都把它加入到结果集合中
    sets.append(path[:])
    return

for i in range(len(nums)):
    if not used[i]:
        if i > 0 and nums[i] == nums[i - 1] and used[i - 1]:
            continue
        # 把本次递归访问的元素加入到 subset 数组中
        path.append(nums[i])

        used[i] = True

        # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        # 本题不需要剪枝
        # 5、做出选择，递归调用该函数，进入下一层继续搜索
        # 递归
        self.backtrack(nums, path, used, sets)

        # 6、撤销选择，回到上一层的状态
        # 取消对 nums[i] 的选择
        used[i] = False
        path.pop()

```

LeetCode 78、子集

一、题目描述

给你一个整数数组 `nums`，数组中的元素 **互不相同**。返回该数组所有可能的子集（幂集）。

解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例 1:

```

输入: nums = [1,2,3]
输出: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]

```

示例 2:

```

输入: nums = [0]
输出: [[], [0]]

```

提示:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有元素 **互不相同**

二、题目解析

一般来说，回溯算法的思考步骤如下：

- 1、画出递归树，找到状态变量(回溯函数的参数)
- 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
- 3、确定选择列表，即需要把什么数据存储到结果里面
- 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
- 5、做出选择，递归调用该函数，进入下一层继续搜索
- 6、撤销选择，回到上一层的状态

翻译成代码为如下的形式，也就是回溯算法解题的一个模板：

```
// 1、画出递归树，找到状态变量(回溯函数的参数)
private void backtrack("原始参数") {

    // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
    if ("终止条件") {
        // 一些逻辑操作（可有可无，视情况而定）
        // 比如，在 N 皇后问题中，在这一步把数据加入到了结果里面
        添加操作
        return;
    }

    // 3、确定选择列表，即需要把什么数据存储到结果里面
    // for 循环就是一个选择的过程
    for ("遍历本层集合中元素") {

        // 一些逻辑操作（可有可无，视情况而定）
        // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过

        // 5、做出选择，递归调用该函数，进入下一层继续搜索
        // 递归
        backtrack("新的参数");

        // 一些逻辑操作（可有可无，视情况而定）

        // 6、撤销选择，回到上一层的状态
    }
}
```

那么对于子集这道问题来说，我们按照模板进行一个思考：

- 1、画出递归树，找到状态变量(回溯函数的参数)

结合最终结果来画递归树。

2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件。

当访问的数组下标超过了 nums 数组的长度时，递归结束。

3、确定选择列表，即需要把什么数据存储到结果里面。

把本次递归访问的元素加入到 subset 数组中

4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过

由于题目要求我们去存储所有的子集，也就意味着不存在丢弃的操作，也就不需要剪枝了。

5、做出选择，递归调用该函数，进入下一层继续搜索

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public List<List<Integer>> subsets(int[] nums) {

        // 结果集合
        List<List<Integer>> sets = new ArrayList<List<Integer>>();

        // 每次的子集
        List<Integer> subset = new ArrayList<Integer>();

        // 执行回溯算法
        backtrack( 0 , nums , subset , sets);

        // 返回结果
        return sets;
    }

    // i 表示递归时正在访问的数组元素下标
    // nums 表示当前集合中的元素
    // subset 表示每次递归后生成的子集，就是路径上的那些元素
    // sets 表示最终生成的所有子集合
    private void backtrack( int i , int[] nums , List<Integer> subset ,
        List<List<Integer>> sets ){

        // 每次确定好一个子集，都把它加入到结果集合中
        sets.add(new ArrayList<Integer>(subset));

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        // 本题中可以不加这个判断，大家可以思考一下为什么可以不加，结合 for 循环的边界来思考
        if( i >= nums.length){
```

```

        return;
    }

    for( int j = i ; j < nums.length ; j++ ){

        // 把本次递归访问的元素加入到 subset 数组中
        subset.add(nums[j]);

        // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        // 本题不需要剪枝

        // 5、做出选择，递归调用该函数，进入下一层继续搜索
        // 递归
        backtrack( j + 1 , nums , subset , sets);

        // 6、撤销选择，回到上一层的状态
        // 取消对 nums[i] 的选择
        subset.remove(subset.size() - 1);

    }

}
}

```

Python

```

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        """
        :param nums: List[int]
        :return: List[List[int]]
        """
        # 结果集合
        sets = []

        # 每次的子集
        subset = []

        # 执行回溯算法
        self.backtrack(0, nums, subset, sets)

        # 返回结果
        return sets

    def backtrack(self, i, nums, subset, sets):
        """
        :param i: int
        :param nums: List[int]
        :param subset: List[int]

```

```

:param sets: List[List[int]]
"""

# 每次确定好一个子集，都把它加入到结果集合中
sets.append(subset[:])

# 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
# 本题中可以不加这个判断，大家可以思考一下为什么可以不加，结合 for 循环的边界来思考
if i >= len(nums):
    return

for j in range(i, len(nums)):
    # 把本次递归访问的元素加入到 subset 数组中
    subset.append(nums[j])

    # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
    # 本题不需要剪枝

    # 5、做出选择，递归调用该函数，进入下一层继续搜索
    # 递归
    self.backtrack(j + 1, nums, subset, sets)

    # 6、撤销选择，回到上一层的状态
    # 取消对 nums[i] 的选择
    subset.pop()

```

LeetCode 90、子集II

一、题目描述

给你一个整数数组 `nums`，其中可能包含重复元素，请你返回该数组所有可能的子集（幂集）。

解集 **不能** 包含重复的子集。返回的解集中，子集可以按 **任意顺序** 排列。

示例 1：

```

输入：nums = [1,2,2]
输出：[[], [1], [1,2], [1,2,2], [2], [2,2]]

```

示例 2：

```

输入：nums = [0]
输出：[[], [0]]

```

提示：

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {

        // 先排序，这样才能对比当前元素和之前元素是否相同
        Arrays.sort(nums);

        // 结果集合
        List<List<Integer>> sets = new ArrayList<List<Integer>>();

        // 每次的子集
        List<Integer> subset = new ArrayList<Integer>();

        // 执行回溯算法
        backtrack( 0 , nums , subset , sets);

        // 返回结果
        return sets;
    }

    // i 表示递归时正在访问的数组元素下标
    // nums 表示当前集合中的元素
    // subset 表示每次递归后生成的子集
    // sets 表示最终生成的所有子集合
    private void backtrack( int i , int[] nums , List<Integer> subset ,
        List<List<Integer>> sets ){

        // 每次确定好一个子集，都把它加入到结果集合中
        sets.add(new ArrayList<Integer>(subset));

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        // 本题中可以不加这个判断，大家可以思考一下为什么可以不加，结合 for 循环的边界来思考
        if( i >= nums.length){
            return;
        }

        for( int j = i ; j < nums.length ; j++ ){

            // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
            if( j > i && nums[j] == nums[ j - 1 ] ) continue;
```



```

        // 把本次递归访问的元素加入到 subset 数组中
        subset.add(nums[j]);

        // 5、做出选择，递归调用该函数，进入下一层继续搜索
        // 递归
        backtrack( j + 1 , nums , subset , sets);

        // 6、撤销选择，回到上一层的状态
        // 取消对 nums[i] 的选择
        subset.remove(subset.size() - 1);

    }

}

}

```

Python

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        """
        :param nums: List[int]
        :return: List[List[int]]
        """
        # 先排序，这样才能对比当前元素和之前元素是否相同
        nums.sort()

        # 结果集合
        sets = []

        # 每次的子集
        subset = []

        # 执行回溯算法
        self.backtrack(0, nums, subset, sets)

        # 返回结果
        return sets

    def backtrack(self, i, nums, subset, sets):
        """
        :param i: int
        :param nums: List[int]
        :param subset: List[int]
        :param sets: List[List[int]]
        """
        # 每次确定好一个子集，都把它加入到结果集合中
        sets.append(subset[:])

```

```

# 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
# 本题中可以不加这个判断，大家可以思考一下为什么可以不加，结合 for 循环的边界来思考
if i >= len(nums):
    return

for j in range(i, len(nums)):

    # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
    if j > i and nums[j] == nums[j - 1]:
        continue

    # 把本次递归访问的元素加入到 subset 数组中
    subset.append(nums[j])

    # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过

    # 5、做出选择，递归调用该函数，进入下一层继续搜索
    # 递归
    self.backtrack(j + 1, nums, subset, sets)

    # 6、撤销选择，回到上一层的状态
    # 取消对 nums[i] 的选择
    subset.pop()

```

四、复杂度分析

时间复杂度： $O(n * 2^n)$ 。一共 2^n 个状态，每种状态需要 $O(n)$ 的时间来构造子集。

空间复杂度： $O(n)$ 。临时数组 subsets 的空间代价是 $O(n)$ ，递归时栈空间的代价为 $O(n)$ 。

第三十三天

LeetCode 17、电话号码的组合

一、题目描述

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按 **任意顺序** 返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

示例 1:

输入：digits = "23" 输出：["ad","ae","af","bd","be","bf","cd","ce","cf"]

示例 2:

输入：digits = "" 输出：[]

示例 3:

输入: digits = "2" 输出: ["a","b","c"]

提示:

- `0 <= digits.length <= 4`
- `digits[i]` 是范围 `['2', '9']` 的一个数字。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 电话号码的字母组合 (LeetCode 17) : https://leetcode.cn/problems/letter-combinations-of-a-phone-number/
class Solution {
    // 映射表
    Map<Character, String> phoneMap;

    // 结果
    List<String> ans;

    // 把 digits 作为全局变量使用方便
    String digits;

    public List<String> letterCombinations(String digits) {

        // 赋值
        this.digits = digits;

        // 初始化
        ans = new ArrayList<String>();

        // 边界处理
        if (digits.length() == 0) {
            return ans;
        }

        // 建立映射关系
        phoneMap = new HashMap<Character, String>() {{
            put('2', "abc");
            put('3', "def");
            put('4', "ghi");
            put('5', "jkl");
```

```

        put('6', "mno");
        put('7', "pqrs");
        put('8', "tuv");
        put('9', "wxyz");
    });

    // 使用深度优先搜索
    dfs( 0, "");

    // 返回结果
    return ans;
}

// 搜索框架
// 在整个搜索过程中，变化的是当前这个数字，比如 2 、3 、4
// index: 当前访问 digits 上的字符索引为多少
// str: 已经拼接的字符串
public void dfs(int index, String str) {

    // 递归的终止条件
    // 当访问完了每个位置的字符时，已经获取到一个完整的字符串了
    if (index == digits.length()) {

        // 记录结果
        ans.add(str);
        return;

    }

    // 获取 index 位置的字符
    // 假设输入的字符是"234"
    // 第一次递归时 index 为 0 , digit 为 2
    // 第二次递归时 index 为 1 , digit 为 3
    // 第三次递归时 index 为 2 , digit 为 4
    char digit = digits.charAt(index);

    // 通过 digit 这个 key 查找出哈希表 phoneMap 中对应的 value
    // 比如 digit 为 2 , 那么 letters 就是 abc
    String letters = phoneMap.get(digit);

    // 获取 letters 的长度
    int lettersCount = letters.length();

    // 访问 letters 中的每个字符
    for (int i = 0; i < lettersCount; i++) {

        // 调用下一层递归
        // 不断的把结果累加到 str 上
        dfs( index + 1, str + letters.charAt(i));
    }
}

```

```
    }  
  }  
}
```

2、**C++** 代码

3、Python 代码

```
#### 登录 AlgoMooc 官网获取更多算法图解  
#### https://www.algomooc.com  
#### 作者：程序员吴师兄  
#### 微信：278166530  
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀  
#### 电话号码的字母组合 (LeetCode 17) : https://leetcode.cn/problems/letter-combinations-of-a-phone-number/  
class Solution:  
    def letterCombinations(self, digits: str) -> List[str]:  
        # 初始化  
        ans = []  
  
        # 边界处理  
        if len(digits) == 0 :  
            return ans  
  
        # 建立映射关系  
        phoneMap = {  
            "2": "abc",  
            "3": "def",  
            "4": "ghi",  
            "5": "jkl",  
            "6": "mno",  
            "7": "pqrs",  
            "8": "tuv",  
            "9": "wxyz"  
        }  
  
        # 搜索框架  
        # 在整个搜索过程中，变化的是当前这个数字，比如 2 、 3 、 4  
        # index: 当前访问 digits 上的字符索引为多少  
        # str: 已经拼接的字符串  
        def dfs(index, str):  
  
            # 递归的终止条件  
            # 当访问完了每个位置的字符时，已经获取到一个完整的字符串了  
            if index == len(digits) :  
  
                # 记录结果  
                ans.append(str)
```

```

        return

    # 获取 index 位置的字符
    # 假设输入的字符是"234"
    # 第一次递归时 index 为 0 , digit 为 2
    # 第二次递归时 index 为 1 , digit 为 3
    # 第三次递归时 index 为 2 , digit 为 4
    digit = digits[index]

    # 通过 digit 这个 key 查找出哈希表 phoneMap 中对应的 value
    # 比如 digit 为 2 , 那么 letters 就是 abc
    letters = phoneMap[digit]

    # 访问 letters 中的每个字符
    for i in letters:

        # 调用下一层递归
        # 不断的把结果累加到 str 上
        dfs( index + 1, str + i)

    # 使用深度优先搜索
    dfs(0 , "")

    # 返回结果
    return ans

```

LeetCode 39、组合总和

一、题目描述

给你一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target`，找出 `candidates` 中可以使数字和为目标数 `target` 的 **所有不同组合**，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。

`candidates` 中的 **同一个** 数字可以 **无限制重复被选取**。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 `target` 的不同组合数少于 150 个。

示例 1：

输入: candidates = [2,3,6,7], target = 7

输出: [[2,2,3],[7]]

解释:

2 和 3 可以形成一组候选, $2 + 2 + 3 = 7$ 。注意 2 可以使用多次。

7 也是一个候选, $7 = 7$ 。

仅有这两种组合。

示例 2:

输入: candidates = [2,3,5], target = 8
输出: [[2,2,2,2],[2,3,3],[3,5]]

示例 3:

输入: candidates = [2], target = 1
输出: []

提示:

- `1 <= candidates.length <= 30`
- `1 <= candidates[i] <= 200`
- `candidate` 中的每个元素都 互不相同
- `1 <= target <= 500`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {

    List<List<Integer>> res = new ArrayList<>();

    public List<List<Integer>> combinationSum(int[] candidates, int target) {

        List<Integer> path = new ArrayList<>();

        backtrack( candidates , target , path , 0 );

        return res;
    }

    // 1、画出递归树，找到状态变量(回溯函数的参数)
    // start 表示递归时正在访问的数组元素下标
    // nums 表示当前集合中的元素
    // target 表示想在当前区间拼凑出的目标值
    // path 表示选择的路径
    private void backtrack(int[] nums,int target,List<Integer> path,int start) {

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        if (target < 0) {
```

```

        // 一些逻辑操作（可有可无，视情况而定）
        // 比如，在 N 皇后问题中，在这一步把数据加入到了结果里面
        return;
    }

    if(target == 0 ){

        // 找到一个组合了
        res.add(new ArrayList(path));

        return;
    }

    // 3、确定选择列表，即需要把什么数据存储到结果里面
    // for 循环就是一个选择的过程
    // i 等于 start 表示，后续可以选的元素一开始只能从 start 开始
    // 比如 nums = [2,3,6,7]
    // i = 1，指向了元素 3 ，表示当前后续选择的过程中，只能从 3 开始选，可以重复选 3 ，但无法选
2 了
    // i = 2，指向了元素 6 ，表示当前后续选择的过程中，只能从 6 开始选，可以重复选 6 ，但无法选
2、3 了
    for (int i = start ; i < nums.length ; i++ ) {

        // 当前路径上可以把 nums[i] 加上
        path.add(nums[i]);

        // 一些逻辑操作（可有可无，视情况而定）
        // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        // 需要剪枝
        // 此时，目标值 target，已经从 target 变成了 target - nums[i]
        // 接下来需要去【某个区间中】拼凑 target - nums[i]
        // 由于 同一个 数字可以 无限制重复被选取
        // 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，依旧可以继续从使用 nums[i]
开始
        // 而 i 前面的元素，比如 num[i-1]、 num[i-2]无法继续使用，实现了剪枝操作
        int nowposition = i ;

        // 5、做出选择，递归调用该函数，进入下一层继续搜索
        // 递归
        backtrack(nums, target - nums[i] , path , nowposition );

        // 一些逻辑操作（可有可无，视情况而定）

        // 6、撤销选择，回到上一层的状态
        path.remove(path.size()-1);
    }
}
}

```


Python

```
class solution:
    def __init__(self):
        self.res = []

    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        path = []
        self.backtrack(candidates, target, path, 0)
        return self.res

# 1、画出递归树，找到状态变量(回溯函数的参数)
# start 表示递归时正在访问的数组元素下标
# nums 表示当前集合中的元素
# target 表示想在当前区间拼凑出的目标值
# path 表示选择的路径
def backtrack(self, nums: List[int], target: int, path: List[int], start: int) ->
None:

    # 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
    if target < 0:
        # 一些逻辑操作（可有可无，视情况而定）
        # 比如，在 N 皇后问题中，在这一步把数据加入到了结果里面
        return

    if target == 0:
        # 找到一个组合了
        self.res.append(list(path))
        return

    # 3、确定选择列表，即需要把什么数据存储到结果里面
    # for 循环就是一个选择的过程
    # i 等于 start 表示，后续可以选的元素一开始只能从 start 开始
    # 比如 nums = [2,3,6,7]
    # i = 1，指向了元素 3，表示当前后续选择的过程中，只能从 3 开始选，可以重复选 3，但无法选
    2 了
    # i = 2，指向了元素 6，表示当前后续选择的过程中，只能从 6 开始选，可以重复选 6，但无法选
    2、3 了
    for i in range(start, len(nums)):
        # 当前路径上可以把 nums[i] 加上
        path.append(nums[i])
        # 一些逻辑操作（可有可无，视情况而定）
        # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        # 需要剪枝
        # 此时，目标值 target，已经从 target 变成了 target - nums[i]
        # 接下来需要去【某个区间中】拼凑 target - nums[i]
        # 由于 同一个 数字可以 无限制重复被选取
        # 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，依旧可以继续从使用 nums[i]
        开始
        # 而 i 前面的元素，比如 num[i-1]、 num[i-2]无法继续使用，实现了剪枝操作
```

```
now_position = i

# 5、做出选择，递归调用该函数，进入下一层继续搜索
# 递归
self.backtrack(nums, target - nums[i], path, now_position)

# 一些逻辑操作（可有可无，视情况而定）

# 6、撤销选择，回到上一层的状态
path.pop()
```

LeetCode 40、组合总和II

一、题目描述

给定一个候选人编号的集合 `candidates` 和一个目标数 `target`，找出 `candidates` 中所有可以使数字和为 `target` 的组合。

`candidates` 中的每个数字在每个组合中只能使用 一次。

注意：解集不能包含重复的组合。

示例 1:

```
输入: candidates = [10,1,2,7,6,1,5], target = 8,
输出:
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

示例 2:

```
输入: candidates = [2,5,2,1,2], target = 5,
输出:
[
  [1,2,2],
  [5]
]
```

提示:

- `1 <= candidates.length <= 100`
- `1 <= candidates[i] <= 50`
- `1 <= target <= 30`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {

    List<List<Integer>> res=new ArrayList<>();

    public List<List<Integer>> combinationSum2(int[] candidates, int target) {

        List<Integer> path = new ArrayList<>();

        // 先排序，方便把相同的元素集中到一起
        Arrays.sort(candidates);

        backtrack( candidates , target , path , 0 );

        return res;
    }

    // 1、画出递归树，找到状态变量(回溯函数的参数)
    // start 表示递归时正在访问的数组元素下标
    // nums 表示当前集合中的元素
    // target 表示想在当前区间拼凑出的目标值
    // path 表示选择的路径
    private void backtrack(int[] nums,int target,List<Integer> path,int start) {

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        if (target < 0) {
            // 一些逻辑操作（可有可无，视情况而定）
            // 比如，在 N 皇后问题中，在这一步把数据加入到了结果里面
            return;
        }

        if(target == 0 ){

            // 找到一个组合了
            res.add(new ArrayList(path));

            return;
        }

        // 3、确定选择列表，即需要把什么数据存储到结果里面
        // for 循环就是一个选择的过程
```

```

// i 等于 start 表示，后续可以选的元素一开始只能从 start 开始
// 比如 nums = [2,3,6,7]
// i = 1，指向了元素 3，表示当前后续选择的过程中，只能从 3 开始选，可以重复选 3，但无法选
2 了
// i = 2，指向了元素 6，表示当前后续选择的过程中，只能从 6 开始选，可以重复选 6，但无法选
2、3 了
for (int i = start ; i < nums.length ; i++ ) {

    // 剪枝操作：同一层相同数值的结点，从第 2 个开始，结果一定发生重复，因此跳过，用
continue
    if( i > start && nums[i] == nums[i - 1]){
        continue;
    }

    // 当前路径上可以把 nums[i] 加上
    path.add(nums[i]);

    // 一些逻辑操作（可有可无，视情况而定）
    // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
    // 需要剪枝
    // 此时，目标值 target，已经从 target 变成了 target - nums[i]
    // 接下来需要去【某个区间中】拼凑 target - nums[i]
    // 由于每个数字在每个组合中只能使用 一次
    // 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，需要从使用 nums[i+1] 开始
    // 而 i 前面的元素，比如 num[i]、 num[i-1]无法继续使用，实现了剪枝操作
    int nowposition = i ;

    // 5、做出选择，递归调用该函数，进入下一层继续搜索
    // 递归
    backtrack(nums, target - nums[i] , path , nowposition + 1 );

    // 一些逻辑操作（可有可无，视情况而定）

    // 6、撤销选择，回到上一层的状态
    path.remove(path.size()-1);
}
}
}

```

Python

```

class Solution:
    def __init__(self):
        self.res = []

    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        path = []

```

```

# 先排序，方便把相同的元素集中到一起
candidates.sort()

self.backtrack(candidates, target, path, 0)
return self.res

# 1、画出递归树，找到状态变量(回溯函数的参数)
# start 表示递归时正在访问的数组元素下标
# nums 表示当前集合中的元素
# target 表示想在当前区间拼凑出的目标值
# path 表示选择的路径
def backtrack(self, nums: List[int], target: int, path: List[int], start: int) ->
None:
    # 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
    if target < 0:
        # 一些逻辑操作（可有可无，视情况而定）
        # 比如，在 N 皇后问题中，在这一步把数据加入到了结果里面
        return

    if target == 0:
        # 找到一个组合了
        self.res.append(list(path))
        return

    # 3、确定选择列表，即需要把什么数据存储到结果里面
    # for 循环就是一个选择的过程
    # i 等于 start 表示，后续可以选的元素一开始只能从 start 开始
    # 比如 nums = [2,3,6,7]
    # i = 1, 指向了元素 3，表示当前后续选择的过程中，只能从 3 开始选，可以重复选 3，但无法选
    # 2 了
    # i = 2, 指向了元素 6，表示当前后续选择的过程中，只能从 6 开始选，可以重复选 6，但无法选
    # 2、3 了
    for i in range(start, len(nums)):

        # 剪枝操作：同一层相同数值的结点，从第 2 个开始，结果一定发生重复，因此跳过，用
        continue

        if i > start and nums[i] == nums[i - 1]:
            continue

        # 当前路径上可以把 nums[i] 加上
        path.append(nums[i])
        # 一些逻辑操作（可有可无，视情况而定）
        # 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
        # 需要剪枝
        # 此时，目标值 target，已经从 target 变成了 target - nums[i]
        # 接下来需要去【某个区间中】拼凑 target - nums[i]
        # 由于 同一个 数字可以 无限制重复被选取
        # 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，依旧可以继续从使用 nums[i]
        开始

```

```
# 而 i 前面的元素, 比如 num[i-1]、 num[i-2]无法继续使用, 实现了剪枝操作
now_position = i + 1

# 5、做出选择, 递归调用该函数, 进入下一层继续搜索
# 递归
self.backtrack(nums, target - nums[i], path, now_position)

# 一些逻辑操作 (可有可无, 视情况而定)

# 6、撤销选择, 回到上一层的状态
path.pop()
```

LeetCode 77、组合

一、题目描述

给定两个整数 `n` 和 `k`, 返回范围 `[1, n]` 中所有可能的 `k` 个数的组合。

你可以按 **任何顺序** 返回答案。

示例 1:

输入: `n = 4, k = 2`

输出:

```
[
  [2,4],
  [3,4],
  [2,3],
  [1,2],
  [1,3],
  [1,4],
]
```

示例 2:

输入: `n = 1, k = 1`

输出: `[[1]]`

提示:

- `1 <= n <= 20`
- `1 <= k <= n`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {

    List<List<Integer>> res = new ArrayList<>();

    public List<List<Integer>> combine(int n, int k) {

        List<Integer> path = new ArrayList<>();

        backtrack( n , k , path , 1 );

        return res;
    }

    // 1、画出递归树，找到状态变量(回溯函数的参数)
    private void backtrack(int n,int k,List<Integer> path,int start) {

        // 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
        if(k == 0 ){

            // 找到一个组合了
            res.add(new ArrayList(path));

            return;
        }

        // 3、确定选择列表，即需要把什么数据存储到结果里面
        // for 循环就是一个选择的过程

        for (int i = start ; i <= n - k + 1 ; i++ ) {

            // 当前路径上可以把 nums[i] 加上
            path.add(i);

            // 一些逻辑操作（可有可无，视情况而定）
            // 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
            // 需要剪枝
            // 此时，目标值 target，已经从 target 变成了 target - nums[i]
            // 接下来需要去【某个区间中】拼凑 target - nums[i]
            // 由于 同一个 数字可以 无限重复被选取
        }
    }
}
```

```

// 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，依旧可以继续从使用 nums[i]
开始

// 而 i 前面的元素，比如 num[i-1]、 num[i-2]无法继续使用，实现了剪枝操作
int nowposition = i + 1 ;

// 5、做出选择，递归调用该函数，进入下一层继续搜索
// 递归
backtrack(n, k - 1 , path , nowposition );

// 一些逻辑操作（可有可无，视情况而定）

// 6、撤销选择，回到上一层的状态
path.remove(path.size()-1);
}
}
}

```

Python代码

```

class Solution:

    def __init__(self):
        self.res = []

    def combine(self, n: int, k: int) -> List[List[int]]:
        """
        :param n: int
        :param k: int
        :return: List[List[int]]
        """
        path = []

        self.backtrack(n, k, path, 1)

        return self.res

# 1、画出递归树，找到状态变量(回溯函数的参数)
def backtrack(self, n: int, k: int, path: List[int], start: int) -> None:
    # 2、寻找结束条件，由于回溯算法是借助递归实现，所以也就是去寻找递归终止条件
    if k == 0:
        # 找到一个组合了
        self.res.append(list(path))
        return

    # 3、确定选择列表，即需要把什么数据存储到结果里面
    # for 循环就是一个选择的过程
    for i in range(start, n - k + 1 + 1):
        # 当前路径上可以把 nums[i] 加上
        path.append(i)

```


开始

```
# 一些逻辑操作（可有可无，视情况而定）
# 4、判断是否需要剪枝，去判断此时存储的数据是否之前已经被存储过
# 需要剪枝
# 此时，目标值 target，已经从 target 变成了 target - nums[i]
# 接下来需要去【某个区间中】拼凑 target - nums[i]
# 由于 同一个 数字可以 无限制重复被选取
# 当前正在使用 nums[i]，那么为了拼凑 target - nums[i]，依旧可以继续从使用 nums[i]

# 而 i 前面的元素，比如 num[i-1]、 num[i-2]无法继续使用，实现了剪枝操作
now_position = i + 1
# 5、做出选择，递归调用该函数，进入下一层继续搜索
# 递归
self.backtrack(n, k - 1, path, now_position)
# 一些逻辑操作（可有可无，视情况而定）
# 6、撤销选择，回到上一层的状态
path.pop()
```

第三十四天

LeetCode 37、解数独

一、题目描述

编写一个程序，通过填充空格来解决数独问题。

数独的解法需遵循如下规则：

1. 数字 1-9 在每一行只能出现一次。
2. 数字 1-9 在每一列只能出现一次。
3. 数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。（请参考示例图）

数独部分空格内已填入了数字，空白格用 '.' 表示。

示例 1：

img

输入: board = `[["5","3",".",".","7",".",".",".","."],`
`["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".","6","."],`
`["8",".",".","6",".",".","3"],["4",".",".","8",".","3",".","1"],`
`["7",".",".","2",".",".","6"],[".","6",".",".",".","2","8","."],`
`[[".","4","1","9",".","5"],[".",".","8",".","7","9"]]`
 输出: `[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],`
`["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],`
`["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],`
`["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],`
`["3","4","5","2","8","6","1","7","9"]]`
 解释: 输入的数独如上图所示, 唯一有效的解决方案如下所示:

提示:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` 是一位数字或者 `'.'`
- 题目数据 保证 输入数独仅有一个解

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 解数独 (LeetCode 37) : https://leetcode.cn/problems/sudoku-solver/
class Solution {
    public void solveSudoku(char[][] board) {

        // 回溯算法
        backTrace(board , 0 , 0 );

    }

    // board : 二维矩阵
    // row    : 第几行, 从 0 开始计数
    // col    : 第几列, 从 0 开始计数
    private boolean backTrace(char[][] board, int row, int col) {

        // 递归终止条件
        // 最后一行已经填充完了所有数字, 来到了第 board.length 行
        // 由于 row 是从 0 开始计数, 说明此时 board 里面的每一行都已经填充好数字
        // 获取到结果了
        if( row == board.length ) return true;
    }
}
```

```

// 如果当前行的最后一列也遍历完毕，开始从下一行的第一列（索引为 0 的位置）开始
if( col == board.length ) return backTrace( board , row + 1 , 0 );

// 如果当前位置已经有数字了，那么直接处理下一个位置，即当前这一行的下一个列
if( board[row][col] != '.' ) return backTrace(board , row , col + 1);

// 经过上面三个判断，当前位置终于可以开始填数字了
// 每个位置可以填充的数字都能从 1 到 9 进行选择
for( char i = '1' ; i <= '9' ; i++){

    // 填充过程中需要判断填充的数字是否遵循了 3 个规则
    // 如果发现不遵循，也就无法继续填充后面的数字了
    // 需要考虑在当前位置填充另外一个数字，i 调整为 i + 1 即可
    if( !isValid(board , row , col , i )) continue;

    // 否则，说明填充的数字 i 符合【目前】的要求
    // 把它填充到 board[row][col]
    board[row][col] = i;

    // 开始考虑下一个位置的填充情况了
    // 由于是一行行、一列列去填充，下一个位置是 col + 1 的位置
    // 只有它能保证正确填充，当前这个位置填充的 i 才是正确
    if( backTrace( board , row , col + 1)) return true;

    // 否则，当前这个 i 只是符合了【目前】的要求，不符合全部的要求
    // 于是，这个位置不能填 i ，还原为空白格再继续分析下去
    board[row][col] = '.';

}

return false;

}

// 判断当前位置是否遵循数独的 3 个规则
private static boolean isValid(char[][] board, int row, int col, char c) {

    for (int i = 0; i < 9; i++) {

        // 1、数字 1-9 在每一行只能出现一次
        // 判断当前这一行是否已经有 c 这个数字存在
        if (board[row][i] == c)
            return false;

        // 2、数字 1-9 在每一列只能出现一次
        // 判断当前这一列是否已经有 c 这个数字存在
        if (board[i][col] == c)
            return false;
    }
}

```

```

        // 3、数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。
        // 判断当前这个九宫格是否已经有 c 这个数字存在
        if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == c)
            return false;
    }
    // 如果遵循了这 3 个规则
    // 说明可以填充 c 这个数字
    return true;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    void solveSudoku(vector<vector<char>>& board) {
        backTrace(board, 0, 0);
    }

    bool backTrace(vector<vector<char>>& board, int row, int col) {
        if (row == board.size()) return true;
        if (col == board[0].size()) return backTrace(board, row + 1, 0);
        if (board[row][col] != '.') return backTrace(board, row, col + 1);

        for (char c = '1'; c <= '9'; c++) {
            if (isValid(board, row, col, c)) {
                board[row][col] = c;
                if (backTrace(board, row, col + 1)) return true;
                board[row][col] = '.';
            }
        }

        return false;
    }

    bool isValid(vector<vector<char>>& board, int row, int col, char c) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == c) return false;
            if (board[i][col] == c) return false;
            if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == c) return false;
        }

        return true;
    }
};

```

3、Python 代码

```
class solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """
        self.backTrace(board, 0, 0)

# board : 二维矩阵
# row    : 第几行, 从 0 开始计数
# col    : 第几列, 从 0 开始计数
def backTrace(self, board: List[List[str]], row: int, col: int) -> bool:

    # 递归终止条件
    # 最后一行已经填充完了所有数字, 来到了第 board.length 行
    # 由于 row 是从 0 开始计数, 说明此时 board 里面的每一行都已经填充好数字
    # 获取到结果了
    if row == len(board):
        return True

    # 如果当前行的最后一列也遍历完毕, 开始从下一行的第一列 (索引为 0 的位置) 开始
    if col == len(board):
        return self.backTrace(board, row + 1, 0)

    # 如果当前位置已经有数字了, 那么直接处理下一个位置, 即当前这一行的下一个列
    if board[row][col] != '.':
        return self.backTrace(board, row, col + 1)

    # 经过上面三个判断, 当前位置终于可以开始填数字了
    # 每个位置可以填充的数字都能从 1 到 9 进行选择
    for i in range(1, 10):
        # 填充过程中需要判断填充的数字是否遵循了 3 个规则
        # 如果发现不遵循, 也就无法继续填充后面的数字了
        # 需要考虑在当前位置填充另外一个数字, i 调整为 i + 1 即可
        c = str(i)
        if not self.isValid(board, row, col, c):
            continue

        # 否则, 说明填充的数字 i 符合【目前】的要求
        # 把它填充到 board[row][col]
        board[row][col] = str(i)

        # 开始考虑下一个位置的填充情况了
        # 由于是一行行、一列列去填充, 下一个位置是 col + 1 的位置
        # 只有它能保证正确填充, 当前这个位置填充的 i 才是正确
        if self.backTrace(board, row, col + 1):
            return True
```

```

        # 否则，当前这个 i 只是符合了【目前】的要求，不符合全部的要求
        # 于是，这个位置不能填 i，还原为空白格再继续分析下去
        board[row][col] = '.'

    return False

# 判断当前位置是否遵循数独的 3
def isValid(self, board: List[List[str]], row: int, col: int, c: str):
    for i in range(9):
        if board[row][i] == c:
            return False
        if board[i][col] == c:
            return False
        if board[3 * (row // 3) + i // 3][3 * (col // 3) + i % 3] == c:
            return False
    return True

```

LeetCode 37、解数独（位运算法）

一、题目描述

编写一个程序，通过填充空格来解决数独问题。

数独的解法需遵循如下规则：

1. 数字 1-9 在每一行只能出现一次。
2. 数字 1-9 在每一列只能出现一次。
3. 数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。（请参考示例图）

数独部分空格内已填入了数字，空白格用 '.' 表示。

示例 1：

img

```

输入: board = [
["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
["8",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".","1"],
["7",".",".","2",".",".","6",".","8","."],
[".",".","4","1","9",".",".","8","7","9"],
["5","3","4","6","7","8","9","1","2"],
["1","9","8","3","4","2","5","6","7"],
["8","5","9","7","6","1","4","2","3"],
["4","2","6","8","5","3","7","9","1"],
["7","1","3","9","2","4","8","5","6"],
["9","6","1","5","3","7","2","8","4"],
["2","8","7","4","1","9","6","3","5"],
["3","4","5","2","8","6","1","7","9"]
]

```

解释：输入的数独如上图所示，唯一有效的解决方案如下所示：

提示：

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` 是一位数字或者 `'.'`
- 题目数据 保证 输入数独仅有一个解

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 有效的数独 (LeetCode 36) (位运算法) : https://leetcode.cn/problems/sudoku-solver/
class Solution {

    // 数独的长宽大小
    final int N = 9;

    // 用数组记录每一行、每一列、每一个小九宫格的情况
    // 000000001 表示当前这一行中出现了数字 1
    // 000000010 表示当前这一行中出现了数字 2
    // 000000011 表示当前这一行中出现了数字 1 和数字 2
    // 100000100 表示当前这一行中出现了数字 3 和数字 9

    // rows 里面的每个元素可以用一个二进制来表示当前这一行数字的出现情况
    private int[] rows = new int[N];

    // columns 里面的每个元素可以用一个二进制来表示当前这一列数字的出现情况
    private int[] columns = new int[N];

    // subboxes 里面的每个元素可以用一个二进制来表示当前这一小九宫格数字的出现情况
    private int[][] subboxes = new int[3][3];

    public void solveSudoku(char[][] board) {

        // 统计未填的个数
        int count = 0;

        // 通过两轮循环, 获取 board 里面有多少个空白格需要填充
        for (int i = 0; i < N; i++) {

            for (int j = 0; j < N; j++) {

                char ch = board[i][j];

                // 如果当前位置为空白格, 记录一下
```

```

        if (ch == '.') {

            count++;

            // 如果当前位置为数字
        } else {
            // 那么这个位置所处的行、列、小九宫格都得更新一下数据
            fillNumber(i, j, ch - '1', true);
        }
    }
}

//上面是一些计算前的准备工作，从这里开始调用回溯算法
backtrace(board, count);
}

private boolean backtrace(char[][] board, int count) {
    // 如果 board 里面的所有空格都填好了数字，没有多余的位置填充
    if (count == 0) {
        // 递归终止
        return true;
    }

    // 开始寻找合适的位置进行分析
    // 即去寻找出某个位置可以选择数字比较少
    int[] pos = getMinOkMaskCountPos(board);

    // 获取到这个位置的坐标
    int x = pos[0], y = pos[1];

    // 再获取这个位置究竟有哪些数字可以进行选择
    // 比如坐标 ( 4 , 1 ) 有 3 、 4 、 5 三个数字符合要求
    int mask = getMask(x, y);

    // 每个位置可以填充的数字都能从 1 到 9 进行选择
    for (char c = '1'; c <= '9'; c++) {

        // 由于数组的索引是从 0 开始的，而小方格的值是从 1 开始的
        int index = c - '1';

        // 判断这个位置是否可以填字符数字 c
        // 比如 mask 为 101111100
        // 可以填充 1 、 2 、 8 ，那么就先尝试填 1 ，发现是可以的，再继续填充下一个
        if (testMask(mask, index)) {

            // 坐标 ( x , y ) 的位置的填充的值为 index 的数字
            // 比如 mask 为 101111100
            // 填入 1 之后
            // mask 可以选择的数字就变成了 2 和 8
            // 即 mask 变成了 101111101

```



```

        fillNumber(x, y, index, true);

        // 把字符数字 c 填入到坐标 ( x , y ) 的位置
        board[x][y] = c;

        // 开始考虑填充下一个位置
        // 如果下一个位置和后续所有位置都填充成功
        // 那么说明当前填充 c 这个数字是正确的
        // 直接返回 true, 不用再去考虑后续的情况了
        if (backtrace(board, count - 1))
            return true;

        // 否则, 还原现场
        // 1、当前位置还原为空白格
        board[x][y] = '.';

        // 2、当前位置填充的数字抹去
        fillNumber(x, y, index, false);
    }
}
return false;
}

// 在 board 上坐标为 ( x , y ) 的位置上填充数字 index
// 填充完毕后, 当前位置所处的行、列、小九宫格记录的数据情况都需要更新
// fill 为 true 表示填充数字
// fill 为 false 表示抹去数字, 还原为空白格
private void fillNumber(int x, int y, int index, boolean fill) {
    if (fill) {
        // 比如 index 为 2
        // mask 就是 000000100
        int mask = 1 << index;
        rows[x] = rows[x] | mask;
        columns[y] = columns[y] | mask;
        subboxes[x / 3][y / 3] = subboxes[x / 3][y / 3] | mask;
    } else {

        // 取反操作
        int mask = ~(1 << index);
        // 通过按位与把这一位更新为 0
        rows[x] = rows[x] & mask;
        columns[y] = columns[y] & mask;
        subboxes[x / 3][y / 3] = subboxes[x / 3][y / 3] & mask;
    }
}

// 当前位置处于第 x 行、第 y 列、第 ( x / 3 , y / 3 ) 个小九宫格
// row[x] 获取第 x 行的数字出现情况, 比如 100000100, 表示这一行出现了 3 、 9 这两个数字

```

数字

```
// columns[y] 获取第 y 列的数字出现情况，比如 010000010，表示这一列出现了 2 、 8 这两个数字

// subboxes[x / 3][y / 3] 获取第第( x / 3 , y / 3 )个小九宫格的数字出现情况
// 这三者通过按位或的操作之后，可以获取到当前位置剩下那些数字可以填写
// 111111100 ，剩下 1 和 2 两个数字可以填
// | 111010100 ，剩下 1 、 2 、 4 、 6 四个数字可以填
// | 001010110 ，剩下 1 、 4 、 6 、 8 、 9 五个数字可以填
// = 111111110
// 只有最后一个位置可以填
private int getMask(int x, int y) {

    // 按位或操作
    return rows[x] | columns[y] | subboxes[x / 3][y / 3];

}

// 统计当前位置可以填充的数字个数有几个
// 比如目前的信息分析出当前位置可以填充 2 、 3 这两个数字
private int getCount(int mask) {
    int count = 0;
    for (int i = 0; i < N; i++) {
        // 只需要获取 mask 的二进制的 0 的个数就可以知道答案
        if ((mask & (1 << i)) == 0)
            count++;
    }
    return count;
}

// 由于 mask 可以填充的数字可以有多个，需要从小到大的顺序去尝试
// mask 为 101111100
// 可以填充 1 、 2 、 8 ，那么就先尝试填 1
private boolean testMask(int mask, int index) {
    return (mask & (1 << index)) == 0;
}

// 查看所有的空白格
// 判断哪个位置的空白格内可填数字比较少
// 那么就优先去填这个位置，所以返回这个位置的下标
private int[] getMinOkMaskCountPos(char[][] board) {

    // 一维数组，包含了两个元素，分别表示坐标 x 和坐标 y
    int[] res = new int[2];

    // 默认每个位置可以选择的数字总数为 10 个（一个不可能达到的数字）
    // 然后通过不断比较的操作获取到最小值
    int min = 10;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            // 当前位置为空白格才去判断
```

```

        if (board[i][j] == '.') {

            // 1、通过对当前位置的行、列、小九宫格同时进行按位或的操作后
            // 可以获取到当前这个位置有哪些数字可以填进去
            // 比如最终结果是 111111100 ， 表示只有 1 、 2 这两个数字符合要求
            int mask = getMask(i, j);

            // 2、通过 mask 可以知道有几个数字符合要求
            int count = getCount(mask);

            // 3、不断的比较，从而知道哪个位置可以选择的数字最小
            // 把它的坐标记录到 res 里面
            if (count < min) {
                min = count;
                res[0] = i;
                res[1] = j;
            }
        }
    }

    // 返回最合适的那个坐标
    return res;
}

```

LeetCode 51、N 皇后

一、题目描述

n 皇后问题 研究的是如何将 **n** 个皇后放置在 **n×n** 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 **n**，返回所有不同的 **n 皇后问题** 的解决方案。

每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 **'Q'** 和 **'.'** 分别代表了皇后和空位。

输入：n = 4
 输出：[[".Q...", "...Q", "Q...", "...Q."],
 ["...Q.", "Q...", "...Q", ".Q..."]]
 解释：4 皇后问题存在两个不同的解法。

二、题目解析

三、参考代码

1、Java 代码

```
class solution {

    // 保存所有符合要求的解
    List<List<String>> res = new ArrayList<>();

    public List<List<String>> solveNQueens(int n) {

        // attack 用来表示皇后的攻击范围
        int[][] attack = new int[n][n];
        // queen 用来记录皇后的位置
        char[][] queen = new char[n][n];

        // 初始化二维数组 queen 中所有的元素为 '.'
        for(char[] c : queen) {
            Arrays.fill(c, '.');
        }

        // 初始化二维数组 attack 中所有的元素为 0
        // 0 代表没有皇后能攻击得到
        // 1 代表出于任意一个皇后的攻击范围内
        for(int[] c : attack) {
            Arrays.fill(c, 0);
        }

        // 从棋盘的第 0 行第 0 列处理 n 皇后的情况
        backtrack(0,n,queen,attack);

        // 最后，返回所有符合要求的解
        return res;
    }

    // 很显然，每一行只能放置一个皇后，所以我们每一行每一行的来放置皇后
    // k 表示当前处理的行
    // n 表示需要放置多少个皇后，同时也代表棋盘的大小为 n * n
    // queen 用来记录皇后的位置
    // attack 用来表示皇后的攻击范围
    private void backtrack(int k ,int n , char[][] queen,int[][] attack){

        // 如果发现在棋盘的最后一行放置好了皇后，那么就说明找到了一组符合要求的解
        if(k == n){
            // 由于 queen 为二维字符数组，所以需要转换为字符串数组
            List<String> list = new ArrayList<>();
            // 遍历二维数组 queen
            // 取出 queen 的每一行字符数组 c
            for (char[] c : queen) {
                // 把字符数组 c 中的所有字符转换为字符串的形式进行拼凑
                // 比如 ['.','Q','.','.']
            }
        }
    }
}
```

```

        // 转换为 '.Q..'
        // 把这个字符串加入到 list 中
        list.add(String.valueOf(c));
    }

    // list 即为一组符合要求的解，把它加入到结果数组中
    res.add(list);
    // 由于遍历完了所有的行，无需再遍历下去，所以返回
    return;
}

// 每一行只能放置一个皇后
// 并且每一列也只能放置一个皇后
// 所以在 k 行中，从 0 列到 n - 1 列，判断皇后应该放置到哪个位置
for(int i = 0 ; i < n ; i++){
    // 如果发现 attack[k][i] == 0
    // 说明这个位置不在任何一个皇后的攻击范围内
    // 所以可以考虑放置皇后
    if(attack[k][i] == 0){

        // 如果在 ( k , i ) 位置放置了皇后，那么就需要考虑在 k + 1 行应该怎么放置其它的皇后了

        // 由于有可能在 ( k , i ) 位置放置了皇后之后，在后续的其它行会无法再放置其它的皇后
        // 那么就需要回到 ( k , i ) 的状态，考虑能不能在 ( k , i + 1 ) 的位置放置
        // 为了能够回到 ( k , i ) 的状态，所以需要先记录此时的 attack
        // 使用一个临时的二维数组，深度拷贝 attack
        // 如果不使用深度拷贝，而是直接使用 int[][] temp = c
        // 会导致 attack 发生改变是 temp 也会发生改变
        // 这样也就无法保存之前的状态了
        int[][] temp = new int[n][n];

        // 通过两个 for 循环，把 attack 中的所有元素深度拷贝到 temp
        for(int l = 0 ; l < n ; l++){
            for( int m = 0 ; m < n ; m++){
                temp[l][m] = attack[l][m];
            }
        }

        // queen 用来记录皇后的位置
        // 那么 ( k , i ) 的位置 queen[k][i] = 'Q'
        queen[k][i] = 'Q';

        // 由于新放置了一个皇后，所以攻击范围又更多了
        // 所以需要更新 attack 数组
        // 新放置皇后的坐标为 ( k , i ) ，同样的需要更新它的八个方向
        checkQueenAttack(k,i,attack);

        // 如果在 ( k , i ) 位置放置了皇后，那么就需要考虑在 k + 1 行应该怎么放置其它的
    }
}

```

```

        // 递归的调用 backtrack 在 k + 1 行放置皇后
        backtrack(k + 1, n, queen, attack);

        // 递归结束后, 拿走皇后, 恢复 attack 的状态, 考虑能不能在 ( k , i + 1 ) 的位置放置
        attack = temp;

        // 恢复 queen 的状态, 说明此时皇后不放置在 ( k , i ) 位置
        queen[k][i] = '.';
    }
}

}

// 坐标 ( x , y ) 为皇后所处的位置
// 更新 attack
private void checkQueenAttack(int x ,int y,int[][] attack){

    // 对于每一个坐标 (x,y) 来说, 都有上、下、左、右、左上、左下、右上、右下 八个方向
    // 【左上】的坐标为 (x - 1, y - 1)
    // 【上】的坐标为 (x - 1, y )
    // 【右上】的坐标为 (x - 1, y + 1)
    // 【左】的坐标为 (x, y + 1)
    // 【右】的坐标为 (x , y - 1)
    // 【左下】的坐标为 (x + 1, y - 1)
    // 【下】的坐标为 (x + 1, y)
    // 【右下】的坐标为 (x + 1, y + 1)
    // 通过两个一维数组可以表示这八个方向
    // dx 表示 x 的方向
    int dx[] = { -1 , -1 , -1 , 0 , 0 , 1 , 1 , 1 };
    // dy 表示 y 的方向
    int dy[] = { -1 , 0 , 1 , -1 , 1 , -1 , 0 , 1 };

    // 皇后所处的坐标肯定是皇后能攻击的位置, 设置为 1
    attack[x][y] = 1;

    // 以坐标 ( x , y ) 为中心, 去更新它八个方向的坐标
    for(int j = 0 ; j < 8; j++){
        // 由内向外的进行更新
        for(int i = 1 ; i < attack.length ; i++){
            // 新的位置的坐标行为 x + i * dx[j]
            int nx = x + i * dx[j];
            // 新的位置的坐标列为 y + i * dy[j]
            int ny = y + i * dy[j];

            // 如果新位置的坐标在 n * n 的棋盘范围内
            if(nx >= 0 && nx < attack.length && ny >= 0 && ny < attack.length){
                // 那么这些位置就是在坐标为 (x, y) 的皇后的攻击范围内, 更新为 1
                attack[nx][ny] = 1;
            }
        }
    }
}

```

```

    }
}
}
}
}

```

2、**C++** 代码

```

class Solution {
    // 保存所有符合要求的解
    vector<vector<string>> res;
public:
    vector<vector<string>> solveNQueens(int n) {

        // attack 用来表示皇后的攻击范围
        vector<vector<int>> attack(n,vector<int>(n));

        for( int i = 0 ; i < n ; i++){
            for(int j = 0 ; j < n ; j++){
                attack[i][j] = 0;
            }
        }

        // queen 用来记录皇后的位置
        vector<vector<string>> queen(n,vector<string>(n));

        for( int i = 0 ; i < n ; i++){
            for(int j = 0 ; j < n ; j++){
                queen[i][j] = ".";
            }
        }

        // 从棋盘的第 0 行第 0 列处理 n 皇后的情况
        backtrack(0,n,queen,attack);

        // 最后，返回所有符合要求的解
        return res;
    }

    // 很显然，每一行只能放置一个皇后，所以我们每一行每一行的来放置皇后
    // k 表示当前处理的行
    // n 表示需要放置多少个皇后，同时也代表棋盘的大小为 n * n
    // queen 用来记录皇后的位置
    // attack 用来表示皇后的攻击范围
    void backtrack(int k, int n , vector<vector<string>> &queen,vector<vector<int>>
&attack){

```

```

// 如果发现在棋盘的最后一行放置好了皇后，那么就说明找到了一组符合要求的解
if(k == n){

    vector<string> ans = {};

    for(int i = 0 ; i < n ; i++){

        string temp = "";

        for( int j = 0 ; j < n ; j++){

            temp += queen[i][j];

        }

        ans.push_back(temp);

    }

    res.push_back(ans);

    // 由于遍历完了所有的行，无需再遍历下去，所以返回
    return;
}

// 每一行只能放置一个皇后
// 并且每一列也只能放置一个皇后
// 所以在 k 行中，从 0 列到 n - 1 列，判断皇后应该放置到哪个位置
for(int i = 0 ; i < n ; i++){
    // 如果发现 attack[k][i] == 0
    // 说明这个位置不在任何一个皇后的攻击范围内
    // 所以可以考虑放置皇后
    if(attack[k][i] == 0){

        // 如果在 ( k , i ) 位置放置了皇后，那么就需要考虑在 k + 1 行应该怎么放置其它的皇后

        // 由于有可能在( k , i ) 位置放置了皇后之后，在后续的其它行会无法再放置其它的皇后
        // 那么就需要回到 ( k , i ) 的状态，考虑能不能在 ( k , i + 1 )的位置放置
        // 为了能够回到 ( k , i ) 的状态，所以需要先记录此时的 attack
        // 使用一个临时的二维数组，深度拷贝 attack
        // 如果不使用深度拷贝，而是直接使用 int[][] temp = c
        // 会导致 attack 发生改变是 temp 也会发生改变
        // 这样也就无法保存之前的状态了
        vector<vector<int>> temp(n,vector<int>(n));

        // 通过两个 for 循环，把 attack 中的所有元素深度拷贝到 temp
        for(int l = 0 ; l < n ; l++){
            for( int m = 0 ; m < n ; m++){

```

后了

皇后

```
        temp[l][m] = attack[l][m];
    }
}

// queen 用来记录皇后的位置
// 那么 ( k , i ) 的位置 queen[k][i] = 'Q'
queen[k][i] = 'Q';

// 由于新放置了一个皇后，所以攻击范围又更多了
// 所以需要更新 attack 数组
// 新放置皇后的坐标为 ( k , i ) ，同样的需要更新它的八个方向
checkQueenAttack(k,i,attack);

// 如果在 ( k , i ) 位置放置了皇后，那么就需要考虑在 k + 1 行应该怎么放置其它的

// 递归的调用 backtrack 在 k + 1 行放置皇后
backtrack(k + 1,n,queen,attack);

// 递归结束后，拿走皇后，恢复 attack 的状态，考虑能不能在 ( k , i + 1 )的位置放置
attack = temp;

// 恢复 queen 的状态，说明此时皇后不放置在( k , i ) 位置
queen[k][i] = '.';
}
}

}

// 坐标 ( x , y ) 为皇后所处的位置
// 更新 attack
void checkQueenAttack(int x ,int y, vector<vector<int>> &attack){

    // 对于每一个坐标 (x,y) 来说，都有上、下、左、右、左上、左下、右上、右下 八个方向
    // 【左上】的坐标为 (x - 1, y - 1)
    // 【上】的坐标为 (x - 1, y )
    // 【右上】的坐标为 (x - 1, y + 1)
    // 【左】的坐标为 (x, y + 1)
    // 【右】的坐标为 (x , y - 1)
    // 【左下】的坐标为 (x + 1, y - 1)
    // 【下】的坐标为 (x + 1, y)
    // 【右下】的坐标为 (x + 1, y + 1)
    // 通过两个一维数组可以表示这八个方向
    // dx 表示 x 的方向
    int dx[8] = { -1 , -1 , -1 , 0 , 0 , 1 , 1 , 1 };
    // dy 表示 y 的方向
    int dy[8] = { -1 , 0 , 1 , -1 , 1 , -1 , 0 , 1 };

    // 皇后所处的坐标肯定是皇后能攻击的位置，设置为 1
```

```

attack[x][y] = 1;

// 以坐标 ( x , y ) 为中心, 去更新它八个方向的坐标
for(int j = 0 ; j < 8; j++){
    // 由内向外的进行更新
    for(int i = 1 ; i < attack.size() ; i++){
        // 新的位置的坐标行为 x + i * dx[j]
        int nx = x + i * dx[j];
        // 新的位置的坐标列为 y + i * dy[j]
        int ny = y + i * dy[j];

        // 如果新位置的坐标在 n * n 的棋盘范围内
        if(nx >= 0 && nx < attack.size() && ny >= 0 && ny < attack.size()){
            // 那么这些位置就是在坐标为 (x, y) 的皇后的攻击范围内, 更新为 1
            attack[nx][ny] = 1;
        }
    }
}
};

```

3、Python 代码

```

class Solution:
    def solvenQueens(self, n: int) -> List[List[str]]:
        # 更新皇后的坐标[x][y]之后更新attack的位置
        def updateAttack(x,y,attack):
            # 更改attack[x][y]上、下、左、右、左上、左下、右上、右下的0为1
            # dx表示x的方向, dy表示y的方向
            dx = [-1, -1, 1, 0, 0, 1, 1, 1]
            dy = [-1, 0, 1, -1, 1, -1, 0, 1]
            # 八个方向更新
            for j in range(8):
                # 内到外更新
                for i in range(n):
                    nx = x + i * dx[j]
                    ny = y + i * dy[j]
                    # 新坐标在棋盘内则更新
                    if nx >= 0 and nx < n and ny >= 0 and ny < n:
                        attack[nx][ny] = 1

        # k为当前处理的行
        # n表示需要放置的皇后数和棋盘大小
        # queen皇后的位置
        # attack攻击范围
        def backtrack(k,n,queen,attack):
            print(k)
            # 如果棋盘最后一行放置好了皇后则找到一组解

```

```

    if k == n:
        r1 = []
        # queen为分隔的二维数组，去转为字符串数组
        for i in queen:
            r1.append(''.join(i))
        res.append(r1)
        return

# 每行每列只能放一个皇后，所以在k行中，从0~n-1列判断皇后放置的位置
for i in range(n):

    # 如果attack[k][i]为0，即不在攻击范围内，可考虑放置皇后
    if attack[k][i] == 0:
        # k行i列放置后需考虑k+1行如何放置
        # 若k+1无法放置，需回退，考虑k行i+1列能否放置
        # 用临时数组记录k行i列时的attack状态
        temp = attack[:]
        # 要使用深度拷贝
        temp = copy.deepcopy(attack)

        queen[k][i] = 'Q'
        # 更新attack数组
        updateAttack(k, i, attack)
        # print(attack, queen, 'after')
        # 递归的调用backtrack考虑在k+1行放置皇后
        backtrack(k+1, n, queen, attack)

        # 递归结束移走皇后恢复[k][i]不能放置状态
        attack = temp
        # print(attack, 3)
        # 恢复queen状态
        queen[k][i] = '.'

res = []
# attack表示皇后攻击范围
# 初始化攻击为0
'''0表示没有皇后能攻击得到，1表示处于任意一个皇后的攻击范围内'''
attack = []
for i in range(n):
    attack.append([0] * n)
# queen记录皇后位置
# 初始化queen所有元素为'.'
queen = []
for i in range(n):
    queen.append(['.'] * n)
# 从棋盘的第0行第0列处理n皇后情况
backtrack(0, n, queen, attack)
# 返回结果
return res

```

第三十五天

LeetCode 1020、飞地的数量

一、题目描述

给你一个大小为 $m \times n$ 的二进制矩阵 `grid`，其中 `0` 表示一个海洋单元格、`1` 表示一个陆地单元格。

一次 **移动** 是指从一个陆地单元格走到另一个相邻（上、下、左、右）的陆地单元格或跨过 `grid` 的边界。

返回网格中 **无法** 在任意次数的移动中离开网格边界的陆地单元格的数量。

示例 1:

img

```
输入: grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]
输出: 3
解释: 有三个 1 被 0 包围。一个 1 没有被包围，因为它在边界上。
```

示例 2:

img

```
输入: grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]
输出: 0
解释: 所有 1 都在边界上或可以到达边界。
```

提示:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 500`
- `grid[i][j]` 的值为 `0` 或 `1`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode.cn/problems/number-of-enclaves/
class Solution {
```

```

public int numEnclaves(int[][] grid) {

    // 矩阵的行数
    int m = grid.length;

    // 矩阵的列数
    int n = grid[0].length;

    for (int i = 0; i < m; i++){

        for (int j = 0; j < n; j++){

            // i == 0: 表示在第 0 行, 处于边界位置
            // j == 0: 表示在第 0 列, 处于边界位置
            // i == (m - 1): 表示在最后一行, 处于边界位置
            // j == (n - 1): 表示在最后一列, 处于边界位置
            // 当格子处于上述任意一种情况的时候, 同时还是一个【陆地格子】
            // 那么需要基于这个格子继续去搜索
            if ((i == 0 || j == 0 || i == (m - 1) || j == (n - 1)) && grid[i][j] ==
1){

                // 在 dfs 搜索过程中, 会不断的把从边界过来的【陆地格子】变成【海洋格子】
                dfs(grid, i, j);
            }
        }
    }

    // 初始化 result, 用来记录飞地的数量
    int result = 0 ;

    // 经过 dfs 搜索之后的 grid 二维矩阵, 从边界过来的【陆地格子】都变成【海洋格子】
    // 那么剩下的那些 1 都是飞地
    for (int i = 0; i < m; i++){

        for (int j = 0; j < n; j++){

            if (grid[i][j] == 1){
                result++;
            }

        }
    }
    return result;
}

public void dfs(int[][] grid, int i, int j) {

```

```

// dfs搜索、递归终止条件
// i < 0 , 越界
// j < 0 , 越界
// i >= grid.length , 越界
// j >= grid[0].length , 越界
// grid[i][j] == 0 , 代表当前岛屿无法延伸下去
if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length || grid[i][j] ==
0) {

    // 直接返回
    return ;

}

// 将当前单元格置为 0 , 避免其它格子 dfs 过程中又把它加入到计算中
grid[i][j] = 0;

// 在当前单元格的四个方向开始搜索
// 上: ( 0 , -1 )
// 下: ( 0 , 1 )
// 左: ( -1 , 0 )
// 右: ( 1 , 0 )

// dx 为行的方向数组
int dx[] = { -1 , 1 , 0 , 0 };

// dy 为行的方向数组
int dy[] = { 0 , 0 , -1 , 1 };

// 朝着这四个方向开始延伸搜索下去
for (int index = 0; index < 4; ++index) {

    // 下一个即将去搜索网格的横坐标
    int next_i = i + dx[index];

    // 下一个即将去搜索网格的纵坐标
    int next_j = j + dy[index];

    // 继续搜索
    dfs(grid,next_i,next_j);

}

}
}

```

2、C++ 代码

```
class solution {
public:
    int numEnclaves(vector<vector<int>>& grid) {

        // 矩阵的行数
        int m = grid.size();

        // 矩阵的列数
        int n = grid[0].size();

        // 从左到右，一行行开始搜索
        for (int i = 0; i < m; i++) {

            // 从上到下，一列列开始搜索
            for (int j = 0; j < n; j++) {

                // i == 0: 表示在第 0 行，处于边界位置
                // j == 0: 表示在第 0 列，处于边界位置
                // i == (m - 1): 表示在最后一行，处于边界位置
                // j == (n - 1): 表示在最后一列，处于边界位置
                // 当格子处于上述任意一种情况的时候，同时还是一个【陆地格子】
                // 那么需要基于这个格子继续去搜索
                if ((i == 0 || j == 0 || i == (m - 1) || j == (n - 1)) && grid[i][j] ==
1){

                    // 在 dfs 搜索过程中，会不断的把从边界过来的【陆地格子】变成【海洋格子】
                    dfs(grid, i, j);

                }

            }

        }

        // 初始化 result，用来记录飞地的数量
        int result = 0 ;

        // 经过 dfs 搜索之后的 grid 二维矩阵，从边界过来的【陆地格子】都变成【海洋格子】
        // 那么剩下的那些 1 都是飞地
        for (int i = 0; i < m; i++){

            for (int j = 0; j < n; j++){

                if (grid[i][j] == 1){
                    result++;
                }

            }

        }

    }
}
```

```

        return result;

    }

public:
    void dfs(vector<vector<int>>& grid, int i, int j) {

        // dfs搜索、递归终止条件
        // i < 0 , 越界
        // j < 0 , 越界
        // i >= board.length , 越界
        // j >= board[0].length , 越界
        // board[i][j] == x , 不需要继续搜索下去
        // board[i][j] == N , 这个格子已经被访问过, 不需要继续搜索下去
        if (i < 0 || j < 0 || i >= grid.size() || j >= grid[0].size() || grid[i][j] ==
0) {

            return;

        }

        // 将当前单元格置为 0 , 避免其它格子 dfs 过程中又把它加入到计算中
        grid[i][j] = 0;

        // 在当前单元格的四个方向开始搜索
        // 上: ( 0 , -1 )
        // 下: ( 0 , 1 )
        // 左: ( -1 , 0 )
        // 右: ( 1 , 0 )

        // dx 为行的方向数组
        int dx[4] = { -1 , 1 , 0 , 0 };

        // dy 为行的方向数组
        int dy[4] = { 0 , 0 , -1 , 1 };

        // 朝着这四个方向开始延伸搜索下去
        for (int index = 0; index < 4; index++) {

            // 下一个即将去搜索网格的横坐标
            int next_i = i + dx[index];

            // 下一个即将去搜索网格的纵坐标
            int next_j = j + dy[index];

            // 继续搜索
            dfs(grid,next_i,next_j);

```



```

    }
}
};

```

3、Python 代码

```

class solution:
    def numEnclaves(self, grid: List[List[int]]) -> int:
        # 矩阵的行数
        m = len(grid)

        # 矩阵的列数
        n = len(grid[0])

        # 从左到右，一行行开始搜索
        # 从上到下，一列列开始搜索
        for i in range(0 , m ) :

            for j in range( 0 , n ):

                if ((i == 0 or j == 0 or i == (m - 1) or j == (n - 1)) and grid[i][j]
== 1) :

                    self.dfs(grid,i,j)

        res = 0

        for i in range(0 , m ) :

            for j in range( 0 , n ):

                if grid[i][j] == 1 :

                    res += 1

        return res

def dfs(self, grid: List[List[str]], i : int , j : int ) -> None:
    # dfs搜索、递归终止条件
    # i < 0 , 越界
    # j < 0 , 越界
    # i >= board.length , 越界
    # j >= board[0].length , 越界
    # board[i][j] == x , 不需要继续搜索下去
    # board[i][j] == N , 这个格子已经被访问过，不需要继续搜索下去
    if i < 0 or j < 0 or i >= len(grid) or j >= len(grid[0]) or grid[i][j] == 0 :
        return

```

```

# 将当前单元格置为 0，避免其它格子 dfs 过程中又把它加入到计算中
grid[i][j] = 0

# 在当前单元格的四个方向开始搜索
# 上：( 0, -1 )
# 下：( 0, 1 )
# 左：( -1, 0 )
# 右：( 1, 0 )
# 朝着这四个方向开始延伸搜索下去
for dx, dy in ((0,1), (1,0), (0,-1), (-1,0)):

    # 下一个即将去搜索网格的横坐标
    next_i = i + dx

    # 下一个即将去搜索网格的纵坐标
    next_j = j + dy

    self.dfs(grid, next_i, next_j)

```

四、复杂度分析

时间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。深度优先搜索过程中，每一个点至多只会被标记一次。

空间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。主要为深度优先搜索的栈的开销。

LeetCode 130、被围绕的区域

一、题目描述

给你一个 $m \times n$ 的矩阵 `board`，由若干字符 `'x'` 和 `'o'`，找到所有被 `'x'` 围绕的区域，并将这些区域里所有的 `'o'` 用 `'x'` 填充。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode.cn/problems/surrounded-regions/
class Solution {
    public void solve(char[][] board) {

        // 矩阵的行数

```

```

int m = board.length;

// 矩阵的列数
int n = board[0].length;

// 利用两个 for 循环，遍历矩阵中的一些特殊的单元格，将它们赋值为 N
// 特殊的单元格：必然是从边界处开始的
for (int i = 0; i < m; i++){

    for (int j = 0; j < n; j++){

        // i == 0: 表示在第 0 行，处于边界位置
        // j == 0: 表示在第 0 列，处于边界位置
        // i == (m - 1): 表示在最后一行，处于边界位置
        // j == (n - 1): 表示在最后一列，处于边界位置
        if (i == 0 || j == 0 || i == (m - 1) || j == (n - 1)){

            // 从这些单元格开始向腹部延伸下去，找到所有和边界单元格想通的那些 O
            dfs(board, i, j);
        }
    }
}

// 第二个 for 循环，开始修改
for (int i = 0; i < m; i++){

    for (int j = 0; j < n; j++){

        // board[i][j] == 'O' 表示无法从边界格子中延伸过来
        // 表示被 'X' 围绕的区域
        if (board[i][j] == 'O'){

            // 修改为 X
            board[i][j] = 'X';
        }

        // board[i][j] == 'N' 表示是从边界格子中延伸过来
        if (board[i][j] == 'N'){

            // 修改为 O
            board[i][j] = 'O';
        }
    }
}

```

```

}

public void dfs(char[][] board, int i, int j) {

    // dfs搜索、递归终止条件
    // i < 0 , 越界
    // j < 0 , 越界
    // i >= board.length , 越界
    // j >= board[0].length , 越界
    // board[i][j] == x , 不需要继续搜索下去
    // board[i][j] == N , 这个格子已经被访问过, 不需要继续搜索下去
    if (i < 0 || j < 0 || i >= board.length || j >= board[0].length || board[i][j]
== 'x' || board[i][j] == 'N') {

        return ;

    }

    // 否则说明当前单元格是 O, 最后需要保留下来的那种
    // 将当前单元格置为 N , 避免其它格子 dfs 过程中又把它加入到计算中
    board[i][j] = 'N';

    // 在当前单元格的四个方向开始搜索
    // 上: ( 0 , -1 )
    // 下: ( 0 , 1 )
    // 左: ( -1 , 0 )
    // 右: ( 1 , 0 )

    // dx 为行的方向数组
    int dx[] = { -1 , 1 , 0 , 0 };

    // dy 为列的方向数组
    int dy[] = { 0 , 0 , -1 , 1 };

    // 朝着这四个方向开始延伸搜索下去
    for (int index = 0; index < 4; index++) {

        // 下一个即将去搜索网格的横坐标
        int next_i = i + dx[index];

        // 下一个即将去搜索网格的纵坐标
        int next_j = j + dy[index];

        // 继续搜索
        dfs(board, next_i, next_j);

    }

}
}

```

```
}
```

2、**C++** 代码

```
class Solution {
public:
    void solve(vector<vector<char>>& board) {

        // 矩阵的行数
        int m = board.size();

        // 矩阵的列数
        int n = board[0].size();

        // 利用两个 for 循环，遍历矩阵中的一些特殊的单元格，将它们赋值为 N
        // 特殊的单元格：必然是从边界处开始的
        for (int i = 0; i < m; i++){

            for (int j = 0; j < n; j++){

                // i == 0: 表示在第 0 行，处于边界位置
                // j == 0: 表示在第 0 列，处于边界位置
                // i == (m - 1): 表示在最后一行，处于边界位置
                // j == (n - 1): 表示在最后一列，处于边界位置
                if (i == 0 || j == 0 || i == (m - 1) || j == (n - 1)){

                    // 从这些单元格开始向腹部延伸下去，找到所有和边界单元格想通的那些 o
                    dfs(board, i, j);
                }
            }
        }

        // 第二个 for 循环，开始修改
        for (int i = 0; i < m; i++){

            for (int j = 0; j < n; j++){

                // board[i][j] == 'O' 表示无法从边界格子中延伸过来
                // 表示被 'X' 围绕的区域
                if (board[i][j] == 'O'){

                    // 修改为 x
                    board[i][j] = 'X';
                }

                // board[i][j] == 'N' 表示是从边界格子中延伸过来
                if (board[i][j] == 'N'){

                    // 修改为 o
                }
            }
        }
    }
};
```

```

        board[i][j] = 'O';
    }

    }

}

public:
    void dfs(vector<vector<char>>& board, int i, int j) {

        // dfs搜索、递归终止条件
        // i < 0 , 越界
        // j < 0 , 越界
        // i >= board.length , 越界
        // j >= board[0].length , 越界
        // board[i][j] == x , 不需要继续搜索下去
        // board[i][j] == N , 这个格子已经被访问过, 不需要继续搜索下去
        if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size() || board[i][j]
== 'x' || board[i][j] == 'N') {

            return ;

        }

        // 否则说明当前单元格是 O, 最后需要保留下来的那种
        // 将当前单元格置为 N , 避免其它格子 dfs 过程中又把它加入到计算中
        board[i][j] = 'N';

        // 在当前单元格的四个方向开始搜索
        // 上: ( 0 , -1 )
        // 下: ( 0 , 1 )
        // 左: ( -1 , 0 )
        // 右: ( 1 , 0 )

        // dx 为行的方向数组
        int dx[4] = { -1 , 1 , 0 , 0 };

        // dy 为行的方向数组
        int dy[4] = { 0 , 0 , -1 , 1 };

        // 朝着这四个方向开始延伸搜索下去
        for (int index = 0; index < 4; index++) {

            // 下一个即将去搜索网格的横坐标
            int next_i = i + dx[index];

            // 下一个即将去搜索网格的纵坐标

```

```

        int next_j = j + dy[index];

        // 继续搜索
        dfs(board, next_i, next_j);

    }

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### https://leetcode.cn/problems/surrounded-regions/
class Solution:
    def solve(self, board: List[List[str]]) -> None:

        # 矩阵的行数
        m = len(board)

        # 矩阵的列数
        n = len(board[0])

        # 利用两个 for 循环，遍历矩阵中的一些特殊的单元格，将它们赋值为 N
        # 特殊的单元格：必然是从边界处开始的
        for i in range(0, m):

            for j in range(0, n):

                # i == 0: 表示在第 0 行，处于边界位置
                # j == 0: 表示在第 0 列，处于边界位置
                # i == (m - 1): 表示在最后一行，处于边界位置
                # j == (n - 1): 表示在最后一列，处于边界位置
                if i == 0 or j == 0 or i == (m - 1) or j == (n - 1):

                    # 从这些单元格开始向腹部延伸下去，找到所有和边界单元格想通的那些 o
                    self.dfs(board, i, j)

        # 第二个 for 循环，开始修改
        for i in range(0, m):

            for j in range(0, n):

                # board[i][j] == 'O' 表示无法从边界格子中延伸过来

```

```

        # 表示被 'x' 围绕的区域
        if board[i][j] == 'O' :

            # 修改为 x
            board[i][j] = 'X'

        # board[i][j] == 'N' 表示是从边界格子中延伸过来
        if board[i][j] == 'N' :

            # 修改为 O
            board[i][j] = 'O'

def dfs(self, board: List[List[str]], i : int , j : int ) -> None:
    # dfs搜索、递归终止条件
    # i < 0 , 越界
    # j < 0 , 越界
    # i >= board.length , 越界
    # j >= board[0].length , 越界
    # board[i][j] == x , 不需要继续搜索下去
    # board[i][j] == N , 这个格子已经被访问过, 不需要继续搜索下去
    if i < 0 or j < 0 or i >= len(board) or j >= len(board[0]) or board[i][j] ==
    'X' or board[i][j] == 'N' :

        return

    # 否则说明当前单元格是 O, 最后需要保留下来的那种
    # 将当前单元格置为 N , 避免其它格子 dfs 过程中又把它加入到计算中
    board[i][j] = 'N'

    # 在当前单元格的四个方向开始搜索
    # 上: ( 0 , -1 )
    # 下: ( 0 , 1 )
    # 左: ( -1 , 0 )
    # 右: ( 1 , 0 )
    # 朝着这四个方向开始延伸搜索下去
    for dx, dy in ((0,1), (1,0), (0,-1), (-1,0)):

        # 下一个即将去搜索网格的横坐标
        next_i = i + dx

        # 下一个即将去搜索网格的纵坐标
        next_j = j + dy

        # 继续搜索
        self.dfs(board,next_i,next_j)

```


四、复杂度分析

时间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。深度优先搜索过程中，每一个点至多只会被标记一次。

空间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。主要为深度优先搜索的栈的开销。

LeetCode 200、岛屿数量

一、题目描述

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

示例 1:

```
输入: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
输出: 1
```

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class Solution {
    public int numIslands(char[][] grid) {

        // count 代表岛屿数量
        int count = 0;

        // mark 标记已经搜索的位置，大小与 grid 一致
        int[][] mark = new int[grid.length][grid[0].length];

        // mark 初始化为 0
        for(int[] c : mark) {
            Arrays.fill(c, 0);
        }

        // 遍历 grid
```

```

        for(int i = 0 ; i < grid.length ; i++){
            for(int j = 0 ; j < grid[i].length;j++){

                // 如果发现位置 (i , j) 是没有标记的陆地
                if(mark[i][j] == 0 && grid[i][j] == '1'){
                    // 对该位置进行搜索
                    DFS(grid,i,j,mark);
                    // 对 (i , j) 完成搜索后, 岛屿数量加 1
                    count++;
                }
            }
        }

        return count;
    }

    // 将 grid 中与 x , y 相连的位置在 mark 中进行标记
    private void DFS(char[][] grid, int x, int y,int[][] mark) {

        // 当前搜索位置在 mark 中标记为 1 , 代表已经遍历了
        mark[x][y] = 1;

        // dx 为行的方向数组
        int dx[] = { -1 , 1 , 0 , 0 };

        // dy 为行的方向数组
        int dy[] = { 0 , 0 , -1 , 1 };

        // 循环遍历当前位置的上、下、左、右四个方向
        for( int i = 0 ; i < 4 ; i++){

            // newX 代表新的行
            int newX = dx[i] + x;

            // newY 代表新的列
            int newY = dy[i] + y;

            // 如果新的位置超出了数组边界
            if(newX < 0 || newX >= mark.length || newY < 0 || newY >=
mark[newX].length){
                // 跳出该位置
                continue;
            }

            // 如果发现新位置是陆地, 并且没有被标记过
            if(grid[newX][newY] == '1' && mark[newX][newY] == 0){
                // 递归搜索新的位置
                DFS(grid,newX,newY,mark);
            }
        }
    }
}

```

```

    }

    }

}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

岛屿数量 (LeetCode 200) :<https://leetcode-cn.com/problems/number-of-islands/submissions/>

class Solution:

def numIslands(self, grid: List[List[str]]) -> int:

count 代表岛屿数量

count = 0

mark 标记已经搜索的位置, 大小与 grid 一致

mark 初始化为 0

mark = [[0 for i in range(len(grid[0]))] for j in range(len(grid))]

遍历 grid

for i in range(len(grid)):

for j in range(len(grid[i])):

如果发现位置 (i , j) 是没有标记的陆地

if mark[i][j] == 0 and grid[i][j] == "1":

对该位置进行搜索

self.DFS(grid,i,j,mark)

对 (i , j) 完成搜索后, 岛屿数量加 1

count += 1

return count

将 grid 中与 x , y 相连的位置在 mark 中进行标记

def DFS(self,grid,x,y,mark: List[List[str]]) :

当前搜索位置在 mark 中标记为 1 , 代表已经遍历了

mark[x][y] = 1

```

# 循环遍历当前位置的上、下、左、右四个方向
for dx, dy in ((0,1), (1,0), (0,-1), (-1,0)):

    # newX 代表新的行
    newX = dx + x

    # newY 代表新的列
    newY = dy + y

    # 如果新的位置超出了数组边界
    if newX < 0 or newX >= len(mark) or newY < 0 or newY >= len(mark[newX]) :
        # 跳出该位置
        continue

    # 如果发现新位置是陆地, 并且没有被标记过
    if grid[newX][newY] == "1" and mark[newX][newY] == 0 :
        # 递归搜索新的位置
        self.DFS(grid,newX,newY,mark)

```

LeetCode 463、岛屿的周长

一、题目描述

给定一个 `row x col` 的二维网格地图 `grid`，其中：`grid[i][j] = 1` 表示陆地，`grid[i][j] = 0` 表示水域。

网格中的格子 **水平和垂直** 方向相连（对角线方向不相连）。整个网格被水完全包围，但其中恰好有一个岛屿（或者说，一个或多个表示陆地的格子相连组成的岛屿）。

岛屿中没有“湖”（“湖”指水域在岛屿内部且不和岛屿周围的水相连）。格子是边长为 1 的正方形。网格为长方形，且宽度和高度均不超过 100。计算这个岛屿的周长。示例 1：

输入: `grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`
 输出: 16
 解释: 它的周长是上面图片中的 16 个黄色的边

示例 2:

输入: `grid = [[1]]`
 输出: 4

示例 3:

输入: `grid = [[1,0]]`
 输出: 4

提示:

- `row == grid.length`
- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` 为 0 或 1

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// https://leetcode.cn/problems/island-perimeter/
class Solution {
    public int islandPerimeter(int[][] grid) {

        // 矩阵的行数
        int m = grid.length;

        // 矩阵的列数
        int n = grid[0].length;

        // 从左到右，一行行开始搜索
        for (int i = 0; i < m; i++) {

            // 从上到下，一列列开始搜索
            for (int j = 0; j < n; j++) {

                // 如果当前格子是陆地，才有资格继续延伸搜索下去
                if(grid[i][j] == 1){
                    // 题目说明，恰好有一个岛屿
                    return dfs(grid,i,j);
                }
            }
        }

        return 0;
    }

    public int dfs(int[][] grid, int i, int j) {
        // dfs搜索、递归终止条件
        // i < 0 , 越界
        // j < 0 , 越界
        // i >= board.length , 越界
```

```

// j >= board[0].length , 越界
// 从边界格子向外面延伸, 会提供一条边的周长
if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length ) {

    return 1;

}

// 如果当前格子是水域, 会提供一条边的周长
if(grid[i][j] == 0){

    return 1;

}

// 如果当前格子已经遍历过的陆地, 无法提供边
if(grid[i][j] == 2){

    return 0;

}

// 否则说明当前位置实际上是陆地, 为 1
// 需要把它修改为 2, 代表是已经访问过陆地
grid[i][j] = 2;

// 在当前单元格的四个方向开始搜索
// 上: ( 0 , -1 )
// 下: ( 0 , 1 )
// 左: ( -1 , 0 )
// 右: ( 1 , 0 )

// dx 为行的方向数组
int dx[] = { -1 , 1 , 0 , 0 };

// dy 为行的方向数组
int dy[] = { 0 , 0 , -1 , 1 };

// 以当前这个陆地格子 grid[i][j] 的四周搜索下去, 计算周长
int res = 0;

// 朝着这四个方向开始延伸搜索下去
for (int index = 0; index < 4; ++index) {

    // 下一个即将去搜索网格的横坐标
    int next_i = i + dx[index];

    // 下一个即将去搜索网格的纵坐标
    int next_j = j + dy[index];

    // 获取从 ( next_i , next_j ) 这个网格开始出发延伸之后的周长

```

```

        int circle = dfs(grid, next_i,next_j);

        // 把这个新增的周长累加到 ( i , j ) 这个网格上
        res += circle;

    }

    // 返回从 grid[i][j] 这个格子开始延伸得到的周长
    return res;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {

        // 矩阵的行数
        int m = grid.size();

        // 矩阵的列数
        int n = grid[0].size();

        // 从左到右，一行行开始搜索
        for (int i = 0; i < m; i++) {

            // 从上到下，一列列开始搜索
            for (int j = 0; j < n; j++) {

                // 如果当前格子是陆地，才有资格继续延伸搜索下去
                if(grid[i][j] == 1){
                    // 题目说明，恰好有一个岛屿
                    return dfs(grid,i,j);
                }

            }

        }

        return 0;

    }
public:
    int dfs(vector<vector<int>>& grid, int i, int j) {

        // dfs搜索、递归终止条件
        // i < 0 , 越界
        // j < 0 , 越界
    }
}

```

```

// i >= board.length , 越界
// j >= board[0].length , 越界
// board[i][j] == x , 不需要继续搜索下去
// board[i][j] == N , 这个格子已经被访问过, 不需要继续搜索下去
if (i < 0 || j < 0 || i >= grid.size() || j >= grid[0].size() ) {

    return 1 ;

}

// 如果当前格子是水域, 会提供一条边的周长
if(grid[i][j] == 0){

    return 1;

}

// 如果当前格子已经遍历过的陆地, 无法提供边
if(grid[i][j] == 2){

    return 0;

}

// 否则说明当前位置实际上是陆地, 为 1
// 需要把它修改为 2, 代表是已经访问过陆地
grid[i][j] = 2;

// 在当前单元格的四个方向开始搜索
// 上: ( 0 , -1 )
// 下: ( 0 , 1 )
// 左: ( -1 , 0 )
// 右: ( 1 , 0 )

// dx 为行的方向数组
int dx[4] = { -1 , 1 , 0 , 0 };

// dy 为行的方向数组
int dy[4] = { 0 , 0 , -1 , 1 };

// 以当前这个陆地格子 grid[i][j] 的四周搜索下去, 计算周长
int res = 0;

// 朝着这四个方向开始延伸搜索下去
for (int index = 0; index < 4; index++) {

    // 下一个即将去搜索网格的横坐标
    int next_i = i + dx[index];

    // 下一个即将去搜索网格的纵坐标
    int next_j = j + dy[index];

```



```

        // 获取从 ( next_i , next_j ) 这个网格开始出发延伸之后的周长
        int circle = dfs(grid, next_i,next_j);

        // 把这个新增的周长累加到 ( i , j ) 这个网格上
        res += circle;

    }

    return res;

}

};

```

3、Python 代码

```

class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        # 矩阵的行数
        m = len(grid)

        # 矩阵的列数
        n = len(grid[0])

        # 利用两个 for 循环，遍历矩阵中的一些特殊的单元格，将它们赋值为 N
        # 特殊的单元格：必然是从边界处开始的
        for i in range(0 , m ) :

            for j in range( 0 , n ):

                if(grid[i][j] == 1):
                    return self.dfs(grid, i, j)

        return 0

    def dfs(self, grid: List[List[str]], i : int , j : int ) -> int:
        # dfs搜索、递归终止条件
        # i < 0 , 越界
        # j < 0 , 越界
        # i >= board.length , 越界
        # j >= board[0].length , 越界
        # board[i][j] == x , 不需要继续搜索下去
        # board[i][j] == N , 这个格子已经被访问过，不需要继续搜索下去
        if i < 0 or j < 0 or i >= len(grid) or j >= len(grid[0]) :
            return 1

        if grid[i][j] == 0 : return 1

```

```

if grid[i][j] == 2 : return 0

# 否则说明当前位置实际上是陆地，为 1
# 需要把它修改为 2，代表是已经访问过陆地
grid[i][j] = 2

res = 0
# 在当前单元格的四个方向开始搜索
# 上： ( 0 , -1 )
# 下： ( 0 , 1 )
# 左： ( -1 , 0 )
# 右： ( 1 , 0 )
# 朝着这四个方向开始延伸搜索下去
for dx, dy in ((0,1), (1,0), (0,-1), (-1,0)):

    # 下一个即将去搜索网格的横坐标
    next_i = i + dx

    # 下一个即将去搜索网格的纵坐标
    next_j = j + dy

    circle = self.dfs(grid,next_i,next_j)

    res += circle

return res

```

四、复杂度分析

时间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。深度优先搜索过程中，每一个点至多只会被标记一次。

空间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。主要为深度优先搜索的栈的开销。

LeetCode 695、岛屿的最大面积

```

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        # 矩阵的行数
        m = len(grid)

        # 矩阵的列数
        n = len(grid[0])

        maxx = 0

        # 利用两个 for 循环，遍历矩阵中的一些特殊的单元格，将它们赋值为 N

```

```

# 特殊的单元格：必然是从边界处开始的
for i in range(0, m):

    for j in range(0, n):

        if(grid[i][j] == 1):

            area = self.dfs(grid,i,j)

            maxx = max(maxx,area)

    return maxx

def dfs(self, grid: List[List[str]], i: int, j: int) -> int:
    # dfs搜索、递归终止条件
    # i < 0, 越界
    # j < 0, 越界
    # i >= board.length, 越界
    # j >= board[0].length, 越界
    # board[i][j] == x, 不需要继续搜索下去
    # board[i][j] == N, 这个格子已经被访问过, 不需要继续搜索下去
    if i < 0 or j < 0 or i >= len(grid) or j >= len(grid[0]) or grid[i][j] == 0:
        return 0

    # 将当前单元格置为 0, 避免其它格子 dfs 过程中又把它加入到计算中
    grid[i][j] = 0

    res = 1
    # 在当前单元格的四个方向开始搜索
    # 上: (0, -1)
    # 下: (0, 1)
    # 左: (-1, 0)
    # 右: (1, 0)
    # 朝着这四个方向开始延伸搜索下去
    for dx, dy in ((0,1), (1,0), (0,-1), (-1,0)):

        # 下一个即将去搜索网格的横坐标
        next_i = i + dx

        # 下一个即将去搜索网格的纵坐标
        next_j = j + dy

        area = self.dfs(grid,next_i,next_j)

        res += area

    return res

```

四、复杂度分析

时间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。深度优先搜索过程中，每一个点至多只会被标记一次。

空间复杂度： $O(n \times m)$ ，其中 n 和 m 分别为矩阵的行数和列数。主要为深度优先搜索的栈的开销。

第三十六天

LeetCode 102、二叉树的层序遍历

一、题目描述

给你一个二叉树，请你返回其按 **层序遍历** 得到的节点值。（即逐层地，从左到右访问所有节点）。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的层序遍历（ LeetCode 102 ） :https://leetcode-cn.com/problems/binary-tree-level-order-traversal
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        // 设置 res 用来保存输出结果
        List<List<Integer>> res = new LinkedList<>();
        // 边界情况处理
        if(root == null) return res;

        // 设置一个队列，用来存储二叉树中的元素
        Queue<TreeNode> queue = new LinkedList<>();
        // 队列添加二叉树的根节点
        queue.add(root);

        // 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
        while(!queue.isEmpty()){
            // 用来记录 queue 的长度，即每层节点的个数
            int size = queue.size();

            // 用来保存每一层节点，保存成功后添加到 res 中
            List<Integer> temp = new ArrayList<>();
```

```

        // 使用 for 循环, 将 queue 中的元素添加的 temp 中
        for(int i = 0 ; i < size ; i++ ){
            // 从 queue 中取出一个节点
            TreeNode node = queue.poll();
            // 把节点存放到 list 中
            temp.add(node.val); //将节点值加入list

            // 判断当前节点的左子节点是否有值, 如果有, 则添加到 queue 中
            if(node.left != null)
                queue.add(node.left);

            // 判断当前节点的右子节点是否有值, 如果有, 则添加到 queue 中
            if(node.right != null)
                queue.add(node.right);
        }

        // 把存放了每一层元素的数组 temp 添加到 res 中
        res.add(temp);
    }

    // 返回 res
    return res;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的层序遍历 ( LeetCode 102 ) :https://leetcode-cn.com/problems/binary-tree-level-order-traversal
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        // 设置 res 用来保存输出结果
        vector<vector<int>> res;
        // 边界情况处理
        if(root == NULL) return res;

        // 设置一个队列, 用来存储二叉树中的元素
        queue<TreeNode*> myqueue;
        // 队列添加二叉树的根节点
        myqueue.push(root);

        // 遍历队列, 直到队列为空, 说明访问了二叉树中所有的节点
        while(!myqueue.empty()){

```

```

        // 用来记录 myqueue 的长度，即每层节点的个数
        int size = myqueue.size();

        // 用来保存每一层节点，保存成功后添加到 res 中
        vector<int> temp;

        // 使用 for 循环，将 myqueue 中的元素添加到 temp 中
        for(int i = 0 ; i < size ; i++){
            // 从 myqueue 中取出一个节点
            TreeNode* node = myqueue.front();
myqueue.pop();
            // 把节点存放到 list 中
            temp.push_back(node->val); //将节点值加入list

            // 判断当前节点的左子节点是否有值，如果有，则添加到 myqueue 中
            if(node->left != NULL)
                myqueue.push(node->left);

            // 判断当前节点的右子节点是否有值，如果有，则添加到 myqueue 中
            if(node->right != NULL)
                myqueue.push(node->right);
        }

        // 把存放了每一层元素的数组 temp 添加到 res 中
        res.push_back(temp);
    }

    // 返回 res
    return res;
}
};

```

3、Python 代码

```

#登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二叉树的层序遍历（ LeetCode 102 ）：https://leetcode-cn.com/problems/binary-tree-level-order-traversal
class solution:
    def levelOrder(self, root: TreeNode) -> List[List[int]]:

        # 设置 res 用来保存输出结果
        res = []

        # 边界情况处理
        if not root :
            return res

```

```
# 设置一个队列，用来存储二叉树中的元素
queue = deque()

# 队列添加二叉树的根节点
queue.append(root)

# 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
while queue :
    # 用来记录 queue 的长度，即每层节点的个数
    size = len(queue)

    # 用来保存每一层节点，保存成功后添加到 res 中
    temp = []

    # 使用 for 循环，将 queue 中的元素添加到 temp 中
    for _ in range(size) :
        # 从 queue 中取出一个节点
        node = queue.popleft()
        # 把节点存放到 list 中
        temp.append(node.val)

        # 判断当前节点的左子节点是否有值，如果有，则添加到 queue 中
        if node.left : queue.append(node.left)

        # 判断当前节点的右子节点是否有值，如果有，则添加到 queue 中
        if node.right : queue.append(node.right)

    # 把存放了每一层元素的数组 temp 添加到 res 中
    res.append(temp)

# 返回 res
return res
```

LeetCode 103、二叉树的锯齿形层序遍历

一、题目描述

给定一个二叉树，返回其节点值的锯齿形层序遍历。（即先从左往右，再从右往左进行下一层遍历，以此类推，层与层之间交替进行）。

例如：给定二叉树 `[3,9,20,null,null,15,7]`，

```
    3
   /\
  9 20
 /\ 
15  7
```

返回锯齿形层序遍历如下：

```
[
  [3],
  [20,9],
  [15,7]
]
```

二、题目解析

该问题需要用到队列，与之前的二叉树的层次遍历类似，不同点在于在偶数层需要翻转一下。

- 建立一个 queue
- 先把根节点放进去，这时候找根节点的左右两个子节点
- 去掉根节点，此时 queue 里的元素就是下一层的所有节点
- 循环遍历，将结果存到一个一维向量里
- 遍历完之后再把这个一维向量存到二维向量里
- 如果该层为偶数层，则 reverse 翻转一下
- 以此类推，可以完成层序遍历

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的锯齿形层序遍历 ( LeetCode 103 ) :https://leetcode-cn.com/problems/binary-tree-zigzag-level-order-traversal/
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {

        // 设置 res 用来保存输出结果
        List<List<Integer>> res = new LinkedList<List<Integer>>();

        // 边界情况处理
        if(root == null) return res;

        // 设置一个队列，用来存储二叉树中的元素
        Queue<TreeNode> nodeQueue = new LinkedList<TreeNode>();

        // 队列添加二叉树的根节点
```



```

nodeQueue.add(root);

// 是否从左至右标志位
boolean isOrderLeft = true;

// 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
while (!nodeQueue.isEmpty()) {

    // 用来记录 queue 的长度，即每层节点的个数
    int size = nodeQueue.size();

    // 用来保存每一层节点，保存成功后添加到 res 中
    // 由于需要既可以在队头添加元素、有需要可以在队尾添加元素，因此使用双端队列
    // 用双端队列是方便调换方向存储不同层的元素
    Deque<Integer> levelList = new LinkedList<Integer>();

    // 使用 for 循环，将 nodeQueue 中的元素添加的 levelList 中
    // 按标志位存入双端队列，再按固定顺序读取下一层节点值进队列
    for (int i = 0; i < size; ++i) {

        // 从 queue 中取出一个节点，此时，nodeQueue 已经抛出了这个节点
        TreeNode curNode = nodeQueue.poll();

        // 先从左往右存储，每次存储在 levelList 的队尾，即 addLast
        if (isOrderLeft) {
            levelList.addLast(curNode.val);

            // 再从右往左存储，每次存储在 levelList 的队头，即 addFirst
        } else {
            levelList.addFirst(curNode.val);
        }

        // 将下一层节点值进入队列，如果有，按照从左到右的顺序
        // 判断当前节点的左子节点是否有值，如果有，则添加到 nodeQueue 中
        if (curNode.left != null) {
            nodeQueue.add(curNode.left);
        }

        // 判断当前节点的右子节点是否有值，如果有，则添加到 nodeQueue 中
        if (curNode.right != null) {
            nodeQueue.add(curNode.right);
        }
    }

    // 把存放了每一层元素的数组 levelList 添加到 res 中，注意格式转换，需要转换为数组的形式
    res.add(new LinkedList<Integer>(levelList));

    // 【从左到右】和【从右到左】的顺序来回交替
    isOrderLeft = !isOrderLeft;
}

```

```

        isOrderLeft = !isOrderLeft;
    }
    // 返回 res
    return res;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的锯齿形层序遍历 ( LeetCode 103 ) :https://leetcode-cn.com/problems/binary-tree-
zigzag-level-order-traversal/
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {

        // 设置 res 用来保存输出结果
        vector<vector<int>> res;

        // 边界情况处理
        if(root == NULL) return res;

        // 设置一个队列，用来存储二叉树中的元素
        queue<TreeNode*> nodeQueue;

        // 队列添加二叉树的根节点
        nodeQueue.push(root);

        // 是否从左至右标志位
        bool isOrderLeft = true;

        // 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
        while (!nodeQueue.empty()) {

            // 用来记录 queue 的长度，即每层节点的个数
            int size = nodeQueue.size();

            // 用来保存每一层节点，保存成功后添加到 res 中
            // 由于需要既可以在队头添加元素、有需要可以在队尾添加元素，因此使用双端队列
            // 用双端队列是方便调换方向存储不同层的元素
            deque<int> levelList;

            // 使用 for 循环，将 nodeQueue 中的元素添加的 levelList 中
            // 按标志位存入双端队列，再按固定顺序读取下一层节点值进队列
            for (int i = 0; i < size; ++i) {

```

```

        // 从 queue 中取出一个节点, 此时, nodeQueue 已经抛出了这个节点
        TreeNode* curNode = nodeQueue.front();
nodeQueue.pop();

        // 先左往右存储, 每次存储在 levelList 的队尾, 即 addLast
        if (isOrderLeft) {
            levelList.push_back(curNode->val);

            // 再从右往左存储, 每次存储在 levelList 的队头, 即 addFirst
        } else {
            levelList.push_front(curNode->val);
        }

        // 将下一层节点值进入队列, 如果有, 按照从左到右的顺序
        // 判断当前节点的左子节点是否有值, 如果有, 则添加到 nodeQueue 中
        if (curNode->left != NULL) {
            nodeQueue.push(curNode->left);
        }

        // 判断当前节点的右子节点是否有值, 如果有, 则添加到 nodeQueue 中
        if (curNode->right != NULL) {
            nodeQueue.push(curNode->right);
        }
    }
    vector<int>v;
    while(!levelList.empty()){
        int element = levelList.front();
        levelList.pop_front();
        v.push_back(element);
    }

    // 把存放了每一层元素的数组 levelList 添加到 res 中, 注意格式转换, 需要转换为数组的形
    式
    res.push_back(v);

    // 【从左到右】和【从右到左】的顺序来回交替
    isOrderLeft = !isOrderLeft;
}
// 返回 res
return res;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者: 程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

```
#### 二叉树的锯齿形层序遍历 ( LeetCode 103 ) :https://leetcode-cn.com/problems/binary-tree-zigzag-level-order-traversal/
class Solution:
    def zigzagLevelOrder(self, root: TreeNode) -> List[List[int]]:

        # 设置 res 用来保存输出结果
        res = [ ]

        # 边界情况处理
        if not root :
            return res

        # 设置一个队列，用来存储二叉树中的元素
        nodeQueue = deque()

        # 队列添加二叉树的根节点
        nodeQueue.append(root)

        # 是否从左至右标志位
        isOrderLeft = True

        # 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
        while nodeQueue :

            # 用来记录 queue 的长度，即每层节点的个数
            size = len(nodeQueue)

            # 用来保存每一层节点，保存成功后添加到 res 中
            # 由于需要既可以在队头添加元素、有需要可以在队尾添加元素，因此使用双端队列
            # 用双端队列是方便调换方向存储不同层的元素
            levelList = collections.deque([ ])

            # 使用 for 循环，将 nodeQueue 中的元素添加的 levelList 中
            # 按标志位存入双端队列，再按固定顺序读取下一层节点值进队列
            for _ in range(size) :

                # 从 queue 中取出一个节点，此时，nodeQueue 已经抛出了这个节点
                curNode = nodeQueue.popleft()

                # 先从左往右存储，每次存储在 levelList 的队尾，即 appendLast
                if isOrderLeft :
                    levelList.append(curNode.val)

                # 再从右往左存储，每次存储在 levelList 的队头，即 appendFirst
                else :
                    levelList.appendleft(curNode.val)

            # 将下一层节点值进入队列，如果有，按照从左到右的顺序
```

```

        # 判断当前节点的左子节点是否有值, 如果有, 则添加到 nodeQueue 中
        if curNode.left :
            nodeQueue.append(curNode.left)

        # 判断当前节点的右子节点是否有值, 如果有, 则添加到 nodeQueue 中
        if curNode.right :
            nodeQueue.append(curNode.right)

    # 把存放了每一层元素的数组 levelList 添加到 res 中, 注意格式转换, 需要转换为数组的形式
    res.append(list(levelList))

    # 【从左到右】和【从右到左】的顺序来回交替
    isOrderLeft = not isOrderLeft

# 返回 res
return res

```

LeetCode 105、从前序与中序遍历序列构造二叉树

一、题目描述

给定一棵树的前序遍历 `preorder` 与中序遍历 `inorder`。请构造二叉树并返回其根节点。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 从前序与中序遍历序列构造二叉树 ( LeetCode 105 ) :https://leetcode-cn.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal
class Solution {
    public TreeNode buildTree(int[] preorder, int[] inorder) {

        // 题目中说 preorder 和 inorder 均无重复元素
        // 通过哈希表把中序遍历序列中的值和顺序建立映射关系
        HashMap<Integer, Integer> map = new HashMap<>();

        // 通过 for 循环, 遍历完中序遍历序列中的所有元素
        for (int i = 0; i < inorder.length; i++) {
            // 以中序序列中的元素值 inorder[i] 作为 key
            // 以位置 i 作为 value
            // 存放到哈希表中

```

```

        // 比如中序遍历序列中的元素是 [ A , D , E , F , G , H , M
,    Z ]

        // 那么这个哈希表就是以下的样子
        // | 索引 | 位置 |
        // | A | 0 |
        // | D | 1 |
        // | E | 2 |
        // | F | 3 |
        // | G | 4 |
        // | H | 5 |
        // | M | 6 |
        // | Z | 7 |
        map.put(inorder[i], i);
    }

    // 下面开始构建二叉树
    // 把前序遍历序列中的第一个元素 preorder[0] 作为二叉树的根节点
    // 因为任意二叉树的前序遍历序列中的第一个元素，一定是二叉树的根节点
    TreeNode root = new TreeNode(preorder[0]);

    // 继续遍历前序遍历序列中的其它元素
    for(int i = 1 ; i < preorder.length ; i++){

        // 把当前遍历的元素构造为一个二叉树的节点
        TreeNode node = new TreeNode(preorder[i]);

        // 把构造的节点插入到以 root 为根节点的二叉树中
        insertNode(root,node,map);

    }

    // 当 preorder 中所有元素都构造并且插入完毕之后
    // 二叉树就完成了构建
    return root;

}

// root : 二叉树的根节点
// node : 待插入的节点
// map : 节点值和中序遍历序列位置的映射
private void insertNode(TreeNode root,TreeNode node,HashMap<Integer,Integer> map){

    // 当 root 和 node 指向的节点相同时，跳出循环
    while(root != node){

        // 如果 node 的中序遍历序列位置小于 root 的中序遍历序列位置
        // 说明 node 应该在 root 的左子树中

```

```

if(map.get(node.val) < map.get(root.val)){

    // 如果此时 root 没有左子树
    if(root.left == null){
        // 那么就把 node 设置为 root 的左子树
        root.left = node;
    }

    // 1、如果之前 root 没有左子树，那么通过上面的代码，就设置好了 root 的左子树
    // 也就是说 node 是 root 的一个叶子节点，完成了插入操作
    // 把 root 指向 root.left 后，root 为 node，跳出了循环

    // 2、如果之前 root 已经有左子树，那么就不能直接把 node 插入到 root 的左子树

    // 需要判断应该把 node 插入到 root 左子树上的孩子节点的那个位置上
    // 比如现在的 root 是这个样子，node 为 A
    //      G
    //     /
    //    D
    //   / \
    //  ①  ②
    // root 已经有左子树 D 了，所以 node 应该考虑插入到 D 中的 ① 还是 ② 上面
    // 所以，把 root 指向 root.left，继续遍历 root 的左子树的情况
    root = root.left;

}

}else{

    // 如果 node 的中序遍历序列位置大于 root 的中序遍历序列位置
    // 说明 node 应该在 root 的右子树中

    // 如果此时 root 没有右子树
    if(root.right == null){
        // 那么就把 node 设置为 root 的右子树
        root.right = node;
    }

    // 把 root 指向 root.right，继续遍历 root 的右子树的情况
    root = root.right;

}

}

}

```

上

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 从前序与中序遍历序列构造二叉树 ( LeetCode 105 ) :https://leetcode-cn.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal
class Solution {

    // 题目中说 preorder 和 inorder 均无重复元素
    // 通过哈希表把中序遍历序列中的值和顺序建立映射关系
    unordered_map<int, int> map;

public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {

        // 通过 for 循环，遍历完中序遍历序列中的所有元素
        for (int i = 0; i < inorder.size(); i++) {
            // 以中序序列中的元素值 inorder[i] 作为 key
            // 以位置 i 作为 value
            // 存放到哈希表中
            // 比如中序遍历序列中的元素是 [ A , D , E , F , G , H , M , Z ]

            // 那么这个哈希表就是以下的样子
            // | 索引 | 位置 |
            // | A | 0 |
            // | D | 1 |
            // | E | 2 |
            // | F | 3 |
            // | G | 4 |
            // | H | 5 |
            // | M | 6 |
            // | Z | 7 |
            map[inorder[i]] = i;
        }

        // 下面开始构建二叉树
        // 把前序遍历序列中的第一个元素 preorder[0] 作为二叉树的根节点
        // 因为任意二叉树的前序遍历序列中的第一个元素，一定是二叉树的根节点
        TreeNode *root = new TreeNode(preorder[0]);

        // 继续遍历前序遍历序列中的其它元素
        for(int i = 1 ; i < preorder.size() ; i++){

            // 把当前遍历的元素构造为一个二叉树的节点
            TreeNode *node = new TreeNode(preorder[i]);

            // 把构造的节点插入到以 root 为根节点的二叉树中
```



```

        insertNode(root,node);

    }

    // 当 preorder 中所有元素都构造并且插入完毕之后
    // 二叉树就完成了构建
    return root;
}

// root : 二叉树的根节点
// node : 待插入的节点
private:
    void insertNode(TreeNode *root,TreeNode *node){

        // 当 root 和 node 指向的节点相同时，跳出循环
        while(root != node){

            // 如果 node 的中序遍历序列位置小于 root 的中序遍历序列位置
            // 说明 node 应该在 root 的左子树中
            if(map[node->val] < map[root->val]){

                // 如果此时 root 没有左子树
                if(root->left == NULL){
                    // 那么就把 node 设置为 root 的左子树
                    root->left = node;
                }

                // 1、如果之前 root 没有左子树，那么通过上面的代码，就设置好了 root 的左子树
                // 也就是说 node 是 root 的一个叶子节点，完成了插入操作
                // 把 root 指向 root.left 后，root 为 node，跳出了循环

                // 2、如果之前 root 已经有左子树，那么就不能直接把 node 插入到 root 的左子树

                // 需要判断应该把 node 插入到 root 左子树上的孩子节点的那个位置上
                // 比如现在的 root 是这个样子，node 为 A
                //          G
                //          /
                //          D
                //         / \
                //        ① ②
                // root 已经有左子树 D 了，所以 node 应该考虑插入到 D 中的 ① 还是 ② 上面
                // 所以，把 root 指向 root.left ，继续遍历 root 的左子树的情况
                root = root->left;

            }else{

                // 如果 node 的中序遍历序列位置大于 root 的中序遍历序列位置
                // 说明 node 应该在 root 的右子树中

```

```

        // 如果此时 root 没有右子树
        if(root->right == NULL){
            // 那么就把 node 设置为 root 的右子树
            root->right = node;
        }

        // 把 root 指向 root.right , 继续遍历 root 的右子树的情况
        root = root->right;

    }

}

};

```

3、Python 代码

```

#登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 从前序与中序遍历序列构造二叉树 ( LeetCode 105 ) :https://leetcode-
cn.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal
class Solution:

    def buildTree(self, preorder: List[int], inorder: List[int]) -> TreeNode:

        # 题目中说 preorder 和 inorder 均无重复元素
        # 通过哈希表把中序遍历序列中的值和顺序建立映射关系
        # 通过 for 循环, 遍历完中序遍历序列中的所有元素
        map = dict()

        for i in range(len(inorder)) :
            # 以中序序列中的元素值 inorder[i] 作为 key
            # 以位置 i 作为 value
            # 存放到哈希表中
            # 比如中序遍历序列中的元素是 [ A , D , E , F , G , H , M
, Z ]

            # 那么这个哈希表就是以下的样子
            # | 索引 | 位置 |
            # | A | 0 |
            # | D | 1 |
            # | E | 2 |
            # | F | 3 |
            # | G | 4 |
            # | H | 5 |
            # | M | 6 |
            # | Z | 7 |

```

```
map[inorder[i]] = i

# 下面开始构建二叉树
# 把前序遍历序列中的第一个元素 preorder[0] 作为二叉树的根节点
# 因为任意二叉树的前序遍历序列中的第一个元素，一定是二叉树的根节点
root = TreeNode(preorder[0])

# 继续遍历前序遍历序列中的其它元素
for i in range(1, len(preorder)):

    # 把当前遍历的元素构造为一个二叉树的节点
    node = TreeNode(preorder[i])

    # 把构造的节点插入到以 root 为根节点的二叉树中
    self.insertNode(root, node, map)

# 当 preorder 中所有元素都构造并且插入完毕之后
# 二叉树就完成了构建
return root

# root : 二叉树的根节点
# node : 待插入的节点
def insertNode(self, root : TreeNode, node : TreeNode, map:dict) :
    # 当 root 和 node 指向的节点相同时，跳出循环
    while root != node :

        # 如果 node 的中序遍历序列位置小于 root 的中序遍历序列位置
        # 说明 node 应该在 root 的左子树中
        if map[node.val] < map[root.val] :

            # 如果此时 root 没有左子树
            if not root.left :
                # 那么就把 node 设置为 root 的左子树
                root.left = node

        # 1、如果之前 root 没有左子树，那么通过上面的代码，就设置好了 root 的左子树
        # 也就是说 node 是 root 的一个叶子节点，完成了插入操作
        # 把 root 指向 root.left 后，root 为 node，跳出了循环

        # 2、如果之前 root 已经有左子树，那么就不能直接把 node 插入到 root 的左子树

        # 需要判断应该把 node 插入到 root 左子树上的孩子节点的那个位置上
        # 比如现在的 root 是这个样子，node 为 A
        #          G
        #         / \
        #        B   C
        #       / \
        #      D   E
```

```

#         /
#         D
#        / \
#       ①  ②
# root 已经有左子树 D 了，所以 node 应该考虑插入到 D 中的 ① 还是 ② 上面
# 所以，把 root 指向 root.left ，继续遍历 root 的左子树的情况
root = root.left

```

```
else:
```

```

# 如果 node 的中序遍历序列位置大于 root 的中序遍历序列位置
# 说明 node 应该在 root 的右子树中

```

```

# 如果此时 root 没有右子树
if not root.right :
    # 那么就把 node 设置为 root 的右子树
    root.right = node

```

```

# 把 root 指向 root.right ，继续遍历 root 的右子树的情况
root = root.right

```

LeetCode 144、二叉树的前序遍历

一、题目描述

给你二叉树的根节点 `root` ，返回它节点值的 前序、中序、后序 遍历。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的前序遍历（ LeetCode 144）：https://leetcode-cn.com/problems/binary-tree-preorder-traversal/
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {

        // 设置一个数组用来保存二叉树前序遍历的结果
        List<Integer> preorderResult = new ArrayList<>();

        // 设置一个数组用来保存二叉树中序遍历的结果
        List<Integer> inorderResult = new ArrayList<>();
    }
}

```

```

// 设置一个数组用来保存二叉树后序遍历的结果
List<Integer> postorderResult = new ArrayList<>();

// 设置一个栈，用来保存路径
Stack<TreeNode> stack = new Stack<>();

// 设置一个节点，一开始指向根节点
TreeNode node = root;

// 设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了
// 每个节点都有 左、右、上 这三种状态
// 按照 左 --> 右 --> 上 的顺序观察每个节点

// 左代表该节点的左右孩子节点都没有遍历
int nodeLeft = 100;

// 右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历
int nodeRight = 200;

// 上代表左右孩子节点都已经遍历，需要返回到它的父节点
int nodeUp = 300;

// 每个节点的初始化状态都是从 左 开始
int nodeState = nodeLeft;

// 对二叉树进行遍历
while(node != null){

    // 位置 ①

    // 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if(nodeState == nodeLeft){
        // 把当前节点加入到二叉树前序遍历的结果数组中
        preorderResult.add(node.val);

        // 如果当前节点有左子树
        if(node.left != null){

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.push(node);

            // 开始观察当前节点的左孩子节点，代码来到了位置 ①
            node = node.left;
        }else{

            // 如果当前节点没有左子树，切换当前节点的状态为【右】
            // 代码来到了位置 ②

```

```

        nodeState = nodeRight;
    }

    }else if(nodeState == nodeRight){ // 如果当前节点的状态是【右】，说明该节点的左孩子
节点已经遍历，右孩子节点还没有遍历

        // 把当前节点加入到二叉树中序遍历的结果数组中
        // inorderResult.add(node.val);

        // 如果当前节点有右子树
        if(node.right != null){

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.push(node);

            // 开始观察当前节点的右孩子节点
            node = node.right;

            // 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
            nodeState = nodeLeft;

            // 执行完上面三行代码，代码来到了位置 ①

        }else{
            // 如果当前节点没有右子树，切换当前节点的状态为【上】
            // 代码来到了位置 ②
            nodeState = nodeUp;
        }
    }else if(nodeState == nodeUp){ // 如果当前节点的状态是【上】，说明左右孩子节点都已
经遍历，需要返回到它的父节点
        // 把当前节点加入到二叉树后序遍历的结果数组中
        // postorderResult.add(node.val);

        // 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
        // 首先构建一个空的节点
        TreeNode parent = null;

        // 如果栈中有元素
        if(!stack.isEmpty()){

            // 那么，栈顶元素就是当前节点的父节点
            parent = stack.pop();

            // 判断一下父节点的左节点是否为当前节点
            // 比如这颗二叉树
            //           1
            //        /  \
            //       2    3
            //      /\    \

```

```

        //      4    5    6
        // 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换
状态为【右】

        // 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切
换状态

        // 如果父节点的左节点为当前节点
        if(parent.left == node){
            // 切换当前节点的状态为【右】
            nodeState = nodeRight;
        }
    }
    // 开始观察当前节点的父节点
    // 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
    // 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
    node = parent;

}
}

// 根据题目要求，返回二叉树前序、中序、后序遍历的结果
return preorderResult;

}
}

```

2、C++ 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的前序遍历（LeetCode 144）：https://leetcode-cn.com/problems/binary-tree-
preorder-traversal/
class Solution {
public:
    vector<int> preorderTraversal(TreeNode* root) {

        // 设置一个数组用来保存二叉树前序遍历的结果
        vector<int> preorderResult;

        // 设置一个数组用来保存二叉树中序遍历的结果
        vector<int> inorderResult;

        // 设置一个数组用来保存二叉树后序遍历的结果
        vector<int> postorderResult;
    }
}

```

```

// 设置一个栈，用来保存路径
stack<TreeNode*> st;

// 设置一个节点，一开始指向根节点
TreeNode* node = root;

// 设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了
// 每个节点都有 左、右、上 这三种状态
// 按照 左 --> 右 --> 上 的顺序观察每个节点

// 左代表该节点的左右孩子节点都没有遍历
int nodeLeft = 100;

// 右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历
int nodeRight = 200;

// 上代表左右孩子节点都已经遍历，需要返回到它的父节点
int nodeUp = 300;

// 每个节点的初始化状态都是从 左 开始
int nodeState = nodeLeft;

// 对二叉树进行遍历
while(node != NULL){

    // 位置 ①

    // 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if(nodeState == nodeLeft){
        // 把当前节点加入到二叉树前序遍历的结果数组中
        preorderResult.push_back(node->val);

        // 如果当前节点有左子树
        if(node->left != NULL){

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            st.push(node);

            // 开始观察当前节点的左孩子节点，代码来到了位置 ①
            node = node->left;
        }else{

            // 如果当前节点没有左子树，切换当前节点的状态为【右】
            // 代码来到了位置 ②
            nodeState = nodeRight;
        }
    }
}

```



```

}else if(nodeState == nodeRight){ // 如果当前节点的状态是【右】，说明该节点的左孩子
节点已经遍历，右孩子节点还没有遍历

    // 把当前节点加入到二叉树中序遍历的结果数组中
    // inorderResult.push_back(node->val);

    // 如果当前节点有右子树
    if(node->right != NULL){

        // 先把当前节点加入到栈中，用来记录节点移动的路径
        st.push(node);

        // 开始观察当前节点的右孩子节点
        node = node->right;

        // 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
        nodeState = nodeLeft;

        // 执行完上面三行代码，代码来到了位置 ①

    }else{
        // 如果当前节点没有右子树，切换当前节点的状态为【上】
        // 代码来到了位置 ②
        nodeState = nodeUp;
    }
}

}else if(nodeState == nodeUp){ // 如果当前节点的状态是【上】，说明左右孩子节点都已
经遍历，需要返回到它的父节点
    // 把当前节点加入到二叉树后序遍历的结果数组中
    // postorderResult.add(node.val);

    // 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
    // 首先构建一个空的节点
    TreeNode* parent = NULL;

    // 如果栈中有元素
    if(!st.empty()){

        // 那么，栈顶元素就是当前节点的父节点
        parent = st.top();
        st.pop();

        // 判断一下父节点的左节点是否为当前节点
        // 比如这颗二叉树
        //          1
        //        /  \
        //       2   3
        //      / \   \
        //     4  5   6
    }
}

```

```

        // 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换
        状态为【右】

        // 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切
        换状态

        // 如果父节点的左节点为当前节点
        if(parent->left == node){
            // 切换当前节点的状态为【右】
            nodeState = nodeRight;
        }
    }
    // 开始观察当前节点的父节点
    // 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
    // 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
    node = parent;

}

// 根据题目要求，返回二叉树前序、中序、后序遍历的结果
return preorderResult;

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二叉树的前序遍历（LeetCode 144）：https://leetcode-cn.com/problems/binary-tree-preorder-traversal/
class Solution:
    def preorderTraversal(self, root: TreeNode) -> List[int]:

        # 设置一个数组用来保存二叉树前序遍历的结果
        preorderResult = []

        # 设置一个数组用来保存二叉树中序遍历的结果
        inorderResult = []

        # 设置一个数组用来保存二叉树后序遍历的结果
        postorderResult = []

        # 设置一个栈，用来保存路径
        stack = list()

```

```

# 设置一个节点，一开始指向根节点
node = root

# 设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了
# 每个节点都有 左、右、上 这三种状态
# 按照 左 --> 右 --> 上 的顺序观察每个节点

# 左代表该节点的左右孩子节点都没有遍历
nodeLeft = 100

# 右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历
nodeRight = 200

# 上代表左右孩子节点都已经遍历，需要返回到它的父节点
nodeUp = 300

# 每个节点的初始化状态都是从 左 开始
nodeState = nodeLeft

# 对二叉树进行遍历
while node :

    # 位置 ①

    # 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if nodeState == nodeLeft :
        # 把当前节点加入到二叉树前序遍历的结果数组中
        preorderResult.append(node.val)

        # 如果当前节点有左子树
        if node.left :

            # 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.append(node)

            # 开始观察当前节点的左孩子节点，代码来到了位置 ①
            node = node.left
        else:

            # 如果当前节点没有左子树，切换当前节点的状态为【右】
            # 代码来到了位置 ②
            nodeState = nodeRight

    elif nodeState == nodeRight:

        # 如果当前节点的状态是【右】，说明该节点的左孩子节点已经遍历，右孩子节点还没有遍历

```

```

# 把当前节点加入到二叉树中序遍历的结果数组中
# inorderResult.add(node.val)

# 如果当前节点有右子树
if node.right :

    # 先把当前节点加入到栈中，用来记录节点移动的路径
    stack.append(node)

    # 开始观察当前节点的右孩子节点
    node = node.right

    # 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
    nodeState = nodeLeft

    # 执行完上面三行代码，代码来到了位置 ①

else:
    # 如果当前节点没有右子树，切换当前节点的状态为【上】
    # 代码来到了位置 ①
    nodeState = nodeUp

elif nodeState == nodeUp :

    # 如果当前节点的状态是【上】，说明左右孩子节点都已经遍历，需要返回到它的父节点
    # 把当前节点加入到二叉树后序遍历的结果数组中
    # postorderResult.add(node.val)

    # 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
    # 首先构建一个空的节点
    parent = None

    # 如果栈中有元素
    if stack :

        # 那么，栈顶元素就是当前节点的父节点
        parent = stack.pop()

        # 判断一下父节点的左节点是否为当前节点
        # 比如这颗二叉树
        #
        #       1
        #      / \
        #     2  3
        #    / \  \
        #   4  5  6
        #
        # 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换状态为【右】

```

态为【右】

```
状态

# 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切换

# 如果父节点的左节点为当前节点
if parent.left == node :
    # 切换当前节点的状态为【右】
    nodeState = nodeRight

# 开始观察当前节点的父节点
# 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
# 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
node = parent

# 根据题目要求，返回二叉树前序、中序、后序遍历的结果
return preorderResult
```

第三十七天

LeetCode 104、二叉树的最大深度

一、题目描述

给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

说明: 叶子节点是指没有子节点的节点。

示例：给定二叉树 `[3,9,20,null,null,15,7]`，



返回它的最大深度 3。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的最大深度 ( LeetCode 104 ) : https://leetcode-cn.com/problems/maximum-depth-of-binary-tree/
class Solution {
    public int maxDepth(TreeNode root) {

        // 如果 root 为空, 直接返回 0
        if(root == null) return 0;

        // 递归调用 maxDepth, 求出当前节点的左子树的最大深度
        int left = maxDepth(root.left);

        // 递归调用 maxDepth, 求出当前节点的右子树的最大深度
        int right = maxDepth(root.right);

        // 求出当前节点的左右子树中较大的值
        int childMaxDepth = Math.max(left, right);

        // 二叉树的最大深度就是它的左右子树中较大的值加上 1
        return childMaxDepth + 1;
    }
}
```

2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的最大深度 ( LeetCode 104 ) : https://leetcode-cn.com/problems/maximum-depth-of-binary-tree/
class Solution {
public:
    int maxDepth(TreeNode* root) {

        // 如果 root 为空, 直接返回 0
        if(root == NULL) return 0;

        // 递归调用 maxDepth, 求出当前节点的左子树的最大深度
        int left = maxDepth(root->left);

        // 递归调用 maxDepth, 求出当前节点的右子树的最大深度
```

```

    int right = maxDepth(root->right);

    // 求出当前节点的左右子树中较大的值
    int childMaxDepth = max(left,right);

    // 二叉树的最大深度就是它的左右子树中较大的值加上 1
    return childMaxDepth + 1;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

二叉树的最大深度 (LeetCode 104) : <https://leetcode-cn.com/problems/maximum-depth-of-binary-tree/>

```

class Solution:
    def maxDepth(self, root: TreeNode) -> int:

        # 如果 root 为空，直接返回 0
        if root == None :
            return 0

        # 递归调用 maxDepth，求出当前节点的左子树的最大深度
        left = self.maxDepth(root.left)

        # 递归调用 maxDepth，求出当前节点的右子树的最大深度
        right = self.maxDepth(root.right)

        # 求出当前节点的左右子树中较大的值
        childMaxDepth = max(left,right)

        # 二叉树的最大深度就是它的左右子树中较大的值加上 1
        return childMaxDepth + 1

```

LeetCode 110、平衡二叉树

一、题目描述

给定一个二叉树，判断它是否是高度平衡的二叉树。

本题中，一棵高度平衡二叉树定义为：

一个二叉树每个节点的左右两个子树的高度差的绝对值不超过 1 。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 平衡二叉树 ( LeetCode 110 ) :https://leetcode-cn.com/problems/balanced-binary-tree
class Solution {
    public boolean isBalanced(TreeNode root) {
        // 自底向上判断，只要返现有叶子树出现非平衡二叉树的情况，那么就是不是平衡二叉树了
        return recur(root) != -1;
    }

    private int recur(TreeNode curNode) {

        // 递归终止条件
        // 如果当前节点 curNode 为空，那么高度就是 0
        if (curNode == null) return 0;

        // 递归的计算当前节点 curNode 的左子树高度
        int left = recur(curNode.left);

        // 如果发现为 -1，即表示左子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        // 这个结果给上层
        // 提前终止了后续的判断
        if(left == -1) return -1;

        // 递归的计算当前节点 curNode 的右子树高度
        int right = recur(curNode.right);

        // 如果发现为 -1，即表示右子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        // 这个结果给上层
        // 提前终止了后续的判断
        if(right == -1) return -1;

        // 1、否则说明在 curNode 的左子树中没有发现不是平衡二叉树的情况
        // 2、否则说明在 curNode 的右子树中没有发现不是平衡二叉树的情况
        // 因此计算一下当前节点 curNode 是否是平衡二叉树
        // 即计算 curNode 的左子树高度与右子树高度差
        // 如果发现小于 2，那么返回以当前节点 curNode 为根节点的子树的最大高度
        // 即节点 curNode 的左右子树中最大高度加 1
        return Math.abs(left - right) < 2 ? Math.max(left, right) + 1 : -1;
    }
}
```


2、**C++** 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 平衡二叉树 ( LeetCode 110 ) :https://leetcode-cn.com/problems/balanced-binary-tree
class Solution {
public:
    bool isBalanced(TreeNode* root) {
        // 自底向上判断，只要返现有叶子树出现非平衡二叉树的情况，那么就是不是平衡二叉树了
        return recur(root) != -1;
    }
private:
    int recur(TreeNode* curNode) {

        // 递归终止条件
        // 如果当前节点 curNode 为空，那么高度就是 0
        if (curNode == NULL) return 0;

        // 递归的计算当前节点 curNode 的左子树高度
        int left = recur(curNode->left);

        // 如果发现为 -1，即表示左子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        // 这个结果给上层
        // 提前终止了后续的判断
        if(left == -1) return -1;

        // 递归的计算当前节点 curNode 的右子树高度
        int right = recur(curNode->right);

        // 如果发现为 -1，即表示右子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        // 这个结果给上层
        // 提前终止了后续的判断
        if(right == -1) return -1;

        // 1、否则说明在 curNode 的左子树中没有发现不是平衡二叉树的情况
        // 2、否则说明在 curNode 的右子树中没有发现不是平衡二叉树的情况
        // 因此计算一下当前节点 curNode 是否是平衡二叉树
        // 即计算 curNode 的左子树高度与右子树高度差
        // 如果发现小于 2，那么返回以当前节点 curNode 为根节点的子树的最大高度
        // 即节点 curNode 的左右子树中最大高度加 1
        return abs(left - right) < 2 ? max(left, right) + 1 : -1;
    }
};
```

3、Python 代码

```
#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 平衡二叉树 ( LeetCode 110 ) :https://leetcode-cn.com/problems/balanced-binary-tree
class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        # 自底向上判断，只要返现有叶子树出现非平衡二叉树的情况，那么就不是平衡二叉树了
        return self.recur(root) != -1

    def recur(self, curNode: TreeNode):

        # 递归终止条件
        # 如果当前节点 curNode 为空，那么高度就是 0
        if not curNode : return 0

        # 递归的计算当前节点 curNode 的左子树高度
        left = self.recur(curNode.left)

        # 如果发现为 -1，即表示左子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        # 这个结果给上层
        # 提前终止了后续的判断
        if left == -1 : return -1

        # 递归的计算当前节点 curNode 的右子树高度
        right = self.recur(curNode.right)

        # 如果发现为 -1，即表示右子树中出现了高度差大于 2 的情况，已经不是平衡二叉树了，可以直接返回
        # 这个结果给上层
        # 提前终止了后续的判断
        if right == -1 : return -1

        # 1、否则说明在 curNode 的左子树中没有发现不是平衡二叉树的情况
        # 2、否则说明在 curNode 的右子树中没有发现不是平衡二叉树的情况
        # 因此计算一下当前节点 curNode 是否是平衡二叉树
        # 即计算 curNode 的左子树高度与右子树高度差
        # 如果发现小于 2，那么返回以当前节点 curNode 为根节点的子树的最大高度
        # 即节点 curNode 的左右子树中最大高度加 1
        return max(left, right) + 1 if abs(left - right) < 2 else -1
```

LeetCode 111、二叉树的最小深度

一、题目描述

给定一个二叉树，找出其最小深度。

最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

说明：叶子节点是指没有子节点的节点。

示例 1:

```
输入: root = [3,9,20,null,null,15,7]
输出: 2
```

示例 2:

```
输入: root = [2,null,3,null,4,null,5,null,6]
输出: 5
```

提示:

- 树中节点数的范围在 $[0, 10^5]$ 内
- $-1000 \leq \text{Node.val} \leq 1000$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的最小深度 ( LeetCode 111 ) :https://leetcode-cn.com/problems/minimum-depth-of-binary-tree/
class Solution {
    public int minDepth(TreeNode root) {
        // 边界情况处理
        if (root == null) return 0;

        // 设置一个队列，用来存储二叉树中的元素
        Queue<TreeNode> nodeQueue = new LinkedList<>();

        // 队列添加二叉树的根节点
        nodeQueue.add(root);

        // 设置 depth 用来保存输出结果
        int depth = 0;
```

```

// 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
while (!nodeQueue.isEmpty()) {

    // 用来记录 queue 的长度，即每层节点的个数
    int size = nodeQueue.size();

    // 每到一层，深度就 +1
    depth++;

    // 使用 for 循环，将 nodeQueue 中的元素统计
    for (int i = 0; i < size; i++) {

        // 从 queue 中取出一个节点，此时，nodeQueue 已经抛出了这个节点
        TreeNode curNode = nodeQueue.poll();

        // curNode.left == null && curNode.right == null
        // 说明是叶子结点
        // 由于【最小深度是从根节点到最近叶子节点的最短路径上的节点数量】
        // 直接返回 depth
        if (curNode.left == null && curNode.right == null){
            return depth;
        }

        // 判断当前节点的左子节点是否有值，如果有，则添加到 nodeQueue 中
        if (curNode.left != null){
            nodeQueue.add(curNode.left);
        }

        // 判断当前节点的右子节点是否有值，如果有，则添加到 nodeQueue 中
        if (curNode.right != null){
            nodeQueue.add(curNode.right);
        }

    }

}

// 返回 depth
return depth;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www->algomooC->com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

```
// 二叉树的最小深度 ( LeetCode 111 ) :https://leetcode-cn->com/problems/minimum-depth-of-binary-tree/
class Solution {
public:
    int minDepth(TreeNode* root) {
        // 边界情况处理
        if(root == NULL) return 0;

        // 设置一个队列，用来存储二叉树中的元素

        queue<TreeNode *> nodeQueue ;

        // 队列添加二叉树的根节点
        nodeQueue.push(root);

        // 设置 depth 用来保存输出结果
        int depth = 0;

        // 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
        while (!nodeQueue.empty()) {

            // 用来记录 queue 的长度，即每层节点的个数
            int size = nodeQueue.size();

            // 每到一层，深度就 +1
            depth++;

            // 使用 for 循环，将 nodeQueue 中的元素统计
            for (int i = 0 ; i < size ; i++) {

                // 从 queue 中取出一个节点，此时，nodeQueue 已经抛出了这个节点
                TreeNode *curNode = nodeQueue.front();

                nodeQueue.pop();

                // curNode->left == NULL && curNode->right == NULL
                // 说明是叶子结点
                // 由于【最小深度是从根节点到最近叶子节点的最短路径上的节点数量】
                // 直接返回 depth
                if(curNode->left == NULL && curNode->right == NULL){
                    return depth;
                }

                // 判断当前节点的左子节点是否有值，如果有，则添加到 nodeQueue 中
                if (curNode->left != NULL){
                    nodeQueue.push(curNode->left);
                }

                // 判断当前节点的右子节点是否有值，如果有，则添加到 nodeQueue 中
```

```

        if (curNode->right != NULL){
            nodeQueue.push(curNode->right);
        }

    }

}

// 返回 depth
return depth;
}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二叉树的最小深度 ( LeetCode 111 ) :https://leetcode-cn.com/problems/minimum-depth-
of-binary-tree/
class Solution:
    def minDepth(self, root: TreeNode) -> int:
        # 边界情况处理
        if root == None :
            return 0

        # 设置一个队列，用来存储二叉树中的元素
        nodeQueue = deque()

        # 队列添加二叉树的根节点
        nodeQueue.append(root)

        # 设置 depth 用来保存输出结果
        depth = 0

        # 遍历队列，直到队列为空，说明访问了二叉树中所有的节点
        while nodeQueue :

            # 用来记录 queue 的长度，即每层节点的个数
            size = len(nodeQueue)

            # 每到一层，深度就 +1
            depth += 1

            # 使用 for 循环，将 nodeQueue 中的元素统计
            for i in range(size):

                # 从 queue 中取出一个节点，此时，nodeQueue 已经抛出了这个节点
                curNode = nodeQueue.popleft()

```

```

# curNode.left == None && curNode.right == None
# 说明是叶子结点
# 由于【最小深度是从根节点到最近叶子节点的最短路径上的节点数量】
# 直接返回 depth
if curNode.left == None and curNode.right == None :
    return depth

# 判断当前节点的左子节点是否有值，如果有，则添加到 nodeQueue 中
if curNode.left :
    nodeQueue.append(curNode.left)

# 判断当前节点的右子节点是否有值，如果有，则添加到 nodeQueue 中
if curNode.right :
    nodeQueue.append(curNode.right)

# 返回 depth
return depth

```

LeetCode 226、翻转二叉树

一、题目描述

你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

示例 1:

img

输入: root = [4,2,7,1,3,6,9]
输出: [4,7,2,9,6,3,1]

示例 2:

img

输入: root = [2,1,3]
输出: [2,3,1]

示例 3:

输入: root = []
输出: []

提示：

- 树中节点数目范围在 `[0, 100]` 内
- `-100 <= Node.val <= 100`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 翻转二叉树 (LeetCode 226) : https://leetcode.cn/problems/invert-binary-tree/
class Solution {
    public TreeNode invertTree(TreeNode root) {

        // 1、递归终止条件一，当前节点为空的时候，就返回 null
        if(root == null) return null;

        // 2、递归终止条件二，当前节点为叶子节点的时候，就返回这个节点
        if( root.left == null && root.right == null){

            // 返回这个节点
            return root;

        }

        // 3、把当前这个节点 root 的左子树进行翻转操作
        TreeNode left = invertTree(root.left);

        // 4、把当前这个节点 root 的右子树进行翻转操作
        TreeNode right = invertTree(root.right);

        // 5、把翻转成功的右子树赋值给 root 的左子树
        root.left = right;

        // 6、把翻转成功的左子树赋值给 root 的右子树
        root.right = left;

        // 7、返回翻转成功的二叉树的根节点
        return root;

    }
}
```


2、C++ 代码

```
class solution {
public:
    TreeNode* invertTree(TreeNode* root) {

        // 1、递归终止条件一，当前节点为空的时候，就返回 nullptr
        if(root == nullptr) return nullptr;

        // 2、递归终止条件二，当前节点为叶子节点的时候，就返回这个节点
        if( root->left == nullptr && root->right == nullptr){

            // 返回这个节点
            return root;

        }

        // 3、把当前这个节点 root 的左子树进行翻转操作
        TreeNode *left = invertTree(root->left);

        // 4、把当前这个节点 root 的右子树进行翻转操作
        TreeNode *right = invertTree(root->right);

        // 5、把翻转成功的右子树赋值给 root 的左子树
        root->left = right;

        // 6、把翻转成功的左子树赋值给 root 的右子树
        root->right = left;

        // 7、返回翻转成功的二叉树的根节点
        return root;

    }
};
```

3、Python 代码

```
class solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:

        # 1、递归终止条件一，当前节点为空的时候，就返回 null
        if not root : return None

        # 2、递归终止条件二，当前节点为叶子节点的时候，就返回这个节点
        if root.left == None and root.right == None :
            # 返回这个节点
            return root

        # 3、把当前这个节点 root 的左子树进行翻转操作
```

```
left = self.invertTree(root.left)

# 4、把当前这个节点 root 的右子树进行翻转操作
right = self.invertTree(root.right)

# 5、把翻转成功的右子树赋值给 root 的左子树
root.left = right

# 6、把翻转成功的左子树赋值给 root 的右子树
root.right = left

# 7、返回翻转成功的二叉树的根节点
return root
```

第三十八天

LeetCode 114、二叉树展开为链表

一、题目描述

给你二叉树的根结点 root，请你将它展开为一个单链表：

- 1、展开后的单链表应该同样使用 TreeNode，其中 right 子指针指向链表中下一个结点，而左子指针始终为 null。
- 2、展开后的单链表应该与二叉树先序遍历顺序相同。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树展开为链表（ LeetCode 114 ）：https://leetcode-cn.com/problems/flatten-binary-tree-to-linked-list/
class Solution {

    public void flatten(TreeNode root) {
        backtrack(root);
    }

    // 传入二叉树的节点，把它转换为链表的形式，返回二叉树的尾节点
    private TreeNode backtrack(TreeNode node){
```

```

// 1、如果 node 为空, 返回 null
if(node == null) return null;

// 2、如果 node 为叶子节点, 返回 node
if(node.left == null && node.right == null) return node;

// 下面开始设置几个指针

// 3、left 为当前节点 node 的左子树
TreeNode left = node.left;

// 4、right 为当前节点 node 的右子树
TreeNode right = node.right;

// 5、leftTail 指向当前节点左子树 left 转换为链表之后的尾节点, 一开始默认为 null
TreeNode leftTail = null;

// 6、rightTail 指向当前节点右子树 right 转换为链表之后的尾节点, 一开始默认为 null
TreeNode rightTail = null;

// 7、tail 指向以当前节点 node 为根节点转换为链表之后的尾节点, 一开始默认为 null
TreeNode tail = null;

// 8、先将 node 的左子树指针置空
// 将 node 的左子树转换为链表之后, node 的右指针指向那个链表
node.left = null;

// 9、如果当前节点存在左子树的时候, 那么把左子树转换为链表的形式
if(left != null){

    // 通过 backtrack 函数递归的把当前节点的左子树转换为链表
    // backtrack 函数指向完之后, left 已经是链表
    // 根据第 5 点的代码, leftTail 指向左子树最后一个节点
    leftTail = backtrack(left);

    // 此时, node 的左子树 left 已经是链表的形式
    // 那么将当前节点 node 的 right 指针指向 left, 完成了当前节点和左子树链表的拼接
    node.right = left;

    // 根据第 7 , tail 指向左子树最后一个节点
    tail = leftTail;
}

// 10、如果当前节点存在右子树的时候, 那么把左子树转换为链表的形式
if(right != null){

    // 通过 backtrack 函数递归的把当前节点的右子树转换为链表
    // backtrack 函数指向完之后, right 已经是链表

```

```

// 根据第 6 点的代码, rightTail 指向右子树最后一个节点
rightTail = backtrack(right);

// 此时, node 的右子树 right 已经是链表的形式
// 如果当前节点 node 不存在左子树, 那么 node.left = null
// 由于 node 的右指针就是 right, 所以不需要执行其它操作
// 但如果存在左子树, 就需要把 left 链表和 right 链表串联起来
// 也就是把 left 链表的尾节点和 right 的头节点拼接起来
if(left != null){
    // 将 leftTail 和 right 转换成的链表链接起来
    leftTail.right = right;
}

// 如果存在右子树, 那么根据第 7 点的代码, tail 指向右子树最后一个节点
tail = rightTail;
}

// 返回链表的尾节点位置继续去拼接其它的递归链表
return tail;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树展开为链表 ( LeetCode 114 ) : https://leetcode-cn.com/problems/flatten-binary-tree-to-linked-list/
class Solution {
public:
    void flatten(TreeNode* root) {
        backtrack(root);
    }

    // 传入二叉树的节点, 把它转换为链表的形式, 返回二叉树的尾节点
    TreeNode* backtrack(TreeNode* node){

        // 1、如果 node 为空, 返回 NULL
        if(node == NULL) return NULL;

        // 2、如果 node 为叶子节点, 返回 node
        if(node->left == NULL && node->right == NULL) return node;

        // 下面开始设置几个指针

        // 3、left 为当前节点 node 的左子树
    }
}

```

```

TreeNode* left = node->left;

// 4、right 为当前节点 node 的右子树
TreeNode* right = node->right;

// 5、leftTail 指向当前节点左子树 left 转换为链表之后的尾节点，一开始默认为 NULL
TreeNode* leftTail = NULL;

// 6、rightTail 指向当前节点右子树 right 转换为链表之后的尾节点，一开始默认为 NULL
TreeNode* rightTail = NULL;

// 7、tail 指向以当前节点 node 为根节点转换为链表之后的尾节点，一开始默认为 NULL
TreeNode* tail = NULL;

// 8、先将 node 的左子树指针置空
// 将 node 的左子树转换为链表之后，node 的右指针指向那个链表
node->left = NULL;

// 9、如果当前节点存在左子树的时候，那么把左子树转换为链表的形式
if(left != NULL){

    // 通过 backtrack 函数递归的把当前节点的左子树转换为链表
    // backtrack 函数指向完之后，left 已经是链表
    // 根据第 5 点的代码，leftTail 指向左子树最后一个节点
    leftTail = backtrack(left);

    // 此时，node 的左子树 left 已经是链表的形式
    // 那么将当前节点 node 的 right 指针指向 left，完成了当前节点和左子树链表的拼接
    node->right = left;

    // 根据第 7，tail 指向左子树最后一个节点
    tail = leftTail;
}

// 10、如果当前节点存在右子树的时候，那么把右子树转换为链表的形式
if(right != NULL){

    // 通过 backtrack 函数递归的把当前节点的右子树转换为链表
    // backtrack 函数指向完之后，right 已经是链表
    // 根据第 6 点的代码，rightTail 指向右子树最后一个节点
    rightTail = backtrack(right);

    // 此时，node 的右子树 right 已经是链表的形式
    // 如果当前节点 node 不存在左子树，那么 node->left = NULL
    // 由于 node 的右指针就是 right，所以不需要执行其它操作
    // 但如果存在左子树，就需要把 left 链表和 right 链表串联起来
    // 也就是把 left 链表的尾节点和 right 的头节点拼接起来
    if(left != NULL){

```

```

        // 将 leftTail 和 right 转换成的链表链接起来
        leftTail->right = right;
    }

    // 如果存在右子树，那么根据第 7 点的代码，tail 指向右子树最后一个节点
    tail = rightTail;
}

// 返回链表的尾节点位置继续去拼接其它的递归链表
return tail;

}

};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 二叉树展开为链表 ( LeetCode 114 ) : https://leetcode-cn.com/problems/flatten-binary-tree-to-linked-list/
class Solution:
    def flatten(self, root: TreeNode) -> None:
        self.backtrack(root)

    # 传入二叉树的节点，把它转换为链表的形式，返回二叉树的尾节点
    def backtrack(self, node: TreeNode) -> TreeNode :

        # 1、如果 node 为空，返回 None
        if not node :
            return None

        # 2、如果 node 为叶子节点，返回 node
        if not node.left and not node.right :
            return node

        # 下面开始设置几个指针

        # 3、left 为当前节点 node 的左子树
        left = node.left

        # 4、right 为当前节点 node 的右子树
        right = node.right

        # 5、leftTail 指向当前节点左子树 left 转换为链表之后的尾节点，一开始默认为 None
        leftTail = None

        # 6、rightTail 指向当前节点右子树 right 转换为链表之后的尾节点，一开始默认为 None

```

```

rightTail = None

# 7、tail 指向以当前节点 node 为根节点转换为链表之后的尾节点，一开始默认为 None
tail = None

# 8、先将 node 的左子树指针置空
# 将 node 的左子树转换为链表之后，node 的右指针指向那个链表
node.left = None

# 9、如果当前节点存在左子树的时候，那么把左子树转换为链表的形式
if left :

    # 通过 backtrack 函数递归的把当前节点的左子树转换为链表
    # backtrack 函数指向完之后，left 已经是链表
    # 根据第 5 点的代码，leftTail 指向左子树最后一个节点
    leftTail = self.backtrack(left)

    # 此时，node 的左子树 left 已经是链表的形式
    # 那么将当前节点 node 的 right 指针指向 left，完成了当前节点和左子树链表的拼接
    node.right = left

    # 根据第 7，tail 指向左子树最后一个节点
    tail = leftTail

# 10、如果当前节点存在右子树的时候，那么把右子树转换为链表的形式
if right :

    # 通过 backtrack 函数递归的把当前节点的右子树转换为链表
    # backtrack 函数指向完之后，right 已经是链表
    # 根据第 6 点的代码，rightTail 指向右子树最后一个节点
    rightTail = self.backtrack(right)

    # 此时，node 的右子树 right 已经是链表的形式
    # 如果当前节点 node 不存在左子树，那么 node.left = None
    # 由于 node 的右指针就是 right，所以不需要执行其它操作
    # 但如果存在左子树，就需要把 left 链表和 right 链表串联起来
    # 也就是把 left 链表的尾节点和 right 的头节点拼接起来
    if left :
        # 将 leftTail 和 right 转换成的链表链接起来
        leftTail.right = right

    # 如果存在右子树，那么根据第 7 点的代码，tail 指向右子树最后一个节点
    tail = rightTail

# 返回链表的尾节点位置继续去拼接其它的递归链表

```

```
return tail
```

LeetCode 199、二叉树的右视图

一、题目描述

给定一个二叉树的 **根节点** root，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的右视图（LeetCode 199）：https://leetcode-cn.com/problems/binary-tree-right-side-view/
class Solution {
    public List<Integer> rightSideView(TreeNode root) {

        // 设置一个数组用来存储二叉树的右视图结果
        // 题目要求存储的是节点的值，所以类型是 Integer
        List<Integer> list = new LinkedList<>();

        // 设置一个队列，用来保存二叉树每一层的节点
        Queue<TreeNode> que = new LinkedList<>();

        // 边界判断
        if (root == null) return list;

        // 首先，把二叉树的根节点加入到队列中
        que.add(root);

        // 观察队列是否为空
        while (!que.isEmpty()) {

            // 1、获取队列的长度
            int levelSize = que.size();

            // 2、通过一个循环，获取队列中每个节点的左右子树，把这些左右子树节点也添加到队列中
            for (int i = 0; i < levelSize; i++) {

                // 3、弹出队列的队首元素
                TreeNode node = que.poll();
```



```

        // 4、判断弹出的节点是否有左子树
        if (node.left != null) {
            // 如果有左子树，把它加入到队列中
            que.add(node.left);
        }

        // 5、判断弹出的节点是否有右子树
        if (node.right != null) {
            // 如果有右子树，把它加入到队列中
            que.add(node.right);
        }

        // 对于每一层的二叉树节点，我们是从左到右依次添加，所以末尾的节点的顺序是 levelSize
- 1

        // 6、这个末尾的节点就是这一层中最右侧的节点
        if (i == levelSize - 1) {
            // 把最右侧的节点值加入到结果中
            list.add(node.val);
        }
    }
}
// 返回结果
return list;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的右视图（ LeetCode 199 ）： https://leetcode-cn.com/problems/binary-tree-right-side-view/
class Solution {
public:
    vector<int> rightSideview(TreeNode* root) {

        // 设置一个数组用来存储二叉树的右视图结果
        // 题目要求存储的是节点的值，所以类型是 Integer
        vector<int> list;

        // 设置一个队列，用来保存二叉树每一层的节点
        queue<TreeNode*> que;

        // 边界判断
        if (root == NULL) return list;

        // 首先，把二叉树的根节点加入到队列中
    }
}

```

```

    que.push(root);

    // 观察队列是否为空
    while (!que.empty()) {

        // 1、获取队列的长度
        int levelSize = que.size();

        // 2、通过一个循环，获取队列中每个节点的左右子树，把这些左右子树节点也添加到队列中
        for (int i = 0; i < levelSize; i++) {

            // 3、弹出队列的队首元素
            TreeNode* node = que.front();
            que.pop();

            // 4、判断弹出的节点是否有左子树
            if (node->left != NULL) {
                // 如果有左子树，把它加入到队列中
                que.push(node->left);
            }

            // 5、判断弹出的节点是否有右子树
            if (node->right != NULL) {
                // 如果有右子树，把它加入到队列中
                que.push(node->right);
            }

            // 对于每一层的二叉树节点，我们是从左到右依次添加，所以末尾的节点的顺序是 levelSize
            - 1

            // 6、这个末尾的节点就是这一层中最右侧的节点
            if (i == levelSize - 1) {
                // 把最右侧的节点值加入到结果中
                list.push_back(node->val);
            }
        }
    }
    // 返回结果
    return list;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解
 ##### <https://www.algomooc.com>
 ##### 作者：程序员吴师兄
 ##### 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

#### 二叉树的右视图 ( LeetCode 199 ) : https://leetcode-cn.com/problems/binary-tree-right-side-view/
class Solution:
    def rightSideView(self, root: TreeNode) -> List[int]:

        # 设置一个数组用来存储二叉树的右视图结果
        # 题目要求存储的是节点的值，所以类型是 Integer
        list = []

        # 设置一个队列，用来保存二叉树每一层的节点
        que = deque()

        # 边界判断
        if not root :
            return list

        # 首先，把二叉树的根节点加入到队列中
        que.append(root)

        # 观察队列是否为空
        while que:

            # 1、获取队列的长度
            levelSize = len(que)

            # 2、通过一个循环，获取队列中每个节点的左右子树，把这些左右子树节点也添加到队列中
            for i in range(levelSize):

                # 3、弹出队列的队首元素
                node = que.popleft()

                # 4、判断弹出的节点是否有左子树
                if node.left :
                    # 如果有左子树，把它加入到队列中
                    que.append(node.left)

                # 5、判断弹出的节点是否有右子树
                if node.right :
                    # 如果有右子树，把它加入到队列中
                    que.append(node.right)

            # 对于每一层的二叉树节点，我们是从左到右依次添加，所以末尾的节点的顺序是 levelSize

            # 6、这个末尾的节点就是这一层中最右侧的节点
            if i == levelSize - 1 :
                # 把最右侧的节点值加入到结果中
                list.append(node.val)

```

```
# 返回结果
return list
```

LeetCode 222 、完全二叉树的节点个数

一、题目描述

给出一个完全二叉树，求出该树的节点个数。

示例 1:

```
输入: root = [1,2,3,4,5,6]
输出: 6
```

示例 2:

```
输入: root = []
输出: 0
```

示例 3:

```
输入: root = [1]
输出: 1
```

提示:

- 树中节点的数目范围是 $[0, 5 * 10^4]$
- $0 \leq \text{Node.val} \leq 5 * 10^4$
- 题目数据保证输入的树是 完全二叉树

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 全二叉树的节点个数 ( LeetCode 222 ) : https://leetcode-cn.com/problems/count-complete-tree-nodes
class Solution {

    public int countNodes(TreeNode root) {

        // 边界判断
        // 递归终止条件，如果当前节点为 null，肯定节点个数为 0
```

```

if(root == null) {
    return 0;
}

// 先获取当前节点的左子树的深度
int leftDepth = getDepth(root.left);

// 再获取当前节点的右子树的深度
int rightDepth = getDepth(root.right);

// 如果 左子树的深度 等于 右子树的深度
// 由于在完全二叉树中，对于某一层的节点来说，只有填充满了这一层的左子树的节点才会填充右子树
// 因此，这意味着左子树是满二叉树
if (leftDepth == rightDepth) {

    // 对于一棵满二叉树来说
    //      1
    //    /  \
    //   2    3
    // 它的节点数是  $2^{\text{depth}} - 1$ 
    // 位移操作，记得加括号，优先级比较低，不加容易出错
    int leftNodes = (1 << leftDepth) - 1;

    // 再去递归的获取它的右子树的节点数
    int rightNodes = countNodes(root.right);

    // 最后返回结果
    // 注意，当前节点也是需要再统计一下的
    return leftNodes + rightNodes + 1;

// 如果 左子树的深度 不等于 右子树的深度
// 由于在完全二叉树中，除了最底层节点可能没填满外，其余每层节点数都达到最大值
// 因此，这意味着右子树是满二叉树
// 这样，才会出现左子树的深度比右子树的深度更深
} else {

    // 对于一棵满二叉树来说
    //      1
    //    /  \
    //   2    3
    // 它的节点数是  $2^{\text{depth}} - 1$ 
    // 位移操作，记得加括号，优先级比较低，不加容易出错
    int rightNodes = (1 << rightDepth) - 1;

    // 再去递归的获取它的右子树的节点数
    int leftNodes = countNodes(root.left);

    // 最后返回结果
    // 注意，当前节点也是需要再统计一下的

```

```

        return leftNodes + rightNodes + 1;
    }
}

// 统计树的深度
private int getDepth(TreeNode curNode) {

    // 默认当前节点 curNode 的深度为 0
    int depth = 0;

    // 只要 curNode 节点有值，就不断的判断
    while (curNode != null) {

        // 由于当前节点 curNode 是在一棵完全二叉树中
        // 最下面一层的节点都集中在该层最左边的若干位置
        // 即只有它的下一层左子树满了，才会填充右子树
        // 因此，如果只需要判断 curNode 的左子树是否有值就行
        curNode = curNode.left;

        // 当前节点 curNode 有值，说明深度加深了一个
        depth++;
    }

    // 返回以 curNode 作为根节点的树的深度
    return depth;
}
}

```

LeetCode 236、二叉树的最近公共祖先

一、题目描述

给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。

最近公共祖先的定义为：“对于有根树 T 的两个节点 **p**、**q**，最近公共祖先表示为一个节点 x，满足 x 是 **p**、**q** 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的最近公共祖先（ LeetCode 236 ）:https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-tree/
class Solution {

```

```

public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {

    // 1、如果此时访问的节点 root 为 null, 那么直接返回 null
    if(root == null ) return null;

    // 2、如果此时访问的节点 root 为指定节点 p, 那么返回 p 这个节点
    if(root == p) return p;

    // 3、如果此时访问的节点 root 为指定节点 q, 那么返回 q 这个节点
    if(root == q) return q;

    // 4、经过上面 1、2、3 的判断之后, root 这个节点必然不是 p、q、null 这三种情况的节点
    // 接下来, 去递归的判断当前节点 root 的左右子树是否包含 p 、q 这两个指定节点

    // 5、递归的去查找 root 的左子树, 判断是否包含p 、q 这两个指定节点
    // 如果 root 的左子树节点和它的左子树所有节点中包含 p, 那么 left 的值就是 p
    // 如果 root 的左子树节点和它的左子树所有节点中包含 q, 那么 left 的值就是 q
    // 如果 root 的左子树节点和它的左子树所有节点中既不包含 p, 也不包含 q, 那么 left 的值就是
null
    TreeNode left = lowestCommonAncestor(root.left, p, q);

    // 6、递归的去查找 root 的右子树, 判断是否包含p 、q 这两个指定节点
    // 如果 root 的右子树节点和它的右子树所有节点中包含 p, 那么 right 的值就是 p
    // 如果 root 的右子树节点和它的右子树所有节点中包含 q, 那么 right 的值就是 q
    // 如果 root 的右子树节点和它的右子树所有节点中既不包含 p, 也不包含 q, 那么 right 的值就是
null
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    // 7、判断完当前节点 root 、 root 的左子树 left、root 的右子树 right 的情况后
    // 开始向父节点传递信息

    // 8、如果 root 节点的左子树 left 和右子树 right 都没有找到指定节点 p、q
    // 并且 root 自己本身也不是指定节点 p、q
    // 那么它给父节点传递的信息就是以 root 为根节点的那颗二叉树没有指定节点 p、q
    if(left == null && right == null){

        // 返回 null, 告诉 root 的父节点, 这里没找到指定节点 p、q
        return null;

    // 9、如果在 root 节点的左子树 left 中没有找到指定节点 p、q
    // 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 right 了
    }else if(left == null){

        // 返回 right , 告诉 root 的父节点, 能不能找到指定节点 p、q 完全取决于 right
        return right;
    }
}

```

```

// 10、如果在 root 节点的右子树 right 中没有找到指定节点 p、q
// 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 left 了
}else if(right == null){

    // 返回 left , 告诉 root 的父节点, 能不能找到指定节点 p、q 完全取决于 left
    return left;

// 11、此时, left != null 并且 right != null
// 这说明在以 root 节点为根节点的那棵二叉树中找到了指定节点 p , 也找到了指定节点 q
// 那么就告诉父节点, root 就是 p、q 的最近公共祖先
}else{

    // 返回 root , 告诉 root 的父节点, root 就是 p、q 的最近公共祖先
    return root;
}
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的最近公共祖先 ( LeetCode 236 ) :https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-tree/
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {

        // 1、如果此时访问的节点 root 为 NULL, 那么直接返回 NULL
        if(root == NULL ) return NULL;

        // 2、如果此时访问的节点 root 为指定节点 p, 那么返回 p 这个节点
        if(root == p) return p;

        // 3、如果此时访问的节点 root 为指定节点 q, 那么返回 q 这个节点
        if(root == q) return q;

        // 4、经过上面 1、2、3 的判断之后, root 这个节点必然不是 p、q、NULL 这三种情况的节点
        // 接下来, 去递归的判断当前节点 root 的左右子树是否包含 p 、 q 这两个指定节点

        // 5、递归的去查找 root 的左子树, 判断是否包含p 、 q 这两个指定节点
        // 如果 root 的左子树节点和它的左子树所有节点中包含 p, 那么 left 的值就是 p
        // 如果 root 的左子树节点和它的左子树所有节点中包含 q, 那么 left 的值就是 q
        // 如果 root 的左子树节点和它的左子树所有节点中既不包含 p, 也不包含 q, 那么 left 的值就是
        NULL

        TreeNode* left = lowestCommonAncestor(root->left, p, q);
    }
}

```



```

// 6、递归的去查找 root 的右子树，判断是否包含p 、q 这两个指定节点
// 如果 root 的右子树节点和它的右子树所有节点中包含 p，那么 right 的值就是 p
// 如果 root 的右子树节点和它的右子树所有节点中包含 q，那么 right 的值就是 q
// 如果 root 的右子树节点和它的右子树所有节点中既不包含 p，也不包含 q，那么 right 的值就是
NULL

TreeNode* right = lowestCommonAncestor(root->right, p, q);

// 7、判断完当前节点 root 、 root 的左子树 left、root 的右子树 right 的情况后
// 开始向父节点传递信息

// 8、如果 root 节点的左子树 left 和右子树 right 都没有找到指定节点 p、q
// 并且 root 自己本身也不是指定节点 p、q
// 那么它给父节点传递的信息就是以 root 为根节点的那颗二叉树没有指定节点 p、q
if(left == NULL && right == NULL){

    // 返回 NULL，告诉 root 的父节点，这里没找到指定节点 p、q
    return NULL;

// 9、如果在 root 节点的左子树 left 中没有找到指定节点 p、q
// 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 right 了
}else if(left == NULL){

    // 返回 right ，告诉 root 的父节点，能不能找到指定节点 p、q 完全取决于 right
    return right;

// 10、如果在 root 节点的右子树 right 中没有找到指定节点 p、q
// 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 left 了
}else if(right == NULL){

    // 返回 left ，告诉 root 的父节点，能不能找到指定节点 p、q 完全取决于 left
    return left;

// 11、此时，left != NULL 并且 right != NULL
// 这说明在以 root 节点为根节点的那棵二叉树中找到了指定节点 p ，也找到了指定节点 q
// 那么就告诉父节点，root 就是 p、q 的最近公共祖先
}else{

    // 返回 root ，告诉 root 的父节点，root 就是 p、q 的最近公共祖先
    return root;
}
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

二叉树的最近公共祖先 (LeetCode 236) :<https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-tree/>

class Solution:

def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':

1、如果此时访问的节点 root 为 null, 那么直接返回 null

if not root:
 return None

2、如果此时访问的节点 root 为指定节点 p, 那么返回 p 这个节点

if root == p : return p

3、如果此时访问的节点 root 为指定节点 q, 那么返回 q 这个节点

if root == q : return q

4、经过上面 1、2、3 的判断之后, root 这个节点必然不是 p、q、null 这三种情况的节点
接下来, 去递归的判断当前节点 root 的左右子树是否包含 p 、 q 这两个指定节点

5、递归的去查找 root 的左子树, 判断是否包含p 、 q 这两个指定节点

如果 root 的左子树节点和它的左子树所有节点中包含 p, 那么 left 的值就是 p

如果 root 的左子树节点和它的左子树所有节点中包含 q, 那么 left 的值就是 q

如果 root 的左子树节点和它的左子树所有节点中既不包含 p, 也不包含 q, 那么 left 的值就是

null

left = self.lowestCommonAncestor(root.left,p,q)

6、递归的去查找 root 的右子树, 判断是否包含p 、 q 这两个指定节点

如果 root 的右子树节点和它的右子树所有节点中包含 p, 那么 right 的值就是 p

如果 root 的右子树节点和它的右子树所有节点中包含 q, 那么 right 的值就是 q

如果 root 的右子树节点和它的右子树所有节点中既不包含 p, 也不包含 q, 那么 right 的值就是

null

right = self.lowestCommonAncestor(root.right,p,q)

7、判断完当前节点 root 、 root 的左子树 left、root 的右子树 right 的情况后
开始向父节点传递信息

8、如果 root 节点的左子树 left 和右子树 right 都没有找到指定节点 p、q

并且 root 自己本身也不是指定节点 p、q

那么它给父节点传递的信息就是以 root 为根节点的那颗二叉树没有指定节点 p、q

if not left and not right :

```

        # 返回 null, 告诉 root 的父节点, 这里没找到指定节点 p、q
        return None

# 9、如果在 root 节点的左子树 left 中没有找到指定节点 p、q
# 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 right 了
elif not left :

    # 返回 right , 告诉 root 的父节点, 能不能找到指定节点 p、q 完全取决于 right
    return right

# 10、如果在 root 节点的右子树 right 中没有找到指定节点 p、q
# 那么以 root 为根节点的那颗二叉树能不能找到指定节点 p、q 完全取决于 left 了
elif not right :

    # 返回 left , 告诉 root 的父节点, 能不能找到指定节点 p、q 完全取决于 left
    return left

# 11、此时, left != null 并且 right != null
# 这说明在以 root 节点为根节点的那棵二叉树中找到了指定节点 p , 也找到了指定节点 q
# 那么就告诉父节点, root 就是 p、q 的最近公共祖先
else:

    # 返回 root , 告诉 root 的父节点, root 就是 p、q 的最近公共祖先
    return root

```

第三十九天

LeetCode 235、二叉搜索树的最近公共祖先

一、题目描述

给定一个二叉搜索树, 找到该树中两个指定节点的最近公共祖先。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉搜索树的最近公共祖先 ( LeetCode 235 ) :https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-search-tree/
class Solution {

```

```

public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {

    // 二叉搜索树的特点是，对于每一个节点 root 来说
    // root.left < root < root.right

    // 如果说 root 是 p、q 的最近公共祖先，也就意味值 p、q 在 root 的两侧
    // 假设 p 在 root 的左侧，q 在 root 的右侧
    // 那么 p.val < root.val < q.val
    // 1、root.val - p.val > 0
    // 2、root.val - q.val < 0
    // 即 ( root.val - p.val ) * ( root.val - q.val ) < 0
    // 所以，如果发现 ( root.val - p.val ) * ( root.val - q.val ) > 0
    // 说明 p、q 在 root 的同一侧，不是最近公共祖先
    // 需要搜索 root 的左右子树

    // 由于所有节点的值都是唯一的，当出现 ( root.val - p.val ) * ( root.val - q.val ) = 0
    // 说明 p、q 中有一个节点是 root 节点，即 p 是 q 的祖先节点，或者 q 是 p 的祖先节点
    while ( (long) (root.val - p.val) * (root.val - q.val) > 0 ){

        // 当进入到这个循环的时候，说明 p、q 在 root 的同一侧

        // 如果 p.val < root.val，也说明 q.val < root.val，因为只有这样，乘积才能大于 0
        if( p.val < root.val ){

            // 接下来往左子树寻找
            root = root.left;

            // 如果 p.val > root.val，也说明 q.val > root.val，因为只有这样，乘积才能大于 0
        }else{

            // 接下来往右子树寻找
            root = root.right;
        }
    }

    // 跳出循环，说明 (root.val - p.val) * (root.val - q.val) <= 0
    // 此时，root 就是 p 、q 的最近公共祖先节点
    return root;
}
}

```

2、C++ 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

```

// 二叉搜索树的最近公共祖先 ( LeetCode 235 ) :https://leetcode-cn.com/problems/lowest-
common-ancestor-of-a-binary-search-tree/
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {

        // 二叉搜索树的特点是, 对于每一个节点 root 来说
        // root->left < root < root->right

        // 如果说 root 是 p、q 的最近公共祖先, 也就意味值 p、q 在 root 的两侧
        // 假设 p 在 root 的左侧, q 在 root 的右侧
        // 那么 p->val < root->val < q->val
        // 1、root->val - p->val > 0
        // 2、root->val - q->val < 0
        // 即 ( root->val - p->val ) * ( root->val - q->val ) < 0
        // 所以, 如果发现 ( root->val - p->val ) * ( root->val - q->val ) > 0
        // 说明 p、q 在 root 的同一侧, 不是最近公共祖先
        // 需要搜索 root 的左右子树

        // 由于所有节点的值都是唯一的, 当出现 ( root->val - p->val ) * ( root->val - q->val )
= 0 时
        // 说明 p、q 中有一个节点是 root 节点, 即 p 是 q 的祖先节点, 或者 q 是 p 的祖先节点
        while ( (root->val - p->val) * (root->val - q->val) > 0 ){

            // 当进入到这个循环的时候, 说明 p、q 在 root 的同一侧

            // 如果 p->val < root->val, 也说明 q->val < root->val, 因为只有这样, 乘积才能大于
0
            if( p->val < root->val ){

                // 接下来往左子树寻找
                root = root->left;

                // 如果 p->val > root->val, 也说明 q->val > root->val, 因为只有这样, 乘积才能大于
0
            }else{

                // 接下来往右子树寻找
                root = root->right;
            }
        }

        // 跳出循环, 说明 (root->val - p->val) * (root->val - q->val) <= 0
        // 此时, root 就是 p 、q 的最近公共祖先节点
        return root;
    }
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

二叉搜索树的最近公共祖先 (LeetCode 235) :<https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-search-tree/>

class Solution:

def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':

二叉搜索树的特点是，对于每一个节点 root 来说

$root.left < root < root.right$

如果说 root 是 p、q 的最近公共祖先，也就意味值 p、q 在 root 的两侧

假设 p 在 root 的左侧，q 在 root 的右侧

那么 $p.val < root.val < q.val$

1、 $root.val - p.val > 0$

2、 $root.val - q.val < 0$

即 $(root.val - p.val) * (root.val - q.val) < 0$

所以，如果发现 $(root.val - p.val) * (root.val - q.val) > 0$

说明 p、q 在 root 的同一侧，不是最近公共祖先

需要搜索 root 的左右子树

由于所有节点的值都是唯一的，当出现 $(root.val - p.val) * (root.val - q.val) = 0$

时

说明 p、q 中有一个节点是 root 节点，即 p 是 q 的祖先节点，或者 q 是 p 的祖先节点

while (root.val - p.val) * (root.val - q.val) > 0 :

当进入到这个循环的时候，说明 p、q 在 root 的同一侧

如果 $p.val < root.val$ ，也说明 $q.val < root.val$ ，因为只有这样，乘积才能大于 0

if $p.val < root.val$:

接下来往左子树寻找

root = root.left

如果 $p.val > root.val$ ，也说明 $q.val > root.val$ ，因为只有这样，乘积才能大于 0

else:

接下来往右子树寻找

root = root.right

跳出循环，说明 $(root.val - p.val) * (root.val - q.val) \leq 0$

此时，root 就是 p、q 的最近公共祖先节点

return root

LeetCode 450、删除二叉搜索树中的节点

一、题目描述

给定一个二叉搜索树的根节点 **root** 和一个值 **key**，删除二叉搜索树中的 **key** 对应的节点，并保证二叉搜索树的性质不变。返回二叉搜索树（有可能被更新）的根节点的引用。

一般来说，删除节点可分为两个步骤：

- 1、首先找到需要删除的节点；
- 2、如果找到了，删除它。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 删除二叉搜索树中的节点（ LeetCode 450 ） : https://leetcode-cn.com/problems/delete-node-in-a-bst/
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {

        // 1、如果 root 为空，那么直接返回空
        if (root == null) return null;

        // 2、如果 root 的节点值等于需要删除的值，那么需要根据以下几种情况进行处理
        if (root.val == key) {

            // 情况 1：当前节点的左子树为空，那么当前节点 root 由 root 的右子树占位就行
            // 比如 key 为 7
            //      6
            //     / \
            //    2  7
            //       \
            //        8
            // 直接将 8 作为根节点的二叉树挪到 7 的位置
            //      6
            //     / \
            //    2  8
            if (root.left == null) return root.right;

            // 情况 2：当前节点的右子树为空，那么当前节点 root 由 root 的左子树占位就行
            // 比如 key 为 2
            //      6
            //     / \
            //    2  7
```

```

//    /
//   1
//  直接将 1 作为根节点的二叉树挪到 2 的位置
//      6
//     / \
//    1   7
if (root.right == null) return root.left;

// 情况 3: 被删除节点既有左子树, 又有右子树
// 比如 key 为 2
//      5
//     / \
//    2   6
//   / \   \
//  1  4   7
//     /
//    3
// 需要找到右子树最小的值, 或者左子树中最大的值
// 这里我们去找右子树最小的值, 为 3
TreeNode minNodeOfRight = findMinNode(root.right);

// 找到右子树最小的值之后, 修改当前节点 root 的值为右子树最小的值
root.val = minNodeOfRight.val;

// 同时, 记得删除掉 root 的右子树最小的值之
// 删除操作就是以 root 作为根节点, key 为右子树最小的值进行删除
root.right = deleteNode(root.right, minNodeOfRight.val);

// 3、如果 root 的节点值小于需要删除的值, 那么就在 root 的右子树中去查找
} else if (root.val < key) {
    // 在 root 的右子树中去查找并删除 key
    root.right = deleteNode(root.right, key);

    // 4、如果 root 的节点值大于需要删除的值, 那么就在 root 的左子树中去查找
} else if (root.val > key) {
    // 在 root 的左子树中去查找并删除 key
    root.left = deleteNode(root.left, key);
}

// 最后返回需要已经删除了 key 的二叉树的根节点
return root;
}

// 通过 findMinNode , 可以找到二叉搜索树中最小的元素
private TreeNode findMinNode(TreeNode node) {

    // 由于二叉搜索树, 左子树所有元素的值都小于根节点的值
    // 所以可以不断的查找, 直到为叶子节点, 那么就找到了
    while (node.left != null) {

```



```

        // 不断的去查找当前节点的左子树
        node = node.left;
    }

    // 返回当前二叉搜索树中最小的元素
    return node;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 删除二叉搜索树中的节点 ( LeetCode 450 ) :https://leetcode-cn.com/problems/delete-node-in-a-bst/
class Solution
{
public:
    TreeNode *deleteNode(TreeNode *root, int key)
    {

        // 1、如果 root 为空，那么直接返回空
        if (root == NULL)
            return NULL;

        // 2、如果 root 的节点值等于需要删除的值，那么需要根据以下几种情况进行处理
        if (root->val == key)
        {

            // 情况 1：当前节点的左子树为空，那么当前节点 root 由 root 的右子树占位就行
            // 比如 key 为 7
            //      6
            //     / \
            //    2   7
            //         \
            //          8
            // 直接将 8 作为根节点的二叉树挪到 7 的位置
            //      6
            //     / \
            //    2   8
            if (root->left == NULL)
                return root->right;

            // 情况 2：当前节点的右子树为空，那么当前节点 root 由 root 的左子树占位就行
            // 比如 key 为 2
            //      6
            //     / \
            //    2   \

```

```

//      2      7
//      /
//     1
// 直接将 1 作为根节点的二叉树挪到 2 的位置
//           6
//          / \
//         1   7
if (root->right == NULL)
    return root->left;

// 情况 3: 被删除节点既有左子树, 又有右子树
// 比如 key 为 2
//           5
//          / \
//         2   6
//        / \   \
//       1  4   7
//        /
//       3
// 需要找到右子树最小的值, 或者左子树中最大的值
// 这里我们去找右子树最小的值, 为 3
TreeNode *minNodeOfRight = findMinNode(root->right);

// 找到右子树最小的值之后, 修改当前节点 root 的值为右子树最小的值
root->val = minNodeOfRight->val;

// 同时, 记得删除掉 root 的右子树最小的值之
// 删除操作就是以 root 作为根节点, key 为右子树最小的值进行删除
root->right = deleteNode(root->right, minNodeOfRight->val);

// 3、如果 root 的节点值小于需要删除的值, 那么就在 root 的右子树中去查找
}
else if (root->val < key)
{
    // 在 root 的右子树中去查找并删除 key
    root->right = deleteNode(root->right, key);

    // 4、如果 root 的节点值大于需要删除的值, 那么就在 root 的左子树中去查找
}
else if (root->val > key)
{
    // 在 root 的左子树中去查找并删除 key
    root->left = deleteNode(root->left, key);
}

// 最后返回需要已经删除了 key 的二叉树的根节点
return root;
}

```

```

// 通过 findMinNode , 可以找到二叉搜索树中最小的元素
TreeNode *findMinNode(TreeNode *node)
{

    // 由于二叉搜索树, 左子树所有元素的值都小于根节点的值
    // 所以可以不断的查找, 直到为叶子节点, 那么就找到了
    while (node->left != NULL)
    {
        // 不断的去查找当前节点的左子树
        node = node->left;
    }

    // 返回当前二叉搜索树中最小的元素
    return node;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者: 程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

删除二叉搜索树中的节点 (LeetCode 450) :<https://leetcode-cn.com/problems/delete-node-in-a-bst/>

class Solution:

def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:

1、如果 root 为空, 那么直接返回空

```

if not root :
    return None

```

2、如果 root 的节点值等于需要删除的值, 那么需要根据以下几种情况进行处理

```

if root.val == key :

```

情况 1: 当前节点的左子树为空, 那么当前节点 root 由 root 的右子树占位就行

比如 key 为 7

```

#      6
#     / \
#    2   7
#         \
#          8

```

```

#      6
#     / \
#    2   7
#         \
#          8

```

```

#      6
#     / \
#    2   7
#         \
#          8

```

```

#      6
#     / \
#    2   7
#         \
#          8

```

```

#      6
#     / \
#    2   7
#         \
#          8

```

直接将 8 作为根节点的二叉树挪到 7 的位置

```

#      6
#     / \
#    2   8

```

```

#      6
#     / \
#    2   8

```

```

#      6
#     / \
#    2   8

```

```

if not root.left:
    return root.right

```

```

# 情况 2: 当前节点的右子树为空, 那么当前节点 root 由 root 的左子树占位就行
# 比如 key 为 2
#       6
#      / \
#     2   7
#    /
#   1
# 直接将 1 作为根节点的二叉树挪到 2 的位置
#       6
#      / \
#     1   7
if not root.right :
    return root.left

# 情况 3: 被删除节点既有左子树, 又有右子树
# 比如 key 为 2
#           5
#          / \
#         2   6
#        / \   \
#       1  4   7
#          /
#         3
# 需要找到右子树最小的值, 或者左子树中最大的值
# 这里我们去找右子树最小的值, 为 3
minNodeOfRight = self.findMinNode(root.right)

# 找到右子树最小的值之后, 修改当前节点 root 的值为右子树最小的值
root.val = minNodeOfRight.val

# 同时, 记得删除掉 root 的右子树最小的值之
# 删除操作就是以 root 的右子树作为根节点, key 为右子树最小的值进行删除
root.right = self.deleteNode(root.right, minNodeOfRight.val)

# 3、如果 root 的节点值小于需要删除的值, 那么就在 root 的右子树中去查找
elif root.val < key :
    # 在 root 的右子树中去查找并删除 key
    root.right = self.deleteNode(root.right, key)

# 4、如果 root 的节点值大于需要删除的值, 那么就在 root 的左子树中去查找
elif root.val > key :
    # 在 root 的左子树中去查找并删除 key
    root.left = self.deleteNode(root.left, key)

# 最后返回需要已经删除了 key 的二叉树的根节点
return root

```

```

# 通过 findMinNode , 可以找到二叉搜索树中最小的元素
def findMinNode(self , node : TreeNode) -> TreeNode :

    # 由于二叉搜索树, 左子树所有元素的值都小于根节点的值
    # 所以可以不断的查找, 直到为叶子节点, 那么就找到了
    while node.left :
        # 不断的去查找当前节点的左子树
        node = node.left

    # 返回当前二叉搜索树中最小的元素
    return node

```

LeetCode 538、把二叉搜索树转换为累加树

一、题目描述

给出二叉搜索树的根节点，该树的节点值各不相同，请你将其转换为累加树（Greater Sum Tree），使每个节点 node 的新值等于原树中大于或等于 node.val 的值之和。

二、题目解析

三、参考代码

```

// 作者: 程序员吴师兄
// 网站: https://www.algomooc.com
class Solution {

    // 定义一个变量, 用来记录已经访问的所有节点值的总和
    int sum = 0;

    public TreeNode convertBST(TreeNode root) {

        // 1、从根节点开始, 修改二叉树上的每一个节点的值
        modify(root);

        // 返回修改后的二叉树的根节点
        return root;
    }

    // 按照右根左的顺序, 遍历二叉树上的每一个节点, 将每个节点的值累加到 sum 上面
    // 同时把 sum 累加到当前节点上
    private void modify(TreeNode node ){

        // 如果当前节点为空, 那么就不需要计算了
        if(node == null) return ;

        // 2、去修改当前节点的右子树上的节点
        // 修改的过程中, 会不断的累加当前节点右子树上所有的节点值之和
    }

```

```
        modify(node.right);

        // 3、把当前节点右子树上所有的节点值之和在加上当前节点的值更新到 sum 上
        sum += node.val;

        // 4、 修改当前节点的值 为 sum
        node.val = sum;

        // 5、去修改当前节点的左子树上的节点
        // 修改的过程中，会不断的累加当前节点左子树上所有的节点值之和
        modify(node.left);
    }
}
```

LeetCode 98、验证二叉搜索树

一、题目描述

给你一个二叉树的根节点 `root`，判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下：

- 节点的左子树只包含 **小于** 当前节点的数。
- 节点的右子树只包含 **大于** 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

示例 1：

img

```
输入：root = [2,1,3]
输出：true
```

示例 2：

img

```
输入：root = [5,1,4,null,null,3,6]
输出：false
解释：根节点的值是 5，但是右子节点的值是 4。
```

提示：

- 树中节点数目范围在 `[1, 104]` 内
- `-231 <= Node.val <= 231 - 1`

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean check(TreeNode node, long rangeLeft, long rangeRight) {

        if (node == null) return true;

        if ( node.val < rangeLeft || node.val > rangeRight ) return false;

        long x = node.val;

        return check(node.left, rangeLeft, x - 1 ) && check(node.right, x + 1,
rangeRight);

    }
}
```

2、**C++** 代码

```
class Solution {
public:
    bool helper(TreeNode* root, long long lower, long long upper) {
        if (root == nullptr) {
            return true;
        }
        if (root -> val <= lower || root -> val >= upper) {
            return false;
        }
        return helper(root -> left, lower, root -> val) && helper(root -> right, root -
> val, upper);
    }
    bool isValidBST(TreeNode* root) {
        return helper(root, LONG_MIN, LONG_MAX);
    }
};
```

3、Python 代码

```
class solution:
    def isValidBST(self, root: TreeNode) -> bool:
        def helper(node, lower = float('-inf'), upper = float('inf')) -> bool:
            if not node:
                return True

            val = node.val
            if val <= lower or val >= upper:
                return False

            if not helper(node.right, val, upper):
                return False
            if not helper(node.left, lower, val):
                return False
            return True

        return helper(root)
```

第四十天

LeetCode 1603、设计停车系统

题目描述

请你给一个停车场设计一个停车系统。停车场总共有三种不同大小的车位：大，中和小，每种尺寸分别有固定数目的车位。

请你实现 `ParkingSystem` 类：

- `ParkingSystem(int big, int medium, int small)` 初始化 `ParkingSystem` 类，三个参数分别对应每种停车位的数目。
- `bool addCar(int carType)` 检查是否有 `carType` 对应的停车位。 `carType` 有三种类型：大，中，小，分别用数字 `1`，`2` 和 `3` 表示。一辆车只能停在 `carType` 对应尺寸的停车位中。如果没有空车位，请返回 `false`，否则将该车停入车位并返回 `true`。

示例 1：

输入：

```
["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]  
[[1, 1, 0], [1], [2], [3], [1]]
```

输出：

```
[null, true, true, false, false]
```

解释：

```
ParkingSystem parkingSystem = new ParkingSystem(1, 1, 0);  
parkingSystem.addCar(1); // 返回 true , 因为有 1 个空的大车位  
parkingSystem.addCar(2); // 返回 true , 因为有 1 个空的中车位  
parkingSystem.addCar(3); // 返回 false , 因为没有空的小车位  
parkingSystem.addCar(1); // 返回 false , 因为没有空的大车位, 唯一一个大车位已经被占据了
```

提示：

- `0 <= big, medium, small <= 1000`
- `carType` 取值为 `1`, `2` 或 `3`
- 最多会调用 `addCar` 函数 `1000` 次

解题思路

- 1、用三个变量分别维护各类车型的剩余空位的数目，每次 `addCar(carType)` 的时候就把对应类型的停车空位数目 `-1`。
- 2、如果当前车型的剩余空位数目 `== 0`，那么说明无法再停车了，就返回 `False`；否则说明可以停车，就返回 `True`。
- 3、`carType` 的取值为 `1,2,3`，因此可以使用数组保存停车位的剩余空位数目，根据下标来获取对应车型的停车位剩余数目。

参考代码

```
class ParkingSystem(object):  
  
    def __init__(self, big, medium, small):  
        # 数组的第 0 位置初始化为 0，起到占位符的作用  
        # 这样在 addCar 方法里面  
        # park[1] 就是 big 的数量  
        # park[2] 就是 medium 的数量  
        # park[3] 就是 small 的数量  
        self.park = [0, big, medium, small]  
  
    def addCar(self, carType):  
        # 当前类型没有车位了，返回 False  
        if self.park[carType] == 0:  
            return False  
  
        # 否则当前类型车位数量减少一个  
        self.park[carType] -= 1
```

```
return True
```

剑指 Offer 29. 顺时针打印矩阵

一、题目描述

输入一个矩阵，按照从外向里以顺时针的顺序依次打印出每一个数字。

示例 1:

输入：matrix = [[1,2,3],[4,5,6],[7,8,9]] 输出：[1,2,3,6,9,8,7,4,5]

示例 2:

输入：matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]] 输出：[1,2,3,4,8,12,11,10,9,5,6,7]

限制:

- `0 <= matrix.length <= 100`
- `0 <= matrix[i].length <= 100`

二、题目解析

对于一个二维矩阵来说，它包含了如下的边界与打印顺序：

- 1、顶层，我们可以定义为 top，在顶层是按照从左到右的顺序进行打印
- 2、右列，我们可以定义为 right，在右列是按照从上到小的顺序进行打印
- 3、底层，我们可以定义为 bottom，在顶层是按照从右到左的顺序进行打印
- 2、左列，我们可以定义为 left，在左列是按照从下到上的顺序进行打印

在打印的过程中，矩阵的可打印区间在不断的发生变化：

- 每当把从左到右把一行打印完毕之后，整个矩阵就在顶部少了一层，即 top 位置向下挪了一层
- 每当把从上到下把一列打印完毕之后，整个矩阵就在右部少了一列，即 right 位置向左挪了一列
- 每当把从右到左把一行打印完毕之后，整个矩阵就在底部少了一层，即 bottom 位置向上挪了一层
- 每当把从下到上把一列打印完毕之后，整个矩阵就在左部少了一列，即 left 位置向右挪了一列

每当 top、right、bottom、left 发生挪到之后，需要判断它们挪动之后的区间是否还存在。

- 1、如果还存在，那么就继续按照 top、right、bottom、left 的顺序进行打印
- 2、如果不存在了，那么说明矩阵中的所有元素打印完毕

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public int[] spiralOrder(int[][] matrix) {
```

```

// 特殊情况，边界处理，比如 matrix = [], 无法获取 matrix[0]
if (matrix.length == 0) {
    return new int[0];
}

// 设置一个一维数组 res 用来记录二维数组 matrix 中的顺时针打印的结果
int[] res = new int[matrix.length * matrix[0].length];

// 在打印的过程中，不断的缩小着打印的区间
// 每当把从左到右把一行打印完毕之后，整个矩阵就在顶部少了一层，后续打印不需要再去处理它们
// 每当把从上到下把一列打印完毕之后，整个矩阵就在右部少了一列，后续打印不需要再去处理它们
// 每当把从右到左把一行打印完毕之后，整个矩阵就在底部少了一层，后续打印不需要再去处理它们
// 每当把从下到上把一列打印完毕之后，整个矩阵就在左部少了一列，后续打印不需要再去处理它们
// 因此，设置四个变量，用来记录打印的区间变化

// top 表示顶部所在的层数位置，一开始在第 0 层
int top = 0 ;

// bottom 表示底部所在的层数位置，一开始在第 matrix.length - 1 层
int bottom = matrix.length - 1 ;

// left 表示左部所在的列数位置，一开始在第 0 列
int left = 0 ;

// right 表示右部所在的列数位置，一开始在第 matrix[0].length - 1 列
int right = matrix[0].length - 1;

// 顺时针打印矩阵过程中，填充 res 数组，从索引位置 0 的地方开始填充
int index = 0;

// 使用一个 while 循环进行打印，只要打印区间中还有值就一直打印
// 直到出现边界越界，即打印区间不存在元素了，跳出循环
while (true) {

    // 1、从左到右，打印这一行
    // 此时，边界从 left 到 right
    for (int i = left; i <= right; i++) {

        // 将当前元素填充到 res 中
        // 此时，一直都是在 top 这一层
        res[index] = matrix[top][i];

        // index 的元素填充完毕之后，开始填充下一个元素
        index++;
    }

    // 经过上面这个循环之后，此时，顶部这一层的所有元素已经打印完毕
    // 整个打印区间需要删除这一行了，因此，将 top 的层数向下挪

```

```

top += 1;

// 如果此时发现顶部位置越过了底部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( top > bottom ) {
    break;
}

// 2、从上到下，打印这一列
// 此时，边界从 top 到 bottom
for (int i = top; i <= bottom; i++) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 right 这一列
    res[index] = matrix[i][right];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;

}

// 经过上面这个循环之后，此时，右部这一列的所有元素已经打印完毕
// 整个打印区间需要删除这一列了，因此，将 right 的层数向左挪
right -= 1;

// 如果此时发现右部位置越过了左部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( right < left ) {
    break;
}

// 3、从右到左，打印这一行
// 此时，边界从 right 到 left
for (int i = right; i >= left; i--) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 bottom 这一层
    res[index] = matrix[bottom][i];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;

}

// 经过上面这个循环之后，此时，底部这一层的所有元素已经打印完毕
// 整个打印区间需要删除这一行了，因此，将 bottom 的层数向上挪
bottom -= 1;

// 如果此时发现顶部位置越过了底部位置，说明整个打印区间已经没有元素了

```

```

// 跳出循环即可
if ( bottom < top ) {
    break;
}

// 4、从下到上，打印这一列
// 此时，边界从 bottom 到 top
for (int i = bottom; i >= top; i--) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 left 这一列
    res[index] = matrix[i][left];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;
}

// 经过上面这个循环之后，此时，左部这一列的所有元素已经打印完毕
// 整个打印区间需要删除这一列了，因此，将 left 的层数向右挪
left += 1;

// 如果此时发现右部位置越过了左部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( left > right ) {
    break;
}

}

// 最后，返回结果即可
return res;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {

        // 特殊情况，边界处理，比如 matrix = [], 无法获取 matrix[0]
        if (matrix.size() == 0) {
            return {};
        }
    }
}

```

```
// 设置一个一维数组 res 用来记录二维数组 matrix 中的顺时针打印的结果

vector<int> res(matrix.size() * matrix[0].size());

// 在打印的过程中，不断的缩小着打印的区间
// 每当把从左到右把一行打印完毕之后，整个矩阵就在顶部少了一层，后续打印不需要再去处理它们
// 每当把从上到下把一列打印完毕之后，整个矩阵就在右部少了一列，后续打印不需要再去处理它们
// 每当把从右到左把一行打印完毕之后，整个矩阵就在底部少了一层，后续打印不需要再去处理它们
// 每当把从下到上把一列打印完毕之后，整个矩阵就在左部少了一列，后续打印不需要再去处理它们
// 因此，设置四个变量，用来记录打印的区间变化

// top 表示顶部所在的层数位置，一开始在第 0 层
int top = 0 ;

// bottom 表示底部所在的层数位置，一开始在第 matrix.length - 1 层
int bottom = matrix.size() - 1 ;

// left 表示左部所在的列数位置，一开始在第 0 列
int left = 0 ;

// right 表示右部所在的列数位置，一开始在第 matrix[0].length - 1 列
int right = matrix[0].size() - 1;

// 顺时针打印矩阵过程中，填充 res 数组，从索引位置 0 的地方开始填充
int index = 0;

// 使用一个 while 循环进行打印，只要打印区间中还有值就一直打印
// 直到出现边界越界，即打印区间不存在元素了，跳出循环
while (true) {

    // 1、从左到右，打印这一行
    // 此时，边界从 left 到 right
    for (int i = left; i <= right; i++) {

        // 将当前元素填充到 res 中
        // 此时，一直都是在 top 这一层
        res[index] = matrix[top][i];

        // index 的元素填充完毕之后，开始填充下一个元素
        index++;

    }

    // 经过上面这个循环之后，此时，顶部这一层的所有元素已经打印完毕
    // 整个打印区间需要删除这一行了，因此，将 top 的层数向下挪
    top += 1;

    // 如果此时发现顶部位置越过了底部位置，说明整个打印区间已经没有元素了
    // 跳出循环即可
```

```

if ( top > bottom ) {
    break;
}

// 2、从上到下，打印这一列
// 此时，边界从 top 到 bottom
for (int i = top; i <= bottom; i++) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 right 这一列
    res[index] = matrix[i][right];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;

}

// 经过上面这个循环之后，此时，右部这一列的所有元素已经打印完毕
// 整个打印区间需要删除这一列了，因此，将 right 的层数向左挪
right -= 1;

// 如果此时发现右部位置越过了左部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( right < left ) {
    break;
}

// 3、从右到左，打印这一行
// 此时，边界从 right 到 left
for (int i = right; i >= left; i--) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 bottom 这一层
    res[index] = matrix[bottom][i];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;

}

// 经过上面这个循环之后，此时，底部这一层的所有元素已经打印完毕
// 整个打印区间需要删除这一行了，因此，将 bottom 的层数向上挪
bottom -= 1;

// 如果此时发现顶部位置越过了底部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( bottom < top ) {
    break;
}

```

```

// 4、从下到上，打印这一列
// 此时，边界从 bottom 到 top
for (int i = bottom; i >= top; i--) {

    // 将当前元素填充到 res 中
    // 此时，一直都是在 left 这一列
    res[index] = matrix[i][left];

    // index 的元素填充完毕之后，开始填充下一个元素
    index++;
}

// 经过上面这个循环之后，此时，左部这一列的所有元素已经打印完毕
// 整个打印区间需要删除这一列了，因此，将 left 的层数向右挪
left += 1;

// 如果此时发现右部位置越过了左部位置，说明整个打印区间已经没有元素了
// 跳出循环即可
if ( left > right ) {
    break;
}

}

// 最后，返回结果即可
return res;
}
};

```

第四十一天

LeetCode 297、二叉树的序列化与反序列化

一、题目描述

请实现两个函数，分别用来序列化和反序列化二叉树。

你需要设计一个算法来实现二叉树的序列化与反序列化。这里不限定你的序列 / 反序列化算法执行逻辑，你只需要保证一个二叉树可以被序列化为一个字符串并且将这个字符串反序列化为原始的树结构。

示例 1：

```

输入：root = [1,2,3,null,null,4,5]
输出：[1,2,3,null,null,4,5]

```


示例 2:

输入: root = []

输出: []

提示:

- 树中节点数范围是 $[0, 10^4]$
- $-1000 \leq \text{Node.val} \leq 1000$

二、题目解析

很多同学认为这道题目难，要么是因为题目一时没看明白，要么是其他人的题解分析写的太复杂、太深奥，自己一看误以为就是一道高难度的题目。

实际上这道题目不难!

一开始，我们先来解释序列化和反序列化，这两个词的概念程序员应该不会陌生，无论你是做哪方面的开发，基本上都用到过这两个操作，简单来说就是把对象转换成另外一种方便存储或者传输的形式状态，使用的时候再把该状态逆转回来。

这道题目的意思就是要我们把二叉树通过某个手段变成一个字符串的形式，同时字符串能通过另外一种手段复原为之前的二叉树，也就是说**二叉树和字符串是一一对应的关系!**

理解清楚题意实际上思路也就有了。

首先，思考二叉树有哪些遍历方式。

前序、中序、后序、层序。

通过这些方式都可以把二叉树转换为数组的形式，但都存在一个问题，不同的二叉树使用不同的遍历方式会生成一个相同的数组。

所以，接下来我们的思考方式就是**采取哪种方式+某种手段可以生成一个特点的数组。**

直接说答案：**把 null 节点放进去，所有的二叉树都变成**完全二叉树**!**

序列化过程**:**

- 使用广度优先搜索按照层次的顺序从上到下遍历所有的节点
- 如果当前节点不为空，以字符串的形式打印出当前节点的值，同时把它的左右子节点加入到队列中
- 如果当前节点为空，直接打印字符串 `null`

反序列化过程:

- 先构造出 `root` 入队，即字符串的第一个元素为 `root`
- 队列元素依次出队，构建当前元素的左右子节点，记为 `node`
- 遍历字符串，如果元素不为空，放入到队列中；如果元素为 `null`，设置为 `node` 的左右子节点
- 队列中的元素依次出队，设置为 `node` 的左右子节点

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉树的序列化与反序列化 ( LeetCode 297 ) :https://leetcode-cn.com/problems/serialize-and-deserialize-binary-tree/
public class Codec {
    public String serialize(TreeNode root) {
        // 边界条件处理
        if(root == null) return "[]";

        // 题目要求序列化后的字符串开头为 [ ,所以初始化 res 为 [
        StringBuilder res = new StringBuilder("[");

        // 生成一个队列，用来存储二叉树中的元素
        Queue<TreeNode> queue = new LinkedList<>();

        // 首先让二叉树的根节点入队
        queue.add(root);

        // 遍历队列，直到队列为空，说明访问了二叉树所有的元素
        while(!queue.isEmpty()) {

            // 队列的队首元素出队
            TreeNode node = queue.poll();

            // 如果出队的元素不为空，把它加入到序列化字符串 res 中，并以逗号结尾
            if(node != null) {
                // 加入到 res 中，以逗号结尾
                res.append(node.val + ",");
                // 在队列中添加当前元素的左节点
                queue.add(node.left);
                // 在队列中添加当前元素的右节点
                queue.add(node.right);
            }else {
                // 如果出队的元素为空，则把 null 加入到序列化字符串 res 中，并以逗号结尾
                // 这样无论输入的是什么样的二叉树，最终都变成了一颗完全二叉树
                res.append("null,");
            }
        }

        // 由于上述的遍历中的每次操作都以逗号结尾，所以此时先需要删除掉最后一个逗号
        // [ 1 , 2 , 3 , ----> [ 1 , 2 , 3
        res.deleteCharAt(res.length() - 1);
    }
}
```

```

// 序列化字符串 res 以 ] 结尾
// [ 1 , 2 , 3 ----> [ 1 , 2 , 3 ]
res.append("]");
// 返回序列化的结果
return res.toString();
}

```

```

public TreeNode deserialize(String data) {
    // 边界情况判断
    if(data.equals("")) return null;

    // 把一个字符串分割成字符串数组，同时删除了 [ 和 ]
    // [1,2,3] --> 1 2 3
    String[] vals = data.substring(1, data.length() - 1).split(",");

    // 二叉树的根节点为字符串的第一个元素，以此构建二叉树
    TreeNode root = new TreeNode(Integer.parseInt(vals[0]));

    // 辅助队列
    Queue<TreeNode> queue = new LinkedList<>();

    // 将根节点入队
    queue.add(root);

    // 设置一个索引，代表二叉树中的节点在字符串中的位置
    // 由于 root 已经入队，所以索引从 1 开始
    int index = 1;

```

```

// 遍历队列，直到队列为空，说明访问了数组中的所有元素
while(!queue.isEmpty()) {
    // 入队
    TreeNode node = queue.poll();
    // 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的左子节点
    if(!vals[index].equals("null")) {
        // 设置为当前出队节点的左子节点
        node.left = new TreeNode(Integer.parseInt(vals[index]));
        // 同时，把当前索引的节点入队
        queue.add(node.left);
    }

```

一个索引元素

```

index++;

```

```

// 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的右子节点
if(!vals[index].equals("null")) {
    // 设置为当前出队节点的右子节点
    node.right = new TreeNode(Integer.parseInt(vals[index]));
    // 同时，把当前索引的节点入队

```

```

        queue.add(node.right);
    }

    // 无论字符串数组中当前索引元素是否为 null，这个元素都已经被操作了，索引向前移动，判断下一个索引元素
    index++;
}

// 返回二叉树的根节点
return root;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 序列化和反序列化二叉搜索树（ LeetCode 449 ）:https://leetcode-cn.com/problems/serialize-and-deserialize-bst/
class Codec
{
public:
    string serialize(TreeNode *root)
    {
        // 边界条件处理
        if (root == NULL)
            return "[]";

        // 题目要求序列化后的字符串开头为 [ ,所以初始化 res 为 [
        string res = "[";

        // 生成一个队列，用来存储二叉树中的元素
        queue<TreeNode *> myqueue;

        // 首先让二叉树的根节点入队
        myqueue.push(root);

        // 遍历队列，直到队列为空，说明访问了二叉树所有的元素
        while (!myqueue.empty())
        {
            // 队列的队首元素出队
            TreeNode *node = myqueue.front();
            myqueue.pop();

            // 如果出队的元素不为空，把它加入到序列化字符串 res 中，并以逗号结尾
            if (node != NULL)

```

```

{
    // 加入到 res 中，以逗号结尾
    res.append(to_string(node->val) + ",");
    // 在队列中添加当前元素的左节点
    myqueue.push(node->left);
    // 在队列中添加当前元素的右节点
    myqueue.push(node->right);
}
else
{
    // 如果出队的元素为空，则把 null 加入到序列化字符串 res 中，并以逗号结尾
    // 这样无论输入的是什么样的二叉树，最终都变成了一颗完全二叉树
    res.append("null,");
}
}

// 由于上述的遍历中的每次操作都以逗号结尾，所以此时先需要删除掉最后一个逗号
// [ 1 , 2 , 3 ,      ----> [ 1 , 2 , 3
res = res.substr(0, res.size() - 1);

// 序列化字符串 res 以 ] 结尾
// [ 1 , 2 , 3      ----> [ 1 , 2 , 3 ]
res.append("]");
// 返回序列化的结果
return res;
}

```

```

TreeNode *deserialize(string data)
{

```

```

    // 边界情况判断
    if (data == "[]")
        return NULL;

```

```

    // 把一个字符串分割成字符串数组，同时删除了 [ 和 ]
    // [1,2,3] --> 1 2 3

```

```

    //手动过滤字符 过滤范围为[1, data.size()-2],因为data[0] = '['data[data.size()-1] =

```

```

    ']'

```

```

    //所以只需要过滤范围内的','即可

```

```

    vector<string> vals;
    for (int i = 1; i < data.size() - 1; i++)
    {
        string s = "";
        while (i < data.size() - 1 && data[i] != ',')
        {
            s = s + data[i];
            i++;
        }
        if (s != "")
            vals.push_back(s);
    }

```

```

}

// 二叉树的根节点为字符串的第一个元素，以此构建二叉树
TreeNode *root = new TreeNode(stoi(vals[0]));

// 辅助队列
queue<TreeNode *> myqueue;

// 将根节点入队
myqueue.push(root);

// 设置一个索引，代表二叉树中的节点在字符串中的位置
// 由于 root 已经入队，所以索引从 1 开始
int index = 1;

// 遍历队列，直到队列为空，说明访问了数组中的所有元素
while (!myqueue.empty() && index < vals.size())
{
    // 入队
    TreeNode *node = myqueue.front();
    myqueue.pop();
    // 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的左子节点
    if (index < vals.size() && vals[index] != "null")
    {
        // 设置为当前出队节点的左子节点
        //stoi: stringToInt std中定义的字符串类型转化成int类型的方法
        node->left = new TreeNode(stoi(vals[index]));

        // 同时，把当前索引的节点入队
        myqueue.push(node->left);
    }

    // 无论字符串数组中当前索引元素是否为 null，这个元素都已经被操作了，索引向前移动，判断下
    // 一个索引元素
    index++;

    // 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的右子节点
    if (index < vals.size() && vals[index] != "null")
    {
        // 设置为当前出队节点的右子节点
        //stoi: stringToInt std中定义的字符串类型转化成int类型的方法
        node->right = new TreeNode(stoi(vals[index]));
        // 同时，把当前索引的节点入队
        myqueue.push(node->right);

        // 无论字符串数组中当前索引元素是否为 null，这个元素都已经被操作了，索引向前移动，判
        // 断下一个索引元素
    }
    index++;
}

```

```

    }
    // 返回二叉树的根节点
    return root;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

二叉树的序列化与反序列化 (LeetCode 297) :<https://leetcode-cn.com/problems/serialize-and-deserialize-binary-tree/>

class Codec:

```

def serialize(self, root):
    # 边界条件处理
    if not root :
        return ""

    res = []

    # 生成一个队列，用来存储二叉树中的元素
    queue = collections.deque()

    # 首先让二叉树的根节点入队
    queue.append(root)

    # 遍历队列，直到队列为空，说明访问了二叉树所有的元素
    while queue:

        # 队列的队首元素出队
        node = queue.popleft()

        # 如果出队的元素不为空，把它加入到序列化字符串 res 中，并以逗号结尾
        if node :
            # 加入到 res 中，以逗号结尾
            res.append(str(node.val))
            # 在队列中添加当前元素的左节点
            queue.append(node.left)
            # 在队列中添加当前元素的右节点
            queue.append(node.right)
        else :
            # 如果出队的元素为空，则把 null 加入到序列化字符串 res 中，并以逗号结尾
            # 这样无论输入的是什么样的二叉树，最终都变成了一颗完全二叉树
            res.append("None")

    return '[' + ','.join(res) + ']'

```

```

def deserialize(self, data):
    # 边界情况判断
    if not data:
        return []

    # 把一个字符串分割成字符串数组，同时删除了 [ 和 ]
    # [1,2,3] --> 1 2 3
    vals = data[1:-1].split(',')

    # 二叉树的根节点为字符串的第一个元素，以此构建二叉树
    root = TreeNode(int(vals[0]))

    # 辅助队列
    queue = collections.deque()

    # 将根节点入队
    queue.append(root)

    # 设置一个索引，代表二叉树中的节点在字符串中的位置
    # 由于 root 已经入队，所以索引从 1 开始
    index = 1

    # 遍历队列，直到队列为空，说明访问了数组中的所有元素
    while queue:
        # 入队
        node = queue.popleft()
        # 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的左子节点
        if vals[index] != 'None':
            # 设置为当前出队节点的左子节点
            node.left = TreeNode(int(vals[index]))
            # 同时，把当前索引的节点入队
            queue.append(node.left)

        # 无论字符串数组中当前索引元素是否为 null，这个元素都已经被操作了，索引向前移动，判断下一个索引元素
        index += 1

        # 如果字符串数组中当前索引元素不为 null，则设置为当前出队节点的右子节点
        if vals[index] != 'None':
            # 设置为当前出队节点的右子节点
            node.right = TreeNode(int(vals[index]))
            # 同时，把当前索引的节点入队
            queue.append(node.right)

        # 无论字符串数组中当前索引元素是否为 null，这个元素都已经被操作了，索引向前移动，判断下一个索引元素

```



```
index += 1

return root
```

LeetCode 530、二叉搜索树的最小绝对差

一、题目描述

给你一个二叉搜索树的根节点 `root`，返回 树中任意两不同节点值之间的最小差值。

差值是一个正数，其数值等于两值之差的绝对值。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉搜索树的最小绝对差（ LeetCode 530 ）：https://leetcode-cn.com/problems/minimum-absolute-difference-in-bst/submissions/
class Solution {
    public int getMinimumDifference(TreeNode root) {

        // 设置一个变量用来记录两不同节点值之间的差值
        int result = Integer.MAX_VALUE;

        // 设置一个栈，用来保存路径
        Stack<TreeNode> stack = new Stack<>();

        // 设置一个节点，一开始指向根节点
        TreeNode curNode = root;

        // 设置一个节点 preNode，用来记录当前遍历节点的上一个节点
        TreeNode preNode = null;

        // 设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了
        // 每个节点都有 左、右、上 这三种状态
        // 按照 左 --> 右 --> 上 的顺序观察每个节点

        // 左代表该节点的左右孩子节点都没有遍历
        int nodeLeft = 100;

        // 右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历
        int nodeRight = 200;
```

```

// 上代表左右孩子节点都已经遍历，需要返回到它的父节点
int nodeUp = 300;

// 每个节点的初始化状态都是从 左 开始
int nodeState = nodeLeft;

// 对二叉树进行遍历
while(curNode != null){

    // 位置 ①

    // 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if(nodeState == nodeLeft){

        // 如果当前节点有左子树
        if(curNode.left != null){

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.push(curNode);

            // 开始观察当前节点的左孩子节点，代码来到了位置 ①
            curNode = curNode.left;
        }else{

            // 如果当前节点没有左子树，切换当前节点的状态为【右】
            // 代码来到了位置 ②
            nodeState = nodeRight;
        }

    }else if(nodeState == nodeRight){ // 如果当前节点的状态是【右】，说明该节点的左孩子
        节点已经遍历，右孩子节点还没有遍历

        // 二叉树中序遍历的处理结果
        if(preNode != null){
            result = Math.min(result, curNode.val - preNode.val);
        }

        preNode = curNode;

        // 如果当前节点有右子树
        if(curNode.right != null){

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.push(curNode);

            // 开始观察当前节点的右孩子节点
            curNode = curNode.right;
        }
    }
}

```

```
// 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
nodeState = nodeLeft;
```

```
// 执行完上面三行代码，代码来到了位置 ①
```

```
}else{
    // 如果当前节点没有右子树，切换当前节点的状态为【上】
    // 代码来到了位置 ①
    nodeState = nodeUp;
}
```

```
}else if(nodeState == nodeUp){ // 如果当前节点的状态是【上】，说明左右孩子节点都已经遍历，需要返回到它的父节点
```

```
// 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
// 首先构建一个空的节点
TreeNode parent = null;
```

```
// 如果栈中有元素
if(!stack.isEmpty()){
```

```
    // 那么，栈顶元素就是当前节点的父节点
    parent = stack.pop();
```

```
    // 判断一下父节点的左节点是否为当前节点
    // 比如这颗二叉树
```

```
    //          1
    //         /  \
    //        2    3
    //       / \   \
    //      4  5   6
```

```
    // 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换
```

状态为【右】

```
    // 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切
```

换状态

```
    // 如果父节点的左节点为当前节点
    if(parent.left == curNode){
        // 切换当前节点的状态为【右】
        nodeState = nodeRight;
    }
```

```
}
// 开始观察当前节点的父节点
// 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
// 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
curNode = parent;
```

```
}
}
```

```

        // 返回结果
        return result;
    }
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 二叉搜索树的最小绝对差 ( LeetCode 530 ) : https://leetcode-cn.com/problems/minimum-absolute-difference-in-bst/submissions/
class Solution
{
public:
    int getMinimumDifference(TreeNode* root)
    {

        // 设置一个变量用来记录两不同节点值之间的差值
        int result = INT_MAX;

        // 设置一个栈，用来保存路径
        stack<TreeNode*> st;

        // 设置一个节点，一开始指向根节点
        TreeNode* curNode = root;

        // 设置一个节点 preNode ，用来记录当前遍历节点的上一个节点
        TreeNode* preNode = NULL;

        // 设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了
        // 每个节点都有 左、右、上 这三种状态
        // 按照 左 --> 右 --> 上 的顺序观察每个节点

        // 左代表该节点的左右孩子节点都没有遍历
        int nodeLeft = 100;

        // 右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历
        int nodeRight = 200;

        // 上代表左右孩子节点都已经遍历，需要返回到它的父节点
        int nodeUp = 300;

        // 每个节点的初始化状态都是从 左 开始
        int nodeState = nodeLeft;

        // 对二叉树进行遍历
        while (curNode != NULL)

```

```

{

    // 位置 ①

    // 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if (nodeState == nodeLeft)
    {

        // 如果当前节点有左子树
        if (curNode->left != NULL)
        {

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            st.push(curNode);

            // 开始观察当前节点的左孩子节点，代码来到了位置 ①
            curNode = curNode->left;
        }
        else
        {

            // 如果当前节点没有左子树，切换当前节点的状态为【右】
            // 代码来到了位置 ①
            nodeState = nodeRight;
        }
    }
    else if (nodeState == nodeRight)
    { // 如果当前节点的状态是【右】，说明该节点的左孩子节点已经遍历，右孩子节点还没有遍历

        // 二叉树中序遍历的处理结果
        if (preNode != NULL)
        {
            result = min(result, curNode->val - preNode->val);
        }

        preNode = curNode;

        // 如果当前节点有右子树
        if (curNode->right != NULL)
        {

            // 先把当前节点加入到栈中，用来记录节点移动的路径
            st.push(curNode);

            // 开始观察当前节点的右孩子节点
            curNode = curNode->right;

            // 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
            nodeState = nodeLeft;
        }
    }
}

```

```

        // 执行完上面三行代码，代码来到了位置 ①
    }
    else
    {
        // 如果当前节点没有右子树，切换当前节点的状态为【上】
        // 代码来到了位置 ①
        nodeState = nodeUp;
    }
}
else if (nodeState == nodeUp)
{ // 如果当前节点的状态是【上】，说明左右孩子节点都已经遍历，需要返回到它的父节点

    // 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
    // 首先构建一个空的节点
    TreeNode* parent = NULL;

    // 如果栈中有元素
    if (!st.empty())
    {

        // 那么，栈顶元素就是当前节点的父节点
        parent = st.top();
        st.pop();

        // 判断一下父节点的左节点是否为当前节点
        // 比如这颗二叉树
        //           1
        //          / \
        //         2  3
        //        / \  \
        //       4  5  6
        // 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换
        // 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切

        // 如果父节点的左节点为当前节点
        if (parent->left == curNode)
        {
            // 切换当前节点的状态为【右】
            nodeState = nodeRight;
        }
    }
    // 开始观察当前节点的父节点
    // 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
    // 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
    curNode = parent;
}

```

状态为【右】

换状态

```

    }
    // 返回结果
    return result;
}
};

```

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

代码有看不懂的地方一定要私聊咨询吴师兄呀

二叉搜索树的最小绝对差 (LeetCode 530) : <https://leetcode-cn.com/problems/minimum-absolute-difference-in-bst/submissions/>

class Solution:

def getMinimumDifference(self, root: TreeNode) -> int:

设置一个数组用来保存二叉树前序遍历的结果

result = 100000

设置一个栈，用来保存路径

stack = list()

设置一个节点，一开始指向根节点

curNode = root

设置一个节点 preNode ，用来记录当前遍历节点的上一个节点

preNode = None

设置三种状态，方便表示当前遍历当前节点的时候进行到哪一步了

每个节点都有 左、右、上 这三种状态

按照 左 --> 右 --> 上 的顺序观察每个节点

左代表该节点的左右孩子节点都没有遍历

nodeLeft = 100

右代表该节点的左孩子节点已经遍历，右孩子节点还没有遍历

nodeRight = 200

上代表左右孩子节点都已经遍历，需要返回到它的父节点

nodeUp = 300

每个节点的初始化状态都是从 左 开始

nodeState = nodeLeft

```

# 对二叉树进行遍历
while curNode :

    # 位置 ①

    # 如果当前节点的状态是【左】，说明该节点的左右孩子节点都没有遍历
    if nodeState == nodeLeft :

        # 如果当前节点有左子树
        if curNode.left :

            # 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.append(curNode)

            # 开始观察当前节点的左孩子节点，代码来到了位置 ①
            curNode = curNode.left
        else:

            # 如果当前节点没有左子树，切换当前节点的状态为【右】
            # 代码来到了位置 ①
            nodeState = nodeRight

    elif nodeState == nodeRight:

        # 如果当前节点的状态是【右】，说明该节点的左孩子节点已经遍历，右孩子节点还没有遍历

        if preNode :
            result = min(result, curNode.val - preNode.val)

        preNode = curNode

        # 如果当前节点有右子树
        if curNode.right :

            # 先把当前节点加入到栈中，用来记录节点移动的路径
            stack.append(curNode)

            # 开始观察当前节点的右孩子节点
            curNode = curNode.right

            # 每个节点开始的状态都是【左】开始，所以需要重新设置一下节点的状态
            nodeState = nodeLeft

            # 执行完上面三行代码，代码来到了位置 ①

        else:
            # 如果当前节点没有右子树，切换当前节点的状态为【上】

```



```
# 代码来到了位置 ①
nodeState = nodeUp
```

```
elif nodeState == nodeUp :
```

```
# 如果当前节点的状态是【上】，说明左右孩子节点都已经遍历，需要返回到它的父节点
# 把当前节点加入到二叉树后序遍历的结果数组中
# postorderResult.add(node.val)
```

```
# 需要返回到当前节点的父节点位置，通过栈顶元素来获取当前节点的父节点
# 首先构建一个空的节点
parent = None
```

```
# 如果栈中有元素
if stack :
```

```
# 那么，栈顶元素就是当前节点的父节点
parent = stack.pop()
```

```
# 判断一下父节点的左节点是否为当前节点
# 比如这颗二叉树
```

```
#           1
#          / \
#         2   3
#        / \   \
#       4  5   6
```

```
# 如果当前节点是 4，那么 4 的父节点是 2，2 的左孩子节点是 4，此时需要切换状
```

态为【右】

```
# 如果当前节点是 5，那么 5 的父节点是 2，2 的左孩子节点是 4，此时不需要切换
```

状态

```
# 如果父节点的左节点为当前节点
if parent.left == curNode :
    # 切换当前节点的状态为【右】
    nodeState = nodeRight
```

```
# 开始观察当前节点的父节点
# 如果上述代码中栈中有元素，那么 parent 有值，代码来到了位置 ①
# 如果上述代码中栈中没有元素，那么 parent 没有值，会跳出循环
curNode = parent
```

```
# 根据题目要求，返回二叉树前序、中序、后序遍历的结果
return result
```

LeetCode 654、最大二叉树

一、题目描述

给定一个不含重复元素的整数数组 `nums` 。一个以此数组直接递归构建的 **最大二叉树** 定义如下：

- 二叉树的根是数组 `nums` 中的最大元素。
- 左子树是通过数组中 **最大值左边部分** 递归构造出的最大二叉树。
- 右子树是通过数组中 **最大值右边部分** 递归构造出的最大二叉树。

返回有给定数组 `nums` 构建的 **最大二叉树** 。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最大二叉树 ( LeetCode 654 ) :https://leetcode-cn.com/problems/maximum-binary-tree
class Solution {
    public TreeNode constructMaximumBinaryTree(int[] nums) {
        // 通过递归的操作可以构建出最大二叉树
        return traversal( nums , 0 , nums.length - 1 );
    }

    private TreeNode traversal( int[] nums, int left, int right ){

        // 递归终止条件
        // 当前区间没有元素
        if (left > right) return null;

        // 或者当前区间只有一个元素，那么这个元素就是这颗子二叉树的根节点
        if ( left == right ) return new TreeNode(nums[left]);

        // 假设 [ left , right ] 这个区间的最大值的索引位置在 left 这个位置
        int maxIndex = left;

        // 遍历 [ left + 1 , right ] 这个区间，找出 [ left , right ] 这个区间的最大值
        for ( int i = left + 1 ; i <= right ; i++){

            // 更新 maxIndex
            if(nums[i] > nums[maxIndex]){
                maxIndex = i;
            }
        }

        // root 是数组 nums 在 [ left , right ] 这个区间中的最大元素
```

```

TreeNode root = new TreeNode(nums[maxValueIndex]);

// 根据 maxValueIndex, 划分为 [ left , maxValueIndex - 1 ]
// 递归的构建这个区间的二叉树
TreeNode leftNode = traversal(nums, left, maxValueIndex - 1);

// 根据 maxValueIndex, 划分为 [ maxValueIndex - 1 , right]
// 递归的构建这个区间的二叉树
TreeNode rightNode = traversal(nums, maxValueIndex + 1, right);

// leftNode 为 root 的左子树
root.left = leftNode;

// rightNode 为 root 的右子树
root.right = rightNode;

// 返回结果
return root;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最大二叉树 ( LeetCode 654 ) :https://leetcode-cn.com/problems/maximum-binary-tree
class Solution
{
public:
    TreeNode *constructMaximumBinaryTree(vector<int> nums)
    {
        // 通过递归的操作可以构建出最大二叉树
        return traversal(nums, 0, nums.size() - 1);
    }

    TreeNode *traversal(vector<int>& nums, int left, int right)
    {
        // 递归终止条件
        // 当前区间没有元素
        if (left > right)
            return NULL;

        // 或者当前区间只有一个元素, 那么这个元素就是这颗子二叉树的根节点
        if (left == right)
            return new TreeNode(nums[left]);
    }
}

```

```

// 假设 [ left , right ] 这个区间的最大值的索引位置在 left 这个位置
int maxValueIndex = left;

// 遍历 [ left + 1 , right ] 这个区间, 找出 [ left , right ] 这个区间的最大值
for (int i = left + 1; i <= right; i++)
{
    // 更新 maxValueIndex
    if (nums[i] > nums[maxValueIndex])
    {
        maxValueIndex = i;
    }
}

// root 是数组 nums 在 [ left , right ] 这个区间中的最大元素
TreeNode *root = new TreeNode(nums[maxValueIndex]);

// 根据 maxValueIndex, 划分为 [ left , maxValueIndex - 1 ]
// 递归的构建这个区间的二叉树
TreeNode *leftNode = traversal(nums, left, maxValueIndex - 1);

// 根据 maxValueIndex, 划分为 [ maxValueIndex - 1 , right]
// 递归的构建这个区间的二叉树
TreeNode *rightNode = traversal(nums, maxValueIndex + 1, right);

// leftNode 为 root 的左子树
root->left = leftNode;

// rightNode 为 root 的右子树
root->right = rightNode;

// 返回结果
return root;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者: 程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 最大二叉树 ( LeetCode 654 ) :https://leetcode-cn.com/problems/maximum-binary-tree
class Solution:
    def constructMaximumBinaryTree(self, nums: List[int]) -> TreeNode:
        # 通过递归的操作可以构建出最大二叉树
        return self.traversal( nums , 0 , len(nums) - 1 )

    def traversal(self, nums: List[int] , left : int ,right : int) -> TreeNode:

```

```

# 递归终止条件
# 当前区间没有元素
if left > right :
    return None

# 或者当前区间只有一个元素，那么这个元素就是这颗子二叉树的根节点
if left == right :
    return TreeNode(nums[left])

# 假设 [ left , right ] 这个区间的最大值的索引位置在 left 这个位置
maxValueIndex = left

# 遍历 [ left + 1 , right ] 这个区间，找出 [ left , right ] 这个区间的最大值
for i in range(left + 1 , right + 1) :
    # 更新 maxValueIndex
    if nums[i] > nums[maxValueIndex] :
        maxValueIndex = i

# root 是数组 nums 在 [ left , right ] 这个区间中的最大元素
root = TreeNode(nums[maxValueIndex])

# 根据 maxValueIndex，划分为 [ left , maxValueIndex - 1 ]
# 递归的构建这个区间的二叉树
leftNode = self.traversal(nums, left, maxValueIndex - 1)

# 根据 maxValueIndex，划分为 [ maxValueIndex - 1 , right]
# 递归的构建这个区间的二叉树
rightNode = self.traversal(nums, maxValueIndex + 1, right)

# leftNode 为 root 的左子树
root.left = leftNode

# rightNode 为 root 的右子树
root.right = rightNode

# 返回结果
return root

```

LeetCode 133、克隆图

一、题目描述

给你无向 [连通](#) 图中一个节点的引用，请你返回该图的 [深拷贝](#)（克隆）。

图中的每个节点都包含它的值 `val`（`int`）和其邻居的列表（`list[Node]`）。

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

测试用例格式：

简单起见，每个节点的值都和它的索引相同。例如，第一个节点值为 1（`val = 1`），第二个节点值为 2（`val = 2`），以此类推。该图在测试用例中使用邻接列表表示。

邻接列表 是用于表示有限图的无序列表的集合。每个列表都描述了图中节点的邻集。

给定节点将始终是图中的第一个节点（值为 1）。你必须将 **给定节点的拷贝** 作为对克隆图的引用返回。

示例 1：

img

```
输入: adjList = [[2,4],[1,3],[2,4],[1,3]]
输出: [[2,4],[1,3],[2,4],[1,3]]
解释:
图中有 4 个节点。
节点 1 的值是 1，它有两个邻居：节点 2 和 4 。
节点 2 的值是 2，它有两个邻居：节点 1 和 3 。
节点 3 的值是 3，它有两个邻居：节点 2 和 4 。
节点 4 的值是 4，它有两个邻居：节点 1 和 3 。
```

示例 2：

img

```
输入: adjList = [[]]
输出: [[]]
解释: 输入包含一个空列表。该图仅仅只有一个值为 1 的节点，它没有任何邻居。
```

示例 3：

```
输入: adjList = []
输出: []
解释: 这个图是空的，它不含任何节点。
```

示例 4:

img

```
输入: adjList = [[2],[1]]
输出: [[2],[1]]
```

提示:

1. 节点数不超过 100。
2. 每个节点值 `Node.val` 都是唯一的, $1 \leq \text{Node.val} \leq 100$ 。
3. 无向图是一个简单图, 这意味着图中没有重复的边, 也没有自环。
4. 由于图是无向的, 如果节点 p 是节点 q 的邻居, 那么节点 q 也必须是节点 p 的邻居。
5. 图是连通图, 你可以从给定节点访问到所有节点。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: 278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 133. 克隆图: https://leetcode.cn/problems/clone-graph/
class Solution {

    // 使用哈希表存储每个节点的信息
    private HashMap <Node, Node> visited = new HashMap <> ();

    public Node cloneGraph(Node node) {

        // 边界处理
        if (node == null) {
            return node;
        }

        // 如果该节点已经被访问过了, 则直接从哈希表中取出对应的克隆节点返回
        if (visited.containsKey(node)) {
            return visited.get(node);
        }

        // 克隆节点使用深拷贝
        Node cloneNode = new Node(node.val, new ArrayList());
```

```

// 使用哈希表存储当前节点信息
visited.put(node, cloneNode);

// 遍历该节点的邻居
for (Node neighbor: node.neighbors) {

    // 克隆当前节点的所有邻居节点
    Node neighborNode = cloneGraph(neighbor);

    // 把克隆成功之后的节点也都添加到克隆节点的邻居节点上
    cloneNode.neighbors.add(neighborNode);

}

// 完成克隆
return cloneNode;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    unordered_map<Node*, Node*> visited;
    Node* cloneGraph(Node* node) {
        if (node == nullptr) {
            return node;
        }

        // 如果该节点已经被访问过了，则直接从哈希表中取出对应的克隆节点返回
        if (visited.find(node) != visited.end()) {
            return visited[node];
        }

        // 克隆节点，注意到为了深拷贝我们不会克隆它的邻居的列表
        Node* cloneNode = new Node(node->val);
        // 哈希表存储
        visited[node] = cloneNode;

        // 遍历该节点的邻居并更新克隆节点的邻居列表
        for (auto& neighbor: node->neighbors) {
            cloneNode->neighbors.emplace_back(cloneGraph(neighbor));
        }
        return cloneNode;
    }
};

```


3、Python 代码

```
class Solution(object):

    def __init__(self):
        self.visited = {}

    def cloneGraph(self, node):
        """
        :type node: Node
        :rtype: Node
        """
        if not node:
            return node

        # 如果该节点已经被访问过了，则直接从哈希表中取出对应的克隆节点返回
        if node in self.visited:
            return self.visited[node]

        # 克隆节点，注意到为了深拷贝我们不会克隆它的邻居的列表
        clone_node = Node(node.val, [])

        # 哈希表存储
        self.visited[node] = clone_node

        # 遍历该节点的邻居并更新克隆节点的邻居列表
        if node.neighbors:
            clone_node.neighbors = [self.cloneGraph(n) for n in node.neighbors]

        return clone_node
```

LeetCode 207、课程表

一、题目描述

你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1`。

在选修某些课程之前需要一些先修课程。先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [ai, bi]`，表示如果要学习课程 `ai` 则 必须 先学习课程 `bi`。

- 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0`，你需要先完成课程 `1`。

请你判断是否可能完成所有课程的学习？如果可以，返回 `true`；否则，返回 `false`。

示例 1：

输入：numCourses = 2, prerequisites = [[1,0]]

输出：true

解释：总共有 2 门课程。学习课程 1 之前，你需要完成课程 0。这是可能的。

示例 2:

输入: numCourses = 2, prerequisites = [[1,0],[0,1]]

输出: false

解释: 总共有 2 门课程。学习课程 1 之前, 你需要先完成课程 0 ; 并且学习课程 0 之前, 你还应先完成课程 1 。这是不可能的。

提示:

- `1 <= numCourses <= 105`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `prerequisites[i]` 中的所有课程对 **互不相同**

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 课程表 (LeetCode 207) : https://leetcode.cn/problems/course-schedule/submissions/
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {

        // 尝试在不具备图的基础知识的情况下来做这题

        // 本题的核心点在于处理任意一个课程的时候, 都需要考虑它是否有前置课程没有完成
        // 只有说当前这个课程没有前置课程才能去处理它

        // 1、记录每一个课程的情况
        // 即, 记录课程号与课程前置课程数量
        // 使用哈希表来记录, 可以快速获取和新增结果
        // 在图这种数据结构里面专业术语叫做记录每个课程的【入度】
        // key: 课号
        // value: 学习这门课程的前置课程数量
        Map<Integer, Integer> inDegree = new HashMap<>();

        // 2、初始化哈希表
        for (int i = 0; i < numCourses; i++) {
            // 3、每个课程的前置课程默认都是 0
            inDegree.put(i, 0);
        }
    }
}
```

```

// 4、记录每个课程直接的依赖关系
// 从而知道完成某个课程的时候，其它哪些课程被影响到了
// 由于被影响的课程可能有很多个，所以采用数组的形式保存被影响的那些课程
// 依赖关系，在图这种数据结构里面专业术语叫做邻接表
// key: 课号
// value: 依赖这门课的后续课，涉及多门，数组保存
Map<Integer, List<Integer>> map = new HashMap<>();

// 5、遍历 prerequisites 数组
// 更新 inDegree 与 map
for (int[] arr : prerequisites) {

    // prerequisites 数组中的每一个元素都是数组的形式
    // 每一个元素记录了两个数据
    // [first , second ]
    // 学 first 的前置课程之一是完成 second
    // 完成 second 会影响 first 的前置课程数量
    int first = arr[0];

    int second = arr[1];

    // 6、更新入度
    // 即更新 first 的前置课程数量，加 1 操作
    inDegree.put(first, inDegree.get(first) + 1);

    // 7、更新依赖
    // 即记录 second 会影响哪些课程
    // 此时，second 必然会影响 first
    if (!map.containsKey(second)) {

        map.put(second, new ArrayList<>());

    }

    // 8、更新 second 影响的课程
    map.get(second).add(first);
}

// 9、开始去选修课程，只有没有前置课程的课程才能去选修
// 即入度为 0 的课程才能去选修
// 完成了一个入度为 0 的课程之后，会影响它的后续课，如果后续课为 0 了，又能选修了
// 以此类推
// 一开始把所有入度为 0 的课程都加入到队列里面
Queue<Integer> q = new LinkedList<>();

for (int key : inDegree.keySet()) {
    if (inDegree.get(key) == 0) {
        q.offer(key);
    }
}

```

```

    }
}

// 10、每个课程依次出队进行处理
// a、减小相关课的入度
// b、如果相关课的入度变为 0，也可以加入到队列里面
// 直到队列为空，即没有课程可以选为止
while (!q.isEmpty()) {

    // 课程出队
    int course = q.poll();

    // 如果发现该课程不会对任何一个课程有影响
    // 即，如果发现依赖关系（邻接表）没有 course
    // 那么直接去查看下一个课程的情况
    if (!map.containsKey(course)) {
        continue;
    }

    // 否则开始更新当前课程的所有后续课程的【入度】情况
    // 获取当前课程的所有的后续课程
    List<Integer> successorList = map.get(course);

    // 更新后续课程的【入度】情况
    for (int k : successorList) {

        // 当前课程 course 选修完毕之后
        // 每个后续课程的前置课程数量减一
        // 即【入度】减一
        inDegree.put(k, inDegree.get(k) - 1);

        // 如果发现后续某个课程【入度】为零
        if (inDegree.get(k) == 0) {
            // 把它也加入到队列处理
            q.offer(k);
        }
    }
}

// 11、由于只有入度为零的课程才能被选修
// 因此，遍历 inDegree，查看是否还有入度为非零的课程
for (int key : inDegree.keySet()) {
    // 如果有，说明当前课程无法被选修
    if (inDegree.get(key) != 0) {
        // 即无法完成所有课程的学习
        return false;
    }
}
}

```

```

        // 12、否则说明可以完成所有课程的学习
        return true;
    }
}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

微信：278166530

代码有看不懂的地方一定要私聊咨询吴师兄呀

class Solution:

```
def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
```

```
    # 尝试在不具备图的基础知识的情况下来做这题
```

```
    # 本题的核心点在于处理任意一个课程的时候，都需要考虑它是否有前置课程没有完成
```

```
    # 只有说当前这个课程没有前置课程才能去处理它
```

```
    # 1、记录每一个课程的情况
```

```
    # 即，记录课程号与课程前置课程数量
```

```
    # 使用哈希表来记录，可以快速获取和新增结果
```

```
    # 在图这种数据结构里面专业术语叫做记录每个课程的【入度】
```

```
    # key: 课号
```

```
    # value: 学习这门课程的前置课程数量
```

```
    # 2、初始化哈希表
```

```
    inDegree = [0] * numCourses
```

```
    # 4、记录每个课程之间的依赖关系
```

```
    # 从而知道完成某个课程的时候，其它哪些课程被影响到了
```

```
    # 由于被影响的课程可能有很多个，所以采用数组的形式保存被影响的那些课程
```

```
    # 依赖关系，在图这种数据结构里面专业术语叫做邻接表
```

```
    # key: 课号
```

```
    # value: 依赖这门课的后续课，涉及多门，数组保存
```

```
    map = defaultdict(list)
```

```
    # 5、遍历 prerequisites 数组
```

```
    # 更新 inDegree 与 map
```

```
    for arr in prerequisites:
```

```
        # prerequisites 数组中的每一个元素都是数组的形式
```

```
        # 每一个元素记录了两个数据
```

```

# [first , second ]
# 学 first 的前置课程之一是完成 second
# 完成 second 会影响 first 的前置课程数量
first = arr[0]

second = arr[1]

# 6、更新入度
# 即更新 first 的前置课程数量，加 1 操作
inDegree[first] += 1

# 7、更新依赖
# 即记录 second 会影响哪些课程
# 此时，second 必然会影响 first
if map.get(second) == None:
    map[second] = []
    # print(map[second])

map[second].append(first)

# 9、开始去选修课程，只有没有前置课程的课程才能去选修
# 即入度为 0 的课程才能去选修
# 完成了一个入度为 0 的课程之后，会影响它的后续课，如果后续课为 0 了，又能选修了
# 以此类推
# 一开始把所有入度为 0 的课程都加入到队列里面
q = deque()

for i in range(len(inDegree)):
    if inDegree[i] == 0:
        q.append(i)

# 10、每个课程依次出列进行处理
# a、减小相关课的入度
# b、如果相关课的入度变为 0，也可以加入到队列里面
# 直到队列为空，即没有课程可以选修为止
while q:

    # 课程出队
    course = q.popleft()
    # print(course)

    # 如果发现该课程不会对任何一个课程有影响
    # 即，如果发现依赖关系（邻接表）没有 course
    # 那么直接去查看下一个课程的情况
    if map.get(course) == None :
        continue

```

```

# 否则开始更新当前课程的所有后续课程的【入度】情况
# 获取当前课程的所有后续课程
successorList = map.get(course)

# 更新后续课程的【入度】情况
for k in successorList :

    # 当前课程 course 选修完毕之后
    # 每个后续课程的前置课程数量减一
    # 即【入度】减一

    inDegree[k] -= 1

# 如果发现后续某个课程【入度】为零
if inDegree[k] == 0 :
    # 把它也加入到队列处理
    q.append(k)
# 11、由于只有入度为零的课程才能被选修
# 因此，遍历 inDegree，查看是否还有入度为非零的课程
for key in inDegree :
    # 如果有，说明当前课程无法被选修
    if key != 0 :
        # 即无法完成所有课程的学习
        return False

# 12、否则说明可以完成所有课程的学习
return True

```

LeetCode 210、课程表II

一、题目描述

现在你总共有 `numCourses` 门课需要选，记为 `0` 到 `numCourses - 1`。给你一个数组 `prerequisites`，其中 `prerequisites[i] = [ai, bi]`，表示在选修课程 `ai` 前 **必须** 先选修 `bi`。

- 例如，想要学习课程 `0`，你需要先完成课程 `1`，我们用一个匹配来表示：`[0,1]`。

返回你为了学完所有课程所安排的学习顺序。可能会有多个正确的顺序，你只要返回 **任意一种** 就可以了。如果不可能完成所有课程，返回 **一个空数组**。

示例 1：

输入：`numCourses = 2, prerequisites = [[1,0]]`

输出：`[0,1]`

解释：总共有 2 门课程。要学习课程 1，你需要先完成课程 0。因此，正确的课程顺序为 `[0,1]`。

示例 2：

输入: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

输出: [0,2,1,3]

解释: 总共有 4 门课程。要学习课程 3, 你应该先完成课程 1 和课程 2。并且课程 1 和课程 2 都应该排在课程 0 之后。

因此, 一个正确的课程顺序是 [0,1,2,3] 。另一个正确的排序是 [0,2,1,3] 。

示例 3:

输入: numCourses = 1, prerequisites = []

输出: [0]

提示:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `ai != bi`
- 所有 `[ai, bi]` 互不相同

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 课程表II (LeetCode 210) : https://leetcode.cn/problems/course-schedule-ii/
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {

        // 1、记录每一个课程的情况
        // 即, 记录课程号与课程前置课程数量
        // 使用哈希表来记录, 可以快速获取和新增结果
        // 在图这种数据结构里面专业术语叫做记录每个课程的【入度】
        // key: 课号
        // value: 学习这门课程的前置课程数量
        Map<Integer,Integer> inDegree = new HashMap<>();

        // 2、初始化哈希表
        for (int i = 0 ; i < numCourses ;i++) {
            // 3、每个课程的前置课程默认都是 0
            inDegree.put(i , 0);
        }
    }
}
```



```

// 4、记录每个课程之间的依赖关系
// 从而知道完成某个课程的时候，其它哪些课程被影响到了
// 由于被影响的课程可能有很多个，所以采用数组的形式保存被影响的那些课程
// 依赖关系，在图这种数据结构里面专业术语叫做邻接表
// key：课号
// value：依赖这门课的后续课，涉及多门，数组保存
Map<Integer ,List<Integer>> map = new HashMap<>();

// 5、遍历 prerequisites 数组
// 更新 inDegree 与 map
for (int[] arr : prerequisites) {

    // prerequisites 数组中的每一个元素都是数组的形式
    // 每一个元素记录了两个数据
    // [first , second ]
    // 学 first 的前置课程之一是完成 second
    // 完成 second 会影响 first 的前置课程数量
    int first = arr[0];

    int second = arr[1];

    // 6、更新入度
    // 即更新 first 的前置课程数量，加 1 操作
    inDegree.put( first , inDegree.get(first) + 1 );

    // 7、更新依赖
    // 即记录 second 会影响哪些课程
    // 此时， second 必然会影响 first
    if (!map.containsKey(second)) {

        map.put(second ,new ArrayList<>());

    }

    // 8、更新 second 影响的课程
    map.get(second).add(first);
}

// 9、开始去选修课程，只有没有前置课程的课程才能去选修
// 即入度为 0 的课程才能去选修
// 完成了一个入度为 0 的课程之后，会影响它的后续课，如果后续课为 0 了，又能选修了
// 以此类推
// 一开始把所有入度为 0 的课程都加入到队列里面

```

```

Queue<Integer> q = new LinkedList<>();

//result
List<Integer> result = new ArrayList<>();

for (int key : inDegree.keySet()) {
    if (inDegree.get(key) == 0) {
        q.offer(key);

        result.add(key);
    }
}

// 10、每个课程依次出队进行处理
// a、减小相关课的入度
// b、如果相关课的入度变为 0，也可以加入到队列里面
// 直到队列为空，即没有课程可以选为止
while (!q.isEmpty()) {

    // 课程出队
    int course = q.poll();
    // System.out.println(course);

    // 如果发现该课程不会对任何一个课程有影响
    // 即，如果发现依赖关系（邻接表）没有 course
    // 那么直接去查看下一个课程的情况
    if (!map.containsKey(course)) {
        continue;
    }

    // 否则开始更新当前课程的所有后续课程的【入度】情况
    // 获取当前课程的所有后续课程
    List<Integer> successorList = map.get(course);

    // 更新后续课程的【入度】情况
    for (int k : successorList) {

        // 当前课程 course 选修完毕之后
        // 每个后续课程的前置课程数量减一
        // 即【入度】减一
        inDegree.put( k ,inDegree.get(k) - 1);

        // 如果发现后续某个课程【入度】为零
        if (inDegree.get(k) == 0) {
            // 把它也加入到队列处理
            q.offer(k);
            result.add(k);
        }
    }
}

```

```

    }
}

// 11、由于只有入度为零的课程才能被选修
// 因此，遍历 inDegree，查看是否还有入度为非零的课程
for (int key : inDegree.keySet()) {
    // 如果有，说明当前课程无法被选修
    if ( inDegree.get(key) != 0 ) {
        // 即无法完成所有课程的学习，返回 一个空数组
        return new int[]{};
    }
}

// 12、否则说明可以完成所有课程的学习，返回 任意一种 顺序
return result.stream().mapToInt(Integer::valueOf).toArray();

}
}

```

LeetCode 684、冗余连接

一、题目描述

树可以看成是一个连通且 **无环** 的 **无向** 图。

给定一棵 n 个节点（节点值 $1 \sim n$ ）的树中添加一条边后的图。添加的边的两个顶点包含在 1 到 n 中间，且这条附加的边不属于树中已存在的边。图的信息记录于长度为 n 的二维数组 `edges`，`edges[i] = [ai, bi]` 表示图中在 `ai` 和 `bi` 之间存在一条边。

请找出一条可以删去的边，删除后可使得剩余部分是一个有着 n 个节点的树。如果有多个答案，则返回数组 `edges` 中最后出现的边。

示例 1:

img

输入: `edges = [[1,2], [1,3], [2,3]]`
 输出: `[2,3]`

示例 2:

img

输入: `edges = [[1,2], [2,3], [3,4], [1,4], [1,5]]`
 输出: `[1,4]`

提示:

- `n == edges.length`
- `3 <= n <= 1000`
- `edges[i].length == 2`
- `1 <= ai < bi <= edges.length`
- `ai != bi`
- `edges` 中无重复元素
- 给定的图是连通的

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {

    // 创建二维数组表示单边数组
    List<List<Integer>> graph;

    // 记录从某个点出发开始访问时，每个节点是否被访问
    List<Boolean> visited;

    // 当前是否找到环
    boolean hasCycle = false;

    public int[] findRedundantConnection(int[][] edges) {
        // n 个节点，节点值在 1 ~ n
        int n = edges.length;

        // 初始化图，由于节点值在 1 ~ n，所以二维矩阵的大小为 n + 1
        // 即索引为 0 的那一行不会记录数据
        graph = new ArrayList<List<Integer>>(n + 1);

        // 初始化每一行
        // 每一行记录一个节点的信息
        for (int i = 0; i <= n; i++) {
            // 使用数组进行记录
            graph.add(new ArrayList<Integer>());
        }

        // 访问 edges
        for (int[] edge : edges) {
            // 每一个 edge 都包含了两个节点的信息
            // 由于是无向图，所以两者都可以相互访问到对方
```

```

graph.get(edge[0]).add(edge[1]);
graph.get(edge[1]).add(edge[0]);

// 每增一条 edge 边, 在原来的已经访问节点的基础上看这条边是否形成了环
hasCycle = false;

// 重新初始化 visited
visited = new ArrayList<Boolean>();

// 默认都是 false
for (int i = 0; i <= n; i++) {
    visited.add(false);
}

// 开始以 edge[0] 这个节点作为初始节点进行访问
// 既然以它作为初始节点, 那么它的父节点就是不存在的, 用 -1 来表示
dfs(edge[0], -1);

// 如果在访问结束后发现找到了环
if(hasCycle){
    // 那么 edge 就是最后出现的边
    return edge;
}
return null;
}

// 图深度优先遍历
// curNode : 出发节点
// fatherNode : 出发节点的父节点
private void dfs(int curNode, int fatherNode) {
    // curNode 这个节点被访问了, 更新为 true
    // visited 的索引值就是节点值
    visited.set(curNode, true);

    // 获取 curNode 可以前往的所有节点信息
    List<Integer> info = graph.get(curNode);

    // 对每个可以前往的节点都访问一下
    for (int nextNode : info) {

        // 如果发现是前往父节点, 不处理, 直接看下一个节点
        if (nextNode == fatherNode)
            continue;

        // 如果发现 nextNode 是还未访问的节点
        if (!visited.get(nextNode)) {

            // 那么以 nextNode 作为初始节点, curNode 作为父节点

```

```

        // 递归的进行搜索访问下去
        dfs(nextNode, curNode);

        // 否则, 如果发现 nextNode 是已经访问过的节点
    } else {

        // 说明从 nextNode 经过一系列访问后来到了 curNode
        // curNode 又能来到 nextNode
        // 有环了
        hasCycle = true;
        return;
    }
}
}
}
}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者: 程序员吴师兄

微信: 278166530

代码有看不懂的地方一定要私聊咨询吴师兄呀

class Solution:

创建二维数组表示单边数组

graph = []

记录从某个点出发开始访问时, 每个节点是否被访问

visited = []

当前是否找到环

hasCycle = False

def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:

n 个节点, 节点值在 1 ~ n

n = len(edges)

初始化图, 由于节点值在 1 ~ n, 所以二维矩阵的大小为 n + 1

即索引为 0 的那一行不会记录数据

初始化每一行

每一行记录一个节点的信息

self.graph = [[] for i in range(n + 1)]

```

# 访问 edges
for edge in edges :
    # 每一个 edge 都包含了两个节点的信息
    # 由于是无向图，所以两者都可以相互访问到对方
    self.graph[edge[0]].append(edge[1])
    self.graph[edge[1]].append(edge[0])

    # 每增一条 edge 边，在原来的已经访问节点的基础上看这条边是否形成了环
    self.hasCycle = False

    # 重新初始化 visited
    self.visited = [ False ] * (n + 1)

    # 默认都是 false

    # 开始以 edge[0] 这个节点作为初始节点进行访问
    # 既然以它作为初始节点，那么它的父节点就是不存在的，用 -1 来表示
    self.dfs(edge[0], -1)

    # 如果在访问结束后发现找到了环
    if self.hasCycle :
        # 那么 edge 就是最后出现的边
        return edge

# 图深度优先遍历
# curNode : 出发节点
# fatherNode : 出发节点的父节点
def dfs(self, curNode, fatherNode) :
    # curNode 这个节点被访问了，更新为 true
    # visited 的索引值就是节点值
    self.visited[curNode] = True

    # 获取 curNode 可以前往的所有节点信息

    # 对每个可以前往的节点都访问一
    for nextNode in self.graph[curNode]:

        # 如果发现是前往父节点，不处理，直接看下一个节点
        if nextNode == fatherNode :
            continue

        # 如果发现 nextNode 是还未访问的节点
        if self.visited[nextNode] == False:

            # 那么以 nextNode 作为初始节点，curNode 作为父节点
            # 递归的进行搜索访问下去

```

```
self.dfs(nextNode, curNode)

# 否则, 如果发现 nextNode 是已经访问过的节点
else:

    # 说明从 nextNode 经过一系列访问后来到了 curNode
    # curNode 又能来到 nextNode
    # 有环了
    self.hasCycle = True
    return
```

LeetCode 743、网络延迟时间

一、题目描述

有 n 个网络节点，标记为 1 到 n 。

给你一个列表 `times`，表示信号经过 有向 边的传递时间。 `times[i] = (ui, vi, wi)`，其中 `ui` 是源节点，`vi` 是目标节点，`wi` 是一个信号从源节点传递到目标节点的时间。

现在，从某个节点 `k` 发出一个信号。需要多久才能使所有节点都收到信号？如果不能使所有节点收到信号，返回 `-1`。

示例 1:

img

```
输入: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
输出: 2
```

示例 2:

```
输入: times = [[1,2,1]], n = 2, k = 1
输出: 1
```

示例 3:

```
输入: times = [[1,2,1]], n = 2, k = 2
输出: -1
```

提示:

- $1 \leq k \leq n \leq 100$
- $1 \leq \text{times.length} \leq 6000$
- $\text{times}[i].\text{length} == 3$
- $1 \leq ui, vi \leq n$
- $ui \neq vi$

- $0 \leq w_i \leq 100$
- 所有 (u_i, v_i) 对都互不相同（即，不含重复边）

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 网络延迟时间 (LeetCode 743) : https://leetcode.cn/problems/network-delay-time/
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {

        // 1、默认任何一个网络节点到其它任何一个网络节点的传递时间为无穷大
        // 这样在对比过程中，获取到一个具体的数值时才能通过取较小值的方式获取到该值
        // 2、更新最短距离的时候，会计算两个距离之和，为了防止溢出
        // 采取除以 2 的操作
        // 3、根据题目的提示，n 最大为 100，传递时间最大为 100
        // 所以，INF 的值设置为 10000、10001、10002、10003 等也是可以的
        int INF = Integer.MAX_VALUE / 2;

        // 使用邻接矩阵存储边信息
        int[][] g = new int[n][n];

        // 初始化
        // 默认节点之间的传递时间为 INF
        for (int i = 0; i < n; ++i) {
            Arrays.fill(g[i], INF);
        }

        // 遍历列表 times
        // times 列表里面的元素是一个一维数组
        // 一维数组里面包含三个元素
        // [ u , v , w ]
        // u 表示源节点
        // v 表示目标节点
        // w 表示 u 到 v 的传递时间
        for (int[] t : times) {

            // 根据题目的说明
            // 由于 u 、 v 的标记范围是 1 到 n
            // 而数组下标索引是从 0 开始的
            // 所以需要进行一个计算

            int uIndex = t[0] - 1;
```

```

    int vIndex = t[1] - 1;

    // 把传递时间更新到图中
    g[uIndex][vIndex] = t[2];

}

// 开始计算所有点到源点的距离
// dist[x] = y 代表从「源点/起点」到 x 的最短距离为 y
// 根据我们的视频动画演示，可以利用一个数组记录
int[] dist = new int[n];

// 默认值也可以设置为一个无穷大的值
Arrays.fill(dist, INF);

// 设置源点到源点的值
// 由于从 k 开始，所以该点距离设为 0，也即源点
// 即源点的最短距离是 0
dist[k - 1] = 0;

// 为了避免节点被反复计算
// 设置一个数组，用来记录节点是否被更新过
boolean[] visited = new boolean[n];

// 迭代 n 次
// 第一次计算可以计算出从源点 k 到源点 k 的距离为 0
// 以示例 1 作为输入，会设置 visited[1] = true
for (int i = 0; i < n; i++) {

    // 每次都去寻找出「最短距离最小」且「未被更新」的点来
    // 这个点一开始不知道在哪个位置
    // 于是可以默认设置为一个不存在的值
    // -1、-2、-3 等都是可以的
    int x = -1;

    // 访问 visited 里面的所有元素
    // 定位出 x 这个节点应该是在哪个位置
    for (int j = 0; j < n; j++) {

        // 1、这个节点需要没有被更新过
        // 2.1、x 还没有确定在哪个节点位置
        // 2.2、x 确定了，但发现有个节点距离源点还更近
        if (!visited[j] && (x == -1 || dist[j] < dist[x])) {
            // 定位了 x 的位置
            x = j;
        }
    }
}

```

```

        // 跳出上面这个 for 循环之后
        // 已经明确知道 x 在哪个节点位置了
        // 标记 x 为 true
        visited[x] = true;

        // 依托于这个节点，从这个节点出发，更新其它点的距离
        for (int p = 0; p < n; p++) {

            // dist[p] 代表从「源点/起点」到 p 的最短距离
            // 对比之前存储的值 dist[p]
            // 和从源点到 x 的最短距离，再从 x 出发到 p 的距离两者的和
            dist[p] = Math.min(dist[p], dist[x] + g[x][p]);

        }

    }

    // 找到距离最远的点
    int ans = -1;

    // 在 dist 找出那个最大的值来
    for( int i = 0 ; i < n ; i++){

        ans = Math.max(dist[i], ans);

    }

    // 如果发现找出的距离是 INF
    // INF 是初始化的值，表示到不了，就返回 -1
    return ans == INF ? -1 : ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        // 1、默认任何一个网络节点到其它任何一个网络节点的传递时间为无穷大
        // 这样在对比过程中，获取到一个具体的数值时才能通过取较小值的方式获取到该值
        // 2、更新最短距离的时候，会计算两个距离之和，为了防止溢出
        // 采取除以 2 的操作
        // 3、根据题目的提示，n 最大为 100，传递时间最大为 100
        // 所以，INF 的值设置为 10000、10001、10002、10003 等也是可以的
        int INF = INT_MAX / 2;

        // 使用邻接矩阵存储边信息
        // 初始化
        // 默认节点之间的传递时间为 INF
    }
}

```

```

vector<vector<int>> g(n, vector<int>(n, INF));

// 遍历列表 times
// times 列表里面的元素是一个一维数组
// 一维数组里面包含三个元素
// [ u , v , w ]
// u 表示源节点
// v 表示目标节点
// w 表示 u 到 v 的传递时间
for (auto &t : times) {

    // 根据题目的说明
    // 由于 u 、 v 的标记范围是 1 到 n
    // 而数组下标索引是从 0 开始的
    // 所以需要进行一个计算

    int uIndex = t[0] - 1;

    int vIndex = t[1] - 1;

    // 把传递时间更新到图中
    g[uIndex][vIndex] = t[2];

}

// 开始计算所有点到源点的距离
// dist[x] = y 代表从「源点/起点」到 x 的最短距离为 y
// 根据我们的视频动画演示，可以利用一个数组记录
// 默认值也可以设置为一个无穷大的值
vector<int> dist(n, INF);

// 设置源点到源点的值
// 由于从 k 开始，所以该点距离设为 0，也即源点
// 即源点的最短距离是 0
dist[k - 1] = 0;

// 为了避免节点被反复计算
// 设置一个数组，用来记录节点是否被更新过
vector<bool> visited(n);

// 迭代 n 次
// 第一次计算可以计算出从源点 k 到源点 k 的距离为 0
// 以示例 1 作为输入，会设置 visited[1] = true
for (int i = 0; i < n; i++) {

    // 每次都去寻找出「最短距离最小」且「未被更新」的点来
    // 这个点一开始不知道在哪个位置
    // 于是可以默认设置为一个不存在的值
    // -1、-2、-3 等都是可以的

```

```

int x = -1;

// 访问 visited 里面的所有元素
// 定位出 x 这个节点应该是在哪个位置
for (int j = 0; j < n; j++) {

    // 1、这个节点需要没有被更新过
    // 2.1、x 还没有确定在哪个节点位置
    // 2.2、x 确定了, 但发现有个节点距离源点还更近
    if (!visited[j] && (x == -1 || dist[j] < dist[x])) {
        // 定位了 x 的位置
        x = j;
    }
}

// 跳出上面这个 for 循环之后
// 已经明确知道 x 在哪个节点位置了
// 标记 x 为 true
visited[x] = true;

// 依托于这个节点, 从这个节点出发, 更新其它点的距离
for (int p = 0; p < n; p++) {

    // dist[p] 代表从「源点/起点」到 p 的最短距离
    // 对比之前存储的值 dist[p]
    // 和从源点到 x 的最短距离, 再从 x 出发到 p 的距离两者的和
    dist[p] = min(dist[p], dist[x] + g[x][p]);

}

}

// 找到距离最远的点
int ans = -1;

// 在 dist 找出那个最大的值来
for (int i = 0; i < n; i++){

    ans = max(dist[i], ans);

}

// 如果发现找出的距离是 INF
// INF 是初始化的值, 表示到不了, 就返回 -1
return ans == INF ? -1 : ans;

}
};

```

3、Python 代码

```
class solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        # 1、默认任何一个网络节点到其它任何一个网络节点的传递时间为无穷大
        # 这样在对比过程中，获取到一个具体的数值时才能通过取较小值的方式获取到该值
        # 2、更新最短距离的时候，会计算两个距离之和，为了防止溢出
        # 采取除以 2 的操作
        # 3、根据题目的提示，n 最大为 100，传递时间最大为 100
        # 所以，INF 的值设置为 10000、10001、10002、10003 等也是可以的
        # 使用邻接矩阵存储边信息
        g = [[float('inf')] * n for _ in range(n)]

        # 初始化
        # 默认节点之间的传递时间为 INF

        # 遍历列表 times
        # times 列表里面的元素是一个一维数组
        # 一维数组里面包含三个元素
        # [ u , v , w ]
        # u 表示源节点
        # v 表示目标节点
        # w 表示 u 到 v 的传递时间
        for x, y, time in times:
            g[x - 1][y - 1] = time

        # 开始计算所有点到源点的距离
        # dist[x] = y 代表从「源点/起点」到 x 的最短距离为 y
        # 根据我们的视频动画演示，可以利用一个数组记录
        dist = [float('inf')] * n

        # 默认值也可以设置为一个无穷大的值

        # 设置源点到源点的值
        # 由于从 k 开始，所以该点距离设为 0，也即源点
        # 即源点的最短距离是 0
        dist[k - 1] = 0

        # 为了避免节点被反复计算
        # 设置一个数组，用来记录节点是否被更新过
        visited = [False] * n

        # 迭代 n 次
        # 第一次计算可以计算出从源点 k 到源点 k 的距离为 0
        # 以示例 1 作为输入，会设置 visited[1] = true
        for _ in range(n) :

            # 每次都去寻找出「最短距离最小」且「未被更新」的点来
```

```

# 这个点一开始不知道在哪个位置
# 于是可以默认设置为一个不存在的值
# -1、-2、-3 等都是可以的
x = -1

# 访问 visited 里面的所有元素
# 定位出 x 这个节点应该是在哪个位置
for j, u in enumerate(visited):
    # 1、这个节点需要没有被更新过
    # 2.1、x 还没有确定在哪个节点位置
    # 2.2、x 确定了，但发现有个节点距离源点还更近
    if not u and (x == -1 or dist[j] < dist[x]):
        # 定位了 x 的位置
        x = j

# 跳出上面这个 for 循环之后
# 已经明确知道 x 在哪个节点位置了
# 标记 x 为 true
visited[x] = True

# 依托于这个节点，从这个节点出发，更新其它点的距离
for p, time in enumerate(g[x]):
    dist[p] = min(dist[p], dist[x] + time)

# 找到距离最远的点
# 在 dist 找出那个最大的值来
ans = max(dist)

# 如果发现找出的距离是 INF
# INF 是初始化的值，表示到不了，就返回 -1
return ans if ans < float('inf') else -1

```

LeetCode 785、判断二分图

一、题目描述

存在一个无向图，图中有 n 个节点。其中每个节点都有一个介于 0 到 $n - 1$ 之间的唯一编号。给你一个二维数组 `graph`，其中 `graph[u]` 是一个节点数组，由节点 `u` 的邻接节点组成。形式上，对于 `graph[u]` 中的每个 `v`，都存在一条位于节点 `u` 和节点 `v` 之间的无向边。该无向图同时具有以下属性：

- 不存在自环（`graph[u]` 不包含 `u`）。
- 不存在平行边（`graph[u]` 不包含重复值）。
- 如果 `v` 在 `graph[u]` 内，那么 `u` 也应该在 `graph[v]` 内（该图是无向图）
- 这个图可能不是连通图，也就是说两个节点 `u` 和 `v` 之间可能不存在一条连通彼此的路径。

二分图 定义：如果能将一个图的节点集合分割成两个独立的子集 **A** 和 **B**，并使图中的每一条边的两个节点一个来自 **A** 集合，一个来自 **B** 集合，就将这个图称为 **二分图**。

如果图是二分图，返回 `true`；否则，返回 `false`。

示例 1:

img

输入: `graph = [[1,2,3],[0,2],[0,1,3],[0,2]]`

输出: `false`

解释: 不能将节点分割成两个独立的子集，以使每条边都连通一个子集中的一个节点与另一个子集中的一个节点。

示例 2:

img

输入: `graph = [[1,3],[0,2],[1,3],[0,2]]`

输出: `true`

解释: 可以将节点分成两组: `{0, 2}` 和 `{1, 3}`。

提示:

- `graph.length == n`
- `1 <= n <= 100`
- `0 <= graph[u].length < n`
- `0 <= graph[u][i] <= n - 1`
- `graph[u]` 不会包含 `u`
- `graph[u]` 的所有值 **互不相同**
- 如果 `graph[u]` 包含 `v`，那么 `graph[v]` 也会包含 `u`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 深度优先遍历的解法
class Solution {
    public boolean isBipartite(int[][] graph) {

        // 每个节点都有一个介于 0 到 n - 1 之间的唯一编号
        // 定义 visited 数组，记录访问每个节点时应该设置为什么颜色
        // 每个索引的元素值代表当前索引（节点）的访问情况
```



```

// 初始值为 0 表示未被访问
// 被访问的时候根据情况赋值为 1 或者 -1
// 根据动画演示
// 1 表示黄色
// -1 表示绿色
int[] visited = new int[graph.length];

// 采取深度优先遍历的方式对 graph 里面的每个节点进行访问
for (int i = 0; i < graph.length; i++) {
    // 只有当前这个节点处于未被渲染的状态，才有资格对它进行处理
    if (visited[i] == 0) {
        // 在访问过程中，如果出现了去设置某个节点的颜色时产生了冲突，那么就直接返回结果
        // 比如 5 这个节点之前被渲染为黄色
        // 再处理其它节点的时候，重新渲染 5，需要把它渲染为绿色
        // 此时，产生了矛盾，表示无法分割成独立的子集
        if(dfs(graph, i, 1, visited) == false){
            return false;
        }
    }
}
return true;
}

// 图深度优先遍历
// 返回值：是否会出现矛盾的节点
// graph：二维矩阵
// v：节点值，同时也是数组 visited 里面的索引值
// color：设置的颜色 1 表示黄色 -1 表示绿色
// visited：记录访问每个节点时应该设置为什么颜色
private boolean dfs(int[][] graph, int v, int color, int[] visited) {

    // 如果当前 v 这个节点已经被染色了
    if (visited[v] != 0) {

        // 判断它的颜色是否与本次要染的颜色相同
        if( visited[v] == color){
            // 如果相同，说明此无向图可以被正确染色
            return true;
        }else{
            // 如果矛盾，说明此无向图无法被正确染色
            return false;
        }
    }

    // 否则，说明当前 v 这个节点还没有被染色
    // 对当前顶点进行染色，设置为 color
    visited[v] = color;

    // 再对和它相邻的所有节点都设置为与 color 相反的颜色

```

```

for (int w: graph[v]) {

    // 每个相邻的节点都使用 dfs 进行渲染为 -color 的颜色
    // 在渲染过程中，记录是否可以正确染色
    boolean res = dfs(graph, w, -color, visited);

    // 如果发现无法正确染色
    if(res == false){

        // 返回 false
        return false;

    }

}

// 否则说明以当前 v 这个节点进行深度优先遍历的方式访问的所有节点
// 均没有出现矛盾的现象
// 可以返回 true
return true;
}
}

```

Java

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：278166530
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 广度优先遍历的解法
class Solution {
    public boolean isBipartite(int[][] graph) {

        // 每个节点都有一个介于 0 到 n - 1 之间的唯一编号
        // 定义 visited 数组，记录访问每个节点时应该设置为什么颜色
        // 每个索引的元素值代表当前索引（节点）的访问情况
        // 初始值为 0 表示未被访问
        // 被访问的时候根据情况赋值为 1 或者 -1
        // 根据动画演示
        // 1 表示黄色
        // -1 表示蓝色
        int[] visited = new int[graph.length];

        // 借助队列实现广度优先遍历
        Queue<Integer> queue = new LinkedList<>(); //广度搜索队列

        // 采取广度优先遍历的方式对 graph 里面的每个节点进行访问
        // 一层一层访问
    }
}

```

```

for (int i = 0; i < graph.length; i++) {

    // 只有当前这个节点处于未被渲染的状态，才有资格对它进行处理
    // 如果当前这个节点已经被渲染了，那么去查看其它的节点
    if (visited[i] != 0) {
        continue;
    }

    // 否则，把当前节点加入到队列里面
    queue.offer(i);

    // 同时设置当前节点的颜色
    // 为什么这里设置为 1，为黄色呢？
    // 如果发现当前这个节点还不是某个节点的相邻节点，就设置为黄色
    visited[i] = 1;

    // 广度优先遍历
    while (!queue.isEmpty()) {

        // 取出队头元素
        int v = queue.poll();

        // graph[v]存放的是与节点 v 相连的所有节点
        // 访问这些节点
        for (int w: graph[v]) {

            // 如果当前节点的某个邻接点已经被染过色了
            // 且颜色和当前顶点相同，产生了矛盾，说明此无向图无法被正确染色
            if (visited[w] == visited[v]) {
                // 返回 false
                // 无需继续访问其它节点了
                return false;
            }

            // 如果当前节点的某个邻接点没有被染过色了
            if (visited[w] == 0) {
                // 把它设置为何当前节点相反的颜色
                visited[w] = -visited[v];
                // 同时把它也加入到队列当中
                queue.offer(w);
            }
        }
    }
}

return true;
}
}

```

2、**C++** 代码

3、Python 代码

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

微信：278166530

代码有看不懂的地方一定要私聊咨询吴师兄呀

广度优先遍历的解法

class Solution:

def isBipartite(self, graph: List[List[int]]) -> bool:

每个节点都有一个介于 0 到 n - 1 之间的唯一编号

定义 visited 数组，记录访问每个节点时应该设置为什么颜色

每个索引的元素值代表当前索引（节点）的访问情况

初始值为 0 表示未被访问

被访问的时候根据情况赋值为 1 或者 -1

根据动画演示

1 表示黄色

-1 表示黄色

n = len(graph)

visited = [0] * n

借助队列实现广度优先遍历

queue = collections.deque()

采取广度优先遍历的方式对 graph 里面的每个节点进行访问

一层一层访问

for i in range(n):

只有当前这个节点处于未被渲染的状态，才有资格对它进行处理

如果当前这个节点已经被渲染了，那么去查看其它的节点

if visited[i] != 0 :

continue

否则，把当前节点加入到队列里面

queue.append(i)

同时设置当前节点的颜色

为什么这里设置为 1，为黄色呢？

如果发现当前这个节点还不是某个节点的相邻节点，就设置为黄色

visited[i] = 1

广度优先遍历

while queue:

```

# 取出队头元素
v = queue.popleft()

# graph[v]存放的是与节点 v 相连的所有节点
# 访问这些节点
for w in graph[v]:

    # 如果当前节点的某个邻接点已经被染过色了
    # 且颜色和当前顶点相同，产生了矛盾，说明此无向图无法被正确染色
    if visited[w] == visited[v] :
        # 返回 false
        # 无需继续访问其它节点了
        return False

    # 如果当前节点的某个邻接点没有被染过色了
    if visited[w] == 0 :
        # 把它设置为何当前节点相反的颜色
        visited[w] = -visited[v]
        # 同时把它也加入到队列当中
        queue.append(w)

return True

```

登录 AlgoMooc 官网获取更多算法图解

<https://www.algomooc.com>

作者：程序员吴师兄

微信：278166530

代码有看不懂的地方一定要私聊咨询吴师兄呀

深度优先遍历的解法

```

class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:

        # 每个节点都有一个介于 0 到 n - 1 之间的唯一编号
        # 定义 visited 数组，记录访问每个节点时应该设置为什么颜色
        # 每个索引的元素值代表当前索引（节点）的访问情况
        # 初始值为 0 表示未被访问
        # 被访问的时候根据情况赋值为 1 或者 -1
        # 根据动画演示
        # 1 表示黄色
        # -1 表示绿色
        n = len(graph)
        visited = [0] * n

        # 采取深度优先遍历的方式对 graph 里面的每个节点进行访问
        for i in range(n):
            # 只有当前这个节点处于未被渲染的状态，才有资格对它进行处理
            if visited[i] == 0 :
                # 在访问过程中，如果出现了去设置某个节点的颜色时产生了冲突，那么就直接返回结果

```

```

        # 比如 5 这个节点之前被渲染为黄色
        # 再处理其它节点的时候, 重新渲染 5, 需要把它渲染为绿色
        # 此时, 产生了矛盾, 表示无法分割成独立的子集
        if self.dfs(graph, i, 1, visited) == False :
            return False

    return True

# 图深度优先遍历
# 返回值: 是否会出现矛盾的节点
# graph: 二维矩阵
# v: 节点值, 同时也是数组 visited 里面的索引值
# color: 设置的颜色 1 表示黄色 -1 表示绿色
# visited: 记录访问每个节点时应该设置为什么颜色
def dfs(self, graph, v, color, visited) -> bool:

    # 如果当前 v 这个节点已经被染色了
    if visited[v] != 0 :

        # 判断它的颜色是否与本次要染的颜色相同
        if visited[v] == color:
            # 如果相同, 说明此无向图可以被正确染色
            return True
        else:
            # 如果矛盾, 说明此无向图无法被正确染色
            return False

    # 否则, 说明当前 v 这个节点还没有被染色
    # 对当前顶点进行染色, 设置为 color
    visited[v] = color

    # 再对和它相邻的所有节点都设置为与 color 相反的颜色
    for w in graph[v]:

        # 每个相邻的节点都使用 dfs 进行渲染为 -color 的颜色
        # 在渲染过程中, 记录是否可以正确染色
        res = self.dfs(graph, w, -color, visited)

        # 如果发现无法正确染色
        if res == False :

            # 返回 false
            return False

```

```
# 否则说明以当前 v 这个节点进行深度优先遍历的方式访问的所有节点
# 均没有出现矛盾的现象
# 可以返回 true
return True
```

第四十三天

LeetCode 509、斐波那契数

一、题目描述

斐波那契数（通常用 $F(n)$ 表示）形成的序列称为 **斐波那契数列**。该数列由 0 和 1 开始，后面的每一项数字都是前面两项数字的和。也就是：

$F(0) = 0$, $F(1) = 1$ $F(n) = F(n - 1) + F(n - 2)$, 其中 $n > 1$

给定 n ，请计算 $F(n)$ 。

示例 1：

输入： $n = 2$ 输出：1 解释： $F(2) = F(1) + F(0) = 1 + 0 = 1$

示例 2：

输入： $n = 3$ 输出：2 解释： $F(3) = F(2) + F(1) = 1 + 1 = 2$

示例 3：

输入： $n = 4$ 输出：3 解释： $F(4) = F(3) + F(2) = 2 + 1 = 3$

提示：

- $0 \leq n \leq 30$

二、题目解析

三、参考代码

1. Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 斐波那契数 (LeetCode 509) : https://leetcode.cn/problems/fibonacci-number/
class Solution {
```

```

int times1;

public int fib(int N) {

    times1 = 0;

    int result = myFib1(N);

    return result ;

}

private int myFib1( int N){

    times1++;

    if ( N == 0 ) return 0 ;

    if ( N == 1 ) return 1 ;

    return myFib1( N - 1 ) + myFib1( N - 2);

}

}

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 斐波那契数 (LeetCode 509) : https://leetcode.cn/problems/fibonacci-number/
class Solution {

    int times1;

    int[] memo;

    public int fib(int N) {

        times1 = 0;

        memo = new int[ N + 1 ];

        Arrays.fill(memo, -1);

        int result = myFib2(N);

        System.out.println("myFib2 调用的次数为 : " + times1 );

        return result ;

    }
}

```



```

}

// 记忆化搜索
// 在递归的基础上增加了一个记忆的功能，依旧是自上向下的解决问题
// 相当于数组从后向前挖坑，挖到最前面在开始填坑
private int myFib2( int N){

    times1++;

    if ( N == 0 ) return 0 ;

    if ( N == 1 ) return 1 ;

    if ( memo[N] == -1) {
        memo[N] = myFib2( N - 1 ) + myFib2( N - 2);
    }
    return memo[N];
}

}

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 斐波那契数 (LeetCode 509) : https://leetcode.cn/problems/fibonacci-number/
class Solution {

    int times1;

    int[] memo;

    public int fib(int N) {

        times1 = 0;

        memo = new int[ N + 1 ];

        Arrays.fill(memo, -1);

        int result = myFib3(N);

        System.out.println("myFib3 调用的次数为 : " + times1 );

        return result ;

    }

}

```

```

// 动态规划：自下向上的解决问题
// 相当于结果数组从前往后进行填充，一个萝卜一个坑
private int myFib3( int N){

    if ( N < 2) return N;

    // 1、确定 dp 数组的含义
    // 即 dp[n] 表示当 n 时是xxxx
    int[] dp = new int[ N + 1];

    // 3、确定 dp 初始状态
    // 初始化默认值、前面几个状态的值
    Arrays.fill(dp,-1);

    dp[0] = 0;

    dp[1] = 1;

    for(int i = 2 ; i <= N ; i++){

        // 2、寻找 dp 数组元素之间的联系
        // 即 dp[i] 的状态可以怎么样通过前面的状态获取到
        dp[i] = dp[i-1] + dp[i-2];
    }

    return dp[N];

}
}

```

1. Python代码

```

class Solution:
    def __init__(self):
        self.times1 = 0

    def fib(self, N):
        self.times1 = 0
        result = self.myFib1(N)
        print(self.times1)
        return result

    def myFib1(self, N):
        self.times1 += 1

        if N == 0:
            return 0

```

```

        if N == 1:
            return 1

        return self.myFib1(N - 1) + self.myFib1(N - 2)
#### 记忆化搜索
class Solution:
    def __init__(self):
        self.times1 = 0
        self.memo = None

    def fib(self, N):
        self.times1 = 0
        self.memo = [-1] * (N + 1)
        result = self.myFib2(N)
        print("myFib2 调用的次数为: ", self.times1)
        return result

    def myFib2(self, N):
        self.times1 += 1

        if N == 0:
            return 0

        if N == 1:
            return 1

        if self.memo[N] == -1:
            self.memo[N] = self.myFib2(N - 1) + self.myFib2(N - 2)

        return self.memo[N]
#### 动态规划
class solution:
    def __init__(self):
        self.times1 = 0
        self.memo = None

    def fib(self, N):
        self.times1 = 0
        self.memo = [-1] * (N + 1)
        result = self.myFib3(N)
        print("myFib3 调用的次数为: ", self.times1)
        return result

    def myFib3(self, N):
        if N < 2:
            return N

        dp = [-1] * (N + 1)

```

```
dp[0] = 0
dp[1] = 1

for i in range(2, N + 1):
    dp[i] = dp[i - 1] + dp[i - 2]

return dp[N]
```

LeetCode 62、不同路径

一、题目描述

一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为“Start”）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。

问总共有多少条不同的路径？

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 不同路径（LeetCode 62）：https://leetcode-cn.com/problems/unique-paths/
class Solution {
    public int uniquePaths(int m, int n) {
        // 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        // dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        // dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        // dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
        int[][] dp = new int[m][n];

        // 初始化 dp[0][0],
        // dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        // 仅此一条，别无分路
        dp[0][0] = 1;

        // i 从 1 遍历到 m - 1
        // 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // 由于每次只能向下或者向右移动一步，此时只能向下移动一步
        // 所以，只能一直向下走，只有这一条路径
        for(int i = 1; i < m ; i++){
```

```

        // 仅此一条，别无分路
        dp[i][0] = 1;
    }

    // j 从 1 遍历到 n - 1
    // 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
    // 由于每次只能向下或者向右移动一步，此时只能向右移动一步
    // 所以，只能一直向右走，只有这一条路径
    for(int j = 1; j < n ; j++){
        // 仅此一条，别无分路
        dp[0][j] = 1;
    }

    // 接下来从第 1 行到第 m - 1 行
    // 从第 1 列到 n - 1 列
    // 填充二维数组 dp 里面的值
    // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
    for (int i = 1; i < m; i++) {

        for (int j = 1; j < n; j++) {
            // 由于每次只能向下或者向右移动一步
            // 位置 (i,j) 的不同路径的数量是由
            // 1、上边位置 dp[ i - 1 ][j] 的不同路径的数量
            // 2、左边位置 dp[i][ j - 1 ] 的不同路径的数量
            // 两者之和获取到的
            dp[i][j] = dp[ i - 1 ][j] + dp[i][ j - 1 ];
        }
    }

    // dp[ m - 1 ][ n - 1 ] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
    // 即到达终点的数量
    // 返回这个结果即可
    return dp[ m - 1 ][ n - 1 ];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        // 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        // dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        // dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        // dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
        auto dp = vector < vector < int>> ( m ,vector<int> ( n));

        // 初始化 dp[0][0],
    }
}

```

```

// dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
// 仅此一条，别无分路
dp[0][0] = 1;

// i 从 1 遍历到 m - 1
// 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
// 由于每次只能向下或者向右移动一步，此时只能向下移动一步
// 所以，只能一直向下走，只有这一条路径
for(int i = 1; i < m ; i++){
    // 仅此一条，别无分路
    dp[i][0] = 1;
}

// j 从 1 遍历到 n - 1
// 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
// 由于每次只能向下或者向右移动一步，此时只能向右移动一步
// 所以，只能一直向右走，只有这一条路径
for(int j = 1; j < n ; j++){
    // 仅此一条，别无分路
    dp[0][j] = 1;
}

// 接下来从第 1 行到第 m - 1 行
// 从第 1 列到 n - 1 列
// 填充二维数组 dp 里面的值
// dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
for (int i = 1; i < m; i++) {

    for (int j = 1; j < n; j++) {
        // 由于每次只能向下或者向右移动一步
        // 位置 (i,j) 的不同路径的数量是由
        // 1、上边位置 dp[ i - 1 ][j] 的不同路径的数量
        // 2、左边位置 dp[i][ j - 1 ] 的不同路径的数量
        // 两者之和获取到的
        dp[i][j] = dp[ i - 1 ][j] + dp[i][ j - 1 ];
    }
}

// dp[ m - 1 ][ n - 1 ] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
// 即到达终点的数量
// 返回这个结果即可
return dp[ m - 1 ][ n - 1 ];
}
};

```

3、Python 代码

```
class solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        # dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        # dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        # dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        # dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
        dp = [[0] * n for _ in range(m)]

        # 初始化 dp[0][0],
        # dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        # 仅此一条, 别无分路
        dp[0][0] = 1

        # i 从 1 遍历到 m - 1
        # 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        # 由于每次只能向下或者向右移动一步, 此时只能向下移动一步
        # 所以, 只能一直向下走, 只有这一条路径
        for i in range(1, m):
            # 仅此一条, 别无分路
            dp[i][0] = 1

        # j 从 1 遍历到 n - 1
        # 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        # 由于每次只能向下或者向右移动一步, 此时只能向右移动一步
        # 所以, 只能一直向右走, 只有这一条路径
        for j in range(1, n):
            # 仅此一条, 别无分路
            dp[0][j] = 1

        # 接下来从第 1 行到第 m - 1 行
        # 从第 1 列到 n - 1 列
        # 填充二维数组 dp 里面的值
        # dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
        for i in range(1, m):
            for j in range(1, n):
                # 由于每次只能向下或者向右移动一步
                # 位置 (i,j) 的不同路径的数量是由
                # 1、上边位置 dp[i - 1][j] 的不同路径的数量
                # 2、左边位置 dp[i][j - 1] 的不同路径的数量
                # 两者之和获取到的
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        # dp[m - 1][n - 1] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
```

```
# 即到达终点的数量
# 返回这个结果即可
return dp[ m - 1 ][ n - 1 ]
```

LeetCode 63、不同路径II

一、题目描述

一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为“Start”）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。

现在考虑网格中有障碍物。问总共有多少条不同的路径？

网格中的障碍物和空位置分别用 1 和 0 来表示。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 不同路径II ( LeetCode 63 ) : https://leetcode-cn.com/problems/unique-paths-ii/
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        // 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        // dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        // dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        // dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量

        // m 行
        int m = obstacleGrid.length;

        // n 列
        int n = obstacleGrid[0].length;

        int[][] dp = new int[m][n];

        // i 从 1 遍历到 m - 1
        // 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // 由于每次只能向下或者向右移动一步，此时只能向下移动一步
        // 所以，只能一直向下走，只有这一条路径
        for(int i = 0; i < m ; i++){
            // 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0
```



```

        if(obstacleGrid[i][0] == 1) break;

        // 仅此一条，别无分路
        dp[i][0] = 1;

    }

    // j 从 1 遍历到 n - 1
    // 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
    // 由于每次只能向下或者向右移动一步，此时只能向右移动一步
    // 所以，只能一直向右走，只有这一条路径
    for(int j = 0; j < n ; j++){
        // 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0
        if(obstacleGrid[0][j] == 1) break;

        // 仅此一条，别无分路
        dp[0][j] = 1;

    }

    // 接下来从第 1 行到第 m - 1 行
    // 从第 1 列到 n - 1 列
    // 填充二维数组 dp 里面的值
    // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
    for (int i = 1; i < m; i++) {

        for (int j = 1; j < n; j++) {
            // 由于每次只能向下或者向右移动一步

            // 如果此时出现了障碍，那么由于无法到达这个位置，因此不用处理
            if (obstacleGrid[i][j] == 1) continue;

            // 位置 (i,j) 的不同路径的数量是由
            // 1、上边位置 dp[ i - 1 ][j] 的不同路径的数量
            // 2、左边位置 dp[i][ j - 1 ] 的不同路径的数量
            // 两者之和获取到的
            dp[i][j] = dp[ i - 1 ][j] + dp[i][ j - 1 ];
        }
    }

    // dp[ m - 1 ][ n - 1 ] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
    // 即到达终点的数量
    // 返回这个结果即可
    return dp[ m - 1 ][ n - 1 ];
}
}

```

2、**C++** 代码

```
class solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
        // 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        // dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        // dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        // dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量

        // m 行
        int m = obstacleGrid.size();

        // n 列
        int n = obstacleGrid[0].size();

        auto dp = vector < vector < int>> ( m ,vector<int> ( n));

        // i 从 1 遍历到 m - 1
        // 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        // 由于每次只能向下或者向右移动一步，此时只能向下移动一步
        // 所以，只能一直向下走，只有这一条路径
        for(int i = 0; i < m ; i++){
            // 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0
            if(obstacleGrid[i][0] == 1) break;

            // 仅此一条，别无分路
            dp[i][0] = 1;
        }

        // j 从 1 遍历到 n - 1
        // 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        // 由于每次只能向下或者向右移动一步，此时只能向右移动一步
        // 所以，只能一直向右走，只有这一条路径
        for(int j = 0; j < n ; j++){
            // 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0
            if(obstacleGrid[0][j] == 1) break;

            // 仅此一条，别无分路
            dp[0][j] = 1;
        }

        // 接下来从第 1 行到第 m - 1 行
        // 从第 1 列到 n - 1 列
        // 填充二维数组 dp 里面的值
        // dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
    }
};
```

```

for (int i = 1; i < m; i++) {

    for (int j = 1; j < n; j++) {
        // 由于每次只能向下或者向右移动一步

        // 如果此时出现了障碍，那么由于无法到达这个位置，因此不用处理
        if (obstacleGrid[i][j] == 1) continue;

        // 位置 (i,j) 的不同路径的数量是由
        // 1、上边位置 dp[ i - 1 ][j] 的不同路径的数量
        // 2、左边位置 dp[i][ j - 1 ] 的不同路径的数量
        // 两者之和获取到的
        dp[i][j] = dp[ i - 1 ][j] + dp[i][ j - 1 ];
    }
}

// dp[ m - 1 ][ n - 1 ] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
// 即到达终点的数量
// 返回这个结果即可
return dp[ m - 1 ][ n - 1 ];

}
};

```

3、Python 代码

```

class Solution:
    def uniquePathsWithObstacles(self, obstacleGrid: List[List[int]]) -> int:
        # 设置二维数组 dp 用来储存到达每个位置时不同路径的数量
        # dp[0][0] 表示从第 0 行第 0 列到达第 0 行第 0 列时不同路径的数量
        # dp[0][i] 表示从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
        # dp[j][0] 表示从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        # dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量

        # m 行
        m = len(obstacleGrid)

        # n 列
        n = len(obstacleGrid[0])

        dp = [[0] * n for _ in range(m)]

        # i 从 0 遍历到 m - 1
        # 获取从第 0 行第 0 列到达第 j 行第 0 列时不同路径的数量
        # 由于每次只能向下或者向右移动一步，此时只能向下移动一步
        # 所以，只能一直向下走，只有这一条路径
        for i in range( 0 , m ):
            # 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0

```

```

if obstacleGrid[i][0] == 1 : break

# 仅此一条，别无分路
dp[i][0] = 1

# j 从 0 遍历到 n - 1
# 获取从第 0 行第 0 列到达第 0 行第 i 列时不同路径的数量
# 由于每次只能向下或者向右移动一步，此时只能向右移动一步
# 所以，只能一直向右走，只有这一条路径
for j in range( 0 , n ):
    # 一旦出现了障碍，那么后面所有的位置都是到达不了的，都是默认的 0
    if obstacleGrid[0][j] == 1 : break

    # 仅此一条，别无分路
    dp[0][j] = 1

# 接下来从第 1 行到第 m - 1 行
# 从第 1 列到 n - 1 列
# 填充二维数组 dp 里面的值
# dp[i][j] 表示从第 0 行第 0 列到达第 i 行第 j 列时不同路径的数量
for i in range( 1 , m ):

    for j in range( 1 , n ):
        # 由于每次只能向下或者向右移动一步

        # 如果此时出现了障碍，那么由于无法到达这个位置，因此不用处理
        if obstacleGrid[i][j] == 1 : continue

        # 位置 (i,j) 的不同路径的数量是由
        # 1、上边位置 dp[ i - 1 ][j] 的不同路径的数量
        # 2、左边位置 dp[i][ j - 1 ] 的不同路径的数量
        # 两者之和获取到的
        dp[i][j] = dp[ i - 1 ][j] + dp[i][ j - 1 ]

# dp[ m - 1 ][ n - 1 ] 表示从第 0 行第 0 列到达第 m - 1 行第 n - 1 列时不同路径的数量
# 即到达终点的数量
# 返回这个结果即可
return dp[ m - 1 ][ n - 1 ]

```

LeetCode 64、最小路径和

一、题目描述

给定一个包含非负整数的 $m \times n$ 网格 grid，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。

说明：每次只能向下或者向右移动一步。

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public int minPathSum(int[][] grid) {

        // m 表示有多少行
        int m = grid.length;

        // n 表示每一行有多少个元素，即 n 表示有多少列
        int n = grid[0].length;

        // 设置二维数组 dp 用来储存每个位置的最优解
        // dp[0][0] 表示第 0 行第 0 列的最优解
        // dp[0][i] 表示第 0 行第 i 列的最优解
        // dp[j][0] 表示第 j 行第 0 列的最优解
        // dp[i][j] 表示第 i 行第 j 列的最优解
        int[][] dp = new int[m][n];

        // 初始化 dp[0][0]，由于只有一个元素
        // 所以 dp[0][0] 的最优解就是 grid[0][0] 这个元素
        dp[0][0] = grid[0][0];

        // i 从 1 遍历到 n - 1
        // 获取第 0 行中第 i 列的最优解
        // 比如 grid[0] 为 [1,3,2...]
        // 由于每次只能向下或者向右移动一步，此时只能向右移动一步
        // 那么 dp[0][i] 依次为 [1,4,6...]
        for(int i = 1; i < n ; i++){
            // 所以对于只有一行的情况，当前位置的最优解等于前一列的最优解加上该列的值
            dp[0][i] = dp[0][i-1] + grid[0][i];
        }

        // i 从 1 遍历到 m - 1
        // 获取第 i 行中第 0 列的最优解
        // 比如 grid 为
        // [1,1,2.....]
        // [5,2,2.....]
        // [2,3,4.....]
```

```

// [...,...,...]
// 由于每次只能向下或者向右移动一步，此时只能向下移动一步
// 那么 dp[i][0] 就是
// [1,.....]
// [6,.....]
// [7,.....]
// [.....]
for(int i = 1; i < m ;i++){
    // 所以对于只有一列的情况，当前位置的最优解等于前一行的最优解加上该行的值
    dp[i][0] = dp[i-1][0] + grid[i][0];
}

// 接下来从第 1 行到第 m - 1 行
// 从第 1 列到底 n - 1 列
// 填充二维数组 dp 里面的值
// dp[i][j] 表示第 i 行第 j 列的最优解
for(int i = 1 ; i < m;i++){
    for(int j = 1;j < n;j++){
        // 由于每次只能向下或者向右移动一步
        // 位置 (i,j) 的最优解
        // 等于当前位置上方位置(i-1,j)的最优解和左侧位置(i,j-1)的最优解的较小值
        // 再加上当前位置的值
        dp[i][j] = Math.min(dp[i-1][j],dp[i][j-1]) + grid[i][j] ;
    }
}

// dp[m-1][n-1] 表示第 m - 1 行第 n - 1 列的最优解
// 返回这个结果即可
return dp[m-1][n-1];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        // m 表示有多少行
        int m = grid.size();

        // n 表示每一行有多少个元素，即 n 表示有多少列
        int n = grid[0].size();

        // 设置二维数组 dp 用来储存每个位置的最优解
        // dp[0][0] 表示第 0 行第 0 列的最优解
        // dp[0][i] 表示第 0 行第 i 列的最优解
        // dp[j][0] 表示第 j 行第 0 列的最优解
        // dp[i][j] 表示第 i 行第 j 列的最优解
        auto dp = vector < vector <int> > (m, vector <int> (n));
    }
};

```

```

// 初始化 dp[0][0], 由于只有一个元素
// 所以 dp[0][0] 的最优解就是 grid[0][0] 这个元素
dp[0][0] = grid[0][0];

// i 从 1 遍历到 n - 1
// 获取第 0 行中第 i 列的最优解
// 比如 grid[0] 为 [1,3,2...]
// 由于每次只能向下或者向右移动一步, 此时只能向右移动一步
// 那么 dp[0][i] 依次为 [1,4,6...]
for(int i = 1; i < n ; i++){
    // 所以对于只有一行的情况, 当前位置的最优解等于前一列的最优解加上该列的值
    dp[0][i] = dp[0][i-1] + grid[0][i];
}

// i 从 1 遍历到 n - 1
// 获取第 i 行中第 0 列的最优解
// 比如 grid 为
// [1,1,2.....]
// [5,2,2.....]
// [2,3,4.....]
// [...,...,...]
// 由于每次只能向下或者向右移动一步, 此时只能向下移动一步
// 那么 dp[i][0] 就是
// [1,.....]
// [6,.....]
// [7,.....]
// [........]
for(int i = 1; i < m ; i++){
    // 所以对于只有一列的情况, 当前位置的最优解等于前一行的最优解加上该行的值
    dp[i][0] = dp[i-1][0] + grid[i][0];
}

// 接下来从第 1 行到第 m - 1 行
// 从第 1 列到底 n - 1 列
// 填充二维数组 dp 里面的值
// dp[i][j] 表示第 i 行第 j 列的最优解
for(int i = 1 ; i < m; i++){
    for(int j = 1; j < n; j++){
        // 由于每次只能向下或者向右移动一步
        // 位置 (i,j) 的最优解
        // 等于当前位置上方位置(i-1,j)的最优解和左侧位置(i,j-1)的最优解的较小值
        // 再加上当前位置的值
        dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j] ;
    }
}

// dp[m-1][n-1] 表示第 m - 1 行第 n - 1 列的最优解
// 返回这个结果即可

```

```

        return dp[m-1][n-1];
    }
};

```

3、Python 代码

```

class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        # m 表示有多少行
        m = len(grid)

        # n 表示每一行有多少个元素，即 n 表示有多少列
        n = len(grid[0])

        # 设置二维数组 dp 用来储存每个位置的最优解
        # dp[0][0] 表示第 0 行第 0 列的最优解
        # dp[0][i] 表示第 0 行第 i 列的最优解
        # dp[j][0] 表示第 j 行第 0 列的最优解
        # dp[i][j] 表示第 i 行第 j 列的最优解
        dp = [[0] * n for _ in range(m)]

        # 初始化 dp[0][0]，由于只有一个元素
        # 所以 dp[0][0] 的最优解就是 grid[0][0] 这个元素
        dp[0][0] = grid[0][0]

        # i 从 1 遍历到 n - 1
        # 获取第 0 行中第 i 列的最优解
        # 比如 grid[0] 为 [1,3,2...]
        # 由于每次只能向下或者向右移动一步，此时只能向右移动一步
        # 那么 dp[0][i] 依次为 [1,4,6...]
        for i in range(1, n):
            # 所以对于只有一行的情况，当前位置的最优解等于前一列的最优解加上该列的值
            dp[0][i] = dp[0][i-1] + grid[0][i]

        # i 从 1 遍历到 n - 1
        # 获取第 i 行中第 0 列的最优解
        # 比如 grid 为
        # [1,1,2.....]
        # [5,2,2.....]
        # [2,3,4.....]
        # [...,...,...]
        # 由于每次只能向下或者向右移动一步，此时只能向下移动一步
        # 那么 dp[i][0] 就是
        # [1,.....]
        # [6,.....]
        # [7,.....]
        # [........]
        for i in range(1, m):

```



```

# 所以对于只有一列的情况，当前位置的最优解等于前一行的最优解加上该行的值
dp[i][0] = dp[i-1][0] + grid[i][0]

# 接下来从第 1 行到第 m - 1 行
# 从第 1 列到底 n - 1 列
# 填充二维数组 dp 里面的值
# dp[i][j] 表示第 i 行第 j 列的最优解
for i in range(1, m):
    for j in range(1, n):
        # 由于每次只能向下或者向右移动一步
        # 位置 (i,j) 的最优解
        # 等于当前位置上方位置(i-1,j)的最优解和左侧位置(i,j-1)的最优解的较小值
        # 再加上当前位置的值
        dp[i][j] = min(dp[i-1][j],dp[i][j-1]) + grid[i][j]

# dp[m-1][n-1] 表示第 m - 1 行第 n - 1 列的最优解
# 返回这个结果即可
return dp[m-1][n-1]

```

LeetCode 70、爬楼梯

一、题目描述

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

示例 1：

输入：n = 2 输出：2 解释：有两种方法可以爬到楼顶。

1. 1 阶 + 1 阶
2. 2 阶

示例 2：

输入：n = 3 输出：3 解释：有三种方法可以爬到楼顶。

1. 1 阶 + 1 阶 + 1 阶
2. 1 阶 + 2 阶
3. 2 阶 + 1 阶

提示：

1 <= n <= 45

二、题目解析

三、参考代码

1. Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {
    public int climbStairs(int n) {

        return calcways(n);

    }

    private int calcways( int n ) {

        if( n == 1 ){
            return 1 ;
        }

        if( n == 2 ){
            return 2;
        }

        return calcways( n - 1 ) + calcways( n - 2);

    }
}

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class Solution {

    int[] memo;
    public int climbStairs(int n) {

        memo = new int[ n + 1 ];

        Arrays.fill(memo , -1);

        return calcways(n);

    }
}
```

```

private int calcways( int n ) {

    if( n == 1 ){
        return 1 ;
    }

    if( n == 2 ){
        return 2;
    }

    if( memo[n] == -1){
        memo[n] = calcways( n - 1 ) + calcways( n - 2);
    }

    return memo[n];

}
}

class Solution {

    int[] memo;
    public int climbStairs(int n) {

        if( n== 1) return 1;

        if( n == 2 ) return 2;

        memo = new int[ n + 1 ];

        memo[1] = 1;

        memo[2] = 2;

        for( int i = 3 ; i <= n ; i++ ){

            memo[i] = memo[i - 1] + memo[ i - 2];

        }

        return memo[n];

    }

}

```

1. Python 代码

```
class Solution:
    def climbStairs(self, n):
        return self.calcways(n)

    def calcways(self, n):
        if n == 1:
            return 1
        if n == 2:
            return 2
        return self.calcways(n - 1) + self.calcways(n - 2)

class Solution:
    def climbStairs(self, n):
        self.memo = [-1] * (n + 1)
        return self.calcways(n)

    def calcways(self, n):
        if n == 1:
            return 1
        if n == 2:
            return 2
        if self.memo[n] == -1:
            self.memo[n] = self.calcways(n - 1) + self.calcways(n - 2)
        return self.memo[n]

class Solution:
    def climbStairs(self, n):
        if n == 1:
            return 1
        if n == 2:
            return 2

        memo = [0] * (n + 1)
        memo[1] = 1
        memo[2] = 2

        for i in range(3, n + 1):
            memo[i] = memo[i - 1] + memo[i - 2]

        return memo[n]
```

LeetCode 1143、最长公共子序列

一、题目描述

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列**，返回 `0`。

一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

- 例如，`"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。

两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

```
输入: text1 = "abcde", text2 = "ace"
输出: 3
解释: 最长公共子序列是 "ace"，它的长度为 3。
```

示例 2：

```
输入: text1 = "abc", text2 = "abc"
输出: 3
解释: 最长公共子序列是 "abc"，它的长度为 3。
```

提示：

- `1 <= text1.length, text2.length <= 1000`
- `text1` 和 `text2` 仅由小写英文字符组成。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最长公共子序列（LeetCode 1143）：https://leetcode-cn.com/problems/longest-common-subsequence
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {

        // 获取字符串 text1 的长度
        int m = text1.length();

        // 获取字符串 text2 的长度
        int n = text2.length();
```

```
// 设置数组 dp, 用来存储 text1 和 text2 最长公共子序列的长度
// dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
// dp[2][3] 表示 text1 前 2 个字符和 text2 前 3 个字符的最长公共子序列的长度
// dp[i][j] 表示 text1 前 i 个字符和 text2 前 j 个字符的最长公共子序列的长度
// 前 i 个字符的区间为 [ 0 , i - 1 ]
// dp[m][n] 表示 text1 前 m 个字符和 text2 前 n 个字符的最长公共子序列的长度
// 前 m 个字符的表示区间为 [ 0 ,m ], 前 n 个字符的表示区间为 [ 0 ,n ]
// 因此, dp 数组的长度为 m + 1 和 n + 1
int[][] dp = new int[m + 1][n + 1];
```

```
// dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
// 此时, text1 没有字符, text2 也没有字符
// 因此, 最长公共子序列的长度为 0
dp[0][0] = 0;
```

```
// 并且, text1 没有字符 或者 text2 没有字符时, 最长公共子序列的长度都为 0
for( int i = 0 ; i <= m ; i++){
    dp[i][0] = 0;
}
```

```
for( int j = 0 ; j <= n ; j++){
    dp[0][j] = 0;
}
```

```
// i 从 1 开始, 直到 m 位置, 遍历 text1 的前 i 个字符
for (int i = 1; i <= m; ++i) {
```

```
    // j 从 1 开始, 直到 n 位置, 遍历 text2 的前 j 个字符
    for (int j = 1; j <= n; ++j) {
```

```
        // 如果发现 text1 的当前字符, 即位置为 i - 1 的字符
        // 与 text2 的当前字符, 即位置为 j - 1 的字符【相同】
        // 此时, 找到了一个公共字符, 最长公共子序列的长度加 1
        if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
```

度

的最长公共子序列的长度

即可

```
            dp[i][j] = dp[i - 1][j - 1] + 1;
```

```
        // 否则, 说明 text1 的当前字符, 即位置为 i - 1 的字符
        // 与 text2 的当前字符, 即位置为 j - 1 的字符【不相同】
    } else {
```

```

        // dp[i - 1][j] 表示 text1 前 i - 1 个字符和 text2 前 j 个字符的最长公共
子序列的长度
        // dp[i][j - 1] 表示 text1 前 i 个字符和 text2 前 j - 1 个字符的最长公共
子序列的长度

        // 需要判断这两者谁更大
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
}
}
// 最后返回结果
return dp[m][n];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        // 获取字符串 text1 的长度
        int m = text1.length();

        // 获取字符串 text2 的长度
        int n = text2.length();

        // 设置数组 dp, 用来存储 text1 和 text2 最长公共子序列的长度
        // dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
        // dp[2][3] 表示 text1 前 2 个字符和 text2 前 3 个字符的最长公共子序列的长度
        // dp[i][j] 表示 text1 前 i 个字符和 text2 前 j 个字符的最长公共子序列的长度
        // 前 i 个字符的区间为 [ 0 , i - 1 ]
        // dp[m][n] 表示 text1 前 m 个字符和 text2 前 n 个字符的最长公共子序列的长度
        // 前 m 个字符的表示区间为 [ 0 , m ], 前 n 个字符的表示区间为 [ 0 , n ]
        // 因此, dp 数组的长度为 m + 1 和 n + 1
        auto dp = vector < vector < int>> ( m + 1 ,vector<int> ( n + 1 ));

        // dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
        // 此时, text1 没有字符, text2 也没有字符
        // 因此, 最长公共子序列的长度为 0
        dp[0][0] = 0;

        // 并且, text1 没有字符 或者 text2 没有字符时, 最长公共子序列的长度都为 0
        for( int i = 0 ; i <= m ; i++){
            dp[i][0] = 0;
        }

        for( int j = 0 ; j <= n ; j++){
            dp[0][j] = 0;
        }
    }
}

```

```

// i 从 1 开始, 直到 m 位置, 遍历 text1 的前 i 个字符
for (int i = 1; i <= m; ++i) {

    // j 从 1 开始, 直到 n 位置, 遍历 text2 的前 j 个字符
    for (int j = 1; j <= n; ++j) {

        // 如果发现 text1 的当前字符, 即位置为 i - 1 的字符
        // 与 text2 的当前字符, 即位置为 j - 1 的字符【相同】
        // 此时, 找到了一个公共字符, 最长公共子序列的长度加 1
        if (text1[i - 1] == text2[j - 1]) {

            // dp[i][j] 表示 text1 前 i 个字符和 text2 前 j 个字符的最长公共子序列的长
            // dp[i - 1][j - 1] 表示 text1 前 i - 1 个字符和 text2 前 j - 1 个字符
            // dp[i][j] 在 dp[i - 1][j - 1] 的基础上又找到了一个公共字符, 只需要加 1
            dp[i][j] = dp[i - 1][j - 1] + 1;

            // 否则, 说明 text1 的当前字符, 即位置为 i - 1 的字符
            // 与 text2 的当前字符, 即位置为 j - 1 的字符【不相同】
        } else {

            // dp[i - 1][j] 表示 text1 前 i - 1 个字符和 text2 前 j 个字符的最长公共
            // dp[i][j - 1] 表示 text1 前 i 个字符和 text2 前 j - 1 个字符的最长公共
            // 需要判断这两者谁更大
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
}
// 最后返回结果
return dp[m][n];
};

```

3、Python 代码

```

class solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        # 获取字符串 text1 的长度
        m = len(text1)

        # 获取字符串 text2 的长度
        n = len(text2)

        # 设置数组 dp, 用来存储 text1 和 text2 最长公共子序列的长度

```



```

# dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
# dp[2][3] 表示 text1 前 2 个字符和 text2 前 3 个字符的最长公共子序列的长度
# dp[i][j] 表示 text1 前 i 个字符和 text2 前 j 个字符的最长公共子序列的长度
# 前 i 个字符的区间为 [ 0 , i - 1 ]
# dp[m][n] 表示 text1 前 m 个字符和 text2 前 n 个字符的最长公共子序列的长度
# 前 m 个字符的表示区间为 [ 0 , m ], 前 n 个字符的表示区间为 [ 0 , n ]
# 因此, dp 数组的长度为 m + 1 和 n + 1
dp = [[0] * (n + 1) for _ in range( m + 1 )]

```

```

# dp[0][0] 表示 text1 前 0 个字符和 text2 前 0 个字符的最长公共子序列的长度
# 此时, text1 没有字符, text2 也没有字符
# 因此, 最长公共子序列的长度为 0
dp[0][0] = 0

```

```

# 并且, text1 没有字符 或者 text2 没有字符时, 最长公共子序列的长度都为 0
for i in range( 0 , m + 1 ) :
    dp[i][0] = 0

for j in range( 0 , n + 1 ) :
    dp[0][j] = 0

```

```

# i 从 1 开始, 直到 m 位置, 遍历 text1 的前 i 个字符
for i in range( 1 , m + 1 ) :

```

```

    # j 从 1 开始, 直到 n 位置, 遍历 text2 的前 j 个字符
    for j in range( 1 , n + 1 ) :

```

```

        # 如果发现 text1 的当前字符, 即位置为 i - 1 的字符
        # 与 text2 的当前字符, 即位置为 j - 1 的字符【相同】
        # 此时, 找到了一个公共字符, 最长公共子序列的长度加 1

```

```

        if text1[i - 1] == text2[j - 1] :
            # dp[i][j] 表示 text1 前 i 个字符和 text2 前 j 个字符的最长公共子序列的长

```

度

```

            # dp[i - 1][j - 1] 表示 text1 前 i - 1 个字符和 text2 前 j - 1 个字符的

```

最长公共子序列的长度

```

            # dp[i][j] 在 dp[i - 1][j - 1] 的基础上又找到了一个公共字符, 只需要加 1

```

即可

```

            dp[i][j] = dp[i - 1][j - 1] + 1

```

```

        # 否则, 说明 text1 的当前字符, 即位置为 i - 1 的字符
        # 与 text2 的当前字符, 即位置为 j - 1 的字符【不相同】

```

```

        else:

```

序列的长度

```

            # dp[i - 1][j] 表示 text1 前 i - 1 个字符和 text2 前 j 个字符的最长公共子

```

序列的长度

```

            # dp[i][j - 1] 表示 text1 前 i 个字符和 text2 前 j - 1 个字符的最长公共子

```

```

            # 需要判断这两者谁更大

```

```

            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

```

```
# 最后返回结果
return dp[m][n]
```

LeetCode 300、最长递增子序列

一、题目描述

给你一个整数数组 `nums`，找到其中最长严格递增子序列的长度。

子序列是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public int lengthOfLIS(int[] nums) {

        // 设置数组 dp，用来存储 nums 中以每个元素结尾的最长递增序列的程度
        // dp[0] 表示以 nums[0] 结尾的最长递增序列的长度
        // dp[1] 表示以 nums[1] 结尾的最长递增序列的长度
        // dp[i] 表示以 nums[i] 结尾的最长递增序列的长度
        int[] dp = new int[nums.length];

        // 首先将数组 dp 里面的值都初始化为 1
        // 1 表示以当前元素结尾的最长递增序列的长度为 1
        // 即最长递增序列就是当前元素本身
        Arrays.fill(dp, 1);

        // 设置一个变量用来存储最长递增序列的结果
        int maxLength = 1;

        // 从 1 开始，直到 dp.length，往 dp 里面填充结果
        for(int i = 1; i < dp.length; i++){

            // 对于 dp[i] 来说，在 nums 中从 0 到 i - 1 都是在 i 的前面的
            // 比如 nums 为 [1,5,2,5,3,7,2]
            // 此时 i 为 3，那么 1,5,2 这些数字都在索引位置为 3 的前面
            // 从 0 遍历到 i - 1，就可以从这些数字中选出小于 i 的数字
            for(int j = 0; j < i; j++){

                // 索引      0  1  2  3  4  5  6
                // nums 为 [ 1, 5, 2, 5, 3, 7, 2 ]
                // 此时 i 为 3，那么 1,5,2 这些数字都在索引位置为 3 的前面
                // 1、从这些数字中选出小于 nums[i] 的数字
                // 2、小于 nums[i] 的那些数字是 1，2，在之前都已经计算过以 1 或者 2 结尾的最长
                // 递增序列的长度
            }
        }
    }
}
```

```

        // 并且这个结果存放在 dp[j] 中
        // 如果数字为 1, 那么此时 j 为 0, dp[0] 存放了以 1 结尾的最长递增序列的长度
        // 如果数字为 2, 那么此时 j 为 2, dp[2] 存放了以 2 结尾的最长递增序列的长度
        // 3、如果发现此时 dp[i] 小于了 dp[j] + 1
        // 4、说明此时 dp[i] 中的值就不是以 nums[i] 结尾的最长递增序列的长度
        // 需要更新 dp[i]
        if(nums[i] > nums[j] && dp[i] < dp[j] + 1){
            // 4、更新 dp[i]
            dp[i] = dp[j] + 1;
        }
    }

    // 在更新 dp[i] 的过程中, 发现了一个更长的子序列
    if(maxLength < dp[i]){
        // 那么把更长的子序列的长度赋值给 maxLength
        maxLength = dp[i];
    }
}

// 最后返回 maxLength 就行
return maxLength;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {

        // 设置数组 dp, 用来存储 nums 中以每个元素结尾的最长递增序列的程度
        // dp[0] 表示以 nums[0] 结尾的最长递增序列的长度
        // dp[1] 表示以 nums[1] 结尾的最长递增序列的长度
        // dp[i] 表示以 nums[i] 结尾的最长递增序列的长度
        // 首先将数组 dp 里面的值都初始化为 1
        // 1 表示以当前元素结尾的最长递增序列的长度为 1
        // 即最长递增序列就是当前元素本身
        vector<int> dp(nums.size(),1);

        // 设置一个变量用来存储最长递增序列的结果
        int maxLength = 1;

        // 从 1 开始, 直到 dp.length, 往 dp 里面填充结果
        for(int i = 1 ; i < dp.size() ; i++){

            // 对于 dp[i] 来说, 在 nums 中从 0 到 i - 1 都是在 i 的前面的
            // 比如 nums 为 [1,5,2,5,3,7,2]
            // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
            // 从 0 遍历到 i - 1, 就可以从这些数字中选出小于 i 的数字

```

```

for(int j = 0; j < i ;j++){

    // 索引      0  1  2  3  4  5  6
    // nums 为 [ 1, 5, 2, 5, 3, 7, 2 ]
    // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
    // 1、从这些数字中选出小于 nums[i] 的数字
    // 2、小于 nums[i] 的那些数字是 1 , 2 , 在之前都已经计算过以 1 或者 2 结尾的最长
递增序列的长度

    // 并且这个结果存放在 dp[j] 中
    // 如果数字为 1, 那么此时 j 为 0, dp[0] 存放了以 1 结尾的最长递增序列的长度
    // 如果数字为 2, 那么此时 j 为 2, dp[2] 存放了以 2 结尾的最长递增序列的长度
    // 3、如果发现此时 dp[i] 小于了 dp[j] + 1
    // 4、说明此时 dp[i] 中的值就不是以 nums[i] 结尾的最长递增序列的长度
    // 需要更新 dp[i]
    if(nums[i] > nums[j] && dp[i] < dp[j] + 1){
        // 4、更新 dp[i]
        dp[i] = dp[j] + 1;
    }
}

// 在更新 dp[i] 的过程中, 发现了一个更长的子序列
if(maxLength < dp[i]){
    // 那么把更长的子序列的长度赋值给 maxLength
    maxLength = dp[i];
}
}

// 最后返回 maxLength 就行
return maxLength;
}
};

```

3、Python 代码

```

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        # 设置数组 dp, 用来存储 nums 中以每个元素结尾的最长递增序列的程度
        # dp[0] 表示以 nums[0] 结尾的最长递增序列的长度
        # dp[1] 表示以 nums[1] 结尾的最长递增序列的长度
        # dp[i] 表示以 nums[i] 结尾的最长递增序列的长度
        # 首先将数组 dp 里面的值都初始化为 1
        # 1 表示以当前元素结尾的最长递增序列的长度为 1
        # 即最长递增序列就是当前元素本身
        dp = [ 1 for _ in range(len(nums))]

        # 设置一个变量用来存储最长递增序列的结果
        maxLength = 1

```

```

# 从 1 开始, 直到 dp.length , 往 dp 里面填充结果
for i in range( 1 , len(dp) ) :

    # 对于 dp[i] 来说, 在 nums 中从 0 到 i - 1 都是在 i 的前面的
    # 比如 nums 为 [1,5,2,5,3,7,2]
    # 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
    # 从 0 遍历到 i - 1, 就可以从这些数字中选出小于 i 的数字
    for j in range ( 0 , i ) :

        # 索引      0  1  2  3  4  5  6
        # nums 为 [ 1, 5, 2, 5, 3, 7, 2 ]
        # 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
        # 1、从这些数字中选出小于 nums[i] 的数字
        # 2、小于 nums[i] 的那些数字是 1 , 2 , 在之前都已经计算过以 1 或者 2 结尾的最长
递增序列的长度

        # 并且这个结果存放在 dp[j] 中
        # 如果数字为 1, 那么此时 j 为 0, dp[0] 存放了以 1 结尾的最长递增序列的长度
        # 如果数字为 2, 那么此时 j 为 2, dp[2] 存放了以 2 结尾的最长递增序列的长度
        # 3、如果发现此时 dp[i] 小于了 dp[j] + 1
        # 4、说明此时 dp[i] 中的值就不是以 nums[i] 结尾的最长递增序列的长度
        # 需要更新 dp[i]
        if nums[i] > nums[j] and dp[i] < dp[j] + 1 :
            # 4、更新 dp[i]
            dp[i] = dp[j] + 1

    # 在更新 dp[i] 的过程中, 发现了一个更长的子序列
    if maxLength < dp[i] :
        # 那么把更长的子序列的长度赋值给 maxLength
        maxLength = dp[i]

# 最后返回 maxLength 就行
return maxLength

```

LeetCode 334、递增的三元组

一、题目描述

给你一个整数数组 `nums` , 判断这个数组中是否存在长度为 3 的递增子序列。

如果存在这样的三元组下标 (i, j, k) 且满足 $i < j < k$, 使得 `nums[i] < nums[j] < nums[k]` , 返回 `true` ; 否则, 返回 `false` 。

示例 1:

输入: `nums = [1,2,3,4,5]` 输出: `true` 解释: 任何 $i < j < k$ 的三元组都满足题意

示例 2:

输入: `nums = [5,4,3,2,1]` 输出: `false` 解释: 不存在满足题意的三元组

示例 3:

输入: nums = [2,1,5,0,4,6] 输出: true 解释: 三元组 (3, 4, 5) 满足题意, 因为 nums[3] == 0 < nums[4] == 4 < nums[5] == 6

提示:

- `1 <= nums.length <= 5 * 105`
- `-231 <= nums[i] <= 231 - 1`

进阶: 你能实现时间复杂度为 `O(n)`, 空间复杂度为 `O(1)` 的解决方案吗?

二、题目解析

三、参考代码

1、Java 代码

```
// 超时代码
class Solution {
    public boolean increasingTriplet(int[] nums) {

        // 设置数组 dp, 用来存储 nums 中以每个元素结尾的最长递增序列的程度
        // dp[0] 表示以 nums[0] 结尾的最长递增序列的长度
        // dp[1] 表示以 nums[1] 结尾的最长递增序列的长度
        // dp[i] 表示以 nums[i] 结尾的最长递增序列的长度
        int[] dp = new int[nums.length];

        // 首先将数组 dp 里面的值都初始化为 1
        // 1 表示以当前元素结尾的最长递增序列的长度为 1
        // 即最长递增序列就是当前元素本身
        Arrays.fill(dp, 1);

        // 设置一个变量用来存储最长递增序列的结果
        int maxLength = 1;

        // 从 1 开始, 直到 dp.length, 往 dp 里面填充结果
        for(int i = 1 ; i < dp.length ; i++){

            // 对于 dp[i] 来说, 在 nums 中从 0 到 i - 1 都是在 i 的前面的
            // 比如 nums 为 [1,5,2,5,3,7,2]
            // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
            // 从 0 遍历到 i - 1, 就可以从这些数字中选出小于 i 的数字
            for(int j = 0; j < i ;j++){

                // 索引      0  1  2  3  4  5  6
                // nums 为 [ 1, 5, 2, 5, 3, 7, 2 ]
                // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
                // 1、从这些数字中选出小于 nums[i] 的数字
                // 2、小于 nums[i] 的那些数字是 1 , 2 , 在之前都已经计算过以 1 或者 2 结尾的最长
                // 递增序列的长度
            }
        }
    }
}
```

```

        // 并且这个结果存放在 dp[j] 中
        // 如果数字为 1, 那么此时 j 为 0, dp[0] 存放了以 1 结尾的最长递增序列的长度
        // 如果数字为 2, 那么此时 j 为 2, dp[2] 存放了以 2 结尾的最长递增序列的长度
        // 3、如果发现此时 dp[i] 小于了 dp[j] + 1
        // 4、说明此时 dp[i] 中的值就不是以 nums[i] 结尾的最长递增序列的长度
        // 需要更新 dp[i]
        if(nums[i] > nums[j] && dp[i] < dp[j] + 1){
            // 4、更新 dp[i]
            dp[i] = dp[j] + 1;
        }
    }

    // 在更新 dp[i] 的过程中, 发现了一个更长的子序列
    if(maxLength < dp[i]){
        // 那么把更长的子序列的长度赋值给 maxLength
        maxLength = dp[i];
    }

    if( maxLength >= 3) return true;
}

// 最后返回 maxLength 就行
return false;

}
}

```

Python

```

class Solution:
    def increasingTriplet(self, nums: List[int]) -> bool:

        # 三元组里面最小的数
        first = float('inf')

        # 三元组里面中间的那个数
        second = float('inf')

        # 开始寻找第三个数
        for third in nums:
            # 1、第三个数 大于 了之前确定好的 第二个数
            # 说明找到了满足条件的三元组
            if third > second: return True

            # 2、第三个数 小于 了之前确定好的 第一个数
            # 为了能让后续 第二个数 的可选范围更大
            # 第一个数总是应该越小越好
            # 这样 第二个数 的可选范围更大
            # 导致 第三个数 的可选范围也更大
            first = third if third < first else first
            second = third if first < third < second else third

```

```
# 更能找到满足条件的三元组
elif third <= first: first = third

# 3、第三个数的值介于 first 和 second 中间
# 更新 second 的值
# 第二个数小一些
# 导致 第三个数 的可选范围也更大
# 更能找到满足条件的三元组
else: second = third

# 返回结果
return False
```

LeetCode 673、最长递增子序列的个数

一、题目描述

给定一个未排序的整数数组 `nums`，返回最长递增子序列的个数。

注意 这个数列必须是 严格 递增的。

示例 1:

输入: [1,3,5,4,7] 输出: 2 解释: 有两个最长递增子序列，分别是 [1, 3, 4, 7] 和 [1, 3, 5, 7]。

示例 2:

输入: [2,2,2,2,2] 输出: 5 解释: 最长递增子序列的长度是1，并且存在5个子序列的长度为1，因此输出5。

提示:

- `1 <= nums.length <= 2000`
- `-106 <= nums[i] <= 106`

二、题目解析

三、参考代码

1、Java 代码

```
public class Solution {
    public int findNumberOfLIS(int[] nums) {

        int n = nums.length;

        // LIS : 最长递增子序列
        // 设置数组 dp，用来存储 nums 中以每个元素结尾的最长递增序列的长度
        // dp[0] 表示以 nums[0] 结尾的 LIS 的长度
        // dp[1] 表示以 nums[1] 结尾的 LIS 的长度
        // dp[i] 表示以 nums[i] 结尾的 LIS 的长度
```



```

int[] dp = new int[n];

// count[i] : 以 nums[i] 为结尾的 LIS 的个数
int[] count = new int[n];

// 初始化
Arrays.fill(dp, 1);
Arrays.fill(count, 1);

// 设置一个变量用来存储最长递增序列的结果
int maxLength = 1;

// 从 0 开始, 直到 dp.length, 往 dp 里面填充结果
for (int i = 0; i < n; i++) {

    // 对于 dp[i] 来说, 在 nums 中从 0 到 i - 1 都是在 i 的前面的
    // 比如 nums 为 [1,5,2,5,3,7,2]
    // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
    // 从 0 遍历到 i - 1, 就可以从这些数字中选出小于 i 的数字
    for (int j = 0; j < i; j++) {

        // 1、如果 nums[i] <= nums[j]
        // nums[i] 无法延伸到 nums[j] 的后面, 形成新的递增序列
        // 于是, 跳过当前的 j 继续查看下一个 j
        if (nums[i] <= nums[j]) {
            continue;
        }

        // 2、否则, 来到这里, 说明 nums[i] > nums[j]
        // nums[i] 可以接到 nums[j] 的后边形成新的递增序列

        // 索引      0  1  2  3  4  5  6
        // nums 为 [ 1, 5, 2, 5, 3, 7, 2 ]
        // 此时 i 为 3, 那么 1,5,2 这些数字都在索引位置为 3 的前面
        // 1、从这些数字中选出小于 nums[i] 的数字
        // 2、小于 nums[i] 的那些数字是 1, 2, 在之前都已经计算过以 1 或者 2 结尾的最长
递增序列的长度

        // 并且这个结果存放在 dp[j] 中
        // 如果数字为 1, 那么此时 j 为 0, dp[0] 存放了以 1 结尾的最长递增序列的长度
        // 如果数字为 2, 那么此时 j 为 2, dp[2] 存放了以 2 结尾的最长递增序列的长度
        // 3、如果发现此时 dp[i] 小于了 dp[j] + 1
        // 4、说明此时 dp[i] 中的值就不是以 nums[i] 结尾的最长递增序列的长度
        // 需要更新 dp[i]
        // 最长递增子序列的长度增加了
        if (dp[i] < dp[j] + 1) {

            // 更新 dp[i]
            dp[i] = dp[j] + 1;

```

```

        // count[i] 就是 count[j]
        // 即有多少这样的 j , 就可以找到多少个以 nums[i] 为结尾的 LIS 的个数

        count[i] = count[j];

    }

    // 如果 dp[i] == dp[j] + 1
    // 说明最长递增子序列的长度并没有增加, 但是出现了长度一样的情况
    // 此时 nums[i] 还没有添加到 nums[j] 的后边
    // 以 nums[i] 为结尾的 LIS 的长度已经是 dp[j] + 1
    // 以 nums[i] 为结尾的 LIS 的个数已经是 count[i]
    // 那么把 nums[i] 加到 nums[j] 之后
    // LIS 的长度不变, 即 dp[i] 不需要更新

    // LIS 包含了 nums[j]
    // 有了 count[j] 个 LIS 的个数
    // 总的 count[i] 更新为 count[i] += count[j]

    // 比如下边的例子
    //           i
    // 1  2  6  5  4  9
    //           j
    // 此时 i 遍历到 9 , j 遍历到 4
    // 在 j 遍历 4 之前, 要先遍历 4 之前的 1 2 6 5
    // 在遍历到 6 的时候, dp[i] 就应该更新为 4 ( 1 2 6 9 这个 LIS 的长度 )
    // 在遍历完 5 的时候, 这个时候的 【count[i] 为 2】
    // (1 2 6 9 和 1 2 5 9 这两个 LIS)

    // 遍历到 4 的时候
    // dp[j] = 3 ( 1 2 4 的长度), count[j] = 1 (就只有 1 2 4 这一个 LIS)
    // 此时满足 dp[j] + 1 = 3 + 1 = 4 == dp[i]
    // 这个时候, 不需要更新 dp[i], 需要更新 count[i] , 【count[i] 已经是 2】
    // 也就是不包含 4 的 1 2 6 9 和 1 2 5 9

    // 如果把 nums[i] 也就是 9 放到 nums[j] 也就是 4 之后
    // count[i] 就会多了一个包含 4 的 LIS , 也就是 1 2 4 9 ,
    // 所以要 count[i] += count[j]
    // 以 nums[i] 为结尾的 LIS 的个数个数为 3
    // (1 2 6 9, 1 2 5 9 和 1 2 4 9 这 3 个 LIS )
    } else if (dp[i] == dp[j] + 1) {

        // 更新 count[i]
        count[i] += count[j];
    }
}

// 记录最长递增子序列的最大长度 maxLength
maxLength = Math.max(maxLength, dp[i]);
}

```

```

// 初始化结果
int result = 0;

// 遍历 dp 数组
for (int i = 0; i < n; i++) {
    // 最长递增子序列的长度是 maxLength
    if (dp[i] == maxLength) {
        // 把所有 dp[i] == maxLength 的那个 i 处对应的 count 进行累加
        result += count[i];
    }
}
// 返回结果
return result;
}
}

```

2、**C++** 代码

3、Python 代码

```

class Solution:
    def findNumberOfLIS(self, nums: List[int]) -> int:
        n = len(nums)

        # LIS: 最长递增子序列
        # 创建一个数组dp, 用于存储以每个元素结尾的最长递增序列的长度
        dp = [1] * n

        # count[i]: 以nums[i]为结尾的LIS的个数
        count = [1] * n

        # 初始化
        maxLength = 1

        # 从0开始, 填充dp数组
        for i in range(n):
            for j in range(i):
                if nums[i] <= nums[j]:
                    continue

                if dp[i] < dp[j] + 1:
                    dp[i] = dp[j] + 1
                    count[i] = count[j]
                elif dp[i] == dp[j] + 1:
                    count[i] += count[j]

            maxLength = max(maxLength, dp[i])

```

```
result = 0
for i in range(n):
    if dp[i] == maxLength:
        result += count[i]

return result
```

LeetCode 718、最长重复子数组（HJ75. 公共子串计算）

一、题目描述

给两个整数数组 **A** 和 **B**，返回两个数组中公共的、长度最长的子数组的长度。

示例：

输入：
A: [1,2,3,2,1]
B: [3,2,1,4,7]
输出：3
解释：
长度最长的公共子数组是 [3, 2, 1] 。

提示：

- `1 <= len(A), len(B) <= 1000`
- `0 <= A[i], B[i] < 100`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最长重复子数组（LeetCode 718）：https://leetcode-cn.com/problems/maximum-length-of-repeated-subarray
class Solution {
    public int findLength(int[] nums1, int[] nums2) {

        // 获取 nums1 的长度
        int m = nums1.length;

        // 获取 nums2 的长度
        int n = nums2.length;

        // 设置数组 dp，用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
```

```
// dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
// dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
// dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
// 前 i 个元素的区间为 [ 0 , i - 1 ]
// dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度
// 前 m 个元素的表示区间为 [ 0 , m ], 前 n 个元素的表示区间为 [ 0 , n ]
// 因此, dp 数组的长度为 m + 1 和 n + 1
int[][] dp = new int[m + 1][n + 1];
```

```
// maxLength 表示 dp 数组中的最大值
int maxLength = 0;
```

```
// dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
// nums1 nums2 也没有元素
// 因此, 公共的、长度最长的子数组的长度为 0
dp[0][0] = 0;
```

```
// nums1 没有元素 或者 nums2 没有字符时, 公共的、长度最长的子数组的长度都为 0
for( int i = 0 ; i <= m ; i++){
    dp[i][0] = 0;
}
```

```
for( int j = 0 ; j <= n ; j++){
    dp[0][j] = 0;
}
```

```
// i 从 1 开始, 直到 m 位置, 遍历 nums1 的前 i 个元素
for (int i = 1; i <= m; i++) {
```

```
    // j 从 1 开始, 直到 n 位置, 遍历 nums2 的前 j 个元素
    for (int j = 1; j <= n; j++) {
```

```
        // 如果发现 nums1 的当前元素, 即位置为 i - 1 的元素
        // 与 nums2 的当前元素, 即位置为 j - 1 的元素【相同】
        // 此时, 找到了一个公共元素, 公共的、长度最长的子数组的长度加 1
        if (nums1[i - 1] == nums2[j - 1]) {
```

```
            // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
            子数组的长度
```

```
            // dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素
            的公共的、长度最长的子数组的长度
```

```
            dp[i][j] = dp[i - 1][j - 1] + 1;
```

```
            // 更新最长的子数组长度
```

```
            maxLength = Math.max(maxLength, dp[i][j]);
```

```
        }
```

```
    }
```

```
}
```

```

        // 返回结果
        return maxLength;
    }
}

```

2、**C++** 代码

```

class Solution {
public:
    int findLength(vector<int>& nums1, vector<int>& nums2) {
        // 获取 nums1 的长度
        int m = nums1.size();

        // 获取 nums2 的长度
        int n = nums2.size();

        // 设置数组 dp, 用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
        // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
        // 前 i 个元素的区间为 [ 0 , i - 1 ]
        // dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度
        // 前 m 个元素的表示区间为 [ 0 , m ], 前 n 个元素的表示区间为 [ 0 , n ]
        // 因此, dp 数组的长度为 m + 1 和 n + 1
        auto dp = vector < vector < int>> ( m + 1 ,vector<int> ( n + 1 ));

        // maxLength 表示 dp 数组中的最大值
        int maxLength = 0;

        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // nums1 nums2 也没有元素
        // 因此, 公共的、长度最长的子数组的长度为 0
        dp[0][0] = 0;

        // nums1 没有元素 或者 nums2 没有字符时, 公共的、长度最长的子数组的长度都为 0
        for( int i = 0 ; i <= m ; i++){
            dp[i][0] = 0;
        }

        for( int j = 0 ; j <= n ; j++){
            dp[0][j] = 0;
        }

        // i 从 1 开始, 直到 m 位置, 遍历 nums1 的前 i 个元素
        for (int i = 1; i <= m; i++) {

            // j 从 1 开始, 直到 n 位置, 遍历 nums2 的前 j 个元素
            for (int j = 1; j <= n; j++) {

```

```

        // 如果发现 nums1 的当前元素，即位置为 i - 1 的元素
        // 与 nums2 的当前元素，即位置为 j - 1 的元素【相同】
        // 此时，找到了一个公共元素，公共的、长度最长的子数组的长度加 1
        if (nums1[i - 1] == nums2[j - 1]) {

            // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
            // 子数组的长度
            // dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素
            // 的公共的、长度最长的子数组的长度
            dp[i][j] = dp[i - 1][j - 1] + 1;

            // 更新最长的子数组长度
            maxLength = max(maxLength, dp[i][j]);

        }
    }

    // 返回结果
    return maxLength;

}

};

```

3、Python 代码

```

class Solution:
    def findLength(self, nums1: List[int], nums2: List[int]) -> int:
        # 获取 nums1 的长度
        m = len(nums1)

        # 获取 nums2 的长度
        n = len(nums2)

        # 设置数组 dp，用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
        # dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        # dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
        # dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
        # 前 i 个元素的区间为 [ 0 , i - 1 ]
        # dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度
        # 前 m 个元素的表示区间为 [ 0 , m ]，前 n 个元素的表示区间为 [ 0 , n ]
        # 因此，dp 数组的长度为 m + 1 和 n + 1
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        # maxLength 表示 dp 数组中的最大值
        maxLength = 0

        # dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        # nums1 nums2 也没有元素
        # 因此，公共的、长度最长的子数组的长度为 0

```

```

dp[0][0] = 0

# nums1 没有元素 或者 nums2 没有字符时，公共的、长度最长的子数组的长度都为 0
for i in range( 0 , m + 1 ) :
    dp[i][0] = 0

for j in range( 0 , n + 1 ) :
    dp[0][j] = 0

# i 从 1 开始，直到 m 位置，遍历 nums1 的前 i 个元素
for i in range( 1 , m + 1 ) :

    # j 从 1 开始，直到 n 位置，遍历 nums2 的前 j 个元素
    for j in range( 1 , n + 1 ) :

        # 如果发现 nums1 的当前元素，即位置为 i - 1 的元素
        # 与 nums2 的当前元素，即位置为 j - 1 的元素【相同】
        # 此时，找到了一个公共元素，公共的、长度最长的子数组的长度加 1
        if nums1[i - 1] == nums2[j - 1] :

            # dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
            # 子数组的长度
            # dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素的
            # 公共的、长度最长的子数组的长度
            dp[i][j] = dp[i - 1][j - 1] + 1

            # 更新最长的子数组长度
            maxLength = max(maxLength, dp[i][j])

# 返回结果
return maxLength

```

第四十五天

LeetCode 121、买卖股票的最佳时机

一、题目描述

给定一个数组 `prices`，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。

你只能选择 **某一天** 买入这只股票，并选择在 **未来的某一个不同的日子** 卖出该股票。设计一个算法来计算你能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 `0`。

示例 1:

输入: [7,1,5,3,6,4] 输出: 5 解释: 在第 2 天 (股票价格 = 1) 的时候买入, 在第 5 天 (股票价格 = 6) 的时候卖出, 最大利润 = 6-1 = 5。注意利润不能是 7-1 = 6, 因为卖出价格需要大于买入价格; 同时, 你不能在买入前卖出股票。

示例 2:

输入: prices = [7,6,4,3,1] 输出: 0 解释: 在这种情况下, 没有交易完成, 所以最大利润为 0。

提示:

- $1 \leq \text{prices.length} \leq 10(5)$
- $0 \leq \text{prices}[i] \leq 10(4)$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机 ( LeetCode 121 ) : https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock/
class Solution {
    public int maxProfit(int[] prices) {

        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为 1, i 的取值范围为数组 prices 的长度, 从 0 开始
        int[][][] dp = new int[length][2][2];

        // dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 0 次交易
        // 的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0][0] = 0;

        // dp[0][1][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 1 次交易
        // 的情况下可以获得的最大收益
```

```

// 此时，就是什么都没做，利润为 0
dp[0][1][0] = 0;

// dp[0][1][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 1 次交易
的情况下可以获得的最大收益
// 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就
是负的，即为 -prices[0]
dp[0][1][1] = -prices[0];

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    // 对于每个坑来说，都有两种状态
    // 今天也就是第 i 天

    // 1、今天【不持有】股票
    // 第 i - 1 天【持有】股票，第 i 天卖出
    // 昨天【持有】股票，今天卖出
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天不操作
    // 昨天【不持有】股票，今天不操作
    dp[i][1][0] = Math.max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] ) ;

    // 2、今天【持有】股票
    // 第 i - 1 天【持有】股票，第 i 天不操作
    // 昨天【持有】股票，今天不操作
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天买入
    // 昨天【不持有】股票，今天买入
    dp[i][1][1] = Math.max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] ) ;

}

// for 循环结束后，dp 数组填充完毕
// dp[length - 1][1][0]
// 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易的情况
下可以获得的最大收益
return dp[length - 1][1][0];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // 先获取数组的长度
    }
}

```

```

int length = prices.size();

// 设置一个三维数组 dp
// dp[i][k][b]
// i 表示天数, dp[i] 表示第 i 天的最大利润
// k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
// 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
// b 表示当前是否持有股票, 取值为 0 和 1
// 其中 0 表示当前持有 0 份股票, 即【不持有】股票
// 而 1 表示当前持有 1 份股票, 即【持有】股票

// 在本题中, k 的值为 1, i 的取值范围为数组 prices 的长度, 从 0 开始
vector<vector<vector<int> > > dp(length, vector<vector<int> >(2, vector<int>
(2)));

// dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 0 次交易
的情况下可以获得的最大收益
// 此时, 就是什么都没做, 利润为 0
dp[0][0][0] = 0;

// dp[0][1][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 1 次交易
的情况下可以获得的最大收益
// 此时, 就是什么都没做, 利润为 0
dp[0][1][0] = 0;

// dp[0][1][1] 表示在第 0 天结束时, 即收盘后, 手上持有 1 份股票, 且此时最多进行了 1 次交易
的情况下可以获得的最大收益
// 手上持有了 1 份股票, 那肯定是执行了买入操作, 然后又还没有卖出, 那么钱都投入了股票中, 利润就
是负的, 即为 -prices[0]
dp[0][1][1] = -prices[0];

// 动态规划: 自底向上, 即从前向后遍历, 实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    // 对于每个坑来说, 都有两种状态
    // 今天也就是第 i 天

    // 1、今天【不持有】股票
    // 第 i - 1 天【持有】股票, 第 i 天卖出
    // 昨天【持有】股票, 今天卖出
    // vs
    // 第 i - 1 天【不持有】股票, 第 i 天不操作
    // 昨天【不持有】股票, 今天不操作
    dp[i][1][0] = max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] );

    // 2、今天【持有】股票
    // 第 i - 1 天【持有】股票, 第 i 天不操作

```

```

        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1][1] = max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] ) ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][1][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易的情况
    // 下可以获得的最大收益
    return dp[length - 1][1][0];

}

};

```

3、Python 代码

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        # b 表示当前是否持有股票，取值为 0 和 1
        # 其中 0 表示当前持有 0 份股票，即【不持有】股票
        # 而 1 表示当前持有 1 份股票，即【持有】股票

        # 在本题中，k 的值为 1，i 的取值范围为数组 prices 的长度，从 0 开始
        # 越在前面的维度，总体上是越出现在后面
        dp = [[[0] * 2 for _ in range(2)] for _ in range(length)]

        # dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
        # 的情况下可以获得的最大收益
        # 此时，就是什么都没做，利润为 0
        dp[0][0][0] = 0

        # dp[0][1][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易
        # 的情况下可以获得的最大收益
        # 此时，就是什么都没做，利润为 0
        dp[0][1][0] = 0

        # dp[0][1][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 1 次交易
        # 的情况下可以获得的最大收益

```

```

# 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]
dp[0][1][1] = -prices[0]

# 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for i in range( 1 ,length) :

    # 对于每个坑来说，都有两种状态
    # 今天也就是第 i 天

    # 1、今天【不持有】股票
    # 第 i - 1 天【持有】股票，第 i 天卖出
    # 昨天【持有】股票，今天卖出
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天不操作
    # 昨天【不持有】股票，今天不操作
    dp[i][1][0] = max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] )

    # 2、今天【持有】股票
    # 第 i - 1 天【持有】股票，第 i 天不操作
    # 昨天【持有】股票，今天不操作
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天买入
    # 昨天【不持有】股票，今天买入
    dp[i][1][1] = max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] )

# for 循环结束后，dp 数组填充完毕
# dp[length - 1][1][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易的情况下可以获得的最大收益
return dp[length - 1][1][0]

```

LeetCode 122、买卖股票的最佳时机 II

一、题目描述

给定一个数组 `prices`，其中 `prices[i]` 是一支给定股票第 `i` 天的价格。

设计一个算法来计算你所能获取的最大利润。你可以尽可能地完成更多的交易（多次买卖一支股票）。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入: prices = [7,1,5,3,6,4] 输出: 7 解释: 在第 2 天 (股票价格 = 1) 的时候买入, 在第 3 天 (股票价格 = 5) 的时候卖出, 这笔交易所能获得利润 = 5-1 = 4 。随后, 在第 4 天 (股票价格 = 3) 的时候买入, 在第 5 天 (股票价格 = 6) 的时候卖出, 这笔交易所能获得利润 = 6-3 = 3 。

示例 2:

输入: prices = [1,2,3,4,5] 输出: 4 解释: 在第 1 天 (股票价格 = 1) 的时候买入, 在第 5 天 (股票价格 = 5) 的时候卖出, 这笔交易所能获得利润 = 5-1 = 4 。注意你不能在第 1 天和第 2 天接连购买股票, 之后再将它们卖出。因为这样属于同时参与了多笔交易, 你必须在再次购买前出售掉之前的股票。

示例 3:

输入: prices = [7,6,4,3,1] 输出: 0 解释: 在这种情况下, 没有交易完成, 所以最大利润为 0。

提示:

- 1 <= prices.length <= 3 * 10⁴
- 0 <= prices[i] <= 10⁴

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机 ( LeetCode 122 ) : https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-ii/submissions/
class Solution {
    public int maxProfit(int[] prices) {
        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为正无穷, i 的取值范围为数组 prices 的长度, 从 0 开始
        int[][] dp = new int[length][2];

        // dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 后续最多再进行 0 次交易的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;
```

// dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 k 次交易的情况下可以获得的最大收益

// 此时，就是什么都没做，利润为 0

dp[0][0] = 0;

// dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，后续最多再进行 k 次交易的情况下可以获得的最大收益

// 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]

dp[0][1] = -prices[0];

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑

for (int i = 1; i < length; i++) {

 // 对于每个坑来说，都有两种状态

 // 今天也就是第 i 天

 // 1、今天【不持有】股票

 // 第 i - 1 天【持有】股票，第 i 天卖出

 // 昨天【持有】股票，今天卖出

 // vs

 // 第 i - 1 天【不持有】股票，第 i 天不操作

 // 昨天【不持有】股票，今天不操作

 dp[i][0] = Math.max(dp[i - 1][0] , dp[i - 1][1] + prices[i]) ;

 // 2、今天【持有】股票

 // 第 i - 1 天【持有】股票，第 i 天不操作

 // 昨天【持有】股票，今天不操作

 // vs

 // 第 i - 1 天【不持有】股票，第 i 天买入

 // 昨天【不持有】股票，今天买入

 dp[i][1] = Math.max(dp[i - 1][1] , dp[i - 1][0] - prices[i]) ;

}

// for 循环结束后，dp 数组填充完毕

// dp[length - 1][k][0]

// 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 k 次交易的情况下可以获得的最大收益

return dp[length - 1][0];

}

}

2、**C++** 代码

```
class solution {
public:
    int maxProfit(vector<int>& prices) {
        // 先获取数组的长度
        int length = prices.size();

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为正无穷, i 的取值范围为数组 prices 的长度, 从 0 开始
        auto dp = vector < vector < int>> ( length ,vector<int> ( 2));

        // dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 0 次交易
        // 的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 k 次交易
        // 的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][1] 表示在第 0 天结束时, 即收盘后, 手上持有 1 份股票, 且此时最多进行了 k 次交易
        // 的情况下可以获得的最大收益
        // 手上持有了 1 份股票, 那肯定是执行了买入操作, 然后又还没有卖出, 那么钱都投入了股票中, 利润就是负的, 即为 -prices[0]
        dp[0][1] = -prices[0];

        // 动态规划: 自底向上, 即从前向后遍历, 实现一个萝卜一个坑
        for (int i = 1; i < length; i++) {

            // 对于每个坑来说, 都有两种状态
            // 今天也就是第 i 天

            // 1、今天【不持有】股票
            // 第 i - 1 天【持有】股票, 第 i 天卖出
            // 昨天【持有】股票, 今天卖出
            // vs
            // 第 i - 1 天【不持有】股票, 第 i 天不操作
            // 昨天【不持有】股票, 今天不操作
            dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] );
        }
    }
};
```



```

        // 2、今天【持有】股票
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1] = max( dp[i - 1][1] , dp[i - 1][0] - prices[i] ) ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][k][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下可以获得的最大收益
    return dp[length - 1][0];
}
};

```

3、Python 代码

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        # b 表示当前是否持有股票，取值为 0 和 1
        # 其中 0 表示当前持有 0 份股票，即【不持有】股票
        # 而 1 表示当前持有 1 份股票，即【持有】股票

        # 在本题中，k 的值为正无穷，i 的取值范围为数组 prices 的长度，从 0 开始
        dp = [[0] * 2 for _ in range(length)]

        # dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易的情况下可以获得的最大收益
        # 此时，就是什么都没做，利润为 0
        dp[0][0] = 0

        # dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下可以获得的最大收益
        # 此时，就是什么都没做，利润为 0
        dp[0][0] = 0

```

```

# dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 k 次交易
的情况下可以获得的最大收益
# 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是
负的，即为 -prices[0]
dp[0][1] = -prices[0]

# 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for i in range( 1 ,length) :

    # 对于每个坑来说，都有两种状态
    # 今天也就是第 i 天

    # 1、今天【不持有】股票
    # 第 i - 1 天【持有】股票，第 i 天卖出
    # 昨天【持有】股票，今天卖出
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天不操作
    # 昨天【不持有】股票，今天不操作
    dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] )

    # 2、今天【持有】股票
    # 第 i - 1 天【持有】股票，第 i 天不操作
    # 昨天【持有】股票，今天不操作
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天买入
    # 昨天【不持有】股票，今天买入
    dp[i][1] = max( dp[i - 1][1] , dp[i - 1][0] - prices[i] )

# for 循环结束后，dp 数组填充完毕
# dp[length - 1][k][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况
下可以获得的最大收益
return dp[length - 1][0]

```

LeetCode 123、买卖股票的最佳时机 III

一、题目描述

给定一个数组，它的第 i 个元素是一支给定的股票在第 i 天的价格。

设计一个算法来计算你所能获取的最大利润。你最多可以完成 **两笔** 交易。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1：

输入: prices = [3,3,5,0,0,3,1,4] 输出: 6 解释: 在第 4 天 (股票价格 = 0) 的时候买入, 在第 6 天 (股票价格 = 3) 的时候卖出, 这笔交易所能获得利润 = 3-0 = 3 。随后, 在第 7 天 (股票价格 = 1) 的时候买入, 在第 8 天 (股票价格 = 4) 的时候卖出, 这笔交易所能获得利润 = 4-1 = 3 。

示例 2:

输入: prices = [1,2,3,4,5] 输出: 4 解释: 在第 1 天 (股票价格 = 1) 的时候买入, 在第 5 天 (股票价格 = 5) 的时候卖出, 这笔交易所能获得利润 = 5-1 = 4 。注意你不能在第 1 天和第 2 天接连购买股票, 之后再将它们卖出。因为这样属于同时参与了多笔交易, 你必须在再次购买前出售掉之前的股票。

示例 3:

输入: prices = [7,6,4,3,1] 输出: 0 解释: 在这个情况下, 没有交易完成, 所以最大利润为 0。

示例 4:

输入: prices = [1] 输出: 0

提示:

- 1 <= prices.length <= 10(5)
- 0 <= prices[i] <= 10(5)

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机III ( LeetCode 123 ) : https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-iii/
class Solution {
    public int maxProfit(int[] prices) {

        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【 k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为 2, i 的取值范围为数组 prices 的长度, 从 0 开始
        int[][][] dp = new int[length][3][2];
```

```

        // dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 0 次交易的
        情况下可以获得的最大收益
        // 此时，就是什么都没做，利润为 0
        dp[0][0][0] = 0;

        // dp[0][1][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 1 次交易的
        情况下可以获得的最大收益
        // 此时，就是什么都没做，利润为 0
        dp[0][1][0] = 0;

        // dp[0][1][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，后续最多再进行 1 次交易的
        情况下可以获得的最大收益
        // 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就
        是负的，即为 -prices[0]
        dp[0][1][1] = -prices[0];

        // dp[0][2][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 2 次交易的
        情况下可以获得的最大收益
        // 此时，就是什么都没做，利润为 0
        dp[0][2][0] = 0;

        // dp[0][2][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，后续最多再进行 2 次交易的
        情况下可以获得的最大收益
        // 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就
        是负的，即为 -prices[0]
        dp[0][2][1] = -prices[0];

        // 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
        for (int i = 1; i < length; i++) {

            // 对于每个坑来说，都有两种状态
            // 今天也就是第 i 天

            // 1、今天【不持有】股票，后续最多交易 2 次
            // 第 i - 1 天【持有】股票，第 i 天卖出
            // 昨天【持有】股票，今天卖出
            // vs
            // 第 i - 1 天【不持有】股票，第 i 天不操作
            // 昨天【不持有】股票，今天不操作
            dp[i][2][0] = Math.max( dp[i - 1][2][0] , dp[i - 1][2][1] + prices[i] ) ;

            // 2、今天【持有】股票，后续最多交易 2 次
            // 第 i - 1 天【持有】股票，第 i 天不操作
            // 昨天【持有】股票，今天不操作
            // vs
            // 第 i - 1 天【不持有】股票，第 i 天买入
            // 昨天【不持有】股票，今天买入
            dp[i][2][1] = Math.max( dp[i - 1][2][1] , dp[i - 1][1][0] - prices[i] ) ;

```

```

        // 3、今天【不持有】股票，后续最多交易 1 次
        // 第 i - 1 天【持有】股票，第 i 天卖出
        // 昨天【持有】股票，今天卖出
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天不操作
        // 昨天【不持有】股票，今天不操作
        dp[i][1][0] = Math.max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] ) ;

        // 4、今天【持有】股票，后续最多交易 1 次
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1][1] = Math.max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] ) ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][1][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 1 次交易的情况下
    // 可以获得的最大收益
    return dp[length - 1][2][0];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // 先获取数组的长度
        int length = prices.size();

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数，dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        // 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票，取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票，即【不持有】股票
        // 而 1 表示当前持有 1 份股票，即【持有】股票

        // 在本题中，k 的值为 2，i 的取值范围为数组 prices 的长度，从 0 开始
        vector<vector<vector<int>>> dp(length, vector<vector<int>>(3, vector<int>
(2)));
    }
}

```

```

// dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
的情况下可以获得的最大收益
// 此时，就是什么都没做，利润为 0
dp[0][0][0] = 0;

// dp[0][1][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易
的情况下可以获得的最大收益
// 此时，就是什么都没做，利润为 0
dp[0][1][0] = 0;

// dp[0][1][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 1 次交易
的情况下可以获得的最大收益
// 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]
dp[0][1][1] = -prices[0];

// dp[0][2][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 2 次交易
的情况下可以获得的最大收益
// 此时，就是什么都没做，利润为 0
dp[0][2][0] = 0;

// dp[0][2][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 2 次交易
的情况下可以获得的最大收益
// 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]
dp[0][2][1] = -prices[0];

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    // 对于每个坑来说，都有两种状态
    // 今天也就是第 i 天

    // 1、今天【不持有】股票，且此时最多进行了 2 次
    // 第 i - 1 天【持有】股票，第 i 天卖出
    // 昨天【持有】股票，今天卖出
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天不操作
    // 昨天【不持有】股票，今天不操作
    dp[i][2][0] = max( dp[i - 1][2][0] , dp[i - 1][2][1] + prices[i] );

    // 2、今天【持有】股票，且此时最多进行了 2 次
    // 第 i - 1 天【持有】股票，第 i 天不操作
    // 昨天【持有】股票，今天不操作
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天买入
    // 昨天【不持有】股票，今天买入
    dp[i][2][1] = max( dp[i - 1][2][1] , dp[i - 1][1][0] - prices[i] );
}

```

```

        // 3、今天【不持有】股票，且此时最多进行了 1 次
        // 第 i - 1 天【持有】股票，第 i 天卖出
        // 昨天【持有】股票，今天卖出
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天不操作
        // 昨天【不持有】股票，今天不操作
        dp[i][1][0] = max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] ) ;

        // 4、今天【持有】股票，且此时最多进行了 1 次
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1][1] = max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] ) ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][1][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 2 次交易的情况下
    // 下可以获得的最大收益
    return dp[length - 1][2][0];
}
};

```

3、Python 代码

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        # b 表示当前是否持有股票，取值为 0 和 1
        # 其中 0 表示当前持有 0 份股票，即【不持有】股票
        # 而 1 表示当前持有 1 份股票，即【持有】股票

        # 在本题中，k 的值为 2，i 的取值范围为数组 prices 的长度，从 0 开始
        # 越在前面的维度，总体上是越出现在后面
        dp = [[[0] * 2 for _ in range(3)] for _ in range(length)]

        # dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
        # 的情况下可以获得的最大收益

```

此时，就是什么都没做，利润为 0

`dp[0][0][0] = 0`

`dp[0][1][0]` 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 1 次交易的情况下可以获得的最大收益

此时，就是什么都没做，利润为 0

`dp[0][1][0] = 0`

`dp[0][1][1]` 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 1 次交易的情况下可以获得的最大收益

手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 `-prices[0]`

`dp[0][1][1] = -prices[0]`

`dp[0][2][0]` 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 2 次交易的情况下可以获得的最大收益

此时，就是什么都没做，利润为 0

`dp[0][2][0] = 0`

`dp[0][2][1]` 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 2 次交易的情况下可以获得的最大收益

手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 `-prices[0]`

`dp[0][2][1] = -prices[0]`

动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑

`for i in range(1, length):`

对于每个坑来说，都有两种状态

今天也就是第 `i` 天

1、今天【不持有】股票，且此时最多进行了 2 次

第 `i - 1` 天【持有】股票，第 `i` 天卖出

昨天【持有】股票，今天卖出

vs

第 `i - 1` 天【不持有】股票，第 `i` 天不操作

昨天【不持有】股票，今天不操作

`dp[i][2][0] = max(dp[i - 1][2][0], dp[i - 1][2][1] + prices[i])`

2、今天【持有】股票，且此时最多进行了 2 次

第 `i - 1` 天【持有】股票，第 `i` 天不操作

昨天【持有】股票，今天不操作

vs

第 `i - 1` 天【不持有】股票，第 `i` 天买入

昨天【不持有】股票，今天买入

`dp[i][2][1] = max(dp[i - 1][2][1], dp[i - 1][1][0] - prices[i])`

3、今天【不持有】股票，且此时最多进行了 1 次

第 `i - 1` 天【持有】股票，第 `i` 天卖出


```

# 昨天【持有】股票，今天卖出
# vs
# 第 i - 1 天【不持有】股票，第 i 天不操作
# 昨天【不持有】股票，今天不操作
dp[i][1][0] = max( dp[i - 1][1][0] , dp[i - 1][1][1] + prices[i] )

# 4、今天【持有】股票，且此时最多进行了 1 次
# 第 i - 1 天【持有】股票，第 i 天不操作
# 昨天【持有】股票，今天不操作
# vs
# 第 i - 1 天【不持有】股票，第 i 天买入
# 昨天【不持有】股票，今天买入
dp[i][1][1] = max( dp[i - 1][1][1] , dp[i - 1][0][0] - prices[i] )

# for 循环结束后，dp 数组填充完毕
# dp[length - 1][1][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 2 次交易的情况下
# 下可以获得的最大收益
return dp[length - 1][2][0]

```

LeetCode 188、买卖股票的最佳时机 IV

一、题目描述

给定一个数组 `prices`，其中 `prices[i]` 是一支给定股票第 `i` 天的价格。

设计一个算法来计算你所能获取的最大利润。你最多可以完成 `k` 笔交易。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

二、题目解析

示例 1：

输入：k = 2, prices = [2,4,1] 输出：2 解释：在第 1 天 (股票价格 = 2) 的时候买入，在第 2 天 (股票价格 = 4) 的时候卖出，这笔交易所能获得利润 = 4-2 = 2。

示例 2：

输入：k = 2, prices = [3,2,6,5,0,3] 输出：7 解释：在第 2 天 (股票价格 = 2) 的时候买入，在第 3 天 (股票价格 = 6) 的时候卖出，这笔交易所能获得利润 = 6-2 = 4。随后，在第 5 天 (股票价格 = 0) 的时候买入，在第 6 天 (股票价格 = 3) 的时候卖出，这笔交易所能获得利润 = 3-0 = 3。

提示：

- `0 <= k <= 100`
- `0 <= prices.length <= 1000`
- `0 <= prices[i] <= 1000`

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机IV ( LeetCode 188 ) : https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-iv/
class Solution {
    public int maxProfit(int k, int[] prices) {

        if (prices == null || prices.length == 0) {
            return 0;
        }

        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为 k, i 的取值范围为数组 prices 的长度, 从 0 开始
        int[][][] dp = new int[length][k + 1][2];

        // 初始化
        for (int j = 1; j <= k; j++) {
            // dp[0][j][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 后续最多再进行 j 次交易的情况下可以获得的最大收益
            dp[0][j][0] = 0;

            // dp[0][j][1] 表示在第 0 天结束时, 即收盘后, 手上持有 1 份股票, 后续最多再进行 j 次交易的情况下可以获得的最大收益
            // 手上持有了 1 份股票, 那肯定是执行了买入操作, 然后又还没有卖出, 那么钱都投入了股票中, 利润就是负的, 即为 -prices[0]
            dp[0][j][1] = -prices[0];
        }

        // 动态规划: 自底向上, 即从前向后遍历, 实现一个萝卜一个坑
        for (int i = 1; i < length; i++) {

            for (int j = 1; j < k + 1; j++){
```

```

        // 对于每个坑来说，都有两种状态
        // 今天也就是第 i 天

        // 1、今天【不持有】股票
        // 第 i - 1 天【持有】股票，第 i 天卖出
        // 昨天【持有】股票，今天卖出
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天不操作
        // 昨天【不持有】股票，今天不操作
        dp[i][j][0] = Math.max( dp[i - 1][j][0] , dp[i - 1][j][1] + prices[i] )
    ;

        // 2、今天【持有】股票
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][j][1] = Math.max( dp[i - 1][j][1] , dp[i - 1][j-1][0] - prices[i]
    ) ;
    }

}

// for 循环结束后，dp 数组填充完毕
// dp[length - 1][1][0]
// 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 1 次交易的情况下
// 可以获得的最大收益
return dp[length - 1][k][0];
}
}

```

2、C++代码

```

class Solution {
public:
    int maxProfit(int k, vector<int>& prices) {

        if (prices.empty()) {
            return 0;
        }

        // 先获取数组的长度
        int length = prices.size();

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数，dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
    }
}

```

```

// 注意：【 k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
// b 表示当前是否持有股票，取值为 0 和 1
// 其中 0 表示当前持有 0 份股票，即【不持有】股票
// 而 1 表示当前持有 1 份股票，即【持有】股票

// 在本题中，k 的值为 k，i 的取值范围为数组 prices 的长度，从 0 开始
vector<vector<vector<int> > > dp(length, vector<vector<int> >( k + 1 ,
vector<int>(2)));

// 初始化
for (int j = 1; j <= k; j++) {
    // dp[0][j][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 j
    次交易的情况下可以获得的最大收益
    dp[0][j][0] = 0;

    // dp[0][j][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 j
    次交易的情况下可以获得的最大收益
    // 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，
    利润就是负的，即为 -prices[0]
    dp[0][j][1] = -prices[0];
}

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    for ( int j = 1 ; j < k + 1 ; j++){
        // 对于每个坑来说，都有两种状态
        // 今天也就是第 i 天

        // 1、今天【不持有】股票
        // 第 i - 1 天【持有】股票，第 i 天卖出
        // 昨天【持有】股票，今天卖出
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天不操作
        // 昨天【不持有】股票，今天不操作
        dp[i][j][0] = max( dp[i - 1][j][0] , dp[i - 1][j][1] + prices[i] ) ;

        // 2、今天【持有】股票
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][j][1] = max( dp[i - 1][j][1] , dp[i - 1][j-1][0] - prices[i] ) ;
    }

}

// for 循环结束后，dp 数组填充完毕

```

```

        // dp[length - 1][k][0]
        // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下可以获得的最大收益
        return dp[length - 1][k][0];
    }
};

```

3、Python 代码

```

class solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        if not prices:
            return 0

        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        # b 表示当前是否持有股票，取值为 0 和 1
        # 其中 0 表示当前持有 0 份股票，即【不持有】股票
        # 而 1 表示当前持有 1 份股票，即【持有】股票

        # 在本题中，k 的值为 k，i 的取值范围为数组 prices 的长度，从 0 开始
        # 越在前面的维度，总体上是越出现在后面
        dp = [[[0] * 2 for _ in range(k + 1)] for _ in range(length)]

        # 初始化
        for j in range(1, k + 1):
            # dp[0][j][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 j 次交易的情况下可以获得的最大收益
            dp[0][j][0] = 0

            # dp[0][j][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 j 次交易的情况下可以获得的最大收益
            # 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]
            dp[0][j][1] = -prices[0]

        # 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
        for i in range(1, length):

            for j in range(1, k + 1):
                # 对于每个坑来说，都有两种状态
                # 今天也就是第 i 天

```

```

# 1、今天【不持有】股票
# 第 i - 1 天【持有】股票，第 i 天卖出
# 昨天【持有】股票，今天卖出
# vs
# 第 i - 1 天【不持有】股票，第 i 天不操作
# 昨天【不持有】股票，今天不操作
dp[i][j][0] = max( dp[i - 1][j][0] , dp[i - 1][j][1] + prices[i] )

# 2、今天【持有】股票
# 第 i - 1 天【持有】股票，第 i 天不操作
# 昨天【持有】股票，今天不操作
# vs
# 第 i - 1 天【不持有】股票，第 i 天买入
# 昨天【不持有】股票，今天买入
dp[i][j][1] = max( dp[i - 1][j][1] , dp[i - 1][j-1][0] - prices[i] )

# for 循环结束后，dp 数组填充完毕
# dp[length - 1][k][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下
# 可以获得的最大收益
return dp[length - 1][k][0]

```

LeetCode 309、最佳买卖股票时机含冷冻期

一、题目描述

给定一个整数数组，其中第 i 个元素代表了第 i 天的股票价格。

设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：

- 你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。
- 卖出股票后，你无法在第二天买入股票 (即冷冻期为 1 天)。

示例：

输入: [1,2,3,0,2] 输出: 3 解释: 对应的交易状态为: [买入, 卖出, 冷冻期, 买入, 卖出]

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最佳买卖股票时机含冷冻期(LeetCode 309):https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-with-cooldown/

```

```

class Solution {
    public int maxProfit(int[] prices) {
        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【 k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为正无穷, i 的取值范围为数组 prices 的长度, 从 0 开始
        int[][] dp = new int[length][2];

        // dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 后续最多再进行 0 次交易的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 后续最多再进行 k 次交易的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][1] 表示在第 0 天结束时, 即收盘后, 手上持有 1 份股票, 后续最多再进行 k 次交易的情况下可以获得的最大收益
        // 手上持有了 1 份股票, 那肯定是执行了买入操作, 然后又还没有卖出, 那么钱都投入了股票中, 利润就是负的, 即为 -prices[0]
        dp[0][1] = -prices[0];

        // 动态规划: 自底向上, 即从前向后遍历, 实现一个萝卜一个坑
        for (int i = 1; i < length; i++) {

            // 对于每个坑来说, 都有两种状态
            // 今天也就是第 i 天

            // 1、今天【不持有】股票
            // 第 i - 1 天【持有】股票, 第 i 天卖出
            // 昨天【持有】股票, 今天卖出
            // vs
            // 第 i - 1 天【不持有】股票, 第 i 天不操作
            // 昨天【不持有】股票, 今天不操作
            dp[i][0] = Math.max( dp[i - 1][0] , dp[i - 1][1] + prices[i] );

            // 2、今天【持有】股票
            // 第 i - 1 天【持有】股票, 第 i 天不操作

```

```

        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1] = Math.max( dp[i - 1][1] ,    (i >= 2 ? dp[i - 2][0] : 0 ) -
prices[i] ) ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][k][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 k 次交易的情况下
    可以获得的最大收益
    return dp[length - 1][0];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // 先获取数组的长度
        int length = prices.size();

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数，dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        // 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票，取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票，即【不持有】股票
        // 而 1 表示当前持有 1 份股票，即【持有】股票

        // 在本题中，k 的值为正无穷，i 的取值范围为数组 prices 的长度，从 0 开始
        auto dp = vector < vector < int>> ( length ,vector<int> ( 2));

        // dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
        的情况下可以获得的最大收益
        // 此时，就是什么都没做，利润为 0
        dp[0][0] = 0;

        // dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易
        的情况下可以获得的最大收益
        // 此时，就是什么都没做，利润为 0
        dp[0][0] = 0;

        // dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 k 次交易
        的情况下可以获得的最大收益
    }
};

```



```

        // 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是负的，即为 -prices[0]
        dp[0][1] = -prices[0];

    // 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
    for (int i = 1; i < length; i++) {

        // 对于每个坑来说，都有两种状态
        // 今天也就是第 i 天

        // 1、今天【不持有】股票
        // 第 i - 1 天【持有】股票，第 i 天卖出
        // 昨天【持有】股票，今天卖出
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天不操作
        // 昨天【不持有】股票，今天不操作
        dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] ) ;

        // 2、今天【持有】股票
        // 第 i - 1 天【持有】股票，第 i 天不操作
        // 昨天【持有】股票，今天不操作
        // vs
        // 第 i - 1 天【不持有】股票，第 i 天买入
        // 昨天【不持有】股票，今天买入
        dp[i][1] = max( dp[i - 1][1] , (i >= 2 ? dp[i - 2][0] : 0 ) - prices[i] )

    ;

    }

    // for 循环结束后，dp 数组填充完毕
    // dp[length - 1][k][0]
    // 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下可以获得的最大收益
    return dp[length - 1][0];
}
};

```

3、Python 代码

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】

```

```

# b 表示当前是否持有股票，取值为 0 和 1
# 其中 0 表示当前持有 0 份股票，即【不持有】股票
# 而 1 表示当前持有 1 份股票，即【持有】股票

# 在本题中，k 的值为正无穷，i 的取值范围为数组 prices 的长度，从 0 开始
dp = [[0] * 2 for _ in range(length)]

# dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
# 的情况下可以获得的最大收益
# 此时，就是什么都没做，利润为 0
dp[0][0] = 0

# dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易
# 的情况下可以获得的最大收益
# 此时，就是什么都没做，利润为 0
dp[0][0] = 0

# dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 k 次交易
# 的情况下可以获得的最大收益
# 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是
# 负的，即为 -prices[0]
dp[0][1] = -prices[0]

# 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for i in range(1, length) :

    # 对于每个坑来说，都有两种状态
    # 今天也就是第 i 天

    # 1、今天【不持有】股票
    # 第 i - 1 天【持有】股票，第 i 天卖出
    # 昨天【持有】股票，今天卖出
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天不操作
    # 昨天【不持有】股票，今天不操作
    dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] )

    # 2、今天【持有】股票
    # 第 i - 1 天【持有】股票，第 i 天不操作
    # 昨天【持有】股票，今天不操作
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天买入
    # 昨天【不持有】股票，今天买入
    dp[i][1] = max( dp[i - 1][1] , (dp[i - 2][0] if i >= 2 else 0) -
prices[i] )

# for 循环结束后，dp 数组填充完毕

```

```
# dp[length - 1][k][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下可以获得的最大收益
return dp[length - 1][0]
```

LeetCode 714、买卖股票的最佳时机含手续费

一、题目描述

给定一个整数数组 `prices`，其中第 `i` 个元素代表了第 `i` 天的股票价格；整数 `fee` 代表了交易股票的手续费用。

你可以无限次地完成交易，但是你每笔交易都需要付手续费。如果你已经购买了一个股票，在卖出它之前你就不能再继续购买股票了。

返回获得利润的最大值。

注意：这里的一笔交易指买入持有并卖出股票的整个过程，每笔交易你只需要为支付一次手续费。

示例 1：

输入：prices = [1, 3, 2, 8, 4, 9], fee = 2 输出：8 解释：能够达到的最大利润: 在此处买入 prices[0] = 1 在此处卖出 prices[3] = 8 在此处买入 prices[4] = 4 在此处卖出 prices[5] = 9 总利润: ((8 - 1) - 2) + ((9 - 4) - 2) = 8

示例 2：

输入：prices = [1,3,7,5,10,3], fee = 3 输出：6

提示：

- $1 \leq \text{prices.length} \leq 5 \times 10^4$
- $1 \leq \text{prices}[i] < 5 \times 10^4$
- $0 \leq \text{fee} < 5 \times 10^4$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 买卖股票的最佳时机含手续费(LeetCode 714):https://leetcode-cn.com/problems/best-time-to-buy-and-sell-stock-with-transaction-fee/
class Solution {
    public int maxProfit(int[] prices, int fee) {
        // 先获取数组的长度
        int length = prices.length;

        // 设置一个三维数组 dp
        // dp[i][k][b]
```

```

// i 表示天数，dp[i] 表示第 i 天的最大利润
// k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
// 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
// b 表示当前是否持有股票，取值为 0 和 1
// 其中 0 表示当前持有 0 份股票，即【不持有】股票
// 而 1 表示当前持有 1 份股票，即【持有】股票

// 在本题中，k 的值为正无穷，i 的取值范围为数组 prices 的长度，从 0 开始
int[][] dp = new int[length][2];

// dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 0 次交易的
// 情况下可以获得的最大收益
// 此时，就是什么都没做，利润为 0
dp[0][0] = 0;

// dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，后续最多再进行 k 次交易的
// 情况下可以获得的最大收益
// 此时，就是什么都没做，利润为 0
dp[0][0] = 0;

// dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，后续最多再进行 k 次交易的
// 情况下可以获得的最大收益
// 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就
// 是负的，即为 -prices[0]
// 由于每笔交易你只需要为支付一次手续费，因此这里可以认为是买入的时候就支付了手续费，
// 那么此时的利润是 - ( prices[0] + fee)
dp[0][1] = - ( prices[0] + fee);

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    // 对于每个坑来说，都有两种状态
    // 今天也就是第 i 天

    // 1、今天【不持有】股票
    // 第 i - 1 天【持有】股票，第 i 天卖出
    // 昨天【持有】股票，今天卖出，不需要支付手续费
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天不操作
    // 昨天【不持有】股票，今天不操作
    dp[i][0] = Math.max( dp[i - 1][0] , dp[i - 1][1] + prices[i] );

    // 2、今天【持有】股票
    // 第 i - 1 天【持有】股票，第 i 天不操作
    // 昨天【持有】股票，今天不操作
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天买入
    // 昨天【不持有】股票，今天买入，需要支付手续费
    dp[i][1] = Math.max( dp[i - 1][1] , dp[i - 1][0] - prices[i] - fee );
}

```

```

    }

    // for 循环结束后, dp 数组填充完毕
    // dp[length - 1][k][0]
    // 表示第 length - 1 天结束时, 即收盘后, 手上持有 0 份股票, 后续最多再进行 k 次交易的情况下
    可以获得的最大收益
    return dp[length - 1][0];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxProfit(vector<int>& prices, int fee) {
        // 先获取数组的长度
        int length = prices.size();

        // 设置一个三维数组 dp
        // dp[i][k][b]
        // i 表示天数, dp[i] 表示第 i 天的最大利润
        // k 表示最多交易次数, 每次交易包含买入和卖出, 这里只在买入的时候将 k - 1
        // 注意: 【k 表示最多交易次数, 而不是实际交易次数, 比如最多交易两次可能实际只交易一次】
        // b 表示当前是否持有股票, 取值为 0 和 1
        // 其中 0 表示当前持有 0 份股票, 即【不持有】股票
        // 而 1 表示当前持有 1 份股票, 即【持有】股票

        // 在本题中, k 的值为正无穷, i 的取值范围为数组 prices 的长度, 从 0 开始
        auto dp = vector < vector < int>> ( length ,vector<int> ( 2));

        // dp[0][0][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了 0 次交易
        的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][0] 表示在第 0 天结束时, 即收盘后, 手上持有 0 份股票, 且此时最多进行了行 k 次交易
        的情况下可以获得的最大收益
        // 此时, 就是什么都没做, 利润为 0
        dp[0][0] = 0;

        // dp[0][k][1] 表示在第 0 天结束时, 即收盘后, 手上持有 1 份股票, 且此时最多进行了 k 次交易
        的情况下可以获得的最大收益
        // 手上持有了 1 份股票, 那肯定是执行了买入操作, 然后又还没有卖出, 那么钱都投入了股票中, 利润就是
        负的, 即为 -prices[0]
        // 由于每笔交易你只需要为支付一次手续费, 因此这里可以认为是买入的时候就支付了手续费,
        // 那么此时的利润是 - ( prices[0] + fee)
        dp[0][1] = - ( prices[0] + fee);
    }
}

```

```

// 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for (int i = 1; i < length; i++) {

    // 对于每个坑来说，都有两种状态
    // 今天也就是第 i 天

    // 1、今天【不持有】股票
    // 第 i - 1 天【持有】股票，第 i 天卖出
    // 昨天【持有】股票，今天卖出，不需要支付手续费
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天不操作
    // 昨天【不持有】股票，今天不操作
    dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] ) ;

    // 2、今天【持有】股票
    // 第 i - 1 天【持有】股票，第 i 天不操作
    // 昨天【持有】股票，今天不操作
    // vs
    // 第 i - 1 天【不持有】股票，第 i 天买入
    // 昨天【不持有】股票，今天买入，需要支付手续费
    dp[i][1] = max( dp[i - 1][1] , dp[i - 1][0] - prices[i] - fee ) ;

}

// for 循环结束后，dp 数组填充完毕
// dp[length - 1][k][0]
// 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况下
// 下可以获得的最大收益
return dp[length - 1][0];
}
};

```

3、Python 代码

```

class Solution:
    def maxProfit(self, prices: List[int], fee: int) -> int:
        # 先获取数组的长度
        length = len(prices)

        # 设置一个三维数组 dp
        # dp[i][k][b]
        # i 表示天数，dp[i] 表示第 i 天的最大利润
        # k 表示最多交易次数，每次交易包含买入和卖出，这里只在买入的时候将 k - 1
        # 注意：【k 表示最多交易次数，而不是实际交易次数，比如最多交易两次可能实际只交易一次】
        # b 表示当前是否持有股票，取值为 0 和 1
        # 其中 0 表示当前持有 0 份股票，即【不持有】股票
        # 而 1 表示当前持有 1 份股票，即【持有】股票

        # 在本题中，k 的值为正无穷，i 的取值范围为数组 prices 的长度，从 0 开始

```

```

dp = [[0] * 2 for _ in range(length)]

# dp[0][0][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 0 次交易
# 的情况下可以获得的最大收益
# 此时，就是什么都没做，利润为 0
dp[0][0] = 0

# dp[0][k][0] 表示在第 0 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易
# 的情况下可以获得的最大收益
# 此时，就是什么都没做，利润为 0
dp[0][0] = 0

# dp[0][k][1] 表示在第 0 天结束时，即收盘后，手上持有 1 份股票，且此时最多进行了 k 次交易
# 的情况下可以获得的最大收益
# 手上持有了 1 份股票，那肯定是执行了买入操作，然后又还没有卖出，那么钱都投入了股票中，利润就是
# 负的，即为 -prices[0]
# 由于每笔交易你只需要为支付一次手续费，因此这里可以认为是买入的时候就支付了手续费，
# 那么此时的利润是 - ( prices[0] + fee)
dp[0][1] = - ( prices[0] + fee)

# 动态规划：自底向上，即从前向后遍历，实现一个萝卜一个坑
for i in range( 1 ,length) :

    # 对于每个坑来说，都有两种状态
    # 今天也就是第 i 天

    # 1、今天【不持有】股票
    # 第 i - 1 天【持有】股票，第 i 天卖出
    # 昨天【持有】股票，今天卖出，不需要支付手续费
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天不操作
    # 昨天【不持有】股票，今天不操作
    dp[i][0] = max( dp[i - 1][0] , dp[i - 1][1] + prices[i] )

    # 2、今天【持有】股票
    # 第 i - 1 天【持有】股票，第 i 天不操作
    # 昨天【持有】股票，今天不操作
    # vs
    # 第 i - 1 天【不持有】股票，第 i 天买入
    # 昨天【不持有】股票，今天买入，需要支付手续费
    dp[i][1] = max( dp[i - 1][1] , dp[i - 1][0] - prices[i] - fee )

# for 循环结束后，dp 数组填充完毕
# dp[length - 1][k][0]
# 表示第 length - 1 天结束时，即收盘后，手上持有 0 份股票，且此时最多进行了 k 次交易的情况
# 下可以获得的最大收益
return dp[length - 1][0]

```

第四十六天

LeetCode 120、三角形最小路径和

一、题目描述

给定一个三角形 `triangle`，找出自顶向下的最小路径和。

每一步只能移动到下一行中相邻的结点上。**相邻的结点** 在这里指的是 **下标与上一层结点的下标相同或者等于上一层结点的下标 + 1** 的两个结点。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 三角形最小路径和 ( LeetCode 120 ) : https://leetcode-cn.com/problems/triangle/
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {

        // triangle 是个二维数组
        // 先获取 triangle 的层数，即一维数组的个数
        int n = triangle.size();

        // 设置一个一维数组，动态的更新每一层中当前节点对应的最短路径
        int[] dp = new int[ n + 1 ];

        // 从最后一层开始计算节点的最短路径，直到顶层 0 层为止
        for (int i = n - 1 ; i >= 0 ; i-- ){
            // dp 中存储的是前 i 个位置存储的是到达第 i 层各个节点的最小路径和
            // 从每一层的第 0 个位置开始
            for(int j = 0 ; j <= i ; j++){
                // dp[j] 表示第 i 层中第 j 个节点的最小路径和
                dp[j] = triangle.get(i).get(j) + Math.min(dp[j],dp[j+1]);
            }
        }

        // 返回结果
        return dp[0];
    }
}
```


2、C++ 代码

```
class Solution {
public:
    int minimumTotal(vector<vector<int>>& triangle) {
        // triangle 是个二维数组
        // 先获取 triangle 的层数，即一维数组的个数
        int n = triangle.size();

        // 设置一个一维数组，动态的更新每一层中当前节点对应的最短路径
        vector<int> dp( n + 1 );

        // 从最后一层开始计算节点的最短路径，直到顶层 0 层为止
        for (int i = n - 1 ; i >= 0 ; i-- ){
            // dp 中存储的是前 i 个位置存储的是到达第 i 层各个节点的最小路径和
            // 从每一层的第 0 个位置开始
            for(int j = 0 ; j <= i ; j++){
                // dp[j] 表示第 i 层中第 j 个节点的最小路径和
                dp[j] = triangle[i][j] + min(dp[j],dp[j+1]);
            }
        }

        // 返回结果
        return dp[0];
    }
};
```

3、Python 代码

```
class Solution:
    def minimumTotal(self, triangle: List[List[int]]) -> int:
        # triangle 是个二维数组
        # 先获取 triangle 的层数，即一维数组的个数
        n = len(triangle)

        # 设置一个一维数组，动态的更新每一层中当前节点对应的最短路径
        dp = [ 0 for _ in range( n + 1 )]

        # 从最后一层开始计算节点的最短路径，直到顶层 0 层为止
        for i in range( n - 1 , -1 , -1 ) :
            # dp 中存储的是前 i 个位置存储的是到达第 i 层各个节点的最小路径和
            # 从每一层的第 0 个位置开始
            for j in range( 0 , i + 1 ) :
                # dp[j] 表示第 i 层中第 j 个节点的最小路径和
                dp[j] = triangle[i][j] + min(dp[j],dp[j+1])

        # 返回结果
        return dp[0]
```

LeetCode 174、地下城游戏

一、题目描述

一些恶魔抓住了公主（P）并将她关在了地下城的右下角。地下城是由 $M \times N$ 个房间组成的二维网格。我们英勇的骑士（K）最初被安置在左上角的房间里，他必须穿过地下城并通过对抗恶魔来拯救公主。

骑士的初始健康点数为一个正整数。如果他的健康点数在某一时刻降至 0 或以下，他会立即死亡。

有些房间由恶魔守卫，因此骑士在进入这些房间时会失去健康点数（若房间里的值为负整数，则表示骑士将损失健康点数）；其他房间要么是空的（房间里的值为 0），要么包含增加骑士健康点数的魔法球（若房间里的值为正整数，则表示骑士将增加健康点数）。

为了尽快到达公主，骑士决定每次只向右或向下移动一步。

编写一个函数来计算确保骑士能够拯救到公主所需的最低初始健康点数。

例如，考虑到如下布局的地下城，如果骑士遵循最佳路径 右 -> 右 -> 下 -> 下，则骑士的初始健康点数至少为 7。

-2 (K)	-3	3
-5	-10	1
10	30	-5 (P)

说明：

- 骑士的健康点数没有上限。
- 任何房间都可能对骑士的健康点数造成威胁，也可能增加骑士的健康点数，包括骑士进入的左上角房间以及公主被监禁的右下角房间。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 地下城游戏（ LeetCode 174 ）：https://leetcode-cn.com/problems/dungeon-game/
class solution {
    public int calculateMinimumHP(int[][] dungeon) {

        // m 表示有多少行
        int m = dungeon.length;

        // n 表示每一行有多少个元素，即 n 表示有多少列
```

```

int n = dungeon[0].length;

// 保存每个位置的最小生命值
// dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
// dp[0][i] 表示骑士来到第 0 行第 i 列时必须拥有的最小生命值
// dp[j][0] 表示骑士来到第 j 行第 0 列时必须拥有的最小生命值
// dp[i][j] 表示骑士来到第 i 行第 j 列时必须拥有的最小生命值
int[][] dp = new int[m][n];

// 先计算最右下角的值，因为骑士必须到达最右下角才能拯救公主
// 最右下角的值有可能是 0、正数、负数
// 如果为 0 或者 正数，那么骑士到达这个位置之前只需要拥有最低存活的生命值 1 滴血即可
// 如果为 负数，那么会在这个位置损失生命值，为了能够继续往后面走，那么必须至少拥有能够抵抗损耗
的生命值
// 比如，负数为 -5，那么必须至少拥有 6 点血，才足够损耗后还拥有最低存活的生命值 1 滴血
// 所以，骑士在这个位置的血量要么是 1，要么是 (1 - 负数)，比如 (1 - (-5)) = 6
dp[m - 1][n - 1] = Math.max(1, 1 - dungeon[m - 1][n - 1]);

// 当骑士来到最后一列的时候，他只能向下移动
for( int i = m - 2 ; i >= 0 ; i-- ){

    // dp[1][0] + dungeon[1][0] >= dp[2][0]
    // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    // 2、dp[1][0] >= 1
    // dp[1][0] = Math.max(1, dp[2][0] - dungeon[1][0])

    dp[i][n - 1] = Math.max(1, dp[i + 1][n - 1] - dungeon[i][n - 1]
]);
}

// 当骑士来到最后一行的时候，他只能向右移动
for( int j = n - 2 ; j >= 0 ; j-- ){

    // dp[1][0] + dungeon[1][0] >= dp[2][0]
    // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    // 2、dp[1][0] >= 1
    // dp[1][0] = Math.max(1, dp[2][0] - dungeon[1][0])

    dp[m - 1][j] = Math.max(1, dp[m - 1][j + 1] - dungeon[m - 1][j
]);
}

// 从右下角开始向左上角推导
// 相当于公主救骑士
for( int i = m - 2 ; i >= 0 ; i--){

    for( int j = n - 2 ; j >= 0 ; j--){

```

```

        // 当骑士来到 ( i , j ) 这个位置时, 可以选择向右移动, 也可以选择向下移动
        // 骑士会选择对血量要求少的那个方向
        int min = Math.min(dp[ i + 1 ][ j ] , dp[ i ][ j + 1 ]);

        // dp[1][0] + dungeon[1][0] >= dp[2][0]
        // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
        // 2、dp[1][0] >= 1
        // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])
        dp[i][j] = Math.max( 1 , min - dungeon[i][j]);

    }
}

// dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
return dp[0][0];
}
}

```

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 地下城游戏 ( LeetCode 174 ) : https://leetcode-cn.com/problems/dungeon-game/
class Solution {
    public int calculateMinimumHP(int[][] dungeon) {

        // m 表示有多少行
        int m = dungeon.length;

        // n 表示每一行有多少个元素, 即 n 表示有多少列
        int n = dungeon[0].length;

        // 保存每个位置的最小生命值
        // dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
        // dp[0][i] 表示骑士来到第 0 行第 i 列时必须拥有的最小生命值
        // dp[j][0] 表示骑士来到第 j 行第 0 列时必须拥有的最小生命值
        // dp[i][j] 表示骑士来到第 i 行第 j 列时必须拥有的最小生命值
        int[][] dp = new int[m][n];

        // 先计算最右下角的值, 因为骑士必须到达最右下角才能拯救公主
        // 最右下角的值有可能是 0 、正数、负数
        // 如果为 0 或者 正数, 那么骑士到达这个位置之前只需要拥有最低存活的生命值 1 滴血即可
        // 如果为 负数, 那么会在这个位置损失生命值, 为了能够继续往后面走, 那么必须至少拥有能够抵抗损耗
        的生命值
        // 比如, 负数为 -5, 那么必须至少拥有 6 点血, 才足够损耗后还拥有最低存活的生命值 1 滴血
        // 所以, 骑士在这个位置的血量要么是 1 , 要么是 (1 - 负数) , 比如 ( 1 - (-5) ) = 6
    }
}

```

```

if( dungeon[ m - 1 ][ n - 1 ] >= 0 ){
    dp[ m - 1 ][ n - 1 ] = 1;
}else{
    dp[ m - 1 ][ n - 1 ] = 1 - dungeon[ m - 1 ][ n - 1 ];
}

// 当骑士来到最后一列的时候，他只能向下移动
for( int i = m - 2 ; i >= 0 ; i-- ){

    // dp[1][0] + dungeon[1][0] >= dp[2][0]
    // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    // 2、dp[1][0] >= 1
    // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])

    dp[ i ][ n - 1 ] = Math.max( 1 , dp[ i + 1 ][ n - 1 ] - dungeon[ i ][ n - 1
]);
}

// 当骑士来到最后一行的时候，他只能向右移动
for( int j = n - 2 ; j >= 0 ; j-- ){

    // dp[1][0] + dungeon[1][0] >= dp[2][0]
    // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    // 2、dp[1][0] >= 1
    // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])

    dp[ m - 1 ][ j ] = Math.max( 1 , dp[ m - 1 ][ j + 1 ] - dungeon[ m - 1 ][ j
]);
}

// 从右下角开始向左上角推导
// 相当于公主救骑士
for( int i = m - 2 ; i >= 0 ; i--){

    for( int j = n - 2 ; j >= 0 ; j--){

        // 当骑士来到 ( i , j ) 这个位置时，可以选择向右移动，也可以选择向下移动
        // 骑士会选择对血量要求少的那个方向
        int min = Math.min(dp[ i + 1 ][ j ] , dp[ i ][ j + 1 ]);

        // dp[1][0] + dungeon[1][0] >= dp[2][0]
        // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
        // 2、dp[1][0] >= 1
        // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])
        dp[i][j] = Math.max( 1 , min - dungeon[i][j]);

    }
}

```

```

    }

    // dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
    return dp[0][0];
}
}

```

2、**C++** 代码

```

class solution {
public:
    int calculateMinimumHP(vector<vector<int>>& dungeon) {

        // m 表示有多少行
        int m = dungeon.size();

        // n 表示每一行有多少个元素，即 n 表示有多少列
        int n = dungeon[0].size();

        // 保存每个位置的最小生命值
        // dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
        // dp[0][i] 表示骑士来到第 0 行第 i 列时必须拥有的最小生命值
        // dp[j][0] 表示骑士来到第 j 行第 0 列时必须拥有的最小生命值
        // dp[i][j] 表示骑士来到第 i 行第 j 列时必须拥有的最小生命值
        auto dp = vector < vector < int>> ( m ,vector<int> (n));

        // 先计算最右下角的值，因为骑士必须到达最右下角才能拯救公主
        // 最右下角的值有可能是 0 、正数、负数
        // 如果为 0 或者 正数，那么骑士到达这个位置之前只需要拥有最低存活的生命值 1 滴血即可
        // 如果为 负数，那么会在这个位置损失生命值，为了能够继续往后面走，那么必须至少拥有能够抵抗损耗
        的生命值
        // 比如，负数为 -5，那么必须至少拥有 6 点血，才足够损耗后还拥有最低存活的生命值 1 滴血
        // 所以，骑士在这个位置的血量要么是 1，要么是 (1 - 负数)，比如 (1 - (-5)) = 6
        if( dungeon[ m - 1 ][ n - 1 ] >= 0 ){
            dp[ m - 1 ][ n - 1 ] = 1;
        }else{
            dp[ m - 1 ][ n - 1 ] = 1 - dungeon[ m - 1 ][ n - 1 ];
        }

        // 当骑士来到最后一列的时候，他只能向下移动
        for( int i = m - 2 ; i >= 0 ; i-- ){

            // dp[1][0] + dungeon[1][0] >= dp[2][0]
            // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
            // 2、dp[1][0] >= 1
            // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])

            dp[ i ][ n - 1 ] = max( 1 , dp[ i + 1 ][ n - 1 ] - dungeon[ i ][ n - 1 ] );
        }
    }
}

```

```

// 当骑士来到最后一行的时候，他只能向右移动
for( int j = n - 2 ; j >= 0 ; j-- ){

    // dp[1][0] + dungeon[1][0] >= dp[2][0]
    // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    // 2、dp[1][0] >= 1
    // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])

    dp[ m - 1][ j ] = max( 1 , dp[ m - 1][ j + 1 ] - dungeon[ m - 1 ][ j ] );
}

// 从右下角开始向左上角推导
// 相当于公主救骑士
for( int i = m - 2 ; i >= 0 ; i--){

    for( int j = n - 2 ; j >= 0 ; j--){

        // 当骑士来到 ( i , j ) 这个位置时，可以选择向右移动，也可以选择向下移动
        // 骑士会选择对血量要求少的那个方向
        int mmin = min(dp[ i + 1 ][ j ] , dp[ i ][ j + 1 ] );

        // dp[1][0] + dungeon[1][0] >= dp[2][0]
        // 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
        // 2、dp[1][0] >= 1
        // dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])
        dp[i][j] = max( 1 , mmin - dungeon[i][j] );

    }
}

// dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
return dp[0][0];

}
};

```

3、Python 代码

```

class Solution:
    def calculateMinimumHP(self, dungeon: List[List[int]]) -> int:
        # m 表示有多少行
        m = len(dungeon)

        # n 表示每一行有多少个元素，即 n 表示有多少列
        n = len(dungeon[0])

```

```

# 保存每个位置的最小生命值
# dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
# dp[0][i] 表示骑士来到第 0 行第 i 列时必须拥有的最小生命值
# dp[j][0] 表示骑士来到第 j 行第 0 列时必须拥有的最小生命值
# dp[i][j] 表示骑士来到第 i 行第 j 列时必须拥有的最小生命值
dp = [[0] * n for _ in range(m)]

# 先计算最右下角的值，因为骑士必须到达最右下角才能拯救公主
# 最右下角的值有可能是 0、正数、负数
# 如果为 0 或者 正数，那么骑士到达这个位置之前只需要拥有最低存活的生命值 1 滴血即可
# 如果为 负数，那么会在这个位置损失生命值，为了能够继续往后面走，那么必须至少拥有能够抵抗损耗
的生命值
# 比如，负数为 -5，那么必须至少拥有 6 点血，才足够损耗后还拥有最低存活的生命值 1 滴血
# 所以，骑士在这个位置的血量要么是 1，要么是 (1 - 负数)，比如 (1 - (-5)) = 6
if dungeon[ m - 1 ][ n - 1 ] >= 0 :
    dp[ m - 1 ][ n - 1 ] = 1
else:
    dp[ m - 1 ][ n - 1 ] = 1 - dungeon[ m - 1 ][ n - 1 ]

# 当骑士来到最后一列的时候，他只能向下移动
for i in range( m - 2 , -1 , -1 ) :

    # dp[1][0] + dungeon[1][0] >= dp[2][0]
    # 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    # 2、dp[1][0] >= 1
    # dp[1][0] = Math.max( 1, dp[2][0] - dungeon[1][0] )

    dp[ i ][ n - 1 ] = max( 1 , dp[ i + 1 ][ n - 1 ] - dungeon[ i ][ n - 1 ] )

# 当骑士来到最后一行的时候，他只能向右移动
for j in range( n - 2 , -1 , -1 ) :

    # dp[1][0] + dungeon[1][0] >= dp[2][0]
    # 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
    # 2、dp[1][0] >= 1
    # dp[1][0] = Math.max( 1, dp[2][0] - dungeon[1][0] )

    dp[ m - 1 ][ j ] = max( 1 , dp[ m - 1 ][ j + 1 ] - dungeon[ m - 1 ][ j ] )

# 从右下角开始向左上角推导
# 相当于公主救骑士
for i in range( m - 2 , -1 , -1 ) :

    for j in range( n - 2 , -1 , -1 ) :

```



```

# 当骑士来到 ( i , j ) 这个位置时, 可以选择向右移动, 也可以选择向下移动
# 骑士会选择对血量要求少的那个方向
mmin = min(dp[ i + 1 ][ j ] , dp[ i ][ j + 1 ])

# dp[1][0] + dungeon[1][0] >= dp[2][0]
# 1、dp[1][0] >= dp[2][0] - dungeon[1][0]
# 2、dp[1][0] >= 1
# dp[1][0] = Math.max( 1,dp[2][0] - dungeon[1][0])
dp[i][j] = max( 1 , mmin - dungeon[i][j])

# dp[0][0] 表示骑士来到第 0 行第 0 列时必须拥有的最小生命值
return dp[0][0]

```

LeetCode 279、完全平方数

一、题目描述

给定正整数 n ，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n 。你需要让组成和的完全平方数的个数最少。

给你一个整数 n ，返回和为 n 的完全平方数的 **最少数量**。

二、题目解析

三、参考代码

1、Java 代码

```

class Solution {
    public int numSquares(int n) {

        // 设置一个数组, 用来存储小于 n 的那些完全平方数
        List<Integer> square = new ArrayList<>();

        // 通过一个下标来计算
        int idx = 1;

        // 直到 idx * idx 超过了 n 为止
        while (idx * idx <= n) {
            // square 存储小于 n 的那些完全平方数
            square.add(idx * idx);

            // idx 累加
            idx++;
        }
    }
}

```

```

// dp[0] 表示数字 0 需要完全平方数的最少数量
// dp[1] 表示数字 0 需要完全平方数的最少数量
// dp[i] 表示数字 i 需要完全平方数的最少数量
int[] dp = new int[ n + 1 ];

// 先让 dp 初始化为 -1, 代表 dp[i] 还没有计算
Arrays.fill(dp,-1);

// dp[0] 表示数字 0 需要完全平方数的最少数量
dp[0] = 0;

// 开始填充 dp[]
for(int i = 1 ; i <= n ;i++){

    // 在每次填充的过程中, 都去遍历 square 数组
    for(int j = 0 ;j < square.size();j++){
        // 如果发现此时 square 的元素值大于了 i
        // 那么 square 后面的那些元素没有必要参与进来计算 i 了
        // 直接退出当前的 j 的循环判断, 让 i++
        if(square.get(j) > i){
            break;
        }

        // 否则, 如果 dp[i] 还没有找到数字 i 需要完全平方数的最少数量
        // 或者此时计算的新值更小, 那么更新 dp[i]
        if(dp[i] == -1 || dp[i] > dp[i-square.get(j)] + 1){

            // 更新 dp[i]
            // dp[i] 表示数字 i 需要完全平方数的最少数量
            // 这个时候 dp[i] 为获取数字为 square.get(j) 的那 1 个完全平方数
            // 加上获取数字为 i-square.get(j) 最少需要 dp[i-square.get(j)] 个数
            dp[i] = dp[i-square.get(j)] + 1;
        }
    }
}

// dp[n] 表示数字 n 需要完全平方数的最少数量
// 返回这个结果就行
return dp[n];
}
}

```

2、**C++** 代码

```
class Solution {
public:
    int numSquares(int n) {
        // 设置一个数组，用来存储小于 n 的那些完全平方数
        vector<int> square;

        // 通过一个下标来计算
        int idx = 1;

        // 直到 idx * idx 超过了 n 为止
        while (idx * idx <= n) {
            // square 存储小于 n 的那些完全平方数
            square.push_back(idx * idx);

            // idx 累加
            idx++;
        }

        // dp[0] 表示数字 0 需要完全平方数的最少数量
        // dp[1] 表示数字 0 需要完全平方数的最少数量
        // dp[i] 表示数字 i 需要完全平方数的最少数量
        // 先让 dp 初始化为 -1，代表 dp[i] 还没有计算
        vector<int> dp(n + 1, -1);

        // dp[0] 表示数字 0 需要完全平方数的最少数量
        dp[0] = 0;

        // 开始填充 dp[]
        for(int i = 1 ; i <= n ; i++){

            // 在每次填充的过程中，都去遍历 square 数组
            for(int j = 0 ; j < square.size(); j++){
                // 如果发现此时 square 的元素值大于了 i
                // 那么 square 后面的那些元素没有必要参与进来计算 i 了
                // 直接退出当前的 j 的循环判断，让 i++
                if(square[j] > i){
                    break;
                }

                // 否则，如果 dp[i] 还没有找到数字 i 需要完全平方数的最少数量
                // 或者此时计算的新值更小，那么更新 dp[i]
                if(dp[i] == -1 || dp[i] > dp[i-square[j]] + 1){

                    // 更新 dp[i]
                    // dp[i] 表示数字 i 需要完全平方数的最少数量
                    // 这个时候 dp[i] 为获取数字为 square.get(j) 的那 1 个完全平方数
                    // 加上获取数字为 i-square.get(j) 最少需要 dp[i-square.get(j)] 个数
                }
            }
        }
    }
};
```

```

        dp[i] = dp[i-square[j]] + 1;
    }
}

// dp[n] 表示数字 n 需要完全平方数的最少数量
// 返回这个结果就行
return dp[n];
}
};

```

3、Python 代码

```

class Solution:
    def numSquares(self, n: int) -> int:
        # 设置一个数组，用来存储小于 n 的那些完全平方数
        square = []

        # 通过一个下标来计算
        idx = 1

        # 直到 idx * idx 超过了 n 为止
        while idx * idx <= n :
            # square 存储小于 n 的那些完全平方数
            square.append(idx * idx)

            # idx 累加
            idx += 1

        # dp[0] 表示数字 0 需要完全平方数的最少数量
        # dp[1] 表示数字 0 需要完全平方数的最少数量
        # dp[i] 表示数字 i 需要完全平方数的最少数量
        # 先让 dp 初始化为 -1，代表 dp[i] 还没有计算
        dp = [-1 for _ in range( n + 1 )]

        # dp[0] 表示数字 0 需要完全平方数的最少数量
        dp[0] = 0

        # 开始填充 dp[]
        for i in range( 1 , n + 1 ) :

            # 在每次填充的过程中，都去遍历 square 数组
            for j in range( 0 , len(square) ) :
                # 如果发现此时 square 的元素值大于了 i
                # 那么 square 后面的那些元素没有必要参与进来计算 i 了
                # 直接退出当前的 j 的循环判断，让 i++
                if square[j] > i:
                    break

```

```

# 否则, 如果 dp[i] 还没有找到数字 i 需要完全平方数的最少数量
# 或者此时计算的新值更小, 那么更新 dp[i]
if dp[i] == -1 or dp[i] > dp[i-square[j]] + 1 :

    # 更新 dp[i]
    # dp[i] 表示数字 i 需要完全平方数的最少数量
    # 这个时候 dp[i] 为获取数字为 square.get(j) 的那 1 个完全平方数
    # 加上获取数字为 i-square.get(j) 最少需要 dp[i-square.get(j)] 个数
    dp[i] = dp[i-square[j]] + 1

# dp[n] 表示数字 n 需要完全平方数的最少数量
# 返回这个结果就行
return dp[n]

```

LeetCode 343、整数拆分

一、题目描述

给定一个正整数 n ，将其拆分为至少两个正整数的和，并使这些整数的乘积最大化。返回你可以获得的最大乘积。

示例 1:

输入: 2
输出: 1
解释: $2 = 1 + 1$, $1 \times 1 = 1$ 。

示例 2:

输入: 10
输出: 36
解释: $10 = 3 + 3 + 4$, $3 \times 3 \times 4 = 36$ 。

说明: 你可以假设 n 不小于 2 且不大于 58。

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 整数拆分 ( LeetCode 343 ) : https://leetcode-cn.com/problems/integer-break/
class Solution {

```

```

public int integerBreak(int n) {

    // dp[2] 表示正整数 2 拆分之后结果的最大乘积
    // dp[3] 表示正整数 3 拆分之后结果的最大乘积
    // dp[i] 表示正整数 i 拆分之后结果的最大乘积
    int[] dp = new int[ n + 1 ];

    // 你可以假设 n 不小于 2 且不大于 58
    // 初始化 dp[2]
    dp[2] = 1;

    // 填充数组 dp 里面的值
    for (int i = 3; i <= n; i++) {

        for (int j = 1; j < i - 1; j++) {
            // 对于正整数 i , 有两种拆法
            // 1、j * ( i - j ) 表示把 i 拆分为 j 和 i - j 这两个数

            // 2、j * dp[ i - j ] 表示把 i 拆分为 j 和 dp[ i - j ] 这两个数
            // dp[ i - j ] 表示正整数 i - j 拆分之后结果的最大乘积，获取到这个乘积之后，再与
j 相乘

            // 并且，并不是说拆分之后乘积就必然大于 i，比如 i = 2，拆分之后的乘积为 1

            // 因此，需要比较这三者，取较大值
            dp[i] = Math.max(dp[i], Math.max(j * ( i - j ), j * dp[ i - j ]));

        }
    }

    // 返回这个结果即可
    return dp[n];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int integerBreak(int n) {
        // dp[2] 表示正整数 2 拆分之后结果的最大乘积
        // dp[3] 表示正整数 3 拆分之后结果的最大乘积
        // dp[i] 表示正整数 i 拆分之后结果的最大乘积
        vector<int> dp( n + 1 );

        // 你可以假设 n 不小于 2 且不大于 58
        // 初始化 dp[2]
        dp[2] = 1;
    }
}

```

```

// 填充数组 dp 里面的值
for (int i = 3; i <= n; i++) {

    for (int j = 1; j < i - 1; j++) {
        // 对于正整数 i , 有两种拆法
        // 1、j * ( i - j ) 表示把 i 拆分为 j 和 i - j 这两个数

        // 2、j * dp[ i - j ] 表示把 i 拆分为 j 和 dp[ i - j ] 这两个数
        // dp[ i - j ] 表示正整数 i - j 拆分之后结果的最大乘积, 获取到这个乘积之后, 再与
j 相乘

        // 并且, 并不是说拆分之后乘积就必然大于 i, 比如 i = 2, 拆分之后的乘积为 1

        // 因此, 需要比较这三者, 取较大值
        dp[i] = max(dp[i], max(j * ( i - j ), j * dp[ i - j ]));

    }
}

// 返回这个结果即可
return dp[n];
}
};

```

3、Python 代码

```

class Solution:
    def integerBreak(self, n: int) -> int:
        # dp[2] 表示正整数 2 拆分之后结果的最大乘积
        # dp[3] 表示正整数 3 拆分之后结果的最大乘积
        # dp[i] 表示正整数 i 拆分之后结果的最大乘积
        dp = [ 0 for _ in range( n + 1 )]

        # 你可以假设 n 不小于 2 且不大于 58
        # 初始化 dp[2]
        dp[2] = 1

        # 填充数组 dp 里面的值
        for i in range( 3 , n + 1 ) :

            for j in range ( 1 , i - 1 ):
                # 对于正整数 i , 有两种拆法
                # 1、j * ( i - j ) 表示把 i 拆分为 j 和 i - j 这两个数

                # 2、j * dp[ i - j ] 表示把 i 拆分为 j 和 dp[ i - j ] 这两个数
                # dp[ i - j ] 表示正整数 i - j 拆分之后结果的最大乘积, 获取到这个乘积之后, 再与
j 相乘

                # 并且, 并不是说拆分之后乘积就必然大于 i, 比如 i = 2, 拆分之后的乘积为 1

```

```
# 因此，需要比较这三者，取较大值
dp[i] = max(dp[i], max(j * (i - j), j * dp[i - j]))

# 返回这个结果即可
return dp[n]
```

第四十七天

LeetCode 516、最长回文子序列

一、题目描述

给你一个字符串 `s`，找出其中最长的回文子序列，并返回该序列的长度。

子序列定义为：不改变剩余字符顺序的情况下，删除某些字符或者不删除任何字符形成的一个序列。

示例 1：

输入：s = "bbbab"
输出：4
解释：一个可能的最长回文子序列为 "bbbb"。

示例 2：

输入：s = "cbbd"
输出：2
解释：一个可能的最长回文子序列为 "bb"。

提示：

- `1 <= s.length <= 1000`
- `s` 仅由小写英文字母组成

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
```


// 最长回文子序列 (LeetCode 516) : <https://leetcode-cn.com/problems/longest-palindromic-subsequence>

```
class Solution {
    public int longestPalindromeSubseq(String s) {

        // 获取字符串 s 的长度
        int length = s.length();

        // 设置数组 dp, 用来存储字符串 s 的最长的回文子序列的长度
        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        // dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子序列的长度
        // i 最大值为 length - 1
        int[][] dp = new int[length][ length ];

        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        // dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        // dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的最长的回文子序列的长度
        // 此时, 这个区间的字符只有一个, 最长的回文子序列的长度为 1
        for (int i = 0; i < length; i++) {
            dp[i][i] = 1;
        }

        // i 从字符串 s 的【尾部】开始向前遍历, j 从 i + 1 开始向后遍历
        // 不断的逼近二维数组最右上角的位置, 即求 dp[0][length - 1]
        for (int i = length - 1; i >= 0; i--) {

            for( int j = i + 1 ; j < length ; j++ ){

                // 如果发现 s.charAt(i) == s.charAt(j)
                if (s.charAt(i) == s.charAt(j)) {

                    // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子
                    // 序列的长度
                    // dp[i][j] 表示字符串 s 第 i + 1 个字符和字符串 s 第 j - 1 个字符之间的最
                    // 长的回文子序列的长度
                    // 由于扩充的两个字符相等, 意味着最长的回文子序列的长度加 2
                    dp[i][j] = dp[ i + 1 ][ j - 1 ] + 2;

                }else{

                    // 字符区间 s[i..j] 里最长回文子序列的长度
                    // 1、去掉头以后的区间的最长的回文子序列的长度, 即 dp[i + 1][j]
                    // 2、去掉尾以后的区间的最长的回文子序列的长度, 即 dp[i][j - 1]
                    // 二者取最大值
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }
    }
}
```

```

    }

    // 最右上角的值就是结果
    return dp[0][length - 1];

}
}

```

2、**C++** 代码

```

class Solution {
public:
    int longestPalindromeSubseq(string s) {
        // 获取字符串 s 的长度
        int length = s.length();

        // 设置数组 dp，用来存储字符串 s 的最长的回文子序列的长度
        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        // dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子序列的长度
        // i 最大值为 length - 1
        auto dp = vector < vector < int>> ( length ,vector<int> ( length ));

        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        // dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        // dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的最长的回文子序列的长度
        // 此时，这个区间的字符只有一个，最长的回文子序列的长度为 1
        for (int i = 0; i < length; i++) {
            dp[i][i] = 1;
        }

        // i 从字符串 s 的【尾部】开始向前遍历，j 从 i + 1 开始向后遍历
        // 不断的逼近二维数组最右上角的位置，即求 dp[0][length - 1]
        for (int i = length - 1; i >= 0; i--) {

            for( int j = i + 1 ; j < length ; j++ ){

                // 如果发现 s.charAt(i) == s.charAt(j)
                if (s[i] == s[j]) {

                    // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子
                    序列的长度

                    // dp[i][j] 表示字符串 s 第 i + 1 个字符和字符串 s 第 j - 1 个字符之间的最
                    长的回文子序列的长度

                    // 由于扩充的两个字符相等，意味着最长的回文子序列的长度加 2
                    dp[i][j] = dp[ i + 1 ][ j - 1 ] + 2;

                }else{

```

```

        // 字符区间 s[i..j] 里最长回文子序列的长度
        // 1、去掉头以后的区间的最长的回文子序列的长度，即 dp[i + 1][j]
        // 2、去掉尾以后的区间的最长的回文子序列的长度，即 dp[i][j - 1]
        // 二者取最大值
        dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
    }
}

// 最右上角的值就是结果
return dp[0][length - 1];
}
};

```

3、Python 代码

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        # 获取字符串 s 的长度
        length = len(s)

        # 设置数组 dp，用来存储字符串 s 的最长的回文子序列的长度
        # dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        # dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        # dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子序列的长度
        # i 最大值为 length - 1

        dp = [[0] * (length) for _ in range(length)]

        # dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的最长的回文子序列的长度
        # dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的最长的回文子序列的长度
        # dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的最长的回文子序列的长度
        # 此时，这个区间的字符只有一个，最长的回文子序列的长度为 1
        for i in range(0, length):
            dp[i][i] = 1

        # i 从字符串 s 的【尾部】开始向前遍历，j 从 i + 1 开始向后遍历
        # 不断的逼近二维数组最右上角的位置，即求 dp[0][length - 1]
        for i in range(length - 1, -1, -1):
            for j in range(i + 1, length):
                # 如果发现 s.charAt(i) == s.charAt(j)
                if s[i] == s[j]:

```

```

# dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的最长的回文子序
列的长度

# dp[i][j] 表示字符串 s 第 i + 1 个字符和字符串 s 第 j - 1 个字符之间的最长
的回文子序列的长度

# 由于扩充的两个字符相等，意味着最长的回文子序列的长度加 2
dp[i][j] = dp[i + 1][j - 1] + 2

else:

    # 字符区间 s[i..j] 里最长回文子序列的长度
    # 1、去掉头以后的区间的最长的回文子序列的长度，即 dp[i + 1][j]
    # 2、去掉尾以后的区间的最长的回文子序列的长度，即 dp[i][j - 1]
    # 二者取最大值
    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

# 最右上角的值就是结果
return dp[0][length - 1]

```

LeetCode 53、最大子数组和

一、题目描述

给定一个整数数组 nums，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

二、题目解析

三、参考代码

1、Java 代码

```

class Solution {
    public int maxSubArray(int[] nums) {

        // 因为后面会使用到 nums 的长度
        // 所以先进行判空操作
        // 如果数组 nums 为空，返回 0
        if(nums.length == 0) return 0;

        // 获取数组的长度
        int n = nums.length;

        // 设置一个数组 dp，长度和数组 nums 长度一致
        // dp[0] 表示以第 0 个元素结尾的最大子数组的和
        // dp[1] 表示以第 1 个元素结尾的最大子数组的和
        // dp[i] 表示以第 i 个元素结尾的最大子数组的和
        int[] dp = new int[n];

        // dp[0] 表示以第 0 个元素结尾的最大子数组的和
    }
}

```

```

// 初始化 dp[0]
dp[0] = nums[0];

// 变量 maxNum 表示数组 dp 中最大的那个值
// 即 maxNum 表示最大的连续字段和
int maxNum = dp[0];

// 从 1 开始遍历数组 nums
for(int i = 1 ; i < n ; i++){

    // 在遍历的过程中，去获取以第 i 个元素结尾的最大子数组的和
    // 如果以 nums[i-1] 结尾的最大字段和为正数
    // 那么以第 i 个元素结尾的最大子数组的和就是自己本身加上以 nums[i-1] 结尾的最大字段和
    if(dp[i-1] > 0){
        // dp[i-1] 是正数
        // 所以 dp[i] 的值为 nums[i] 加上 dp[i-1]
        // 因为 正数 + 变量 > 变量
        // dp[i -1] + nums[i] > nums[i]
        dp[i] = dp[i -1] + nums[i];

        // 否则 dp[i-1] 不是正数，为负数或者 0
    }else{
        // 那么 dp[i] 的值为 nums[i]
        // 因为 负数 + 变量 < 变量
        // dp[i -1] + nums[i] < nums[i]
        dp[i] = nums[i];
    }

    // 在更新 dp[i] 的过程中，更新 maxNum 的值
    // 如果此时 dp[i] 的值大于了 maxNum
    if(maxNum < dp[i]){
        // 那么 maxNum 更新为 dp[i]
        maxNum = dp[i];
    }
}

// 最后返回 maxNum
return maxNum;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        // 因为后面会使用到 nums 的长度
        // 所以先进行判空操作
        // 如果数组 nums 为空，返回 0
    }
}

```

```

if(nums.size() == 0) return 0;

// 获取数组的长度
int n = nums.size();

// 设置一个数组 dp, 长度和数组 nums 长度一致
// dp[0] 表示以第 0 个元素结尾的最大子数组的和
// dp[1] 表示以第 1 个元素结尾的最大子数组的和
// dp[i] 表示以第 i 个元素结尾的最大子数组的和
vector<int> dp( n , 0 );

// dp[0] 表示以第 0 个元素结尾的最大子数组的和
// 初始化 dp[0]
dp[0] = nums[0];

// 变量 maxNum 表示数组 dp 中最大的那个值
// 即 maxNum 表示最大的连续字段和
int maxNum = dp[0];

// 从 1 开始遍历数组 nums
for(int i = 1 ; i < n ; i++){

    // 在遍历的过程中, 去获取以第 i 个元素结尾的最大子数组的和
    // 如果以 nums[i-1]结尾的最大字段和为正数
    // 那么以第 i 个元素结尾的最大子数组的和就是自己本身加上以 nums[i-1]结尾的最大字段和
    if(dp[i-1] > 0){
        // dp[i-1] 是正数
        // 所以 dp[i] 的值为 nums[i] 加上 dp[i-1]
        // 因为 正数 + 变量 > 变量
        // dp[i -1] + nums[i] > nums[i]
        dp[i] = dp[i -1] + nums[i];

        // 否则 dp[i-1] 不是正数, 为负数或者 0
    }else{
        // 那么 dp[i] 的值为 nums[i]
        // 因为 负数 + 变量 < 变量
        // dp[i -1] + nums[i] < nums[i]
        dp[i] = nums[i];
    }

    // 在更新 dp[i] 的过程中, 更新 maxNum 的值
    // 如果此时 dp[i] 的值大于了 maxNum
    if(maxNum < dp[i]){
        // 那么 maxNum 更新为 dp[i]
        maxNum = dp[i];
    }
}

// 最后返回 maxNum

```

```
        return maxNum;
    }
};
```

3、Python 代码

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # 因为后面会使用到 nums 的长度
        # 所以先进行判空操作
        # 如果数组 nums 为空, 返回 0
        if len(nums) == 0 : return 0

        # 获取数组的长度
        n = len(nums)

        # 设置一个数组 dp, 长度和数组 nums 长度一致
        # dp[0] 表示以第 0 个元素结尾的最大子数组的和
        # dp[1] 表示以第 1 个元素结尾的最大子数组的和
        # dp[i] 表示以第 i 个元素结尾的最大子数组的和
        dp = [0 for _ in range(n)]

        # dp[0] 表示以第 0 个元素结尾的最大子数组的和
        # 初始化 dp[0]
        dp[0] = nums[0]

        # 变量 maxNum 表示数组 dp 中最大的那个值
        # 即 maxNum 表示最大的连续字段和
        maxNum = dp[0]

        # 从 1 开始遍历数组 nums
        for i in range( 1 , n ) :

            # 在遍历的过程中, 去获取以第 i 个元素结尾的最大子数组的和
            # 如果以 nums[i-1] 结尾的最大字段和为正数
            # 那么以第 i 个元素结尾的最大子数组的和就是自己本身加上以 nums[i-1] 结尾的最大字段和
            if dp[i-1] > 0 :
                # dp[i-1] 是正数
                # 所以 dp[i] 的值为 nums[i] 加上 dp[i-1]
                # 因为 正数 + 变量 > 变量
                # dp[i -1] + nums[i] > nums[i]
                dp[i] = dp[i -1] + nums[i]

            # 否则 dp[i-1] 不是正数, 为负数或者 0
            else:
                # 那么 dp[i] 的值为 nums[i]
                # 因为 负数 + 变量 < 变量
                # dp[i -1] + nums[i] < nums[i]
                dp[i] = nums[i]
```

```
        # 在更新 dp[i] 的过程中, 更新 maxNum 的值
        # 如果此时 dp[i] 的值大于了 maxNum
        if maxNum < dp[i] :
            # 那么 maxNum 更新为 dp[i]
            maxNum = dp[i]

    # 最后返回 maxNum
    return maxNum
```

LeetCode 5、最长回文子串

一、题目描述

给你一个字符串 `s`，找到 `s` 中最长的回文子串。

示例 1：

```
输入: s = "babad"
输出: "bab"
解释: "aba" 同样是符合题意的答案。
```

示例 2：

```
输入: s = "cbbd"
输出: "bb"
```

示例 3：

```
输入: s = "a"
输出: "a"
```

示例 4：

```
输入: s = "ac"
输出: "a"
```

提示：

- `1 <= s.length <= 1000`
- `s` 仅由数字和英文字母（大写和/或小写）组成

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最长回文子串 ( LeetCode 5 ) : https://leetcode-cn.com/problems/longest-palindromic-substring/
class Solution {
    public String longestPalindrome(String s) {

        // 获取字符串 s 的长度
        int length = s.length();

        // 设置数组 dp, 用来存储字符串 s 的 [i,j] 区间的子串是否是回文子串
        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的子串是否是回文子串
        // dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的子串是否是回文子串
        // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的子串是否是回文子串
        // i 最大值为 length - 1
        boolean[][] dp = new boolean[length][length];

        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的子串是否是回文子串
        // dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的子串是否是回文子串
        // dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的子串是否是回文子串
        // 此时, 这个区间的字符只有一个, 肯定是回文子串
        for (int i = 0; i < length; i++) {
            dp[i][i] = true;
        }

        // 设置变量记录最长的回文子串的长度
        int maxLen = 1;

        // 设置变量记录最长的回文子串的开始位置
        // 从后向前寻找
        int begin = length - 1;

        // i 从字符串 s 的【尾部】开始向前遍历, j 从 i + 1 开始向后遍历
        // 不断的逼近二维数组最右上角的位置, 即求 dp[0][length - 1]
        for (int i = length - 1; i >= 0; i--) {

            for (int j = i + 1; j < length; j++) {

                // 如果发现 s.charAt(i) == s.charAt(j)
                if (s.charAt(i) == s.charAt(j)) {

                    // 如果 [i, j] 这个区间中只有 2 个字符, 并且此时两个字符还是一样的
```

间

```
// 那么肯定是回文子串
// 假设 i = 5, j = 6, [i, j] 这个区间中只有 2 个字符
// 如果不加这个判断的话, dp[i][j] = dp[i + 1][j - 1]
// 此时, i + 1 = 6, j - 1 = 5
// [6, 5] 这个区间不存在, 默认值为 false
if( (j - i + 1) == 2){

    dp[i][j] = true;

}

// 否则, 当前这个区间是否是回文子串区间取决于 [i + 1, j - 1] 这个区

dp[i][j] = dp[i + 1][j - 1];

}

}else{
    // 如果发现 s.charAt(i) != s.charAt(j)
    // 那说明 [i, j] 这个区间必然不是回文子串
    dp[i][j] = false;

}

// 更新最长的回文子串长度
if(dp[i][j] && j - i + 1 > maxLen){
    // 更新最长的回文子串长度
    maxLen = j - i + 1;
    // 更新最长的回文子串的开始位置
    begin = i;
}

}

}

// 通过截取的方式返回最长的回文子串
// Java substring() 方法
// beginIndex -- 起始索引 (包括), 索引从 0 开始
// endIndex -- 结束索引 (不包括)
// 0 1 2 3 4
// b a b a d
// 此时, begin = 1, maxLen = 3
// substring(1, 4), 即截取 [1, 4), 左闭右开的区间
// 获取子串 aba
return s.substring(begin, begin + maxLen);
}
}
```

2、**C++** 代码

```
class solution {
public:
    string longestPalindrome(string s) {
        // 获取字符串 s 的长度
        int length = s.length();

        // 设置数组 dp, 用来存储字符串 s 的 [i,j] 区间的子串是否是回文子串
        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的子串是否是回文子串
        // dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的子串是否是回文子串
        // dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的子串是否是回文子串
        // i 最大值为 length - 1
        auto dp = vector < vector < bool>> ( length ,vector<bool> ( length ));

        // dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的的子串是否是回文子串
        // dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的的子串是否是回文子串
        // dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的的子串是否是回文子串
        // 此时, 这个区间的字符只有一个, 肯定是回文子串
        for (int i = 0; i < length; i++) {
            dp[i][i] = true;
        }

        // 设置变量记录最长的回文子串的长度
        int maxLen = 1;

        // 设置变量记录最长的回文子串的开始位置
        // 从后向前寻找
        int begin = length - 1;

        // i 从字符串 s 的【尾部】开始向前遍历, j 从 i + 1 开始向后遍历
        // 不断的逼近二维数组最右上角的位置, 即求 dp[0][length - 1]
        for (int i = length - 1; i >= 0; i--) {
            for( int j = i + 1 ; j < length ; j++ ){

                // 如果发现 s.charAt(i) == s.charAt(j)
                if (s[i] == s[j]) {

                    // 如果 [i , j] 这个区间中只有 2 个字符, 并且此时两个字符还是一样的
                    // 那么肯定是回文子串
                    // 假设 i = 5, j = 6, [i , j] 这个区间中只有 2 个字符
                    // 如果不加这个判断的话, dp[i][j] = dp[ i + 1 ][ j - 1 ]
                    // 此时, i + 1 = 6, j - 1 = 5
                    // [ 6 , 5 ] 这个区间不存在, 默认值为 false
                    if( (j - i + 1) == 2){

                        dp[i][j] = true;
                    }
                }
            }
        }

        return s.substr(begin, maxLen);
    }
};
```

```

        }else{

            // 否则，当前这个区间是否是回文子串区间取决于 [ i + 1 , j - 1 ] 这个区
            间

            dp[i][j] = dp[ i + 1 ][ j - 1 ];

        }

    }else{
        // 如果发现 s.charAt(i) != s.charAt(j)
        // 那说明 [i , j] 这个区间必然不是回文子串
        dp[i][j] = false;

    }

    // 更新最长的回文子串长度
    if(dp[i][j] && j - i + 1 > maxLen){
        // 更新最长的回文子串长度
        maxLen = j - i + 1;
        // 更新最长的回文子串的开始位置
        begin = i;
    }

}

}

// 通过截取的方式返回最长的回文子串
// Java substring() 方法
// beginIndex -- 起始索引（包括），索引从 0 开始
// endIndex -- 结束索引（不包括）
// 0 1 2 3 4
// b a b a d
// 此时, begin = 1, maxLen = 3
// substring( 1 , 4 ), 即截取 [1 , 4 ) , 左闭右开的区间
// 获取子串  aba
return s.substr(begin,  maxLen);

}

};

```

3、Python 代码

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        # 获取字符串 s 的长度
        length = len(s)

```

```

# 设置数组 dp，用来存储字符串 s 的 [i,j] 区间的子串是否是回文子串
# dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的子串是否是回文子串
# dp[2][3] 表示字符串 s 第 2 个字符和字符串 s 第 3 个字符之间的子串是否是回文子串
# dp[i][j] 表示字符串 s 第 i 个字符和字符串 s 第 j 个字符之间的子串是否是回文子串
# i 最大值为 length - 1
dp = [[False] * length for _ in range(length)]

# dp[0][0] 表示字符串 s 第 0 个字符和字符串 s 第 0 个字符之间的子串是否是回文子串
# dp[3][3] 表示字符串 s 第 3 个字符和字符串 s 第 3 个字符之间的子串是否是回文子串
# dp[i][i] 表示字符串 s 第 i 个字符和字符串 s 第 i 个字符之间的子串是否是回文子串
# 此时，这个区间的字符只有一个，肯定是回文子串
for i in range(length):
    dp[i][i] = True

# 设置变量记录最长的回文子串的长度
maxLen = 1

# 设置变量记录最长的回文子串的开始位置
# 从后向前寻找
begin = length - 1

# i 从字符串 s 的【尾部】开始向前遍历，j 从 i + 1 开始向后遍历
# 不断的逼近二维数组最右上角的位置，即求 dp[0][length - 1]
for i in range( length - 1 , -1 , -1 ):

    for j in range( i + 1 , length ):

        # 如果发现 s.charAt(i) == s.charAt(j)
        if s[i] == s[j]:

            # 如果 [i , j] 这个区间中只有 2 个字符，并且此时两个字符还是一样的
            # 那么肯定是回文子串
            # 假设 i = 5, j = 6, [i , j] 这个区间中只有 2 个字符
            # 如果不加这个判断的话，dp[i][j] = dp[ i + 1 ][ j - 1 ]
            # 此时，i + 1 = 6, j - 1 = 5
            # [ 6 , 5 ] 这个区间不存在，默认值为 false
            if (j - i + 1) == 2 :

                dp[i][j] = True

            else:

                # 否则，当前这个区间是否是回文子串区间取决于 [ i + 1 , j - 1 ] 这个区间
                dp[i][j] = dp[ i + 1 ][ j - 1 ]

        else:

```

```

        # 如果发现 s.charAt(i) != s.charAt(j)
        # 那说明 [i , j] 这个区间必然不是回文子串
        dp[i][j] = False

    # 更新最长的回文子串长度
    if dp[i][j] and j - i + 1 > maxLen :
        # 更新最长的回文子串长度
        maxLen = j - i + 1
        # 更新最长的回文子串的开始位置
        begin = i

# 通过截取的方式返回最长的回文子串
return s[begin:begin + maxLen]

```

LeetCode 718、最长重复子数组

一、题目描述

给两个整数数组 **A** 和 **B** ，返回两个数组中公共的、长度最长的子数组的长度。

示例：

输入：
A: [1,2,3,2,1]
B: [3,2,1,4,7]
输出：3
解释：
长度最长的公共子数组是 [3, 2, 1] 。

提示：

- $1 \leq \text{len}(A), \text{len}(B) \leq 1000$
- $0 \leq A[i], B[i] < 100$

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最长重复子数组（ LeetCode 718 ）：https://leetcode-cn.com/problems/maximum-length-of-repeated-subarray

```

```

class Solution {
    public int findLength(int[] nums1, int[] nums2) {

        // 获取 nums1 的长度
        int m = nums1.length;

        // 获取 nums2 的长度
        int n = nums2.length;

        // 设置数组 dp, 用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
        // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
        // 前 i 个元素的区间为 [ 0 , i - 1 ]
        // dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度
        // 前 m 个元素的表示区间为 [ 0 , m ], 前 n 个元素的表示区间为 [ 0 , n ]
        // 因此, dp 数组的长度为 m + 1 和 n + 1
        int[][] dp = new int[m + 1][n + 1];

        // maxLength 表示 dp 数组中的最大值
        int maxLength = 0;

        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // nums1 nums2 也没有元素
        // 因此, 公共的、长度最长的子数组的长度为 0
        dp[0][0] = 0;

        // nums1 没有元素 或者 nums2 没有字符时, 公共的、长度最长的子数组的长度都为 0
        for( int i = 0 ; i <= m ; i++){
            dp[i][0] = 0;
        }

        for( int j = 0 ; j <= n ; j++){
            dp[0][j] = 0;
        }

        // i 从 1 开始, 直到 m 位置, 遍历 nums1 的前 i 个元素
        for (int i = 1; i <= m; i++) {

            // j 从 1 开始, 直到 n 位置, 遍历 nums2 的前 j 个元素
            for (int j = 1; j <= n; j++) {

                // 如果发现 nums1 的当前元素, 即位置为 i - 1 的元素
                // 与 nums2 的当前元素, 即位置为 j - 1 的元素【相同】
                // 此时, 找到了一个公共元素, 公共的、长度最长的子数组的长度加 1
                if (nums1[i - 1] == nums2[j - 1]) {

                    // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
子数组的长度

```

```

        // dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素
        的公共的、长度最长的子数组的长度
        dp[i][j] = dp[i - 1][j - 1] + 1;

        // 更新最长的子数组长度
        maxLength = Math.max(maxLength, dp[i][j]);
    }
}

// 返回结果
return maxLength;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int findLength(vector<int>& nums1, vector<int>& nums2) {
        // 获取 nums1 的长度
        int m = nums1.size();

        // 获取 nums2 的长度
        int n = nums2.size();

        // 设置数组 dp, 用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
        // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
        // 前 i 个元素的区间为 [ 0 , i - 1 ]
        // dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度
        // 前 m 个元素的表示区间为 [ 0 , m ], 前 n 个元素的表示区间为 [ 0 , n ]
        // 因此, dp 数组的长度为 m + 1 和 n + 1
        auto dp = vector < vector < int >> ( m + 1 , vector<int> ( n + 1 ));

        // maxLength 表示 dp 数组中的最大值
        int maxLength = 0;

        // dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        // nums1 nums2 也没有元素
        // 因此, 公共的、长度最长的子数组的长度为 0
        dp[0][0] = 0;

        // nums1 没有元素 或者 nums2 没有字符时, 公共的、长度最长的子数组的长度都为 0
        for( int i = 0 ; i <= m ; i++){
            dp[i][0] = 0;
        }
    }
}

```



```

for( int j = 0 ; j <= n ; j++){
    dp[0][j] = 0;
}

// i 从 1 开始, 直到 m 位置, 遍历 nums1 的前 i 个元素
for (int i = 1; i <= m; i++) {

    // j 从 1 开始, 直到 n 位置, 遍历 nums2 的前 j 个元素
    for (int j = 1; j <= n; j++) {

        // 如果发现 nums1 的当前元素, 即位置为 i - 1 的元素
        // 与 nums2 的当前元素, 即位置为 j - 1 的元素【相同】
        // 此时, 找到了一个公共元素, 公共的、长度最长的子数组的长度加 1
        if (nums1[i - 1] == nums2[j - 1]) {

            // dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
            // 子数组的长度
            // dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素
            // 的公共的、长度最长的子数组的长度
            dp[i][j] = dp[i - 1][j - 1] + 1;

            // 更新最长的子数组长度
            maxLength = max(maxLength, dp[i][j]);
        }
    }
}

// 返回结果
return maxLength;
}
};

```

3、Python 代码

```

class Solution:
    def findLength(self, nums1: List[int], nums2: List[int]) -> int:
        # 获取 nums1 的长度
        m = len(nums1)

        # 获取 nums2 的长度
        n = len(nums2)

        # 设置数组 dp, 用来存储 nums1 和 nums2 公共的、长度最长的子数组的长度
        # dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
        # dp[2][3] 表示 nums1 前 2 个元素和 nums2 前 3 个元素的公共的、长度最长的子数组的长度
        # dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的子数组的长度
        # 前 i 个元素的区间为 [ 0 , i - 1 ]
        # dp[m][n] 表示 nums1 前 m 个元素和 nums2 前 n 个元素的公共的、长度最长的子数组的长度

```

```

# 前 m 个元素的表示区间为 [ 0 , m ]，前 n 个元素的表示区间为 [ 0 , n ]
# 因此，dp 数组的长度为 m + 1 和 n + 1
dp = [[0] * (n + 1) for _ in range( m + 1 )]

# maxLength 表示 dp 数组中的最大值
maxLength = 0

# dp[0][0] 表示 nums1 前 0 个元素和 nums2 前 0 个元素的公共的、长度最长的子数组的长度
# nums1 nums2 也没有元素
# 因此，公共的、长度最长的子数组的长度为 0
dp[0][0] = 0

# nums1 没有元素 或者 nums2 没有字符时，公共的、长度最长的子数组的长度都为 0
for i in range( 0 , m + 1 ) :
    dp[i][0] = 0

for j in range( 0 , n + 1 ) :
    dp[0][j] = 0

# i 从 1 开始，直到 m 位置，遍历 nums1 的前 i 个元素
for i in range( 1 , m + 1 ) :

    # j 从 1 开始，直到 n 位置，遍历 nums2 的前 j 个元素
    for j in range( 1 , n + 1 ) :

        # 如果发现 nums1 的当前元素，即位置为 i - 1 的元素
        # 与 nums2 的当前元素，即位置为 j - 1 的元素【相同】
        # 此时，找到了一个公共元素，公共的、长度最长的子数组的长度加 1
        if nums1[i - 1] == nums2[j - 1] :

            # dp[i][j] 表示 nums1 前 i 个元素和 nums2 前 j 个元素的公共的、长度最长的
            # 子数组的长度
            # dp[i - 1][j - 1] 表示 nums1 前 i - 1 个元素和 nums2 前 j - 1 个元素的
            # 公共的、长度最长的子数组的长度
            dp[i][j] = dp[i - 1][j - 1] + 1

            # 更新最长的子数组长度
            maxLength = max(maxLength, dp[i][j])

# 返回结果
return maxLength

```

LeetCode 139、单词拆分

一、题目描述

给你一个字符串 `s` 和一个字符串列表 `wordDict` 作为字典。请你判断是否可以利用字典中出现的单词拼接出 `s`。

注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例 1：

输入: `s = "leetcode"`, `wordDict = ["leet", "code"]` 输出: `true` 解释: 返回 `true` 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

示例 2：

输入: `s = "applepenapple"`, `wordDict = ["apple", "pen"]` 输出: `true` 解释: 返回 `true` 因为 "applepenapple" 可以由 "apple" "pen" "apple" 拼接成。注意，你可以重复使用字典中的单词。

示例 3：

输入: `s = "catsanddog"`, `wordDict = ["cats", "dog", "sand", "and", "cat"]` 输出: `false`

提示：

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` 和 `wordDict[i]` 仅有小写英文字母组成
- `wordDict` 中的所有字符串 互不相同

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 单词拆分 (LeetCode 139) : https://leetcode.cn/problems/word-break/submissions/
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {

        // wordDict 可能包含重复的字符串，采取一个去重的操作
        Set<String> wordDictSet = new HashSet(wordDict);

        // dp[i] 表示 s 的前 i 位是否可以用 wordDict 中的单词表示
        boolean[] dp = new boolean[s.length() + 1];

        // 一个空字符可以用 wordDict 中的单词表示
```

```

dp[0] = true;

// 一个优化的方案：先去获取 wordDict 最长字符的长度
int maxWordLength = 0;

// 先去获取 wordDict 最长字符的长度
for( int i = 0 ; i < wordDict.size() ; i++){

    String word = wordDict.get(i);

    maxWordLength = Math.max(maxWordLength , word.length());

}

// 开始填充 dp 数组
// 遍历字符串的所有子串
for( int i = 1 ; i <= s.length() ; i++){

    // 在遍历过程中，获取 s 中的子串
    // 判断 s( 0 , i ) 能否用 wordDict 中的单词表示
    // s( 0 , i ) 可以划分为两个部分 s( 0 , j ) 和 s( j , i )
    // 如果发现 ( 0 , j ) 可以用 wordDict 中的单词表示
    // 同时，wordDictSet 还包含了 s( j , i )，那说明 s( 0 , i ) 能否用 wordDict 中的
单词表示

    // 这里采取了一个优化，极大提高了计算效率
    // j 不需要每次都从 0 开始，可以直接提前到距离 i 差 maxWordLength 个字符的位置
    // 因为 wordDict 中最长字符的长度是 maxWordLength
    // 那么 j 从 i - maxWordLength 开始 s( j , i ) 不会忽略掉判断这个最长字符
    // 并且在 i < maxWordLength 这个阶段，index 依旧是从 0 开始，涵盖了 s( 0 , i -
maxWordLength )这些子串的判断
    int index = Math.max( i - maxWordLength , 0 );

    // j 不需要每次都从 0 开始，可以直接提前到距离 i 差 maxWordLength 个字符的位置
    // 接下来的核心点就是判断 s( j , i ) 是否可以被构成一个单词
    for(int j = index ; j < i ; j++){

        // 1、如果发现 s 的前 j 位是否可以用 wordDict 中的单词表示
        // 2、wordDict 还包含了 s.substring(j , i ) 这个子串
        // 即 s 中的前部分 ( 0 , j ) 可以被表示
        // s 中的后部分 ( j , i ) 也可以被表示
        // 那么子串 s ( 0 , i ) 可以被表示
        if(dp[j] && wordDictSet.contains(s.substring(j , i ))){

            // 即 s 的前 i 位可以用 wordDict 中的单词表示
            dp[i] = true;

            // 那么 j 也就没有必要继续执行 ++ 操作了
            break;

```

```

        }
    }

    }

    return dp[s.length()];
}
}

```

LeetCode 198、打家劫舍

一、题目描述

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。**

给定一个代表每个房屋存放金额的非负整数数组，计算你 **不触动警报装置的情况下**，一夜之内能够偷窃到的最高金额。

示例 1：

输入：[1,2,3,1]

输出：4

解释：偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3）。

偷窃到的最高金额 = 1 + 3 = 4。

二、题目解析

三、参考代码

1、Java 代码

```

class Solution {
    public int rob(int[] nums) {

        // 先获取全部房间的总数
        int n = nums.length;

        // 如果数组为空，表示没有房间，那么自然偷不到东西，直接返回 0
        if(n == 0) return 0;

        // 如果数组只有 1 个元素，表示只有 1 个房间，那么只能偷这个房间的东西，直接返回这个房间的金额
        if(n == 1) return nums[0];

        // 设置一个数组 value 用来存放前 n 个房间可以偷取的最大金额
        // value[0] 表示前 0 个房间可以偷取的最大金额
        // value[1] 表示前 1 个房间可以偷取的最大金额
        // value[i] 表示前 i 个房间可以偷取的最大金额
    }
}

```

```

int[] value = new int[n + 1];

// value[0] 表示前 0 个房间可以偷取的最大金额
// 0 个房间自然偷不到东西，所以存放结果为 0
value[0] = 0;

// value[1] 表示前 1 个房间可以偷取的最大金额
// 只有 1 个房间，那么只能偷这个房间的东西，所以存放结果为这个房间的金额
value[1] = nums[0];

// 从 i = 2 直到 i = n, value 中存放的结果由前 i - 2 和 i - 1 共同决定
for(int i = 2 ; i <= n ; i++){

    // 转移方程: value[i] 等于 value[ i - 1 ] 和 value[ i - 2 ] + num[ i - 1 ] 中
    的较大值
    value[i] = Math.max(value[i - 1] ,value[i - 2] + nums[i - 1]);
}

// 最后返回 value 的最后一个值
return value[n];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int rob(vector<int>& nums) {
        // 先获取全部房间的总数
        int n = nums.size();

        // 如果数组为空，表示没有房间，那么自然偷不到东西，直接返回 0
        if(n == 0) return 0;

        // 如果数组只有 1 个元素，表示只有 1 个房间，那么只能偷这个房间的东西，直接返回这个房间的金额
        if(n == 1) return nums[0];

        // 设置一个数组 value 用来存放前 n 个房间可以偷取的最大金额
        // value[0] 表示前 0 个房间可以偷取的最大金额
        // value[1] 表示前 1 个房间可以偷取的最大金额
        // value[i] 表示前 i 个房间可以偷取的最大金额
        vector<int> value( n + 1 );

        // value[0] 表示前 0 个房间可以偷取的最大金额
        // 0 个房间自然偷不到东西，所以存放结果为 0
        value[0] = 0;

        // value[1] 表示前 1 个房间可以偷取的最大金额
    }
}

```

```

// 只有 1 个房间，那么只能偷这个房间的东西，所以存放结果为这个房间的金额
value[1] = nums[0];

// 从 i = 2 直到 i = n, value 中存放的结果由前 i - 2 和 i - 1 共同决定
for(int i = 2 ; i <= n ; i++){

    // 转移方程: value[i] 等于 value[ i - 1 ] 和 value[ i - 2 ] + num[ i - 1 ] 中
    的较大值
    value[i] = max(value[i - 1] ,value[i - 2] + nums[i - 1]);
}

// 最后返回 value 的最后一个值
return value[n];
}
};

```

3、Python 代码

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        # 先获取全部房间的总数
        n = len(nums)

        # 如果数组为空，表示没有房间，那么自然偷不到东西，直接返回 0
        if n == 0 : return 0

        # 如果数组只有 1 个元素，表示只有 1 个房间，那么只能偷这个房间的东西，直接返回这个房间的金额
        if n == 1 : return nums[0]

        # 设置一个数组 value 用来存放前 n 个房间可以偷取的最大金额
        # value[0] 表示前 0 个房间可以偷取的最大金额
        # value[1] 表示前 1 个房间可以偷取的最大金额
        # value[i] 表示前 i 个房间可以偷取的最大金额
        value = [ 0 for _ in range( n + 1)]

        # value[0] 表示前 0 个房间可以偷取的最大金额
        # 0 个房间自然偷不到东西，所以存放结果为 0
        value[0] = 0

        # value[1] 表示前 1 个房间可以偷取的最大金额
        # 只有 1 个房间，那么只能偷这个房间的东西，所以存放结果为这个房间的金额
        value[1] = nums[0]

        # 从 i = 2 直到 i = n, value 中存放的结果由前 i - 2 和 i - 1 共同决定
        for i in range( 2 , n + 1 ) :

            # 转移方程: value[i] 等于 value[ i - 1 ] 和 value[ i - 2 ] + num[ i - 1 ] 中的
            较大值
            value[i] = max(value[i - 1] ,value[i - 2] + nums[i - 1])

```

```
# 最后返回 value 的最后一个值
return value[n]
```

LeetCode 213、打家劫舍II

LeetCode 213、打家劫舍II

一、题目描述

你是一个专业的小偷，计划偷窃沿街的房屋，每间房内都藏有一定的现金。这个地方所有的房屋都围成一圈，这意味着第一个房屋和最后一个房屋是紧挨着的。同时，相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你 **在不触动警报装置的情况下**，今晚能够偷窃到的最高金额。

示例 1：

输入：nums = [2,3,2]

输出：3

解释：你不能先偷窃 1 号房屋（金额 = 2），然后偷窃 3 号房屋（金额 = 2），因为他们是相邻的。

示例 2：

输入：nums = [1,2,3,1]

输出：4

解释：你可以先偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3）。

偷窃到的最高金额 = 1 + 3 = 4。

示例 3：

输入：nums = [0]

输出：0

提示：

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 1000`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 打家劫舍II ( LeetCode 213 ) : https://leetcode-cn.com/problems/house-robber-ii/
class Solution {

    public int rob(int[] nums) {

        // 先获取全部房间的总数
        int n = nums.length;

        // 如果数组为空，表示没有房间，那么自然偷不到东西，直接返回 0
        if(n == 0) return 0;

        // 如果数组只有 1 个元素，表示只有 1 个房间，那么只能偷这个房间的东西，直接返回这个房间的金额
        if(n == 1) return nums[0];

        // 不选择第一个房子
        int[] numsExceptFirstRoom = Arrays.copyOfRange(nums, 1, nums.length);

        // 不选择最后一个房子
        int[] numsExceptLastRoom = Arrays.copyOfRange(nums, 0, nums.length - 1);

        // 从这两个选择中，挑选出最大值
        return Math.max(myRob(numsExceptFirstRoom), myRob(numsExceptLastRoom));

    }

    public int myRob(int[] nums) {

        // 先获取全部房间的总数
        int n = nums.length;

        // 设置一个数组 value 用来存放前 n 个房间可以偷取的最大金额
        // value[0] 表示前 0 个房间可以偷取的最大金额
        // value[1] 表示前 1 个房间可以偷取的最大金额
        // value[i] 表示前 i 个房间可以偷取的最大金额
        int[] value = new int[n + 1];

        // value[0] 表示前 0 个房间可以偷取的最大金额
        // 0 个房间自然偷不到东西，所以存放结果为 0
        value[0] = 0;

        // value[1] 表示前 1 个房间可以偷取的最大金额
        // 只有 1 个房间，那么只能偷这个房间的东西，所以存放结果为这个房间的金额
```

```

value[1] = nums[0];

// 从 i = 2 直到 i = n, value 中存放的结果由前 i - 2 和 i - 1 共同决定
for(int i = 2 ; i <= n ; i++){

    // 转移方程: value[i] 等于 value[ i - 1 ] 和 value[ i - 2 ] + num[ i - 1 ] 中
    的较大值
    value[i] = Math.max(value[i - 1] ,value[i - 2] + nums[i - 1]);
}

// 最后返回 value 的最后一个值
return value[n];

}
}

```

3、Python 代码

```

class Solution:
    def rob(self, nums):
        # 获取房间总数
        n = len(nums)

        # 如果数组为空, 返回0
        if n == 0:
            return 0

        # 如果数组只有一个元素, 返回该元素的金额
        if n == 1:
            return nums[0]

        # 不选择第一个房间
        nums_except_first_room = nums[1:]

        # 不选择最后一个房间
        nums_except_last_room = nums[:-1]

        # 从这两个选择中挑选出最大值
        return max(self.myRob(nums_except_first_room),
self.myRob(nums_except_last_room))

    def myRob(self, nums):
        # 获取房间总数
        n = len(nums)

        # 创建一个数组value来存放前n个房间可以偷取的最大金额
        value = [0] * (n + 1)

        # 前0个房间无法偷取, 金额为0

```

```
value[0] = 0

# 前1个房间只有一个房间可偷取，金额为该房间的金额
value[1] = nums[0]

# 从i = 2到i = n, value中的结果由前i-2和i-1共同决定
for i in range(2, n + 1):
    # 转移方程: value[i]等于value[i-1]和value[i-2]+nums[i-1]中的较大值
    value[i] = max(value[i - 1], value[i - 2] + nums[i - 1])

# 返回value的最后一个值
return value[n]
```

LeetCode 264、丑数II（动态规划）

一、题目描述

我们把只包含质因子 2、3 和 5 的数称作丑数（Ugly Number）。求按从小到大的顺序的第 n 个丑数。

示例:

输入: n = 10 输出: 12 解释: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12 是前 10 个丑数。

说明:

1. 1 是丑数。
2. n 不超过1690。

二、题目解析

对于任意一个丑数来说，它都可以和 2、3、5 进行相乘得到一个丑，那么每个丑数都可以得到三个数，这三个数分别属于不同的数组中。

假设当前已经知道丑数序列 nums 是这些元素：1, 2, 3, 4, 5, 6, 8, 9。

1、nums2 数组存储了这个序列中所有丑数与 2 相乘的结果， $\text{nums2} = \{1 * 2, 2 * 2, 3 * 2, 4 * 2, 5 * 2, 6 * 2, 8 * 2, \dots\}$

2、nums3 数组存储了这个序列中所有丑数与 3 相乘的结果， $\text{nums3} = \{1 * 3, 2 * 3, 3 * 3, 4 * 3, 5 * 3, 6 * 3, 8 * 3, \dots\}$

3、nums5 数组存储了这个序列中所有丑数与 5 相乘的结果， $\text{nums5} = \{1 * 5, 2 * 5, 3 * 5, 4 * 5, 5 * 5, 6 * 5, 8 * 5, \dots\}$

那么，想填充 nums 的话，它的选择来源肯定只能来源于 nums2、nums3、nums5 这三个数组。

并且 nums 序列接下来填充的数是 nums2、nums3、nums5 这三个数组除去已经在 nums 序列的丑数时剩下的最小值。

也就是说，每次寻找丑数的过程是在 nums2、nums3、nums5 这三个数组中寻找最小值的过程。

- 1、在寻找过程中，因为丑数的序列在不断的扩充，nums2、nums3、nums5 这三个数组也在不断的扩充。
- 2、每次找到那个最小值之后，接下来的寻找过程应该忽略它了。

那么，现在来看 nums2、nums3、nums5 这三个数组的起始状态，此时，丑数序列只有一个元素 `ugly[0] = 1`。

- `nums2 : {1 * 2.....}`
- `nums3: {1 * 3.....}`
- `nums5 : {1 * 5.....}`

第二个丑数序列需要去获取 nums2、nums3、nums5 这三个数组的第一个元素的最小值。

那么，此时可以设置三个索引，分别指向 nums2、nums3、nums5 数组开始的位置。

- `index2` 指向 `nums2` 数组
- `index3` 指向 `nums3` 数组
- `index5` 指向 `nums5` 数组

比较的结果是发现 `1 * 2` 最小，于是第二个丑数找到了，此时 `ugly` 就变成 `{1, 2}`，`nums2`、`nums3`、`nums5` 这三个数组也需要得到扩充。

- `nums2 : {1 * 2, 2 * 2, .....}`
- `nums3: {1 * 3, 2 * 3, .....}`
- `nums5 : {1 * 5, 2 * 5, .....}`

刚才我们说过，每次找到那个最小值之后，接下来的寻找过程应该忽略它了，用代码的形式来表示就是索引值向后移动。

比如在 `nums2` 上找到了 `1 * 2` 之后，`index2` 开始移动到 `2 * 2` 的位置。

第三个丑数序列需要去获取 `nums2`、`nums3`、`nums5` 这三个数组中 `index2`、`index3`、`index5` 指向元素的最小值。

- `index2` 指向了 `nums2` 中的 `2 * 2`，即 `nums2[index2]`
- `index3` 指向了 `nums3` 中的 `1 * 3`，即 `nums3[index3]`
- `index5` 指向了 `nums5` 中的 `1 * 5`，即 `nums5[index5]`

如此不断的循环，就可以把 `ugly` 数组填充到 `n` 的位置，得到 `ugly[n]`。

这里需要注意到，`nums2`、`nums3`、`nums5` 这三个数组并没有具体的额外创造出来，它们都在 `ugly` 数组中。

- `nums2` 实际上就是 `ugly[]*2` 的结果
- `nums3` 实际上就是 `ugly[]*3` 的结果
- `nums5` 实际上就是 `ugly[]*5` 的结果

所以比较的过程，实际上就是比较 `nums2[index2] = ugly[index2] * 2`、`nums3[index3] = ugly[index3] * 3`、`nums5[index5] = ugly[index5] * 5` 三者大小的过程，获取到最小值就填充到 `ugly` 数组中，再将对应的索引向后移动，由于 `nums2`、`nums3`、`nums5` 这三个数组中会存在重复元素，所以需要执行去重的操作。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
class solution {
    public int nthUglyNumber(int n) {

        // 设置一个数组，用来存放结果
        // nums[0] 表示第 1 个丑数，结果是 1
        // nums[1] 表示第 2 个丑数，结果是 2
        // nums[n-1] 表示第 n 个丑数，结果需要计算
        int[] nums = new int[n];

        // nums[0] 表示第 1 个丑数，结果是 1
        nums[0] = 1;

        // 对于任意一个丑数来说，它总是可以由前面的丑数与 2、3、5 其中一个进行相乘而来的
        // 那么每个丑数都可以得到三个数，这三个数分别属于不同的数组中
        // nums2 = {1*2, 2*2, 3*2, 4*2, 5*2, 6*2, 8*2...}
        // nums3 = {1*3, 2*3, 3*3, 4*3, 5*3, 6*3, 8*3...}
        // nums5 = {1*5, 2*5, 3*5, 4*5, 5*5, 6*5, 8*5...}
        // nums2 按照从小到大的顺序存放了所有丑数和 2 相乘的结果
        // nums3 按照从小到大的顺序存放了所有丑数和 3 相乘的结果
        // nums5 按照从小到大的顺序存放了所有丑数和 5 相乘的结果

        // nums[i] 的值就是在这三个数组中进行挑选
        // 设置三个索引，分别指向 nums2、nums3、nums5 数组开始的位置
        // 每当 nums[i] 的值从哪个数组中获取到，该索引就在这个数组中向后移动

        // index2 指向 nums2 数组
        int index2 = 0;

        // index3 指向 nums3 数组
        int index3 = 0;

        // index5 指向 nums5 数组
        int index5 = 0;

        // 开始填充 nums 数组
        for (int i = 1; i < n; i++) {

            // nums[i] 的值就是在这三个数组中进行挑选最小值
            // 但是，这三个数组并没有具体的额外创造出来，它们都在 nums 数组中
            // 其中 nums[index2] 表示的就是第 index2 个丑数，nums[index2] * 2 在数组 nums2
            中的位置为 index2
            // 其中 nums[index3] 表示的就是第 index3 个丑数，nums[index3] * 3 在数组 nums3
            中的位置为 index3
```

```

        // 其中 nums[index5] 表示的就是第 index5 个丑数, nums[index5] * 5 在数组 nums5
        中的位置为 index5
        nums[i] = Math.min(Math.min(nums[index2] * 2, nums[index3] * 3),
        nums[index5] * 5);

        // 执行完上面的代码之后, nums[i] 已经被填充完毕
        // 那么和它相等的数需要被挪出接下来的比较

        // nums[i] == nums[index2] * 2
        // 移动 index2 到下一个数
        if (nums[i] == nums[index2] * 2) {
            index2++;
        }

        // nums[i] == nums[index3] * 3
        // 移动 index3 到下一个数
        if (nums[i] == nums[index3] * 3) {
            index3++;
        }

        // nums[i] == nums[index5] * 5
        // 移动 index5 到下一个数
        if (nums[i] == nums[index5] * 5) {
            index5++;
        }
    }

    // 返回结果
    return nums[n - 1];
}
}

```

四、复杂度分析

- 时间复杂度 $O(N)$ ：其中 $N = n$ ，有个 for 循环，需遍历列表。
- 空间复杂度 $O(N)$ ：创造了长度为 N 的结果数组，需要使用 $O(N)$ 的额外空间。

LeetCode 322、零钱兑换

一、题目描述

给你一个整数数组 `coins`，表示不同面额的硬币；以及一个整数 `amount`，表示总金额。

计算并返回可以凑成总金额所需的 **最少的**硬币个数****。如果没有任何一种硬币组合能组成总金额，返回 `-1`。

你可以认为每种硬币的数量是无限的。

示例 1：

输入: coins = [1, 2, 5], amount = 11
输出: 3
解释: 11 = 5 + 5 + 1

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public int coinChange(int[] coins, int amount) {

        // 初始化数组 dp，长度为 amount + 1，因为在 dp 数组中还会存储金额为 0 的情况
        // dp[i] 表示想要凑齐 i 元需要的最少硬币个数
        // dp[0] 表示想要凑齐 0 元需要的最少硬币个数
        // dp[1] 表示想要凑齐 1 元需要的最少硬币个数
        // dp[14] 表示想要凑齐 14 元需要的最少硬币个数
        int[] dp = new int[amount + 1];

        // 首先将数组 dp 里面的值都初始化为 -1
        // -1 表示当前的金额还没有找到需要的最少硬币个数
        Arrays.fill(dp, -1);

        // 想要凑齐 0 元的最少硬币个数是 0 个
        dp[0] = 0;

        // 依次计算想要凑齐 1 元到 amount 的最少硬币个数是多少
        for(int i = 1 ; i <= amount ; i++){

            // 对于每个金额 i 来说，coins 中的每个面值小于 i 的硬币都可以尝试去拼凑 i
            // 比如 i = 8 , coins 为 [1,2,5,7,10]
            // 其中 1, 2, 5, 7 都小于 8
            // 1 可以尝试去拼凑 8
            // 2 可以尝试去拼凑 8
            // 5 可以尝试去拼凑 8
            // 7 可以尝试去拼凑 8
            // 所以，设置一个变量 j ，遍历数组 coins
            for(int j = 0 ; j < coins.length;j++){

                // 1、如果当前的硬币面值 coins[j] 小于了 i，表示这枚硬币有可能可以拼凑到 i
                // 2、那么 i - coins[j] 表示面值 coins[j] 的硬币想要拼凑 i 需要那些面值的硬币金
                // 额
                // 3、而 dp[i-coins[j]] 表示想要凑齐 i - coins[j] 元需要的最少硬币个数
                // 4、如果 dp[i-coins[j]] != -1 ，表示想要凑齐 i - coins[j] 元需要的最少硬币
                // 个数有结果
                if(coins[j] <= i && dp[i-coins[j]] != -1){
                    // 这个时候，对于金额 i 来说
                    // 1、如果它之前还没有找到凑齐 i 元需要的最少硬币个数
```

```

        // 2、如果此时计算的最少硬币个数比之前保存的结果 dp[i] 更小
        // 那么更新 dp[i]
        if(dp[i] == -1 || dp[i-coins[j]] + 1 < dp[i]){
            // 更新 dp[i]
            // dp[i] 表示想要凑齐 i 元需要的最少硬币个数
            // 这个时候 dp[i] 为获取面值为 j 的那 1 个硬币
            // 加上获取面值为 i - coins[j] 最少需要 dp[i - coins[j]] 个硬币
            dp[i] = dp[i - coins[j]] + 1;
        }

    }

}

// dp[amount] 表示想要凑齐 amount 元需要的最少硬币个数
// 返回这个结果就行
return dp[amount];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        // 初始化数组 dp，长度为 amount + 1，因为在 dp 数组中还会存储金额为 0 的情况
        // dp[i] 表示想要凑齐 i 元需要的最少硬币个数
        // dp[0] 表示想要凑齐 0 元需要的最少硬币个数
        // dp[1] 表示想要凑齐 1 元需要的最少硬币个数
        // dp[14] 表示想要凑齐 14 元需要的最少硬币个数
        // 首先将数组 dp 里面的值都初始化为 -1
        // -1 表示当前的金额还没有找到需要的最少硬币个数
        vector<int> dp( amount + 1 , -1 );

        // 想要凑齐 0 元的最少硬币个数是 0 个
        dp[0] = 0;

        // 依次计算想要凑齐 1 元到 amount 的最少硬币个数是多少
        for(int i = 1 ; i <= amount ; i++){

            // 对于每个金额 i 来说，coins 中的每个面值小于 i 的硬币都可以尝试去拼凑 i
            // 比如 i = 8 , coins 为 [1,2,5,7,10]
            // 其中 1, 2, 5, 7 都小于 8
            // 1 可以尝试去拼凑 8
            // 2 可以尝试去拼凑 8
            // 5 可以尝试去拼凑 8
            // 7 可以尝试去拼凑 8
            // 所以，设置一个变量 j，遍历数组 coins
            for(int j = 0 ; j < coins.size();j++){

```


额

个数有结果

```
// 1、如果当前的硬币面值 coins[j] 小于了 i, 表示这枚硬币有可能可以拼凑到 i
// 2、那么 i - coins[j] 表示面值 coins[j] 的硬币想要拼凑 i 需要那些面值的硬币金

// 3、而 dp[i-coins[j]] 表示想要凑齐 i - coins[j] 元需要的最少硬币个数
// 4、如果 dp[i-coins[j]] != -1 , 表示想要凑齐 i - coins[j] 元需要的最少硬币

if(coins[j] <= i && dp[i-coins[j]] != -1){
    // 这个时候, 对于金额 i 来说
    // 1、如果它之前还没有找到凑齐 i 元需要的最少硬币个数
    // 2、如果此时计算的最少硬币个数比之前保存的结果 dp[i] 更小
    // 那么更新 dp[i]
    if(dp[i] == -1 || dp[i-coins[j]] + 1 < dp[i]){
        // 更新 dp[i]
        // dp[i] 表示想要凑齐 i 元需要的最少硬币个数
        // 这个时候 dp[i] 为获取面值为 j 的那 1 个硬币
        // 加上获取面值为 i - coins[j] 最少需要 dp[i - coins[j]] 个硬币
        dp[i] = dp[i - coins[j]] + 1;
    }
}

}

// dp[amount] 表示想要凑齐 amount 元需要的最少硬币个数
// 返回这个结果就行
return dp[amount];
}

};
```

3、Python 代码

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # 初始化数组 dp, 长度为 amount + 1, 因为在 dp 数组中还会存储金额为 0 的情况
        # dp[i] 表示想要凑齐 i 元需要的最少硬币个数
        # dp[0] 表示想要凑齐 0 元需要的最少硬币个数
        # dp[1] 表示想要凑齐 1 元需要的最少硬币个数
        # dp[14] 表示想要凑齐 14 元需要的最少硬币个数
        # 首先将数组 dp 里面的值都初始化为 -1
        # -1 表示当前的金额还没有找到需要的最少硬币个数
        dp = [ -1 for _ in range( amount + 1 )]

        # 想要凑齐 0 元的最少硬币个数是 0 个
        dp[0] = 0

        # 依次计算想要凑齐 1 元到 amount 的最少硬币个数是多少
        for i in range ( 1 , amount + 1 ) :
```

额

数有结果

```
# 对于每个金额 i 来说, coins 中的每个面值小于 i 的硬币都可以尝试去拼凑 i
# 比如 i = 8 , coins 为 [1,2,5,7,10]
# 其中 1, 2, 5, 7 都小于 8
# 1 可以尝试去拼凑 8
# 2 可以尝试去拼凑 8
# 5 可以尝试去拼凑 8
# 7 可以尝试去拼凑 8
# 所以, 设置一个变量 j , 遍历数组 coins
for j in range( 0 , len(coins) ):

    # 1、如果当前的硬币面值 coins[j] 小于了 i, 表示这枚硬币有可能可以拼凑到 i
    # 2、那么 i - coins[j] 表示面值 coins[j] 的硬币想要拼凑 i 需要那些面值的硬币金

    # 3、而 dp[i-coins[j]] 表示想要凑齐 i - coins[j] 元需要的最少硬币个数
    # 4、如果 dp[i-coins[j]] != -1 , 表示想要凑齐 i - coins[j] 元需要的最少硬币个

    if coins[j] <= i and dp[i-coins[j]] != -1 :
        # 这个时候, 对于金额 i 来说
        # 1、如果它之前还没有找到凑齐 i 元需要的最少硬币个数
        # 2、如果此时计算的最少硬币个数比之前保存的结果 dp[i] 更小
        # 那么更新 dp[i]
        if dp[i] == -1 or dp[i-coins[j]] + 1 < dp[i] :
            # 更新 dp[i]
            # dp[i] 表示想要凑齐 i 元需要的最少硬币个数
            # 这个时候 dp[i] 为获取面值为 j 的那 1 个硬币
            # 加上获取面值为 i - coins[j] 最少需要 dp[i - coins[j]] 个硬币
            dp[i] = dp[i - coins[j]] + 1

# dp[amount] 表示想要凑齐 amount 元需要的最少硬币个数
# 返回这个结果就行
return dp[amount]
```

LeetCode 337、打家劫舍III

一、题目描述

在上次打劫完一条街道之后和一圈房屋后，小偷又发现了一个新的可行窃的地区。

这个地区只有一个入口，我们称之为根。

除了“根”之外，每栋房子有且只有一个“父”房子与之相连。

一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。如果两个直接相连的房子在同一天晚上被打劫，房屋将自动报警。计算在不触动警报的情况下，小偷一晚能够盗取的最高金额。

示例 1:

输入: [3,2,3,null,3,null,1]

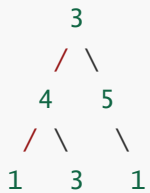


输出: 7

解释: 小偷一晚能够盗取的最高金额 = 3 + 3 + 1 = 7.

示例 2:

输入: [3,4,5,1,3,null,1]



输出: 9

解释: 小偷一晚能够盗取的最高金额 = 4 + 5 = 9.

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 打家劫舍III ( LeetCode 337 ) : https://leetcode-cn.com/problems/house-robber-iii/submissions/
class Solution {

    public int rob(TreeNode root) {

        // 每个节点都有两种选择: 偷、不偷
        // 从叶子节点开始判断, 直到来到根节点
        int[] res = chooseNode(root);

        // 选择当前节点偷和不偷抉择时的较大值
        return Math.max(res[0], res[1]);
    }

    private int[] chooseNode(TreeNode node) {
        // 递归终止条件
        // 即在叶子节点的时候, 叶子节点的左右子节点为 null
```

```

// 继续调用下去就会返回给叶子节点，它的左右子树的选择的金额都是 0
if (node == null) {
    return new int[]{0, 0};
}

// 否则的话，说明当前节点有值
// 需要判断，当前节点是偷还是不偷

// 1、如果选择偷，那么左子节点不能偷，这个时候需要获取不偷左子节点时的最大金额
// 2、如果选择偷，那么右子节点不能偷，这个时候需要获取不偷右子节点时的最大金额
// 3、如果选择不偷，那么可以偷左右子节点，这个时候的最大金额为左右子节点最大金额之和

// dp[0] 表示的是以当前 node 为根节点的子树能够偷取的最大金额，并且此时采取【不偷】 node 这个节点的策略
// dp[1] 表示的是以当前 node 为根节点的子树能够偷取的最大金额，并且此时采取【偷】 node 这个节点的策略
int[] dp = new int[2];

// 先去计算左子节点的选择
int[] left = chooseNode(node.left);

// 再去计算右子节点的选择
int[] right = chooseNode(node.right);

// 如果选择不偷，那么可以偷左右子节点，这个时候的最大金额为左右子节点最大金额之和
dp[0] = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);

// 1、如果选择偷，那么左子节点不能偷，这个时候需要获取不偷左子节点时的最大金额
// 2、如果选择偷，那么右子节点不能偷，这个时候需要获取不偷右子节点时的最大金额
// 3、再加上当前节点的金额
dp[1] = left[0] + right[0] + node.val;

return dp;
}
}

```

LeetCode 72、编辑距离

一、题目描述

给你两个单词 word1 和 word2，请你计算出将 word1 转换成 word2 所使用的最少操作数。

你可以对一个单词进行如下三种操作：

- 1、插入一个字符
- 2、删除一个字符
- 3、替换一个字符

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {
    public int minDistance(String word1, String word2) {

        // 获取字符串 word1 的长度
        int L1 = word1.length();

        // 获取字符串 word2 的长度
        int L2 = word2.length();

        // 设置二维数组 dp 用来储存 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[0][0] 表示 word1 的前 0 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
        int[][] dp = new int[L1 + 1][L2 + 1];

        // dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[i][0] 相当于给二维数组 dp 的第一列进行赋值操作
        // 只需要每次执行删除操作，就可以把 word1 的字符删除到 0 个字符
        for(int i = 0 ; i <= L1 ; i++){
            // dp[1][0] 表示需要删除 1 次才能删除到 0 个字符
            // dp[2][0] 表示需要删除 2 次才能删除到 2 个字符
            // dp[i][0] 表示需要删除 i 次才能删除到 2 个字符
            dp[i][0] = i;
        }

        // dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[0][j] 相当于给二维数组 dp 的第一行进行赋值操作
        // 只需要每次执行插入操作，就可以把 word1 的字符转行为 j 个字符
        for(int j = 0 ; j <= L2 ; j++){
            // dp[0][1] 表示需要插入 1 次才能得到 1 个字符
            // dp[0][2] 表示需要插入 2 次才能得到 2 个字符
            // dp[0][j] 表示需要插入 j 次才能得到 j 个字符
            dp[0][j] = j;
        }

        // 通过两个 for 循环，设置二维数组 dp 中的所有元素的值
        // i 从 word1 的第 1 个字符一直到 L1 个字符
        for(int i = 1 ; i <= L1 ; i++){

            // j 从 word2 的第 1 个字符一直到 L2 个字符
            for(int j = 1 ; j <= L2 ; j++){
```

```

// 二维数组 dp 的下标是从 (0,0) 开始的
// 所以 dp[i][j] 表示的是 word1 中前 i 个字符即位置为 i - 1 的字符
// 转换为 word2 中前 j 个字符即位置为 j - 1 的字符所需要的最少操作数
// 如果此时 word1 中第 i - 1 个位置的字符 (即当前遍历 i 时的字符)
// 与 word2 中第 j - 1 个位置的字符【相等】 (即当前遍历 j 时的字符)
// 说明 word1 的前 i - 1 个字符转换为 word2 中前 j - 1 个字符成功后,
// word1 的前 i 个字符也就成功转换为 word2 的前 j 个字符了
if(word1.charAt(i - 1) == word2.charAt(j - 1)){

    // dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
    // dp[i - 1][j - 1] 表示 word1 的前 i - 1 个字符转换为 word2 的前 j - 1
个字符最少操作数

    // 比如 word1 为 abcd, word2 为 efcd
    // 此时 i = 4, j = 4
    // 由于 word1.charAt(4 - 1) = d
    // word2.charAt(4 - 1) = d
    // 所以如果知道了 abc 转换为 efg 的最少操作数, 那么结果也就出来了
    // d --> d 不需要执行任何操作
    dp[i][j] = dp[i - 1][j - 1];

    // 否则, 说明 word1 中第 i - 1 个位置的字符 (即当前遍历 i 时的字符)
    // 与 word2 中第 j - 1 个位置的字符【不相等】 (即当前遍历 j 时的字符)
}else{

    // 那么 dp[i][j] 可以从以下三种状态转换过来
    // 1、word1(L1-1) --> word2(L2-1)
    // 2、word1(L1-1) --> word2(L2)
    // 3、word1(L1) --> word2(L2-1)
    // word1(L1-1) --> word2(L2-1) 的最少操作数是 dp[i - 1][j - 1]
    // word1(L1-1) --> word2(L2) 的最少操作数是 dp[i - 1][j]
    // word1(L1) --> word2(L2-1) 的最少操作数是 dp[i][j - 1]
    // 选这三种状态的较小值后,
    // 1、如果是 word1(L1-1) --> word2(L2-1) 过来的, 再执行 1 次替换操作
    // 2、如果是 word1(L1-1) --> word2(L2) 过来的, 再执行 1 次删除操作
    // 3、如果是 word1(L1) --> word2(L2-1) 过来的, 再执行 1 次插入操作
    dp[i][j] = Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j
- 1])) + 1;

}

}

}

// dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
// 把这个结果进行返回
return dp[L1][L2];

}

}

```

2、**C++** 代码

```
class solution {
public:
    int minDistance(string word1, string word2) {

        // 获取字符串 word1 的长度
        int L1 = word1.length();

        // 获取字符串 word2 的长度
        int L2 = word2.length();

        // 设置二维数组 dp 用来储存 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[0][0] 表示 word1 的前 0 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
        auto dp = vector< vector<int> > ( L1 + 1,vector<int> ( L2 + 1 ));

        // dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        // dp[i][0] 相当于给二维数组 dp 的第一列进行赋值操作
        // 只需要每次执行删除操作，就可以把 word1 的字符删除到 0 个字符
        for(int i = 0 ; i <= L1 ; i++){
            // dp[1][0] 表示需要删除 1 次才能删除到 0 个字符
            // dp[2][0] 表示需要删除 2 次才能删除到 2 个字符
            // dp[i][0] 表示需要删除 i 次才能删除到 2 个字符
            dp[i][0] = i;
        }

        // dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        // dp[0][j] 相当于给二维数组 dp 的第一行进行赋值操作
        // 只需要每次执行插入操作，就可以把 word1 的字符转行为 j 个字符
        for(int j = 0 ; j <= L2 ; j++){
            // dp[0][1] 表示需要插入 1 次才能得到 1 个字符
            // dp[0][2] 表示需要插入 2 次才能得到 2 个字符
            // dp[0][j] 表示需要插入 j 次才能得到 j 个字符
            dp[0][j] = j;
        }

        // 通过两个 for 循环，设置二维数组 dp 中的所有元素的值
        // i 从 word1 的第 1 个字符一直到 L1 个字符
        for(int i = 1 ; i <= L1 ; i++){

            // j 从 word2 的第 1 个字符一直到 L2 个字符
            for(int j = 1 ; j <= L2 ; j++){

                // 二维数组 dp 的下标是从 (0,0) 开始的
```

个字符最少操作数

```
// 所以 dp[i][j] 表示的是 word1 中前 i 个字符即位置为 i - 1 的字符
// 转换为 word2 中前 j 个字符即位置为 j - 1 的字符所需要的最少操作数
// 如果此时 word1 中第 i - 1 个位置的字符（即当前遍历 i 时的字符）
// 与 word2 中第 j - 1 个位置的字符【相等】（即当前遍历 j 时的字符）
// 说明 word1 的前 i - 1 个字符转换为 word2 中前 j - 1 个字符成功后，
// word1 的前 i 个字符也就成功转换为 word2 的前 j 个字符了
if(word1[i - 1] == word2[j - 1]){
    // dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
    // dp[i - 1][j - 1] 表示 word1 的前 i - 1 个字符转换为 word2 的前 j - 1
    // 个字符最少操作数

    // 比如 word1 为 abcd, word2 为 efcd
    // 此时 i = 4, j = 4
    // 由于 word1.charAt(4 - 1) = d
    // word2.charAt(4 - 1) = d
    // 所以如果知道了 abc 转换为 efc 的最少操作数，那么结果也就出来了
    // d --> d 不需要执行任何操作
    dp[i][j] = dp[i - 1][j - 1];

    // 否则，说明 word1 中第 i - 1 个位置的字符（即当前遍历 i 时的字符）
    // 与 word2 中第 j - 1 个位置的字符【不相等】（即当前遍历 j 时的字符）
}else{

    // 那么 dp[i][j] 可以从以下三种状态转换过来
    // 1、word1(L1-1) --> word2(L2-1)
    // 2、word1(L1-1) --> word2(L2)
    // 3、word1(L1) --> word2(L2-1)
    // word1(L1-1) --> word2(L2-1) 的最少操作数是 dp[i - 1][j - 1]
    // word1(L1-1) --> word2(L2) 的最少操作数是 dp[i - 1][j]
    // word1(L1) --> word2(L2-1) 的最少操作数是 dp[i][j - 1]
    // 选这三种状态的较小值后，
    // 1、如果是 word1(L1-1) --> word2(L2-1) 过来的，再执行 1 次替换操作
    // 2、如果是 word1(L1-1) --> word2(L2) 过来的，再执行 1 次删除操作
    // 3、如果是 word1(L1) --> word2(L2-1) 过来的，再执行 1 次插入操作
    dp[i][j] = min(dp[i - 1][j - 1], min(dp[i - 1][j], dp[i][j - 1])) +
1;
    }
}
}
// dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
// 把这个结果进行返回
return dp[L1][L2];
}
};
```


3、Python 代码

```
class solution:
    def minDistance(self, word1: str, word2: str) -> int:
        # 获取字符串 word1 的长度
        L1 = len(word1)

        # 获取字符串 word2 的长度
        L2 = len(word2)

        # 设置二维数组 dp 用来储存 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        # dp[0][0] 表示 word1 的前 0 个字符转换为 word2 的前 0 个字符最少操作数
        # dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        # dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        # dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
        # dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
        dp = [[0] * (L2 + 1) for _ in range( L1 + 1)]

        # dp[i][0] 表示 word1 的前 i 个字符转换为 word2 的前 0 个字符最少操作数
        # dp[i][0] 相当于给二维数组 dp 的第一列进行赋值操作
        # 只需要每次执行删除操作，就可以把 word1 的字符删除到 0 个字符
        for i in range(L1 + 1):
            # dp[1][0] 表示需要删除 1 次才能删除到 0 个字符
            # dp[2][0] 表示需要删除 2 次才能删除到 2 个字符
            # dp[i][0] 表示需要删除 i 次才能删除到 2 个字符
            dp[i][0] = i

        # dp[0][j] 表示 word1 的前 0 个字符转换为 word2 的前 j 个字符最少操作数
        # dp[0][j] 相当于给二维数组 dp 的第一行进行赋值操作
        # 只需要每次执行插入操作，就可以把 word1 的字符转行为 j 个字符
        for j in range( L2 + 1):
            # dp[0][1] 表示需要插入 1 次才能得到 1 个字符
            # dp[0][2] 表示需要插入 2 次才能得到 2 个字符
            # dp[0][j] 表示需要插入 j 次才能得到 j 个字符
            dp[0][j] = j

        # 通过两个 for 循环，设置二维数组 dp 中的所有元素的值
        # i 从 word1 的第 1 个字符一直到 L1 个字符
        for i in range( 1 , L1 + 1):

            # j 从 word2 的第 1 个字符一直到 L2 个字符
            for j in range( 1 , L2 + 1):

                # 二维数组 dp 的下标是从 (0,0) 开始的
                # 所以 dp[i][j] 表示的是 word1 中前 i 个字符即位置为 i - 1 的字符
                # 转换为 word2 中前 j 个字符即位置为 j - 1 的字符所需要的最少操作数
```

个字符最少操作数

```
# 如果此时 word1 中第 i - 1 个位置的字符（即当前遍历 i 时的字符）
# 与 word2 中第 j - 1 个位置的字符【相等】（即当前遍历 j 时的字符）
# 说明 word1 的前 i - 1 个字符转换为 word2 中前 j - 1 个字符成功后，
# word1 的前 i 个字符也就成功转换为 word2 的前 j 个字符了
if word1[i - 1] == word2[j - 1] :

    # dp[i][j] 表示 word1 的前 i 个字符转换为 word2 的前 j 个字符最少操作数
    # dp[i - 1][j - 1] 表示 word1 的前 i - 1 个字符转换为 word2 的前 j - 1
    # 个字符最少操作数

    # 比如 word1 为 abcd, word2 为 efcd
    # 此时 i = 4, j = 4
    # 由于 word1.charAt(4 - 1) = d
    # word2.charAt(4 - 1) = d
    # 所以如果知道了 abc 转换为 efc 的最少操作数，那么结果也就出来了
    # d --> d 不需要执行任何操作
    dp[i][j] = dp[i - 1][j - 1]

    # 否则，说明 word1 中第 i - 1 个位置的字符（即当前遍历 i 时的字符）
    # 与 word2 中第 j - 1 个位置的字符【不相等】（即当前遍历 j 时的字符）
else:

    # 那么 dp[i][j] 可以从以下三种状态转换过来
    # 1、word1(L1-1) --> word2(L2-1)
    # 2、word1(L1-1) --> word2(L2)
    # 3、word1(L1) --> word2(L2-1)
    # word1(L1-1) --> word2(L2-1) 的最少操作数是 dp[i - 1][j - 1]
    # word1(L1-1) --> word2(L2) 的最少操作数是 dp[i - 1][j]
    # word1(L1) --> word2(L2-1) 的最少操作数是 dp[i][j - 1]
    # 选这三种状态的较小值后，
    # 1、如果是 word1(L1-1) --> word2(L2-1) 过来的，再执行 1 次替换操作
    # 2、如果是 word1(L1-1) --> word2(L2) 过来的，再执行 1 次删除操作
    # 3、如果是 word1(L1) --> word2(L2-1) 过来的，再执行 1 次插入操作
    dp[i][j] = min(dp[i - 1][j - 1], min(dp[i - 1][j], dp[i][j - 1])) + 1

# dp[L1][L2] 表示 word1 的前 L1 个字符转换为 word2 的前 L2 个字符最少操作数
# 把这个结果进行返回
return dp[L1][L2]
```

第四十九天

LeetCode 1049、最后一块石头的重量II

一、题目描述

有一堆石头，用整数数组 `stones` 表示。其中 `stones[i]` 表示第 `i` 块石头的重量。

每一回合，从中选出任意两块石头，然后将它们一起粉碎。假设石头的重量分别为 `x` 和 `y`，且 `x <= y`。那么粉碎的可能结果如下：

- 如果 `x == y`，那么两块石头都会被完全粉碎；
- 如果 `x != y`，那么重量为 `x` 的石头将会完全粉碎，而重量为 `y` 的石头新重量为 `y-x`。

最后，最多只会剩下一块石头。返回此石头最小的可能重量。如果没有石头剩下，就返回 `0`。

示例 1：

输入：`stones = [2,7,4,1,8,1]` 输出：`1` 解释：组合 2 和 4，得到 2，所以数组转化为 `[2,7,1,8,1]`，组合 7 和 8，得到 1，所以数组转化为 `[2,1,1,1]`，组合 2 和 1，得到 1，所以数组转化为 `[1,1,1]`，组合 1 和 1，得到 0，所以数组转化为 `[1]`，这就是最优值。

示例 2：

输入：`stones = [31,26,33,21,40]` 输出：`5`

提示：

- `1 <= stones.length <= 30`
- `1 <= stones[i] <= 100`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最后一块石头的重量II (LeetCode 1049) : https://leetcode.cn/problems/last-stone-weight-ii/submissions/
class Solution {
    public int lastStoneweightII(int[] stones) {

        // 在两块石头碰撞过程中，可以每次把大的石头划分到 A 部分，小的石头划分到 B 部分
        // 因此可以将数组分成 A 和 B 两个部分
        // A 部分的石头可以被部分留下
        // B 部分的石头全部被粉碎

        // 接下来就是需要思考到底哪些石头放入 A 部分，哪些石头放入 B 部分
        // 其中 A 部分的石头重量和是 sumA
        // 其中 B 部分的石头重量和是 sumB
        // 为了使得最后剩下的石头尽可能的小，那么总是会想去构造 { a , b } 这两个石头的差值尽可能的小
        // 于是问题就变成了：从 stones 中选择一些元素，总和不超过 sum / 2 的最大值
```

```

// 先去计算总和
int sum = 0;

for (int num : stones) {

    sum += num;

}

int target = sum / 2;

// dp[i][j] 代表考虑前 i 个物品（数值），凑成总和不超过 j 的最大价值
int dp[][] = new int[stones.length + 1][target + 1];

// 01 背包问题开始填充二维数组
for( int i = 1 ; i <= stones.length ; i++){

    for( int j = 0 ; j <= target ; j++){

        // 注意到 i 是从下标 1 开始访问的
        // 1、背包容量小于当前元素
        // 背包无法放入 stones[i - 1]
        if ( j < stones[i - 1]){

            dp[i][j] = dp[ i - 1 ][j];

            // 2、背包容量大于等于当前元素
            // 背包可以放入 stones[i - 1]
        }else{

            // 不选：方案数为 dp[i - 1][j]
            // 选：方案数为 dp[i - 1][j - stones[i - 1]] + stones[i-1]
            dp[i][j] = Math.max(dp[i-1][j] , dp[i-1][j - stones[i-1]] +
stones[i-1]);

        }

    }

}

// 这两个部分最大值是 dp[stones.length][target]，剩余的部分就是剩下的石头
return Math.abs(sum - dp[stones.length][target] - dp[stones.length][target]);

}

}

```

滚动数组代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最后一块石头的重量II (LeetCode 1049) : https://leetcode.cn/problems/last-stone-weight-ii/submissions/
class Solution {
    public int lastStoneweightII(int[] stones) {

        // 在两块石头碰撞过程中，可以每次把大的石头划分到 A 部分，小的石头划分到 B 部分
        // 因此可以将数组分成 A 和 B 两个部分
        // A 部分的石头可以被部分留下
        // B 部分的石头全部被粉碎

        // 接下来就是需要思考到底哪些石头放入 A 部分，哪些石头放入 B 部分
        // 其中 A 部分的石头重量和是 sumA
        // 其中 B 部分的石头重量和是 sumB
        // 为了使得最后剩下的石头尽可能的小，那么总是会想去构造 { a , b } 这两个石头的差值尽可能的小
        // 于是问题就变成了：从 stones 中选择一些元素，总和不超过 sum / 2 的最大值

        // 先去计算总和
        int sum = 0;

        for (int num : stones) {

            sum += num;

        }

        int target = sum / 2;

        // dp[i][j] 代表考虑前 i 个物品（数值），凑成总和不超过 j 的最大价值
        int dp[] = new int[target + 1];

        // 01 背包问题开始填充二维数组
        for( int i = 1 ; i <= stones.length ; i++){

            for( int j = target ; j >= 0 ; j--){

                // 注意到 i 是从下标 1 开始访问的
                if ( j >= stones[i - 1]){

                    dp[j] = Math.max(dp[j] , dp[j - stones[i-1]] + stones[i-1]);

                }

            }

        }

    }
}
```

```

    }

    // 这两个部分最大值是 dp[stones.length][target]，剩余的部分就是剩下的石头
    return Math.abs(sum - dp[target] - dp[target]);
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int lastStoneWeightII(vector<int>& stones) {

        // 在两块石头碰撞过程中，可以每次把大的石头划分到 A 部分，小的石头划分到 B 部分
        // 因此可以将数组分成 A 和 B 两个部分
        // A 部分的石头可以被部分留下
        // B 部分的石头全部被粉碎

        // 接下来就是需要思考到底哪些石头放入 A 部分，哪些石头放入 B 部分
        // 其中 A 部分的石头重量和是 sumA
        // 其中 B 部分的石头重量和是 sumB
        // 为了使得最后剩下的石头尽可能的小，那么总是会想去构造 { a , b } 这两个石头的差值尽可能的小
        // 于是问题就变成了：从 stones 中选择一些元素，总和不超过 sum / 2 的最大值

        // 先去计算总和
        int sum = accumulate(stones.begin(), stones.end(), 0);

        int target = sum / 2;

        // dp[i][j] 代表考虑前 i 个物品（数值），凑成总和不超过 j 的最大价值
        vector<vector<int>> dp(stones.size() + 1, vector<int>(target + 1));

        // 01 背包问题开始填充二维数组
        for( int i = 1 ; i <= stones.size() ; i++){

            for( int j = 0 ; j <= target ; j++){

                // 注意到 i 是从下标 1 开始访问的
                // 1、背包容量小于当前元素
                // 背包无法放入 stones[i - 1]
                if ( j < stones[i - 1]){

                    dp[i][j] = dp[ i - 1 ][j];

                }

                // 2、背包容量大于等于当前元素
                // 背包可以放入 stones[i - 1]
            }else{

```

```

        // 不选: 方案数为 dp[i - 1][j]
        // 选: 方案数为 dp[i - 1][j - stones[i - 1]] + stones[i-1]
        dp[i][j] = max(dp[i-1][j] , dp[i-1][j - stones[i-1]] + stones[i-1]);
    }
}

// 这两个部分最大值是 dp[stones.length][target], 剩余的部分就是剩下的石头
return abs(sum - dp[stones.size()][target] - dp[stones.size()][target]);
}
};

```

3、Python 代码

```

class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        # 在两块石头碰撞过程中, 可以每次把大的石头划分到 A 部分, 小的石头划分到 B 部分
        # 因此可以将数组分成 A 和 B 两个部分
        # A 部分的石头可以被部分留下
        # B 部分的石头全部被粉碎

        # 接下来就是需要思考到底哪些石头放入 A 部分, 哪些石头放入 B 部分
        # 其中 A 部分的石头重量和是 sumA
        # 其中 B 部分的石头重量和是 sumB
        # 为了使得最后剩下的石头尽可能的小, 那么总是会想去构造 { a , b } 这两个石头的差值尽可能的小
        # 于是问题就变成了: 从 stones 中选择一些元素, 总和不超过 sum / 2 的最大值

        # 先去计算总和
        total = sum(stones)

        target = total // 2

        # dp[i][j] 代表考虑前 i 个物品 (数值), 凑成总和不超过 j 的最大价值
        dp = [[0] * (target + 1) for _ in range(len(stones) + 1)]

        # 01 背包问题开始填充二维数组
        for i in range(1, len(stones) + 1):
            for j in range(target + 1):

                # 注意到 i 是从下标 1 开始访问的
                # 1、背包容量小于当前元素
                # 背包无法放入 stones[i - 1]
                if j < stones[i - 1]:

                    dp[i][j] = dp[i - 1][j]

```

```

        # 2、背包容量大于等于当前元素
        # 背包可以放入 stones[i - 1]
    else:

        # 不选：方案数为 dp[i - 1][j]
        # 选：方案数为 dp[i - 1][j - stones[i - 1]] + stones[i-1]
        dp[i][j] = max(dp[i-1][j] , dp[i-1][j - stones[i-1]] + stones[i-1])

    # 这两个部分最大值是 dp[stones.length][target]，剩余的部分就是剩下的石头
    return total - 2*dp[len(stones)][target]
#### 滚动数组
class Solution:
    def lastStoneWeightII(self, stones: List[int]) -> int:
        # 在两块石头碰撞过程中，可以每次把大的石头划分到 A 部分，小的石头划分到 B 部分
        # 因此可以将数组分成 A 和 B 两个部分
        # A 部分的石头可以被部分留下
        # B 部分的石头全部被粉碎

        # 接下来就是需要思考到底哪些石头放入 A 部分，哪些石头放入 B 部分
        # 其中 A 部分的石头重量和是 sumA
        # 其中 B 部分的石头重量和是 sumB
        # 为了使得最后剩下的石头尽可能的小，那么总是会想去构造 { a , b } 这两个石头的差值尽可能的小
        # 于是问题就变成了：从 stones 中选择一些元素，总和不超过 sum / 2 的最大值

        # 先计算总和
        sum = 0
        for num in stones:
            sum += num

        target = sum // 2

        # dp[i] 代表考虑前 i 个物品（数值），凑成总和不超过 target 的最大价值
        dp = [0] * (target + 1)

        # 01 背包问题开始填充一维数组
        for i in range(1, len(stones) + 1):
            for j in range(target, -1, -1):
                # 注意到 i 是从下标 1 开始访问的
                if j >= stones[i - 1]:
                    dp[j] = max(dp[j], dp[j - stones[i - 1]] + stones[i - 1])

        # 这两个部分最大值是 dp[len(stones)][target]，剩余的部分就是剩下的石头
        return abs(sum - dp[target] - dp[target])

```


LeetCode 322、零钱兑换（完全背包解法）

一、题目描述

给你一个整数数组 `coins`，表示不同面额的硬币；以及一个整数 `amount`，表示总金额。

计算并返回可以凑成总金额所需的 **最少的**硬币个数****。如果没有任何一种硬币组合能组成总金额，返回 `-1`。

你可以认为每种硬币的数量是无限的。

示例 1：

输入：coins = [1, 2, 5], amount = 11 输出：3 解释：11 = 5 + 5 + 1

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 零钱兑换（LeetCode 322）完全背包解法：https://leetcode.cn/problems/coin-change/
class Solution {
    public int coinChange(int[] coins, int amount) {

        int n = coins.length;

        // 二维数组：状态定义:dp[i][j]表示从前 i 个物品中选择不超过 j 重量的物品的最大价值
        int[][] dp = new int[n + 1][amount + 1];

        for( int i = 0 ; i <= n ; i++){
            for( int j = 0 ; j <= amount ;j++){
                dp[i][j] = amount + 1;
            }
        }

        dp[0][0] = 0;

        // Tips: 前 i 个物品表示的是编号从 [ 0 , i - 1 ] 这些物品，即最后一个物品编号为 i - 1
        for( int i = 1 ; i <= n ; i++){

            for( int j = 0 ; j <= amount; j++){

                // 背包容量小于当前物品的容量，即背包容量已经不足以拿第 i 个物品了
                // 第 i 个物品即下标为 i - 1 的那个物品
                if( j < coins[i-1]){
```

```

        dp[i][j] = dp[i-1][j];

        // 背包容量足够拿第 i 个物品，那么可拿也可不拿
        // 1、拿了，那么最大价值是前 i 个物品扣除第 i 个物品的重量的情况下的最大价值，再加上【第 i 个】物品的价值
        // 2、不拿，那么就是从前 i - 1 个物品中选择出的最大价值
    }else{

        dp[i][j] = Math.min(dp[i-1][j],dp[i][j - coins[i-1]] + 1);
    }
}

return dp[n][amount] == amount + 1 ? -1 : dp[n][amount];

}
}

```

2、**C++** 代码

```

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        int n = coins.size();

        // 二维数组：状态定义:dp[i][j]表示从前 i 个物品中选择不超过 j 重量的物品的最大价值
        vector<vector<int>> dp(n + 1, vector<int>(amount + 1, amount + 1));

        dp[0][0] = 0;

        // Tips: 前 i 个物品表示的是编号从 [ 0 , i - 1 ] 这些物品，即最后一个物品编号为 i - 1
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j <= amount; j++) {
                // 背包容量小于当前物品的容量，即背包容量已经不足以拿第 i 个物品了
                // 第 i 个物品即下标为 i - 1 的那个物品
                if (j < coins[i - 1]) {
                    dp[i][j] = dp[i - 1][j];
                }
                // 背包容量足够拿第 i 个物品，那么可拿也可不拿
                // 1、拿了，那么最大价值是前 i 个物品扣除第 i 个物品的重量的情况下的最大价值，再加上【第 i 个】物品的价值
                // 2、不拿，那么就是从前 i - 1 个物品中选择出的最大价值
                else {
                    dp[i][j] = min(dp[i - 1][j], dp[i][j - coins[i - 1]] + 1);
                }
            }
        }

        return dp[n][amount] == amount + 1 ? -1 : dp[n][amount];
    }
}

```

```
}  
};
```

3、Python 代码

```
class Solution:  
    def coinChange(self, coins: List[int], amount: int) -> int:  
  
        n = len(coins)  
  
        # 二维数组: 状态定义: dp[i][j] 表示从前 i 个物品中选择不超过 j 重量的物品的最大价值  
        dp = [[amount + 1] * (amount + 1) for _ in range(n + 1)]  
  
        dp[0][0] = 0  
  
        # Tips: 前 i 个物品表示的是编号从 [0, i - 1] 这些物品, 即最后一个物品编号为 i - 1  
        for i in range(1, n + 1):  
            for j in range(amount + 1):  
                # 背包容量小于当前物品的容量, 即背包容量已经不足以拿第 i 个物品了  
                # 第 i 个物品即下标为 i - 1 的那个物品  
                if j < coins[i - 1]:  
                    dp[i][j] = dp[i - 1][j]  
                # 背包容量足够拿第 i 个物品, 那么可拿也可不拿  
                # 1、拿了, 那么最大价值是前 i 个物品扣除第 i 个物品的重量的情况下的最大价值, 再加上  
                # 【第 i 个】物品的价值  
                # 2、不拿, 那么就是从前 i - 1 个物品中选择出的最大价值  
                else:  
                    dp[i][j] = min(dp[i - 1][j], dp[i][j - coins[i - 1]] + 1)  
  
        return -1 if dp[n][amount] == amount + 1 else dp[n][amount]
```

LeetCode 416、分割等和子集

一、题目描述

给你一个只包含正整数的非空数组 nums。请你判断是否可以将这个数组分割成两个子集, 使得两个子集的元素和相等。

二、题目解析

三、参考代码

1、Java 代码

```
class Solution {  
    public boolean canPartition(int[] nums) {  
  
        // 使用 sum 来保存数组中全部元素的和  
        int sum = 0;
```

```

// 遍历 nums 数组
for (int i = 0 ; i < nums.length ; i++){
    // 把 nums 数组上的所有元素值累加到 sum 上
    sum += nums[i];
}

// 如果发现 sum 为奇数，那么必然无法拆分为两个相等的整数
if(sum % 2 == 1){
    // 所以无法将这个数组分割成两个子集，返回false
    return false;
}

// 获取数组的长度
int n = nums.length;

// 获取数组中全部元素之后的一半
// 接下来需要在数组 nums 中寻找一些数字去拼凑为 target
// 如果能找到一些数字之和为 target，那么剩下的数字之和也是 target
int target = sum / 2;

// dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
// dp[0][0] 表示 nums 的前 0 个元素是否可以组成和为 0 的结果
// dp[2][6] 表示 nums 的前 2 个元素是否可以组成和为 6 的结果
// dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
// i 的值从 0 到 n - 1
// j 的值从 0 到 target
boolean[][] dp = new boolean[n][target + 1];

// 先初始化 dp[0][nums[0]]
// 如果 nums 的第 0 个元素 nums[0] 小于 target
if (nums[0] <= target) {
    // 那么 nums 的前 0 个元素是否可以组成和为 nums[0] 的结果是 true
    // 因为 nums 的前 0 个元素就是 nums[0]
    dp[0][nums[0]] = true;
}

// 接下来开始往 dp 数组中填充结果
// i 的值从 1 到 n - 1
for (int i = 1; i < n; i++) {
    // j 的值从 0 到 target
    for (int j = 0; j <= target; j++) {
        // dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
        // dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
        // 对于 dp[i][j] 来说，如果 dp[i - 1][j] 为 true，那么 dp[i][j] 必然也为
true

        // 所以，先初始化 dp[i][j] 的值为 dp[i - 1][j] 的值
        // 再通过后面的代码修改 dp[i][j] 中还为 false 的值
    }
}

```

```

        dp[i][j] = dp[i - 1][j];

        // 如果此时 nums[i] 恰巧为 j
        // 那么对于 dp[i][j] 来说, nums 的前 i 个元素可以组成和为 j 的结果
        if (nums[i] == j) {
            // 所以 dp[i][j] 为 true
            dp[i][j] = true;
            // 同时继续
            continue;
        }

        // 如果发现 nums[i] 小于 j
        if (nums[i] < j) {
            // dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
            // dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
            // dp[i - 1][j - nums[i]] 表示 nums 的前 i - 1 个元素是否可以组成和为 j
- nums[i] 的结果

            // 那么 dp[i][j] 的结果要为 true
            // 1、nums 的前 i - 1 个元素可以组成和为 j
            // 2、nums 的前 i - 1 个元素可以组成和为 j - nums[i]
            // 对于 1 来说, 不用使用 nums[i] 就组成了 j
            // 对于 2 来说, 前 i - 1 个元素可以组成和为 j - nums[i], 那么加上此时的值
nums[i], 也组成了 j

            dp[i][j] = dp[i - 1][j] || dp[i - 1][j - nums[i]];
        }
    }
}

// dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
// 返回这个结果
return dp[n - 1][target];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    bool canPartition(vector<int>& nums) {
        // 使用 sum 来保存数组中全部元素的和
        int sum = 0;

        // 遍历 nums 数组
        for (int i = 0 ; i < nums.size() ; i++){
            // 把 nums 数组上的所有元素值累加到 sum 上
            sum += nums[i];
        }
    }
}

```

```

// 如果发现 sum 为奇数，那么必然无法拆分为两个相等的整数
if(sum % 2 == 1){
    // 所以无法将这个数组分割成两个子集，返回false
    return false;
}

// 获取数组的长度
int n = nums.size();

// 获取数组中全部元素之后的一半
// 接下来需要在数组 nums 中寻找一些数字去拼凑为 target
// 如果能找到一些数字之和为 target，那么剩下的数字之和也是 target
int target = sum / 2;

// dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
// dp[0][0] 表示 nums 的前 0 个元素是否可以组成和为 0 的结果
// dp[2][6] 表示 nums 的前 2 个元素是否可以组成和为 6 的结果
// dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
// i 的值从 0 到 n - 1
// j 的值从 0 到 target
auto dp = vector < vector < bool>> ( n ,vector<bool> (target + 1));

// 先初始化 dp[0][nums[0]]
// 如果 nums 的第 0 个元素 nums[0] 小于 target
if (nums[0] <= target) {
    // 那么 nums 的前 0 个元素是否可以组成和为 nums[0] 的结果是 true
    // 因为 nums 的前 0 个元素就是 nums[0]
    dp[0][nums[0]] = true;
}

// 接下来开始往 dp 数组中填充结果
// i 的值从 1 到 n - 1
for (int i = 1; i < n; i++) {
    // j 的值从 0 到 target
    for (int j = 0; j <= target; j++) {
        // dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
        // dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
        // 对于 dp[i][j] 来说，如果 dp[i - 1][j] 为 true，那么 dp[i][j] 必然也为
true

        // 所以，先初始化 dp[i][j] 的值为 dp[i - 1][j] 的值
        // 再通过后面的代码修改 dp[i][j] 中还为 false 的值
        dp[i][j] = dp[i - 1][j];

        // 如果此时 nums[i] 恰巧为 j
        // 那么对于 dp[i][j] 来说，nums 的前 i 个元素可以组成和为 j 的结果
        if (nums[i] == j) {
            // 所以 dp[i][j] 为 true
            dp[i][j] = true;
        }
    }
}

```

```

        // 同时继续
        continue;
    }

    // 如果发现 nums[i] 小于 j
    if (nums[i] < j) {
        // dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
        // dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
        // dp[i - 1][j - nums[i]] 表示 nums 的前 i - 1 个元素是否可以组成和为 j
- nums[i] 的结果
        // 那么 dp[i][j] 的结果要为 true
        // 1、nums 的前 i - 1 个元素可以组成和为 j
        // 2、nums 的前 i - 1 个元素可以组成和为 j - nums[i]
        // 对于 1 来说, 不用使用 nums[i] 就组成了 j
        // 对于 2 来说, 前 i - 1 个元素可以组成和为 j - nums[i], 那么加上此时的值
nums[i], 也组成了 j
        dp[i][j] = dp[i - 1][j] || dp[i - 1][j - nums[i]];
    }
}
}

// dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
// 返回这个结果
return dp[n - 1][target];
}
};

```

3、Python 代码

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        # 使用 total 来保存数组中全部元素的和
        total = sum(nums)

        # 如果发现 sum 为奇数, 那么必然无法拆分为两个相等的整数
        if total & 1:
            return False

        # 获取数组的长度
        n = len(nums)

        # 获取数组中全部元素之后的一半
        # 接下来需要在数组 nums 中寻找一些数字去拼凑为 target
        # 如果能找到一些数字之和为 target, 那么剩下的数字之和也是 target
        target = total // 2

        # dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
        # dp[0][0] 表示 nums 的前 0 个元素是否可以组成和为 0 的结果
        # dp[2][6] 表示 nums 的前 2 个元素是否可以组成和为 6 的结果

```

```

# dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
# i 的值从 0 到 n - 1
# j 的值从 0 到 target
dp = [[False] * (target + 1) for _ in range(n)]

# 先初始化 dp[0][nums[0]]
# 如果 nums 的第 0 个元素 nums[0] 小于 target
if nums[0] <= target :
    # 那么 nums 的前 0 个元素是否可以组成和为 nums[0] 的结果是 true
    # 因为 nums 的前 0 个元素就是 nums[0]
    dp[0][nums[0]] = True

# 接下来开始往 dp 数组中填充结果
# i 的值从 1 到 n - 1
for i in range(1, n):
    # j 的值从 0 到 target
    for j in range(0, target + 1):
        # dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
        # dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
        # 对于 dp[i][j] 来说, 如果 dp[i - 1][j] 为 true, 那么 dp[i][j] 必然也为 true
        # 所以, 先初始化 dp[i][j] 的值为 dp[i - 1][j] 的值
        # 再通过后面的代码修改 dp[i][j] 中还为 false 的值
        dp[i][j] = dp[i - 1][j]

        # 如果此时 nums[i] 恰巧为 j
        # 那么对于 dp[i][j] 来说, nums 的前 i 个元素可以组成和为 j 的结果
        if nums[i] == j :
            # 所以 dp[i][j] 为 true
            dp[i][j] = True
            # 同时继续
            continue

        # 如果发现 nums[i] 小于 j
        if nums[i] < j :
            # dp[i][j] 表示 nums 的前 i 个元素是否可以组成和为 j 的结果
            # dp[i - 1][j] 表示 nums 的前 i - 1 个元素是否可以组成和为 j 的结果
            # dp[i - 1][j - nums[i]] 表示 nums 的前 i - 1 个元素是否可以组成和为 j -
nums[i] 的结果

            # 那么 dp[i][j] 的结果要为 true
            # 1、nums 的前 i - 1 个元素可以组成和为 j
            # 2、nums 的前 i - 1 个元素可以组成和为 j - nums[i]
            # 对于 1 来说, 不使用 nums[i] 就组成了 j
            # 对于 2 来说, 前 i - 1 个元素可以组成和为 j - nums[i], 那么加上此时的值
nums[i], 也组成了 j

            dp[i][j] = dp[i - 1][j] | dp[i - 1][j - nums[i]]

```



```
# dp[n - 1][target] 表示 nums 的前 n - 1 个元素是否可以组成和为 target 的结果
# 返回这个结果
return dp[n - 1][target]
```

LeetCode 474、一和零

一、题目描述

给你一个二进制字符串数组 `strs` 和两个整数 `m` 和 `n`。

请你找出并返回 `strs` 的最大子集的长度，该子集中最多有 `m` 个 0 和 `n` 个 1。

如果 `x` 的所有元素也是 `y` 的元素，集合 `x` 是集合 `y` 的子集。

示例 1：

输入：strs = ["10", "0001", "111001", "1", "0"], m = 5, n = 3

输出：4

解释：最多有 5 个 0 和 3 个 1 的最大子集是 {"10","0001","1","0"}，因此答案是 4。

其他满足题意但较小的子集包括 {"0001","1"} 和 {"10","1","0"}。{"111001"} 不满足题意，因为它含 4 个 1，大于 n 的值 3。

示例 2：

输入：strs = ["10", "0", "1"], m = 1, n = 1

输出：2

解释：最大的子集是 {"0", "1"}，所以答案是 2。

提示：

- `1 <= strs.length <= 600`
- `1 <= strs[i].length <= 100`
- `strs[i]` 仅由 '0' 和 '1' 组成
- `1 <= m, n <= 100`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 一和零 (LeetCode 474)：https://leetcode.cn/problems/ones-and-zeroes/
class Solution {
    public int findMaxForm(String[] strs, int m, int n) {
```

```

// 获取字符串数组的长度
int len = strs.length;

// dp[i][j][k] 表示
// 1、从前 i 个字符串中选出若干个
// 2、在使得其中 0 的数目不超过 j
// 3、1 的数目不超过 k 的情况下能够选出的最多的字符串数目
int[][][] dp = new int[len + 1][m + 1][n + 1];

// 开始填充这个三维数组
// 1、物品一个一个尝试
// 2、容量一点一点尝试
// 3、每个物品分类讨论：选与不选
for (int i = 1; i <= len; i++) {

    // 注意到 i 是从下标 1 开始访问的
    // 对于字符串数组 strs 来说，区间 [ 0 , 1 ] 的这个字符串是 strs[0]
    // 也就是说，区间 [ 0 , i ] 的最后一个字符串是 strs[i - 1]
    // 需要讨论 strs[i - 1] 能否加入到我们的【背包当中】
    String currentStr = strs[ i - 1 ];

    // 统计 strs[ i - 1 ] 中 0 的数量
    int zeroNum = countZeroNumber(currentStr);

    // 统计 strs[ i - 1 ] 中 1 的数量
    int oneNum = countOneNumber(currentStr);

    // 01 背包问题，开始讨论
    for( int j = 0 ; j <= m ; j++){

        for( int k = 0 ; k <= n ; k++){

            // 1、如果发现目前能够使用的 0 的个数 j 小于了当前字符串中 0 的数量
            // 2、如果发现目前能够使用的 1 的个数 k 小于了当前字符串中 1 的数量
            if( j < zeroNum || k < oneNum){

                // 意味着当前这个字符串 currentStr 无法放入到背包当中
                // 此时，不选择当前考虑的这个字符串
                // 能够得到的字符串数目的最大值可以通过前 i - 1 个字符串中得到结果
                dp[ i ][ j ][ k ] = dp[ i - 1 ][ j ][ k ];

                // 否则，当前这个字符串有资格加入都背包当中
            }else{

                // 在有资格的情况下，可以选，也可以不选
                // 1、不选， 能够得到的字符串数目的最大值可以通过前 i - 1 个字符串中得到

                // 2、选，那么当前这个字符串必然加入进去，子集长度加 1
            }
        }
    }
}

```

结果

```

        // 同时 0 的个数剩下 j - zeroNum, 1 的个数剩下 k - oneNum
        // 需从前 i - 1 个字符串中去凑这些 0 和 1
        dp[i][j][k] = Math.max(dp[i - 1][j][k], dp[i - 1][j - zeroNum]
[k - oneNum] + 1);
    }
}
}

// 返回结果
return dp[len][m][n];
}

// 统计字符串 str 中 0 的个数
private int countZeroNumber(String str){

    int count = 0;

    for(char c : str.toCharArray()){

        if( c == '0'){

            count++;

        }

    }

    return count;
}

// 统计字符串 str 中 1 的个数
private int countOneNumber(String str){

    int count = 0;

    for(char c : str.toCharArray()){

        if( c == '1'){

            count++;

        }

    }

    return count;
}

```

```
}
```

2、**C++** 代码

```
class Solution {
public:
    int findMaxForm(vector<string>& strs, int m, int n) {

        // 获取字符串数组的长度
        int len = strs.size();

        // dp[i][j][k] 表示
        // 1、从前 i 个字符串中选出若干个
        // 2、在使得其中 0 的数目不超过 j
        // 3、1 的数目不超过 k 的情况下能够选出的最多的字符串数目
        vector<vector<vector<int>>> dp(len + 1, vector<vector<int>>(m + 1, vector<int>(n + 1)));

        // 开始填充这个三维数组
        // 1、物品一个一个尝试
        // 2、容量一点一点尝试
        // 3、每个物品分类讨论：选与不选
        for (int i = 1; i <= len; i++) {

            // 注意到 i 是从下标 1 开始访问的
            // 对于字符串数组 strs 来说，区间 [ 0 , 1 ] 的这个字符串是 strs[0]
            // 也就是说，区间 [ 0 , i ] 的最后一个字符串是 strs[i - 1]
            // 需要讨论 strs[i - 1] 能否加入到我们的【背包当中】
            string currentStr = strs[ i - 1 ];

            // 统计 strs[ i - 1 ] 中 0 的数量
            int zeroNum = countZeroNumber(currentStr);

            // 统计 strs[ i - 1 ] 中 1 的数量
            int oneNum = countOneNumber(currentStr);

            // 01 背包问题，开始讨论
            for( int j = 0 ; j <= m ; j++){

                for( int k = 0 ; k <= n ; k++){

                    // 1、如果发现目前能够使用的 0 的个数 j 小于了当前字符串中 0 的数量
                    // 2、如果发现目前能够使用的 1 的个数 k 小于了当前字符串中 1 的数量
                    if( j < zeroNum || k < oneNum){

                        // 意味着当前这个字符串 currentStr 无法放入到背包当中
                        // 此时，不选择当前考虑的这个字符串
                        // 能够得到的字符串数目的最大值可以通过前 i - 1 个字符串中得到结果
                        dp[ i ][ j ][ k ] = dp[ i - 1 ][ j ][ k ];
                    }
                }
            }
        }
    }
};
```

结果

```
// 否则，当前这个字符串有资格加入都背包当中
}else{

    // 在有资格的情况下，可以选，也可以不选
    // 1、不选， 能够得到的字符串数目的最大值可以通过前 i - 1 个字符串中得到

    // 2、选，那么当前这个字符串必然加入进去，子集长度加 1
    // 同时 0 的个数剩下 j - zeroNum, 1 的个数剩下 k - oneNum
    // 需要从前 i - 1 个字符串中去凑这些 0 和 1
    dp[i][j][k] = max(dp[i - 1][j][k], dp[i - 1][j - zeroNum][k -
oneNum] + 1);
    }
}

// 返回结果
return dp[len][m][n];

}

private:
int countZeroNumber(string str){

    int count = 0;

    for(int i = 0 ; i < str.size() ; i++){

        if( str[i] == '0'){

            count++;

        }

    }

    return count;
}

private:
int countOneNumber(string str){

    int count = 0;

    for(int i = 0 ; i < str.size() ; i++){

        if( str[i] == '1'){
```

```

        count++;

    }

}

return count;
}

};

```

3、Python 代码

```

class Solution:
    def findMaxForm(self, strs: List[str], m: int, n: int) -> int:

        length = len(strs)

        # dp[i][j][k] 表示
        # 1、从前 i 个字符串中选出若干个
        # 2、在使得其中 0 的数目不超过 j
        # 3、1 的数目不超过 k 的情况下能够选出的最多的字符串数目
        dp = [[[0] * (n + 1) for _ in range(m + 1)] for _ in range(length+1)]

        # 开始填充这个三维数组
        # 1、物品一个一个尝试
        # 2、容量一点一点尝试
        # 3、每个物品分类讨论：选与不选
        for i in range(1, length+1):

            # 注意到 i 是从下标 1 开始访问的
            # 对于字符串数组 strs 来说，区间 [ 0 , 1 ] 的这个字符串是 strs[0]
            # 也就是说，区间 [ 0 , i ] 的最后一个字符串是 strs[i - 1]
            # 需要讨论 strs[i - 1] 能否加入到我们的【背包当中】

            # 统计 strs[ i - 1 ] 中 0 的数量
            c0 = strs[i-1].count('0')

            # 统计 strs[ i - 1 ] 中 1 的数量
            c1 = len(strs[i-1]) - c0
            for j in range(m+1):
                for k in range(n+1):
                    # 1、如果发现目前能够使用的 0 的个数 j 小于了当前字符串中 0 的数量
                    # 2、如果发现目前能够使用的 1 的个数 k 小于了当前字符串中 1 的数量
                    if j < c0 or k < c1:
                        dp[i][j][k] = dp[i-1][j][k]
                    else:
                        # 在有资格的情况下，可以选，也可以不选
                        # 1、不选， 能够得到的字符串数目的最大值可以通过前 i - 1 个字符串中得到结

```

```

        # 2、选，那么当前这个字符串必然加入进去，子集长度加 1
        # 同时 0 的个数剩下 j - zeroNum, 1 的个数剩下 k - oneNum
        # 需要从前 i - 1 个字符串中去凑这些 0 和 1
        dp[i][j][k] = max( dp[i-1][j][k], dp[i-1][j-c0][k-c1] + 1 )

    return dp[length][m][n]

```

LeetCode 494、目标和

```

class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        # 每个数字都有两种状态：被进行“+”， 或者被进行“-”
        # 因此可以将数组分成 A 和 B 两个部分
        # A 部分的数字全部进行“+”操作
        # B 部分的数字全部进行“-”操作

        # 设数组的和为 sum，A 部分的和为 sumA，B 部分的和为 sumB

        # sumA + sumB = sum (1)
        # 同时有： sumA - sumB = target (2)
        # 将(1)式与(2)式相加，可以得到： 2sumA = sum + target (3)
        # 即 sumA = (sum + target) / 2
        # 所以，原问题转换为：
        # 有一些物品，第 i 个物品的重量为 nums[i]，背包的容量为 sumA，问：有多少种方式将背包【恰好填满】

        # 先去计算总和
        total = sum(nums)

        # 再去计算背包容量
        bagSize = (target + total) // 2

        # 题目有可能出现 target 是一个很小的负数
        # 比如 nums = [ 100 ]，target = -200
        # 这个时候 bagSize 就是负数了，需要调整为正数
        if bagSize < 0 : bagSize = -bagSize

        # System.out.println(bagSize)

        # 如果发现 target + sum 为奇数，则无解
        # 比如 nums = [ 1 ]，target = 2
        # target + sum = 2 + 1 = 3，此时无解
        # 因为 bagSize 必然是整数，即 (target + sum) / 2 必然是整数
        # 那么只有 target + sum 为偶数才能得到整数
        if (target + total) % 2 == 1 : return 0

        # 在前 i 个数字中，凑出和为 j 的组合有多少种方法
        # dp[0][0] 表示在前 0 个数字中，凑出和为 0 的组合，有多少种方法

```

```

dp = [[0] * (bagSize + 1) for _ in range(len(nums) + 1)]
# 初始化
for i in range(len(nums) + 1) :
    # 在前 i 个数字中，凑出和为 0 的组合有多少种方法
    # 答案是 1 种方法
    # 即每次不选择第 i 个元素就行
    dp[i][0] = 1

# 01 背包问题开始填充二维数组
for i in range( 1 , len(nums) + 1 ) :

    for j in range(bagSize + 1) :

        # 注意到 i 是从下标 1 开始访问的
        # 1、背包容量小于当前元素
        # 背包无法放入 nums[i - 1]
        if j < nums[i - 1] :

            dp[i][j] = dp[i - 1][j]

        # 2、背包容量大于等于当前元素
        # 背包可以放入 nums[i - 1]
        else:

            # 不选：方案数为 dp[i - 1][j]
            # 选：方案数为 dp[i - 1][j - nums[i - 1]]
            # 两者之和
            dp[i][j] = dp[i - 1][j] + dp[i - 1][j - nums[i - 1]]

# 返回结果
return dp[len(nums)][bagSize]

```

LeetCode 518、零钱兑换II

一、题目描述

给你一个整数数组 `coins` 表示不同面额的硬币，另给一个整数 `amount` 表示总金额。

请你计算并返回可以凑成总金额的硬币组合数。如果任何硬币组合都无法凑出总金额，返回 `0` 。

假设每一种面额的硬币有无限个。

题目数据保证结果符合 32 位带符号整数。

示例 1：

输入: amount = 5, coins = [1, 2, 5]

输出: 4

解释: 有四种方式可以凑成总金额:

5=5

5=2+2+1

5=2+1+1+1

5=1+1+1+1+1

示例 2:

输入: amount = 3, coins = [2]

输出: 0

解释: 只用面额 2 的硬币不能凑成总金额 3 。

示例 3:

输入: amount = 10, coins = [10]

输出: 1

提示:

- `1 <= coins.length <= 300`
- `1 <= coins[i] <= 5000`
- `coins` 中的所有值 互不相同
- `0 <= amount <= 5000`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 零钱兑换II (LeetCode 518) : https://leetcode.cn/problems/coin-change-2/submissions/
class Solution {
    public int change(int amount, int[] coins) {
        // 经典的完全背包问题

        // 二维数组: 状态定义:dp[i][j]表示从数组 coins 中前 i 个硬币中选择得到价值为 j 的硬币组合数
        int[][] dp = new int[coins.length + 1][ amount + 1 ];

        // 只有当不选取任何硬币时, 金额之和才为 0, 此时只有 1 种硬币组合
        dp[0][0] = 1;
```

数

```
// Tips: 前 i 个物品表示的是编号从 [ 0 , i - 1] 这些物品, 即最后一个物品编号为 i - 1
// coins 数组的大小就是硬币的个数
for( int i = 1 ; i <= coins.length ; i++){
    // 遍历背包容量
    for( int j = 0 ; j <= amount ; j++){

        // 背包容量小于当前硬币的值, 即背包容量已经不足以拿第 i 个硬币了
        // 第 i 个硬币即下标为 i - 1 的那个硬币
        if( j < coins[i - 1]){

            // 硬币组合数就是去考虑当前背包容量情况下, 从前 i - 1 个物品中选择出的硬币组合数

            dp[i][j] = dp[i - 1][j];

            // 背包容量足够拿第 i 个物品, 那么可拿也可不拿
            // 1、拿了, 那么硬币组合数是前 i 个硬币扣除第 i 个硬币的情况下的硬币组合数
            // 2、不拿, 那么就是从前 i - 1 个硬币中选择出的硬币组合数
        }else{

            dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]];
        }

        // System.out.print(dp[i][j] + "");
    }
    // System.out.println();
}

return dp[coins.length][amount];

}
}
```

一维**DP**

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 零钱兑换II (LeetCode 518) : https://leetcode.cn/problems/coin-change-2/submissions/
class Solution {
    public int change(int amount, int[] coins) {
        // 经典的完全背包问题

        // 二维数组: 状态定义:dp[i][j]表示从数组 coins 中前 i 个硬币中选择得到价值为 j 的硬币组合数

        int[] dp = new int[ amount + 1 ];
```

```

// 只有当不选取任何硬币时，金额之和才为 0，此时只有 1 种硬币组合
dp[0] = 1;

// Tips: 前 i 个物品表示的是编号从 [ 0 , i - 1] 这些物品，即最后一个物品编号为 i - 1
// coins 数组的大小就是硬币的个数
for( int i = 1 ; i <= coins.length ; i++){
    // 遍历背包容量
    for( int j = 0 ; j <= amount ; j++){

        // 背包容量足够拿第 i 个物品，那么可拿也可不拿
        // 1、拿了，那么硬币组合数是前 i 个硬币扣除第 i 个硬币的情况下的硬币组合数
        // 2、不拿，那么就是从前 i - 1 个硬币中选择出的硬币组合数
        if( j >= coins[i - 1]){
            // 硬币组合数就是去考虑当前背包容量情况下，从前 i - 1 个物品中选择出的硬币组合数
            dp[j] = dp[j] + dp[j-coins[i-1]];
        }
    }
}

return dp[amount];
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int change(int amount, vector<int>& coins) {

        // 经典的完全背包问题

        // 二维数组：状态定义:dp[i][j]表示从数组 coins 中前 i 个硬币中选择得到价值为 j 的硬币组合数
        int n = coins.size();
        vector<vector<int>>>dp(n + 1, vector<int>(amount + 1 , 0));

        // 只有当不选取任何硬币时，金额之和才为 0，此时只有 1 种硬币组合
        dp[0][0] = 1;

        // Tips: 前 i 个物品表示的是编号从 [ 0 , i - 1] 这些物品，即最后一个物品编号为 i - 1
        // coins 数组的大小就是硬币的个数
        for( int i = 1 ; i <= n ; i++){
            // 遍历背包容量
            for( int j = 0 ; j <= amount ; j++){

                // 背包容量小于当前硬币的值，即背包容量已经不足以拿第 i 个硬币了
            }
        }
    }
}

```

数

```
// 第 i 个硬币即下标为 i - 1 的那个硬币
if( j < coins[i - 1]){

    // 硬币组合数就是去考虑当前背包容量情况下，从前 i - 1 个物品中选择出的硬币组合

    dp[i][j] = dp[i - 1][j];

    // 背包容量足够拿第 i 个物品，那么可拿也可不拿
    // 1、拿了，那么硬币组合数是前 i 个硬币扣除第 i 个硬币的情况下的硬币组合数
    // 2、不拿，那么就是从前 i - 1 个硬币中选择出的硬币组合数
}else{

    dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]];
}

}

}

return dp[n][amount];

}

};
```

3、Python 代码

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:

        n = len(coins)
        dp = [[0]*(amount+1) for _ in range(n+1)]

        # 初始化
        dp[0][0] = 1

        # 完全背包
        # 第一层循环：遍历硬币
        for i in range(1, n+1):

            # 第二层循环：遍历背包
            for j in range(amount+1):
                # 容量有限，无法选择第i个硬币
                if j < coins[i-1]:
                    dp[i][j] = dp[i-1][j]
                # 可选择第i个硬币
                else:
                    dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]]

        return dp[n][amount]
```

第五十天

LeetCode 208、实现Trie（前缀树）

一、题目描述

Trie（发音类似 "try"）或者说 **前缀树** 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补完和拼写检查。

请你实现 Trie 类：

- `Trie()` 初始化前缀树对象。
- `void insert(String word)` 向前缀树中插入字符串 `word`。
- `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true`（即，在检索之前已经插入）；否则，返回 `false`。
- `boolean startswith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix`，返回 `true`；否则，返回 `false`。

示例：

输入

```
["Trie", "insert", "search", "search", "startswith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

输出

```
[null, null, true, false, true, null, true]
```

解释

```
Trie trie = new Trie();  
trie.insert("apple");  
trie.search("apple"); // 返回 True  
trie.search("app");   // 返回 False  
trie.startswith("app"); // 返回 True  
trie.insert("app");  
trie.search("app");   // 返回 True
```

提示：

- `1 <= word.length, prefix.length <= 2000`
- `word` 和 `prefix` 仅由小写英文字母组成
- `insert`、`search` 和 `startswith` 调用次数 总计 不超过 `3 * 104` 次

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 实现Trie（前缀树）（LeetCode 208）：https://leetcode.cn/problems/implement-trie-prefix-tree/
class Trie {
    class TrieNode {
        // 节点表示从根节点到本节点的路径构成的字符串是否有效
        boolean val;
        // 每个节点均有 26 个孩子节点
        TrieNode[] children = new TrieNode[26];
    }

    // 根节点不存储 val 值
    private TrieNode root;

    public Trie() {
        // 初始 root
        root = new TrieNode();
    }

    public void insert(String word) {
        // 插入操作

        // 从根节点位置出发开始操作
        TrieNode cur = root;

        // 把字符串 word 里面的每个字符都插入到树中
        for (char c : word.toCharArray()) {
            // 通过 ASCII 求差定位当前这个字符的索引位置
            // 比如当前字符为 a，那么 a - a = 0，索引位置为 0
            // 比如当前字符为 b，那么 b - a = 1，索引位置为 1
            // 比如当前字符为 c，那么 c - a = 2，索引位置为 2
            int index = c - 'a';

            // 判断当前索引的元素是否存在
            // 如果不存在，那么就初始化一个
            if (cur.children[index] == null) {
                cur.children[index] = new TrieNode();
            }

            // 如果存在，那么就可以直接使用它
        }
    }
}
```

```

        // 接下来，顺着它来到它的子节点位置
        // 比如插入 algo
        // 一开始 cur 来到了 a 这个字符
        // 再来到 l 这个字符
        cur = cur.children[index];
    }

    // 跳出 for 循环之后，cur 到达了单词最后一个字符的位置
    // 节点表示从根节点到本节点的路径构成的字符串是否有效
    // 因此，当前节点的值更新为 true
    cur.val = true;
}

public boolean search(String word) {
    // 查找操作

    // 从根节点位置出发开始操作
    TrieNode cur = root;

    // 访问字符串 word 里面的每个字符
    // 查看能否在树中找到这条路径
    for (char c : word.toCharArray()) {

        // 通过 ASCII 求差定位当前这个字符的索引位置
        int index = c - 'a';

        // 如果发现这条路径上的某个节点都没有被初始
        if (cur.children[index] == null) {

            // 那么说明这个单词之前并没有完整保存下来
            // 直接返回 false
            return false;

        }

        // 接下来，顺着它来到它的子节点位置
        cur = cur.children[index];
    }

    // 跳出 for 循环之后，cur 到达了单词最后一个字符的位置
    // 节点表示从根节点到本节点的路径构成的字符串是否有效
    // 如果存在路径，那么 cur.val 就是 true
    // 如果存在这条路径，那么 cur.val 就是默认 false
    return cur.val;
}

public boolean startswith(String prefix) {

    // 查找前缀操作

```

```

TrieNode cur = root;

// 访问字符串 word 里面的每个字符
for (char c : prefix.toCharArray()) {
    // 通过 ASCII 求差定位当前这个字符的索引位置
    int index = c - 'a';

    // 如果发现这条路径上的某个节点都没有被初始
    if (cur.children[index] == null){

        // 那么说明这个前缀之前并没有完整保存下来
        // 直接返回 false
        return false;
    }

    // 接下来, 顺着它来到它的子节点位置
    cur = cur.children[index];
}

// 跳出 for 循环之后, 说明 prefix 里面的每个字符均能按照顺序在树中找到
// 所以这个前缀存在, 返回 true
return true;
}
}

```

2、**C++** 代码

```

class Trie {
private:
    vector<Trie*> children;
    bool isEnd;

    Trie* searchPrefix(string prefix) {
        Trie* node = this;
        for (char ch : prefix) {
            ch -= 'a';
            if (node->children[ch] == nullptr) {
                return nullptr;
            }
            node = node->children[ch];
        }
        return node;
    }

public:
    Trie() : children(26), isEnd(false) {}

    void insert(string word) {
        Trie* node = this;

```



```

    for (char ch : word) {
        ch -= 'a';
        if (node->children[ch] == nullptr) {
            node->children[ch] = new Trie();
        }
        node = node->children[ch];
    }
    node->isEnd = true;
}

bool search(string word) {
    Trie* node = this->searchPrefix(word);
    return node != nullptr && node->isEnd;
}

bool startswith(string prefix) {
    return this->searchPrefix(prefix) != nullptr;
}
};

```

3、Python 代码

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.isEnd = False

    def searchPrefix(self, prefix: str) -> "Trie":
        node = self
        for ch in prefix:
            ch = ord(ch) - ord("a")
            if not node.children[ch]:
                return None
            node = node.children[ch]
        return node

    def insert(self, word: str) -> None:
        node = self
        for ch in word:
            ch = ord(ch) - ord("a")
            if not node.children[ch]:
                node.children[ch] = Trie()
            node = node.children[ch]
        node.isEnd = True

    def search(self, word: str) -> bool:
        node = self.searchPrefix(word)
        return node is not None and node.isEnd

```

```
def startswith(self, prefix: str) -> bool:
    return self.searchPrefix(prefix) is not None
```

LeetCode 211、添加与搜索单词 - 数据结构设计

一、题目描述

请你设计一个数据结构，支持 添加新单词 和 查找字符串是否与任何先前添加的字符串匹配。

实现词典类 `wordDictionary`：

- `wordDictionary()` 初始化词典对象
- `void addWord(word)` 将 `word` 添加到数据结构中，之后可以对它进行匹配
- `bool search(word)` 如果数据结构中存在字符串与 `word` 匹配，则返回 `true`；否则，返回 `false`。
。 `word` 中可能包含一些 `'.'`，每个 `.` 都可以表示任何一个字母。

示例：

输入：

```
["wordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
```

输出：

```
[null,null,null,null,false,true,true,true]
```

解释：

```
wordDictionary wordDictionary = new wordDictionary();
wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // 返回 False
wordDictionary.search("bad"); // 返回 True
wordDictionary.search(".ad"); // 返回 True
wordDictionary.search("b.."); // 返回 True
```

提示：

- `1 <= word.length <= 25`
- `addWord` 中的 `word` 由小写英文字母组成
- `search` 中的 `word` 由 `'.'` 或小写英文字母组成
- 最多调用 `104` 次 `addWord` 和 `search`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 添加与搜索单词 - 数据结构设计 (LeetCode 211) : https://leetcode.cn/problems/design-add-and-search-words-data-structure/
class wordDictionary {

    class TrieNode {
        // 节点表示从根节点到本节点的路径构成的字符串是否有效
        boolean val;
        // 每个节点均有 26 个孩子节点
        TrieNode[] children = new TrieNode[26];
    }

    // 根节点不存储 val 值
    private TrieNode root;

    public wordDictionary() {

        // 初始 root
        root = new TrieNode();
    }

    public void addWord(String word) {
        // 插入操作

        // 从根节点位置出发开始操作
        TrieNode cur = root;

        // 把字符串 word 里面的每个字符都插入到树中
        for (char c : word.toCharArray()) {
            // 通过 ASCII 求差定位当前这个字符的索引位置
            // 比如当前字符为 a , 那么 a - a = 0 , 索引位置为 0
            // 比如当前字符为 b , 那么 b - a = 1 , 索引位置为 1
            // 比如当前字符为 c , 那么 c - a = 2 , 索引位置为 2
            int index = c - 'a';

            // 判断当前索引的元素是否存在
            // 如果不存在, 那么就初始化一个
            if (cur.children[index] == null) {
                cur.children[index] = new TrieNode();
            }

            // 如果存在, 那么就可以直接使用它
        }
    }
}
```

```

        // 接下来，顺着它来到它的子节点位置
        // 比如插入 algo
        // 一开始 cur 来到了 a 这个字符
        // 再来到 l 这个字符
        cur = cur.children[index];
    }

    // 跳出 for 循环之后，cur 到达了这个单词最后一个字符的位置
    // 节点表示从根节点到本节点的路径构成的字符串是否有效
    // 因此，当前节点的值更新为 true
    cur.val = true;
}

public boolean search(String word) {

    return dfs( word , root , 0 );

}

public boolean dfs(String word, TrieNode node, int index) {

    // 如果 node 为空，代表当前这个位置没有存储任何信息
    // 也就意味着从 root 到 node 这个位置没有存储 word 这个单词的字符路径
    // 返回 false
    if( node == null) return false;

    // 递归终止条件
    // word 从 0 到 word.length() - 2 的字符都匹配成功
    // 返回 node 处于 word.length() - 1 层时的结果就行，即最后一个单词是否匹配成功的结果
    if( index == word.length() ) return node.val;

    // 通过 ASCII 求差定位当前这个字符的索引位置
    int i = word.charAt(index) - 'a';

    // 在匹配的过程中，如果遇到了 '.'
    if( word.charAt(index) == '.' ){

        // 需要对当前节点的所有子树都进行遍历
        for( int j = 0 ; j < 26 ; j++){

            // 只要有任何一个子树能最终匹配完成，那么就代表能匹配完成，返回 true
            if( dfs(word, node.children[j] , index + 1)) return true;

        }

        // for 循环结束之后，意味着没有任何一个子树能最终匹配完成
        // 返回 false
        return false;
    }

```

```

        // 在匹配的过程中，如果遇到的不是 '.'
        // 那么就是普通的小写字母
    }else{

        // 需要继续查看下一个字符是否匹配
        return dfs(word , node.children[i] , index + 1);
    }

}

}
}

```

LeetCode 648、单词替换

一、题目描述

在英语中，我们有一个叫做 **词根 (root)** 的概念，可以词根后面添加其他一些词组成另一个较长的单词——我们称这个词为 **继承词 (successor)**。例如，词根 **an**，跟随着单词 **other** (其他)，可以形成新的单词 **another** (另一个)。

现在，给定一个由许多词根组成的词典 **dictionary** 和一个用空格分隔单词形成的句子 **sentence**。你需要将句子中的所有**继承**词**用**词根替换掉**。如果继承词有许多可以形成它的词根，则用**最短**的词根**替换它。

你需要输出替换之后的句子。

示例 1:

输入: dictionary = ["cat","bat","rat"], sentence = "the cattle was rattled by the battery"
输出: "the cat was rat by the bat"

示例 2:

输入: dictionary = ["a","b","c"], sentence = "aadsfasf absbs bbab cadsfafs"
输出: "a a b c"

提示:

- `1 <= dictionary.length <= 1000`
- `1 <= dictionary[i].length <= 100`
- `dictionary[i]` 仅由小写字母组成。
- `1 <= sentence.length <= 10^6`
- `sentence` 仅由小写字母和空格组成。
- `sentence` 中单词的总量在范围 `[1, 1000]` 内。
- `sentence` 中每个单词的长度在范围 `[1, 1000]` 内。
- `sentence` 中单词之间由一个空格隔开。

- `sentence` 没有前导或尾随空格。

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 单词替换 (LeetCode 648) :https://leetcode.cn/problems/replace-words/submissions/
class Solution {

    class TrieNode {
        // 节点表示从根节点到本节点的路径构成的字符串是否有效
        boolean val;
        // 每个节点均有 26 个孩子节点
        TrieNode[] children = new TrieNode[26];
    }

    // 根节点不存储 val 值
    // 初始 root
    TrieNode root = new TrieNode();

    public void insert(String word) {
        // 插入操作

        // 从根节点位置出发开始操作
        TrieNode cur = root;

        // 把字符串 word 里面的每个字符都插入到树中
        for (char c : word.toCharArray()) {
            // 通过 ASCII 求差定位当前这个字符的索引位置
            // 比如当前字符为 a , 那么 a - a = 0 , 索引位置为 0
            // 比如当前字符为 b , 那么 b - a = 1 , 索引位置为 1
            // 比如当前字符为 c , 那么 c - a = 2 , 索引位置为 2
            int index = c - 'a';

            // 判断当前索引的元素是否存在
            // 如果不存在, 那么就初始化一个
            if (cur.children[index] == null) {
                cur.children[index] = new TrieNode();
            }

            // 如果存在, 那么就可以直接使用它

            // 接下来, 顺着它来到它的子节点位置
        }
    }
}
```

```

        // 比如插入 algo
        // 一开始 cur 来到了 a 这个字符
        // 再来到 l 这个字符
        cur = cur.children[index];
    }

    // 跳出 for 循环之后, cur 到达了这个单词最后一个字符的位置
    // 节点表示从根节点到本节点的路径构成的字符串是否有效
    // 因此, 当前节点的值更新为 true
    cur.val = true;
}

public String search(String word) {
    // 查找操作

    // 从根节点位置出发开始操作
    TrieNode cur = root;

    // 访问字符串 word 里面的每个字符
    // 查看能否在树中找到这条路径
    for (int i = 0 ; i < word.length() ; i++) {

        // 通过 ASCII 求差定位当前这个字符的索引位置
        int index = word.charAt(i) - 'a';

        // 如果发现这条路径上的某个节点都没有被初始
        if (cur.children[index] == null) {

            // 那么说明这个单词之前并没有完整保存下来
            // 直接跳出循环
            break;

        }

        if(cur.children[index].val) {

            return word.substring( 0 , i + 1);

        }

        // 接下来, 顺着它来到它的子节点位置
        cur = cur.children[index];
    }

    // 跳出 for 循环, 说明 word 这个单词在 dictionary 没有找到前缀
    // 需要把这个单词完全返回
    return word;
}

```

```
public String replacewords(List<String> dictionary, String sentence) {  
  
    for( String str : dictionary ){  
  
        insert(str);  
  
    }  
  
    StringBuilder sb = new StringBuilder();  
  
    for(String str : sentence.split(" ")){  
  
        sb.append(search(str)).append(" ");  
  
    }  
  
    return sb.substring( 0 , sb.length() - 1);  
  
}
```

LeetCode 676、实现一个魔法字典

一、题目描述

设计一个使用单词列表进行初始化的数据结构，单词列表中的单词 **互不相同** 。如果给出一个单词，请判定能否只将这个单词中一个字母换成另一个字母，使得所形成的新单词存在于你构建的字典中。

实现 `MagicDictionary` 类：

- `MagicDictionary()` 初始化对象
- `void buildDict(String[] dictionary)` 使用字符串数组 `dictionary` 设定该数据结构，`dictionary` 中的字符串互不相同
- `bool search(String searchword)` 给定一个字符串 `searchword` ，判定能否只将字符串中 一个 字母换成另一个字母，使得所形成的新字符串能够与字典中的任一字符串匹配。如果可以，返回 `true` ；否则，返回 `false` 。

示例：

输入

```
["MagicDictionary", "buildDict", "search", "search", "search", "search"]  
[[], [{"hello", "leetcode"}], ["hello"], ["hhlllo"], ["hell"], ["leetcoded"]]
```

输出

```
[null, null, false, true, false, false]
```

解释

```
MagicDictionary magicDictionary = new MagicDictionary();  
magicDictionary.buildDict(["hello", "leetcode"]);  
magicDictionary.search("hello"); // 返回 False  
magicDictionary.search("hhlllo"); // 将第二个 'h' 替换为 'e' 可以匹配 "hello" , 所以返回 True  
magicDictionary.search("hell"); // 返回 False  
magicDictionary.search("leetcoded"); // 返回 False
```

提示：

- `1 <= dictionary.length <= 100`
- `1 <= dictionary[i].length <= 100`
- `dictionary[i]` 仅由小写英文字母组成
- `dictionary` 中的所有字符串 互不相同
- `1 <= searchword.length <= 100`
- `searchword` 仅由小写英文字母组成
- `buildDict` 仅在 `search` 之前调用一次
- 最多调用 `100` 次 `search`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解  
// https://www.algomooc.com  
// 作者：程序员吴师兄  
// 添加微信 278166530 获取最新课程  
// 代码有看不懂的地方一定要私聊咨询吴师兄呀  
// 实现一个魔法字典 (LeetCode 676) : https://leetcode.cn/problems/implement-magic-dictionary/submissions/  
class MagicDictionary {  
  
    class TrieNode {  
        // 节点表示从根节点到本节点的路径构成的字符串是否有效  
        boolean val;  
        // 每个节点均有 26 个孩子节点  
        TrieNode[] children = new TrieNode[26];  
    }  
  
    // 根节点不存储 val 值  
    private TrieNode root;
```

```

public MagicDictionary() {

    // 初始 root
    root = new TrieNode();

}

public void buildDict(String[] dictionary) {

    // 把字符串 word 里面的每个字符都插入到树中
    for (String word : dictionary) {

        // 从根节点位置出发开始操作
        TrieNode cur = root;

        // 把字符串 word 里面的每个字符都插入到树中
        for (char c : word.toCharArray()) {
            // 通过 ASCII 求差定位当前这个字符的索引位置
            // 比如当前字符为 a , 那么 a - a = 0 , 索引位置为 0
            // 比如当前字符为 b , 那么 b - a = 1 , 索引位置为 1
            // 比如当前字符为 c , 那么 c - a = 2 , 索引位置为 2
            int index = c - 'a';

            // 判断当前索引的元素是否存在
            // 如果不存在, 那么就初始化一个
            if (cur.children[index] == null) {
                cur.children[index] = new TrieNode();
            }

            // 如果存在, 那么就可以直接使用它

            // 接下来, 顺着它来到它的子节点位置
            // 比如插入 algo
            // 一开始 cur 来到了 a 这个字符
            // 再来到 l 这个字符
            cur = cur.children[index];
        }

        // 跳出 for 循环之后, cur 到达了单词最后一个字符的位置
        // 节点表示从根节点到本节点的路径构成的字符串是否有效
        // 因此, 当前节点的值更新为 true
        cur.val = true;
    }
}

public boolean search(String searchword) {

    // 对单词 searchword 里面的每个字符都尝试替换一下

```

```

// 即固定其它的字符，仅修改第 i 个字符
for( int i = 0 ; i < searchword.length() ; i++){

    // 如果可以替换成功，返回 true
    if( dfs( searchword , root , 0 , i)) return true;

}

// 跳出 for 循环后，说明无法匹配成功
// 返回 false
return false;

}

public boolean dfs(String searchword, TrieNode node, int index, int changeID) {

    // 如果 node 为空，代表当前这个位置没有存储任何信息
    // 也就意味着从 root 到 node 这个位置没有存储 word 这个单词的字符路径
    // 返回 false
    if( node == null) return false;

    // 递归终止条件
    if( index == searchword.length() ){

        return node.val;

    }

    // 通过 ASCII 求差定位当前这个字符的索引位置
    int i = searchword.charAt(index) - 'a';

    // 在匹配的过程中，当前这个字符是开始判断需要替换的字符
    if( index == changeID){

        // 需要对当前节点的所有子树都进行遍历
        for( int j = 0 ; j < 26 ; j++){

            if( j == i ) continue;

            // 在这个前缀树中
            // 沿着其它路径走
            if(dfs(searchword, node.children[j] , index + 1 , changeID)) return
true;

        }

        // for 循环结束之后，意味着没有任何一个子树能最终匹配完成
        // 返回 false

```

```

        return false;

    }else{

        // 需要继续查看下一个字符是否匹配
        return dfs(searchword , node.children[i] , index + 1, changeID);
    }

}

}

```

第五十一天

LeetCode 200、岛屿数量（并查集解法）

一、题目描述

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

示例 1:

输入: grid = [["1","1","1","1","0"], ["1","1","0","1","0"], ["1","1","0","0","0"], ["0","0","0","0","0"]] 输出: 1

示例 2:

输入: grid = [["1","1","0","0","0"], ["1","1","0","0","0"], ["0","0","1","0","0"], ["0","0","0","1","1"]] 输出: 3

提示:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` 的值为 '0' 或 '1'

二、题目解析

三、参考代码

1、Java 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀

```

```
// 岛屿数量 (LeetCode 200) (并查集解法) : https://leetcode.cn/problems/number-of-islands/
class Solution {
    private int rows;

    private int cols;

    public int numIslands(char[][] grid) {

        rows = grid.length;

        cols = grid[0].length;

        // 水格的数量
        int spaces = 0;

        UnionFind unionFind = new UnionFind(rows * cols);

        // 沿着每个网格的右方、下方去查找
        int[][] directions = {{1, 0}, {0, 1}};

        for (int i = 0; i < rows; i++) {

            for (int j = 0; j < cols; j++) {

                // 当前网格是水
                if (grid[i][j] == '0') {
                    // 水格的数量增加
                    spaces++;

                    // 当前网格是陆地
                } else {

                    // 沿着这个陆地的右方、下方这两个方向去查找
                    for (int[] direction : directions) {

                        // 新网格的 x 坐标
                        int newX = i + direction[0];

                        // 新网格的 y 坐标
                        int newY = j + direction[1];

                        // 如果新网格的坐标位于矩阵内
                        // 并且是陆地
                        if (newX < rows && newY < cols && grid[newX][newY] == '1') {
                            // 那么就把这两个网格连接起来
                            unionFind.union(getIndex(i, j), getIndex(newX, newY));
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
}
return unionFind.getCount() - spaces;

}

// 把这些二维网格进行编号
// 比如第 0 行第 0 列网格的编号是 0
// 比如第 0 行第 1 列网格的编号是 1
// 比如第 1 行第 1 列网格的编号是 5 (一列有 5 个元素)
private int getIndex(int i, int j) {
    return i * cols + j;
}
}

// 并查集
class UnionFind {

    int[] roots;

    int count;

    public UnionFind(int n) {
        // 使用一维数组用来记录每个网格的出发位置
        roots = new int[n];
        for (int i = 0; i < n ; i++) {
            // 默认每个网格的出发位置是自己本身
            roots[i] = i;
        }
        // 所以一开始有 n 个岛屿
        count = n ;
    }

    // 寻找当前网格的出发位置
    public int find(int i){

        if( i == roots[i]) return i ;

        roots[i] = find(roots[i]);

        return roots[i];

    }

    // 连通操作，把陆地连接起来
    public void union(int p, int q) {
        // 寻找陆地 p 的出发位置
        int pRoot = find(p);

```

```

// 寻找陆地 q 的出发位置
int qRoot = find(q);

// 如果两者的出发位置不同
// 那么需要采取操作把 p 和 q 所在的两堆网格合到一起
if (pRoot != qRoot) {
    // 也就是把其中一个的最上面的出发位置挂过来就行
    roots[pRoot] = qRoot;
    // 与此同时，两堆陆地变成了一堆陆地
    count--;
}
}

// 获取当前区间的所有连通量
public int getCount() {

    // 也就是有多少陆地网格 + 水网格
    return count;
}

}

```

2、C++ 代码

3、Python 代码

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        class UnionFind:
            def __init__(self, n):
                self.count = n
                self.parent = [i for i in range(n)]

            def get_count(self):
                return self.count

            def find(self, p):
                while p != self.parent[p]:
                    self.parent[p] = self.parent[self.parent[p]]
                    p = self.parent[p]
                return p

            def union(self, p, q):
                p_root = self.find(p)
                q_root = self.find(q)
                if p_root == q_root:
                    return
                self.parent[p_root] = q_root

```

```

        self.count -= 1

    row = len(grid)
    if row == 0:
        return 0
    col = len(grid[0])

    def get_index(x, y):
        return x * col + y

    spaces = 0
    uf = UnionFind(row * col)

    for i in range(row):
        for j in range(col):
            if grid[i][j] == '0':
                spaces += 1
            else:
                if i + 1 < row and grid[i + 1][j] == '1':
                    uf.union(get_index(i, j), get_index(i + 1, j))
                if j + 1 < col and grid[i][j + 1] == '1':
                    uf.union(get_index(i, j), get_index(i, j + 1))

    return uf.get_count() - spaces

```

LeetCode 547、省份数量

一、题目描述

有 n 个城市，其中一些彼此相连，另一些没有相连。如果城市 a 与城市 b 直接相连，且城市 b 与城市 c 直接相连，那么城市 a 与城市 c 间接相连。

省份 是一组直接或间接相连的城市，组内不含其他没有相连的城市。

给你一个 $n \times n$ 的矩阵 `isConnected`，其中 `isConnected[i][j] = 1` 表示第 i 个城市和第 j 个城市直接相连，而 `isConnected[i][j] = 0` 表示二者不直接相连。

返回矩阵中 **省份** 的数量。

示例 1:

```

输入: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
输出: 2

```

示例 2:

img

输入: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
输出: 3

提示:

- $1 \leq n \leq 200$
- $n == \text{isConnected.length}$
- $n == \text{isConnected}[i].\text{length}$
- $\text{isConnected}[i][j]$ 为 1 或 0
- $\text{isConnected}[i][i] == 1$
- $\text{isConnected}[i][j] == \text{isConnected}[j][i]$

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 省份数量 (LeetCode 547) : https://leetcode.cn/problems/number-of-provinces/
class Solution {
    public int findCircleNum(int[][] isConnected) {

        // 城市的数量
        int n = isConnected.length;

        // 初始化并查集
        UnionFind uf = new UnionFind(n);

        // 遍历每个顶点, 将当前顶点与其邻接点进行合并
        for (int i = 0; i < n; i++) {
            // 由于这个二维数组是镜像对称的
            // 所以 j 可以从 i + 1 开始
            for (int j = i + 1; j < n; j++) {
                // 如果第 i 个城市和第 j 个城市直接相连
                if (isConnected[i][j] == 1) {
                    // 连通起来
                    uf.union(i, j);
                }
            }
        }

        // 返回最终合并后的集合的数量
        return uf.count;
    }
}
```

```

    }
}

// 并查集
class UnionFind {

    // 使用一维数组用来记录每个城市的父城市
    // 如果自己就是根城市，那么 roots[i] = i，即自己指向自己
    // 如果自己不是根城市，则 roots[i] = roots id
    int[] roots;

    // 省份数量
    int count;

    public UnionFind(int n) {
        // 使用一维数组用来记录每个城市的父城市
        roots = new int[n];
        for (int i = 0; i < n ; i++) {
            // 默认每个城市的父城市是自己本身
            roots[i] = i;
        }
        // 所以一开始有 n 个省份
        count = n ;
    }

    // 寻找 i 的父城市
    public int find(int i) {

        // 如果发现当前城市的父城市是自己本身
        if (i == roots[i]) {
            // 返回自己就行
            return i;
        }

        // 否则不断的去向上寻找
        // 这里采取了路径压缩的方法
        // roots[i] 记录的是 i 这个城市的父城市
        // find(roots[i]) 表示去寻找 roots[i] 的父城市
        // 遍历父城市的同时，把父城市的父城市也改成根城市
        roots[i] = find(roots[i]);

        // 返回结果
        return roots[i];
    }

    // 连通操作
    public void union(int p, int q) {
        // 寻找 p 的父城市
        int pRoot = find(p);
    }
}

```

```

// 寻找 q 的父城市
int qRoot = find(q);

// 如果两者的父城市不同
// 那么需要采取操作把 p 和 q 所在的两堆城市合到一起
if (pRoot != qRoot) {
    // 也就是把其中一个的最上面的城市挂过来就行
    roots[pRoot] = qRoot;
    // 与此同时，两堆城市变成了一堆城市
    count--;
}
}
}

```

2、**C++** 代码

3、Python 代码

```

class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:

        class UnionFind:
            def __init__(self, n):
                self.count = n
                self.parent = [i for i in range(n)]

            def get_count(self):
                return self.count

            def find(self, p):
                while p != self.parent[p]:
                    self.parent[p] = self.parent[self.parent[p]]
                    p = self.parent[p]
                return p

            def union(self, p, q):
                p_root = self.find(p)
                q_root = self.find(q)
                if p_root == q_root:
                    return
                self.parent[p_root] = q_root
                self.count -= 1

        n = len(isConnected)

```

```
uf = UnionFind(n)

for i in range(n):
    for j in range(n):
        if isConnected[i][j] == 1:

            uf.union(i , j)

return uf.get_count()
```

第五十二天

LeetCode 138 、复杂链表的复制

一、题目描述

请实现 `copyRandomList` 函数，复制一个复杂链表。在复杂链表中，每个节点除了有一个 `next` 指针指向下一个节点，还有一个 `random` 指针指向链表中的任意节点或者 `null`。

示例 1：

复杂链表的复制

```
输入：head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
输出：[[7,null],[13,0],[11,4],[10,2],[1,0]]
```

提示：

- `-10000 <= Node.val <= 10000`
- `Node.random` 为空（null）或指向链表中的节点。
- 节点数目不超过 1000 。

二、题目解析

对于链表中的每个节点来说，它都有三个特征：

- 值为 `val`
- 一个指向下一个节点的指针 `next`
- 一个指向随机节点的指针 `random`

要想复制这样一个复杂**链表**必须要考虑到这三个特征。

如果没有 `random` 指针的话，那就是普通的链表，只需要遍历链表，然后每轮创建新节点，同时赋值 `val` 和调整前驱指针指向当前节点就行。

这题出现了 `random` 指针，由于它可以指向 `null`、前面的节点或者后面的节点，无法做到在一次遍历的过程中就确定下来，因为如果是指向后面的节点，但后面的节点还没有创建生成，无法确定。

所以，我们需要在一开始把所有的节点都创建出来，避免 `random` 找不到指向，同时观察上图，每个节点都通过 `random` 对应着一个新的节点，这种一一对应的关系，符合哈希表的特征。

此时的哈希表以原**链表的节点作为键，新创建的节点作为值**。

原链表（Key）中的每个节点都有 `next` 和 `random` 指针，而新链表（Value）没有 `next` 和 `random` 指针。

需要通过第二次的遍历过程进行指针指向的调整。

在第二次遍历过程中，以原链表中的节点作为键，查找当前原节点的指针指向，然后调整新节点的指针指向。

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class Solution {
    public Node copyRandomList(Node head) {
        // 边界判断，一般链表的题目都需要判断头节点是否为空
        if(head == null ) return null;
        // 从链表的头节点开始遍历
        Node cur = head;
        // 使用一一对应的哈希表结构 Map 存放已经创建的节点
        Map<Node,Node> map = new HashMap<>();

        // 遍历原链表
        while( cur != null ) {
            // 以原链表的节点为 key，构建一个 Map
            // Map 的 value 为一个新链表中的节点
            // 新节点的值 val 和原链表的值 val 一样
            // 但原链表中的每个节点都有 next 和 random 指针，而 Map 中的 value 没有 next 和
            random 指针
            // map.put(Key,value)
            Node newNode = new Node(cur.val);

            map.put(cur,newNode);

            // 查看下一个节点的情况
            cur = cur.next;
        }

        // 再次从链表的头节点开始遍历
        cur = head;

        // 遍历原链表
        while( cur != null ) {
```

```

// 原链表节点 ---- 新链表节点
// key      ----- value
// cur      ----- map.get(cur)

// 0、在字典中找到一个 cur 为 key 对应的那个 value 值
Node valueCur = map.get(cur);
// 接下来，需要去寻找 valueCur 的 next 节点和 random 节点

// 寻找 valueCur 的 next 节点
// 1、获取当前节点 cur 在原链表中的 next 指针指向的节点
Node keyNextNode = cur.next;
// 2、在字典中找到以 keyNextNode 为 key 对应的那个 value 值
Node valueNextNode = map.get(keyNextNode);
// 3、那么新链表中的这个节点的 next 指针就是 valueNextNode
valueCur.next = valueNextNode;

// 寻找 valueCur 的 random 节点
// 1、获取当前节点 cur 在原链表中的 random 指针指向的节点
Node keyRandomNode = cur.random;
// 2、在字典中找到以 valueRandomNode 为 key 对应的那个 value 值
Node valueRandomNode = map.get(keyRandomNode);
// 4、那么新链表中的这个节点的 next 指针就是 valueNextNode
valueCur.random = valueRandomNode;

//遍历下去，查看下一个节点
cur = cur.next;

}
// 原链表节点 ---- 新链表节点
// key      ----- value
// cur      ----- map.get(cur)
// head     ----- map.get(head)
return map.get(head);
}
}

```

LeetCode 445、两数相加II

一、题目描述

给你两个 非空 链表来代表两个非负整数。数字最高位位于链表开始位置。它们的每个节点只存储一位数字。将这两数相加会返回一个新的链表。

你可以假设除了数字 0 之外，这两个数字都不会以零开头。

二、题目解析

三、参考代码

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

        // 构建两个栈，用来储存两个链表中的元素

        // stack1 用来存储链表 l1 中的元素
        Stack<Integer> stack1 = new Stack<Integer>();

        // stack2 用来存储链表 l2 中的元素
        Stack<Integer> stack2 = new Stack<Integer>();

        // 依次把链表 l1 中的元素加入到 stack1 中
        while (l1 != null) {
            // 把当前节点加入到 stack1 中
            stack1.push(l1.val);
            // 继续向后遍历
            l1 = l1.next;
        }

        // 依次把链表 l2 中的元素加入到 stack2 中
        while (l2 != null) {
            // 把当前节点加入到 stack2 中
            stack2.push(l2.val);
            // 继续向后遍历
            l2 = l2.next;
        }

        // 设置一个进位，初始化为 0
        // 两个个位数相加，进位只能是 1 或者 0
        // 比如 7 + 8 = 15，进位是 1
        // 比如 2 + 3 = 6，没有进位，或者说进位是 0
        int carryBit = 0;

        // 构建一个链表用来存放 l1 和 l2 两个链表相加的结果
        // 其中 dummy 这个节点为虚拟头结点
        ListNode dummy = new ListNode(-1);

        // 只要任意一个栈或者 carryBit 有值，那么就要求和
        while (!stack1.isEmpty() || !stack2.isEmpty() || carryBit != 0) {

            // 三目运算符，如果栈为空，就用 0 来占位，否则使用栈顶元素值

            // 获取 stack1 的栈顶元素值
            int x = stack1.isEmpty() ? 0 : stack1.pop();
```

```

// 获取 stack2 的栈顶元素值
int y = stack2.isEmpty() ? 0 : stack2.pop();

// 每一位计算的同时需要考虑上一位的进位情况
int sum = x + y + carryBit;

// 获取当前计算结果的十位数
// 比如 7 + 8 = 15
// 那么 sum / 10 = 1, 进位为 1
carryBit = sum / 10;

// 获取当前计算结果的个位数
// 比如 7 + 8 = 15
// 那么 sum % 10 = 5
int diget = sum % 10;

// 构建一个节点用来存放这个个位数
ListNode digetNode = new ListNode(diget);

// 把这个节点插入到结果链表中
// 注意是插入, 越后面计算的结果越在链表的前面

// 一开始, dummy 后面没有其它节点, 当发生求和计算时
if(dummy.next == null){
    // 第一个计算结果直接加入到 dummy 后面
    dummy.next = digetNode;
}else{

    // 新的节点的 next 指针为之前的虚拟头节点 dummy 后面的那个节点
    digetNode.next = dummy.next;

    // 更新 dummy 的 next 指针指向的节点
    dummy.next = digetNode;
}
}

// 最后返回结果链表的头节点就行, 即虚拟头结点的下一个节点
return dummy.next;
}
}

```

C++

```

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        stack<int> s1, s2;
        while (l1) {
            s1.push(l1->val);

```



```

        l1 = l1 -> next;
    }
    while (l2) {
        s2.push(l2 -> val);
        l2 = l2 -> next;
    }
    int carry = 0;
    ListNode* ans = nullptr;
    while (!s1.empty() or !s2.empty() or carry != 0) {
        int a = s1.empty() ? 0 : s1.top();
        int b = s2.empty() ? 0 : s2.top();
        if (!s1.empty()) s1.pop();
        if (!s2.empty()) s2.pop();
        int cur = a + b + carry;
        carry = cur / 10;
        cur %= 10;
        auto curnode = new ListNode(cur);
        curnode -> next = ans;
        ans = curnode;
    }
    return ans;
}
};

```

Python

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        s1, s2 = [], []
        while l1:
            s1.append(l1.val)
            l1 = l1.next
        while l2:
            s2.append(l2.val)
            l2 = l2.next
        ans = None
        carry = 0
        while s1 or s2 or carry != 0:
            a = 0 if not s1 else s1.pop()
            b = 0 if not s2 else s2.pop()
            cur = a + b + carry
            carry = cur // 10
            cur %= 10
            curnode = ListNode(cur)
            curnode.next = ans
            ans = curnode
        return ans

```

四、复杂度分析

时间复杂度： $O(\max(m,n))$ ，其中 m 和 n 分别为两个链表的长度。我们需要遍历两个链表的全部位置，而处理每个位置只需要 $O(1)$ 的时间。

空间复杂度： $O(m + n)$ ，其中 m 和 n 分别为两个链表的长度。空间复杂度主要取决于我们把链表内容放入栈中所用的空间。

LeetCode 811、子域名访问次数

一、题目描述

网站域名 `"discuss.leetcode.com"` 由多个子域名组成。顶级域名为 `"com"`，二级域名为 `"leetcode.com"`，最低一级为 `"discuss.leetcode.com"`。当访问域名 `"discuss.leetcode.com"` 时，同时也会隐式访问其父域名 `"leetcode.com"` 以及 `"com"`。

计数配对域名 是遵循 `"rep d1.d2.d3"` 或 `"rep d1.d2"` 格式的一个域名表示，其中 `rep` 表示访问域名的次数，`d1.d2.d3` 为域名本身。

- 例如，`"9001 discuss.leetcode.com"` 就是一个 **计数配对域名**，表示 `discuss.leetcode.com` 被访问了 `9001` 次。

给你一个 **计数配对域名** 组成的数组 `cpdomains`，解析得到输入中每个子域名对应的 **计数配对域名**，并以数组形式返回。可以按 **任意顺序** 返回答案。

示例 1：

```
输入: cpdomains = ["9001 discuss.leetcode.com"]
输出: ["9001 leetcode.com","9001 discuss.leetcode.com","9001 com"]
解释: 例子中仅包含一个网站域名: "discuss.leetcode.com"。
按照前文描述，子域名 "leetcode.com" 和 "com" 都会被访问，所以它们都被访问了 9001 次。
```

示例 2：

```
输入: cpdomains = ["900 google.mail.com", "50 yahoo.com", "1 intel.mail.com", "5 wiki.org"]
输出: ["901 mail.com","50 yahoo.com","900 google.mail.com","5 wiki.org","5 org","1 intel.mail.com","951 com"]
解释: 按照前文描述，会访问 "google.mail.com" 900 次，"yahoo.com" 50 次，"intel.mail.com" 1 次，"wiki.org" 5 次。
而对于父域名，会访问 "mail.com" 900 + 1 = 901 次，"com" 900 + 50 + 1 = 951 次，和 "org" 5 次。
```

提示：

- `1 <= cpdomain.length <= 100`
- `1 <= cpdomain[i].length <= 100`
- `cpdomain[i]` 会遵循 `"repi d1i.d2i.d3i"` 或 `"repi d1i.d2i"` 格式
- `repi` 是范围 `[1, 10^4]` 内的一个整数

- d1i、d2i 和 d3i 由小写英文字母组成

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 子域名访问次数 (LeetCode 811) : https://leetcode.cn/problems/subdomain-visit-count/
class Solution {
    public List<String> subdomainVisits(String[] cpdomains) {

        // 使用哈希表记录每个子域名的计数
        Map<String, Integer> map = new HashMap<>();

        // 遍历数组
        for (String s : cpdomains) {

            // 获取当前字符串的长度
            int n = s.length();

            // 每个字符串都是由数字 + 域名组成
            // 先去获取这个字符串中的数字
            // 从 0 开始，向后扫描到空格位置
            int idx = 0;

            // 从前向后扫描到空格位置
            while (idx < n && s.charAt(idx) != ' ') idx++;

            // 截取出数字来
            // tips: substring() 方法返回字符串的子字符串
            // beginIndex -- 起始索引 (包括)，索引从 0 开始
            // endIndex -- 结束索引 (不包括)
            int cnt = Integer.parseInt(s.substring(0, idx));

            // 从后往前处理域名部分，直到处理完毕
            int start = idx + 1;

            // 从后往前处理域名部分
            idx = n - 1;

            // 直到处理完毕
            while (idx >= start) {

                // 每个域名由多个子域名组成
```

```

        // 通过 . 来截取
        while (idx >= start && s.charAt(idx) != '.') idx--;

        // 获取当前子域名
        String cur = s.substring(idx + 1);

        // 更新这个子域名的计数
        map.put(cur, map.getOrDefault(cur, 0) + cnt);

        // idx 继续向前移动
        idx--;
    }
}

// 从哈希表中构造出数组答案来
List<String> ans = new ArrayList<>();

// keySet() 方法返回映射中所有 key 组成的 Set 视图
for (String key : map.keySet()) {

    // key 是域名, map.get(key) 获取 value
    ans.add(map.get(key) + " " + key);

}

// 返回结果
return ans;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    vector<string> subdomainVisits(vector<string>& cpdomains) {

        // 使用哈希表记录每个子域名的计数
        unordered_map<string, int> map;

        // 遍历数组
        for (string s : cpdomains) {

            // 获取当前字符串的长度
            int n = s.length();

            // 每个字符串都是由数字 + 域名组成
            // 先去获取这个字符串中的数字
            // 从 0 开始, 向后扫描到空格位置
            int idx = 0;

```

```

// 从前向后扫描到空格位置
while (idx < n && s[idx] != ' ') {
    idx++;
}

// 截取出数字来
// tips: substr() 方法返回字符串的子字符串
// pos -- 起始索引, 索引从 0 开始
// len -- 子串长度
int cnt = stoi(s.substr(0, idx));

// 从后往前处理域名部分, 直到处理完毕
int start = idx + 1;

// 从后往前处理域名部分
idx = n - 1;

// 直到处理完毕
while (idx >= start) {

    // 每个域名由多个子域名组成
    // 通过 . 来截取
    while (idx >= start && s[idx] != '.') {
        idx--;
    }

    // 获取当前子域名
    string cur = s.substr(idx + 1);

    // 更新这个子域名的计数
    map[cur] += cnt;

    // idx 继续向前移动
    idx--;
}
}

// 从哈希表中构造出数组答案来
vector<string> ans;

// keySet() 方法返回映射中所有 key 组成的 Set 视图
for (auto& [key, value] : map) {
    // key 是域名, value 获取计数
    ans.push_back(to_string(value) + " " + key);
}

// 返回结果
return ans;

```

```
}  
};
```

3、Python 代码

```
class Solution:  
    def subdomainvisits(self, cpdomains: List[str]) -> List[str]:  
  
        # 使用哈希表记录每个子域名的计数  
        map = {}  
  
        # 遍历数组  
        for s in cpdomains:  
  
            # 获取当前字符串的长度  
            n = len(s)  
  
            # 每个字符串都是由数字 + 域名组成  
            # 先去获取这个字符串中的数字  
            # 从 0 开始, 向后扫描到空格位置  
            idx = 0  
  
            # 从前向后扫描到空格位置  
            while idx < n and s[idx] != ' ':  
                idx += 1  
  
            # 截取出数字来  
            cnt = int(s[:idx])  
  
            # 从后往前处理域名部分, 直到处理完毕  
            start = idx + 1  
            idx = n - 1  
  
            # 直到处理完毕  
            while idx >= start:  
  
                # 每个域名由多个子域名组成  
                # 通过 . 来截取  
                while idx >= start and s[idx] != '.':  
                    idx -= 1  
  
                # 获取当前子域名  
                cur = s[idx + 1:]  
  
                # 更新这个子域名的计数  
                map[cur] = map.get(cur, 0) + cnt  
  
                # idx 继续向前移动  
                idx -= 1
```

```
# 从哈希表中构造出数组答案来
ans = []

for key in map:

    # key 是域名, map[key] 获取 value
    ans.append(str(map[key]) + " " + key)

# 返回结果
return ans
```

LeetCode 874、模拟行走机器人

一、题目描述

机器人在一个无限大小的 XY 网格平面上行走，从点 $(0, 0)$ 处开始出发，面向北方。该机器人可以接收以下三种类型的命令 `commands`：

- `-2`：向左转 90 度
- `-1`：向右转 90 度
- `1 <= x <= 9`：向前移动 `x` 个单位长度

在网格上有一些格子被视为障碍物 `obstacles`。第 `i` 个障碍物位于网格点 `obstacles[i] = (xi, yi)`。

机器人无法走到障碍物上，它将会停留在障碍物的前一个网格方块上，但仍然可以继续尝试进行该路线的其余部分。

返回从原点到机器人所有经过的路径点（坐标为整数）的最大欧式距离的平方。（即，如果距离为 `5`，则返回 `25`）

注意：

- 北表示 `+Y` 方向。
- 东表示 `+X` 方向。
- 南表示 `-Y` 方向。
- 西表示 `-X` 方向。

示例 1：

```
输入: commands = [4,-1,3], obstacles = []
输出: 25
解释:
机器人开始位于 (0, 0):
1. 向北移动 4 个单位，到达 (0, 4)
2. 右转
3. 向东移动 3 个单位，到达 (3, 4)
距离原点最远的是 (3, 4)，距离为  $3^2 + 4^2 = 25$ 
```

示例 2:

输入: `commands = [4,-1,4,-2,4]`, `obstacles = [[2,4]]`

输出: 65

解释: 机器人开始位于 (0, 0):

1. 向北移动 4 个单位, 到达 (0, 4)
 2. 右转
 3. 向东移动 1 个单位, 然后被位于 (2, 4) 的障碍物阻挡, 机器人停在 (1, 4)
 4. 左转
 5. 向北走 4 个单位, 到达 (1, 8)
- 距离原点最远的是 (1, 8) , 距离为 $1^2 + 8^2 = 65$

提示:

- `1 <= commands.length <= 10^4`
- `commands[i]` is one of the values in the list `[-2,-1,1,2,3,4,5,6,7,8,9]`.
- `0 <= obstacles.length <= 100^4`
- `-3 * 100^4 <= xi, yi <= 3 * 100^4`
- 答案保证小于 `20^31`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 模拟行走机器人 (LeetCode 874) : https://leetcode.cn/problems/walking-robot-simulation/submissions/
class Solution {
    public int robotSim(int[] commands, int[][] obstacles) {

        // 初始化结果
        int result = 0;

        // 从原点 ( 0 , 0 ) 位置开始出发
        int x = 0, y = 0;

        // 机器人前进的方向
        // 初始方向: 正北
        // 0 表示 向北
        // 1 表示 向东
        // 2 表示 向南
        // 3 表示 向西
        int direction = 0;
```


点

```
// 定义四个方向
int[][] direxy = new int[][]{{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

// 障碍物有多个，所以需要有一个障碍物坐标点集合
// 使用 集合 Set 存储障碍物的坐标，用来检查下一步是否受阻
// 机器人每试图走一个位置，就用此位置与障碍物集合列表里的坐标进行比较，看是否刚好是障碍物坐标

HashSet<String> obstaclesSet = new HashSet<String>();

// 存储障碍物坐标点
for (int[] obs : obstacles) {

    // 以坐标的形式存储
    obstaclesSet.add(obs[0] + "," + obs[1]);

}

// 机器人开始行走
for (int com : commands) {

    // -2: 向左转 90 度
    if (com == -2) {
        // 1、当前方向为正北方向，即 direction == 0，向左转 90 度
        // 来到 向西 方向，即 direction 需要更新为 3
        // 2、当前方向【不是】正北方向，即 direction != 0，向左转 90 度
        // 西 -> 南，即 3 -> 2
        // 南 -> 东，即 2 -> 1
        // 东 -> 北，即 1 -> 0
        // 即 direction 需要更新为 direction - 1
        direction = (direction == 0) ? 3 : direction - 1;

    // -1: 向右转 90 度
    } else if (com == -1) {
        // 1、当前方向为正西方向，即 direction == 3，向右转 90 度
        // 来到 向北 方向，即 direction 需要更新为 0
        // 2、当前方向【不是】正西方向，即 direction != 3，向右转 90 度
        // 北 -> 东，即 0 -> 1
        // 东 -> 南，即 1 -> 2
        // 南 -> 西，即 2 -> 3
        // 即 direction 需要更新为 direction + 1
        direction = (direction == 3) ? 0 : direction + 1;

    // 1 <= x <= 9 : 向前移动 x 个单位长度
    } else {
        // 没有遇到障碍物，一直往前走 com 个单位长度
        while (com-- > 0 && !obstaclesSet.contains((x + direxy[direction][0]) +
            "," + (y + direxy[direction][1]))) {
            x += direxy[direction][0];
```

```

        y += direxy[direction][1];
    }

    // 机器人停下来后开始计算最大欧式距离的平方
    // 更新结果
    result = Math.max(result, x * x + y * y);
}
}
return result;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int robotSim(vector<int>& commands, vector<vector<int>>& obstacles) {

        // 初始化结果
        int result = 0;

        // 从原点 ( 0 , 0 ) 位置开始出发
        int x = 0;

        int y = 0;

        // 机器人前进的方向
        // 初始方向: 正北
        // 0 表示 向北
        // 1 表示 向东
        // 2 表示 向南
        // 3 表示 向西
        int direction = 0;

        // 定义四个方向

        int dx[4] = {0, 1, 0, -1};

        int dy[4] = {1, 0, -1, 0};

        // 障碍物有多个, 所以需要有一个障碍物坐标点集合
        // 使用 集合 Set 存储障碍物的坐标, 用来检查下一步是否受阻
        // 机器人每试图走一个位置, 就用此位置与障碍物集合列表里的坐标进行比较, 看是否刚好是障碍物坐标
        unordered_set<pair<int, int>, pair_hash> obstacleSet;
        for (vector<int> obstacle: obstacles)
            obstacleSet.insert(make_pair(obstacle[0], obstacle[1]));

        // 机器人开始行走
    }
}

```

点

```

for (int com : commands) {

    // -2: 向左转 90 度
    if (com == -2) {
        // 1、当前方向为正北方向, 即 direction == 0 , 向左转 90 度
        // 来到 向西 方向, 即 direction 需要更新为 3
        // 2、当前方向【不是】正北方向, 即 direction != 0 , 向左转 90 度
        // 西 -> 南, 即 3 -> 2
        // 南 -> 东, 即 2 -> 1
        // 东 -> 北, 即 1 -> 0
        // 即 direction 需要更新为 direction - 1
        direction = (direction == 0) ? 3 : direction - 1;

    // -1: 向右转 90 度
    } else if (com == -1) {
        // 1、当前方向为正西方向, 即 direction == 3 , 向右转 90 度
        // 来到 向北 方向, 即 direction 需要更新为 0
        // 2、当前方向【不是】正西方向, 即 direction != 3 , 向右转 90 度
        // 北 -> 东, 即 0 -> 1
        // 东 -> 南, 即 1 -> 2
        // 南 -> 西, 即 2 -> 3
        // 即 direction 需要更新为 direction + 1
        direction = (direction == 3) ? 0 : direction + 1;

    // 1 <= x <= 9 : 向前移动 x 个单位长度
    } else {
        // 没有遇到障碍物, 一直往前走 com 个单位长度
        for (int k = 0; k < com; ++k) {
            int nx = x + dx[direction];
            int ny = y + dy[direction];
            if (obstacleSet.find(make_pair(nx, ny)) == obstacleSet.end()) {
                x = nx;
                y = ny;
                result = max(result, x*x + y*y);
            }
        }
    }
}

return result;

}

private:
    struct pair_hash
    {
        template <class T1, class T2>
        std::size_t operator () (std::pair<T1, T2> const &pair) const
        {
            std::size_t h1 = std::hash<T1>()(pair.first);

```

```

        std::size_t h2 = std::hash<T2>()(pair.second);

        return h1 ^ h2;
    }
};
};

```

3、Python 代码

```

class solution:
    def robotSim(self, commands: List[int], obstacles: List[List[int]]) -> int:
        # 初始化结果
        result = 0

        # 从原点 ( 0 , 0 ) 位置开始出发
        x = 0

        y = 0

        # 机器人前进的方向
        # 初始方向: 正北
        # 0 表示 向北
        # 1 表示 向东
        # 2 表示 向南
        # 3 表示 向西
        direction = 0

        # 定义四个方向
        dx = [0, 1, 0, -1]

        dy = [1, 0, -1, 0]

        # 障碍物有多个, 所以需要有一个障碍物坐标点集合
        # 使用 集合 Set 存储障碍物的坐标, 用来检查下一步是否受阻
        # 机器人每试图走一个位置, 就用此位置与障碍物集合列表里的坐标进行比较, 看是否刚好是障碍物坐标点
        obstacleSet = set(map(tuple, obstacles))

        # 机器人开始行走
        for com in commands :

            # -2: 向左转 90 度
            if com == -2 :
                # 1、当前方向为正北方向, 即 direction == 0 , 向左转 90 度
                # 来到 向西 方向, 即 direction 需要更新为 3
                # 2、当前方向【不是】正北方向, 即 direction != 0 , 向左转 90 度
                # 西 -> 南, 即 3 -> 2
                # 南 -> 东, 即 2 -> 1
                # 东 -> 北, 即 1 -> 0
                # 即 direction 需要更新为 direction - 1

```

```

        if direction == 0:
            direction = 3
        else:
            direction = direction - 1
        # direction = (direction == 0) ? 3 : direction - 1

# -1: 向右转 90 度
elif com == -1 :
    # 1、当前方向为正西方向，即 direction == 3 ，向右转 90 度
    # 来到 向北 方向，即 direction 需要更新为 0
    # 2、当前方向【不是】正西方向，即 direction != 3 ，向右转 90 度
    # 北 -> 东，即 0 -> 1
    # 东 -> 南，即 1 -> 2
    # 南 -> 西，即 2 -> 3
    # 即 direction 需要更新为 direction + 1
    if direction == 3:
        direction = 0
    else:
        direction = direction + 1
    # direction = (direction == 3) ? 0 : direction + 1

# 1 <= x <= 9 : 向前移动 x 个单位长度
else :
    # 没有遇到障碍物，一直往前走 com 个单位长度
    for k in range(com):
        if (x+dx[direction], y+dy[direction]) not in obstacleSet:
            x += dx[direction]
            y += dy[direction]
            result = max(result, x*x + y*y)

return result

```

第五十三天

LeetCode 1109、航班预订统计

一、题目描述

这里有 `n` 个航班，它们分别从 `1` 到 `n` 进行编号。

有一份航班预订表 `bookings`，表中第 `i` 条预订记录 `bookings[i] = [firsti, lasti, seatsi]` 意味着在从 `firsti` 到 `lasti`（包含 `firsti` 和 `lasti`）的每个航班上预订了 `seatsi` 个座位。

请你返回一个长度为 `n` 的数组 `answer`，里面的元素是每个航班预定的座位总数。

示例 1:

输入: bookings = [[1,2,10],[2,3,20],[2,5,25]], n = 5 输出: [10,55,45,25,25] 解释: 航班编号 1 2 3 4 5 预订记录 1 : 10 10 预订记录 2 : 20 20 预订记录 3 : 25 25 25 25 总座位数: 10 55 45 25 25 因此, answer = [10,55,45,25,25]

示例 2:

输入: bookings = [[1,2,10],[2,2,15]], n = 2 输出: [10,25] 解释: 航班编号 1 2 预订记录 1 : 10 10 预订记录 2 : 15 总座位数: 10 25 因此, answer = [10,25]

提示:

- `1 <= n <= 2 * 10^4`
- `1 <= bookings.length <= 2 * 10^4`
- `bookings[i].length == 3`
- `1 <= firsti <= lasti <= n`
- `1 <= seatsi <= 10^4`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 微信: wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 航班预订统计 (LeetCode 1109) : https://leetcode.cn/problems/corporate-flight-bookings/
class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {

        // 初始化结果数组
        // 填充 counter 的过程可以理解为乘客乘坐公交车
        // 上车就表示该时刻 t1 即新区间加入乘客 +1
        // 下车就表示区间结束, 即下一个时刻 t2 + 1 乘客 -1
        int[] counter = new int[n];

        // 遍历 bookings
        for (int[] booking : bookings) {

            // 因为航班编号为 1 - n , 因此左偏移一位才是 counter 索引
            counter[booking[0] - 1] += booking[2];

            // 在右侧区间有效的情况下, 进行修改
            // 这里原本有 +1 , 向左偏移一位就是原本的, 要判断区间是否合法
            if (booking[1] < n) {
                // counter[booking[1]] 表示 booking[2] 这个数量有效的最右端位置
                counter[booking[1]] -= booking[2];
            }
        }
    }
}
```

```
}

// 求出差分数组的前缀和即可得到目标数组
// 该站人数 = 上一站人数 + 该站变化量
// 注意最初的一站就是 0 + counter[0] = counter[0]
for (int i = 1; i < n; i++) {

    counter[i] += counter[i - 1];

}

// 返回结果
return counter;
}
}
```

LeetCode 376 、摆动序列

一、题目描述

如果连续数字之间的差严格地在正数和负数之间交替，则数字序列称为 **摆动序列** 。第一个差（如果存在的话）可能是正数或负数。仅有一个元素或者含两个不等元素的序列也视作摆动序列。

- 例如， `[1, 7, 4, 9, 2, 5]` 是一个 **摆动序列** ，因为差值 `(6, -3, 5, -7, 3)` 是正负交替出现的。
- 相反， `[1, 4, 7, 2, 5]` 和 `[1, 7, 4, 5, 5]` 不是摆动序列，第一个序列是因为它的前两个差值都是正数，第二个序列是因为它的最后一个差值为零。

子序列 可以通过从原始序列中删除一些（也可以不删除）元素来获得，剩下的元素保持其原始顺序。

给你一个整数数组 `nums` ，返回 `nums` 中作为 **摆动序列** 的 **最长子序列的长度** 。

示例 1：

输入： `nums = [1,7,4,9,2,5]`

输出： 6

解释： 整个序列均为摆动序列，各元素之间的差值为 `(6, -3, 5, -7, 3)` 。

示例 2：

输入： `nums = [1,17,5,10,13,15,10,5,16,8]`

输出： 7

解释： 这个序列包含几个长度为 7 摆动序列。

其中一个 `[1, 17, 10, 13, 10, 16, 8]` ，各元素之间的差值为 `(16, -7, 3, -3, 6, -8)` 。

示例 3：

输入： `nums = [1,2,3,4,5,6,7,8,9]`

输出： 2

提示：

- `1 <= nums.length <= 1000`
- `0 <= nums[i] <= 1000`

进阶：你能否用 `O(n)` 时间复杂度完成此题？

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 摆动序列(376):https://leetcode-cn.com/problems/wiggle-subsequence/
class Solution {
    public int wiggleMaxLength(int[] nums) {

        // 每个元素都有三种状态

        // 1、每个元素的初始状态
        int beginState = 0;

        // 2、如果当前元素的值大于它之前元素的值
        // 比如 nums[i] > nums[i-1]
        // 那么说明当前元素处于上升阶段，状态设置为 up
        int upState = 1;

        // 3、如果当前元素的值小于它之前元素的值
        // 比如 nums[i] < nums[i-1]
        // 那么说明当前元素处于下降阶段，状态设置为 down
        int downState = 2;

        // 如果 nums 长度小于 2
        if(nums.length < 2){
            // 由于仅有一个元素或者含两个不等元素的序列也视作摆动序列
            // 直接返回数组的长度
            return nums.length;
        }

        // 以第一个元素作为初始的摇摆序列，此时，长度为 1
        int length = 1;

        // 一开始，状态为 begin
        int state = beginState;
```



```

// 接下来, 开始遍历 nums 中的所有元素
for( int i = 1 ; i < nums.length ; i++ ){
    // 每个元素都有三种状态, 在这三种状态下去判断这个元素应该怎么操作

    // 1、如果是在 begin 状态
    if(state == beginState){
        // 那么比较 nums[i] 和 nums[i-1] 的大小

        // 此时的元素值为 nums[i], 它前面的元素值为 nums[i-1]
        // 如果 nums[i] > nums[i-1], 代表着现在处于上升过程
        if(nums[i] > nums[i-1]){
            // 状态修改为 up
            state = upState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;

            // 此时的元素值为 nums[i], 它前面的元素值为 nums[i-1]
            // 如果 nums[i] < nums[i-1], 代表着现在处于下降过程
        }else if(nums[i] < nums[i-1]){
            // 状态修改为 down
            state = downState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;
        }else{

            // do nothing

        }

    }

    // 2、如果是在 up 状态
    // 说明 nums[i] 的前面两个元素 nums[i-2] < nums[i-1], 正在处于上升过程
    }else if(state == upState){

        // 只有此时 nums[i] < nums[i-1]
        // 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波峰 ^
        if(nums[i] < nums[i-1]){

            // 此时, 由于 nums[i] < nums[i-1], 开始处于下降状态了
            // 状态修改为 down
            state = downState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;
        }else{

            // do nothing

        }

    }

    // 3、如果是在 up 状态
    // 说明 nums[i] 的前面两个元素 nums[i-2] > nums[i-1], 正在处于下降过程
    }else if(state == downState){

```

```

        // 只有此时 nums[i] > nums[i-1]
        // 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波谷 v
        if(nums[i] > nums[i-1]){

            // 此时, 由于 nums[i] > nums[i-1], 开始处于上升状态了
            // 状态修改为 up
            state = upState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;
        }else{
            // do nothing
        }
    }
}

// 返回结果 length
// 不需要返回具体序列
return length;
}
}

```

2、**C++** 代码

```

// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 摆动序列(376):https://leetcode-cn.com/problems/wiggle-subsequence/
class Solution {
public:
    int wiggleMaxLength(vector<int>& nums) {
        // 每个元素都有三种状态

        // 1、每个元素的初始状态
        int beginState = 0;

        // 2、如果当前元素的值大于它之前元素的值
        // 比如 nums[i] > nums[i-1]
        // 那么说明当前元素处于上升阶段, 状态设置为 up
        int upState = 1;

        // 3、如果当前元素的值小于它之前元素的值
        // 比如 nums[i] < nums[i-1]
        // 那么说明当前元素处于下降阶段, 状态设置为 down
        int downState = 2;
    }
}

```

```

// 如果 nums 长度小于 2
if(nums.size() < 2){
    // 由于仅有一个元素或者含两个不等元素的序列也视作摆动序列
    // 直接返回数组的长度
    return nums.size();
}

// 以第一个元素作为初始的摇摆序列, 此时, 长度为 1
int length = 1;

// 一开始, 状态为 begin
int state = beginState;

// 接下来, 开始遍历 nums 中的所有元素
for( int i = 1 ; i < nums.size() ; i++ ){
    // 每个元素都有三种状态, 在这三种状态下去判断这个元素应该怎么操作

    // 1、如果是在 begin 状态
    if(state == beginState){
        // 那么比较 nums[i] 和 nums[i-1] 的大小

        // 此时的元素值为 nums[i], 它前面的元素值为 nums[i-1]
        // 如果 nums[i] > nums[i-1], 代表着现在处于上升过程
        if(nums[i] > nums[i-1]){
            // 状态修改为 up
            state = upState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;

            // 此时的元素值为 nums[i], 它前面的元素值为 nums[i-1]
            // 如果 nums[i] < nums[i-1], 代表着现在处于下降过程
        }else if(nums[i] < nums[i-1]){
            // 状态修改为 down
            state = downState;
            // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
            length++;
        }else{

            // do nothing

        }

    }

    // 2、如果是在 up 状态
    // 说明 nums[i] 的前面两个元素 nums[i-2] < nums[i-1], 正在处于上升过程
    }else if(state == upState){

        // 只有此时 nums[i] < nums[i-1]
        // 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波峰 ^
        if(nums[i] < nums[i-1]){

```

```

        // 此时, 由于 nums[i] < nums[i-1], 开始处于下降状态了
        // 状态修改为 down
        state = downState;
        // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
        length++;
    }else{
        // do nothing
    }

    // 3、如果是在 up 状态
    // 说明 nums[i] 的前面两个元素 nums[i-2] > nums[i-1], 正在处于下降过程
}else if(state == downState){

    // 只有此时 nums[i] > nums[i-1]
    // 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波谷 v
    if(nums[i] > nums[i-1]){

        // 此时, 由于 nums[i] > nums[i-1], 开始处于上升状态了
        // 状态修改为 up
        state = upState;
        // 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
        length++;
    }else{
        // do nothing
    }
}

}

// 返回结果 length
// 不需要返回具体序列
return length;

}

};

```

3、Python 代码

```

#### https://www.algomooc.com
#### 作者: 程序员吴师兄
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 摆动序列(376):https://leetcode-cn.com/problems/wiggle-subsequence/
class solution:
    def wiggleMaxLength(self, nums: List[int]) -> int:
        # 每个元素都有三种状态

        # 1、每个元素的初始状态
        beginState = 0

```

```

# 2、如果当前元素的值大于它之前元素的值
# 比如 nums[i] > nums[i-1]
# 那么说明当前元素处于上升阶段，状态设置为 up
upState = 1

# 3、如果当前元素的值小于它之前元素的值
# 比如 nums[i] < nums[i-1]
# 那么说明当前元素处于下降阶段，状态设置为 down
downState = 2

# 如果 nums 长度小于 2
if len(nums) < 2 :
    # 由于仅有一个元素或者含两个不等元素的序列也视作摆动序列
    # 直接返回数组的长度
    return len(nums)

# 以第一个元素作为初始的摇摆序列，此时，长度为 1
length = 1

# 一开始，状态为 begin
state = beginState

# 接下来，开始遍历 nums 中的所有元素
for i in range(1, len(nums)) :
    # 每个元素都有三种状态，在这三种状态下去判断这个元素应该怎么操作

    # 1、如果是在 begin 状态
    if state == beginState :
        # 那么比较 nums[i] 和 nums[i-1] 的大小

        # 此时的元素值为 nums[i]，它前面的元素值为 nums[i-1]
        # 如果 nums[i] > nums[i-1]，代表着现在处于上升过程
        if nums[i] > nums[i-1] :
            # 状态修改为 up
            state = upState
            # 摆动序列中增加了 nums[i] 这个元素，所以 length 需要加 1
            length += 1

        # 此时的元素值为 nums[i]，它前面的元素值为 nums[i-1]
        # 如果 nums[i] < nums[i-1]，代表着现在处于下降过程
        elif nums[i] < nums[i-1] :
            # 状态修改为 down
            state = downState
            # 摆动序列中增加了 nums[i] 这个元素，所以 length 需要加 1
            length += 1

    # 2、如果是在 up 状态

```

```

# 说明 nums[i] 的前面两个元素 nums[i-2] < nums[i-1], 正处于上升过程
elif state == upState :

    # 只有此时 nums[i] < nums[i-1]
    # 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波峰 ^
    if nums[i] < nums[i-1] :

        # 此时, 由于 nums[i] < nums[i-1], 开始处于下降状态了
        # 状态修改为 down
        state = downState
        # 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
        length += 1
# 3、如果是在 up 状态
# 说明 nums[i] 的前面两个元素 nums[i-2] > nums[i-1], 正处于下降过程
elif state == downState :

    # 只有此时 nums[i] > nums[i-1]
    # 那么 nums[i-2], nums[i-1], nums[i] 这三者形成一个波谷 v
    if nums[i] > nums[i-1] :

        # 此时, 由于 nums[i] > nums[i-1], 开始处于上升状态了
        # 状态修改为 up
        state = upState
        # 摆动序列中增加了 nums[i] 这个元素, 所以 length 需要加 1
        length += 1

# 返回结果 length
# 不需要返回具体序列
return length

```

四、复杂度分析

时间复杂度：O(n)，其中 n 是序列的长度。我们只需要遍历该序列一次。

空间复杂度：O(1)。我们只需要常数空间来存放若干变量。

LeetCode 45、跳跃游戏II

一、题目描述

给定一个长度为 n 的 0 索引整数数组 `nums`。初始位置为 `nums[0]`。

每个元素 `nums[i]` 表示从索引 i 向前跳转的最大长度。换句话说，如果你在 `nums[i]` 处，你可以跳转到任意 `nums[i + j]` 处：

- $0 \leq j \leq \text{nums}[i]$
- $i + j < n$

返回到达 `nums[n - 1]` 的最小跳跃次数。生成的测试用例可以到达 `nums[n - 1]`。

示例 1:

输入: `nums = [2,3,1,1,4]`

输出: 2

解释: 跳到最后一个位置的最小跳跃数是 2。

从下标为 0 跳到下标为 1 的位置, 跳 1 步, 然后跳 3 步到达数组的最后一个位置。

示例 2:

输入: `nums = [2,3,0,1,4]`

输出: 2

提示:

- `1 <= nums.length <= 10^4`
- `0 <= nums[i] <= 1000`

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 添加微信 278166530 获取最新课程
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 跳跃游戏II (LeetCode 45) : https://leetcode.cn/problems/jump-game-ii/
class Solution {
    public int jump(int[] nums) {
        // 当前的位置, 索引
        int nowPos = 0 ;

        // 最小的跳跃次数
        int step = 0;

        // nowPos 不断的调整位置
        // 由于题目明确说明可以达到最后一个位置
        // 因此, 当 nowPos 超过或者等于 nums.length - 1 时
        // 即跳出 while 循环, 也就得到结果了
        while( nowPos < nums.length - 1){

            // nowPos 对应的元素 nums[nowPos] 表示可以向后跳转的最大距离
            // nowPos + nums[nowPos] 表示从 nowPos 跳跃一次, 可以来到的最远的一个位置
            // 这个位置也就是选择跳跃时的一个右边界
            // 但不一定会选择跳到 nowPos + nums[nowPos] 这个位置
            int right = nowPos + nums[nowPos];
```

```

// 除非发现 right 可以到达末尾位置或者超过末尾位置
// 那么只需要在原来的 step 基础上跳跃一次即得到答案
if( right >= nums.length - 1) return step + 1;

// 否则就需要开始考虑下一次跳跃应该跳到 [ nowPos + 1 , right ] 之间的哪个位置
// 而选择的依据就是在上述区间选择的位置点，可以使得它的最大距离是所有选择中最大的

// 于是初始化两个变量

// nextRight 表示来到下一个跳跃点后，它的最大距离
int nextRight = right ;

// nextPos 表示下一个跳跃点的位置
int nextPos = nowPos;

// 访问 [ nowPos + 1 , right ] 之间的元素
for ( int i = nowPos + 1 ; i <= right ; i++){

    // 不断更新最大距离
    if( i + nums[i] > nextRight){

        // 不断更新最大距离
        nextRight = i + nums[i];
        // 不断更新跳跃点的位置
        nextPos = i;

    }

}

// 跳出循环后，nextPos 就是跳跃的新位置
// nowPos 来到 nextPos 的位置
nowPos = nextPos;

// 完成一次跳跃，step 进行更新
step++;
}

// 返回结果
return step;
}
}

```

2、**C++** 代码

```

class Solution {
public:
    int jump(vector<int>& nums) {
        // 当前的位置，索引
    }
}

```



```

int nowPos = 0 ;

// 最小的跳跃次数
int step = 0;

// nowPos 不断的调整位置
// 由于题目明确说明可以达到最后一个位置
// 因此, 当 nowPos 超过或者等于 nums.length - 1 时
// 即跳出 while 循环, 也就得到结果了
while( nowPos < nums.size() - 1){

    // nowPos 对应的元素 nums[nowPos] 表示可以向后跳转的最大距离
    // nowPos + nums[nowPos] 表示从 nowPos 跳跃一次, 可以来到的最远的一个位置
    // 这个位置也就是选择跳跃时的一个右边界
    // 但不一定会选择跳到 nowPos + nums[nowPos] 这个位置
    int right = nowPos + nums[nowPos];

    // 除非发现 right 可以到达末尾位置或者超过末尾位置
    // 那么只需要在原来的 step 基础上跳跃一次即得到答案
    if( right >= nums.size() - 1) return step + 1;

    // 否则就需要开始考虑下一次跳跃应该跳到 [ nowPos + 1 , right ] 之间的哪个位置
    // 而选择的依据就是在上述区间选择的位置点, 可以使得它的最大距离是所有选择中最大的

    // 于是初始化两个变量

    // nextRight 表示来到下一个跳跃点后, 它的最大距离
    int nextRight = right ;

    // nextPos 表示下一个跳跃点的位置
    int nextPos = nowPos;

    // 访问 [ nowPos + 1 , right ] 之间的元素
    for ( int i = nowPos + 1 ; i <= right ; i++){

        // 不断更新最大距离
        if( i + nums[i] > nextRight){

            // 不断更新最大距离
            nextRight = i + nums[i];
            // 不断更新跳跃点的位置
            nextPos = i;

        }

    }

    // 跳出循环后, nextPos 就是跳跃的新位置
    // nowPos 来到 nextPos 的位置
    nowPos = nextPos;
}

```

```

        // 完成一次跳跃, setp 进行更新
        step++;
    }

    // 返回结果
    return step;
}
};

```

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 添加微信 278166530 获取最新课程
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 跳跃游戏II (LeetCode 45) : https://leetcode.cn/problems/jump-game-ii/
class Solution:
    def jump(self, nums: List[int]) -> int:
        # 当前的位置, 索引
        nowPos = 0

        # 最小的跳跃次数
        step = 0

        # nowPos 不断的调整位置
        # 由于题目明确说明可以达到最后一个位置
        # 因此, 当 nowPos 超过或者等于 nums.length - 1 时
        # 即跳出 while 循环, 也就得到结果了
        while nowPos < len(nums) - 1 :

            # nowPos 对应的元素 nums[nowPos] 表示可以向后跳转的最大距离
            # nowPos + nums[nowPos] 表示从 nowPos 跳跃一次, 可以来到的最远的一个位置
            # 这个位置也就是选择跳跃时的一个右边界
            # 但不一定会选择跳到 nowPos + nums[nowPos] 这个位置
            right = nowPos + nums[nowPos]

            # 除非发现 right 可以到达末尾位置或者超过末尾位置
            # 那么只需要在原来的 step 基础上跳跃一次即得到答案
            if right >= len(nums) - 1 : return step + 1

            # 否则就需要开始考虑下一次跳跃应该跳到 [ nowPos + 1 , right ] 之间的哪个位置
            # 而选择的依据就是在上述区间选择的位置点, 可以使得它的最大距离是所有选择中最大的

            # 于是初始化两个变量

            # nextRight 表示来到下一个跳跃点后, 它的最大距离

```

```

nextRight = right

# nextPos 表示下一个跳跃点的位置
nextPos = nowPos

# 访问 [ nowPos + 1 , right ] 之间的元素
for i in range( nowPos + 1 , right + 1) :

    # 不断更新最大距离
    if i + nums[i] > nextRight :

        # 不断更新最大距离
        nextRight = i + nums[i]
        # 不断更新跳跃点的位置
        nextPos = i

# 跳出循环后, nextPos 就是跳跃的新位置
# nowPos 来到 nextPos 的位置
nowPos = nextPos

# 完成一次跳跃, setp 进行更新
step += 1

# 返回结果
return step

```

LeetCode 946、验证栈序列

一、题目描述

给定 `pushed` 和 `popped` 两个序列，每个序列中的 **值都不重复**，只有当它们可能是在最初空栈上进行的推入 `push` 和弹出 `pop` 操作序列的结果时，返回 `true`；否则，返回 `false`。

示例 1：

输入：pushed = [1,2,3,4,5], popped = [4,5,3,2,1] 输出：true 解释：我们可以按以下顺序执行：push(1), push(2), push(3), push(4), pop() -> 4, push(5), pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1

示例 2：

输入：pushed = [1,2,3,4,5], popped = [4,3,5,1,2] 输出：false 解释：1 不能在 2 之前弹出。

提示：

- `1 <= pushed.length <= 1000`
- `0 <= pushed[i] <= 1000`
- `pushed` 的所有元素 **互不相同**

- `popped.length == pushed.length`
- `popped` 是 `pushed` 的一个排列

二、题目解析

三、参考代码

Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class Solution {
    public boolean validateStackSequences(int[] pushed, int[] popped) {

        // 利用栈来模拟入栈和出栈操作
        Stack<Integer> s = new Stack<Integer>();

        // index 表示 popped 数组中元素的下标
        // 比如 popped 是 [4,5,3,2,1]
        // 那么第 0 个下标元素是 4 这个数字
        // 先去判断这个数字能否正常的出栈
        int index = 0;

        // 遍历 pushed 数组中的每个元素
        for(int i = 0 ; i < pushed.length; i ++){

            // 在遍历 pushed 数组时，把当前遍历的元素加入到栈中
            s.push(pushed[i]);

            // 加入完之后，不断的执行以下的判断
            // 1、栈中是否有元素
            // 2、栈顶元素是否和 popped 当前下标的元素相同
            // 如果同时满足这两个条件
            // 说明这个元素可以满足要求，即可以在最初空栈上进行推入 push 和弹出 pop 操作
            while(!s.isEmpty() && popped[index] == s.peek()){

                // 那么就把栈顶元素弹出
                s.pop();

                // 同时 index++, 观察 popped 下一个元素
                index++;
            }
        }

        // 遍历完 pushed 数组中的每个元素之后，如果发现栈不为空
        if(!s.isEmpty()){

            // 那么说明出栈序列不合法，返回 false
            return false;
        }
    }
}
```

```

    }

    // 否则返回 true
    return true;
}
}

```

Python 代码

```

class solution:
    def validateStackSequences(self, pushed: List[int], popped: List[int]) -> bool:
        # 利用列表来模拟入栈和出栈操作
        stack = []

        # index 表示 popped 数组中元素的下标
        # 比如 popped 是 [4,5,3,2,1]
        # 那么第 0 个下标元素是 4 这个数字
        # 先去判断这个数字能否正常的出栈
        index = 0

        # 遍历 pushed 数组中的每个元素
        for i in range(len(pushed)):
            # 在遍历 pushed 数组时, 把当前遍历的元素加入到列表中
            stack.append(pushed[i])

            # 加入完之后, 不断的执行以下的判断
            # 1、列表中是否有元素
            # 2、列表末尾元素是否和 popped 当前下标的元素相同
            # 如果同时满足这两个条件
            # 说明这个元素可以满足要求, 即可以在最初空列表上进行推入 push 和弹出 pop 操作
            while stack and popped[index] == stack[-1]:
                # 那么就把列表末尾元素弹出
                stack.pop()

                # 同时 index++, 观察 popped 下一个元素
                index += 1

        # 遍历完 pushed 数组中的每个元素之后, 如果发现列表不为空
        if stack:
            # 那么说明出栈序列不合法, 返回 False
            return False

        # 否则返回 True
        return True

```

第五十四天

LeetCode 231、2 的幂

一、题目描述

给你一个整数 n ，请你判断该整数是否是 2 的幂次方。如果是，返回 true；否则，返回 false。

示例 1：

```
输入：n = 1
输出：true
解释： $2^0 = 1$ 
```

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class Solution {
    public boolean isPowerOfTwo(int n) {

        // 考察异或操作
        return (n > 0) && (n & (n - 1)) == 0;
    }
}
```

LeetCode 268、丢失的数字

一、题目描述

给定一个包含 $[0, n]$ 中 n 个数的数组 `nums`，找出 $[0, n]$ 这个范围内没有出现在数组中的那个数。

```
输入：nums = [9,6,4,2,3,5,7,0,1]
输出：8
解释：n = 9，因为有 9 个数字，所以所有的数字都在范围  $[0, 9]$  内。8 是丢失的数字，因为它没有出现在 nums 中。
```

提示：

- `n == nums.length`
- `1 <= n <= 104`
- `0 <= nums[i] <= n`
- `nums` 中的所有数字都独一无二

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
class Solution {
    public int missingNumber(int[] nums) {

        // 定义变量 missing 为零
        int missing = 0;

        // 循环遍历 nums 中的每个数字
        for(int i = 0 ; i < nums.length; i++){

            // 将 nums 中的数字和 1 到 n 这这些数字全部进行异或
            // 由于两个相同的数字进行异或的结果是 0
            // 比如  $5 \wedge 5 = 0$ 
            // 又由于任何一个数和 0 进行异或还是它本身
            // 比如  $3 \wedge 0 = 3$ 
            // 并且，题目中说明 nums 中的所有数字都独一无二
            // 所以，最终 missing 就是那个只出现过一次的数字
            missing = missing ^ nums[i] ^ (i + 1);

        }

        // missing 就是那个丢失的数字，返回就行
        return missing;
    }
}
```

LeetCode 318、最大单词长度乘积

一、题目描述

给你一个字符串数组 `words`，找出并返回 `length(words[i]) * length(words[j])` 的最大值，并且这两个单词不含有公共字母。如果不存在这样的两个单词，返回 `0`。

示例 1:

```
输入: words = ["abcw","baz","foo","bar","xtfn","abcdef"]
输出: 16
解释: 这两个单词为 "abcw", "xtfn"。
```

示例 2:

输入: words = ["a","ab","abc","d","cd","bcd","abcd"]

输出: 4

解释: 这两个单词为 "ab", "cd"。

示例 3:

输入: words = ["a","aa","aaa","aaaa"]

输出: 0

解释: 不存在这样的两个单词。

提示:

- `2 <= words.length <= 1000`
- `1 <= words[i].length <= 1000`
- `words[i]` 仅包含小写字母

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最大单词长度乘积 ( LeetCode 318 ) : https://leetcode-cn.com/problems/maximum-product-of-word-lengths/
class Solution {
    public int maxProduct(String[] words) {

        // 获取字符串数组的长度
        int n = words.length;

        // 创建一个与 words 同等长度的 masks
        int[] masks = new int[n];

        // 每一个字母是否出现在某个单词的状态用一个 int 表示, int 的每一位代表 a - z 中的每个字母是否出现过
        // 比如 a 对应的是 000001
        // 比如 b 对应的是 000010
        // 比如 c 对应的是 000100
        // 这样的话, ab 对应的是 000011

        for (int i = 0 ; i < n ; i++) {

            String word = words[i];

            int bit = 0;
```



```

// 对于 word 中的每个字母来说，都拿出来查看
// 假设字符串为 "abc"
// 令字符串中的每一个字符命名为 c
// 则 (c - 'a') 为 0 1 2
// bit |= 1 << (c - 'a') 则是将 1 左移 0 位、1位、2位
// bit 开始位为 0
// 0 | 1 = 1
// 1 | 10 = 11
// 11 | 100 = 111
// 因此，字符串为 "abc" 对应的就是 111
for (int j = 0; j < word.length(); j++) {

    int c = word.charAt(j) - 'a';

    // bit |= 1 << (c - 'a') 则是将 1 左移 (c - 'a')位
    bit |= (1 << c);

}

// 因此，字符串为 word 对应的就是 bit
masks[i] = bit;
}

// 开始计算最大值
int ans = 0;

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // 在查看的过程中，如果发现两个单词不含有公共字母才计算它们的最大值
        if ((masks[i] & masks[j]) == 0) {
            // 计算之后更新
            ans = Math.max(ans, words[i].length() * words[j].length());
        }
    }
}

// 返回结果
return ans;
}
}

```

LeetCode 85、最大矩形

一、题目描述

给定一个仅包含 0 和 1、大小为 `rows x cols` 的二维二进制矩阵，找出只包含 1 的最大矩形，并返回其面积。

示例 1:

img

输入: `matrix = [[ "1","0","1","0","0"],[ "1","0","1","1","1"],[ "1","1","1","1","1"],[ "1","0","0","1","0"]]`

输出: 6

解释: 最大矩形如上图所示。

示例 2:

输入: `matrix = []`

输出: 0

示例 3:

输入: `matrix = [ ["0"]]`

输出: 0

示例 4:

输入: `matrix = [ ["1"]]`

输出: 1

示例 5:

输入: `matrix = [ ["0","0"]]`

输出: 0

提示:

- `rows == matrix.length`
- `cols == matrix[0].length`
- `1 <= row, cols <= 200`
- `matrix[i][j]` 为 '0' 或 '1'

二、题目解析

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 微信：wzb_3377
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 最大矩形 (LeetCode 85) : https://leetcode.cn/problems/maximal-rectangle/submissions/
class Solution {
    public int maximalRectangle(char[][] matrix) {

        // 对于每一行来说，它都是一维的
        int[] heights = new int[matrix[0].length];

        // 记录最大值
        int maxArea = 0;

        // 遍历每一行
        for( int row = 0 ; row < matrix.length ; row++){

            // 遍历每一列
            for( int col = 0 ; col < matrix[0].length ; col++){

                // 在遍历过程中
                // 如果发现当前位置为 1，那么把之前记录的 1 累加下来
                if(matrix[row][col] == '1'){

                    heights[col] += 1;

                }else{

                    // 【重点】：注意，一旦发现某个位置为 0 了
                    // 那么代表这一列的 1 断掉了，需要重置为 0
                    heights[col] = 0;

                }

            }

            // 获取当前行的最大矩形
            // 再更新
            maxArea = Math.max(maxArea, largestRectangleArea(heights));

        }

        return maxArea;

    }
}
```

```

public int largestRectangleArea(int[] heights) {

    // 初始化最终结果为0
    int res = 0;

    // 使用单调栈
    Stack<Integer> stack = new Stack<>();

    // 将给定的原数组左右各添加一个元素0，方便处理
    int[] newHeights = new int[heights.length + 2];

    // 左边界为 0
    newHeights[0] = 0 ;

    // 右边界边界为 0
    newHeights[heights.length + 1] = 0;

    // 其余不变
    for ( int i = 1 ; i < heights.length + 1 ; i++) {
        newHeights[i] = heights[i - 1];
    }

    // 整体思路：
    // 对于一个高度，如果能得到向左和向右的边界
    // 那么就能对每个高度求一次面积
    // 遍历所有高度，即可得出最大面积

    // 开始遍历
    for (int i = 0 ; i < newHeights.length ; i++) {
        // 如果栈不为空且当前考察的元素值小于栈顶元素值，
        // 则表示以栈顶元素值为高的矩形面积可以确定
        while (!stack.isEmpty() && newHeights[i] < newHeights[stack.peek()]) {
            // 弹出栈顶元素
            int cur = stack.pop();
            // 获取栈顶元素对应的高
            int curHeight = newHeights[cur];

            // 栈顶元素弹出后，新的栈顶元素就是其左侧边界
            int leftIndex = stack.peek();

            // 右侧边界是当前考察的索引

            int rightIndex = i;

            // 计算矩形宽度
            int curwidth = rightIndex - leftIndex - 1;

            // 计算面积

```

```

        res = Math.max(res, curHeight * curwidth);

    }

    // 当前考察索引入栈
    stack.push(i);
}

// 返回结果
return res;
}
}

class Solution {
public:
    int maximalRectangle(vector<vector<char>>& matrix) {
        // 对于每一行来说，它都是一维的
        std::vector<int> heights(matrix[0].size(), 0);

        // 记录最大值
        int maxArea = 0;

        // 遍历每一行
        for (int row = 0; row < matrix.size(); ++row) {
            // 遍历每一列
            for (int col = 0; col < matrix[0].size(); ++col) {

                // 在遍历过程中
                // 如果发现当前位置为 '1', 那么把之前记录的 1 累加下来
                if (matrix[row][col] == '1') {
                    heights[col] += 1;
                }
                else {
                    // 【重点】：注意，一旦发现某个位置为 '0' 了
                    // 那么代表这一列的 1 断掉了，需要重置为 0
                    heights[col] = 0;
                }
            }

            // 获取当前行的最大矩形
            // 再更新
            maxArea = std::max(maxArea, largestRectangleArea(heights));
        }

        return maxArea;
    }

    int largestRectangleArea(std::vector<int>& heights) {
        // 初始化最终结果为 0
        int res = 0;
    }
}

```

```

// 使用单调栈
std::stack<int> stk;

// 将给定的原数组左右各添加一个元素 0，方便处理
std::vector<int> newHeights(heights.size() + 2, 0);

// 左边界为 0
newHeights[0] = 0;

// 右边界为 0
newHeights[heights.size() + 1] = 0;

// 其余不变
for (int i = 1; i <= heights.size(); ++i) {
    newHeights[i] = heights[i - 1];
}

// 整体思路：
// 对于一个高度，如果能得到向左和向右的边界
// 那么就能对每个高度求一次面积
// 遍历所有高度，即可得出最大面积

// 开始遍历
for (int i = 0; i < newHeights.size(); ++i) {
    // 如果栈不为空且当前考察的元素值小于栈顶元素值，
    // 则表示以栈顶元素值为高的矩形面积可以确定
    while (!stk.empty() && newHeights[i] < newHeights[stk.top()]) {
        // 弹出栈顶元素
        int cur = stk.top();
        stk.pop();

        // 获取栈顶元素对应的高
        int curHeight = newHeights[cur];

        // 栈顶元素弹出后，新的栈顶元素就是其左侧边界
        int leftIndex = stk.top();

        // 右侧边界是当前考察的索引
        int rightIndex = i;

        // 计算矩形宽度
        int curwidth = rightIndex - leftIndex - 1;

        // 计算面积
        res = std::max(res, curHeight * curwidth);
    }

    // 当前考察索引入栈

```

```

        stk.push(i);
    }

    // 返回结果
    return res;
}

};

int main() {
    Solution solution;
    std::vector<std::vector<char>> matrix = {
        {'1', '0', '1', '0', '0'},
        {'1', '0', '1', '1', '1'},
        {'1', '1', '1', '1', '1'},
        {'1', '0', '0', '1', '0'}
    };
    int maximalArea = solution.maximalRectangle(matrix);
    std::cout << "Maximal Rectangle Area: " << maximalArea << std::endl;
    return 0;
}

#### 登录 AlgoMooc 官网获取更多算法图解
#### https://www.algomooc.com
#### 作者：程序员吴师兄
#### 微信：wzb_3377
#### 代码有看不懂的地方一定要私聊咨询吴师兄呀
#### 最大矩形 (LeetCode 85) : https://leetcode.cn/problems/maximal-rectangle/submissions/
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:

        # 对于每一行来说，它都是一维的
        heights = [0] * len(matrix[0])

        # 记录最大值
        maxArea = 0

        # 遍历每一行
        for row in range(len(matrix)):
            # 遍历每一列
            for col in range(len(matrix[0])):

                # 在遍历过程中
                # 如果发现当前位置为 1，那么把之前记录的 1 累加下来
                if matrix[row][col] == '1' :

                    heights[col] += 1

                else:

```

```

        # 【重点】：注意，一旦发现某个位置为 0 了
        # 那么代表这一列的 1 断掉了，需要重置为 0
        heights[col] = 0

        # 获取当前行的最大矩形
        # 再更新
        maxArea = max(maxArea, self.largestRectangleArea(heights))

    return maxArea

def largestRectangleArea(self, heights: List[int]) -> int:

    # 初始化最终结果为0
    res = 0

    # 使用单调栈
    stk = list()

    # 将给定的原数组左右各添加一个元素0，方便处理
    newHeights = [0] * (len(heights) + 2)

    # 左边界为 0
    newHeights[0] = 0

    # 右边界边界为 0
    newHeights[len(heights) + 1] = 0

    # 其余不变
    for i in range(1, len(newHeights) - 1):
        newHeights[i] = heights[i - 1]

    # 整体思路：
    # 对于一个高度，如果能得到向左和向右的边界
    # 那么就能对每个高度求一次面积
    # 遍历所有高度，即可得出最大面积

    # 开始遍历
    for i in range(len(newHeights)):
        # 如果栈不为空且当前考察的元素值小于栈顶元素值，
        # 则表示以栈顶元素值为高的矩形面积可以确定
        while stk and newHeights[i] < newHeights[stk[-1]]:
            # 弹出栈顶元素
            cur = stk[-1]

            # 计算面积
            res = max(res, (i - cur) * newHeights[cur])

            stk.pop()

        stk.append(i)

```



```
# 获取栈顶元素对应的高
curHeight = newHeights[cur]

# 栈顶元素弹出后，新的栈顶元素就是其左侧边界
leftIndex = stk[-1]

# 右侧边界是当前考察的索引

rightIndex = i

# 计算矩形宽度
curwidth = rightIndex - leftIndex - 1

# 计算面积
res = max(res, curHeight * curwidth)

# 当前考察索引入栈
stk.append(i)

# 返回结果
return res
```

第五十五天

LeetCode 295、数据流中的中位数

一、题目描述

如何得到一个数据流中的**中位数**？**

如果从数据流中读出奇数个数值，那么中位数就是所有数值排序之后位于中间的数值。

如果从数据流中读出偶数个数值，那么中位数就是所有数值排序之后中间两个数的平均值。

例如，

[2,3,4] 的中位数是 3

[2,3] 的中位数是 $(2 + 3) / 2 = 2.5$

设计一个支持以下两种操作的数据结构：

- void addNum(int num) - 从数据流中添加一个整数到数据结构中。
- double findMedian() - 返回目前所有元素的中位数。

示例 1：

```
输入: ["MedianFinder","addNum","addNum","findMedian","addNum","findMedian"]
      [[],[1],[2],[],[3],[ ]]
输出: [null,null,null,1.50000,null,2.00000]
```

示例 2:

```
输入:
["MedianFinder","addNum","findMedian","addNum","findMedian"]
[[],[2],[],[3],[ ]]
输出: [null,null,2.00000,null,2.50000]
```

限制:

- 最多会对 `addNum`、`findMedian` 进行 50000 次调用。

二、题目解析

这道题目得先了解以下几个基础概念:

- 1、中位数指的是**排序数组**的中间元素值, 如果是奇数, 那么直接就是中间的数值; 如果是偶数, 那么就是中间两个数的平均值。
- 2、数据流指的是数据的长度是动态变化的, 就像流水一样, 在不断的新增数据过来。

这意味着, **数据流的**中位数在不断的变化, 不仅值在变化, 求解方式也是在动态变化****。

并且, 我们是需要不断的将数据流中的全部数字进行排序, 那么这里就要借助堆的知识了。

设置两个堆, 一个是大顶堆 `maxHeap`, 一个是小顶堆 `minHeap`。

- **大顶堆 `maxHeap`** 来存储数据流中较小一半的值
- **小顶堆 `minHeap`** 来存储数据流中较大一半的值

由于大顶堆的堆顶为它的存储区间的最大值, 小顶堆的堆顶为它的存储区间的最小值, 那么如果用着两个堆来存储数据流的所有数据, 我们可以组成一个**递增有序**的数组。

- 1、大顶堆从左到右递增
- 2、小顶堆从左到右递增

在动态存储数据流的数据过程中, 中位数也就是这两种情况:

- 1、数据流为奇数时, 保证两个堆的长度相差 1, 小顶堆的堆顶就是中位数。
- 2、数据流为偶数时, 保证两个堆的长度相等, 两个堆的堆顶相加除二就是中位数。

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 剑指 Offer 41. 数据流中的中位数:https://leetcode-cn.com/problems/shu-ju-liu-zhong-de-zhong-wei-shu-lcof/
class MedianFinder {
```

```

// PriorityQueue, 优先队列
// 优先队列的作用是能保证每次取出的元素都是队列中权值最小的 ( Java 的优先队列每次取最小元素,
C++的优先队列每次取最大元素)
// 由于 Java 的优先队列每次取最小元素, 即默认函数是实现小顶堆
// 大顶堆的概念: 每个节点的值大于等于左右孩子节点的值, 堆顶为最大值
// 因此, 大顶堆的初始化需要额外处理
// maxHeap 存储数据流中较小一半的值
PriorityQueue<Integer> maxHeap;

// 小顶堆的概念: 每个节点的值小于等于左右孩子节点的值, 堆顶为最小值
// minHeap 来存储数据流中较大一半的值
PriorityQueue<Integer> minHeap;

public MedianFinder() {

    // 初始化操作
    maxHeap = new PriorityQueue<Integer>((x, y) -> (y - x));

    minHeap = new PriorityQueue<Integer>();
}

// 根据我们的设定, 一直维护大顶堆、小顶堆的特性
// 使得 maxHeap堆底 <= maxHeap堆顶 <= minHeap堆顶 <= minHeap堆底
// 那么, 中位数就是在【大顶堆的堆顶】与【小顶堆的堆顶】中间的位置
// 在添加元素的过程中, 需要判断添加的元素应该添加到哪个堆中
// 小的值应该插入到 maxHeap, 大的值应该插入到 minHeap

public void addNum(int num) {

    // 数据流的长度有奇数和偶数两种情况, 并且是在动态变化的

    // 1、【大顶堆】与【小顶堆】的长度不相等, 由于两者的长度至多相差 1, 那么数据流的总长度就是奇数
    // 假设 minHeap 的长度为 n, 则 maxHeap 的长度为 n - 1
    // 那么 maxHeap 是应该需要加入一个【新的元素】的, 这样就能使得 minHeap 和 maxHeap 的长度
均为 n
    // 那么加入新元素之后, 中位数就是 ( minHeap 的堆顶 + maxHeap 的堆顶) / 2
    // 但如果直接把 num 加入到 maxHeap 中, 如果 num 是一个很大的值
    // 由于 maxHeap 是存储数据流中较小一半的值, 这样就会破坏我们维护的属性
    // 因此, 我们可以先把 num 加入到 minHeap 中, 然后从 minHeap 挤出一个最小值来, 重新加入到
maxHeap
    // 一来一回, minHeap 的长度依旧为 n, maxHeap 的长度变成了 n
    if(maxHeap.size() != minHeap.size()) {

        // 先将元素添加到小顶堆 minHeap 中
        // 由于 minHeap 添加了新的元素, PriorityQueue 会自动的将 minHeap 之前的元素和 num
进行操作
        // 使得 minHeap 的每个节点的值小于等于左右孩子节点的值, 堆顶为最小值
        // 这个时候, minHeap 的长度变成了 n + 1

```

```

minHeap.add(num);

// 由于 minHeap 来存储数据流中较大一半的值，而新添加的元素 num 有可能是一个很小的值
// 理论上应该要加入到 maxHeap 才对
// 所以，先去获取此时 minHeap 的堆顶元素（不一定值是 num），即最小值，把它抛出后加入到
maxHeap 中
minHeap.add(minHeap.poll());

// 2、【大顶堆】与【小顶堆】的长度相等，那么数据流的总长度就是偶数
// 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n
// 我们把新的元素加入到 minHeap 中，使得 minHeap 的长度变成了 n + 1
// 那么中位数就是 minHeap 的堆顶元素了
// 但如果直接把 num 加入到 minHeap 中，如果 num 是一个很小的值
// 由于 minHeap 是存储数据流中较大一半的值，这样就会破坏我们维护的属性
// 因此，我们可以先把 num 加入到 maxHeap 中，然后从 maxHeap 挤出一个最大值来，重新加入到
minHeap
// 一来一回，maxHeap 的长度依旧为 n，minHeap 的长度变成了 n + 1
} else {

// 先将元素添加到大顶堆 maxHeap 中
// 由于 maxHeap 添加了新的元素，PriorityQueue 会自动的将 maxHeap 之前的元素和 num
进行操作
// 使得 maxHeap 的每个节点的值大于等于左右孩子节点的值，堆顶为最大值
// 这个时候，maxHeap 的长度变成了 n + 1
maxHeap.add(num);

// 由于 maxHeap 来存储数据流中较小一半的值，而新添加的元素 num 有可能是一个很大的值
// 理论上应该要加入到 minHeap 才对
// 所以，先去获取此时 maxHeap 的堆顶元素（不一定值是 num），即最大值，把它抛出后加入到
minHeap 中
minHeap.add(maxHeap.poll());

}
}

public double findMedian() {

// 数据流的长度有奇数和偶数两种情况，并且是在动态变化的

// 1、【大顶堆】与【小顶堆】的长度不相等，由于两者的长度至多相差 1，那么数据流的总长度就是奇数
// 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n - 1
// 那么中位数出现在 minHeap 的堆顶位置
if(maxHeap.size() != minHeap.size()) {
    return minHeap.peek();

// 2、【大顶堆】与【小顶堆】的长度相等，那么数据流的总长度就是偶数
// 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n

```

```

        // 那么中位数就是 ( minHeap 的堆顶 + maxHeap 的堆顶) / 2
    } else {
        return (maxHeap.peak() + minHeap.peak()) / 2.0;
    }
}
}

```

2、**C++** 代码

3、Python 代码

```

#### 登录 AlgoMooc 官网获取更多算法图解
# https://www.algomooc.com
# 作者：程序员吴师兄
# 代码有看不懂的地方一定要私聊咨询吴师兄呀
# 剑指 offer 41. 数据流中的中位数:https://leetcode-cn.com/problems/shu-ju-liu-zhong-de-zhong-wei-shu-lcof/
class MedianFinder:

    # 初始化操作
    def __init__(self):

        # 优先队列的作用是能保证每次取出的元素都是队列中权值最小的（ Java 的优先队列每次取最小元素，
        C++的优先队列每次取最大元素）
        # 由于 Java 的优先队列每次取最小元素，即默认函数是实现小顶堆
        # 大顶堆的概念：每个节点的值大于等于左右孩子节点的值，堆顶为最大值
        # 因此，大顶堆的初始化需要额外处理
        # maxHeap 存储数据流中较小一半的值
        self.maxHeap = []

        # 小顶堆的概念：每个节点的值小于等于左右孩子节点的值，堆顶为最小值
        # minHeap 来存储数据流中较大一半的值
        self.minHeap = []

        # 根据我们的设定，一直维护大顶堆、小顶堆的特性
        # 使得 maxHeap堆底 <= maxHeap堆顶 <= minHeap堆顶 <= minHeap堆底
        # 那么，中位数就是在【大顶堆的堆顶】与【小顶堆的堆顶】中间的位置
        # 在添加元素的过程中，需要判断添加的元素应该添加到哪个堆中
        # 小的值应该插入到 maxHeap，大的值应该插入到 minHeap

    def addNum(self, num: int) -> None:

        # 数据流的长度有奇数和偶数两种情况，并且是在动态变化的

        # 1、【大顶堆】与【小顶堆】的长度不相等，由于两者的长度至多相差 1，那么数据流的总长度就是奇数

```

```

# 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n - 1
# 那么 maxHeap 是应该需要加入一个【新的元素】的，这样就能使得 minHeap 和 maxHeap 的长度
均为 n

# 那么加入新元素之后，中位数就是 (minHeap 的堆顶 + maxHeap 的堆顶) / 2
# 但如果直接把 num 加入到 maxHeap 中，如果 num 是一个很大的值
# 由于 maxHeap 是存储数据流中较小一半的值，这样就会破坏我们维护的属性
# 因此，我们可以先把 num 加入到 minHeap 中，然后从 minHeap 挤出一个最小值来，重新加入到
maxHeap

# 一来一回，minHeap 的长度依旧为 n，maxHeap 的长度变成了 n
if len(self.maxHeap) != len(self.minHeap) :

    # 先将元素添加到小顶堆 minHeap 中
    # 由于 minHeap 添加了新的元素，PriorityQueue 会自动的将 minHeap 之前的元素和 num
    进行操作

    # 使得 minHeap 的每个节点的值小于等于左右孩子节点的值，堆顶为最小值
    # 这个时候，minHeap 的长度变成了 n + 1
    heappush(self.minHeap, num)
    # 由于 minHeap 来存储数据流中较大一半的值，而新添加的元素 num 有可能是一个很小的值
    # 理论上应该要加入到 maxHeap 才对
    # 所以，先去获取此时 minHeap 的堆顶元素（不一定值是 num），即最小值，把它抛出后加入到
    maxHeap 中

    heappush(self.maxHeap, -heappop(self.minHeap))

# 2、【大顶堆】与【小顶堆】的长度相等，那么数据流的总长度就是偶数
# 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n
# 我们把新的元素加入到 minHeap 中，使得 minHeap 的长度变成了 n + 1
# 那么中位数就是 minHeap 的堆顶元素了
# 但如果直接把 num 加入到 minHeap 中，如果 num 是一个很小的值
# 由于 minHeap 是存储数据流中较大一半的值，这样就会破坏我们维护的属性
# 因此，我们可以先把 num 加入到 maxHeap 中，然后从 maxHeap 挤出一个最大值来，重新加入到
minHeap

# 一来一回，maxHeap 的长度依旧为 n，minHeap 的长度变成了 n + 1
else :

    # 先将元素添加到大顶堆 maxHeap 中
    # 由于 maxHeap 添加了新的元素，PriorityQueue 会自动的将 maxHeap 之前的元素和 num
    进行操作

    # 使得 maxHeap 的每个节点的值大于等于左右孩子节点的值，堆顶为最大值
    # 这个时候，maxHeap 的长度变成了 n + 1
    heappush(self.maxHeap, -num)

    # 由于 maxHeap 来存储数据流中较小一半的值，而新添加的元素 num 有可能是一个很大的值
    # 理论上应该要加入到 minHeap 才对
    # 所以，先去获取此时 maxHeap 的堆顶元素（不一定值是 num），即最大值，把它抛出后加入到
    minHeap 中

    heappush(self.minHeap, -heappop(self.maxHeap))

def findMedian(self) -> float:

```

```

# 数据流的长度有奇数和偶数两种情况，并且是在动态变化的

# 1、【大顶堆】与【小顶堆】的长度不相等，由于两者的长度至多相差 1，那么数据流的总长度就是奇数
# 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n - 1
# 那么中位数出现在 minHeap 的堆顶位置
if len(self.maxHeap) != len(self.minHeap) :
    return self.minHeap[0]

# 2、【大顶堆】与【小顶堆】的长度相等，那么数据流的总长度就是偶数
# 假设 minHeap 的长度为 n，则 maxHeap 的长度为 n
# 那么中位数就是 （ minHeap 的堆顶 + maxHeap 的堆顶） / 2
else :
    return (-self.maxHeap[0] + self.minHeap[0]) / 2.0

```

四、复杂度分析

空间复杂度：O(N)，即为数据流中的元素数量。

时间复杂度：

获取中位数：O(1)。

添加元素：O(logN)，堆添加元素的时间复杂度为O(logN)。

LeetCode 461、汉明距离

一、题目描述

两个整数之间的 [汉明距离](#) 指的是这两个数字对应二进制位不同的位置的数目。

给你两个整数 `x` 和 `y`，计算并返回它们之间的汉明距离。

示例 1：

输入：x = 1, y = 4

输出：2

解释：

1 (0 0 0 1)

4 (0 1 0 0)

 ↑ ↑

上面的箭头指出了对应二进制位不同的位置。

示例 2：

输入：x = 3, y = 1

输出：1

提示：

- $0 \leq x, y \leq 2^{31} - 1$

二、题目解析

三、参考代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者：程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 汉明距离 ( LeetCode 461 ) : https://leetcode-cn.com/problems/hamming-distance/
class Solution {
    public int hammingDistance(int x, int y) {

        // 将 x 与 y 执行一个异或操作
        // 异或的特征：不同为 1，相同为 0
        // 这样，s 的二进制中 1 的个数就是这两个数字对应二进制位不同的位置的数目
        int s = x ^ y;

        // 统计 s 上 1 的个数
        int count = 0;

        while (s != 0) {
            // 每次让 s 与 s - 1 执行相与操作
            s &= s - 1;
            // 每循环一次，都会删去其二进制表示中最右侧的 1，最终循环的次数即为 s 中 1 的个数
            count++;
        }

        // 返回结果
        return count;
    }
}
```

剑指 Offer 51、数组中的逆序对

一、题目描述

在数组中的两个数字，如果前面一个数字大于后面的数字，则这两个数字组成一个逆序对。输入一个数组，求出这个数组中的逆序对的总数。

示例 1:

输入：[7,5,6,4]
输出：5

限制：

0 <= 数组长度 <= 50000

二、题目解析

这道题目首先得掌握归并排序的基础知识。

- 1、分解 (Divide) : 将 n 个元素分成个含 $n / 2$ 个元素的子序列。
- 2、解决 (Conquer) : 用合并排序法对两个子序列递归的排序。
- 3、合并 (Combine) : 合并两个已排序的子序列已得到排序结果。

而在第二步解决的过程中,不断的通过比较的方式合并两个排序数组,在这个操作中,如果遇到 **左子数组当前元素 > 右子数组当前元素**,意味着「左子数组当前元素 至 末尾元素」与「右子数组当前元素」构成了若干「逆序对」。

比如 4 与 2 进行比较, $4 > 2$, 它们是一组逆序对, 又因为黄色区域从左到右是递增的, 那也就意味着从 start1 到 end1 所有的元素都大于了 2, 都和 2 构成了逆序对。

所以,我们只需要在归并排序的代码上添加一行统计逆序对的代码就行。

三、参考代码

1、Java 代码

```
// 登录 AlgoMooc 官网获取更多算法图解
// https://www.algomooc.com
// 作者: 程序员吴师兄
// 代码有看不懂的地方一定要私聊咨询吴师兄呀
// 剑指 offer 51. 数组中的逆序对: https://leetcode-cn.com/problems/shu-zu-zhong-de-ni-xu-dui-lcof/solution/
class Solution {

    // 记录逆序对的数量
    private int count;

    public int reversePairs(int[] nums) {

        // 初始化为 0
        count = 0 ;

        int[] result = new int[nums.length];

        // 通过归并排序的方式,不断的记录逆序对的数量
        merge_sort_recursive(nums,result,0,nums.length - 1);

        return count;
    }

    public void merge_sort_recursive(int[] arr, int[] result, int start, int end) {

        // 1、分解到最小需要解决的地步,无法再分解了
        if (start >= end){
```

```

        // 需要开始解决问题
        return;
    }

    // 2、解决

    // 计算数组 arr 的中间位置
    int mid = (start + end) / 2;

    // start1 为左区间的开始位置
    int start1 = start;

    // end1 为左区间的结束位置
    int end1 = mid;

    // start2 为右区间的开始位置
    int start2 = mid + 1;

    // end2 为右区间的结束位置
    int end2 = end;

    // 调用 merge_sort_recursive 函数，将 arr 数组中的 start1 到 end1 这一区间的数字排序
    // 后，并存放到 result 中
    merge_sort_recursive(arr, result, start1, end1);

    // 调用 merge_sort_recursive 函数，将 arr 数组中的 start2 到 end2 这一区间的数字排序
    // 后，并存放到 result 中
    merge_sort_recursive(arr, result, start2, end2);

    // 3、合并

    // 将左右区间中较小的数字存放到 result 中，从 k 开始
    int k = start;

    while (start1 <= end1 && start2 <= end2){
        // 如果 arr[start1] < arr[start2])
        if(arr[start1] <= arr[start2]){
            // result[k] 存放的是 arr[start1]
            result[k] = arr[start1] ;
            start1++;
            k++;
        }else{
            // 否则，result[k] 存放的是 arr[start2]
            result[k] = arr[start2] ;

            // 在这里统计逆序对
            count += (end1 - start1 + 1);

            start2++;
        }
    }

```

```

        k++;
    }

}

// 如果左区间中还有元素，那么把它都添加到 result 中
while (start1 <= end1){
    result[k] = arr[start1];
    k++;
    start1++;
}

// 如果右区间中还有元素，那么把它都添加到 result 中
while (start2 <= end2){
    result[k] = arr[start2];
    k++;
    start2++;
}

// 最后，把结果赋值到 arr 中
for (k = start; k <= end; k++)
    arr[k] = result[k];
}

}

```

2、C++ 代码

3、Python 代码

```

class Solution:

    # 记录逆序对的数量
    # 初始化为 0
    count = 0

    def reversePairs(self, nums: List[int]) -> int:

        result = [0] * len(nums)

        # 通过归并排序的方式，不断的记录逆序对的数量
        self.merge_sort_recursive(nums, result, 0, len(nums) - 1)

        return self.count

```

```

def merge_sort_recursive(self, arr:List[int], result:List[int], start, end) :

    # 1、分解到最小需要解决的地步，无法再分解了
    if start >= end :
        # 需要开始解决问题
        return

    # 2、解决

    # 计算数组 arr 的中间位置
    mid = (start + end ) // 2

    # start1 为左区间的开始位置
    start1 = start

    # end1 为左区间的结束位置
    end1 = mid

    # start2 为右区间的开始位置
    start2 = mid + 1

    # end2 为右区间的结束位置
    end2 = end

    # 调用 merge_sort_recursive 函数，将 arr 数组中的 start1 到 end1 这一区间的数字排序
    # 后，并存放到 result 中
    self.merge_sort_recursive(arr, result, start1, end1)

    # 调用 merge_sort_recursive 函数，将 arr 数组中的 start2 到 end2 这一区间的数字排序
    # 后，并存放到 result 中
    self.merge_sort_recursive(arr, result, start2, end2)

    # 3、合并

    # 将左右区间中较小的数字存放到 result 中，从 k 开始
    k = start

    while start1 <= end1 and start2 <= end2 :
        # 如果 arr[start1] < arr[start2])
        if arr[start1] <= arr[start2] :
            # result[k] 存放的是 arr[start1]
            result[k] = arr[start1]
            start1 += 1
            k += 1
        else :
            # 否则，result[k] 存放的是 arr[start2]
            result[k] = arr[start2]

```

```
        # 在这里统计逆序对
        self.count += (end1 - start1 + 1)

        start2 += 1

        k += 1

    # 如果左区间中还有元素，那么把它都添加到 result 中
    while start1 <= end1 :
        result[k] = arr[start1]
        k += 1
        start1 += 1

    # 如果右区间中还有元素，那么把它都添加到 result 中
    while start2 <= end2 :
        result[k] = arr[start2]
        k += 1
        start2 += 1

    # 最后，把结果赋值到 arr 中
    for k in range( start , end + 1) :
        arr[k] = result[k]
```
